

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Secrétariat Général

Université Nazi BONI

École Doctorale Sciences et Techniques

N° d'ordre :

000004



Thèse de doctorat unique en informatique

En vue de l'obtention du grade de docteur de l'Université Nazi BONI

Spécialité : Traitement d'Images

Vers une boîte à outils de la transformée de Hough pour la détection des objets discrets dans des grilles virtuelles

Présentée par : OUEDRAOGO Moïse

Soutenue le 9 Janvier 2025, devant le jury composé de :

Président : Pr Oumarou SIE, Professeur titulaire en informatique, Université Aube Nouvelle

Examineur : Pr Malo SADOUANOUAN, Professeur titulaire en informatique, Université Nazi BONI

Rapporteur : Pr Frédéric T. OUEDRAOGO, Professeur titulaire en informatique, Université Norbert Zongo

Rapporteur : Dr Telesphore B. TIENDREBEOGO, Maître de Conférences en informatique, Université Nazi BONI

Rapporteur : Dr Tiémoman KONE, Maître de recherche en informatique, Université virtuelle de Côte d'Ivoire

Directeur de thèse : Dr Abdoulaye SERE, Maître de Conférences en informatique, Université Nazi BONI

Année universitaire 2024/2025

Dédicace

Je dédie ce modeste travail :

À Jésus-Christ, pour le don de l'intelligence, de la sagesse, de la protection et de la vie.

À mes parents, OUEDRAOGO Moumouni Sylvain et OUEDRAOGO Nongasida Rosalie. Ils ont pris soin de moi et m'ont toujours encouragé. Ils prient pour moi tous les jours.

Une dédicace spéciale à ma maman. Elle a veillé sans relâche sur moi et s'est sacrifiée pour mes études. Elle est, en grande partie, la raison pour laquelle je me suis toujours battu dans la vie.

À ma grande sœur OUEDRAOGO Monique et à mes petits frères OUEDRAOGO Élisée Wendéfangé, OUEDRAOGO Ezékiel Bouwendmanegré, OUEDRAOGO Enock Relwendé qui m'ont été d'un grand soutien.

À l'amour de ma vie, BAFIOGO Marcelline, qui me soutient sans faille, même lorsque tout le monde est contre moi. Durant ces trois années de travaux, elle a été ma force et mon inspiration.

À mon Pasteur, l'Apôtre OUEDRAOGO Paul, qui n'a cessé de prier pour moi jour et nuit.

- Moïse OUEDRAOGO

Remerciements

Je tiens à exprimer ma profonde gratitude aux entités et aux personnes sans lesquelles ce projet n'aurait jamais vu le jour. Mes sincères remerciements vont en particulier :

Aux membres du jury qui ont accepté de lire ce travail et de l'évaluer.

Au Dr Abdoulaye SERE. Il a eu confiance en moi et a été d'un soutien indéfectible durant les moments difficiles de la thèse. C'est un homme très ouvert, rigoureux, travailleur et très humble. j'ai beaucoup appris à ses côtés. C'est un mentor pour moi.

Au Pr Tiemonan KONE, Directeur Général de l'Université Virtuelle de Côte d'Ivoire (UVCI), et ses collaborateurs pour l'accueil chaleureux au sein de l'Unité de Recherche et d'Expertise Numérique (UREN) lors de mon séjour scientifique à l'UVCI.

Au Dr André Conseibo, Responsable du Laboratoire L@mia de l'Université Norbert Zongo de m'avoir ouvert les portes du laboratoire L@mia pour mon séjour de recherche. Le Pr Frédéric Ouedraogo, malgré qu'il était en voyage, a veillé à ce que je sois bien installé et n'a cessé de prendre de mes nouvelles durant tous mon séjour. Les doctorants du laboratoire, notamment Emile OUEDRAOGO, Jean SAMA, Augustin KABORE, ont favorisé mon intégration au sein du laboratoire.

Au Pr Boureima SANGARE, Dr Telesphore TIENDREBEOGO, Dr SOME Borlli Michel Jonas, pour leurs soutiens, conseils et encouragements.

Au Dr Ouedraogo Thomas, Dr Boulou Mahamadi, Dr Traoré Cheick Amed, Traoré Go Issa, Bamogo Moussa, Somda Augustin, Dabonné Hoda, Ouedraogo Arthur, Kaboré Abdoulaye. Nous avons partagé de nombreuses expériences durant ces années de thèse.

Abstract

The main objective of this thesis is to develop a toolbox for the Hough transform to efficiently detect discrete objects in images, by integrating temporal optimization techniques, parallel programming and specific adaptation for different types of applications. To this end, discrete mathematics (topology and geometry) has enabled us, on the one hand, to propose and demonstrate results relating to digital images. On the other hand, we developed algorithms simulated to corroborate our theoretical results. These theoretical results and their simulations concern straight line detection and fingerprint recognition.

In the case of discrete line detection, the proposed method consists in superimposing a virtual grid (rectangular, triangular, hexagonal or octagonal) on the image and calculating the dual of the cells in this virtual grid whose rate or percentage of lit pixels exceeds a given percentage ($0\% \leq \alpha \leq 100\%$). The voting threshold of the Standard Hough transform is a criterion that determines the number of votes required for a line to be considered detected. It is an important parameter influencing the quality and reliability of line detection in an image. A low vote threshold includes noise and therefore degrades detection quality. Our approach takes advantage of this limitation to optimize the analytical recognition of straight lines. Simulations were carried out on an image of a building and on an image of a freeway. The results demonstrated the method's ability to detect straight lines, while offering additional flexibility thanks to the possibility of adjusting the virtual grid parameters. The virtual grid approach enabled a reduction in computation time with an acceleration factor $A = 3,26$ (on the building image) and $A = 1,9$ (on the freeway image) with the rectangular grid, an acceleration factor $A = 2,28$ (on the building image) and $A = 1,31$ (on the freeway image) with the triangular grid, an acceleration factor $A = 3,5$ (on the building image) and $A = 1,05$ (on the freeway image) with the hexagonal grid, an acceleration factor $A = 2,22$ (on the building image) and $A = 1,9$ (on the freeway image) with the octagonal grid. By reducing the number of pixels to be calculated thanks to the α variable and the size of the virtual grid cells, the method offers significant computational savings. However, despite its advantages, the method proposed for discrete line detection is not without its limitations. These are : the loss of information present in the full set of lit pixels in the virtual grid cells, sensitivity to *alpha* rates of lit pixels and to the size of the virtual grid cells, the complexity of its implementation and context dependence.

A comparative analysis of the rectangular Hough transform and the triangular Hough transform reveals that the rectangular Hough transform offers superior processing speed to the triangular Hough transform, making it ideal for applications requiring fast detection. As far as accuracy is concerned, visual illustrations of the results confirm the relevance of the lines detected by both methods, although parameter adjustments may be necessary to optimize performance. However, the rectangular Hough

transform is more flexible than the triangular Hough transform in that it offers targeted detection of vertical and horizontal lines.

In addition, to further improve our method, we combined parallel programming and virtual grids. Simulation results show that this hybrid approach reduces processing time for low α rates and small cell sizes. However, this efficiency diminishes as the value of α and/or cell size increases. It's worth pointing out that combining parallelism with triangular grids delivers better results than with rectangular, hexagonal and octagonal grids.

With regard to fingerprint recognition, our method consists, firstly, in using a virtual rectangular grid to reduce processing time, and secondly, in combining the virtual grid and parallelism to further improve processing time. Experimental results reveal that the virtual rectangular grid approach improved processing time with an acceleration factor $A = 1,60$. The combination of parallelism and virtual grid accelerated the fingerprint identification process for large databases (at least 500 fingerprints) with a speed-up factor of $A = 2,19$. For smaller databases (less than 500 fingerprints), this hybrid approach is less effective.

Finally, a toolbox named HoughVG with a graphical user interface, incorporating all algorithm implementations, has been developed.

Keywords : Hough transform, patterns recognition, virtual grid, parallelism, reconstruction.

Résumé

L'objectif principal de cette thèse est de développer une boîte à outils pour la transformée de Hough afin de détecter efficacement des objets discrets dans les images, en intégrant des techniques d'optimisation temporelle, de programmation parallèle et d'adaptation spécifique pour différents types d'applications. Pour ce faire, les mathématiques discrètes (topologie et géométrie) nous ont permis, d'une part de proposer et de démontrer des résultats relatifs aux images numériques. D'autre part, nous avons développé des algorithmes que nous avons simulés afin de corroborer nos résultats théoriques. Ces résultats théoriques et leurs simulations concernent la détection de lignes droites et la reconnaissance d'empreintes digitales.

Dans le cas de la détection des droites discrètes, la méthode proposée consiste à superposer une grille virtuelle (rectangulaire, triangulaire, hexagonale ou octogonale) sur l'image et à calculer le dual des cellules de cette grille virtuelle dont le taux ou pourcentage de pixels allumés dépasse un certain pourcentage α donné ($0\% \leq \alpha \leq 100\%$). Le seuil de vote de la transformée de Hough Standard est un critère qui détermine le nombre de votes nécessaires pour qu'une ligne soit considérée comme détectée. C'est un paramètre important qui influence la qualité et la fiabilité de la détection des lignes dans une image. Un faible seuil de vote inclut le bruit et, par conséquent, dégrade la qualité de la détection. Notre approche exploite cette limitation pour optimiser la reconnaissance analytique des lignes droites. Les simulations ont été réalisées sur des images d'un immeuble et d'une autoroute. Les résultats ont démontré la capacité de la méthode à détecter les lignes droites, tout en offrant une flexibilité supplémentaire grâce à la possibilité d'ajuster les paramètres de la grille virtuelle. L'approche des grilles virtuelles a permis la réduction du temps de calcul avec un facteur d'accélération $A = 3,26$ (sur l'image de l'immeuble) et $A = 1,9$ (sur l'image de l'autoroute) avec la grille rectangulaire, un facteur d'accélération $A = 2,28$ (sur l'image de l'immeuble) et $A = 1,31$ (sur l'image de l'autoroute) avec la grille triangulaire, un facteur d'accélération $A = 3,5$ (sur l'image de l'immeuble) et $A = 1,05$ (sur l'image de l'autoroute) avec la grille hexagonale, un facteur d'accélération $A = 2,22$ (sur l'image de l'immeuble) et $A = 1,9$ (sur l'image de l'autoroute) avec la grille octogonale. Du fait de la réduction du nombre de pixels à calculer grâce à la variable α et la taille des cellules de la grille virtuelle, la méthode offre des économies computationnelles importantes. Cependant, malgré ses avantages, la méthode proposée dans le cadre de la détection des droites discrètes n'est pas exempte de limitations. Ce sont : la perte d'informations présentes dans l'ensemble complet des pixels allumés des cellules de la grille virtuelle, la sensibilité aux taux α de pixels allumés et à la taille des cellules de la grille virtuelle, la complexité de sa mise en œuvre et la dépendance au contexte.

Une analyse comparative de la transformée de Hough rectangulaire et de la transformée de Hough triangulaire révèle que La transformée de Hough rectangulaire traite plus rapidement les données que

la version triangulaire, la rendant idéale pour les applications nécessitant une détection rapide. En ce qui concerne la précision, les illustrations visuelles des résultats confirment la pertinence des lignes détectées par les deux méthodes, bien que des ajustements de paramètres puissent être nécessaires pour optimiser les performances. Cependant, la transformée de Hough rectangulaire est plus flexible que la transformée de Hough triangulaire en ce sens qu'elle offre une capacité de détection ciblée des droites verticales et horizontales.

De plus, dans l'objectif d'améliorer d'avantage notre méthode, nous avons combiné la programmation parallèle et les grilles virtuelles. Les résultats des simulations démontrent que cette approche hybride réduit le temps de traitement pour les valeurs basses du taux α ainsi que pour les cellules de petite taille. Cependant, cette efficacité diminue lorsque la valeur de α et/ou la taille des cellules augmente. Il convient de souligner que la combinaison du parallélisme avec les grilles triangulaires offre de meilleurs résultats qu'avec les grilles rectangulaires, hexagonales et octogonales.

En ce qui concerne la reconnaissance des empreintes digitales, notre méthode consiste, dans un premier temps, à utiliser une grille rectangulaire virtuelle pour réduire le temps de traitement, dans un deuxième temps, à combiner la grille virtuelle et le parallélisme en vue d'améliorer d'avantage le temps de traitement. Les résultats expérimentaux révèlent que l'approche basée sur la grille rectangulaire virtuelle a amélioré le temps de traitement avec un facteur d'accélération $A = 1.60$. La combinaison du parallélisme et de la grille virtuelle a accéléré le processus d'identification des empreintes digitales dans le cas des bases de données volumineuses (au moins 500 empreintes digitales) avec un facteur d'accélération $A = 2.19$. Pour les bases de données moins volumineuses (moins de 500 empreintes digitales), cette approche hybride est moins efficace.

Enfin, une boîte à outils nommée HoughVG doté d'une interface graphique utilisateur, regroupant l'ensemble des implémentations des algorithmes, a été implémentée.

Mots clés : transformée de Hough, reconnaissance de formes, grille virtuelle, parallélisme, reconstruction

Table des matières

Dédicace	i
Remerciements	ii
Abstract	iii
Résumé	v
Table des matières	xi
Sigles et acronymes	xi
Listes des algorithmes	xiii
Liste des figures	xix
Liste des tableaux	xxii
Introduction générale	1
1 Contexte de la recherche	1
2 Problématique	2
3 Objectif de la thèse	2
4 Hypothèses de recherche	3
5 Organisation du document	3
Publications personnelles	5
1 Panorama de la Transformée de Hough	6
Introduction	6
1 Notions de géométrie discrète	7
1.1 Notion de pavage	7

1.2	Topologie discrète	8
1.3	Modèles de discrétisation	9
1.4	Droite analytique passant par des hexagones ou des octogones	11
2	Notions des grilles virtuelles	12
3	Origine et historique de la transformation de Hough	13
4	Définition et étapes de la transformation de Hough Standard	14
5	Variantes de la Transformée de Hough	16
5.1	Transformée de Hough probabiliste	16
5.2	Transformée de Hough probabiliste progressive	17
5.3	Transformée de Hough pour la détection de courbes	18
5.4	Transformée de Hough pour la détection de cercles, d'arcs, et d'ellipses	18
5.5	Transformée de Hough pour la détection d'ellipses	18
5.6	Transformée de Hough généralisée	19
5.7	Transformée de Hough adaptative	19
5.8	Transformée de Hough basée sur l'algorithme Map-Reduce	19
5.9	Transformée de Hough rapide	20
5.10	Transformée de Hough rapide intégrée (Integrated Fast Hough Transform)	20
6	Avancées de la Transformée de Hough	20
6.1	Optimisation de l'espace d'accumulation	21
6.2	Utilisation d'espaces d'accumulation pyramidaux	21
6.3	Adaptation à des formes complexes	22
6.4	Détection multi-objet	22
6.5	Intégration avec l'apprentissage automatique	22
6.6	Optimisation matérielle	23
7	Applications de la transformée de Hough	23
7.1	Transport	24
7.2	Images satellitaires	25
7.3	Industrie	26
7.4	Domaine de l'intelligence artificielle	27
7.5	Médecine	27
7.6	Élevage	28
7.7	Réseaux de communication optique	29
7.8	Construction et bâtiment	29
7.9	Agriculture	30
7.10	Sécurité	32
8	Domaines émergents	33

Conclusion	34
2 Transformée de Hough Rectangulaire	35
Introduction	35
1 Description de la méthode	36
1.1 Maillage de l'image	36
1.2 Sélection des cellules	36
1.3 Parallélisation de la transformée de Hough rectangulaire	37
1.4 Reconnaissance des droites discrètes	39
2 Complexité de la Transformée de Hough Rectangulaire	40
3 Simulations et discussions	42
3.1 Cas de l'image de l'immeuble	43
3.2 Cas de l'image de l'autoroute	46
3.3 Comparaison de la Transformée de Hough Rectangulaire et Transformée de Hough Standard	49
3.4 Comparaison de la Transformée de Hough Rectangulaire et sa version parallélisée	51
3.5 Comparaison avec d'autres approches	54
Conclusion	55
3 Transformée de Hough Triangulaire	57
Introduction	57
1 Description de la méthode	58
1.1 Maillage triangulaire de l'image	58
1.2 Parallélisation de la transformée de Hough triangulaire	60
1.3 Sélection des cellules triangulaires	61
1.4 Reconnaissance des droites discrètes	63
2 Complexité de la transformée de Hough triangulaire	65
3 Simulations et discussions	66
3.1 Cas de l'image de l'immeuble	67
3.2 Cas de l'image de l'autoroute	69
4 Analyse comparative	72
4.1 Transformée de Hough Triangulaire et Transformée de Hough Standard	73
4.2 Transformée de Hough Triangulaire et sa version parallélisée	75
4.3 Transformée de Hough Rectangulaire et Transformée de Hough Triangulaire . .	77
Conclusion	81

4	Transformée de Hough Hexagonale	83
	Introduction	83
1	Description de la méthode	84
1.1	Maillage Hexagonal de l'image	84
1.2	Sélection des cellules de la grille virtuelle	85
1.3	Parallélisation de la transformée de Hough hexagonale	87
1.4	Reconnaissance des droites discrètes	89
2	Complexité de la transformée de Hough hexagonale	90
3	Simulations et discussions	93
3.1	Cas de l'image de l'immeuble	94
3.2	Cas de l'image de l'autoroute	97
3.3	Comparaison de transformée de Hough hexagonale et transformée de Hough standard	100
3.4	Comparaison de THH et sa version parallélisée	103
	Conclusion	106
5	Transformée de Hough Octogonale	107
	Introduction	107
1	Description de la méthode	108
1.1	Maillage de l'image	108
1.2	Sélection des cellules	108
1.3	Parallélisation de la transformée de Hough octogonale	111
1.4	Reconnaissance des droites discrètes	112
2	Complexité de la transformée de Hough octogonale	113
3	Simulations et discussions	115
3.1	Cas de l'image de l'immeuble	117
3.2	Cas de l'image de l'autoroute	119
3.3	Comparaison de la Transformée de Hough Octogonale et Transformée de Hough Standard	121
3.4	Comparaison de la Transformée de Hough Octogonale et la Transformée de Hough Octogonale Parallélisée	124
	Conclusion	126
6	Reconnaissance d'Empreintes Digitales dans une Grille Virtuelle	128
	Introduction	128
1	La biométrie	129
1.1	Définition	129

1.2	Les systèmes biométriques	129
2	Les empreintes digitales	130
2.1	Historique	130
2.2	Caractéristiques des empreintes digitales	130
3	Structure d'un système de reconnaissance d'empreintes digitales	131
3.1	Acquisition d'empreintes digitales	133
3.2	Pré-traitement	133
3.3	Extraction des minuties	133
3.4	Comparaison des minuties	134
4	Méthodologie	136
5	Expérimentations et discussions	138
5.1	Performance temporelle	139
5.2	Précision	141
	Conclusion	142
7	Architecture de la Boîte à Outils	143
	Introduction	143
1	Conception	144
1.1	Cas d'utilisation	144
1.2	Description	145
1.3	Les dépendances	148
2	Interface graphique	148
2.1	Scénarios d'utilisation du système de détection de droites	148
2.2	Scénarios d'utilisation du système de reconnaissance d'empreintes digitales	150
3	Impact	150
4	Limitations	151
5	Validation des hypothèses de recherche	151
	Conclusion	152
	Conclusion générale	154

Liste des Algorithmes

1	Reconnaissance de droites discrètes par la Transformée de Hough Standard	17
2	Mise à jour de l'accumulateur	18
3	Construction de la grille rectangulaire	36
4	Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire	38
5	Mise à jour de l'accumulateur	38
6	Taux de pixels allumés	39
7	Mise à jour parallèle de l'accumulateur	40
8	Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire Pa- rallélisée	41
9	Reconstruction des droites détectées	41
10	Construction de la grille triangulaire	59
11	Mise à jour de l'accumulateur	60
12	Taux de pixels allumés	61
13	Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire	62
14	Mise à jour parallèle de l'accumulateur	63
15	Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire Parallélisée	64
16	Reconstruction des droites détectées	65
17	Construction de la grille hexagonale	85
18	Calcul du taux de pixels allumés	87
19	Mise à jour de l'accumulateur	88
20	Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 1)	89
21	Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 2)	90
22	Sélection de tous les pixels compris entre A et B.	91
23	Mise à jour parallèle de l'accumulateur	91

24	Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée (partie 1)	92
25	Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée (partie 2)	93
26	Reconstruction des droites détectées	93
27	Construction de la grille octogonale	109
28	Mise à jour de l'accumulateur	111
29	Taux de pixels allumes	112
30	Reconnaissance de droites discrètes par la transformée de Hough octogonale	113
31	Mise à jour parallèle de l'accumulateur	114
32	Reconnaissance de droite discrètes par la Transformée de Hough Octogonale Parallélisée	115
33	Reconstruction des droites détectées	115
34	Transformée de Hough Généralisée	135
35	Recherche des paires de minuties correspondantes de deux empreintes digitales	136
36	Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle.	137
37	Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle combiné au parallélisme	138

Table des figures

1	Organisation du document	4
1.1	Pavage régulier [1]	8
1.2	Pavage irrégulier [1]	8
1.3	Exemple de voisinage en dimension 2 : (a) Un pixel et ses quatre 1-voisins. (b) Le même pixel et ses huit 0-voisins. [2]	9
1.4	Représentation de la droite discrète arithmétique de Reveillès $D(5, 8, -1, 8)$, les inégalités sont $-1 \leq 5x - 8y < 7$. (a) chaque point discret (x, y) est étiqueté par $5x - 8y$. (b) la droite discrète obtenue [2]	10
1.5	Segments de droites discrètes : (a) Mince, (b) Naïf, (c) Standard, (d) Épais	10
1.6	(a) Un hexagone dans un pixel carré; (b) Un octogone dans un pixel carré [3]	12
1.7	Une grille virtuelle isothétique	13
1.8	Un point A et son dual	15
1.9	Une droite et son dual	15
1.10	Les étapes de la technique de la transformée de Hough standard	16
1.11	Images pour les expérimentations	16
1.12	Images binaires pour les expérimentations	17
2.1	Cellule rectangulaire	37
2.2	Grille rectangulaire superposée à l'espace image	37
2.3	Les étapes de la technique de la transformée de Hough rectangulaire	42
2.4	Courbe représentative des données du tableau 2.1	44
2.5	(a) Détection de lignes verticales de l'immeuble avec la THR ($l = 168, L = 1, S_R = 168, \alpha = 30\%, seuil = 50$), (b) détection de bords de l'autoroute avec la THR ($l = 29, L = 238, S_R = 6902, \alpha = 10\%, seuil = 40$)	45
2.6	Courbe représentative des données du tableau 2.2	46
2.7	Courbe représentative des données du tableau 2.3	47
2.8	Courbe représentative des données du tableau 2.4	48

2.9	(a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40); (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)	49
2.10	(a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20); (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)	50
2.11	Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=40, $\alpha = 40\%$, $L = 5$ et $l = 3$; (b) Seuil=40, $\alpha = 40\%$, $L = 5$ et $l = 3$	51
2.12	Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=51, $\alpha = 40\%$, $L = 5$ et $l = 3$; (b) Seuil=45, $\alpha = 25\%$, $L = 7$ et $l = 5$	52
2.13	Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et $seuil = 70$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres ($seuil = 50$, $\alpha = 30\%$) des algorithmes 4 et 8	52
2.14	Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et $seuil = 40$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres ($seuil = 40$, $\alpha = 30\%$) des algorithmes 4 et 8	53
2.15	Lignes droites détectées sur le pentagone avec l'approche de Traoré et al [4]	54
2.16	Lignes droites détectées sur l'autoroute avec l'approche de Traoré et al [4]	54
2.17	Lignes droites détectées sur l'immeuble avec l'approche de Traoré et al [4]	54
2.18	Lignes droites détectées avec notre méthode	55
3.1	Deux triangles rectangles (ABC) et (FGH) avec respectivement des droites parallèles (AB) et (DE), et (FG) et (KL).	59
3.2	Une grille triangulaire superposée à l'espace image	59
3.3	Sélection des pixels d'une cellule triangulaire	62
3.4	Les étapes de la technique de la transformée de Hough triangulaire	65
3.5	Représentation graphique des données du tableau 3.1	68
3.6	(a) THT sur l'immeuble ($h = 219$, $b = 178$, $\alpha = 48$, $seuil = 61$), (b) THT sur l'autoroute ($h = 92$, $b = 300$, $\alpha = 10$, $seuil = 60$)	69
3.7	Représentation graphique des données du tableau 3.2	70
3.8	Représentation graphique des données du tableau 3.3	71
3.9	Représentation graphique des données du tableau 3.4	72

3.10 (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40); (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)	73
3.11 (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20); (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)	74
3.12 Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=38, $\alpha = 45\%$, $h = 5$ et $b = 3$; (b) Seuil=38, $\alpha = 40\%$, $h = 7$ et $b = 5$	74
3.13 Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=57, $\alpha = 30\%$, $h = 5$ et $b = 3$; (b) Seuil=35, $\alpha = 30\%$, $h = 7$ et $b = 5$	75
3.14 Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($h = 3$, $b = 5$ et $Seuil = 60$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules triangulaires et les paramètres ($Seuil = 80$, $\alpha = 20\%$) des l'algorithmes 13 et 15	76
3.15 Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($h = 4$, $b = 2$ et $Seuil = 60$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules triangulaires et les paramètres ($Seuil = 60$, $\alpha = 20\%$) des l'algorithmes 13 et 15	77
3.16 Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=35, $\alpha = 0.35$, $L = 7$ et $l = 5$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.4$, $h = 7$ et $b = 5$.	79
3.17 Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=90, $\alpha = 0.1$, $L = 5$ et $l = 3$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.45$, $h = 5$ et $b = 3$.	80
3.18 Détection de lignes droites sur l'image 1.11 de l'autoroute avec, (a) le THR avec les paramètres : Seuil=51, $\alpha = 0.4$, $L = 5$ et $l = 3$; (b) le THT avec les paramètres : Seuil=57, $\alpha = 0.3$, $h = 5$ et $b = 3$.	80
4.1 Un hexagone ABCDEF	85
4.2 Représentation des pixels d'une cellule hexagonale	85
4.3 Une grille hexagonale superposée sur un espace image	86
4.4 La prise en compte des pixels exclus d'une cellule hexagonale	86
4.5 Les étapes de la technique de la transformée de Hough hexagonale	94
4.6 Courbe représentative des données du tableau 4.1	96

4.7	Courbe représentative des données du tableau 4.2	96
4.8	Courbe représentative des données du tableau 4.3	98
4.9	Courbe représentative des données du tableau 4.4	99
4.10	(a) THH sur l'immeuble ($\gamma = 64, \alpha = 14, seuil = 62$), (b) THH sur l'autoroute ($\gamma = 30, \alpha = 10, seuil = 60$)	100
4.11	(a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=25); (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)	101
4.12	(a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=67); (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)	101
4.13	Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres : (a) Seuil=50, $\alpha = 30\%$, $\gamma = 3$; (b) Seuil=25, $\alpha = 37\%$, $\gamma = 4$	102
4.14	Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres : (a) Seuil=39, $\alpha = 50\%$, $S_H = 6$ pour $\gamma = 2$; (b) Seuil=67, $\alpha = 25\%$, $S_H = 20$ pour $\gamma = 3$	103
4.15	Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=60, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=50, $\alpha = 30\%$) des l'algorithmes 20 et 24	104
4.16	Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=60, $\gamma = 3$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=60, $\alpha = 10\%$) des l'algorithmes 20 et 24	105
5.1	Octogone ABCDEFGH	109
5.2	Grille octogonale superposée à l'espace image.	110
5.3	Mise en évidence des pixels de l'octogone ABCDEFGH	110
5.4	Les étapes de la technique de la transformée de Hough octogonale	116
5.5	Courbe représentative des données du tableau 5.1	118
5.6	Courbe représentative des données du tableau 5.2	119
5.7	Courbe représentative des données du tableau 5.3	120
5.8	Courbe représentative des données du tableau 5.4	121
5.9	(a) THO sur l'immeuble ($\gamma = 54, \alpha = 11\%, seuil = 50$), (b) THO sur l'autoroute ($\gamma = 33, \alpha = 10, seuil = 40$)	122

5.10	(a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40); (b) Détection de lignes droites sur l'image1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)	123
5.11	(a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20); (b) Détection de lignes droites sur l'image1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)	124
5.12	Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=53, $\alpha = 30\%$, $\gamma = 2$; (b) Seuil=51, $\alpha = 25\%$, $\gamma = 3$	124
5.13	Détection de lignes droites sur l'image1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=38, $\alpha = 30\%$, $\gamma = 2$; (b) Seuil=40, $\alpha = 20\%$, $\gamma = 3$	125
5.14	Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=100, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=70, $\alpha = 20\%$) des l'algorithmes 30 et 32	125
5.15	Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (threshold=70, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (threshold=50, $\alpha = 0, 2$) des l'algorithmes 30 et 32	126
6.1	Empreinte digitale [5]	131
6.2	Les différents types de minutie [5]	131
6.3	Les trois principales classes d'empreintes, boucle (a), spire (b), arche (c) [5]	132
6.4	Architecture générale d'un système complet de reconnaissance d'empreintes	132
6.5	Capteur capacitif [6]	133
6.6	(a) Bifurcation; (b) Terminaison	134
6.7	Superposition de deux empreintes digitales	135
6.8	Parallélisation de l'appariement des empreintes digitales. (a) Empreinte identifiée; (b) Empreinte non identifiée	139
6.9	Appariement de deux empreintes digitales avec une grille rectangulaire	140
6.10	Comparaison du temps de traitement en fonction du nombre d'empreintes	140
6.11	Matrices de confusion	142
7.1	Interactions entre les utilisateurs et le système	144
7.2	Activités dans la boîte à outils	145
7.3	Description de la structure interne de HoughVG	146
7.4	Tableau de bord	149

7.5	Séquence des scénarios d'utilisation du système de détection des droites	149
7.6	Séquence des scénarios d'utilisation du système de reconnaissance d'empreintes digitales	150

Liste des tableaux

2.1	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres fixes ($l = 3$, $L = 5$ and $Seuil = 70$) et α variant entre 10% et 45% où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.	43
2.2	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres fixes ($\alpha = 30\%$ et $Seuil = 50$) et $1 \leq L \leq Nl$ et $1 \leq l \leq Nc$ où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.	44
2.3	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres fixes ($l = 3$, $L = 5$ and $Seuil = 40$) et α variant entre 10% et 50% où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.	47
2.4	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres fixes ($\alpha = 30\%$ et $Seuil = 40$), $1 \leq L \leq Nl$ et $1 \leq l \leq Nc$ où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.	48
2.5	Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)	49
2.6	Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)	49
2.7	Résultats de simulation avec la THR appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)	51
2.8	Résultats de simulation avec la THR appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)	51
2.9	Approche proposée	53
2.10	Approche de Traoré et al [4]	55

3.1	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres fixes ($h = 5$, $b = 3$ et $Seuil = 60$) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.	68
3.2	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres fixes ($\alpha = 20\%$ et $Seuil = 80$), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$, où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.	69
3.3	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres fixes ($h = 4$, $b = 2$ et $Seuil = 60$) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.	70
3.4	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres fixes ($\alpha = 20\%$ et $Seuil = 60$), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$, où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.	72
3.5	Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)	73
3.6	Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites; time2=temps de détection d'une seule droite)	73
3.7	Résultats de simulation avec la THT sur l'image de l'immeuble (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)	74
3.8	Résultats de simulation avec la THT sur l'image de l'autoroute (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)	75
4.1	Resultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres fixes ($Seuil=62$ et $\alpha = 14\%$) et γ variant entre 2 et 64	95
4.2	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres fixes ($Seuil=60$ and $\gamma = 4$) et α variant entre 10% et 35%	96
4.3	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la THH avec les paramètres fixes ($Seuil=60$ and $\alpha = 10\%$) et γ variant entre 2 et 40	98
4.4	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la THH avec les paramètres fixes ($Seuil=60$ and $\gamma = 3$) et α variant entre 10% et 30%	99
4.5	Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites; time2=temps de détection d'une seule droite)	100
4.6	Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites; time2=temps de détection d'une seule droite)	100
4.7	Résultats de simulation avec la THH appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)	102

4.8	Résultats de simulation avec la THH appliquée sur l'image 1.11(a) de l'autoroute (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)	102
5.1	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres fixes ($\gamma = 3$ et $Seuil = 60$) et α variant entre 10% et 35% .	117
5.2	Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres fixes ($Seuil = 50$ et $\alpha = 20\%$) et γ variant entre 2 et 82 . .	118
5.3	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la THO avec les paramètres fixes ($\gamma = 3$ et $Seuil = 40$) et α variant entre 10% et 30% .	120
5.4	Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la THO avec les paramètres fixes ($Seuil=40$ et $\alpha = 20\%$) et γ variant entre 2 et 56 . . .	121
5.5	Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)	122
5.6	Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)	123
5.7	Résultats de simulation avec la THO appliquée sur l'image 1.11(c) de l'immeuble (temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite détectée)	123
5.8	Résultats de simulation avec la THO appliquée sur l'image 1.11(a) de l'autoroute (temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite détectée)	124
6.1	Comparaison du temps de traitement en fonction du nombre d'empreintes	141
6.2	Structure générale d'une matrice de confusion	142
7.1	Facteurs d'accélération	152
7.2	Comparaison du temps de traitement de l'approche hybride et celle basée sur les grilles virtuelles	152

Introduction Générale

1 Contexte de la recherche

Dans le domaine de la vision par ordinateur et du traitement d'image, la détection d'objets discrets est une problématique essentielle qui trouve de nombreuses applications pratiques. Que ce soit pour la reconnaissance de formes [7], la conduite autonome [8], la détection des routes [9–12], la surveillance de sécurité, l'élevage [13], la construction des bâtiments [14], la chiologie [15], l'agriculture [16–18], la médecine [19, 20], les véhicules autonomes [21–23], la capacité à identifier et localiser des objets spécifiques dans une image de façon précise et rapide est cruciale de nos jours où la rapidité et l'efficacité sont devenues des exigences quotidiennes.

Pour illustrer ce fait, prenons l'exemple¹ de l'incident qui s'est produit en Arizona au États-Unis le 18 mars 2018 où une piétonne traversant un passage piéton de nuit a été mortellement heurté par un automobile autonome de type Volvo réaménagé. Cet exemple montre à quel point les algorithmes des systèmes des véhicules autonome doivent être optimisé pour les permettre de détecter rapidement et avec précision les obstacles pour éviter les accidents.

Parmi les différentes approches de détection des objets discrets, la transformée de Hough est largement utilisée depuis son introduction en 1962 par Paul Hough [24]. Au fil des années, les chercheurs ont développé plusieurs variations [7, 25–42] de cette technique qui a fait ses preuves en identifiant avec succès des formes géométriques, telles que des lignes [43–45], des cercles [46] ou des ellipses [47], des courbes [32, 48–51], ainsi que des formes arbitraires [52, 53] dans des images complexes, même en présence de bruit et d'occlusions partielles.

La transformation de Hough repose sur une représentation paramétrique des formes utilisant un espace image et un espace de paramètre. Cette approche permet de détecter des objets indépendamment de leur position, leur orientation et leur échelle, offrant ainsi une grande robustesse dans des scénarios variés. En effet, au lieu de rechercher directement les objets dans l'image, la transformée de Hough convertit le problème de détection en une recherche de correspondances dans un espace de paramètres.

1. https://www.bfmtv.com/economie/entreprises/industries/accident-mortel-le-logiciel-d-uber-etait-incapable-de-reconnaitre-un-pieton-traversant-hors-des-clous_AN-201911060101.html

Cette représentation paramétrique permet de traiter les variations géométriques des objets, ce qui la rend particulièrement adaptée à la détection de formes régulières telles que les lignes, les cercles et les ellipses. Cependant, elle peut être coûteuse en termes de temps de traitement. La méthode traditionnelle de la transformée de Hough nécessite de calculer et d'accumuler les votes pour chaque pixel de l'image, ce qui peut être intensif en ressources computationnelles, en particulier pour des images de haute résolution ou dans des applications nécessitant un traitement en temps réel. Cette contrainte de temps de traitement peut limiter l'applicabilité de la transformée de Hough dans des applications pratiques où des performances rapides sont essentielles.

2 Problématique

La détection d'objets discrets dans des images reste un défi majeur en vision par ordinateur. Bien que la transformation de Hough soit une méthode bien établie pour cette tâche, son implémentation peut être complexe et exigeante en termes de ressources.

Ainsi, comment concevoir une boîte à outils basée sur la transformée de Hough, optimisée pour des performances temporelles accrues et adaptée spécifiquement à la détection d'objets discrets, tels que les lignes dans des images et les caractéristiques des empreintes digitales ? Pour résoudre cette problématique, nous devons répondre à plusieurs questions clés :

- comment réduire le temps de traitement de la transformée de Hough en utilisant des grilles virtuelles, pour améliorer la détection des objets discrets ?
- quel impact la combinaison des grilles virtuelles et du parallélisme a-t-elle sur les performances de la détection des objets discrets par la transformée de Hough ?
- quels sont les défis et les opportunités rencontrés lors de l'implémentation d'une boîte à outils intégrant les solutions proposées ?
- comment cette boîte à outils peut-elle être utilisée dans des applications pratiques de traitement d'images ?

3 Objectif de la thèse

L'objectif principal de cette thèse est de développer une boîte à outils de la transformée de Hough en vue de détecter efficacement des objets discrets dans les images, en intégrant des techniques d'optimisation temporelle, pour différents types d'applications, notamment la détection de lignes droites et la reconnaissance d'empreintes digitales.

Pour atteindre cet objectif global, nous nous fixons plusieurs sous-objectifs :

- optimiser le temps de traitement de la transformée de Hough en utilisant des grilles virtuelles : pour améliorer la détection des objets discrets dans les images, notre premier sous-objectif

consiste à optimiser le temps de traitement de la transformée de Hough en utilisant des grilles virtuelles. Nous cherchons à développer des méthodes efficaces pour superposer ces grilles sur les images, en explorant différents types telles que les grilles rectangulaires, triangulaires, hexagonales et octogonales et à évaluer les performances des différents types de grilles virtuelles pour la transformée de Hough, en termes de précision et de temps de calcul.

- combiner la méthode des grilles virtuelles et le parallélisme : notre deuxième sous-objectif est de combiner l'approche des grilles virtuelles et la programmation parallèle. Nous visons à évaluer l'impact de cette approche hybride sur le temps de traitement. Notre objectif est d'identifier les configurations de grille pour lesquelles le parallélisme offre les meilleurs gains de performance.
- développer une boîte à outils : notre troisième sous-objectif est de développer une boîte à outils intégrant les implémentations des algorithmes de l'approche basée sur les grilles virtuelles et de l'approche Hybride.

4 Hypothèses de recherche

Pour orienter notre recherche, nous formulons plusieurs hypothèses clés qui guideront le développement et l'optimisation de la transformée de Hough :

- l'intégration des grilles virtuelles dans la méthode de la transformée de Hough contribue à réduire le temps de traitement tout en maintenant une bonne précision dans la détection d'objets discrets.
- La combinaison du parallélisme et des grilles virtuelles permet d'accélérer le temps de traitement et de maintenir une bonne précision.

5 Organisation du document

Ce document est organisé comme suit :

- L'introduction générale présente le sujet de la recherche, explique le contexte et expose les questions, les objectifs et les hypothèses de recherche.
- Le chapitre 1 (Panorama de la transformée de Hough) examine les bases théoriques de la TH, son évolution historique, ainsi que les avancées récentes dans le domaine. Elle explore également les différentes variantes de la TH et leurs applications dans divers domaines, de la vision par ordinateur à la médecine en passant par l'industrie et l'agriculture.
- Les chapitres 2, 3, 4 et 5 explorent en détails les variantes de la transformée de Hough proposées dans le cadre de cette thèse, notamment la transformée de Hough rectangulaire, la transformée de Hough triangulaire, la transformée de Hough hexagonale et la transformée de Hough octogonale respectivement.

- Le chapitre 6 présente une application pratique de la THG dans le domaine de la biométrie, en particulier dans la reconnaissance d’empreintes digitales. Elle décrit une approche basée sur l’utilisation des grilles virtuelles pour accélérer le processus de reconnaissance d’empreintes digitales.
- Le chapitre 7 présente l’environnement de travail et l’architecture logicielle de la boîte à outils. Il détaille les différentes ses fonctionnalités et offre des conseils pratiques pour son utilisation.
- La conclusion générale résume les principaux résultats et contributions de la recherche, discute des limites et propose des pistes pour des recherches futures.

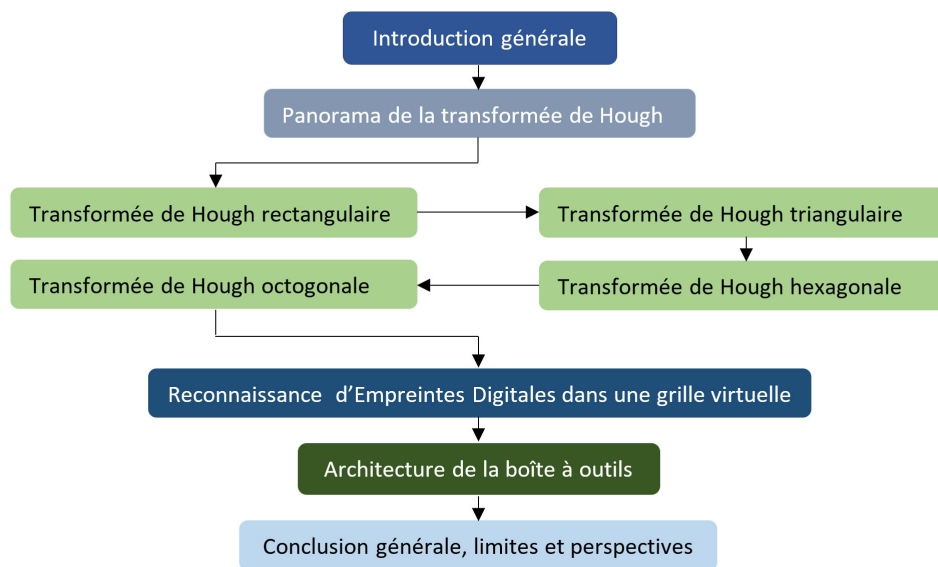


FIGURE 1 – Organisation du document

Publications personnelles

- Moïse Ouedraogo, Abdoulaye Sere, Cheick Amed Diloma Gabriel Traore¹, HoughVG :Hough Transform Toolbox for Straight-Line Detection and Fingerprint Recognition, Elsevier, Software Impacts, 2024, <https://doi.org/10.1016/j.simpa.2024.100709>
- Moïse Ouedraogo, Abdoulaye Sere, B.M.J. Some, C.A.G.D. Traore. *Straight-Line Recognition Using a Triangular Grid*. In : Arai, K. (eds) *Advances in Information and Communication*. FICC 2022. Lecture Notes in Networks and Systems, vol 438. Springer, Cham. 2022 https://doi.org/10.1007/978-3-030-98012-2_45
- Moïse Ouedraogo, Abdoulaye Séré, *Straight-Line Recognition in a Virtual Hexagonal Grid using Hough Transform*, in EAI INTERSOL, 2024
- Abdoulaye Sere, Moïse Ouedraogo and Armand Kodjo Atiampo, *Parallel Hough Transform based on Object Dual and Pympp Library*. International Journal of Advanced Computer Science and Applications(IJACSA), 13(10), 2022. <http://dx.doi.org/10.14569/IJACSA.2022.0131084>
- A. Zerbo, M. Ouedraogo, A. Sere (2023). *Parallel Fingerprint Recognition Using Generalized Hough Transform in a Virtual Grid*. In : Arai, K. (eds) *Proceedings of the Future Technologies Conference (FTC) 2023*, Volume 3. FTC 2023. Lecture Notes in Networks and Systems, vol 815. Springer, Cham. https://doi.org/10.1007/978-3-031-47457-6_35
- Ali Zerbo, Moïse Ouedraogo, Abdoulaye Sere, Mamadou Diarra, *Comparison of Joblib and Pympp for Parallel Fingerprint Recognition*, EAI JRI, 2024, <http://dx.doi.org/10.4108/eai.18-12-2023.2348107>

Panorama de la Transformée de Hough

Introduction	6
1 Notions de géométrie discrète	7
2 Notions des grilles virtuelles	12
3 Origine et historique de la transformation de Hough	13
4 Définition et étapes de la transformation de Hough Standard	14
5 Variantes de la Transformée de Hough	16
6 Avancées de la Transformée de Hough	20
7 Applications de la transformée de Hough	23
8 Domaines émergents	33
Conclusion	34

Introduction

La Transformée de Hough, proposée initialement en 1962 par Paul Hough, est une méthode adaptée à la détection de formes géométriques dans des images numériques bruitées [24]. Au fil des années, cette technique a été largement utilisée dans le domaine de la vision par ordinateur pour détecter des objets discrets tels que des lignes droites, des cercles et des ellipses, et des formes géométriques paramétriques et arbitraires.

La Transformée de Hough a connu de nombreuses évolutions et des variantes ont été développées pour relever les défis associés à la détection d’objets dans des images réelles. Des recherches appro-

fondies ont été entreprises pour améliorer l'efficacité et la précision de cette méthode, tout en tenant compte des contraintes de temps de calcul et des ressources matérielles.

Plusieurs travaux scientifiques sur la transformée de Hough ont été réalisés dans plusieurs domaines clés. Parmi ces travaux, les techniques d'optimisation visant à réduire la complexité du calcul de l'accumulateur, les approches probabilistes [54] permettant une détection plus rapide et efficace d'objets discrets, ainsi que les adaptations spécifiques pour la détection de formes non-linéaires et complexes [7, 32] dans des images ont été réalisées.

Les applications de la Transformée de Hough s'étendent à divers domaines tels que le transport [55–57] pour améliorer la détection des voies par les véhicules autonomes, les images satellitaires [22, 58] améliorer la précision de la détection des satellites, l'industrie [59, 60] dans le but d'améliorer les systèmes automatique de production, la sécurité [61–64], la détection d'objets dans les images médicales [19, 20, 65]. Son utilité dans des environnements variés témoigne de sa polyvalence et de son efficacité dans la résolution de problèmes pratiques.

Cependant, malgré les succès obtenus, des défis subsistent et suscitent un intérêt continu pour la recherche en Transformée de Hough. Parmi ces défis, on peut citer la prise en compte d'occlusions, la détection d'objets dans des environnements bruités, l'optimisation de l'accumulateur pour la détection rapide et précise, ainsi que l'extension de la méthode pour des tâches de détection plus complexes et spécifiques.

Dans ce chapitre, nous donnerons d'abord quelques notions de la géométrie discrète, ensuite nous explorerons l'origine et l'historique de la Transformée de Hough, ses principes fondamentaux, les avancées de la transformée de Hough. Nous mettrons également l'accent sur les différentes applications et les domaines émergents où la Transformée de Hough continue de jouer un rôle essentiel dans l'analyse d'images numériques.

1 Notions de géométrie discrète

Cette section est consacrée aux notations et définitions des éléments de la géométrie discrète. Quelques relations topologiques (voisinage, connexité) entre les différents éléments composant les espaces discrets sont présentées.

1.1 Notion de pavage

Définition 1 (Pavé, espace discret, pavage, objet discret, partie pavable [66])

- (i) *Un pavé est une cellule (composant élémentaire) d'un espace discret ;*
- (ii) *Un espace discret est un sous-ensemble de points isolés (points discrets) de l'espace euclidien $\mathbb{R}^n$;*

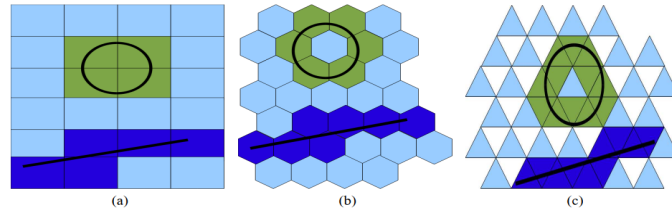


FIGURE 1.1 – Pavage régulier [1]

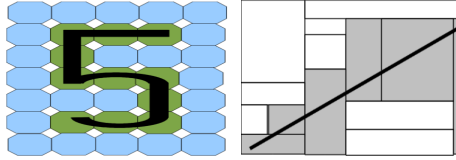


FIGURE 1.2 – Pavage irrégulier [1]

- (iii) Un pavage est une partition dénombrable de compacts (pavés) d'intérieurs non vide dans un espace euclidien ;
- (iv) Un pavage régulier (respectivement irrégulier) est une partition dénombrable de compacts (pavés) identiques (respectivement non-identiques) d'intérieurs non vide dans un espace euclidien ;
- (v) Un objet discret est un ensemble de pavés ;
- (vi) Une partie pavable de $\mathbb{R}^n$ est toute réunion d'un nombre fini de pavés fermés et bornés de $\mathbb{R}^n$.

Définition 2

- (i) Une grille régulière est un espace discret constitué de pavés identiques
- (ii) Une grille irrégulière est un espace discret constitué de pavés non-identiques

La figure 1.1 montre trois exemples de grilles régulières et la figure 1.2 montre deux exemples de grilles irrégulières.

Remarque 1

- Une image numérique est vue comme un espace discret où chaque point lumineux de l'image est un point discret dans l'espace discret.
- La discrétisation est le fait de décomposer un espace (une image) en points discrets.

1.2 Topologie discrète

Dans cette section, nous introduisons les notions de voisinage et de connexité entre points discrets afin établir des relations d'adjacence et d'incidence entre les pavés d'un espace discret.

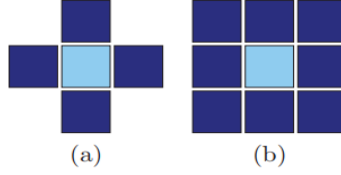


FIGURE 1.3 – Exemple de voisinage en dimension 2 : (a) Un pixel et ses quatre 1-voisins. (b) Le même pixel et ses huit 0-voisins. [2]

Dans le cadre des espaces discrets classiques, nous utilisons la définition 3 de voisinage entre des points discrets.

Définition 3 (k -voisinage [67]) Soit $k \in \llbracket 0, n-1 \rrbracket$. Deux points discrets $p = (p_1, \dots, p_n) \in \mathbb{Z}^n$ et $q = (q_1, \dots, q_n) \in \mathbb{Z}^n$ sont dits k -voisins si $\forall i \in \llbracket 1, n \rrbracket, |p_i - q_i| \leq 1$ et $k \leq n - \sum_{i=1}^n |p_i - q_i|$.

En pratique, dans un espace discret de dimension 2, deux pixels sont 1-voisins s'ils ont une arête commune et 0-voisins s'ils ont une arête commune ou un sommet commun. La figure 1.3 donne une illustration de voisinage en dimension 2.

Remarque 2 ([4]) Deux points discrets k -voisins sont aussi k -adjacents. D'où l'équivalence des notions de k -voisinage et de k -adjacence. En dimension 2 si : p et q sont 0-voisins sans être 1-voisins alors $d(p; q) = \sqrt{2}$ unité ; p et q sont 1-voisins alors $d(p; q) = 1$ unité.

1.3 Modèles de discrétisation

La discrétisation est l'opération qui transforme un objet continu dans un espace euclidien en un objet discret décrit dans un espace discret. La discrétisation d'un objet continu se fait grâce à l'utilisation de méthodes ou modèles de discrétisation telles que les modèles Naïfs, Super-ouverture et Standard [68].

Définition 4 (Droite discrète analytique arithmétique [69]) Une droite discrète de paramètres (a, b, μ) et épaisseur arithmétique ω est définie comme l'ensemble des points entiers (x, y) vérifiant la double inégalité $\mu \leq ax + by < \mu + \omega$ avec $(a, b, \mu, \omega) \in \mathbb{Z}^4$ telle que a et b soient premiers entre eux. Une telle droite est notée $D(a, b, \mu, \omega)$. μ est la borne inférieure de la droite discrète et $\frac{a}{b}$ sa pente.

Cette première définition de droite analytique discrète fut généralisée aux hyperplans analytiques discrets par Jean Pierre Reveillès [69].

Définition 5 (Hyperplans analytiques discrets [69]) En dimension n , l'hyperplan analytique discret de paramètres $A = (a_1, \dots, a_n) \in \mathbb{R}_n$, $\mu \in \mathbb{R}$ et $\omega \in \mathbb{R}$ est l'ensemble des points discrets $X = (x_1, \dots, x_n) \in \mathbb{R}^n$ vérifiant $\mu \leq \sum_{i=1}^n a_i x_i < \mu + \omega$.

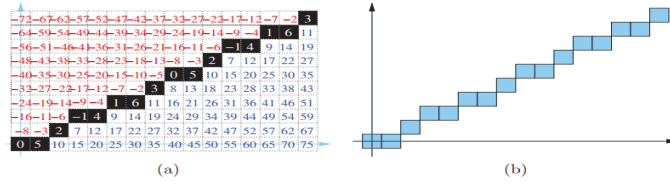


FIGURE 1.4 – Représentation de la droite discrète arithmétique de Reveillès $D(5, 8, -1, 8)$, les inégalités sont $-1 \leq 5x - 8y < 7$. (a) chaque point discret (x, y) est étiqueté par $5x - 8y$. (b) la droite discrète obtenue [2]

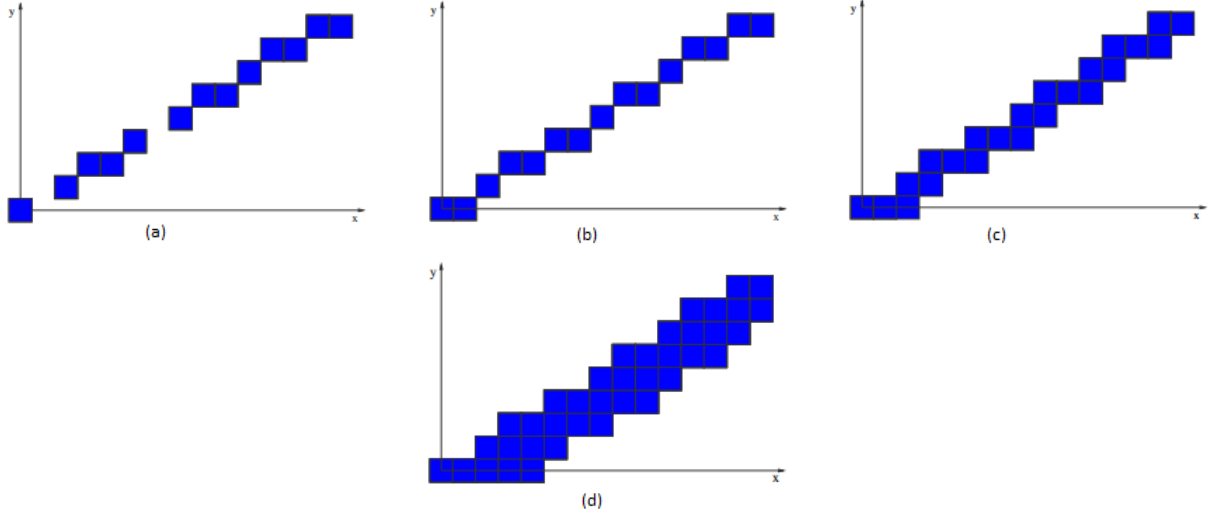


FIGURE 1.5 – Segments de droites discrètes : (a) Mince, (b) Naïf, (c) Standard, (d) Épais

Si l'inégalité de droite est large, on parle d'hyperplan analytique discret fermé.

En fonction de son épaisseur arithmétique, un hyperplan analytique peut être qualifié de diverses manières.

Définition 6 (Hyperplan analytique discret [67]) Soit H un hyperplan analytique discret de paramètres $A = (a_1, \dots, a_n) \in \mathbb{R}_n$, $\mu \in \mathbb{R}$ et $\omega \in \mathbb{R}$. Alors :

- H est un hyperplan **Naïf** si $\omega = \max_{1 \leq i \leq n} |a_i|$;
- H est un hyperplan **Standard** si $\omega = \sum_{i=1}^n |a_i|$;
- H est dit **mince** si $\omega < \max_{1 \leq i \leq n} |a_i|$;
- H est dit **épais** si $\omega > \sum_{i=1}^n |a_i|$.

La figure 1.5 montre quatre exemples en dimension 2 de droites discrètes mince, naïve, standard et épaisse. Dans ce document, il est question de la détection des droites discrètes standard.

1.4 Droite analytique passant par des hexagones ou des octogones

Les pixels sont généralement représentés sous forme carré. Séré et al [3] ont proposé de nouvelles définitions de droites analytiques à travers des hexagones ou des octogones virtuels. La figure 1.6 présent deux pixels contenant un hexagone et un octogone respectivement.

Définition 7 (Droite analytique hexagonale croissante [3]) Soit $A(x_a, y_a)$ et $B(x_b, y_b)$ le centre de deux hexagones respectivement. nous fixons $U = (y_b - y_a)$, $V = (x_a - x_b)$ et $T = -x_a(y_b - y_a) - y_a(x_a - x_b)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite hexagonale croissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

- si $0 \leq \alpha \leq \frac{\pi}{3}$ alors $-(\frac{d \cdot |U|}{4} + \frac{d \cdot |V| \sqrt{3}}{4}) \leq U \cdot x + V \cdot y + T \leq (\frac{d \cdot |U|}{4} + \frac{d \cdot |V| \sqrt{3}}{4})$;
- si $\frac{\pi}{3} \leq \alpha \leq \frac{\pi}{2}$ alors $-\frac{d \cdot |U|}{4} \leq U \cdot x + V \cdot y + T \leq \frac{d \cdot |U|}{4}$

Définition 8 (Droite analytique hexagonale décroissante [3]) Soit $A(x_a, y_a)$ et $B(x_b, y_b)$ le centre de deux hexagones respectivement. nous fixons $U = (y_b - y_a)$, $V = (x_a - x_b)$ et $T = -x_a(y_b - y_a) - y_a(x_a - x_b)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite hexagonale décroissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

- si $\frac{2\pi}{3} \leq \alpha \leq \pi$ alors $-(\frac{d \cdot |U|}{4} + \frac{d \cdot |V| \sqrt{3}}{4}) \leq U \cdot x + V \cdot y + T \leq (\frac{d \cdot |U|}{4} + \frac{d \cdot |V| \sqrt{3}}{4})$;
- si $\frac{\pi}{2} \leq \alpha \leq \frac{2\pi}{3}$ alors $-\frac{d \cdot |U|}{4} \leq U \cdot x + V \cdot y + T \leq \frac{d \cdot |U|}{4}$

Définition 9 (Droite analytique octogonale croissante [3]) Soit $A(x_a, y_a)$ et $B(x_b, y_b)$ le centre de deux octogones respectivement. nous fixons $U = (y_b - y_a)$, $V = (x_a - x_b)$ et $T = -x_a(y_b - y_a) - y_a(x_a - x_b)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite octogonale croissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

- si $\frac{\pi}{2} \geq \alpha \geq \frac{\pi}{4}$ alors $-(\frac{|U|}{2} + \frac{\sqrt{2}|V|}{2(2+\sqrt{2})}) \leq U \cdot x + V \cdot y + T \leq \frac{|U|}{2} + \frac{\sqrt{2}|V|}{2(2+\sqrt{2})}$;
- si $0 \leq \alpha \leq \frac{\pi}{4}$ alors $-(\frac{|V|}{2} + \frac{\sqrt{2}|U|}{2(2+\sqrt{2})}) \leq U \cdot x + V \cdot y + T \leq \frac{|V|}{2} + \frac{\sqrt{2}|U|}{2(2+\sqrt{2})}$

Définition 10 (Droite analytique octogonale décroissante [3]) Soit $A(x_a, y_a)$ et $B(x_b, y_b)$ le centre de deux octogones respectivement. nous fixons $U = (y_b - y_a)$, $V = (x_a - x_b)$ et $T = -x_a(y_b - y_a) - y_a(x_a - x_b)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite octogonale décroissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

- si $\frac{3\pi}{4} \geq \alpha \geq \frac{\pi}{2}$ alors $-(\frac{|U|}{2} + \frac{\sqrt{2}|V|}{2(2+\sqrt{2})}) \leq U \cdot x + V \cdot y + T \leq \frac{|U|}{2} + \frac{\sqrt{2}|V|}{2(2+\sqrt{2})}$;
- si $\frac{3\pi}{4} \leq \alpha \leq \pi$ alors $-(\frac{|V|}{2} + \frac{\sqrt{2}|U|}{2(2+\sqrt{2})}) \leq U \cdot x + V \cdot y + T \leq \frac{|V|}{2} + \frac{\sqrt{2}|U|}{2(2+\sqrt{2})}$

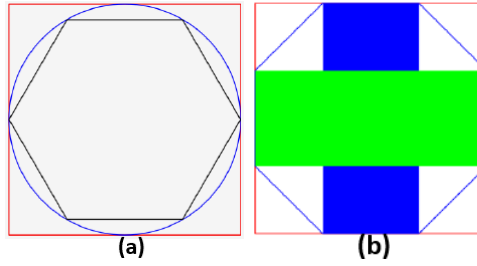


FIGURE 1.6 – (a) Un hexagone dans un pixel carré ; (b) Un octogone dans un pixel carré [3]

2 Notions des grilles virtuelles

Une grille virtuelle est une structure imaginaire composée de lignes horizontales, verticales et/ou oblique qui divisent une image en cellules.

Il existe deux types de grille virtuelle : les grilles régulières et les grilles irrégulières. La grille régulière est la plus simple, avec des cellules de même type et de taille égale réparties de manière régulière sur toute l'image. Par exemple, pour une grille dont les cellules sont des rectangles, on parle de grille rectangulaire, pour une grille dont les cellules sont des triangles, on parle de grille triangulaire, pour une grille dont les cellules sont des hexagones, on parle de grille hexagonale et pour une grille dont les cellules sont des octogones, on parle de grille octogonale. Nous pouvons voir une illustration de ces grilles virtuelles sur les figures 2.2, 3.2, 4.3 et 5.2 respectivement. Quant à la grille irrégulière, elle est composée de formes différentes de cellules réparties sur l'image.

Antoine Vacavant [70], dans ses travaux de thèse en 2008, a présenté les grilles irrégulières isothétiques. Une grille irrégulière isothétique est une grille composée de cellules de formes et de tailles variées, mais dont les côtés restent parallèles aux axes horizontalement et verticalement comme illustré sur la figure 1.7. Il a fait mention également des grilles triangulaires, hexagonales abordées par Herbert Freeman [71] en 1979 et E. Luczak [72] en 1976 respectivement.

D'après B. Kumar [73], Les structures de pixels hexagonaux offrent plusieurs avantages par rapport aux pixels carrés, comme une symétrie circulaire plus élevée, un ajustement uniforme des pixels, une complexité de traitement réduite, une meilleure qualité d'image, une connectivité cohérente et une isotropie plus élevée.

En 2019, Traoré et al [4] ont proposé une nouvelle approche de la transformée de Hough standard qui consiste à appliquer la TH sur une grille rectangulaire virtuelle superposée sur l'image dans le but de réduire son temps de traitement.

L'application de la TH, implique généralement des calculs intensifs. En considérant l'ensemble des pixels de l'image, ces calculs peuvent devenir coûteux en termes de temps de traitement. La grille virtuelle permet de filtrer les cellules de la grille qui ne contiennent pas un nombre suffisant de pixels allumés pour contribuer de manière significative à la détection de motifs. Cette approche permet de réduire le nombre total de calculs. Au lieu de traiter l'ensemble des pixels de l'image,

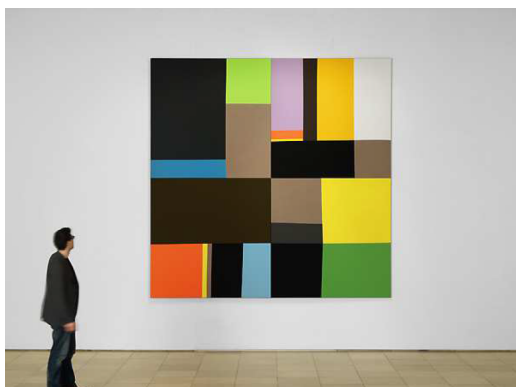


FIGURE 1.7 – Une grille virtuelle isothétique

nous ne considérons que les cellules dont le taux de pixel allumé dépasse un pourcentage α donné, économisant ainsi des ressources computationnelles. Comment reconnaître des objets discrets, notamment des droites discrètes ?

3 Origine et historique de la transformation de Hough

La vision par ordinateur a connu des avancées spectaculaires au cours des dernières décennies, permettant aux machines de comprendre et d'analyser des images numériques de manière semblable à la perception humaine. Au cœur de cette discipline, la détection d'objets discrets occupe une place prépondérante, permettant l'identification et la localisation précise d'entités spécifiques dans une image. Pour réaliser cette tâche complexe, la transformation de Hough émerge comme une méthode classique et puissante qui s'appuie sur une approche paramétrique, offrant une grande robustesse face aux variations géométriques des objets dans l'image.

La Transformation de Hough est une méthode de traitement d'image novatrice qui a été proposée pour la première fois par Paul Hough dans un article [24] publié en 1962 intitulé "Méthode et moyens pour reconnaître des motifs complexes". Cependant, la méthode n'a pas été largement reconnue à l'époque et est restée relativement méconnue pendant plusieurs années.

C'est en 1972 que la Transformation de Hough a été réintroduite et popularisée par Richard O. Duda et Peter E. Hart dans leur article intitulé "Use of the Hough Transformation to Detect Lines and Curves in Pictures" (Utilisation de la Transformation de Hough pour détecter des lignes et des courbes dans des images) [32]. Dans cet article, Duda et Hart ont démontré comment la méthode de Hough pouvait être utilisée pour détecter des lignes droites dans des images en représentant ces lignes dans un espace de paramètres appelé espace de Hough.

L'idée centrale de la Transformation de Hough réside dans sa capacité à représenter des formes géométriques, telles que des lignes droites, des cercles, des ellipses, et d'autres objets, sous forme de points dans un espace de paramètres. Cette représentation paramétrique permet de détecter ces formes indépendamment de leur position, leur orientation et leur échelle dans l'image, ce qui la rend

très robuste aux variations géométriques.

Initialement utilisée pour détecter des lignes dans des images, la Transformation de Hough a été étendue pour détecter d'autres formes géométriques en adaptant la représentation paramétrique appropriée pour chaque forme. Ainsi, des variantes de la méthode de Hough ont été développées pour détecter des cercles, des ellipses et d'autres objets discrets, élargissant considérablement son champ d'application.

Depuis sa réintroduction par Duda et Hart, la Transformation de Hough est devenue une méthode fondamentale de traitement d'images et un outil puissant pour la détection de formes géométriques et d'objets discrets dans de nombreux domaines de la vision par ordinateur. Elle a été largement utilisée dans des applications pratiques telles que la conduite autonome, la reconnaissance de formes, la robotique, la vision industrielle, la détection d'objets médicaux, et bien d'autres.

Au fil des années, de nombreuses améliorations et optimisations, notamment la transformée de Hough combinée à l'intelligence artificielle [44, 74–81], ont été apportées à la méthode de Hough, rendant son utilisation plus efficace et plus précise. Aujourd'hui, la Transformation de Hough continue d'être un sujet de recherche actif, avec de nouvelles avancées et applications émergentes dans le domaine de la vision par ordinateur.

4 Définition et étapes de la transformation de Hough Standard

Définition 11 (Transformée de Hough standard) Soient $I_2 \subset \mathbb{R}^2$ l'espace image c'est-à-dire une image de largeur l , de hauteur h et (x, y) un point de I_2 de paramètres (θ, ρ) dans l'espace des paramètres P_2 . La transformée de Hough standard de (x, y) est la courbe sinusoïdale $S(x, y)$ dans P_2 définie par $S(x, y) = \{(\theta, \rho) \in [0, \pi] \times [-\sqrt{l^2 + h^2}, \sqrt{l^2 + h^2}] / \rho = x \cos \theta + y \sin \theta\}$.

Par exemple pour $A(1; 1) \in I_2$, la transformée de Hough standard du point A est $S(1, 1) = \{(\theta, \rho) \in [0, \pi] \times [-\sqrt{l^2 + h^2}, \sqrt{l^2 + h^2}] / \rho = \cos \theta + \sin \theta\}$. Ce resultat est illustré dans la figure 1.8

Propriete 1 (Transformée de Hough standard [82])

- Un point dans l'espace d'image I_2 , correspond à une sinusoïde dans l'espace des paramètres P_2 . Cela est illustré par la Figure 1.8 en (a) et (b).
- Des points appartenant à la même sinusoïde dans l'espace des paramètres P_2 correspondent à des droites qui se coupent en un même point dans l'espace image I_2
- Un point de l'espace des paramètres P_2 correspond à une droite dans l'espace image I_2 . La figure 1.9 présente un exemple.
- Des points appartenant à une même droite (D) dans l'espace d'observation I_2 correspondent à des sinusoïdes qui se coupent en un même point dans l'espace des paramètres P_2 . La figure 1.9 présente également cette relation.

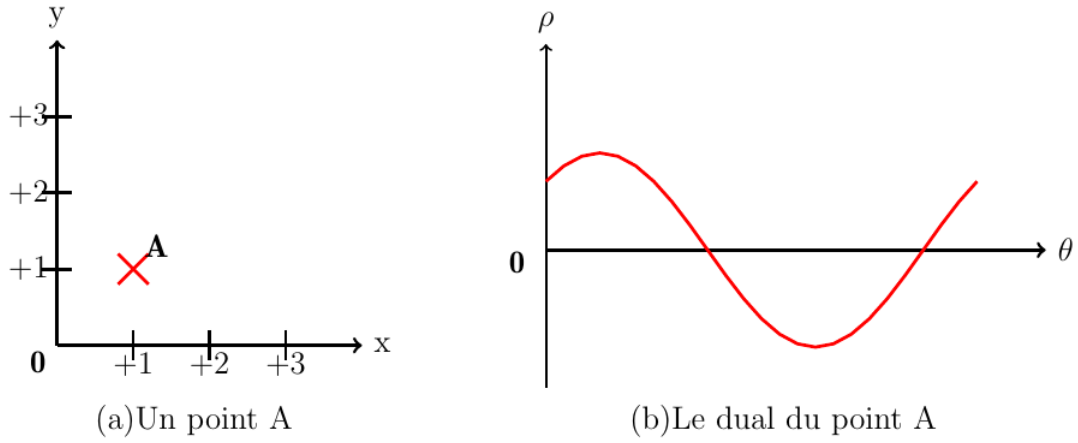


FIGURE 1.8 – Un point A et son dual

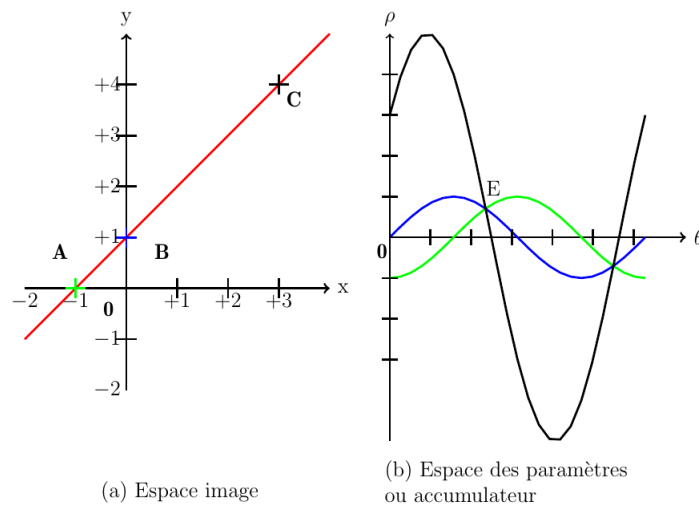


FIGURE 1.9 – Une droite et son dual

Les étapes de l'application de la transformée de Hough standard sur une image sont :

- pré-traitement de l'image : Il s'agit d'appliquer un filtre sur l'image pour mettre en évidence les contours des objets discrets. La figure 1.11 illustre respectivement une image d'une route, un pentagone et une image d'un bâtiment. La figure 1.12 est le résultat de l'application du filtre de Canny sur les images de la figure 1.11.

- mise à jours de l'accumulateur : la mise à jours de l'accumulateur consiste à calculer le dual des points de l'image et à mettre à jours l'espace des paramètres par incrémentations des valeurs des cellules de l'accumulateur comme illustre l'algorithme 1 et 2. Le dual de chaque pixel représente un vote dans l'accumulateur.

- détection des maxima dans l'accumulateur : une fois que tous les votes ont été accumulés, on recherche les maxima dans l'accumulateur en utilisant un seuil de vote. les maxima sont les points $(\theta_{max}, \rho_{max})$ de l'accumulateur ayant un nombre de votes supérieur au seuil de vote donné.

- reconstruction : Les paramètres $(\theta_{max}, \rho_{max})$ de l'espace des paramètres sont associés à la droite $\Delta : y = -\frac{\cos\theta}{\sin\theta}x + \frac{\rho}{\sin\theta}$ dans l'espace image.

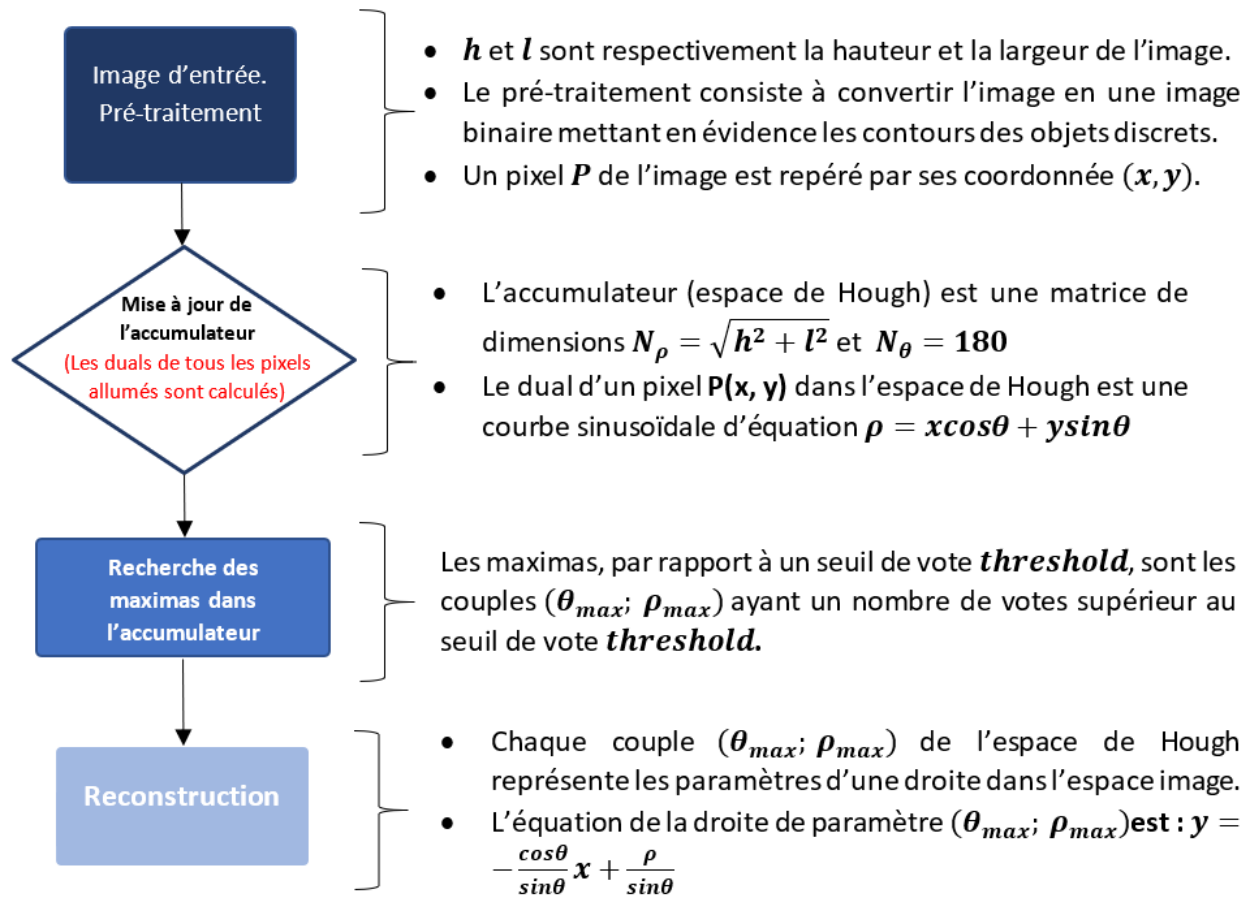


FIGURE 1.10 – Les étapes de la technique de la transformée de Hough standard

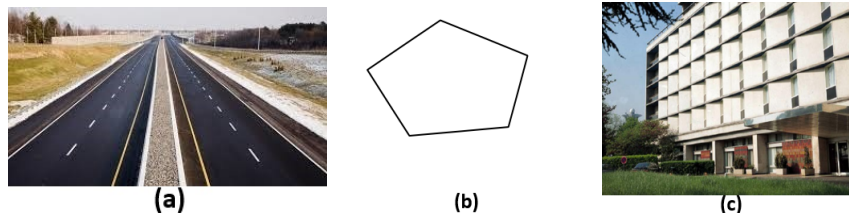


FIGURE 1.11 – Images pour les expérimentations

5 Variantes de la Transformée de Hough

Les variantes de la Transformée de Hough sont des extensions ou des adaptations de l'algorithme de base pour répondre à des problèmes spécifiques de détection d'objets géométriques dans les images numériques. Voici quelques-unes des principales variantes de la Transformée de Hough :

5.1 Transformée de Hough probabiliste

La Transformée de Hough probabiliste (PHT) est une variante de la Transformée de Hough classique utilisée pour détecter des formes géométriques, (telles que des lignes, des cercles, des ellipses) dans des images. Elle a été développée par Kiryati et al [54] pour améliorer la robustesse et l'efficacité de la Transformée de Hough classique, en particulier dans des conditions où l'exactitude des détections

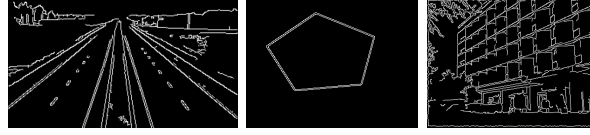


FIGURE 1.12 – Images binaires pour les expérimentations

Algorithme 1 : Reconnaissance de droites discrètes par la Transformée de Hough Standard

```

Fonction Standard(image, threshold) :tableau
  Pre-condition :  $0 < \text{threshold}$ 
  Variables : accum_ligne, accum_colon, irho, Nl, Nc : entier ; accum : matrice
               accumulateur ;
  dtheta, drho, rho, theta : réels;
  Debut
    % dimension de l'image
     $Nl \leftarrow$  la hauteur de l'image ;  $Nc \leftarrow$  la largeur de l'image;
    % dimensions de la matrice 'accum'
     $\text{accum\_ligne} \leftarrow 180$  ;  $\text{accum\_column} \leftarrow E[\sqrt{(Nc^2 + Nl^2)}]$ ;
     $dtheta = \pi/180$ ;
    pour  $0 \leq y \leq Nc$  faire
      pour  $0 \leq x \leq Nl$  faire
        | Acc_update();
      fin
    fin
  fin
  Rechercher les maxima dans l'accumulateur et placer les couples  $(\rho, \theta)$  dans une liste
  'lignes';
  Retourner la liste 'lignes';
fin

```

est compromise par des données bruitées ou des objets partiels. Contrairement à la Transformée de Hough classique, qui accumule des votes pour les points d'intersection dans un espace discret, la Transformée de Hough probabiliste attribue des probabilités aux paramètres des formes candidates, permettant ainsi de gérer de manière plus flexible les incertitudes liées à la détection.

5.2 Transformée de Hough probabiliste progressive

La transformée de Hough probabiliste progressive (PPHT) présentée par Matas et al [83], offre des avantages significatifs par rapport à la transformée de Hough probabiliste [54]. Contrairement à PHT, qui effectue la transformation de Hough standard sur une fraction pré-sélectionnée de points d'entrée, PPHT réalise la détection de ligne avec un calcul minimal en exploitant les différentes fractions de votes nécessaires pour une détection fiable des lignes avec un nombre variable de points de support. Cela élimine la nécessité de spécifier la fraction de points utilisée pour voter ad hoc ou sur la base de connaissances préalables. Au lieu de cela, l'algorithme s'adapte à la complexité inhérente des données. PPHT est particulièrement bien adapté aux applications en temps réel avec un temps de traitement fixe, car le vote et la détection de ligne se produisent simultanément, les fonctionnalités les plus

Algorithme 2 : Mise à jour de l'accumulateur

```
Procédure Acc_update() : vide
    % Mise à jour de l'accumulateur
    if img[z, t]  $\neq$  0 then
        for itheta dans range(accum_row) do
            theta  $\leftarrow$  itheta  $\times$  dtheta;
            rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
            irho  $\leftarrow$  int(rho);
            if irho > 0 et irho < accum_column then
                accum[itheta][irho]  $\leftarrow$  accum[itheta][irho] + 1
            end
        end
    end
fin
```

importantes étant identifiées en premier. Les résultats expérimentaux démontrent que PPHT surpasse la transformée de Hough standard dans de nombreux scénarios.

5.3 Transformée de Hough pour la détection de courbes

Dans les années 1972, soit 10 ans après l'introduction de la transformée de Hough classique [24], Duda et Hart [32] ont introduit une nouvelle approche pour détecter des courbes plus généraux. Cette approche est basée sur la transformée de Hough classique et met en avant le fait que l'utilisation de paramètres angle-rayon, par opposition aux paramètres pente-interception, simplifie davantage les calculs nécessaires. En outre, des interprétations alternatives sont proposées, offrant une perspective supplémentaire sur les raisons sous-jacentes à l'efficacité de cette méthode.

5.4 Transformée de Hough pour la détection de cercles, d'arcs, et d'ellipses

Cette variante est spécifiquement conçue pour la détection de cercles dans les images. En 1975 Kimme et al dans leur article [84] décrivent une procédure qui se concentre sur l'identification efficace de cercles approximatifs et d'arcs circulaires dans des images numérisées avec des bords améliorés. La procédure est une extension et une amélioration d'un concept de détection de courbes (arcs, cercles et ellipses en particulier) introduit par Duda et Hart [32]. Cela implique que la nouvelle procédure s'inspire de leur travail et développe leurs idées pour améliorer la précision, l'efficacité ou l'applicabilité de la détection de cercles. Princen et al dans leur article [85] ont étudié un certain nombre de méthodes de détection de cercles basées sur des variations de la transformée de Hough. Coelho et al [86] quant à eux utilisent une approche basée sur MapReduce pour la détection des cercles.

5.5 Transformée de Hough pour la détection d'ellipses

La Transformée de Hough est une technique précieuse pour détecter des lignes droites dans des images, en particulier lorsque les contours sont accentués. Cependant, lorsqu'il s'agit d'étendre cette

méthode pour détecter des ellipses, un problème se pose : le temps de calcul requis peut devenir excessivement long. Dans ce contexte, Tsuji et Matsumoto dans leurs travaux [87] en 1978 ont proposés une approche modifiée visant à surmonter cette limitation. L'approche fonctionne de manière itérative. Elle recherche des groupes d'ellipses presque complètes en explorant deux espaces de paramètres différents. Cette approche séparée dans les espaces de paramètres permet de réduire la complexité du calcul et d'accélérer le processus de détection. Suite à ces travaux, Davies [88] a montré par une autre approche qu'un seul plan dans l'espace des paramètres peut être utilisé, avec un gain d'efficacité conséquent. La méthode est particulièrement adaptée aux ellipses de faible excentricité.

5.6 Transformée de Hough généralisée

Ballard et al [7] ont proposé une généralisation de la Transformée de Hough classique, permettant de détecter des formes géométriques arbitraires, telles que des polygones, des courbes complexes ou d'autres formes définies par un ensemble de points. Elle utilise un modèle de référence de l'objet à détecter et recherche les occurrences de ce modèle dans l'image en utilisant la Transformée de Hough. La Transformée de Hough classique est limitée à la détection de lignes droites et de cercles. En généralisant la Transformée de Hough pour détecter des formes arbitraires, cette approche constitue une avancée significative dans le domaine de la détection d'objets et de formes dans des images. La méthode proposée a ouvert de nouvelles possibilités pour la détection de formes complexes, élargissant ainsi l'applicabilité de la Transformée de Hough dans diverses applications dans le domaine du traitement d'images.

5.7 Transformée de Hough adaptative

Dans les années 1987 les chercheurs, Illingworth et Kittler [89] ont présenté une approche novatrice appelée "Transformée de Hough Adaptative (AHT)" pour la détection efficace de formes bidimensionnelles. Cette méthode étend la Transformée de Hough en utilisant un réseau d'accumulateurs plus restreint tout en intégrant une stratégie d'accumulation et de recherche itérative. Cette approche permet d'identifier de manière précise les pics significatifs dans les espaces de paramètres de la Transformée de Hough. L'AHT se révèle nettement supérieure à l'implémentation standard de la TH en termes de stockage et de calculs requis.

5.8 Transformée de Hough basée sur l'algorithme Map-Reduce

La Transformée de Hough basée sur l'algorithme Map-Reduce est une version parallélisée de la transformée de Hough traditionnelle, conçue pour tirer parti des capacités de calcul distribué offertes par le modèle de programmation Map-Reduce. Abdoulaye SERE et al [90] ont proposés une composition de la transformée de Hough et de Map-Reduce dans l'environnements informatiques distribués

Hadoop. Ces travaux se sont focalisés sur la détection des droites, mais ils ont été étendus par Mateus Coelho et al [86] pour proposer la détection de cercle avec Map-Reduce. Ces variantes de la Transformée de Hough offrent des solutions adaptées à différents types de formes géométriques et aux exigences spécifiques des applications.

5.9 Transformée de Hough rapide

La transformée de Hough rapide est une approche, intitulée "Fast Hough Transform" (FHT), introduite en 1986 par Li et al [91] qui vise à améliorer l'efficacité du processus de détection en utilisant une méthode hiérarchique pour réduire les calculs nécessaires.

Les auteurs considèrent que les caractéristiques de l'image "votent" pour des ensembles de points qui sont associés à des hyperplans dans un espace de paramètres. Plutôt que de traiter l'ensemble de l'espace des paramètres, l'algorithme divise cet espace en hypercubes de résolutions variables. Ils effectuent ensuite la transformée de Hough uniquement sur les hypercubes où les votes dépassent un certain seuil.

L'approche hiérarchique permet de réduire les coûts de calcul et de stockage, elle ne traite que les parties de l'espace des paramètres susceptibles de contenir des informations importantes.

5.10 Transformée de Hough rapide intégrée (Integrated Fast Hough Transform)

La détection de lignes, de plans et d'hyperplans au sein de données multidimensionnelles trouve de nombreuses applications dans les domaines de la vision artificielle et de l'intelligence artificielle. La contribution de Li et al [92] consiste en la proposition d'une approche novatrice nommée Transformée de Hough Rapide Intégrée ou "Integrated Fast Hough Transform" (IFHT).

L'espace des paramètres dans l'IFHT est représenté par un unique arbre de type k , ce qui permet d'adopter une stratégie de stockage hiérarchique. L'IFHT modifie fondamentalement la manière dont les données sont ajustées aux moindres carrés, par rapport à la méthode de Transformée de Hough Rapide (FHT) développée par Li et al. Dans l'IFHT, cet ajustement se réalise en prenant en compte les erreurs d'observation sur toutes les dimensions, ce qui le rend plus adapté et résistant au bruit des données.

Dans des tests effectués sur des données simulées avec divers niveaux de bruit et de paramètres, l'IFHT surpasse significativement la méthode de Transformée de Hough Rapide de Li [91] en termes de robustesse et de précision.

6 Avancées de la Transformée de Hough

Les avancées de la Transformée de Hough ont été nombreuses au fil des années, et elles ont permis d'améliorer son efficacité, sa précision et son adaptabilité à différents domaines d'application. Voici

quelques-unes des principales avancées de la Transformée de Hough

6.1 Optimisation de l'espace d'accumulation

L'optimisation de l'espace d'accumulation est une technique importante pour accélérer la Transformée de Hough et réduire la complexité computationnelle en réduisant la taille de l'espace d'accumulation tout en maintenant une précision raisonnable pour la détection d'objets discrets. L'espace d'accumulation est une matrice multidimensionnelle utilisée pour enregistrer les votes lors de la recherche de lignes ou de formes dans l'image. Cependant, cette matrice peut devenir très grande, ce qui rend le processus de vote coûteux en termes de mémoire et de temps de calcul.

La Transformée de Hough probabiliste est une variante de la Transformée de Hough classique introduite par Kiryati et al [54]. Les auteurs ont proposé une variation de la transformée de Hough classique qui vise à résoudre certains des problèmes inhérents à la détection de formes complexes dans des images bruitées. La transformée de Hough probabiliste opère en échantillonnant aléatoirement un sous-ensemble des pixels de l'image et en utilisant ces échantillons pour déterminer les paramètres de la forme recherchée. Cette approche probabiliste permet de réduire les temps de calcul et de mieux gérer les scénarios où les objets peuvent être partiellement cachés ou présenter des variations d'échelle.

L'optimisation de l'espace d'accumulation dans le contexte de la Transformée de Hough probabiliste repose sur l'idée que, plutôt que de stocker l'accumulation pour chaque point de l'image, on peut estimer les paramètres de transformation en utilisant uniquement un échantillon aléatoire de points. Ce processus probabiliste permet de maintenir une représentation compacte de l'espace d'accumulation tout en réduisant la complexité temporelle de l'algorithme.

6.2 Utilisation d'espaces d'accumulation pyramidaux

Pour des échelles différentes ou des niveaux de résolution d'image, on peut créer des espaces d'accumulation pyramidaux. Cela permet de détecter des formes à différentes échelles, tout en réduisant la complexité computationnelle. En 2004, Lorenz et Hardeberg dans [93] ont présentés une généralisation multiscale de la Transformée de Hough pour l'analyse d'images. La méthode proposée vise à améliorer la robustesse de la Transformée de Hough aux variations d'échelle dans les images, ce qui est une limitation courante de la version traditionnelle de la Transformée de Hough.

La généralisation multiscale repose sur la construction d'une pyramide d'échelles dans l'espace d'accumulation. Au lieu d'utiliser une échelle fixe pour détecter les objets discrets, les auteurs utilisent plusieurs échelles de manière pyramidale pour représenter les objets à différentes résolutions. Cela permet de détecter des objets de différentes tailles dans l'image.

Le processus de détection commence par la création de plusieurs niveaux de résolution dans l'espace d'accumulation. Ensuite, des seuils adaptatifs sont appliqués pour détecter les objets candidats à

chaque niveau. Les détections à différents niveaux sont ensuite combinées pour obtenir les détections finales.

Les résultats expérimentaux démontrent l'efficacité de la méthode proposée. La généralisation multiscale améliore la détection d'objets discrets dans des images contenant des variations d'échelle, et elle est robuste face au bruit et aux déformations.

6.3 Adaptation à des formes complexes

L'adaptation de la Transformée de Hough à des formes complexes est un domaine de recherche important en vision par ordinateur. La Transformée de Hough classique est conçue pour détecter des formes géométriques simples telles que des lignes droites, des cercles, et des ellipses. Cependant, de nombreuses applications réelles impliquent la détection de formes plus complexes et non linéaires, telles que des objets naturels, des structures anatomiques, ou des motifs abstraits. Pour adapter la Transformée de Hough à des formes complexes, une approche a été proposée. Il s'agit de l'extension de l'espace des paramètres abordée par Ballard et Brown dans [7], où ils proposent la THG, qui étend l'espace des paramètres pour inclure des descripteurs plus riches et contextuels. Au lieu de se limiter à des paramètres géométriques, l'espace des paramètres est étendu pour inclure des descripteurs locaux ou globaux qui décrivent des caractéristiques spécifiques des formes recherchées.

6.4 Détection multi-objet

Plutôt que de se limiter à détecter un seul objet à la fois, cette approche permet la détection simultanée de plusieurs occurrences d'objets dans une seule passe de la Transformée de Hough. Cette approche offre une manière efficace de détecter des objets multiples dans des images complexes, ce qui est crucial dans de nombreuses applications de vision par ordinateur telles que la robotique, la reconnaissance de formes, la surveillance et l'analyse d'images médicales. Eric GABRIEL et al [93] a proposé une approche innovante pour la détection automatique de piétons et de voitures dans des images numériques basée sur la transformée de Hough généralisée discriminante et les réseaux de neurones à convolution profonde.

6.5 Intégration avec l'apprentissage automatique

L'utilisation de techniques d'optimisation avancées et d'apprentissage automatique (comme les réseaux de neurones) permet d'améliorer la précision de la détection en optimisant les paramètres de l'algorithme en fonction des données spécifiques. Des chercheurs ont proposés des approches pour intégrer la transformée de Hough avec l'apprentissage automatique.

Gall et al [94] ont proposé une approche novatrice appelée "Hough Forest" pour la détection d'objets, le suivi et la reconnaissance d'actions dans des vidéos. L'Hough Forest est une méthode d'apprentissage automatique basée sur les arbres de décision, inspirée par la Transformée de Hough,

qui vise à améliorer la robustesse et la précision des tâches de détection et de suivi d'objets dans des scènes complexes.

Qi Han et al [44] ont présenté également une méthode qui intègre la Transformée de Hough classique avec des techniques d'apprentissage profond (deep learning) pour la détection sémantique des lignes dans les images. En effet, le problème de détection des lignes sémantiques dans le domaine spatial est transformé en repérage de points individuels dans le domaine paramétrique, rendant les étapes de post-traitement, c'est-à-dire la suppression non maximale, plus efficaces.

Divas Karimanzira et Helge Renkewitz [95] utilisent la transformée de Hough généralisée combiné avec l'apprentissage automatique (machine learning) pour la détection et la localisation de stations d'accueil sous-marines dans des images acoustiques.

6.6 Optimisation matérielle

L'optimisation matérielle dans le cadre de la Transformée de Hough vise à améliorer l'efficacité et la rapidité de cette transformation en utilisant des techniques spécifiques au matériel informatique sur lequel elle est implémentée. La Transformée de Hough est une opération coûteuse en termes de temps de calcul, surtout pour les images de grande taille, en raison de la complexité de son algorithme. L'optimisation matérielle cherche à exploiter les caractéristiques du matériel cible pour accélérer le processus de transformation de Hough et réduire la consommation de ressources. Les architectures de processeurs modernes offrent des possibilités de parallélisation, notamment en utilisant des unités de calcul vectorielles et des unités de calcul graphiques (GPU). Ce qui a favorisé Yam-Uicab et al [96] de présenter des stratégies d'optimisation pour accélérer la Transformée de Hough appliquée à la détection de caractéristiques linéaires en utilisant le GPU. Les auteurs exploitent les capacités de parallélisation offertes par les GPU pour accélérer le processus de transformation de Hough.

Ces avancées ont rendu la Transformée de Hough plus performante et polyvalente dans la détection d'objets discrets dans les images numériques. Elles ont ouvert de nouvelles perspectives pour l'utilisation de cette méthode dans divers domaines, allant de la robotique à la vision industrielle, en passant par la reconnaissance de formes et la détection d'objets médicaux. Les progrès continus dans ce domaine promettent de rendre la Transformée de Hough encore plus efficace et pertinente pour des applications futures en vision par ordinateur.

7 Applications de la transformée de Hough

Cette section explore les diverses applications de la transformée de Hough dans différents domaines, mettant en évidence son importance et son efficacité dans des contextes tels que le transport, la surveillance environnementale, l'industrie, la médecine, l'élevage et l'agriculture. Ces applications démontrent comment la transformée de Hough contribue de manière significative à la résolution de

problèmes pratiques.

7.1 Transport

Une méthode pour améliorer la précision de la détection des voies dans le contexte des véhicules autonomes a été présentée en 2023 par Javeed et al [55]. Au cours des dernières décennies, les véhicules autonomes ont gagné en popularité en raison de leur potentiel pour réduire les accidents de la route causés par des erreurs humaines. Dans ce contexte, il devient crucial d'intégrer des systèmes intelligents de détection des voies dans les véhicules autonomes. La détection précise des voies est une fonctionnalité essentielle pour les systèmes avancés de conduite. Cette étude propose une approche améliorée pour détecter les voies en utilisant une version étendue de l'algorithme de détection des bords de Canny, soutenue par la transformée de Hough rapide. Pour améliorer la qualité des données en entrée, une étape de lissage de l'image avec un filtre de flou gaussien est effectuée, réduisant ainsi le bruit indésirable. Cette méthodologie a été testée sur différentes séquences d'images. Ensuite, un ajustement des moindres carrés a été appliqué dans cette région pour suivre la trajectoire de la voie. Les résultats des expériences ont montré que le système proposé présente une détection des voies robuste et précise. L'approche s'est avérée efficace en termes de rapidité de traitement et de précision d'identification, répondant ainsi aux exigences de la détection des voies pour les systèmes légers de conduite automatique.

Somda et Sere [57] ont proposés une approche novatrice pour résoudre le problème de la congestion routière en exploitant la technologie du Système de Positionnement Global (GPS) intégrée aux véhicules, associée à des serveurs de station de contrôle. La solution proposée vise à atténuer la congestion routière sans nécessiter l'installation de nombreuses caméras ou d'infrastructures supplémentaires sur les routes. Le cœur de cette étude tourne autour de la mise en œuvre de la méthode de transformation de Hough au sein des Réseaux Véhiculaires Ad-Hoc (VANET) pour une détection et une surveillance efficaces de la congestion routière. En établissant un réseau VANET, l'échange de données liées à la sécurité entre les véhicules devient possible, améliorant ainsi la sécurité des usagers de la route. En réponse aux préoccupations croissantes concernant la congestion routière et l'augmentation des accidents de la route, divers scénarios de circulation ont été formulés, conçus et testés.

Katru Amandeep et Anil Kumar [8] ont proposés une technique utilisant la transformée de Hough pour améliorer la détection des voies dans les systèmes Ad-Hoc véhiculaires en temps réel. La recherche se concentre sur l'optimisation de la détection des voies en exploitant la transformée de Hough. Cette méthode est utilisée pour identifier les voies courbes, tandis que la conversion du parallélisme des données vise à augmenter la vitesse de la technique proposée..

Pour une comparaison exhaustive des performances, les résultats de l'algorithme proposé seront mis en parallèle avec ceux des algorithmes de détection de voies déjà existants. Dans le contexte des transports intelligents, qui sont en constante évolution pour accroître la sécurité routière et réduire les

incidents, cette nouvelle technique basée sur la transformée de Hough vise à surmonter les limites des approches existantes.

L'algorithme proposé a été soigneusement conçu et implémenté dans l'environnement MATLAB, fournissant ainsi une plate-forme de développement et d'expérimentation solide pour évaluer son efficacité et ses performances.

Le domaine de l'aviation présente d'importants défis environnementaux, notamment en ce qui concerne l'impact du transport aérien sur le changement climatique. Les traînées de condensation émises par les avions contribuent particulièrement à ce problème en raison de leur potentiel à aggraver le réchauffement planétaire. Cependant, la détection des traînées de condensation à partir d'images satellites s'avère être une tâche complexe et ancienne. Les méthodes classiques de traitement d'images informatique montrent leurs limites dans des conditions d'imagerie changeantes. De plus, les approches traditionnelles de l'apprentissage automatique, impliquant des réseaux de neurones convolutifs classiques, se heurtent à des obstacles tels que le manque de données annotées spécifiquement pour les traînées de condensation et des processus d'apprentissage inadaptés à ce domaine. Dans ce présent article de Sun et al [97], une approche novatrice est introduite, reposant sur l'utilisation de l'apprentissage par transfert amélioré. Cette approche permet d'identifier de manière précise les traînées de condensation en utilisant un minimum de données. En parallèle, une nouvelle fonction de perte appelée "SR Loss" est proposée, laquelle améliore la détection des traînées en transformant l'espace de l'image en espace de Hough. Cette recherche ouvre de nouvelles perspectives pour la détection basée sur l'apprentissage automatique dans le domaine de l'aéronautique, en offrant des solutions pour pallier le manque d'ensembles de données volumineux annotés manuellement. De plus, ces avancées contribuent significativement à l'amélioration des modèles de détection des traînées de condensation.

7.2 Images satellitaires

Les travaux de Maja Michałowska et al [58] décrivent l'utilisation de données de balayage laser terrestre dans la gestion et la planification forestières. Ces données permettent d'obtenir des informations géospatiales pour évaluer les caractéristiques des arbres et des peuplements forestiers. L'approche présentée dans l'article combine la transformée de Hough circulaire, le débruitage des données et une méthode robuste d'ajustement au cercle des moindres carrés pour extraire avec précision les positions des troncs d'arbres à partir des données de balayage laser. L'étude commence par la détection initiale des positions des tiges d'arbres grâce à la transformée de Hough circulaire. Ensuite, les résultats sont filtrés pour éliminer les points non pertinents, ce qui améliore la qualité de la détection. Les données tridimensionnelles du balayage laser sont ensuite analysées en utilisant une méthode d'ajustement au cercle des moindres carrés, permettant d'identifier précisément les positions des tiges d'arbres. Les résultats obtenus sont prometteurs. Environ 94,8 % des arbres ont été localisés avec une précision moyenne de 87,2 %. Les erreurs moyennes pour la position du tronc et le rayon de l'arbre mesuré au

sol sont respectivement de 1,64 cm et 1,15 cm. Cette méthodologie offre des applications potentielles étendues. Elle peut être utilisée comme base pour analyser automatiquement des caractéristiques telles que l'essence, la hauteur et l'étendue du houppier des arbres. De plus, cette approche peut être adaptée à la détection d'autres objets de forme circulaire, comme des lampes ou des pylônes électriques, lorsque des données de balayage laser sont disponibles. Les positions détectées peuvent être intégrées dans des systèmes d'information géographique ou des applications industrielles pour répondre à des exigences légales liées à la position géodésique des objets.

En 2010, Poncelet et al [22] ont développés une technique basée sur la méthode de la transformée de Hough pour identifier des linéaments dans des images et appliquée aux résultats obtenus par des détecteurs de contours. Après avoir testé ses performances sur une image bien-structurée, représentant un paysage régulier de la région de Hesbaye, cette méthode a été mise en œuvre sur une image Landsat TM5 et un modèle numérique de terrain couvrant les massifs montagneux des Cima d'Asta et des Lagorai dans les Dolomites. Les résultats issus de ces deux types de données spatiales sont comparés et mis en relation avec la structure géologique de la zone. Les avantages et inconvénients spécifiques de chaque approche sont examinés. Il est évoqué que la détection des formations naturelles présente des challenges et que la question de l'échelle spatiale de reconnaissance est importante. Par ailleurs, les limitations du programme implémenté sont étudiées. Bien que fonctionnel, l'efficacité du détecteur de contours joue un rôle crucial, et les effets de normalisation appliqués aux zones d'intérêt peuvent introduire des biais dans le calcul des fréquences, qui ne sont pas encore complètement maîtrisés.

7.3 Industrie

En raison des normes de qualité de plus en plus exigeantes dans l'industrie mécanique moderne concernant la création de trous, les méthodes conventionnelles de détection manuelle ne sont plus viables. Il est essentiel de mettre en place un système mécanisé pour assurer un contrôle de qualité en temps réel et une production automatisée de trous. C'est dans cette optique que Wenfeng Hou [59] combine la technique de détection de cercles de Hough avec un système automatisé de repérage de trous, opéré par un robot industriel. La démarche de recherche implique une analyse approfondie de la structure globale du système de localisation automatique des trous. Cela englobe la confirmation des emplacements critiques ainsi que des paramètres nécessaires pour la détection. Par la suite, la méthode de détection de cercles de Hough est mise en œuvre et son efficacité est évaluée au travers d'expérimentations. Les résultats montrent une précision moyenne de détection de 98,0 % pour quatre types de défauts. Cette réussite souligne l'efficacité et la faisabilité de l'intégration de la détection de cercles de Hough au système de perçage automatisé utilisant des robots industriels. De plus, cette approche propose des perspectives réalisables pour des applications connexes.

7.4 Domaine de l'intelligence artificielle

La correction du désalignement des documents pose un problème fondamental lors du traitement d'images de documents. Les approches existantes présentent des limitations. Par exemple, la méthode de transformation de ligne de Hough peut induire une inversion erronée des images, tandis que les modèles d'apprentissage profond exigent d'importantes ressources humaines et informatiques, sans garantir une correction complète incluant l'orientation. Les méthodes basées sur l'OCR éprouvent également des difficultés à interpréter le texte en cas d'inclinaison. Shafi et al [98] ont proposés, en 2023, une nouvelle méthode d'apprentissage profond, à la fois simple et économique, pour remédier à l'inclinaison et à l'orientation des documents. L'approche restreint l'espace de recherche du modèle d'apprentissage automatique en prédisant si une image est inversée, sans nécessiter la prédiction d'un angle entre 0 et 360 degrés. Ils ont raffiné un modèle MobileNetV2, pré-entraîné sur Imagenet, en utilisant seulement 200 images, obtenant ainsi des résultats prometteurs. Cette méthode se révèle utile pour des tâches automatisées telles que l'extraction de données via la technologie OCR, réduisant substantiellement la nécessité d'interventions manuelles.

La reconnaissance des mouvements de la main joue un rôle essentiel dans l'interaction entre l'humain et la machine. Avec l'avancement des caméras capables de capturer la profondeur, la fusion des images en couleur et en profondeur permet d'obtenir des informations plus riches pour la détection des gestes de la main. Dans ce document, nous présentons un système de détection de gestes de la main qui se base sur les données enregistrées par la caméra frontale Intel RealSense SR300. Étant donné que les pixels des images en profondeur provenant du dispositif RealSense diffèrent de ceux des images en couleur, le système de détection proposé par Liao et al [15] associe les images en profondeur aux images en couleur en utilisant une technique appelée transformée de Hough généralisée. Cette méthode permet de séparer la main du contexte complexe de l'image en couleur en se basant sur les informations de profondeur. Ensuite, le système identifie divers gestes de la main grâce à un réseau de neurones convolutif à double canal spécialement conçu, qui intègre deux flux d'entrée distincts : les images en couleur et les images en profondeur. De plus, les auteurs ont constitué une base de données comprenant 24 types de gestes de la main, représentant chacun une lettre de l'alphabet anglais. Cette base contient un total de 168 000 images, réparties équitablement entre les images RVB et les images de profondeur, soit 84 000 de chaque. Les résultats expérimentaux obtenus en utilisant cette nouvelle base de données de gestes de la main démontrent l'efficacité de l'approche, avec un taux de précision de reconnaissance atteignant 99,4 %.

7.5 Médecine

La méthode de la transformée de Hough est un algorithme traditionnel de détection de formes qui a été largement employé pour repérer des lignes droites, des cercles ainsi que des ellipses au sein

d’images. Récemment, des concepts issus de la géométrie algébrique ont été employés pour étendre cette approche à des catégories particulières de courbes. Campi Cristina et al exposent dans l’article [19] des résultats préliminaires qui illustrent la potentialité d’utiliser cette technique pour identifier des structures osseuses denses dans des images cliniques obtenues par tomodensitométrie (scanner) et qui pourrait ainsi servir d’outil informatique efficace pour des applications en hématologie et oncologie.

Le cancer du sein constitue l’un des problèmes majeurs en matière de santé à l’échelle mondiale. La détection précoce des anomalies liées au cancer du sein offre les meilleures perspectives de guérison. Parmi les techniques les plus performantes et largement adoptées pour repérer et diagnostiquer ce cancer figure la mammographie. L’article de Vijayarajeswari et al [20] se focalisent sur la catégorisation des mammographies en utilisant les propriétés extraites via la transformée de Hough. La transformée de Hough, une méthode bidimensionnelle, est employée pour isoler les caractéristiques spécifiques d’une forme dans une image. La détection des micro-calcifications et des masses représente deux indicateurs cruciaux du risque, et leur identification automatisée revêt une importance primordiale pour le dépistage précoce du cancer du sein. Étant donné que les masses sont souvent ambiguës par rapport au tissu environnant, la localisation et la caractérisation automatisées des masses s’avèrent également complexes. Cet article explore les approches de classification et d’extraction de caractéristiques. Ici, la transformée de Hough est mise en œuvre pour repérer les éléments distinctifs au sein des images mammographiques, lesquels sont ensuite classés à l’aide d’une machine à vecteurs de support (SVM). L’intégration du SVM se traduit par une précision de classification accrue. La méthodologie est testée sur un échantillon de 95 images mammographiques recueillies et catégorisées par le biais d’un SVM. Les résultats obtenus démontrent l’efficacité de l’approche proposée pour la classification précise des anomalies au sein des mammographies.

7.6 Élevage

Dans leur papier publié en 2018, C. Yang et al [13] ont présenté une nouvelle méthode pour améliorer le suivi des abeilles. Dans le cadre de cette étude, des vidéos en 2D ont été enregistrées à une fréquence de 50 Hz. La première étape du modèle consiste à suivre les abeilles en combinant le seuillage des couleurs et la détection du premier plan. Ensuite, les abeilles sont représentées sous forme de taches dans une image binaire, et ces taches sont analysées pour estimer les positions et les directions de déplacement des abeilles. Néanmoins, le suivi devient difficile lorsque les abeilles se croisent dans l’image. L’article résout ce problème en appliquant la transformée de Hough standard à la recherche des abeilles lorsqu’elles se croisent. Ensuite, un filtre de Kalman est utilisé pour suivre les abeilles en utilisant leurs positions estimées. Étant donné la fréquence élevée des images (50 Hz), les mouvements des abeilles présentent une grande variabilité, ce qui rend leur suivi fiable complexe. Pour pallier cela, des ajustements sont apportés au filtre de Kalman pour s’adapter à cette situation. Étant donné que plusieurs abeilles sont suivies en même temps, l’algorithme d’affectation hongrois

est mis en œuvre pour attribuer les prédictions et les mesures à chaque abeille individuellement. Les résultats de l'expérience démontrent que le suivi des abeilles dans les vidéos 2D à 50 Hz est réalisé de manière fiable, surtout en situation de vue rapprochée.

7.7 Réseaux de communication optique

Les réseaux optiques flexibles en pleine émergence nécessitent des méthodes de surveillance intelligentes pour garantir la qualité du signal et atteindre les niveaux de performance souhaités. Ainsi, Sindhumitha et al [99] ont introduit un réseau de neurones à convolution multitâche (MT-CNN) conçu pour une surveillance polyvalente à l'aide d'une technique novatrice de transformation combinée d'image de Hough (CIHT). Ce système innovant permet de surveiller efficacement les performances optiques à multiples paramètres en utilisant la transformation de Hough (HT) pour extraire les caractéristiques de manière efficiente. Ils ont réalisé des simulations approfondies afin de démontrer les avantages de l'utilisation de la transformation de Hough ainsi que de l'augmentation des données, confirmant ainsi les résultats obtenus.

7.8 Construction et bâtiment

Malgré l'utilisation croissante des drones pour inspecter les façades des bâtiments, leurs avantages restent sous-exploités en raison des difficultés à obtenir une vue d'ensemble rapide et complète de l'état actuel des bâtiments à partir d'images morcelées. Ainsi, Cheng Zhang et al [21] ont présenté une méthode visant à résoudre ce problème. Cette méthode se déroule en trois étapes principales : premièrement, les informations de position et les paramètres optiques des images capturées par les drones sont utilisés pour configurer des caméras virtuelles dans un modèle d'information du bâtiment (BIM), créant ainsi des images modélisées. Deuxièmement, une technique améliorée basée sur la transformée de Hough généralisée est employée pour extraire les éléments de la façade du bâtiment, quelles que soient leurs formes. Enfin, dans la troisième étape, les images des drones sont projetées sur une orthophoto et intégrées dans des vues en deux dimensions et en trois dimensions. Pour automatiser ce processus, un prototype appelé Dynamo a été développé. Les expérimentations informatiques et sur le terrain ont validé cette approche en montrant une marge d'erreur moyenne de 21 mm lors de l'enregistrement des images dans le modèle BIM (Building Information Modeling). Ces résultats illustrent le potentiel de l'approche proposée pour faciliter l'inspection des façades de bâtiments en utilisant des drones.

De multiples applications urbaines requièrent des polygones de bâtiments en tant que données d'entrée. Cependant, extraire manuellement ces polygones à partir de nuages de points implique un investissement considérable en termes de temps et d'efforts. La méthode de la transformée de Hough est reconnue pour extraire des caractéristiques linéaires, mais les approches actuelles basées sur cette méthode manquent de souplesse pour extraire de manière efficace les contours de bâtiments complexes

et variés. L'information concernant l'ordre des points est souvent négligée. En exploitant les points de bord de bâtiments ordonnés, Elyta et al [14] ont développé une nouvelle variante de la transformée de Hough appelée "transformée de Hough assistée par des points ordonnés" (OHT). Cette méthode permet d'extraire avec précision les contours de bâtiments à partir de nuages de points LiDAR aéroportés. Cette approche consiste tout d'abord à construire une matrice d'accumulation de Hough en utilisant un système de vote dans un espace paramétrique linéaire (θ, r) . La dispersion des angles dans chaque colonne est employée pour identifier les orientations prédominantes des bâtiments. Les auteurs ont mis au point une méthode de filtrage et de regroupement hiérarchique pour obtenir des lignes précises à partir des points saillants détectés et des points de bord de bâtiments ordonnés. L'utilisation d'une matrice de liste de points ordonnés, comprenant les points de bord de bâtiments disposés dans un ordre spécifique, permet de détecter des segments de ligne dans des directions variées, ce qui aboutit à l'obtention de polygones de toit de bâtiments de grande qualité. La méthode a été soumise à des tests sur trois ensembles de données distincts : un nouvel ensemble à Makassar, en Indonésie, ainsi que deux ensembles de référence à Vaihingen, en Allemagne, caractérisés par des particularités différentes. L'algorithme de la méthode est la première application de la transformée de Hough offrant une grande adaptabilité, étant en mesure de traiter des bâtiments aux contours de longueurs et d'orientations variées. Les résultats montrent des taux élevés d'exhaustivité (entre 90,1 % et 96,4 %) ainsi que de précision (tous supérieurs à 96 %). L'exactitude de l'alignement angulaire des bâtiments présente un écart type moyen entre 0,2 et 0,57 m. Par ailleurs, le score de qualité (89,6 %) pour le benchmark Vaihingen-B dépasse les performances des méthodes concurrentes de pointe. Bien que les solutions pour l'ensemble de données difficile Vaihingen-A ne soient pas encore disponibles, nous obtenons un score de qualité de 93,2 %. Enfin, ils ont démontré l'efficacité de la méthode sur des bâtiments complexes à orientations variées près du musée EYE à Amsterdam.

7.9 Agriculture

Ces dernières années, la recherche dans le domaine de l'automatisation agricole a particulièrement mis l'accent sur le développement des tracteurs autonomes. Une tâche cruciale pour la planification de la conduite autonome concerne la détection des frontières du sol labouré. Récemment, plusieurs études ont exploré des approches basées sur la vision artificielle pour cette détection, et des progrès significatifs sont en cours. Cependant, la plupart des méthodes existantes se concentrent principalement sur des caractéristiques locales, ce qui entraîne des performances en deçà du potentiel optimal. En outre, les méthodes précédentes souffrent généralement d'interférences locales, nécessitant ainsi une étape de post-traitement heuristique afin d'obtenir les contours définitifs des zones labourées. C'est dans ce contexte que Oh Knaghan et al [100] ont introduit une nouvelle approche de détection des frontières du sol labouré, basée sur l'apprentissage profond, capable d'appréhender les caractéristiques contextuelles des lignes. L'approche proposée repose sur une fusion entre le soft-argmax et la trans-

formation de Hough. Grâce à l'utilisation du soft-argmax de Hough, les auteurs transforment les caractéristiques profondes issues des réseaux de neurones convolutifs en caractéristiques contextuelles liées aux frontières du sol labouré. Ensuite, ils effectuent une régression directe des paramètres de la transformation de Hough en explorant l'espace vectoriel transformé. Étant donné que les lignes droites sont paramétrées par ces paramètres, la méthode que nous avançons propose un apprentissage intégral et ne nécessite aucune phase de traitement ultérieur. Pour évaluer notre proposition, les auteurs mènent une expérimentation en créant une importante base de données spécifique à la détection des frontières du sol labouré. Les résultats des expériences confirment l'efficacité de notre méthode par rapport aux approches existantes en matière de détection des frontières du sol labouré.

G. lin et al [16] ont présenté une nouvelle méthode pour repérer des fruits dans des milieux naturels, adaptée aux robots de récolte automatisée, aux systèmes d'évaluation des rendements et aux dispositifs de contrôle de qualité. Contrairement aux techniques couramment basées sur la couleur, qui souffrent de sensibilité aux variations de lumière et aux contrastes faibles entre les fruits et les feuilles, l'approche proposée se fonde sur les caractéristiques des contours. Tout d'abord, un indicateur de forme distinctif est dérivé pour représenter les propriétés géométriques d'une partie quelconque, et est utilisé dans une correspondance partielle à double sens des formes. Cela permet de repérer les parties d'intérêt correspondant à des parties d'un contour de référence. Ensuite, une nouvelle transformation de Hough probabiliste est élaborée pour combiner ces morceaux afin de détecter les candidats-fruits. Enfin, un classificateur à vecteur de support, formé sur des caractéristiques de couleur et de texture, vérifie tous les candidats-fruits. Des ensembles de données couvrant différents types de fruits tels que les agrumes, les tomates, les citrouilles, les courges amères, les serviettes et les mangues ont été mis à disposition. Les expériences menées sur ces ensembles de données ont démontré que cette approche est compétitive pour la détection d'une variété de fruits, qu'ils soient verts ou oranges, de forme circulaire ou non, dans des environnements naturels.

La détection des rangs de café sur des champs imaged par UAV reste un défi à relever. Bien que de nombreuses méthodes aient abordé cette problématique en utilisant la transformée de Hough, cette approche présuppose à tort que les rangées de culture sont linéaires, ce qui n'est pas le cas dans les images prises depuis les airs. La contribution de Soares et al [17] consiste en un système de tuiles qui parvient à approximer de manière satisfaisante les rangées au sein de chaque tuile sous forme de segments droits, rendant ainsi possible l'utilisation de la transformée de Hough. Les résultats expérimentaux, comparés à la réalité du terrain, suggèrent que notre méthode permet d'effectivement se rapprocher des rangées de culture réelles.

Dans le contexte de l'agriculture durable, il y a une demande croissante pour des solutions robotiques. En particulier, les robots conçus pour l'agriculture de précision ouvrent de nouvelles perspectives. Ces applications requièrent généralement une grande précision dans leur système de navigation. Un élément clé pour une navigation fiable est la capacité à détecter de manière robuste les rangées de

cultures. Cependant, repérer les cultures à partir de données visuelles ou laser devient particulièrement complexe lorsque les plantes sont soit très petites, soit si grandes qu'elles se confondent. Winterhalter et al [18] ont exposé une méthodologie pour segmenter de manière fiable les plantes à différents stades de croissance. De plus, ils introduisent un nouvel algorithme destiné à détecter avec robustesse les rangées de cultures. Cet algorithme adapte la transformée de Hough utilisée pour la détection de lignes, permettant ainsi de repérer un modèle de lignes parallèles équidistantes. L'algorithme est en mesure d'estimer simultanément l'angle, le décalage latéral et l'espacement entre les rangées de cultures. Il se révèle particulièrement adapté pour identifier les plantes très petites. Des expérimentations approfondies, menées sur divers jeux de données réelles provenant de cultures de types et de tailles différentes, démontrent que notre algorithme produit des résultats fiables et précis.

7.10 Sécurité

La transformée de Hough est une technique de traitement d'image qui permet de détecter des formes géométriques telles que des lignes, des cercles ou des ellipses. Cette technique est largement utilisée dans le domaine de la sécurité pour la détection d'objets ou de personnes dans des images de vidéo-surveillance. Elle peut également être utilisée pour la reconnaissance des empreintes digitales. Les empreintes digitales sont un moyen courant d'identification et de vérification de l'identité dans de nombreux domaines, tels que les services de police, les douanes, les contrôles d'accès. L'article [61] présente une étude sur les algorithmes d'alignement d'empreintes digitales basés sur la Transformée de Hough. Les auteurs classifient les différentes approches basées sur la Transformée de Hough en trois catégories : l'alignement basé sur la correspondance locale, l'alignement basé sur la rotation discrétisée et l'alignement basé sur les paires correspondantes. Les performances de ces approches sont comparées en termes de précision, de temps de calcul et d'utilisation de la mémoire. Les expériences ont été réalisées sur une base de données d'empreintes digitales capturées dans différentes orientations et positions. Les résultats montrent que chaque approche a ses avantages et ses inconvénients en fonction des conditions spécifiques des empreintes digitales. Cette approche offre une méthode robuste et fiable pour l'identification biométrique basée sur les empreintes digitales.

La Transformée de Hough joue un rôle crucial dans la méthode de correspondance d'empreintes digitales. Elle est utilisée par K. Anitha [62] pour aligner et faire correspondre les empreintes digitales en extrayant des lignes droites qui approximent chaque crête d'empreinte digitale individuellement. La Transformée de Hough est appliquée à chaque crête séparément pour détecter les lignes droites qui correspondent le mieux à la crête. Les pics de l'espace de Hough pour chaque crête sont ensuite utilisés pour estimer les paramètres de rotation et de translation nécessaires pour enregistrer les images d'empreintes digitales de requête et de modèle. Ce processus d'alignement simplifie la correspondance des empreintes digitales en calculant un score de correspondance basé sur l'alignement des crêtes. Dans l'ensemble, la Transformée de Hough est essentielle pour faciliter l'alignement et la comparaison des

empreintes digitales dans l'algorithme présenté dans le document.

La transforme de Hough linéaire a été utilisé par Nitika et al [63], en combinaison avec la correspondance des minuties pour la reconnaissance des empreintes digitales.

La technologie d'identification par empreintes digitales partielles, qui est principalement utilisée dans les appareils dotés d'une petite surface de détection tels que les téléphones portables, les disques durs et les ordinateurs, a fait l'objet d'une attention accrue ces dernières années en raison des avantages uniques qu'elle présente. Toutefois, en raison de l'absence de points caractéristiques suffisants, la méthode conventionnelle ne donne pas de bons résultats dans la situation décrite ci-dessus. Qin, Jin et al [64] ont proposé une nouvelle technique de comparaison d'empreintes digitales qui utilise les crêtes comme caractéristiques pour traiter les images d'empreintes digitales partielles et combine la transformée de Hough généralisée modifiée et la stratégie de notation basée sur l'apprentissage automatique. L'algorithme peut répondre efficacement aux exigences de temps réel et d'économie d'espace des appareils à ressources limitées. Les expériences menées sur une base de données interne indiquent que l'algorithme proposé présente d'excellentes performances.

8 Domaines émergents

Les avancées rapides en vision par ordinateur et le développement de nouvelles technologies ont ouvert la voie à de nombreux domaines émergents où la Transformée de Hough pourrait jouer un rôle important. Voici quelques-uns de ces domaines émergents :

—Véhicules autonomes : la détection d'objets et de lignes est essentielle pour la navigation des véhicules autonomes. La Transformée de Hough peut être utilisée pour détecter les lignes de la route, les panneaux de signalisation, les véhicules et les piétons, contribuant ainsi à la sécurité et à la prise de décision dans ces véhicules.

—Réalité augmentée et réalité virtuelle : la Transformée de Hough peut être utilisée pour détecter et suivre des objets en temps réel, ce qui est essentiel pour des applications de réalité augmentée et de réalité virtuelle. Elle peut aider à suivre les mouvements des utilisateurs et à intégrer des objets virtuels dans des environnements réels.

—Agriculture de précision : l'utilisation de drones et de robots dans l'agriculture de précision nécessite la détection et la reconnaissance d'objets tels que les cultures, les maladies des plantes et les débris dans les champs. La Transformée de Hough pourrait être utile pour détecter et suivre ces objets dans des images aériennes.

—Médecine et imagerie médicale : la détection d'objets anatomiques et de structures médicales est essentielle dans l'imagerie médicale. La Transformée de Hough pourrait être appliquée pour détecter des organes, des vaisseaux sanguins et des lésions dans des images médicales, contribuant ainsi au diagnostic et à la planification de traitements médicaux.

—Surveillance et sécurité : la Transformée de Hough peut être utilisée dans des systèmes de surveillance pour détecter des objets suspects ou des comportements anormaux. Elle peut être appliquée dans des systèmes de vidéo surveillance pour détecter des intrusions, des véhicules suspects ou des colis abandonnés.

—Fabrication et contrôle qualité : dans le domaine industriel, la Transformée de Hough peut être utilisée pour détecter des défauts dans des pièces fabriquées, vérifier la qualité des produits et aider à l’automatisation des processus de fabrication.

—Robotique collaborative : la détection et la localisation précises des objets sont essentielles dans la robotique collaborative où les robots doivent interagir de manière sûre et efficace avec des objets et des humains.

—Détection d’objets dans des vidéos en temps réel : avec la demande croissante de surveillance vidéo en temps réel, la Transformée de Hough peut jouer un rôle essentiel dans la détection rapide d’objets dans des vidéos en direct.

Ces domaines émergents représentent des opportunités prometteuses pour la Transformée de Hough et son intégration avec d’autres techniques avancées de vision par ordinateur et d’apprentissage automatique. En explorant ces domaines, la Transformée de Hough pourrait continuer à évoluer et à se développer pour répondre aux besoins en constante évolution de la société moderne.

Conclusion

La Transformée de Hough demeure une méthode importante dans le domaine du traitement d’images. Depuis son introduction initiale pour la détection de lignes droites dans les années 1960, elle a été étendue et adaptée pour détecter une gamme variée de formes géométriques, allant des cercles et des ellipses aux formes arbitraires définies par un ensemble de points.

Les nombreuses variantes de la Transformée de Hough offrent des solutions spécifiques aux défis rencontrés dans différents domaines d’application, que ce soit dans l’industrie, la médecine, l’agriculture, les véhicules autonomes ou la sécurité. Que ce soit pour la détection d’objets dans des images de surveillance, l’identification de structures anatomiques dans des images médicales, ou la navigation des véhicules autonomes, la Transformée de Hough continue d’être un outil indispensable.

Avec les avancées rapides en intelligence artificielle, ainsi que l’émergence de nouveaux domaines tels que la réalité augmentée et la médecine personnalisée, il est probable que la Transformée de Hough continuera à jouer un rôle important dans l’analyse et la compréhension des données visuelles.

Dans la suite, nous proposerons la reconnaissance des objets discrets par de nouvelles approches de la transformée de Hough introduite dans le cadre de ces travaux.

Transformée de Hough Rectangulaire

Introduction	35
1 Description de la méthode	36
2 Complexité de la Transformée de Hough Rectangulaire	40
3 Simulations et discussions	42
Conclusion	55

Introduction

La détection de lignes droites dans les images est un problème fondamental en vision par ordinateur, largement étudié pour ses applications dans des domaines tels que la robotique, la reconnaissance de formes, la cartographie. Parmi les nombreuses techniques développées pour résoudre ce problème, la Transformée de Hough est l'une des approches les plus connues et les plus utilisées.

Ce chapitre fournit une nouvelle approche de la transformée de Hough pour la détection des droites discrètes. Il s'agit de la transformée de Hough rectangulaire. Nous décrivons sa méthode de fonctionnement, ses étapes clés et ses paramètres. Nous menons également des simulations sur des images représentatives pour évaluer les performances de la transformée de Hough rectangulaire dans la détection de lignes droites.

De plus, nous comparons les performances de la transformée de Hough rectangulaire avec celles de la Transformée de Hough Standard, avec sa version parallélisée THRP, ainsi qu'avec une approche proposée par Traore et al [4]. Cette comparaison nous permet d'évaluer l'efficacité de la THR par rapport aux méthodes existantes en termes de précision de détection et de temps d'exécution.

1 Description de la méthode

Dans cette section, nous décrivons la technique de la transformée de Hough rectangulaire. Les différentes étapes sont illustrées dans la figure 2.3. En outre, nous détaillons l'algorithme 6 de calcul du taux de pixels allumés dans une région rectangulaire. Enfin, nous discutons également des considérations pratiques pour déterminer la taille optimale des cellules, afin d'assurer une bonne détection des droites tout en minimisant le temps de traitement.

1.1 Maillage de l'image

L'algorithme 3 est l'algorithme de maillage rectangulaire. Il prend en paramètre les dimensions de l'image (Nl : la hauteur de l'image, Nc : la largeur de l'image), les dimensions du rectangle (L : longueur, l : largeur) et retourne un tableau de 4 entiers dont les deux premières valeurs sont les dimensions de la cellule rectangulaire et les deux dernières valeurs sont les résidus respectivement de la hauteur et de la largeur de l'image. Les résidus de la hauteur sont les pixels restants après avoir subdivisé la hauteur de l'image en parties égales de longueur l . c'est le reste de la division euclidienne de Nl par l . De même, les résidus de la largeur sont les pixels restants après avoir subdivisé la largeur de l'image en parties égales de longueur L . c'est le reste de la division euclidienne de Nc par L .

Algorithme 3 : Construction de la grille rectangulaire

```
Fonction maillage_rect( $Nl, Nc, l, L$  : entiers) : tableau d'entiers
  Variables : tab : tableau de 6 entiers
  Output : tab;
  begin
    tab[0]  $\leftarrow L$ ; % Longueur du rectangle
    tab[1]  $\leftarrow l$ ; % Largeur du rectangle
    tab[2]  $\leftarrow Nl \bmod l$ ; % le reste de la division euclidienne de  $Nl$  par  $l$ 
    tab[3]  $\leftarrow Nc \bmod L$ ; % le reste de la division euclidienne de  $Nl$  par  $l$ 
    return tab;
  fin
```

La grille rectangulaire, représenté par la figure 2.2, issue du maillage de l'espace image est constitué de cellules rectangulaires de longueur AB et de largeur AD comme illustré dans la figure 2.1.

1.2 Sélection des cellules

Le critère de sélection d'un rectangle dans la grille repose sur le pourcentage de pixels allumés à l'intérieur de ce rectangle. Ce pourcentage est déterminé à l'aide de l'algorithme 6 décrit comme suit :

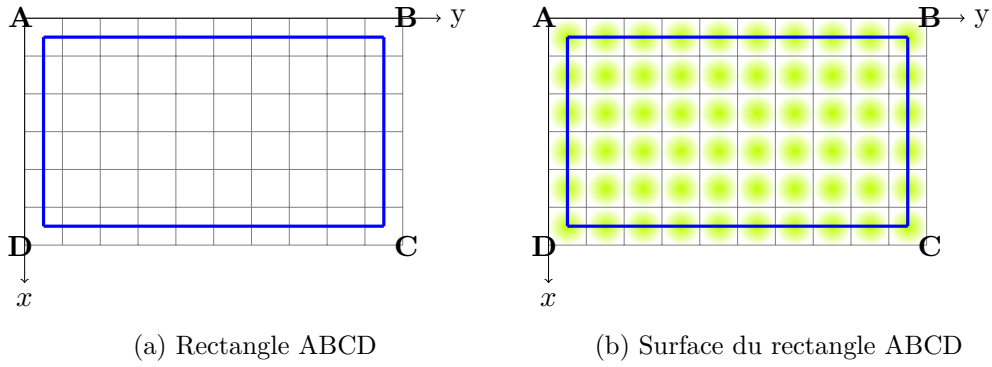


FIGURE 2.1 – Cellule rectangulaire

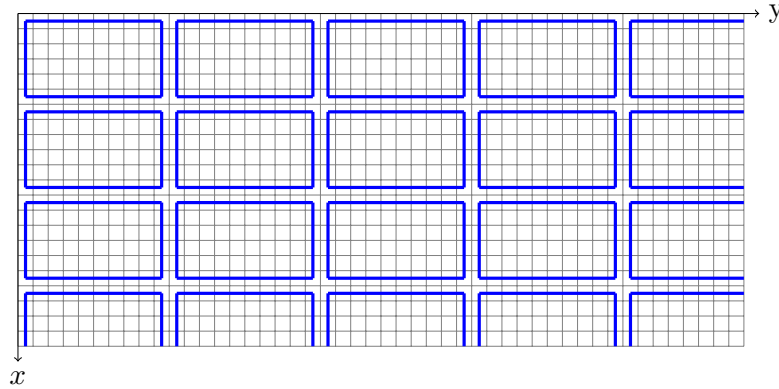


FIGURE 2.2 – Grille rectangulaire superposée à l'espace image

- Paramètres d'entrées : img , représentant l'image sur laquelle l'algorithme opère, et A, B, D trois tableaux de deux entiers chacun, représentant les coordonnées de trois sommet d'un rectangle.
- Paramètre de sorties : l'algorithme renvoie un nombre réel représentant le taux de pixels allumés dans la région rectangulaire spécifiée.
- Variables : k et l sont des variables entières initialisées à 0. Elles sont utilisées pour compter les pixels allumés et éteints, respectivement.
- Les boucles : deux boucles imbriquées parcourent les pixels dans la région rectangulaire spécifiée.
- L'analyse des pixels : pour chaque pixel aux coordonnées (x, y) , l'algorithme vérifie si la valeur du pixel dans l'image n'est pas égale à 0. Si c'est vrai, il incrémente k (nombre de pixels allumés) ; sinon, il incrémente l (nombre de pixels éteints).
- Enfin, l'algorithme renvoie le rapport de pixels allumés sur le nombre total de pixels dans la région, calculé comme $k/(k + l)$.

1.3 Parallélisation de la transformée de Hough rectangulaire

La parallélisation de la transformée de Hough rectangulaire nécessite une adaptation de l'algorithme pour tirer parti des capacités de traitement simultané offertes par le parallélisme. La parallélisation concerne l'accumulateur c'est-à-dire l'espace des paramètres encore appelé l'espace de

Algorithme 4 : Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire

```
Fonction Rectangular(imge, l, L,  $\alpha$ , seuil) : tableau
Pre-condition :  $1 \leq L \leq Nl$ ,  $1 \leq l \leq Nc$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
Variables : tab : tableau de 6 entiers ;
A, B, D : tableau de 2 entiers ; accum_row, accum_column, irho, Nl, Nc : entiers ;
accum : matrice de dimensions "accum_row  $\times$  accum_column";
dtheta, rho, theta : real;
Begin
    % les dimensions de l'image ;
    Nl  $\leftarrow$  nombre de ligne de l'image ; Nc  $\leftarrow$  nombre de colonne de l'image ;
    % the dimensions of "accum" ;
    accum_row  $\leftarrow$  180 ; accum_column  $\leftarrow E(\sqrt{(Nc^2 + Nl^2)})$  ;
    dtheta =  $\pi/180$  ; tab  $\leftarrow$  maillage_rect(Nl, Nc, l, L) ;
    H = Nl - tab[2] ; L = Nc - tab[3] ; H1 = H + tab[1] + 1 ; L1 = L + tab[0] + 1 ;
    for x allant de tab[1] à H1 par pas de tab[1] + 1 do
        for y allant de tab[0] à L1 par pas de tab[0] + 1 do
            A[0] = x - tab[1] ; A[1] = y - tab[0] ; B[0] = x - tab[1] ; B[1] = y ; D[0] = x ; D[1] = y - tab[0] ;
            if count(imge, A, B, D)  $\geq \alpha$  then
                for z allant de A[0] à D[0] do
                    for t allant de A[1] à B[1] do
                        Acc.update();
                    end
                end
            end
        end
    end
    Rechercher les maxima dans l'accumulateur et placer les couples ( $\rho$ ,  $\theta$ ) dans une liste 'lignes';
    Retourner la liste 'lignes';
fin
```

Algorithme 5 : Mise à jour de l'accumulateur

```
Procédure Acc_update() : vide
    % Mise à jour de l'accumulateur
    if img[z, t]  $\neq 0$  then
        for itheta dans range(accum_row) do
            theta  $\leftarrow$  itheta  $\times$  dtheta;
            rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
            irho  $\leftarrow$  int(rho);
            if irho > 0 et irho < accum_column then
                accum[itheta][irho]  $\leftarrow$  accum[itheta][irho] + 1
            end
        end
    end
fin
```

Hough. Les algorithmes 8 et 7 sont les versions parallélisées des algorithmes 4 et 5. Les étapes principales de la parallélisation sont :

- importation de la bibliothèque *pypm*² : importer une bibliothèque en Python est nécessaire pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction *import pypm* est utilisée pour charger la bibliothèque *pypm*.
- création de l'accumulateur de type *pypm* : dans l'algorithme séquentiel, l'espace des paramètres (l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la transformée de Hough, est transformé en une structure de données partagée grâce à *pypm*. En effet, *pypm* permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.

2. <https://github.com/classner/pypm/tree/main>

Algorithme 6 : Taux de pixels allumés

Fonction *count_rect*(*img* : *image*, *A*, *B*, *D* : *tableaux*) : *réel*
 Variables : *k*, *l* : entiers;
 begin
 $k \leftarrow 0$; % initialisation du compteur du nombre de pixels allumés
 $l \leftarrow 0$; % initialisation du compteur du nombre de pixels éteints
 for *x* allant de *A*[0] à *D*[0] **do**
 for *y* allant de *A*[1] à *B*[1] **do**
 if *img*[*x*, *y*] $\neq 0$ **then**
 $k \leftarrow k + 1$;
 else
 $l \leftarrow l + 1$;
 end
 end
 end
 return $\frac{k}{k+l}$; % le taux de pixels allumés
fin

L’instruction “*accum_pymp* $\leftarrow$ *pymp.shared.array([accum_row, accum_column])*” de l’algorithme 8 crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l’affecte à la variable “*accum_pymp*”.

- construction parallèle : l’instruction “*with pymp.Parallel(4) as p*” de l’algorithme 8 indique que nous souhaitons exécuter le bloc de code à l’intérieur de la déclaration “*with*” en parallèle avec 4 processus.
- parallélisation de l’algorithme 5 de mise à jours de l’accumulateur : la fonction “*p.range*” génère une séquence d’indices à partir de laquelle chaque cœur du CPU obtiendra une plage d’indices spécifique. Cela permet de diviser le travail entre les cœurs du CPU de manière équitable. Ainsi la boucle “*for itheta in p.range(accum_row)*” de l’algorithme 7 utilise les indices compris entre 0 et *accum_row* − 1 pour itérer à travers l’accumulateur par sections, où chaque section est traitée par un processus différent.

1.4 Reconnaissance des droites discrètes

L’algorithme 4 de reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire prend une image en entrée et utilise une grille générée avec des paramètres spécifiques. Il parcourt ensuite cette grille, définissant des régions rectangulaires à partir de points délimités par les points A, B, C, D. Si le taux de pixels allumés dans une région dépasse un seuil défini par α , la fonction *Acc_update* (l’algorithme 5) est appelée pour mettre à jour la matrice accumulateur. Après avoir exploré toutes les régions, l’algorithme recherche les maxima dans l’accumulateur, puis place les couples $(\theta; \rho)$ (les coordonnées des maxima dans l’accumulateur) dans une liste. Les paramètres des droites détectées correspondent aux coordonnées des maxima de l’accumulateur. Cette liste est ensuite ren-

Algorithme 7 : Mise à jour parallèle de l'accumulateur

```
Procedure Acc_updateP() : vide
  % Mise à jour de l'accumulateur
  if img[z,t]  $\neq$  0 then
    for itheta dans p.range(accum_row) do
      % Calcul des coordonnées polaire du pixel "img[z,t]".  $\theta \leftarrow \theta \times d\theta$ ;
       $\rho \leftarrow t \times \cos(\theta) + z \times \sin(\theta)$ ;
       $i\rho \leftarrow \text{int}(\rho)$ ;
      if  $i\rho > 0$  et  $i\rho < \text{accum\_column}$  then
        | accum_pymp[itheta][irho]  $\leftarrow$  accum_pymp[itheta][irho] + 1
      end
    end
  end
fin
```

voquée, fournissant les paramètres des droites discrètes détectées dans l'image.

L'algorithme 9 a pour objectif de visualiser les droites détectées par la méthode de la Transformée de Hough. Il prend en entrée une image (*image*), la liste de paramètres de droites détectées (*lignes*) renvoyée par l'algorithme 4, et une couleur (*colors*). L'algorithme parcourt la liste des paramètres de droites détectées et reconstruit chaque droite correspondante sur l'image. Les droites reconstruites sont de couleur *colors*. Cette reconstruction permet de mettre en évidence les droites détectées visuellement sur l'image d'origine, facilitant ainsi l'interprétation et l'analyse des résultats de la Transformée de Hough.

2 Complexité de la Transformée de Hough Rectangulaire

La fonction *maillage_rect* dans l'algorithme 3 récupère les dimensions d'une image et assigne des valeurs à un tableau, puis retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

La fonction *count_rect* dans l'algorithme 6 : initialise des variables k et l_1 , puis elle parcourt les pixels à l'intérieur du rectangle à l'aide de boucles dont le nombre d'itérations est $L \times l$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité du bloc formé par les boucles est $O(L \times l)$, où $L \times l$ est le nombre de pixels à l'intérieur du rectangle (l'aire du rectangle). Ainsi, la complexité totale de la fonction *count_rect* est $O(L \times l)$.

La fonction *Acc_update* dans l'algorithme 5 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de $\text{accum_row} = 180$, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction *Acc_update* est $O(1)$.

L'algorithme 4 de la Transformée de Hough rectangulaire (THR) commence par l'initialisation des

Algorithme 8 : Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire Parallélisée

```

Fonction RectangularP(image, l, L,  $\alpha$ , seuil, cpu_count) : tableau
  Pre-condition :  $1 \leq L \leq Nl$ ,  $1 \leq l \leq Nc$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
  Variables : tab : tableau de 6 entiers ;
  A, B, D : tableau de 2 entiers ; accum_row, accum_column, irho, Nl, Nc : entiers ;
  accum : matrice de dimensions "accum_row  $\times$  accum_column";
  dtheta, rho, theta : real;
  Begin
    import pypm; % importation de la bibliothèque pypm
    % les dimensions de l'image ;
    Nl  $\leftarrow$  nombre de ligne de l'image ; Nc  $\leftarrow$  nombre de colonne de l'image ;
    % the dimensions of "accum";
    accum_row  $\leftarrow$  180 ; accum_column  $\leftarrow$   $E(\sqrt{(Nc^2 + Nl^2)})$ ;
    accum_pypm  $\leftarrow$  pypm.shared.array([accum_row, accum_column]) % création de l'accumulateur de type pypm;
    dtheta =  $\pi/180$  ; tab  $\leftarrow$  maillage_rectP(Nl, Nc, l, L) ;
    H = Nl - tab[4] ; L = Nc - tab[5] ; H1 = H + tab[3] + 1 ; L1 = L + tab[1] + 1 ;
    % Démarrage du traitement parallèle
    With pypm.parallel(cpu_count) as p :
      for x allant de tab[1] à H1 par pas de tab[1]+1 do
        for y allant de tab[0] à L1 par pas de tab[0]+1 do
          A[0] = x - tab[1] ; A[1] = y - tab[0] ; B[0] = x - tab[1] ; B[1] = y ; C[0] = x ; C[1] = y - tab[0] ;
          if count(img, A, B, D)  $\geq \alpha$  then
            for z allant de A[0] à D[0] do
              for t allant de A[1] à B[1] do
                Acc.update();
              end
            end
          end
        end
      end
    fin
    Rechercher les maxima dans l'accumulateur et placer les couples ( $\rho$ ,  $\theta$ ) dans une liste 'lignes';
    Retourner la liste 'lignes';
  fin

```

Algorithme 9 : Reconstruction des droites détectées

```

Fonction PlotHoughLine(image, lignes, colors) : image
  Parcourir 'lignes' la liste des paramètres des droites détectées, puis construire chaque
  droite correspondant en couleur 'colors' sur l'image 'image'.
fin

```

variables et du tableau *tab*, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction *maillage_rect* dont la complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux boucles imbriquées parcourant les points à l'intérieur de chaque rectangle à l'intérieur du quelle se trouve des opérations de coût constant et la fonction *Acc.update* de complexité $O(1)$. La complexité de ces boucles est donc $O(L \times l)$.

Le coût des deux boucles principales imbriquées parcourant les rectangles de la grille, dont le nombre total d'itérations dépend de la taille de l'image et de la taille des cellules rectangulaires, contenant un bloc d'opérations d'affectation de coût constant, un test de condition sur la fonction *count_rect* de complexité $O(L \times l)$, les deux boucles imbriquées de coût $L \times l$ parcourant les pixels à l'intérieur du rectangle, est $(Nl/L \times Nc/l) \times (1 + (L \times l) + 1 + (L \times l))$, donc de complexité $O((Nl/L \times Nc/l) \times (1 + (L \times l) + 1 + (L \times l))) = O(Nl \times Nc)$.

En considérant ces points, la complexité totale de l'algorithme 4 de la transformée de Hough rectangulaire est égal à $O(Nl \times Nc)$, où *Nl* et *Nc* sont le nombre de lignes et de colonnes de l'image respectivement. Ce qui correspond à la complexité de la transformée de Hough standard.

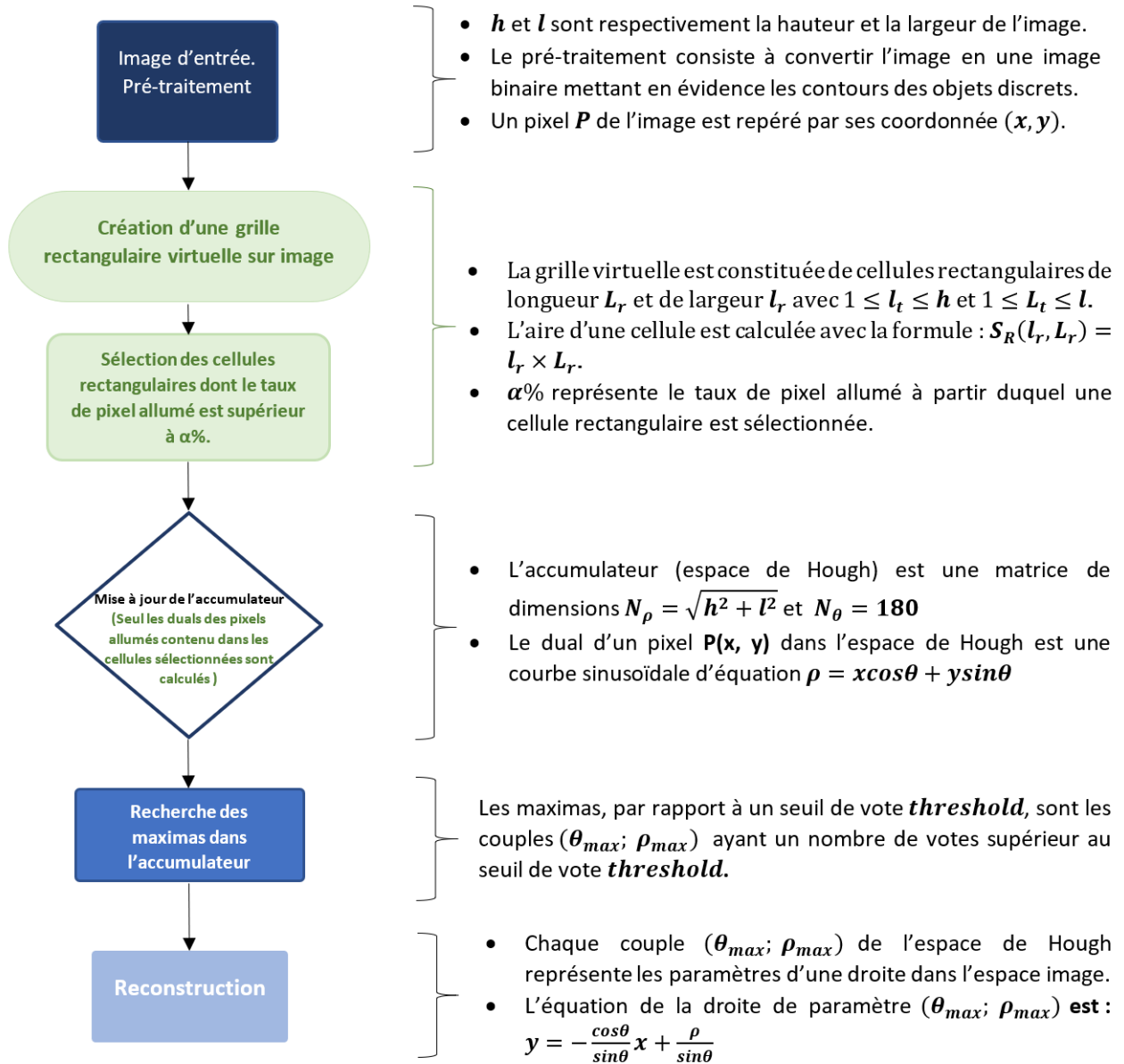


FIGURE 2.3 – Les étapes de la technique de la transformée de Hough rectangulaire

3 Simulations et discussions

Cette section se concentre sur les résultats de simulations obtenus en appliquant l'algorithme de la transformée de Hough rectangulaire à deux cas d'étude distincts : une image représentant un immeuble et une autre dépeignant une autoroute illustré dans la figure 1.11. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Les simulations ont été menées en variant différents paramètres de l'algorithme, tels que α , la taille des cellules rectangulaires S_R , et le seuil de vote. Les résultats sont analysés en termes du nombre de lignes

TABLEAU 2.1 – Résultats des simulations sur l’image 1.11(c) de l’immeuble avec l’algorithme 4 de la THR avec les paramètres fixes ($l = 3$, $L = 5$ and $Seuil = 70$) et α variant entre 10% et 45% où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

α	Droites détectées	temps(sec)
10	155	0,49
15	134	0,48
20	134	0,49
25	51	0,39
30	32	0,33
35	2	0,20
40	2	0,20
45	0	0,13

détectées, de la précision et du temps de calcul. Cette section présente également une comparaison entre la transformée de Hough rectangulaire et sa version parallélisée, ainsi qu’une confrontation avec l’approche de Traoré et al [4]. Les discussions qui suivent mettent en lumière les tendances observées dans les résultats des simulations, ainsi que les avantages et les limitations de chaque approche.

3.1 Cas de l’image de l’immeuble

Dans cette section, nous examinons les performances de l’algorithme de détection de lignes de la THR appliqué à une image représentant un immeuble. L’objectif est d’explorer l’impact des paramètres clés de l’algorithme, à savoir L , l et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l’algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l’algorithme de la transformée de Hough rectangulaire en fonction des caractéristiques de l’image de l’immeuble.

3.1.1 Variation de α

Le tableau 2.1 présente les résultats de simulations sur l’image 1.11(c) de l’immeuble en utilisant l’algorithme 4 de la THR. Les simulations ont été réalisées avec des paramètres fixes ($l = 3$, $L = 5$ et $Seuil = 70$), tandis que le paramètre α a été modifié entre 10% et 45%. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 45%. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L’analyse des résultats de la simulation sur l’image de l’immeuble avec l’algorithme de la THR et les paramètres fixes ($l = 3$, $L = 5$ et $Seuil = 70$) avec α variant entre 10% et 45% révèle deux tendances majeures. Premièrement, en ce qui concerne le nombre de lignes détectées, on observe une diminution

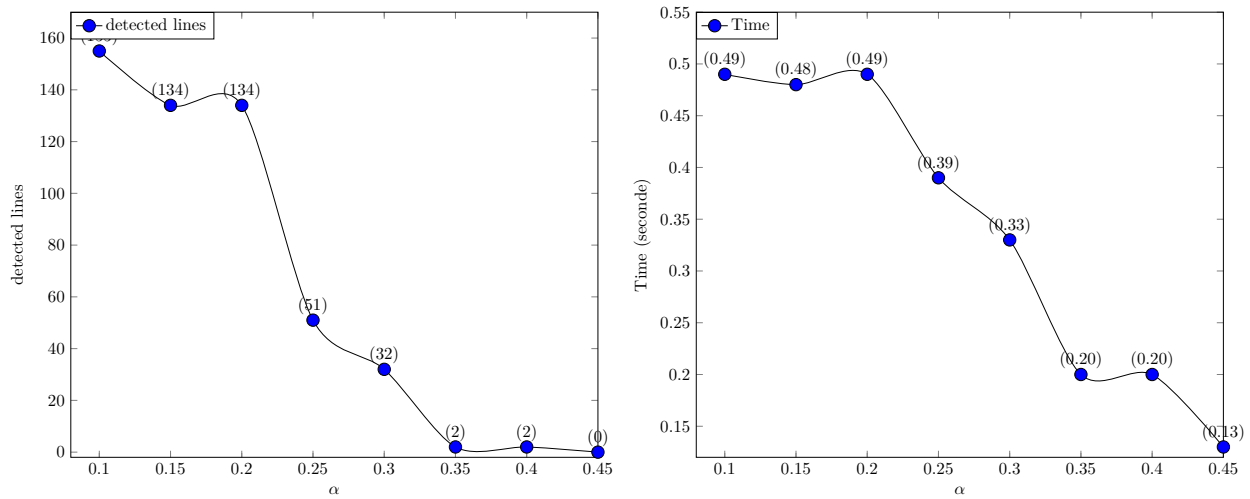


FIGURE 2.4 – Courbe représentative des données du tableau 2.1

TABLEAU 2.2 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres fixes ($\alpha = 30\%$ et $Seuil = 50$) et $1 \leq L \leq Nl$ et $1 \leq l \leq Nc$ où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

l	L	S_R	Nbre de droites détectées	temps(sec)
2	4	8	263	0,35
3	5	15	162	0,33
4	6	24	27	0,23
5	7	35	22	0,22
6	8	48	5	0,18
7	9	63	2	0,18
8	10	80	2	0,16
9	11	99	0	0,13
168	1	168	14	0.10
171	1	171	14	0.10

progressive à mesure que α augmente. À $\alpha = 10\%$, 155 lignes sont détectées, mais ce nombre diminue régulièrement pour atteindre 0 lignes à $\alpha = 45\%$. Cette tendance suggère que des valeurs plus élevées de α conduisent à une détection moins nombreuse mais plus sélective des lignes droites. Cependant, à partir des certaines valeurs de α , THR ne détecte aucune droites. Deuxièmement, en ce qui concerne le temps de calcul, on observe une diminution correspondante à mesure que α augmente. Pour $\alpha = 10\%$, le temps de calcul est de 0,49 secondes, et il diminue progressivement pour atteindre 0,13 secondes à $\alpha = 45\%$. La diminution du temps de calcul est plus marquée après $\alpha = 30\%$, passant de 0,33 à 0,13 secondes. Cette tendance suggère qu'opter pour des valeurs élevées de α peut améliorer le temps de traitement. La courbe représentative dans la figure 2.4 confirme visuellement ces tendances, avec une décroissance du nombre de lignes détectées d'une part, et une décroissance du temps de calcul d'autre part.



FIGURE 2.5 – (a) Détection de lignes verticales de l'immeuble avec la THR ($l = 168$, $L = 1$, $S_R = 168$, $\alpha = 30\%$, $seuil = 50$), (b) détection de bords de l'autoroute avec la THR ($l = 29$, $L = 238$, $S_R = 6902$, $\alpha = 10\%$, $seuil = 40$)

3.1.2 Variation de la taille des cellules rectangulaires

Le tableau 2.2 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR. Les simulations ont été réalisées avec des paramètres fixes ($\alpha = 30\%$ et $Seuil = 50$), $1 \leq L \leq Nl$ et $1 \leq l \leq Nc$. Le tableau comporte cinq colonnes : La première et la deuxième colonne représentent respectivement les différentes valeurs de la largeur l et la longueur L du rectangle. La troisième colonne représente l'aire S_R du rectangle. La quatrième colonne montre le nombre de droites détectées pour chaque valeur de S_R . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse des données présentées dans le tableau et les graphiques correspondant de la figure 2.6) permet de tirer plusieurs conclusions importantes.

Premièrement, en ce qui concerne le nombre de droites détectées, une tendance claire émerge. En effet, il y a une diminution régulière du nombre de lignes détectées à mesure que la surface des cellules augmente. Mais à partir d'un certain range, à $S_R = 168$, cette tendance est compromise. Ainsi l'augmentation de la taille des cellules de la grille n'entraîne pas toujours la diminution des droites détectées. Deuxièmement, le temps de calcul varie également en fonction de la surface des cellules rectangulaires. Bien que les cellules de plus petite surface nécessitent généralement plus de temps de calcul, avec un maximum à 0,35 secondes pour la cellule de surface 8, le temps de calcul diminue globalement avec l'augmentation de la surface des cellules. La cellule de plus grande surface ($l = 9$, $L = 11$) atteint un temps de calcul minimal de 0,13 secondes, suggérant une amélioration du temps de traitement pour les cellules de grande taille.

La courbe représentative dans la figure 2.6 confirme visuellement ces tendances, montrant une décroissance du nombre de lignes détectées et une diminution du temps de calcul à mesure que la surface des cellules rectangulaires augmente.

Ainsi, les simulations réalisées sur l'image représentant l'immeuble démontrent que l'algorithme de la transformée de Hough rectangulaire est capable de détecter, avec une précision acceptable, les droites discrètes présentes dans l'image de l'immeuble. En ajustant les paramètres appropriés, comme

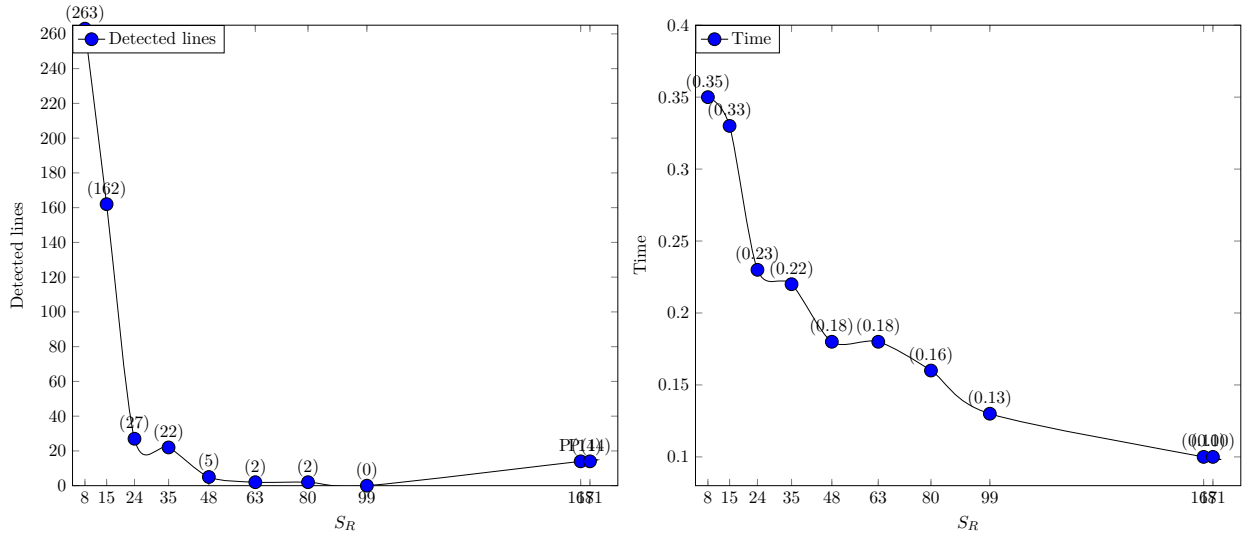


FIGURE 2.6 – Courbe représentative des données du tableau 2.2

la taille des cellules rectangulaires défini par S_R , α et le seuil de vote, nous avons pu minimiser le temps de calcul tout en maintenant une précision raisonnable comme illustre la première image la figure 2.5.

3.2 Cas de l'image de l'autoroute

Dans cette section, nous explorons l'application de la transformée de Hough rectangulaire à le cas de la détection de lignes dans l'image d'une autoroute. Nous étudions les performances de la transformée de Hough rectangulaire en ajustant différents paramètres, tels que le pourcentage α de pixels allumés des cellules rectangulaires et la taille des cellules rectangulaires.

3.2.1 Variation de α

Le tableau 2.3 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute en utilisant l'algorithme 4 de la transformée de Hough rectangulaire. Les simulations ont été réalisées avec des paramètres fixes ($l = 3$, $L = 5$ and $Seuil = 40$) et α variant entre 10% et 50%. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 50%. Ce paramètre définit la taille de la cellule Rectangulaire. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α .

Les résultats de la simulation sur l'image de l'autoroute en utilisant l'algorithme de la transformée de Hough rectangulaire avec des paramètres fixes ($l = 3$, $L = 5$ et $Seuil = 40$) et une variation de α entre 10% et 50% sont analysés.

En ce qui concerne le nombre de lignes détectées, on observe une tendance à la diminution à mesure que α augmente. À $\alpha = 10\%$, 493 lignes sont détectées, et ce nombre diminue progressivement pour atteindre 0 lignes à $\alpha = 50\%$. Ainsi, des valeurs trop élevées de α peuvent entraîner une détection Nulle.

TABLEAU 2.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres fixes ($l = 3$, $L = 5$ and $Seuil = 40$) et α variant entre 10% et 50% où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

α (%)	Droites détectées	temps(sec)
10	493	0,21
15	384	0,20
20	384	0,21
25	195	0,16
30	118	0,15
35	25	0,11
40	25	0,11
45	4	0,07
50	0	0,05

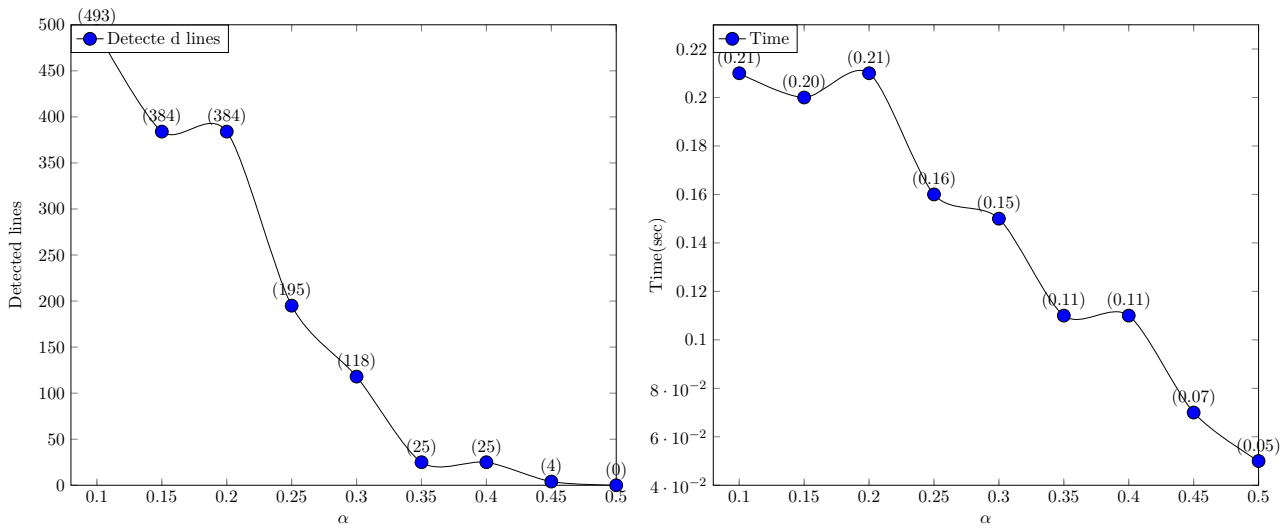


FIGURE 2.7 – Courbe représentative des données du tableau 2.3

Le temps de calcul, quant à lui, montre une décroissance avec l'augmentation de α . À $\alpha = 10\%$, le temps de calcul est de 0,21 seconde, et il diminue progressivement pour atteindre 0,05 seconde à $\alpha = 50\%$. La réduction du temps de calcul est plus marquée après $\alpha = 35\%$, passant de 0,15 à 0,05 secondes.

Le graphique des données du tableau 2.3, représenté dans la figure 2.7, confirme visuellement ces tendances, montrant une décroissance du nombre de lignes détectées et du temps de calcul. Ces résultats suggèrent que des valeurs plus élevées de α conduisent à une détection plus sélective des lignes, tout en améliorant le temps de calcul.

3.2.2 Variation de la taille des cellules rectangulaires

Les résultats de la simulation sur l'image de l'autoroute en utilisant l'algorithme 4 de la transformée de Hough rectangulaire avec une variation de S_R sont analysés. S_R représente l'aire d'une cellule rectangulaire. Les paramètres fixes incluent un taux α de 30% et un seuil de 40 votes, tandis que les dimensions du rectangle, représentées par l et L , varient respectivement entre 2 et 6 et entre 4 et 8.

TABLEAU 2.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres fixes ($\alpha = 30\%$ et $Seuil = 40$), $1 \leq L \leq Nl$ et $1 \leq l \leq Nc$ où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

l	L	S_R	Droites détectées	temps(sec)
2	4	8	184	0,16
3	5	15	118	0,14
4	6	24	21	0,12
5	7	35	2	0,08
6	8	48	0	0,06

l	L	S_R	Droites détectées	temps(sec)
$\alpha = 10$	29	84	111	0,12
	29	217	10	0,07
	29	238	17	0,08

Les résultats sont présentés dans le Tableau 2.4 et visualisés dans la Figure 2.8. Le tableau comporte cinq colonnes : La première et la deuxième colonne représentent respectivement les différentes valeurs de la largeur l et la longueur L du rectangle. La troisième colonne représente l'aire S_R du rectangle. La quatrième colonne montre le nombre de droites détectées pour chaque valeur de S_R . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

En ce qui concerne le nombre de lignes détectées, à mesure que l'aire des cellules augmente, il diminue, passant de 184 droites détectées pour une surface de 8 px^2 à 0 pour les aires supérieures ou égales 48 px^2 . Cependant, en diminuant le pourcentage de pixels allumés α à 10%, nous remarquons que il est possible de détecter, avec une précision acceptable, des droites avec des cellules de grandes taille comme le montre le tableau et la deuxième image de la figure 2.5. Aussi, le temps de calcul diminue avec l'augmentation de l'aire S_R des cellules rectangulaires, indiquant une amélioration du temps de traitement avec les grandes cellules rectangulaires. Les représentations graphiques 2.8 confirme ces résultats.

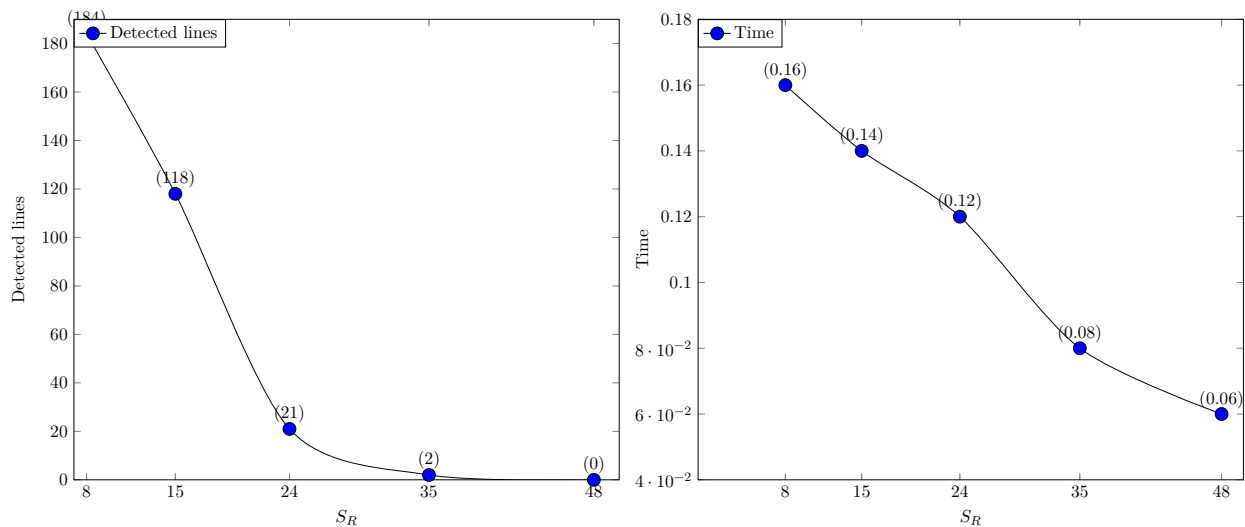


FIGURE 2.8 – Courbe représentative des données du tableau 2.4

Ainsi, les résultats obtenus ont démontré la capacité de cette méthode à identifier avec une précision

TABLEAU 2.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)

Seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
40	9281	0,49	0,00005
116	10	0,49	0,049

TABLEAU 2.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)

Seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
51	169	0,24	0,0014
100	10	0,21	0,021

acceptable les lignes de l'immeuble ainsi que celles délimitant l'autoroute. Les données des tableaux 2.2 et 2.4 ainsi que les résultats visuels représentés dans la figure 2.5 suggèrent qu'avec les grandes cellules, il est possible d'avoir des économies computationnelles tout en gardant une précision acceptable. Avec la transformée de Hough rectangulaire, il est possible de cibler et détecter les droites en fonction de leurs position : soit verticale, soit horizontale. Pour les droites verticales, il faut fixer la longueur L de la cellule rectangulaire à 1 et prendre la largeur l de la cellule rectangulaire dans un voisinage de la hauteur de l'image. Pour les droites horizontales, il faut fixer la largeur l à 1 et prendre la longueur L dans un voisinage de la largeur de l'image.

3.3 Comparaison de la Transformée de Hough Rectangulaire et Transformée de Hough Standard



FIGURE 2.9 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40); (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

Nous procédons dans cette section à une comparaison de la transformée de Hough rectangulaire, méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough rectangulaire sont illustré dans les tableaux 2.5 et 2.7 respectivement. En ce qui concerne la précision, pour un seuil de 40 votes, la THS a effectué une détection de 9281 droites. Cette détection est non pertinente comme le montre la première image de la figure 2.9. par contre, avec le même seuil de votes, la THR a détecté 11 droites. Les résultats visuels illustrés par la



FIGURE 2.10 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

deuxième image de la figure 2.11 montre que les droites détectées suivent bien les lignes de l'immeuble suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough rectangulaire pour parvenir à ce résultat sont : les dimensions $(l; L)$ et le pourcentage α de pixels allumés de chaque cellule rectangulaire.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 2.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 2.9 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'immeuble indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 2.5 montre que la transformée de Hough standard a détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La transformée de Hough rectangulaire a détecté quasiment le même nombre de droite en 0.15 seconde soit 0.013 seconde par droite détecté comme illustré dans le tableau 2.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,49}{0,15} = 3,26$ (T_h représente le temps d'exécution de l'algorithme de notre méthode, tandis que T_s signifie le temps d'exécution de l'algorithme standard).

De même avec l'image de l'autoroute, les résultats des expériences avec la THS et la THR sont illustré dans les tableaux 2.6 et 2.8 respectivement.

En ce qui concerne la précision, pour un seuil de 51 votes, la THS a effectué une détection de 169 droites. Cette détection est non pertinente comme le montre la première image de la figure 2.10. par contre, avec le même seuil de votes, la transformée de Hough rectangulaire a détecté 11 droites. Les résultats visuels illustrés par la première image de la figure 2.12 montre que les droites détectées suivent bien les lignes de l'autoroute suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough rectangulaire pour parvenir à ce résultat sont : les dimensions $(l; L)$ et le pourcentage α de pixels allumés de chaque cellule rectangulaire.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 2.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 2.10 illustre bien ces résultats. Les 10

TABLEAU 2.7 – Résultats de simulation avec la THR appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

(l, L)	S_R	α	seuil	number of detected lines	time1(sec)	time2(sec)
(3,5)	15	0,4	40	39	0,20	0,005
(5,7)	35	0,35	35	11	0,15	0,013

TABLEAU 2.8 – Résultats de simulation avec la THR appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

(l, L)	S_R	α	seuil	number of detected lines	time1(sec)	time2(sec)
(3,5)	15	0,4	51	11	0,11	0,01
(5,7)	35	0,25	45	20	0,13	0,0065

droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 2.6 montre que la THS a détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La THR a détecté quasiment le même nombre de droite en 0.11 seconde soit 0.01 seconde par droite détecté comme illustré dans le tableau 2.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,21}{0,11} = 1,9$ (T_h représente le temps d'exécution de l'algorithme de notre méthode, tandis que T_s signifie le temps d'exécution de l'algorithme standard).



FIGURE 2.11 – Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=40, $\alpha = 40\%$, $L = 5$ et $l = 3$; (b) Seuil=40, $\alpha = 40\%$, $L = 5$ et $l = 3$

3.4 Comparaison de la Transformée de Hough Rectangulaire et sa version parallélisée

Dans cette section, nous procédons à une analyse comparative des performances de la transformée de Hough rectangulaire et de sa version parallélisée (THRP) sur deux images tests : une image d'immeuble et une image d'autoroute représentées sur la figure 1.11.

3.4.1 Cas de l'image de l'immeuble

La figure 2.13 présente deux graphiques comparant les performances des algorithmes de transformée de Hough rectangulaire et de transformée de Hough rectangulaire parallèle. Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et S_R).



FIGURE 2.12 – Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=51, $\alpha = 40\%$, $L = 5$ et $l = 3$; (b) Seuil=45, $\alpha = 25\%$, $L = 7$ et $l = 5$

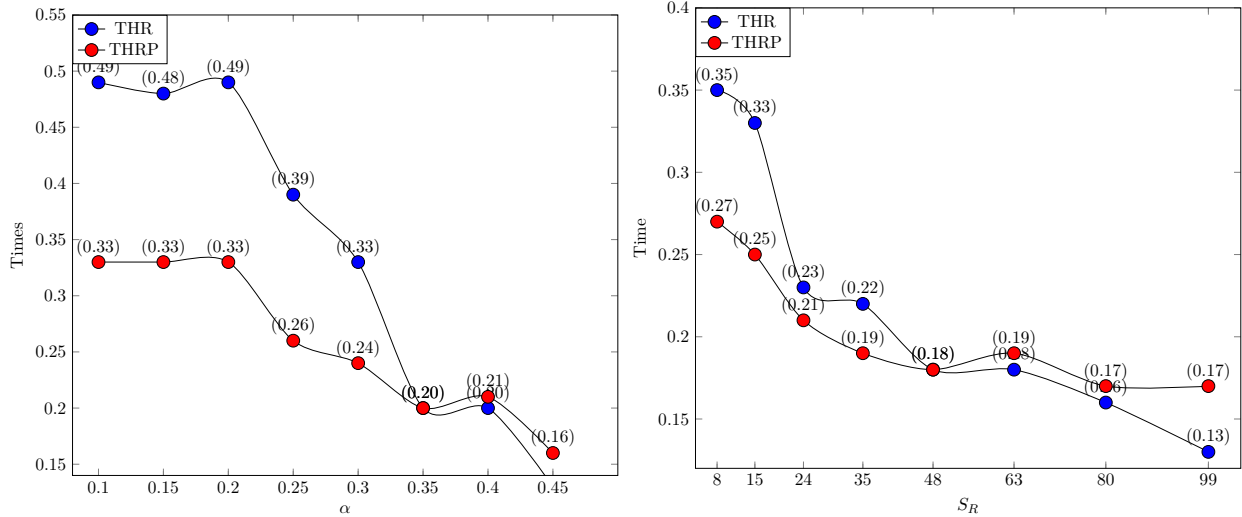


FIGURE 2.13 – Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et $seuil = 70$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres ($seuil = 50$, $\alpha = 30\%$) des l'algorithme 4 et 8

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 45%. Pour des valeurs de α plus basses (entre 10% et 35%), la courbe de la transformée de Hough rectangulaire parallélisée est en dessous de celle de la transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire parallélisée est plus rapide pour les petites valeurs de α . Pour des valeurs de α plus élevées (entre 35% et 45%), la courbe de la transformée de Hough rectangulaire parallélisée passe au dessus de celle de la transformée de Hough rectangulaire. Cela suggère que la version parallélisée est moins rapide pour les valeurs élevées de α .

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de la surface des cellules rectangulaires, avec les deux algorithmes de la THR et la THRP. Les surfaces des cellules varient de 8 px^2 à 99 px^2 . La courbe de la transformée de Hough rectangulaire parallélisée est en dessous de celle de la transformée de Hough rectangulaire pour les surfaces inférieure a 48 px^2 . A 48 px^2 , la courbe de la transformée de Hough rectangulaire parallélisée passe au dessus de la courbe de la transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire est moins rapide que sa version parallélisée pour les petites valeurs de S_R et est plus rapide pour les grandes valeurs de S_R .

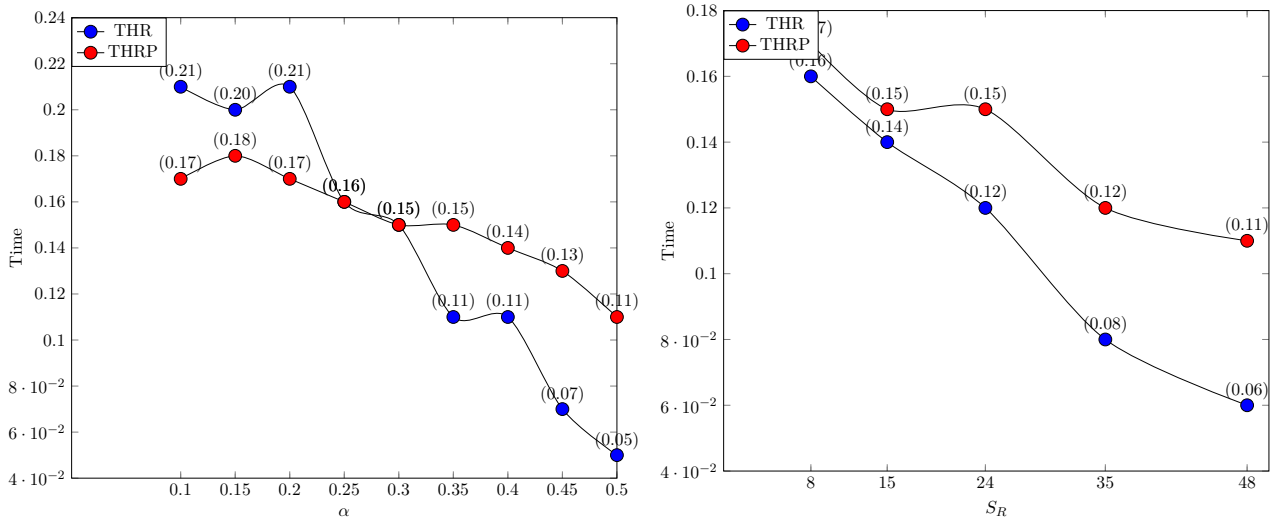


FIGURE 2.14 – Cas de l’autoroute : (à gauche) Comparaison des variations du temps d’exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et $seuil = 40$), (à droite) Comparaison des variations du temps d’exécution en fonction de la surface des cellules rectangulaires et les paramètres ($seuil = 40$, $\alpha = 30\%$) des l’algorithmes 4 et 8

3.4.2 Cas de l’image de l’autoroute

La figure 2.14 présente deux graphiques comparant les performances des algorithmes de la THR et la THRP. Les graphiques comparent les variations du temps d’exécution en fonction des paramètres α et S_R .

Dans le premier graphique, les variations du temps d’exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 50%. Pour les petites valeurs de α (entre 10% et 25%), la courbe de la transformée de Hough rectangulaire parallélisée est en dessous de celle de transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire parallélisée est plus rapide pour ces valeurs de α . Pour des valeurs de α plus élevés (entre 30% et 50%), la courbe de la transformée de Hough rectangulaire parallélisée passe au dessus de celle de la transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire parallélisée est moins rapide pour des valeurs de α supérieur à 30%.

Dans le deuxième graphique, les variations du temps d’exécution sont comparées en fonction de la surface des cellules rectangulaires, avec les deux algorithmes de la transformée de Hough rectangulaire et sa version parallélisée. Les surfaces des cellules varient de 8 à 48 px^2 . La courbe de transformée de Hough rectangulaire parallélisée est au dessus de celle de la transformée de Hough rectangulaire pour toutes les valeurs de S_R . Cela suggère que la transformée de Hough rectangulaire parallélisée est moins rapide que la transformée de Hough rectangulaire dans ce cas.

α (%)	(L,l)	seuil	Droites	Temps (sec)
20	(3,6)	30	5	0,10

TABEAU 2.9 – Approche proposée

3.5 Comparaison avec d'autres approches

Dans cette section, nous comparons notre approche de détection de lignes à celle proposée par Traoré et al [4]. Cette comparaison vise à évaluer les performances relatives des deux méthodes en termes de précision dans la détection des lignes d'une part, et de leur performance temporelle d'autre part. Nous examinerons en détail les résultats visuels obtenus ainsi que les temps d'exécution pour éclairer les forces et les faiblesses de chaque approche. Cette analyse comparative permettra de déterminer les avantages et les limitations de chaque méthode, fournissant ainsi des indications précieuses pour le choix de l'approche la plus appropriée en fonction des besoins spécifiques de l'application envisagée.

Dans l'approche de la transformée de Hough rectangulaire proposée dans ce document, les dimensions des cellules rectangulaires sont paramétrées contrairement à l'approche proposée par Traoré et al [4]. Ce qui donne une plus grande flexibilité dans l'ajustement des paramètres.

3.5.1 Précision

les résultats visuels de la simulation de l'algorithme de l'approche de Traoré et al [4] dans la figure 2.15 montre une détection acceptable de deux côtés du pentagone et une détection non-parfaite des trois autres côtés avec les paramètres mentionnés dans le tableau 2.10. La figure 2.16 illustre la détection des lignes délimitant l'autoroute avec l'approche de Amed [4]. Trois de ces lignes ont été bien détectées. Les quatre autres détections ont été défectueuse. La figure 2.17 illustre la détection des lignes de l'immeuble avec l'approche de Traoré et al [4]. Aucune ligne de l'immeuble n'a été détecté.

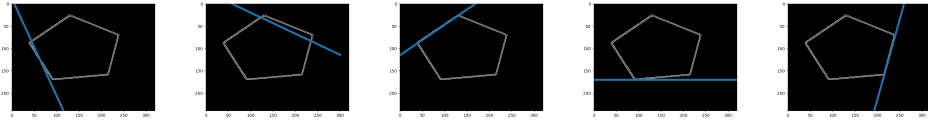


FIGURE 2.15 – Lignes droites détectées sur le pentagone avec l'approche de Traoré et al [4]

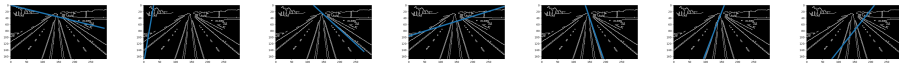


FIGURE 2.16 – Lignes droites détectées sur l'autoroute avec l'approche de Traoré et al [4]

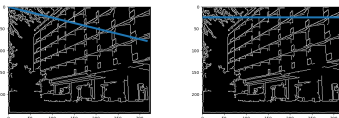


FIGURE 2.17 – Lignes droites détectées sur l'immeuble avec l'approche de Traoré et al [4]

La première de la figure 2.18 illustre les résultats visuels de simulations de notre algorithme sur l'image du pentagone en utilisant les paramètres mentionnés dans le tableau 2.9. Elle montre une

détection acceptable de quatre côtés du pentagone et un côté non détecté. La deuxième image de la figure 2.18 montre une détection acceptable de cinq lignes délimitant l'autoroute et la troisième image de la même figure montre une détection claire de lignes horizontale et verticale.

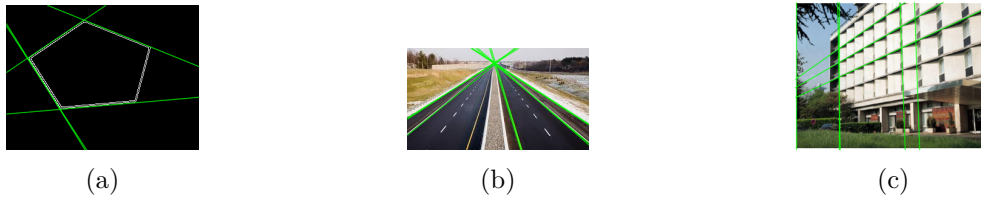


FIGURE 2.18 – Lignes droites détectées avec notre méthode

α (%)	seuil	Droites	Temps (sec)
10	80	5	0.17

TABEAU 2.10 – Approche de Traoré et al [4]

Ainsi, les résultats obtenus montrent que notre méthode présente des performances supérieures dans plusieurs aspects par rapport à l'approche de Traoré et al.

Dans le cas de la détection des côtés du pentagone, notre méthode parvient à détecter quatre côtés sur cinq de manière acceptable, tandis que l'approche de Traoré et al ne parvient qu'à une détection acceptable de deux côtés. De même, pour la détection des lignes délimitant l'autoroute, notre méthode réussit à détecter cinq lignes sur sept de manière satisfaisante, alors que l'approche de Traoré et al n'en détecte que trois correctement. En outre, alors que l'approche de Traoré et al ne parvient pas à détecter les lignes de l'immeuble, notre méthode parvient à identifier les lignes de l'immeuble.

3.5.2 Performance temporelle

En ce qui concerne la performance temporelle, les résultats obtenus montrent des différences entre l'approche de Traoré et al et la méthode proposée dans ce chapitre. L'approche de Traoré et al nécessite un temps d'exécution moyen de 0,17 secondes pour détecter cinq lignes avec un seuil de 80 et un paramètre α de 10%, tandis que notre méthode atteint une meilleure performance avec un temps d'exécution moyen de seulement 0,10 secondes pour détecter cinq lignes, utilisant un seuil de 30 et un paramètre α de 20%, avec des dimensions (L,l) fixées à (3,6). Ces résultats mettent en évidence l'amélioration du temps de traitement de la méthode proposée.

Conclusion

L'étude de la technique de la transformée de Hough rectangulaire a révélé ses capacités prometteuses dans la détection de lignes droites dans les images. Nous avons décrit en détail la méthode de fonctionnement de la transformée de Hough rectangulaire, mettant en lumière ses étapes clés et ses pa-

ramètres ajustables, ainsi que ses avantages par rapport à d'autres approches telles que la transformée de Hough standard.

Les simulations ont démontré la capacité de la transformée de Hough rectangulaire à détecter avec une précision acceptable les lignes droites, tout en offrant une flexibilité supplémentaire grâce à la possibilité d'ajuster les paramètres de maillage et la réduction du temps de calcul. Aussi, avec la réduction du nombre de pixels à calculer grâce à la variable α , la transformée de Hough rectangulaire offre des économies computationnelles importantes. aussi, avec la transformée de Hough rectangulaire, il est possible de cibler et détecter les droites en fonction de leurs position : soit verticale, soit horizontale.

De plus, nos comparaisons avec l'approche proposée par Traoré et al [4] ont mis en évidence les performances supérieures de la transformée de Hough rectangulaire en termes de précision de détection et de temps d'exécution. Aussi, le choix entre transformée de Hough rectangulaire et transformée de Hough rectangulaire parallélisée dépend des valeurs spécifiques de α et de S_R dans un contexte donné. Globalement, THRP semble être plus performante pour les petites valeurs de α et de S_R , tandis que transformée de Hough rectangulaire pourrait être préférable pour les valeurs plus élevées de ces paramètres. Néanmoins, même si elle présente des avantages, la transformée de Hough rectangulaire comporte des limitations. Celles-ci incluent la perte d'informations provenant de l'ensemble complet des pixels allumés des cellules rectangulaires, la sensibilité aux taux α de pixels allumés et à la taille des cellules rectangulaires, la complexité de sa mise en œuvre, sa dépendance au contexte ainsi que la sensibilité aux paramètres. Dans le chapitre suivant, nous aborderons la transformée de Hough triangulaire.

Transformée de Hough Triangulaire

Introduction	57
1 Description de la méthode	58
2 Complexité de la transformée de Hough triangulaire	65
3 Simulations et discussions	66
4 Analyse comparative	72
Conclusion	81

Introduction

La détection de lignes droites dans les images constitue une étape cruciale dans de nombreuses applications de vision par ordinateur. Parmi les nombreuses méthodes développées à cet effet, la transformée de Hough (TH) est largement reconnue pour sa robustesse et sa capacité à détecter des structures linéaires dans des images bruitées. Cependant, son utilisation standard comporte des limites, notamment en termes de performance temporelle.

Dans ce chapitre, nous introduisons une nouvelle approche de la transformée de Hough standard (THS) dénommée transformée de Hough triangulaire (THT). La transformée de Hough triangulaire repose sur le concept de grille virtuelle, où des cellules triangulaires virtuelles sont générées sur l'image. Seules les cellules ayant un certains taux de pixel allumés sont considéré dans les calculs.

Dans ce chapitre, nous présentons en détail la méthode de la transformée de Hough triangulaire, en commençant par la description du maillage triangulaire de l'image et de la sélection des cellules triangulaires. Nous discutons ensuite des techniques de reconnaissance des droites discrètes utilisées dans la transformée de Hough triangulaire et de son application pratique dans la détection de lignes droites dans les images. Aussi, nous nous penchons sur la comparaison des performances entre la

transformée de Hough triangulaire et sa version parallélisée (THTP).

Nous évaluons les performances de la transformée de Hough triangulaire à travers des simulations sur des images, en comparant ses résultats avec ceux de la transformée de Hough standard. Enfin, nous discutons des avantages et des limitations de la transformée de Hough triangulaire, ainsi que de ses implications potentielles pour les applications de vision par ordinateur.

1 Description de la méthode

La description de la méthode présentée dans cette section vise à détailler le processus de maillage triangulaire de l'image ainsi que les techniques de reconnaissance des droites discrètes utilisant la transformée de Hough triangulaire. Les étapes de la méthode sont illustrées dans la figure 3.4. Le maillage triangulaire, fondé sur le théorème de Thalès et accompagné d'algorithmes spécifiques, permet de subdiviser l'espace image en une grille virtuelle de triangles. Cette subdivision sert de base à la détection de droites discrètes dans l'image.

Par la suite, des simulations pour la détection des droites discrètes sont effectuées à l'aide de la transformée de Hough triangulaire, où chaque pixel allumé du triangle de la grille est utilisé comme élément de vote dans l'accumulateur. Les détails des algorithmes utilisés pour parcourir la grille, compter les votes et détecter les droites sont discutés en détail, offrant ainsi un aperçu complet du processus de reconnaissance.

1.1 Maillage triangulaire de l'image

Le maillage triangulaire de l'image consiste à générer une grille triangulaire virtuelle sur l'espace image. La grille est constituée de cellules triangulaires. Le théorème de Thalès est utilisé ici pour calculer le dual de chaque cellule. Nous rappelons ce théorème comme suit :

Theoreme 1 (Thales) *Soit (ABC) un triangle rectangle. Soient D et E deux points sur les droites (AC) et (BC) respectivement, de manière à avoir la droite (DE) parallèle à la droite (AB) . Alors :*

$$\frac{CD}{CA} = \frac{DE}{AB} = \frac{CE}{CB}$$

Cela signifie que le théorème de Thalès met en évidence des relations de proportionnalité entre des droites parallèles. Par exemple, avec la relation $\left(\frac{CD}{CA} = \frac{DE}{AB}\right)$ du théorème de Thalès, nous obtenons donc $DE = \frac{CD \times AB}{CA}$, comme illustré dans la figure 3.1.

Dans la figure 3.1, les segments $[AB]$, $[AC]$ sont respectivement la base et la hauteur du triangle (ABC) : la base est liée à l'axe (y) , tandis que la hauteur est associée à l'axe (x) .

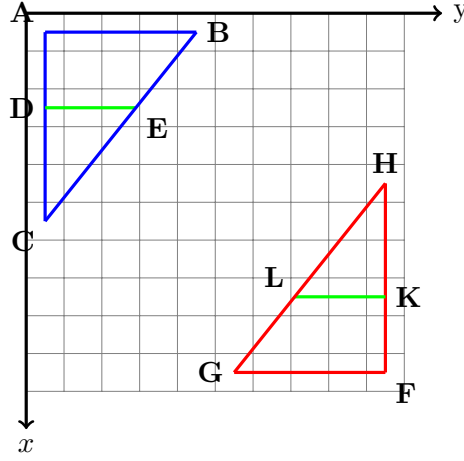


FIGURE 3.1 – Deux triangles rectangles (ABC) et (FGH) avec respectivement des droites parallèles (AB) et (DE), et (FG) et (KL).

La hauteur et la base des triangles rectangles sont passées en paramètres dans l'algorithme 10. Une illustration de deux triangles rectangles est présentée dans la figure 3.1.

Algorithme 10 : Construction de la grille triangulaire

Fonction *maillage_tri*(Nl, Nc, h, b : entiers) : tableau d'entiers
Variables : tab : tableau de 6 entiers
Output : tab;
begin
 $tab[1] \leftarrow b$; % la base du triangle
 $tab[3] \leftarrow h$; % la hauteur du triangle
 $tab[0] \leftarrow \text{int}[Nl/h]$; % nombre de partis de longueur $tab[3]$ sur la hauteur de l'image
 $tab[2] \leftarrow \text{int}[Nc/b]$; % nombre de partis de longueur $tab[1]$ sur la largeur de l'image
 $tab[4] \leftarrow Nl \bmod h$; % le reste dans la division euclidienne de Nl par h
 $tab[5] \leftarrow Nc \bmod b$; % le reste dans la division euclidienne de Nc par b
return tab;
fin

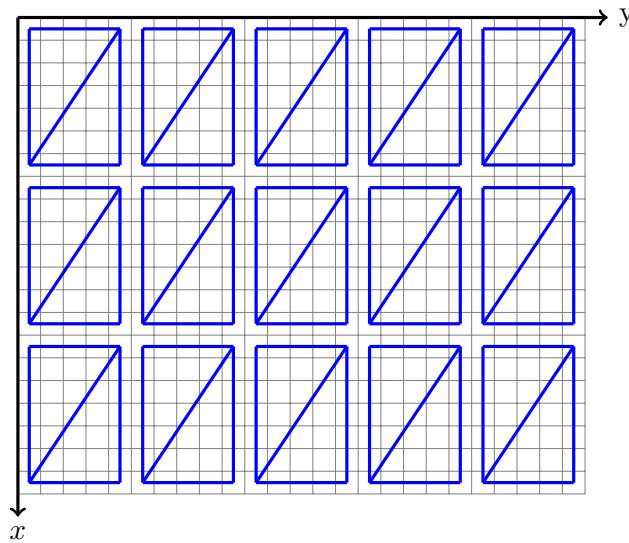


FIGURE 3.2 – Une grille triangulaire superposée à l'espace image

Algorithme 11 : Mise à jour de l'accumulateur

```
Procédure Acc_update() : vide
    % Mise à jour de l'accumulateur
    if img[z, t]  $\neq$  0 then
        for itheta dans range(accum_row) do
            % Calcul des coordonnées polaires du pixel "img[z,t]".
            theta  $\leftarrow$  itheta  $\times$  dtheta;
            rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
            irho  $\leftarrow$  int(rho);
            if irho > 0 et irho < accum_column then
                | accum[itheta][irho]  $\leftarrow$  accum[itheta][irho] + 1
            end
        end
    end
fin
```

Après la création de la grille triangulaire, présenté dans la figure 3.2, le calcul du dual du triangle dépend du nombre de pixels allumés dans chaque triangle.

1.2 Parallélisation de la transformée de Hough triangulaire

La parallélisation de la transformée de Hough triangulaire nécessite une adaptation de l'algorithme afin de tirer pleinement parti des capacités de traitement simultané offertes par le parallélisme. Cette parallélisation concerne principalement l'accumulateur, également appelé l'espace des paramètres ou l'espace de Hough. Les algorithmes 15 et 14 sont les versions parallélisées des algorithmes 13 et 11. Les principales étapes de cette parallélisation sont les suivantes :

- importation de la bibliothèque *pypm*³ : importer une bibliothèque en Python est nécessaire pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction *import pypm* est utilisée pour charger la bibliothèque *pypm*.

- création de l'accumulateur de type *pypm* : dans l'algorithme séquentiel, l'espace des paramètres (l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la transformée de Hough, est transformé en une structure de données partagée grâce à *pypm*. En effet, *pypm* permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.

L'instruction "*accum_pypm* $\leftarrow$ *pypm.shared.array([accum_row, accum_column])*" de l'algorithme 15 crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable "*accum_pypm*".

- construction parallèle : l'instruction "*with pypm.Parallel(4) as p*" de l'algorithme 15 indique que nous souhaitons exécuter le bloc de code à l'intérieur de la déclaration "with" en parallèle avec 4 processus.

3. <https://github.com/classner/pypm/tree/main>

Algorithme 12 : Taux de pixels allumés

```

Fonction count_tri(img : image, A, B, C : 3 tableaux de deux entiers) : réel
  Variables : k, l : integers;
  D : table of two integers
  begin
    k ← 0; % Initialisation du nombre de pixels allumés
    l ← 0; % Initialisation du nombre de pixels éteints
    if B[1]δ=A[1] and C[0]δ=A[0] then
      for x allant de A[0] à C[0] do
         $DE \leftarrow \frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|}$ ;
        y0 ← A[1] + round(DE);
        for y allant de A[1] à y0 do
          if img[x, y] ≠ 0 then
            | k ← k + 1;
          else
            | l ← l + 1;
          end
        end
      end
    if B[1]i=A[1] and C[0]i=A[0] then
      for x allant de C[0] à A[0] do
         $DE \leftarrow \frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|}$ ;
        y0 ← A[1] - E[DE];
        % E[DE] est la partie entière de DE
        for y allant de y0 à A[1] do
          if img[x, y] ≠ 0 then
            | k ← k + 1;
          else
            | l ← l + 1;
          end
        end
      end
    end
    return  $\frac{k}{k + l}$ ;
  fin

```

- parallélisation de l'algorithme 11 de mise à jours de l'accumulateur : la fonction "*p.range*" génère une séquence d'indices à partir de laquelle chaque cœur du CPU obtiendra une plage d'indices spécifique. Cela permet de diviser le travail entre les cœurs de manière équitable. Ainsi la boucle "*for theta in p.range(accum_row)*" de l'algorithme 14 utilise les indices compris entre 0 et *accum_row* − 1 pour itérer à travers l'accumulateur par sections, où chaque section est traitée par un processus différent.

1.3 Sélection des cellules triangulaires

La sélection d'une cellule triangulaire dépend de son taux de pixel allumé. L'algorithme 12 calcule le taux de pixels allumés dans une région triangulaire définie par trois points spécifiés par les tableaux *A*, *B*, et *C*. L'image d'entrée est représentée par *img*. L'algorithme utilise deux boucles pour parcourir les pixels de la région délimitée par les points du triangle. Selon la configuration du triangle (si la coordonnée *y* de *B* est supérieure ou égale à celle de *A*, et la coordonnée *x* de *C* est supérieure ou égale à celle de *A*), l'algorithme calcule la valeur de *DE* et détermine la portée de l'itération *y*. Ensuite, il compte le nombre de pixels allumés (*k*) et éteints (*l*) dans la région définie par le triangle. Le résultat renvoyé est le taux de pixels allumés dans cette région, calculé comme le rapport de *k* sur la somme de *k* et *l*.

Le parcours des pixels du triangle se fait à l'aide du théorème de Thalès avec l'instruction suivante :

$$DE \leftarrow \frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|} \quad (3.1)$$

Il convient de noter que, pour une cellule particulière de la grille, tous les pixels ne sont pas pris

Algorithme 13 : Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire

```

Fonction Triangular(img, h, b,  $\alpha$ , seuil) : Tableau
Pre-condition :  $1 \leq h \leq Nl$  et  $1 \leq b \leq Nc$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
Variables : tab : tableau de 6 entiers; A, B, C : tableau de 2 entiers; accum_row, accum_column, irho, Nl, Nc : entiers;
accum : matrice de dimensions "accum_row  $\times$  accum_column"; dtheta, rho, theta, DE : réels;
Begin
    Nl  $\leftarrow$  nombre de ligne de l'image; Nc  $\leftarrow$  nombre de colonne de l'image; % the dimensions of the image ;
    accum_row  $\leftarrow$  180; accum_column  $\leftarrow$   $E(\sqrt{(Nc^2 + Nl^2)})$ ; % les dimensions de la matrice "accum";
    dtheta =  $\pi/180$ ; tab  $\leftarrow$  maillage_tri(Nl, Nc) ;
    H = Nl - tab[4]; L = Nc - tab[5] ;
    % parcours des triangles de la grille sans tenir compte des résidus
    for x allant de tab[3] à Nl par pas de tab[3]+1 do
        for y allant de tab[1] à Nc par pas de tab[1]+1 do
            A[0] = x - tab[3]; A[1] = y - tab[1]; B[0] = x - tab[3]; B[1] = y; C[0] = x; C[1] = y - tab[1]; D = A;
            if B[1]i=A[1] et C[0]i=A[0] then
                if count(img, A, B, C)  $\geq \alpha$  then
                    for z allant de A[0] à C[0] do
                        DE  $\leftarrow \frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|}$ ; y0  $\leftarrow$  A[1] + round(DE);
                        for t allant de A[1] à y0 do
                            Acc.update();
                        end
                    end
                end
            end
            A[0] = x; A[1] = y; B[0] = x - tab[3]; B[1] = y; C[0] = x; C[1] = y - tab[1]; D = A;
            if B[1]i=A[1] et C[0]i=A[0] then
                if count(img, A, B, C)  $\geq \alpha$  then
                    for z allant de C[0] à A[0] do
                        DE  $\leftarrow \frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|}$ ; y0  $\leftarrow$  A[1] - E[DE];
                        for t allant de y0 à A[1] do
                            Acc.update();
                        end
                    end
                end
            end
        end
    end
    % traitement des résidus if tab[4]  $\neq$  0 then
        for z allant de H et Nl do
            for t allant de 0 et Nc do
                Acc.update();
            end
        end
    if tab[5]  $\neq$  0 then
        for z allant de 0 and Nl do
            for t allant de L and Nc do
                Acc.update();
            end
        end
    end
    Rechercher les maxima dans l'accumulateur et placer les couples (ρ, θ) dans une liste 'lignes';
    Retourner la liste 'lignes';
fin

```

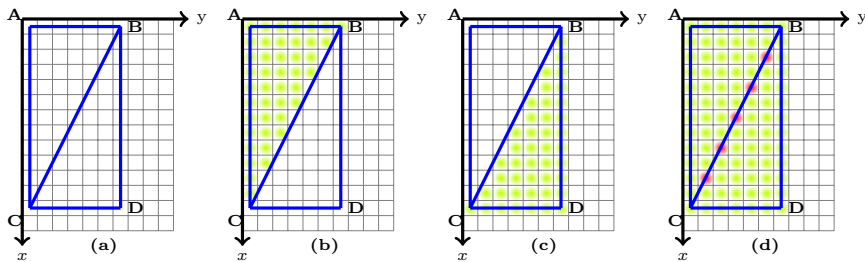


FIGURE 3.3 – Sélection des pixels d'une cellule triangulaire

en compte lors du calcul du taux. Les pixels exclus, en revanche, seront considérés dans les cellules voisines, comme illustré dans la figure 3.3. Cette exclusion vise à garantir qu'un même pixel ne soit pas pris en compte deux fois dans deux triangles distincts. L'aire d'une cellule triangulaire est calculé selon le lemme 1.

Lemme 1 (L'aire d'une cellule triangulaire) Soit b la base et h la hauteur du triangle. L'aire du

Algorithme 14 : Mise à jour parallèle de l'accumulateur

```
Procédure Acc_updateP() : vide
  % Mise à jour de l'accumulateur
  if img[z, t]  $\neq$  0 then
    for itheta dans p.range(accum_row) do
      theta  $\leftarrow$  itheta  $\times$  dtheta;
      rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
      irho  $\leftarrow$  int(rho);
      if irho > 0 et irho < accum_column then
        | accum_pymp[itheta][irho]  $\leftarrow$  accum_pymp[itheta][irho] + 1
      end
    end
  end
fin
```

triangle, représentée par la partie verte de la figure 3.3(b), est calculée à l'aide de la formule suivante :

$$S_T(b, h) = \frac{b(h-1) + 2}{2} \quad (3.2)$$

Preuve 1 (lemme 1) Considérons *b* la base et *h* la hauteur du triangle.

La formule d'un triangle ordinaire est donnée par $\frac{b \times h}{2}$. L'espace image est discret et les cellules triangulaires ont leur diagonale commune deux à deux. Ainsi, les pixels qui sont sur une diagonale commune à deux cellules seront exclu du calcul de l'aire du triangle pour n'est pas que des cellules soit pris en compte deux fois dans les calculs. Par conséquent, la surface du triangle dans ce cas est $\frac{b(h-1)}{2} + 1$. Ce qui donne $S_T(b, h) = \frac{b(h-1)+2}{2}$.

Remarque 3 Étant donné que l'espace image est discrète, toute valeur de $S_T(b, h)$ non entier doit être arrondi à l'entier qui lui est immédiatement supérieur.

Par exemple, pour $b = 7$ et $h = 13$, $S_T(7, 13) = \frac{7(13-1)+2}{2} = 43 \text{ px}^2$, comme illustré dans la figure 3.3. L'unité de mesure de la surface est en pixels carrés (px^2). Aussi pour $b = 5$ et $h = 8$, $S_T(5, 8) = \frac{5(8-1)+2}{2} = 18,5 \text{ px}^2$. d'après la remarque 3, 18,5 est arrondi à 19. Par conséquent on écrit $S_T(5, 8) = 19$.

1.4 Reconnaissance des droites discrètes

L'algorithme 13 de reconnaissance de droites discrètes par la transformée de Hough triangulaire vise à détecter des droites discrètes dans une image en utilisant une variante triangulaire de la Transformée de Hough. Les paramètres de l'algorithme incluent l'image d'entrée (*img*), les dimensions de la grille triangulaire (*h*, *b*), un pourcentage α de pixels allumés dans chaque cellule de la grille, et un seuil global (*Seuil*). L'algorithme parcourt une grille triangulaire générée par la fonction *maillage_tri*(Algorithme 10) et explore les triangles formés en utilisant des coordonnées spécifiques (*A*, *B*, *C*). Pour chaque triangle, les pixels de l'image sont examinés en fonction de certaines conditions et un comptage est

Algorithme 15 : Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire Parallélisée

```

Fonction TriangularP(img, h, b, α, seuil, cpu_count) : Tableau
Pre-condition :  $1 \leq h \leq Nl$  et  $1 \leq b \leq Nc$ ,  $0 \leq \alpha \leq 1$ ,  $0 < seuil$ 
Variables : tab : tableau de 6 entiers; A, B, C : tableau de 2 entiers; accum_row, accum_column, irho, Nl, Nc : entiers;
accum : matrice de dimensions "accum_row × accum_column"; dtheta, rho, theta, DE : réels;
Begin
    import pypm; % importation de la bibliothèque pypm
    Nl ← nombre de ligne de l'image; Nc ← nombre de colonne de l'image; % the dimensions of the image;
    accum_row ← 180; accum_column ←  $E(\sqrt{Nc^2 + Nl^2})$ ; % les dimensions de la matrice "accum";
    accum_pymp ← pypm.shared.array([accum_row, accum_column]) % création de l'accumulateur de type pypm;
    dtheta =  $\pi/180$ ; tab ← maillage_tri(Nl, Nc) ;
    H = Nl - tab[4]; L = Nc - tab[5] ;
    % parcours des triangles de la grille sans tenir compte des résidus
    % Démarrage du traitement parallèle
    With pypm.parallel(cpu_count) as p :
        for x allant de tab[3] à Nl par pas de tab[3]+1 do
            for y allant de tab[1] à Nc par pas de tab[1]+1 do
                A[0] = x - tab[3]; A[1] = y - tab[1]; B[0] = x - tab[3]; B[1] = y; C[0] = x; C[1] = y - tab[1]; D = A;
                if B[1]δ=A[1] et C[0]δ=A[0] then
                    if count(img, A, B, C) ≥  $\alpha$  then
                        for z allant de A[0] à C[0] do
                            DE ←  $\frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|}$ ; y0 ← A[1] + round(DE);
                            for t allant de A[1] à y0 do
                                Acc.update();
                            end
                        end
                    end
                end
                A[0] = x; A[1] = y; B[0] = x - tab[3]; B[1] = y; C[0] = x; C[1] = y - tab[1]; D = A;
                if B[1]i=A[1] et C[0]i=A[0] then
                    if count(img, A, B, C) ≥  $\alpha$  then
                        for z allant de C[0] à A[0] do
                            DE ←  $\frac{|x - C[0]| \times |A[1] - B[1]|}{|A[0] - C[0]|}$ ; y0 ← A[1] - E[DE];
                            for t allant de y0 à A[1] do
                                Acc.update();
                            end
                        end
                    end
                end
            end
        end
        % traitement des résidus if tab[4] ≠ 0 then
            for z allant de H et Nl do
                for t allant de 0 et Nc do
                    Acc.update();
                end
            end
        if tab[5] ≠ 0 then
            for z allant de 0 à Nl do
                for t allant de L à Nc do
                    Acc.update();
                end
            end
        end
    fin
    Rechercher les maxima dans l'accumulateur et placer les couples (ρ, θ) dans une liste 'lignes';
    Retourner la liste 'lignes';
fin

```

effectué. La mise à jour de l'accumulateur se fait en fonction de ces pixels. Les triangles de la grille sont parcourus sans tenir compte des résidus, et ensuite, les résidus sont traités séparément. Finalement, les maxima de l'accumulateur sont recherchés, les couples (θ, ρ) correspondants sont ajoutés à une liste 'lignes', qui est ensuite renvoyée.

L'algorithme 16 génère une représentation visuelle des droites détectées par la méthode de la Transformée de Hough. Il reçoit en entrée une image (*image*), une liste de paramètres de droites détectées (*lignes*) générée par l'algorithme 13, et une couleur (*colors*). L'algorithme parcourt la liste des paramètres de droites détectées et reconstruit chaque droite correspondante sur l'image. Les droites reconstruites sont colorées avec la couleur *colors*. Cette reconstruction permet de visualiser les droites détectées sur l'image d'origine, facilitant ainsi l'interprétation et l'analyse des résultats de la Trans-

formée de Hough.

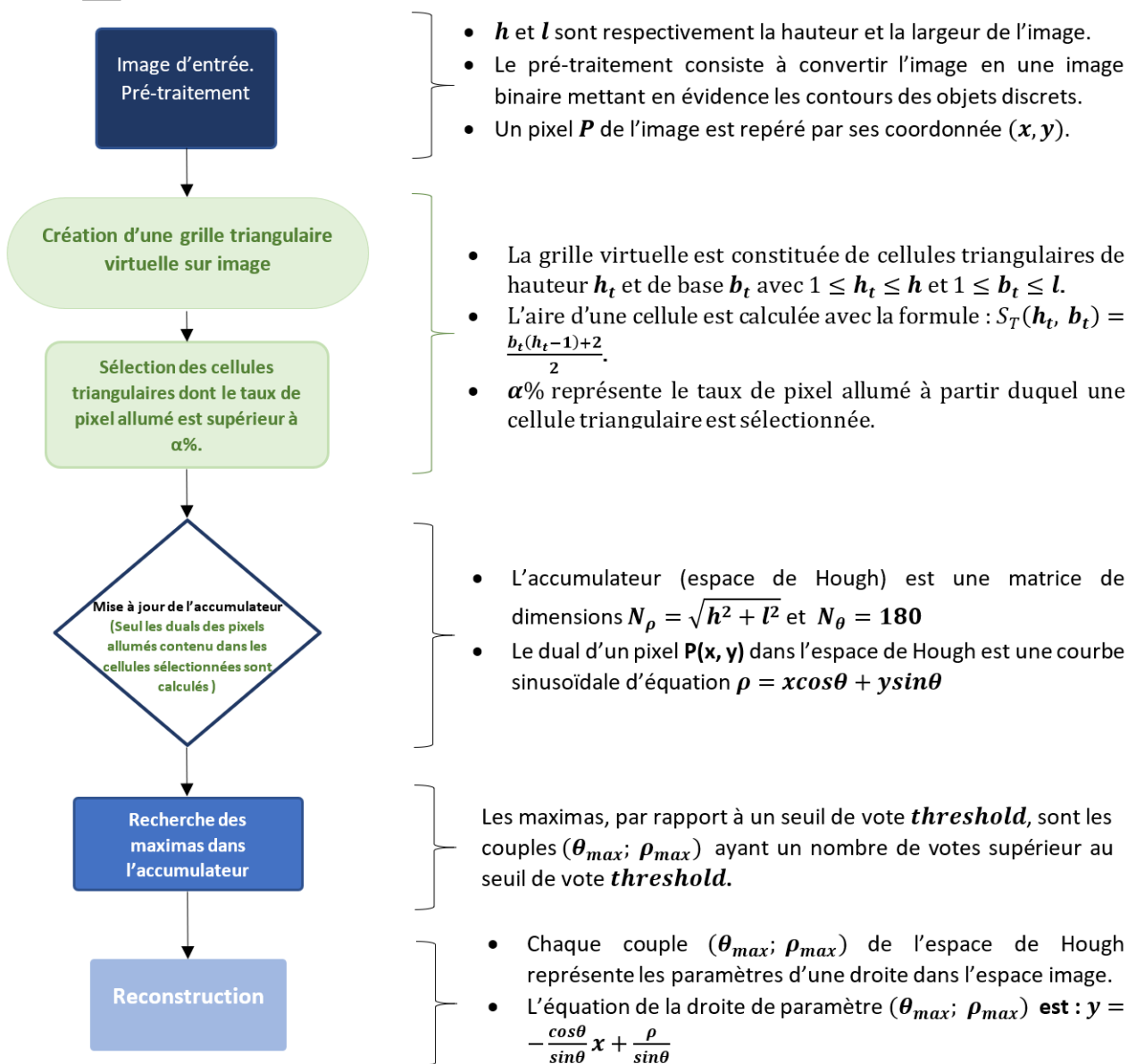


FIGURE 3.4 – Les étapes de la technique de la transformée de Hough triangulaire

Algorithme 16 : Reconstruction des droites détectées

Fonction *PlotHoughLine*(*image*, *lignes*, *colors*) : *image*

Parcourir '*lignes*' la liste des paramètres des droites détectés, puis construire chaque droite correspondant en couleur '*colors*' sur l'image '*image*'.

fin

2 Complexité de la transformée de Hough triangulaire

La fonction *maillage_tri* dans l'algorithme 10 initialise un tableau et lui assigne des valeurs, puis retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

La fonction *count_tri* dans l'algorithme 12 : initialise des variables k , l et D , puis effectue des tests de conditions pour déterminer l'orientation du triangle. Toutes ces opérations sont de coût constant, donc $O(1)$. Ensuite, elle parcourt les pixels à l'intérieur du triangle à l'aide de boucles dont le nombre d'itérations est $\frac{b(h-1)+2}{2}$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité d'un bloc formé par les boucles est $O(\frac{b(h-1)+2}{2}) = O(b \times h)$, où $\frac{b(h-1)+2}{2}$ est le nombre de pixels à l'intérieur du triangle. Ainsi, la complexité totale de la fonction *count_tri* est $O(b \times h)$.

La fonction *Acc_update* dans l'algorithme 11 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de *accum_row* = 180, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction *Acc_update* est $O(1)$.

L'algorithme 13 de la transformée de Hough triangulaire commence par l'initialisation des variables et du tableau *tab*, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction *maillage_tri2* dont la complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux boucles imbriquées parcourant les points à l'intérieur de chaque triangle à l'intérieur duquel se trouve des opérations de coût constant et la fonction *Acc_update* de complexité $O(1)$. La complexité de ces boucles est donc $O(\frac{b(h-1)+2}{2}) = O(b \times h)$, où $\frac{b(h-1)+2}{2}$ est le nombre de pixels à l'intérieur du triangle. Le coût des deux boucles principales imbriquées parcourant les triangles de la grille, dont le nombre total d'itérations dépend de la taille de l'image et de la taille des cellules triangulaires, contenant un test de condition de coût constant, un deuxième test de condition sur la fonction *count_tri2* de complexité $O(b \times h)$, deux boucles imbriquées de coût $\frac{b(h-1)+2}{2}$ parcourant les pixels à l'intérieur du triangle, est $Nl/h \times Nc/b \times (1 + 1 + 1 + (\frac{b(h-1)+2}{2}))$, donc de complexité $O(Nl/h \times Nc/b \times (1 + 1 + 1 + (\frac{b(h-1)+2}{2}))) = O(Nl \times Nc)$. Aussi la complexité des deux boucles parcourant les résidus dans les dimensions de l'image est $O(Nc)$ et $O(Nl)$ respectivement.

En considérant ces points, la complexité totale de l'algorithme 13 de la THT est égal à $O(Nl \times Nc) + O(Nc) + O(Nl) = O(Nl \times Nc)$, où Nl et Nc sont le nombre de lignes et de colonnes de l'image respectivement, h et b sont les dimensions des cellules triangulaire. Ce qui correspond à la complexité de la THS.

3 Simulations et discussions

Cette section offre une analyse détaillée des résultats obtenus à partir des simulations menées sur les algorithmes de la transformée de Hough triangulaire appliqués à deux images : l'image d'un bâtiment et l'image d'une autoroute représenté dans la figure 1.11. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Ces simulations ont été réalisées en variant les paramètres clés tels que le seuil de vote, la taille S_T des cellules triangulaires, et le paramètre α . Les résultats obtenus ont été examinés afin de déterminer l'impact de ces paramètres sur la performance de l'algorithme en termes de détection de lignes droites. De plus, une comparaison entre la transformée de Hough triangulaire et sa version parallélisée (THTP) a été effectuée pour évaluer l'algorithme parallèle. Cette section offre ainsi une compréhension des performances et des limitations de la transformée de Hough triangulaire dans les contextes de l'image de l'immeuble et de l'autoroute, ainsi que des perspectives pour d'éventuelles améliorations.

3.1 Cas de l'image de l'immeuble

Dans cette section, nous explorons l'application de la transformée de Hough triangulaire à le cas de la détection de lignes dans l'image d'un immeuble. Nous étudions les performances de la transformée de Hough triangulaire en ajustant différents paramètres, tels que le pourcentage α de pixels allumés et la taille des cellules rectangulaires.

3.1.0.1 Variation de α

Le tableau 3.1 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble en utilisant l'algorithme 13 de la transformée de Hough triangulaire. Les simulations ont été réalisées avec les paramètres fixes ($h = 5$, $b = 3$ et $Seuil = 60$) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 55%. Ce paramètre définit la taille de la cellule triangulaire. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse des résultats de simulation sur l'image de l'immeuble, en utilisant l'algorithme de la transformée de Hough triangulaire, met en lumière l'influence du paramètre α sur la performance de l'algorithme :

- Tout d'abord, en examinant le nombre de lignes détectées, on constate qu'il diminue à mesure que α augmente. En effet, de 388 droites détectées avec $\alpha = 10\%$, ce nombre a diminué jusqu'à 0 avec $\alpha = 55\%$. Des valeurs trop élevées de α peuvent entraîner une détection nulle.

- En ce qui concerne le temps de traitement, il y a une amélioration. Le temps de calcul reste relativement constant pour α compris entre 0,1 et 0,45, puis diminue à $\alpha = 50\%$ et $\alpha = 55\%$.

TABLEAU 3.1 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres fixes ($h = 5$, $b = 3$ et $Seuil = 60$) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

$\alpha(\%)$	Nbre de droites détectées	temps(sec)
10	388	0,49
15	154	0,44
20	154	0,44
25	154	0,43
30	28	0,33
35	28	0,34
40	9	0,27
45	4	0,21
50	4	0,21
55	0	0,16

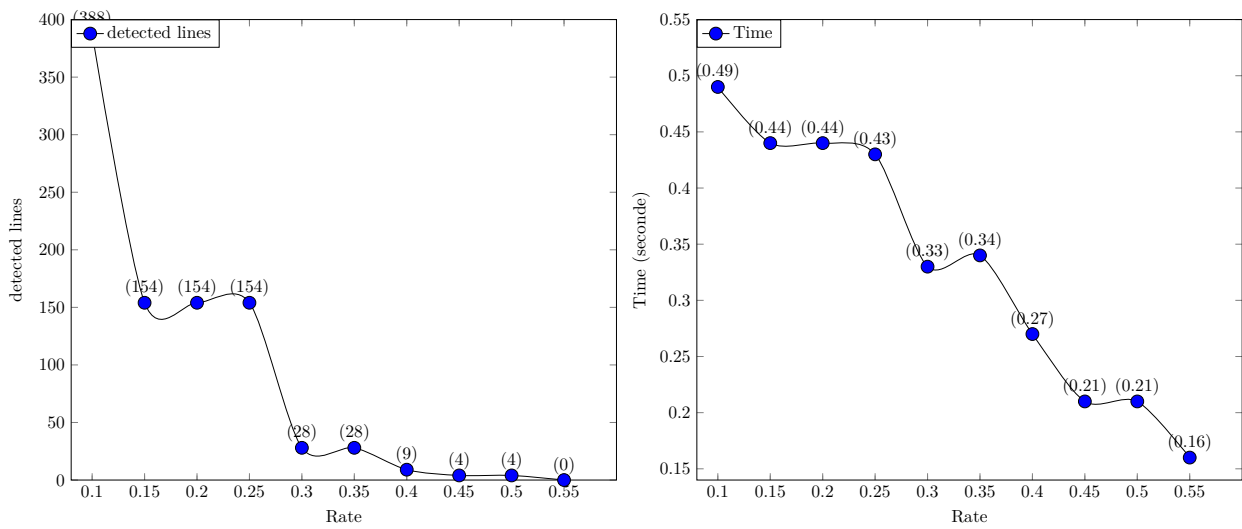


FIGURE 3.5 – Représentation graphique des données du tableau 3.1

La représentation graphique des données du tableau représenté dans la figure 3.5 confirme ces observations, montrant la décroissance du nombre de lignes détectées à mesure que α augmente, et une baisse du temps de calcul pour les valeurs de α plus élevées.

3.1.0.2 Variation de la taille des cellules triangulaires

Le tableau 3.2 et les graphiques associés, illustrés dans la Figure 3.7, présentent les résultats de simulations utilisant l'Algorithme 13 de la transformée de Hough triangulaire sur l'image 1.11(c) de l'immeuble. L'algorithme est appliqué avec des paramètres fixes ($\alpha = 20\%$ et $Seuil = 80$) tout en faisant varier les paramètres h et b , représentant respectivement la hauteur et la base de la cellule triangulaire.

Dans l'analyse du tableau 3.2, on observe que les valeurs de h et b augmentent progressivement, entraînant une augmentation correspondante de l'aire S_T des cellules triangulaires. Cette croissance est attendue, car h et b définissent la taille de la cellule triangulaire. A partir de $S_T = 4$ jusqu'à $S_T = 29$, le nombre de lignes détectées diminue à mesure que la taille de la cellule augmente. Pour



FIGURE 3.6 – (a) THT sur l'immeuble ($h = 219, b = 178, \alpha = 48, seuil = 61$), (b) THT sur l'autoroute ($h = 92, b = 300, \alpha = 10, seuil = 60$)

TABLEAU 3.2 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres fixes ($\alpha = 20\%$ et $Seuil = 80$), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$, où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

h	b	$S_T(h, b)$	Nbre de droites détectées	temps(sec)
2	4	4	221	0,59
3	5	7	105	0,53
4	6	11	85	0,52
5	7	16	49	0,48
6	8	22	39	0,41
7	9	29	29	0,40
131	117	7606	59	0,36

	h	b	$S_T(h, b)$	Nbre de droites détectées	temps(sec)
$seuil = 61, \alpha = 48$	2	163	800	42	0,28
	2	187	95	25	0,25
	219	178	19403	38	0,26

$S_T = 7606$, 59 droites ont été détecté contre 29 pour $S_T = 29$ l'aire d'une plus petite cellule. Ainsi, de façon générale, le nombre de droites détectées n'a pas de tendance spécifique par rapport à la taille de cellules. En ce qui concerne le temps de traitement, il diminue lorsque la taille des cellules augmente.

3.2 Cas de l'image de l'autoroute

Dans cette section, nous examinons les performances de l'algorithme de détection de lignes de la transformée de Hough triangulaire appliqué à une image représentant une autoroute. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir h , b et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la transformée de Hough triangulaire en fonction des caractéristiques de l'image de l'autoroute.

3.2.0.1 Variation de α Le tableau 3.3 présente les résultats de simulations utilisant l'algorithme 13 de la transformée de Hough triangulaire sur l'image 1.11(a) de l'autoroute. Les paramètres de l'algorithme sont fixés à $h = 4$, $b = 2$, et $Seuil = 60$, tandis que le paramètre α varie entre 10%

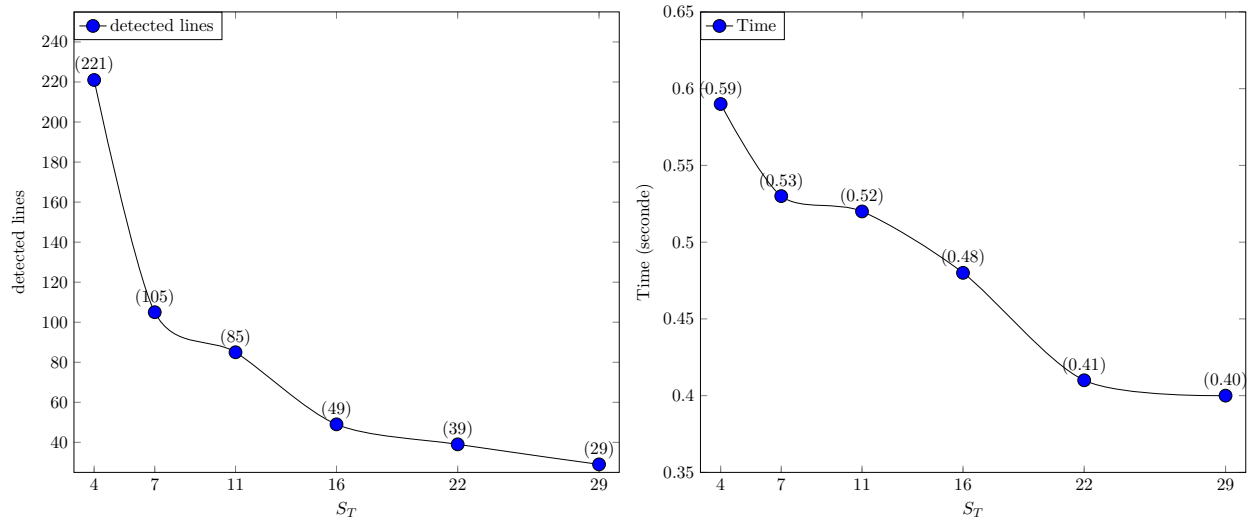


FIGURE 3.7 – Représentation graphique des données du tableau 3.2

TABLEAU 3.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres fixes ($h = 4$, $b = 2$ et $Seuil = 60$) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

$\alpha(\%)$	Droites détectées	temps(sec)
10	80	0,24
15	80	0.24
20	80	0.24
25	51	0.22
30	18	0,19
35	18	0.19
40	18	0,19
45	8	0.16
50	8	0,16
55	2	0.12

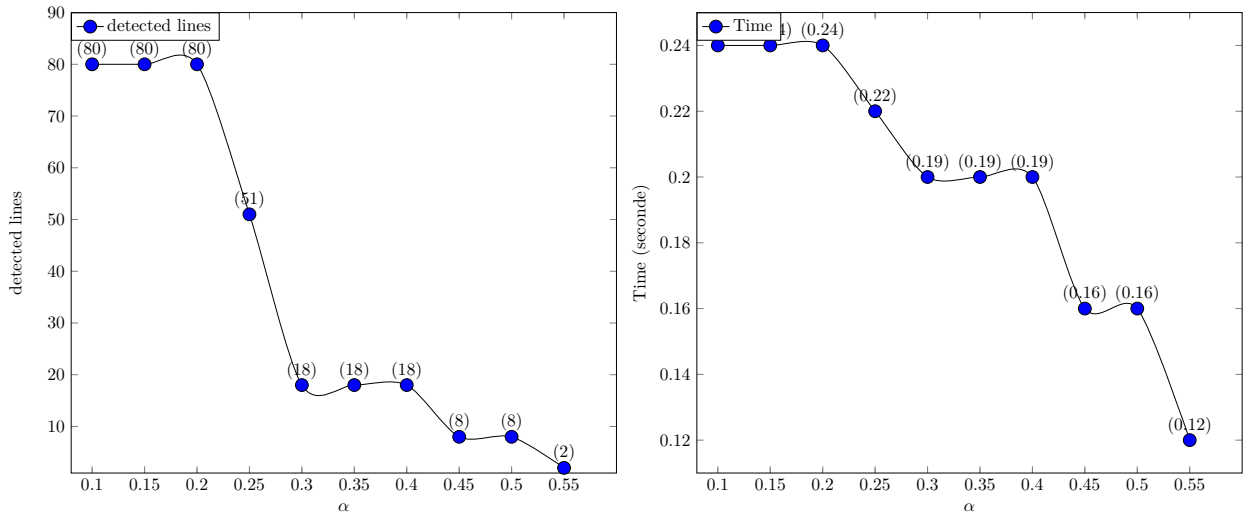


FIGURE 3.8 – Représentation graphique des données du tableau 3.3

et 55%. Les colonnes indiquent le nombre de lignes détectées et le temps de traitement pour chaque valeur de α .

Les résultats indiquent que, pour des valeurs de α comprises entre 10% et 20%, le nombre de lignes détectées reste constant à 80, et le temps de calcul associé est également stable à 0.24 seconde. À partir de $\alpha = 25\%$, on observe une diminution progressive du nombre de lignes détectées, atteignant 2 à $\alpha = 55\%$. Le temps de calcul diminue légèrement à mesure que α augmente.

L'analyse des graphiques illustrés dans la figure 3.8 montre que jusqu'à $\alpha = 20\%$, la performance de l'algorithme est constante. Au-delà de cette valeur, le nombre de lignes détectées diminue progressivement, et à $\alpha = 55\%$, seulement 2 lignes sont détectées. Cependant, le temps de calcul diminue légèrement avec l'augmentation de α .

Ainsi, ces résultats suggèrent que l'algorithme de détection de lignes est sensible au paramètre α . Des valeurs plus élevées de α conduisent à une réduction du nombre de lignes détectées mais également à une diminution du temps de traitement, ce qui peut être bénéfique pour des applications nécessitant une exécution rapide de l'algorithme. Cependant, des valeurs trop élevées de α peuvent entraîner de détections nulles

3.2.0.2 Variation de la taille des cellules triangulaire Le tableau 3.4 présente les résultats de simulations utilisant l'algorithme 13 de la transformée de Hough triangulaire sur l'image 1.11(a) de l'autoroute. Les paramètres de l'algorithme sont fixés avec $\alpha = 20\%$ et $Seuil = 60$, tandis que $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$. Les colonnes indiquent la hauteur h , la base b , la surface S_T , le nombre de lignes détectées, et le temps de traitement pour chaque configuration de paramètres. L'analyse du tableau 3.4, dont les graphiques sont représentés dans la figure 3.4, révèle des variations significatives du nombre de lignes détectées et du temps de traitement en fonction des valeurs de h et b . À mesure que la taille de la cellule triangulaire augmente, allant de $4px_2$ à $29px_2$, le nombre de lignes détectées

TABLEAU 3.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres fixes ($\alpha = 20\%$ et $Seuil = 60$), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$, où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

h	b	S_T	Droites détectées	temps(sec)
2	4	4	165	0,25
3	5	7	96	0,23
4	6	11	84	0,22
5	7	16	50	0,20
6	8	22	15	0,17
7	9	29	17	0,16

h	b	S_T	Droites détectées	temps(sec)
(pour $\alpha = 10$)				
168	151	12 610	29	0.15
92	300	13 651	8	0.13

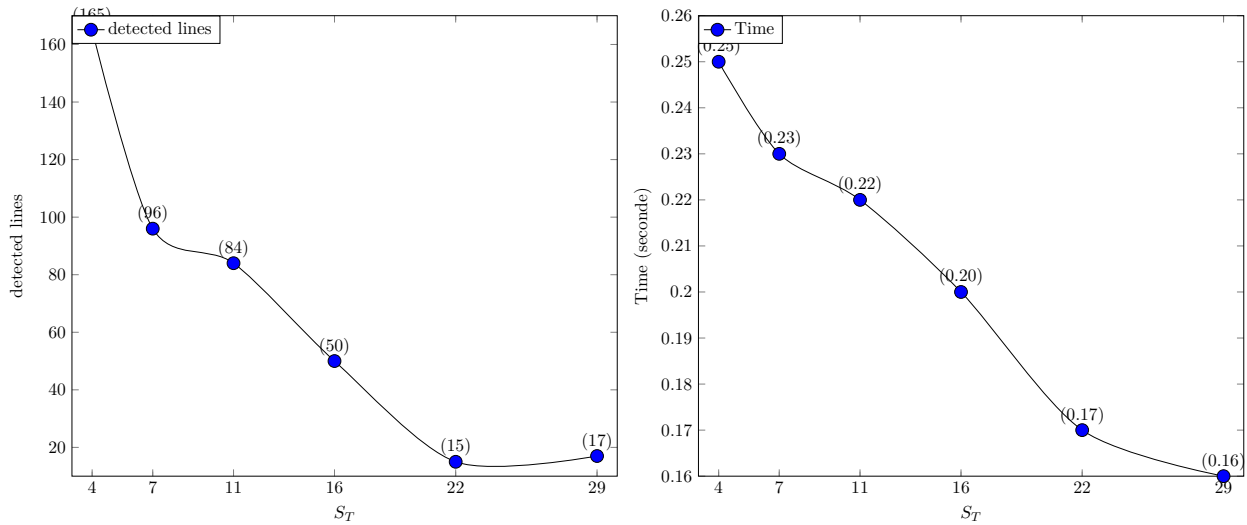


FIGURE 3.9 – Représentation graphique des données du tableau 3.4

diminue, passant de 165 droites à 15 droites. Pour $S_T = 29$ le nombre de droites détectées est monté à 29. Ainsi, la tendance observée précédemment n'est pas générale. Le temps de traitement diminue à mesure que la taille de la cellule augmente, indiquant un temps de traitement réduit avec des cellules triangulaires plus grandes.

Ainsi, ces résultats mettent en évidence l'importance des paramètres h et b dans l'algorithme de la transformée de Hough triangulaire. Les cellules de grandes taille offre une précision acceptable et un temps de traitement plus rapide, en témoignent les données des tableaux 3.2 et 3.4, ainsi que les résultats visuels dans la figure 3.6.

4 Analyse comparative

Dans cette section, nous comparons les performances de deux méthodes de détection de lignes droites : la transformée de Hough triangulaire et sa version standard. Nous appliquons ces deux méthodes sur deux images différentes : une image d'un immeuble et une image d'une autoroute. Les

TABLEAU 3.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)

Seuil	Droites détectées	temps1(sec)	temps2(sec)
38	11271	0,55	0,000049
116	10	0,49	0,049

TABLEAU 3.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites; time2=temps de détection d'une seule droite)

Seuil	Droites détectées	temps1(sec)	temps2(sec)
57	99	0,23	0,0023
100	10	0,21	0,021

résultats de ces comparaisons sont illustrés et analysés en termes de précision de détection et de temps de traitement, fournissant ainsi une évaluation complète de l'efficacité et de la rapidité des algorithmes. De plus, nous examinerons les performances de la transformée de Hough triangulaire par rapport à sa version parallélisée (THTP). Nous comparons également la performance temporelle et la précision de la Transformée de Hough Rectangulaire et de la transformée de Hough triangulaire. Ces analyses permettront de démontrer les avantages et les inconvénients de chaque méthode dans différents contextes d'application.

4.1 Transformée de Hough Triangulaire et Transformée de Hough Standard

Nous procédons dans cette section à une comparaison de la transformée de Hough triangulaire, méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.



FIGURE 3.10 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40); (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough triangulaire sont illustrés dans les tableaux 3.5 et 3.7 respectivement.

En ce qui concerne la précision, pour un seuil de 38 votes, la THS a effectué une détection de 11271 droites. Cette détection est non pertinente comme le montre la première image de la figure 3.10. par contre, avec le même seuil de votes, la THT a détecté 11 droites. Les résultats visuels illustrés par la deuxième image de la figure 3.12 montre que les droites détectées suivent bien les lignes de l'immeuble suggérant des détections pertinentes.



FIGURE 3.11 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

TABLEAU 3.7 – Résultats de simulation avec la THT sur l'image de l'immeuble (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

(h, b)	S_T	$\alpha(\%)$	seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
(5,3)	7	45	38	22	0.21	0.0095
(7,5)	16	40	38	11	0.17	0.015

Les paramètres supplémentaires utilisés par la transformée de Hough triangulaire pour parvenir à ce résultat sont : les dimensions $(h; b)$ et le pourcentage α de pixels allumés de chaque cellule triangulaire. Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 3.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 3.10 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'immeuble indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 3.5 montre que la transformée de Hough standard à détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La transformée de Hough triangulaire a détecté quasiment le même nombre de droite en 0.17 seconde soit 0.015 seconde par droite détecté comme illustré dans le tableau 3.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,49}{0,17} = 2,88$ (T_h représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).



FIGURE 3.12 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=38, $\alpha = 45\%$, $h = 5$ et $b = 3$; (b) Seuil=38, $\alpha = 40\%$, $h = 7$ et $b = 5$

De même avec l'image de l'autoroute, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough triangulaire sont illustré dans les tableaux 3.6 et 3.8 respectivement.

En ce qui concerne la précision, pour un seuil de 57 votes, la THS a effectué une détection de 99

TABLEAU 3.8 – Résultats de simulation avec la THT sur l'image de l'autoroute (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)

(h, b)	S_T	$\alpha(\%)$	Seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
(5,3)	7	30	57	11	0.16	0.014
(7,5)	16	30	35	25	0.12	0.0048

droites. Cette détection est non pertinente comme le montre la première image de la figure 3.11. par contre, avec le même seuil de votes, la THT a détecté 11 droites. Les résultats visuels illustrés par la première image de la figure 3.13 montre que les droites détectées suivent bien les lignes de l'autoroute suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough triangulaire pour parvenir à ce résultat sont : les dimensions $(h; b)$ et le pourcentage α de pixels allumés de chaque cellule triangulaire. Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 3.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 3.11 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 3.6 montre que la transformée de Hough standard a détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La transformée de Hough triangulaire a détecté quasiment le même nombre de droite en 0.16 seconde soit 0.014 seconde par droite détecté comme illustré dans le tableau 3.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,21}{0,16} = 1,31$ (T_h représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).



FIGURE 3.13 – Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=57, $\alpha = 30\%$, $h = 5$ et $b = 3$; (b) Seuil=35, $\alpha = 30\%$, $h = 7$ et $b = 5$

4.2 Transformée de Hough Triangulaire et sa version parallélisée

4.2.1 Cas de l'image de l'immeuble

La figure 3.14 présente deux graphiques comparant les performances des algorithmes de transformée de Hough triangulaire et de transformée de Hough triangulaire parallélisée (THTP). Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et S_T).

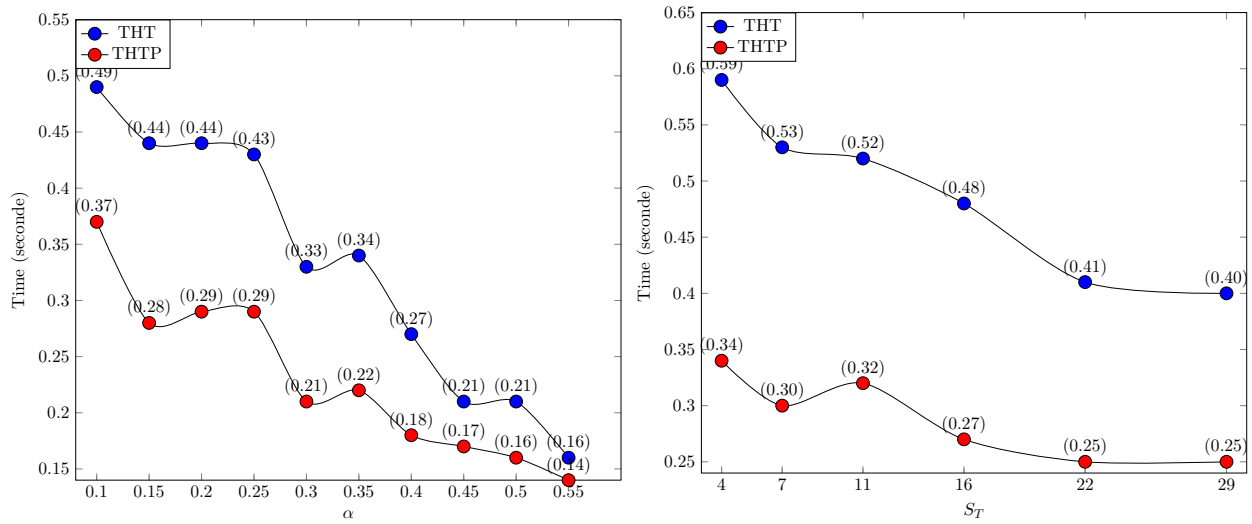


FIGURE 3.14 – Cas de l’immeuble : (à gauche) Comparaison des variations du temps d’exécution en fonction de α et les paramètres ($h = 3$, $b = 5$ et $Seuil = 60$), (à droite) Comparaison des variations du temps d’exécution en fonction de la surface des cellules triangulaires et les paramètres ($Seuil = 80$, $\alpha = 20\%$) des l’algorithmes 13 et 15

Dans le premier graphique (à gauche), les variations du temps d’exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 55%. Pour toutes ces valeurs de α , la courbe de la transformée de Hough triangulaire parallélisée est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire parallélisée est plus rapide que la transformée de Hough triangulaire.

Dans le deuxième graphique (à droite), les variations du temps d’exécution sont comparées en fonction de la surface des cellules rectangulaires, avec les deux algorithmes de la THT et la THTP. Les surfaces des cellules varient de 4 à 29 px^2 . Pour toutes ces valeurs, la courbe de la transformée de Hough triangulaire parallélisée est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire parallélisée est plus rapide que la transformée de Hough triangulaire.

4.2.2 Cas de l’image de l’autoroute

La figure 3.15 présente deux graphiques comparant les performances des algorithmes de transformée de Hough triangulaire et de transformée de Hough triangulaire parallélisée (THTP). Les graphiques comparent les variations du temps d’exécution en fonction de différents paramètres (α et S_T). Dans le premier graphique (à gauche), les variations du temps d’exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 55%. Pour toutes ces valeurs de α , la courbe de la transformée de Hough triangulaire parallélisée est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire est moins rapide que sa version parallélisée.

Dans le deuxième graphique (à droite), les variations du temps d’exécution sont comparées en

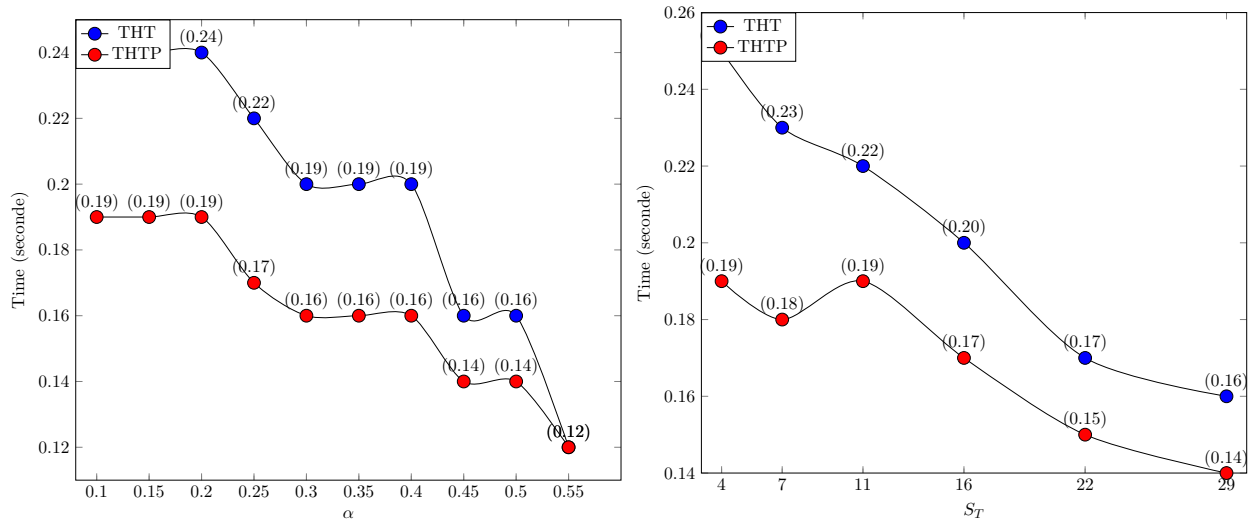


FIGURE 3.15 – Cas de l’autoroute : (à gauche) Comparaison des variations du temps d’exécution en fonction de α et les paramètres ($h = 4$, $b = 2$ et $Seuil = 60$), (à droite) Comparaison des variations du temps d’exécution en fonction de la surface des cellules triangulaires et les paramètres ($Seuil = 60$, $\alpha = 20\%$) des l’algorithmes 13 et 15

fonction de la surface des cellules triangulaires, avec les deux algorithmes de la THT et la THTP. Les surfaces des cellules varient de 4 à 29 px^2 . Pour toutes ces valeurs, la courbe de la transformée de Hough triangulaire est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire parallélisée est plus rapide que la transformée de Hough triangulaire.

Ainsi, l’analyse des deux graphiques met en lumière les performances relatives de la transformée de Hough triangulaire et sa version parallélisée en fonction des paramètres : α et la taille des cellules triangulaires. Pour les petites valeurs de α , la transformée de Hough triangulaire parallélisée montre des temps d’exécution plus rapides par rapport à THT. Cependant, cette supériorité diminue lorsque α devient plus élevé. De plus, en ce qui concerne la taille des cellules triangulaire, la transformée de Hough triangulaire parallélisée est plus performante tant pour les cellules de petites tailles que pour les cellules de taille plus grande. De plus la parallélisation n’a pas affecté la précision de la détection. La version parallélisé de la transformée de Hough triangulaire est donc la meilleur option pour une détection rapide dans le cas de l’image de l’immeuble et de l’autoroute. L’article [101] corobore ces résultats.

4.3 Transformée de Hough Rectangulaire et Transformée de Hough Triangulaire

Dans cette section, nous entreprenons une analyse comparative de la transformée de Hough rectangulaire et de la transformée de Hough triangulaire, examinant leurs méthodologies respectives, leurs complexités computationnelles et leurs performances en matière de détection de lignes droites. Cela permettra de mettre en lumière les avantages et les limitations de chaque approche, fournissant des indications pour choisir la méthode la plus adaptée à des applications spécifiques.

4.3.1 Performance temporelle

Dans cette sous-section, nous examinons en détails les temps de traitement associés à la transformée de Hough rectangulaire et à la transformée de Hough triangulaire pour la détection de lignes dans des images d'immeubles et d'autoroutes. Nous présentons les résultats de nos simulations, mettant en évidence les différences de temps de détection entre les deux méthodes. Cette analyse nous permettra de déterminer la méthode la plus efficace en termes de vitesse de traitement pour chaque scénario d'imagerie spécifique.

4.3.1.1 Cas de l'image de l'immeuble Le tableau 2.7 contient les résultats des simulations de la transformée de Hough rectangulaire appliquée à l'image de l'immeuble. Au niveau de la deuxième ligne, 11 lignes droites ont été détectées en 0.15 secondes ou 0.013 secondes par ligne, avec les paramètres $(l, L) = (5, 7)$, $S_R = 35$, $\alpha = 0.35$ et $Seuil = 35$ où $S_R(l, L)$ représente la surface du rectangle de largeur l et de longueur L .

Le tableau 3.7 contient les résultats des simulations de la transformée de Hough triangulaire appliquée à l'image de l'immeuble. Au niveau de la deuxième ligne, 11 lignes droites ont également été détectées, mais avec un temps de détection relatif élevé. Le temps de détection est de 0.17 secondes ou 0.015 secondes par ligne, avec les paramètres $(h, b) = (7, 5)$, $S_T = 16$, $\alpha = 0.4$ et $Seuil = 38$ où $S_T(h, b)$ représente l'aire du rectangle de hauteur h et de base b .

Le temps de détection est relativement faible dans les deux cas, ce qui indique que la THR et la THT sont efficaces pour détecter les lignes droites dans les images.

Les résultats des deux tableaux (tableau 2.7, tableau 3.7) révèlent que la transformée de Hough rectangulaire est plus rapide que la transformée de Hough triangulaire pour détecter les lignes droites dans l'image de l'immeuble.

4.3.1.2 Cas de l'image de l'autoroute Le tableau 2.8 contient les résultats des simulations de la transformée de Hough rectangulaire appliquée à l'image de l'autoroute. Au niveau de la première ligne, 11 lignes droites ont été détectées en 0.11 secondes ou 0.01 secondes par ligne, avec les paramètres $(l, L) = (3, 5)$, $S_R = 15$, $\alpha = 0.4$ et $Seuil = 51$ où $S_R(l, L)$ représente l'aire du rectangle de largeur l et de longueur L .

Le tableau 3.8 contient les résultats des simulations de la transformée de Hough triangulaire appliquée à l'image de l'autoroute. Au niveau de la première ligne, 11 lignes droites ont également été détectées, mais avec un temps de détection relatif élevé. 0,16 secondes ou 0,014 secondes par ligne, avec les paramètres $(h, b) = (5, 3)$, $S_T = 7$, $\alpha = 0.3$ et $Seuil = 57$ où $S_T(h, b)$ représente l'aire du rectangle de hauteur h et de base b .

Le temps de détection est relativement faible dans les deux cas, ce qui indique que THR et THT sont efficaces pour détecter les lignes droites dans les images.

Les résultats des deux tableaux montrent que la transformée de Hough rectangulaire est plus rapide que la transformée de Hough triangulaire pour détecter les lignes droites dans l'image de la route.

En résumé, les deux méthodes de détection de lignes ont démontré des performances dans les deux scénarios d'imagerie, avec des résultats comparables en termes de précision et de vitesse. Cependant, la méthode de la transformée de Hough rectangulaire a montré un léger avantage en termes de temps de traitement dans les deux cas, ce qui suggère son application potentielle dans des environnements où la vitesse de traitement est cruciale.

4.3.2 Précision

La précision est un aspect crucial dans l'évaluation des performances des algorithmes de détection de lignes, car elle détermine la fiabilité des résultats obtenus. Dans cette sous-section, nous examinons attentivement la précision de deux méthodes de détection de lignes : la transformée de Hough rectangulaire et la transformée de Hough triangulaire, appliquées à des images d'immeubles et d'autoroutes. Nous analysons les résultats visuels des deux méthodes, en évaluant la pertinence des lignes détectées par rapport aux contours réels des objets dans les images.

4.3.2.1 Cas de l'image de l'immeuble Examinons la première image de la figure 3.16, la sortie visuelle de l'algorithme de la transformée de Hough rectangulaire. Il s'agit de la sortie visuelle des résultats du tableau 2.7. Les 11 lignes détectées sont toutes pertinentes. Aucune ligne indésirable n'a été détectée. Il est clair que 4 arêtes de bâtiment ont été détectées. Cela signifie que la même arête a été détectée plusieurs fois. Les lignes détectées sont toutes obliques par rapport au plan de l'image et correspondent aux lignes horizontales de l'immeuble. Aucune ligne verticale de l'immeuble n'a été détectée.



FIGURE 3.16 – Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=35, $\alpha = 0.35$, $L = 7$ et $l = 5$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.4$, $h = 7$ et $b = 5$.

Il en va de même pour la deuxième image de la figure 3.16, le résultat de l'algorithme de la THT. Il s'agit de la sortie visuelle des données du tableau 3.7. Comme pour l'algorithme de la THR, les 11 lignes détectées sont toutes pertinentes (mais ce ne sont pas les mêmes). Aucune ligne indésirable n'a été détectée. Il est clair que 4 arêtes du bâtiment ont été détectées. Cela implique que les arêtes ont

été détectées plusieurs fois. Les lignes détectées sont toutes obliques par rapport au plan de l'image et correspondent aux lignes horizontales du bâtiment. Aucune ligne verticale du bâtiment n'a été détectée.

Remarque 4 *Les paramètres peuvent être affinés en fonction des besoins de l'application afin d'obtenir un rendu visuel plus efficace et plus précis. Par exemple, la première image de la figure 3.17, résultant de la transformée de Hough rectangulaire, illustre la détection de lignes horizontales et verticales avec les paramètres : $\text{Seuil}=90$, $\alpha = 0.1$, $L = 5$ et $l = 3$. Il en va de même pour la seconde image, résultant de la transformée de Hough triangulaire, avec les paramètres : $\text{Seuil}=38$, $\alpha = 0.45$, $h = 5$ et $b = 3$.*



FIGURE 3.17 – Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : $\text{Seuil}=90$, $\alpha = 0.1$, $L = 5$ et $l = 3$; (b) la THT avec les paramètres : $\text{Seuil}=38$, $\alpha = 0.45$, $h = 5$ et $b = 3$.

4.3.2.2 Cas de l'image de l'autoroute Examinons la première image de la figure 3.18, qui représente le rendu visuel de l'algorithme de la THR. Cette illustration est la sortie graphique des résultats énumérés dans le tableau 2.8. Dans ce contexte, les 11 lignes détectées sont toutes pertinentes, sans aucune ligne superflue. Il est à noter que quatre bords de la route ont été identifiés, ce qui suggère que la même détection a pu être faite plusieurs fois. Ce qui est intéressant, c'est que les bords délimitant la route ont été clairement identifiés. De même, la deuxième image de la figure 3.18 résulte



FIGURE 3.18 – Détection de lignes droites sur l'image 1.11 de l'autoroute avec, (a) le THR avec les paramètres : $\text{Seuil}=51$, $\alpha = 0.4$, $L = 5$ et $l = 3$; (b) le THT avec les paramètres : $\text{Seuil}=57$, $\alpha = 0.3$, $h = 5$ et $b = 3$.

de l'application de l'algorithme de la THT. Cette représentation graphique correspond à la sortie visuelle des données présentées dans le tableau 3.8. Comme dans le cas de l'algorithme de la THR,

les 11 lignes détectées sont pertinentes, bien qu'elles ne correspondent pas nécessairement aux mêmes lignes. Une ligne superflue est identifiée. Cette fois, cinq bords de la route sont clairement détectés, ce qui suggère la possibilité d'une détection répétée de certains éléments.

Dans cette analyse comparative de la précision entre la transformée de Hough rectangulaire et la transformée de Hough triangulaire, nous avons examiné les résultats visuels des deux méthodes appliquées à des images d'immeubles et d'autoroutes. Dans chaque cas, les lignes détectées ont été évaluées par rapport aux contours réels des objets dans les images.

Les résultats obtenus ont montré que dans l'ensemble, les lignes détectées par les deux méthodes étaient pertinentes, avec peu ou pas de lignes superflues. Cependant, des cas de détections répétées de certaines arêtes ont été observés.. Cela souligne l'importance de l'ajustement précis des paramètres de chaque méthode pour optimiser la pertinence des détections.

Conclusion

la transformée de Hough triangulaire offre une approche novatrice pour la détection de lignes droites dans les images. En exploitant le maillage triangulaire de l'image, la transformée de Hough triangulaire permet une détection acceptable des lignes droites, tout en réduisant le temps de traitement, et ayant une complexité comparable à celle de la Transformée de Hough Standard.

Au cours de cette étude, nous avons examiné la méthode de la transformée de Hough triangulaire, en mettant en lumière ses différentes étapes, de la génération de la grille triangulaire virtuelle à la reconnaissance des droites discrètes. Nous avons également réalisé des simulations sur deux images pour évaluer les performances de la transformée de Hough triangulaire, en comparant ses résultats avec ceux de la transformée de Hough standard. L'étude de la complexité temporelle de l'algorithme de la transformée de Hough triangulaire a révélé que la transformée de Hough triangulaire a la même complexité que la transformée de Hough standard.

Après ajustement des paramètres de la transformée de Hough triangulaire, nous avons pu observer une amélioration de la vitesse de détection par rapport à la THS pour les faibles seuils de vote, tout en maintenant une précision raisonnable. De plus, avec la réduction du nombre de pixels à calculer grâce à la variable α , la transformée de Hough triangulaire offre des économies computationnelles importantes. Les cellules de grandes tailles offre une précision acceptable et un temps de traitement plus rapide. L'analyse comparative entre la transformée de Hough triangulaire et sa version parallélisée THTP révèle que la THTP est plus rapide pour des petites valeurs de seuil α , offrant des temps d'exécution plus courts. Cependant, cette efficacité diminue avec l'augmentation de α . Ainsi, la version parallélisée de la transformée de Hough triangulaire est recommandée pour une détection rapide, notamment dans les cas des images d'immeuble et d'autoroute.

L'analyse comparative de la transformée de Hough rectangulaire et de la transformée de Hough

triangulaire a mis en lumière plusieurs constats importants. Tout d'abord, en ce qui concerne les performances temporelles, nous avons constaté que la transformée de Hough rectangulaire a tendance à être plus rapide que la transformée de Hough triangulaire dans la détection de lignes. En ce qui concerne la précision, nos résultats ont montré que les deux méthodes ont produit des résultats acceptables. Cependant, la transformée de Hough rectangulaire est plus flexible que la transformée de Hough triangulaire en ce sens qu'elle offre une capacité de détection ciblée des droites verticales et horizontales.

Cependant, malgré ses avantages, la transformée de Hough triangulaire présente également certaines limites, notamment en termes de sensibilité aux paramètres (α et (b, h)), de complexité de mise en œuvre, de perte d'informations provenant de l'ensemble complet des pixels allumés des cellules triangulaires et la dépendance du contexte. Dans le chapitre suivant, nous aborderons la transformée de Hough hexagonale.

Transformée de Hough Hexagonale

Introduction	83
1 Description de la méthode	84
2 Complexité de la transformée de Hough hexagonale	90
3 Simulations et discussions	93
Conclusion	106

Introduction

La détection de lignes droites dans les images constitue une tâche fondamentale dans le domaine de la vision par ordinateur et de l'analyse d'images. Parmi les méthodes les plus couramment utilisées pour cette tâche, la Transformée de Hough (TH) est largement répandue pour sa capacité à détecter des motifs linéaires, même en présence de bruit.

Cependant, malgré sa robustesse, la méthode de la Transformée de Hough Standard (THS) présente des limitations, notamment en termes de temps de traitement et de sensibilité aux paramètres. Pour surmonter ces défis, une approche innovante a émergé : la Transformée de Hough Hexagonale (THH).

Dans cette étude, nous examinons en détail la méthode de la transformée de Hough hexagonale, explorant ses principes fondamentaux, ses mécanismes de fonctionnement et ses applications potentielles. Nous menons également des simulations pour évaluer les performances de la transformée de Hough hexagonale dans divers scénarios, en comparant ses résultats avec ceux de la méthode de la THS. Aussi, nous discutons des implications de nos résultats et des perspectives pour l'avenir de la détection de lignes dans les images. Enfin, nous examinons les performances comparatives de la transformée de Hough hexagonale et de sa version parallélisée (THHP) en fonction du taux α et de la taille des cellules hexagonales γ .

1 Description de la méthode

Dans cette section, nous présentons une approche novatrice de la Transformée de Hough standard appliquée sur une grille hexagonale virtuelle. Cette technique, dont les étapes sont décrites dans la figure 4.5, repose sur la subdivision de l'image en cellules hexagonales régulières, offrant ainsi une représentation alternative qui permet de réduire le temps de traitement et d'améliorer la détection des lignes. Nous décrivons en détail le processus de création de la grille hexagonale virtuelle, ainsi que l'algorithme de la THH.

En outre, nous discutons des techniques de sélection de cellules de la grille et de reconnaissance de droites discrètes. Enfin, nous présentons une méthode de visualisation des lignes détectées pour une interprétation des résultats.

1.1 Maillage Hexagonal de l'image

La conception d'une grille hexagonale virtuelle à l'aide du maillage de l'espace image consiste à diviser une image en cellules hexagonales et à positionner ces cellules de manière à couvrir l'image de manière cohérente. La grille hexagonale virtuelle est composée de cellules hexagonales ayant les mêmes caractéristiques. Il s'agit donc d'un maillage régulier comme illustré dans la figure 4.2.

Définition 12 (Hexagone) *Un hexagone est un polygone à six côtés.*

γ représente la taille de l'hexagone.

AB, BC, CD, DE, EF et FA sont les côtés de l'hexagone illustré dans la figure 4.1.

L'hexagone est subdivisé en trois parties comme illustré dans la figure 4.1(a) :

- le triangle isocèle ABF
- le rectangle BCEF
- le triangle isocèle CDE

Le théorème de Thalès est utilisé pour parcourir les pixels dans la zone triangulaire isocèle de chaque hexagone.

Le théorème de Thalès montre la proportionnalité entre les lignes parallèles. Par exemple, en utilisant l'équation $\left(\frac{AP}{AG} = \frac{NP}{BG}\right)$ du théorème, on peut trouver que $NP = \frac{AP \times BG}{AG}$. Si $BG = \gamma$ et $AG = \gamma$, alors $NP = AP$. Ainsi, $NP = PK$ car ABF est un triangle isocèle. De manière similaire, nous avons $MQ = QR = QD$ comme indiqué dans la figure 4.1(b).

L'algorithme 17 est utilisé pour la conception de la grille hexagonale virtuelle. Avec les paramètres d'entrée (γ et image), l'algorithme 17 renvoie un tableau de 3 entiers. La première valeur dans le

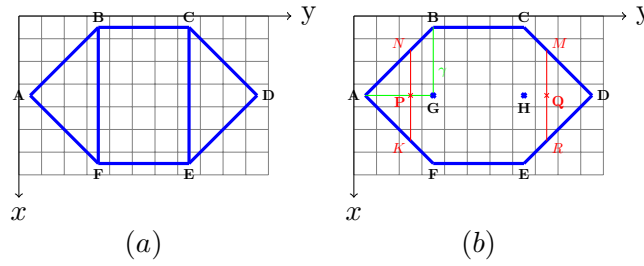


FIGURE 4.1 – Un hexagone ABCDEF

Algorithme 17 : Construction de la grille hexagonale

Fonction *maillage_hex(image, γ : entier) : tableau d'entiers*
 γ est la taille de l'hexagone;
Variables : tab : tableau of 3 entiers ;
Sorties : tab : tableau of 3 entiers ;
Debut
 $Nl \leftarrow$ la hauteur de l'image;
 $Nc \leftarrow$ la largeur de l'image ;
 $tab[0] \leftarrow \gamma$;
 $tab[1] \leftarrow Nl \bmod (2\gamma - 2)$;% résidus sur la hauteur de l'image
 $tab[2] \leftarrow Nc \bmod (4\gamma - 2)$;% résidus sur la largeur de l'image
Retourner : tab;
fin
fin

tableau est γ exprimé en pixels et noté px , la deuxième et la troisième valeurs sont les résidus de ligne et de colonne, respectivement.

1.2 Sélection des cellules de la grille virtuelle

Le critère de sélection d'un hexagone dans la grille d'hexagones est le taux de pixels allumés dans l'hexagone. Ce taux est calculé avec l'algorithme 18. Il convient de noter que, pour un hexagone donné de la grille, tous les pixels ne sont pas pris en compte dans le calcul du taux. Dans la figure 4.2, les pixels de couleur verte sont ceux pris en compte dans le calcul. Les autres seront pris en compte dans les hexagones voisins, comme illustré dans la figure 4.4. L'exclusion des pixels de couleur rouge et

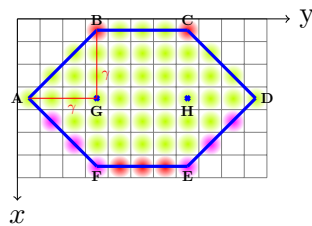


FIGURE 4.2 – Représentation des pixels d'une cellule hexagonale

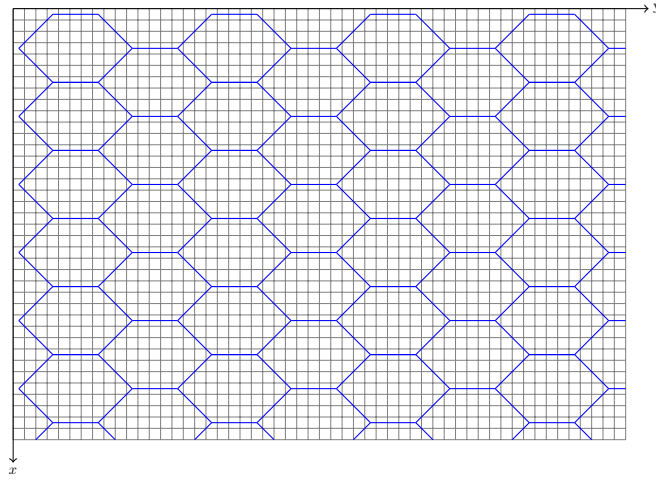


FIGURE 4.3 – Une grille hexagonale superposée sur un espace image

magenta (avec l'algorithme 22) permet de ne pas prendre en compte deux fois le même pixel dans deux hexagones différents.

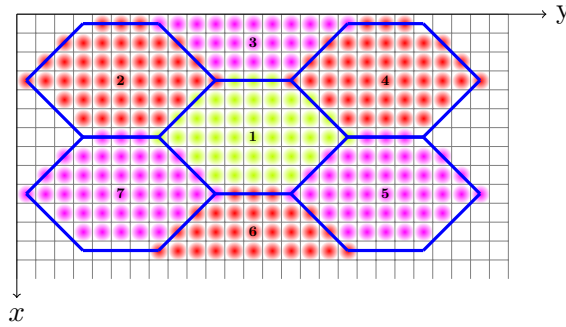


FIGURE 4.4 – La prise en compte des pixels exclus d'une cellule hexagonale

Le volume d'une cellule Hexagonale est calculé selon le lemme 2.

Lemme 2 (L'aire d'une cellule hexagonale) Soit γ la taille d'un hexagone. La surface de l'hexagone, représentée par la partie verte de la figure 4.2, est calculée à l'aide de la formule suivante :

$$S_H(\gamma) = 4(\gamma - 1)(\gamma - \frac{1}{2}) \quad (4.1)$$

Preuve 2 (lemme 2) Considérons γ comme la taille de l'hexagone.

Calcul de la surface $S_r(\gamma)$ du rectangle dans lequel l'hexagone est inscrit :

$S_r(\gamma) = L \times l = (3\gamma - 1)(2\gamma - 1)$ où $L = (3\gamma - 1)$ et $l = (2\gamma - 1)$ représentent respectivement la longueur et la largeur du rectangle.

Calcul de la surface $S_t(\gamma)$ des quatre triangles rectangles (aux sommets du rectangle) : $S_t(\gamma) = 4(\frac{\gamma^2 - \gamma}{2})$

Algorithme 18 : Calcul du taux de pixels allumés

```

Fonction count_hex(img : image, A, B, C, D, E, F, G, H : tableau de deux entiers) : réel
Variables : k, l : entiers ;
Debut
    k ← 0 ; l ← 0 ; MAT1 ← Rely(A, F) ; MAT2 ← Rely(F, E) ; MAT3 ← Rely(E, D) ;
    for A[1] ≤ y ≤ D[1] do
        if A[1] ≤ y ≤ G[1] then
            NP ← |y - A[1]|
            for A[0] - NP ≤ x ≤ A[0] + NP do
                if (x, y) est un element de MAT1 then
                    ne rien faire ;
                else
                    if 0 ≤ x ≤ Nl et 0 ≤ y ≤ Nc then
                        if img[x, y] ≠ 0 then
                            k ← k + 1 ;
                        else
                            l ← l + 1 ;
                        end
                    end
                end
            end
        end
    end
    if G[1] ≤ y ≤ H[1] then
        for B[0] ≤ x ≤ F[0] do
            if (x, y) est un element de MAT2 then
                ne rien faire ;
            else
                if 0 ≤ x ≤ Nl and 0 ≤ y ≤ Nc then
                    if img[x, y] ≠ 0 then
                        k ← k + 1 ;
                    else
                        l ← l + 1 ;
                    end
                end
            end
        end
    end
    if H[1] ≤ y ≤ D[1] then
        MQ ← |y - D[1]|
        for A[0] - MQ ≤ x ≤ A[0] + MQ do
            if (x, y) est un element de MAT3 then
                ne rien faire ;
            else
                if 0 ≤ x ≤ Nl and 0 ≤ y ≤ Nc then
                    if img[x, y] ≠ 0 then
                        k ← k + 1 ;
                    else
                        l ← l + 1 ;
                    end
                end
            end
        end
    end
    end
    Retourner :  $\frac{k}{k+l}$  ;
fin

```

Calcul de la surface $S_e(\gamma)$ de l'ensemble des pixels exclus : $S_e(\gamma) = (\gamma - 1) + 2(\gamma - 1) + 2 = 3\gamma - 1$ où $(\gamma - 1)$ représente le nombre de pixels rouges en bas de l'hexagone, $2 \times (\gamma - 1)$ représente le nombre de pixels de couleur magenta sur les côtés gauche et droit de l'hexagone, et 2 représente les deux pixels rouges en haut de l'hexagone.

Calcul de la surface de l'hexagone (seulement la partie verte de la figure 4.2) : $S_H(\gamma) = S_r(\gamma) - (S_t(\gamma) + S_e(\gamma))$. Ce qui donne, après simplification et factorisation : $S_H(\gamma) = 4(\gamma - 1)(\gamma - \frac{1}{2})$

Par exemple, pour $\gamma = 4$, $S_H(4) = 4(4 - 1)(4 - \frac{1}{2}) = 4 \times 3 \times 3.5 = 42 \text{ px}^2$, comme illustré dans la figure 4.2. L'unité de mesure de la surface est en pixels carrés (px^2).

1.3 Parallélisation de la transformée de Hough hexagonale

La parallélisation de la transformée de Hough hexagonal nécessite une adaptation de l'algorithme pour tirer parti des capacités de traitement simultané offertes par le parallélisme. La parallélisation concerne l'accumulateur c'est-à-dire l'espace des paramètre encore appelé l'espace de Hough. Les

Algorithme 19 : Mise à jour de l'accumulateur

```
Procedure Acc_update() : vide
    % Mise à jour de l'accumulateur
    if img[z, t]  $\neq$  0 then
        for itheta dans range(accum_row) do
            theta  $\leftarrow$  itheta  $\times$  dtheta;
            rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
            irho  $\leftarrow$  int(rho);
            if irho > 0 et irho < accum_column then
                accum[itheta][irho]  $\leftarrow$  accum[itheta][irho] + 1
            end
        end
    end
fin
```

algorithmes 24 et 23 sont les versions parallélisées des algorithmes 20 et 19. Les étapes principales de la parallélisation sont :

- importation de la bibliothèque *pymmp*⁴ : Importer une bibliothèque en Python est nécessaire pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction *import pymmp* est utilisée pour charger la bibliothèque *pymmp*.

- création de l'accumulateur de type *pymmp* : Dans l'algorithme séquentiel, l'espace des paramètres (l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la transformée de Hough, est transformé en une structure de données partagée grâce à *pymmp*. En effet, *pymmp* permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.

L'instruction "*accum_pymmp* $\leftarrow$ *pymmp.shared.array([accum_row, accum_column])*" de l'algorithme 24 crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable "*accum_pymmp*".

- construction parallèle : L'instruction "*with pymmp.Parallel(4) as p*" de l'algorithme 24 indique que nous souhaitons exécuter le bloc de code à l'intérieur de la déclaration "with" en parallèle avec 4 processus.

- parallélisation de l'algorithme 19 de mise à jour de l'accumulateur : La fonction "*p.range*" génère une séquence d'indices à partir de laquelle chaque cœur du CPU obtiendra une plage d'indices spécifique. Cela permet de diviser le travail entre les cœurs du CPU de manière équitable. Ainsi la boucle "*for itheta in p.range(accum_row)*" de l'algorithme 23 utilise les indices compris entre 0 et *accum_row* - 1 pour itérer à travers l'accumulateur par sections, où chaque section est traitée par un processus différent.

4. <https://github.com/classner/pymmp/tree/main>

Algorithme 20 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 1)

```

Fonction Hexagonal(img,  $\gamma$ ,  $\alpha$ , seuil) : tableau
Pre-condition :  $2 \leq \gamma \leq \min(\frac{Nl+2}{2}, \frac{Nc+2}{2})$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
Variables : tab : tableau de 6 entiers; A, B, C, D, E, F, G, H : tableaux de deux entiers; accum_ligne, accum_colon, irho, Nl, Nc : entiers; accum : matrice; dtheta, drho, rho, theta, DE : réels;

Debut
    Nl  $\leftarrow$  la hauteur de l'image; Nc  $\leftarrow$  la largeur de l'image; % les dimensions de l'image ;
    accum_row  $\leftarrow$  180; accum_column  $\leftarrow$   $E(\sqrt{(Nc^2 + Nl^2)})$ ; % les dimensions de la matrice "accum";
    dtheta  $= \pi/180$ ; tab  $\leftarrow$  maillage_hex(img,  $\gamma$ ); Ht  $\leftarrow$  Nl - tab[1]; La  $\leftarrow$  Nc - tab[2]; H1  $\leftarrow$  Ht +  $2\gamma - 2$ ; L1  $\leftarrow$  La +  $4\gamma - 2$ ;
    for  $3\gamma - 2 \leq y \leq L1$  par pas de  $4\gamma - 2$  do
        for  $2\gamma - 2 \leq x \leq H1$  par pas de  $2\gamma - 2$  do
            A[0]  $= x - \text{tab}[0] + 1$ ; A[1]  $= y - 3\text{tab}[0] + 2$ ; B[0]  $= x - 2\text{tab}[0] + 2$ ; B[1]  $= y - 2\text{tab}[0] + 1$ ; C[0]  $= x - 2\text{tab}[0] + 2$ ;
            C[1]  $= y - \text{tab}[0] + 1$ ; D[0]  $= x - \text{tab}[0] + 1$ ; D[1]  $= y$ ; E[0]  $= x$ ; E[1]  $= y - \text{tab}[0] + 1$ ; F[0]  $= x$ ;
            F[1]  $= y - 2\text{tab}[0] + 1$ ; G[0]  $= x - \text{tab}[0] + 1$ ; G[1]  $= y - 2\text{tab}[0] + 1$ ; H[0]  $= x - \text{tab}[0] + 1$ ; H[1]  $= y - \text{tab}[0] + 1$ ;
            if count(img, A, B, D, E, F, G, H)  $\geq \alpha$  then
                MAT1  $\leftarrow$  Rely(A, F); MAT2  $\leftarrow$  Rely(F, E); MAT3  $\leftarrow$  Rely(E, D);
                for  $A[1] \leq j \leq D[1]$  do
                    if  $A[1] \leq j \leq G[1] - 1$  then
                        NP  $\leftarrow |j - A[1]|$ 
                        for  $A[0] - NP \leq i \leq A[0] + NP$  do
                            if (i, j) est un élément de MAT1 then
                                ne rien faire;
                            else
                                Acc.update();
                            end
                        end
                    end
                if  $G[1] \leq j \leq H[1]$  then
                    for  $B[0] \leq i \leq F[0]$  do
                        if (i, j) est un élément de MAT2 then
                            ne rien faire;
                        else
                            Acc.update();
                        end
                    end
                end
                if  $H[1] \leq j \leq D[1]$  then
                    MQ  $\leftarrow |j - D[1]|$ 
                    for  $A[0] - MQ \leq i \leq A[0] + MQ$  do
                        if (i, j) est un élément de MAT3 then
                            ne rien faire;
                        else
                            Acc.update();
                        end
                    end
                end
            end
        end
    end

```

1.4 Reconnaissance des droites discrètes

L'algorithme 20 de la reconnaissance de droites discrètes par la Transformée de Hough Hexagonale prend en entrée une image (*img*), un paramètre γ représentant la taille de l'hexagone, un paramètre *alpha* représentant le taux de pixels allumés dans l'hexagone, et le seuil de votes.

L'algorithme commence par initialiser les variables nécessaires, notamment les tableaux pour les points de l'hexagone (*A*, *B*, *C*, *D*, *E*, *F*, *G*, *H*), les dimensions de l'image, et la matrice d'accumulateur. Ensuite, il effectue un maillage hexagonal de l'image à l'aide de la fonction *maillage_hex* représenté par l'algorithme 17.

Le processus de détection des droites discrètes se fait en parcourant les hexagones de la grille générée. Pour chaque hexagone, il vérifie s'il y a suffisamment de points allumés dans la région correspondante et met à jour l'accumulateur en conséquence.

Le cœur de l'algorithme se trouve dans la double boucle **for** qui parcourt les positions des hexagones dans l'image. Pour chaque position, il calcule les coordonnées des points de l'hexagone, vérifie le nombre de points allumés à l'intérieur de l'hexagone, et met à jour l'accumulateur à l'aide de l'algorithme 18

Algorithme 21 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 2)

```

for  $\gamma - 1 \leq y \leq L1$  par pas de  $4\gamma - 2$  do
  for  $\gamma - 1 \leq x \leq H1$  par pas de  $2\gamma - 2$  do
     $A[0] = x - \text{tab}[0] + 1$ ;  $A[1] = y - 3\text{tab}[0] + 2$ ;  $B[0] = x - 2\text{tab}[0] + 2$ ;  $B[1] = y - 2\text{tab}[0] + 1$ ;  $C[0] = x - 2\text{tab}[0] + 2$ ;
     $C[1] = y - \text{tab}[0] + 1$ ;  $D[0] = x - \text{tab}[0] + 1$ ;  $D[1] = y$ ;  $E[0] = x$ ;  $E[1] = y - \text{tab}[0] + 1$ ;  $F[0] = x$ ;
     $F[1] = y - 2\text{tab}[0] + 1$ ;  $G[0] = x - \text{tab}[0] + 1$ ;  $G[1] = y - 2\text{tab}[0] + 1$ ;  $H[0] = x - \text{tab}[0] + 1$ ;  $H[1] = y - \text{tab}[0] + 1$ ;
    if  $\text{count}(\text{img}, A, B, C, D, E, F, G, H) \geq \alpha$  then
       $\text{MAT1} \leftarrow \text{Rely}(A, F)$ ;  $\text{MAT2} \leftarrow \text{Rely}(F, E)$ ;  $\text{MAT3} \leftarrow \text{Rely}(E, D)$ ;
      for  $A[1] \leq j \leq D[1]$  do
        if  $A[1] \leq j \leq G[1] - 1$  then
           $NP \leftarrow |j - A[1]|$ 
          for  $A[0] - NP \leq i \leq A[0] + NP$  do
            if  $(i, j)$  est un element de  $\text{MAT1}$  then
              ne rien faire;
            else
              Acc.update()
            end
          end
        end
        if  $G[1] \leq j \leq H[1]$  then
          for  $B[0] \leq i \leq F[0]$  do
            if  $(i, j)$  est un element de  $\text{MAT2}$  then
              ne rien faire;
            else
              Acc.update();
            end
          end
        end
        if  $H[1] \leq j \leq D[1]$  then
           $MQ \leftarrow |j - D[1]|$ 
          for  $A[0] - MQ \leq i \leq A[0] + MQ$  do
            if  $(i, j)$  est un element de  $\text{MAT3}$  then
              ne rien faire;
            else
              Acc.update()
            end
          end
        end
      end
    end
  end
end
end
Rechercher les maxima dans l'accumulateur et placer les couples  $(\rho, \theta)$  dans une liste 'lignes';
Retourner la liste 'lignes';
fin
fin

```

et de l'algorithme 19 respectivement.

Enfin, il retourne une liste appelée *lignes* contenant les couples (θ, ρ) correspondant aux paramètres des droites détectées.

L'objectif de l'algorithme 26 est de visualiser les droites identifiées à l'aide de la méthode de la Transformée de Hough. Il requiert une image en entrée (*image*), la liste des paramètres de droites détectées (*lignes*) renvoyée par l'algorithme 20, ainsi qu'une spécification de couleur (*colors*). L'algorithme parcourt la liste des paramètres de droites détectées et reconstitue chaque droite correspondante sur l'image, attribuant la couleur *colors* à ces droites reconstruites. Cette étape de reconstruction vise à mettre en évidence de manière visuelle les droites détectées sur l'image d'origine, ce qui facilite l'interprétation et l'analyse des résultats obtenus par la Transformée de Hough.

2 Complexité de la transformée de Hough hexagonale

La fonction *Relate* dans l'algorithme 22 assigne des valeurs à un tableau, puis retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

La fonction *maillage_hex* dans l'algorithme 17 initialise un tableau et lui assigne des valeurs, puis

Algorithme 22 : Sélection de tous les pixels compris entre A et B.

Fonction *Relate*(*A*, *B* : tableaux de deux entiers) : liste**Variables** : *MAT* : une liste de couple d'entiers;**Debut***MAT* $\leftarrow$ les coordonnées des pixels
compris entre A et B, A et B exclus ;**Retourner** : *MAT* ;**fin****fin**

Algorithme 23 : Mise à jour parallèle de l'accumulateur

Procédure *Acc_updateP*() : vide

% Mise à jour de l'accumulateur

if *img*[*z*, *t*] $\neq$ 0 **then****for** *itheta* dans *p.range(accum_row)* **do***theta* $\leftarrow$ *itheta* $\times$ *dtheta*;*rho* $\leftarrow$ *t* $\times$ *cos(theta)* + *z* $\times$ *sin(theta)*;*irho* $\leftarrow$ *int(rho)*;**if** *irho* > 0 et *irho* < *accum_column* **then***accum_pymp*[*itheta*][*irho*] $\leftarrow$ *accum_pymp*[*itheta*][*irho*] + 1**end****end****end****fin**

retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

La fonction *count_hex* dans l'algorithme 18 : initialise des variables *k*, *l* et *D*, puis elle parcourt les pixels à l'intérieur de l'hexagone à l'aide de boucles dont le nombre d'itérations est $4(\gamma - 1)(\gamma - \frac{1}{2})$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité du bloc formé par les boucles est $O(4(\gamma - 1)(\gamma - \frac{1}{2})) = O(\gamma^2)$, où $4(\gamma - 1)(\gamma - \frac{1}{2})$ est le nombre de pixels à l'intérieur de l'hexagone. Ainsi, la complexité totale de la fonction *count_hex* est $O(\gamma^2)$.

La fonction *Acc_update* dans l'algorithme 19 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de *accum_row* = 180, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction *Acc_update* est $O(1)$.

L'algorithme 20 de la transformée de Hough hexagonal commence par l'initialisation des variables et du tableau *tab*, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction *maillage_hex* dont la complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux boucles imbriquées parcourant les points à l'intérieur de chaque hexagone à l'intérieur du quelle se trouve des opérations de coût constant et la fonction *Acc_update* de complexité $O(1)$. La complexité

Algorithme 24 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée (partie 1)

```

Fonction HexagonalP(image,  $\gamma$ ,  $\alpha$ , seuil, cpu_count) : tableau
Pre-condition :  $2 \leq \gamma \leq \min(\frac{Nl+2}{2}, \frac{Nc+2}{2})$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
Données :  $\alpha$ 
Variables : tab : tableau de 6 entiers; A, B, C, D, E, F, G, H : tableau de deux entiers;
accum_ligne, accum_colon, irho, Nl, Nc : entiers; accum : accumulator; Seuil,  $\alpha$ , dtheta, drho, rho, theta, DE : réels;
Debut
    import pypm; % importation de la bibliothèque pypm
    Nl  $\leftarrow$  number of rows of image; Nc  $\leftarrow$  number of columns of image; % the dimensions of the image ;
     $\alpha = 0.8$  % The rate of lit pixels from which the triangle is selected; threshold  $\leftarrow$  150 % the minimum number of votes ;
    accum_row  $\leftarrow$  180; accum_column  $\leftarrow$   $E(\sqrt{(Nc^2 + Nl^2)})$ ; % the dimensions of the matrix "accum";
    accum_pypm  $\leftarrow$  pypm.shared.array(accum_row, accum_column); % creation de l'accumulateur de type pypm
     $\gamma \leftarrow 3$ ; dtheta =  $\pi/180$ ; tab  $\leftarrow$  mesh(image,  $\gamma$ ); Ht  $\leftarrow$  Nl - tab[3]; La  $\leftarrow$  Nc - tab[4]; H1  $\leftarrow$  Ht +  $2\gamma - 2$ ; L1  $\leftarrow$  La +  $4\gamma - 2$ ;
    % Démarrage du traitement parallèle
    With pypm.parallel(cpu_count) as p :
        for  $3\gamma - 2 \leq y \leq L1$  par pas de  $4\gamma - 2$  do
            for  $2\gamma - 2 \leq x \leq H1$  par pas de  $2\gamma - 2$  do
                A[0] = x - tab[0] + 1; A[1] = y - 3tab[0] + 2; B[0] = x - 2tab[0] + 2; B[1] = y - 2tab[0] + 1;
                C[0] = x - 2tab[0] + 2;
                C[1] = y - tab[0] + 1; D[0] = x - tab[0] + 1; D[1] = y; E[0] = x; E[1] = y - tab[0] + 1; F[0] = x;
                F[1] = y - 2tab[0] + 1; G[0] = x - tab[0] + 1; G[1] = y - 2tab[0] + 1; H[0] = x - tab[0] + 1; H[1] = y - tab[0] + 1;
                if count(img, A, B, D, E, F, G, H)  $\geq \alpha$  then
                    MAT1  $\leftarrow$  Rely(A, F); MAT2  $\leftarrow$  Rely(F, E); MAT3  $\leftarrow$  Rely(E, D);
                    for A[1]  $\leq j \leq D[1] do
                        if A[1]  $\leq j \leq G[1] - 1 then
                            NP  $\leftarrow$  |j - A[1]|
                            for A[0] - NP  $\leq i \leq A[0] + NP do
                                if (i, j) est un élément de MAT1 then
                                    ne rien faire;
                                else
                                    Acc.updateP();
                                end
                            end
                        end
                        if G[1]  $\leq j \leq H[1] then
                            for B[0]  $\leq i \leq F[0] do
                                if (i, j) est un élément de MAT2 then
                                    ne rien faire;
                                else
                                    Acc.updateP();
                                end
                            end
                        end
                        if H[1]  $\leq j \leq D[1] then
                            MQ  $\leftarrow$  |j - D[1]|
                            for A[0] - MQ  $\leq i \leq A[0] + MQ do
                                if (i, j) est un élément de MAT3 then
                                    ne rien faire;
                                else
                                    Acc.updateP();
                                end
                            end
                        end
                    end
                end
            end
        end
    end$$$$$$$ 
```

de ces boucles est donc $O(4(\gamma - 1)(\gamma - \frac{1}{2})) = O(\gamma^2)$

Le coût des deux boucles principales imbriquées parcourant les hexagones de la grille, dont le nombre total d'itérations dépend de la taille de l'image et de la taille des cellules hexagonales, contenant un bloc d'opérations d'affectation de coût constant, un test de condition sur la fonction *count_hex* de complexité $O(\gamma^2)$, trois affectations appelant la fonction *Rely* de coût constant, les deux boucles imbriquées de coût $4(\gamma - 1)(\gamma - \frac{1}{2})$ parcourant les pixels à l'intérieur de l'hexagone, est $Nl/(4\gamma - 2) \times Nc/(2\gamma - 2) \times (1 + 4(\gamma - 1)(\gamma - \frac{1}{2}) + 1 + 4(\gamma - 1)(\gamma - \frac{1}{2}))$, donc de complexité $O(Nl/(4\gamma - 2) \times Nc/(2\gamma - 2) \times (1 + 4(\gamma - 1)(\gamma - \frac{1}{2}) + 1 + 4(\gamma - 1)(\gamma - \frac{1}{2}))) = O(Nl \times Nc)$.

En considérant ces points, la complexité totale de l'algorithme 20 de la THH est égal à $O(Nl \times Nc)$, où *Nl* et *Nc* sont le nombre de lignes et de colonnes de l'image respectivement, γ est la taille des cellules hexagonale. Ce qui correspond à la complexité de la THS.

Algorithme 25 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée (partie 2)

```

for  $\gamma - 1 \leq y \leq L1$  par pas de  $4\gamma - 2$  do
  for  $\gamma - 1 \leq x \leq H1$  par pas de  $2\gamma - 2$  do
     $A[0] = x - tab[0] + 1$ ;  $A[1] = y - 3tab[0] + 2$ ;  $B[0] = x - 2tab[0] + 2$ ;  $B[1] = y - 2tab[0] + 1$ ;
     $C[0] = x - 2tab[0] + 2$ ;
     $C[1] = y - tab[0] + 1$ ;  $D[0] = x - tab[0] + 1$ ;  $D[1] = y$ ;  $E[0] = x$ ;  $E[1] = y - tab[0] + 1$ ;  $F[0] = x$ ;
     $F[1] = y - 2tab[0] + 1$ ;  $G[0] = x - tab[0] + 1$ ;  $G[1] = y - 2tab[0] + 1$ ;  $H[0] = x - tab[0] + 1$ ;  $H[1] = y - tab[0] + 1$ ;
    if  $count(img, A, B, C, D, E, F, G, H) \geq \alpha$  then
       $MAT1 \leftarrow Rely(A, F)$ ;  $MAT2 \leftarrow Rely(F, E)$ ;  $MAT3 \leftarrow Rely(E, D)$ ;
      for  $A[1] \leq j \leq D[1]$  do
        if  $A[1] \leq j \leq G[1] - 1$  then
           $NP \leftarrow |j - A[1]|$ 
          for  $A[0] - NP \leq i \leq A[0] + NP$  do
            if  $(i, j)$  est un element de  $MAT1$  then
              ne rien faire;
            else
              Acc.updateP();
            end
          end
        end
      end
      if  $G[1] \leq j \leq H[1]$  then
        for  $B[0] \leq i \leq F[0]$  do
          if  $(i, j)$  est un element de  $MAT2$  then
            ne rien faire;
          else
            Acc.updateP();
          end
        end
      end
      if  $H[1] \leq j \leq D[1]$  then
         $MQ \leftarrow |j - D[1]|$ 
        for  $A[0] - MQ \leq i \leq A[0] + MQ$  do
          if  $(i, j)$  est un element de  $MAT3$  then
            ne rien faire;
          else
            Acc.updateP();
          end
        end
      end
    end
  end
end
end
fin
Rechercher les maxima dans l'accumulateur et placer le couple  $(\rho, \theta)$  dans une liste 'lignes';
Retourner la liste 'lignes';
fin

```

Algorithme 26 : Reconstruction des droites détectées

Fonction *PlotHoughLine*(*imge*, *lignes*, *colors*) : *image*

Parcourir '*lignes*' la liste des paramètres des droites détectées, puis construire chaque droite correspondant en couleur '*colors*' sur l'image '*imge*'.

fin

3 Simulations et discussions

Dans cette section, nous présentons les résultats des simulations et des discussions sur l'application de la Transformée de Hough Hexagonale à deux images différentes : celle d'un immeuble et celle d'une autoroute représentées sur la figure 1.11. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Nous examinons les effets de trois paramètres clés, le seuil de vote, γ et α , sur les performances de

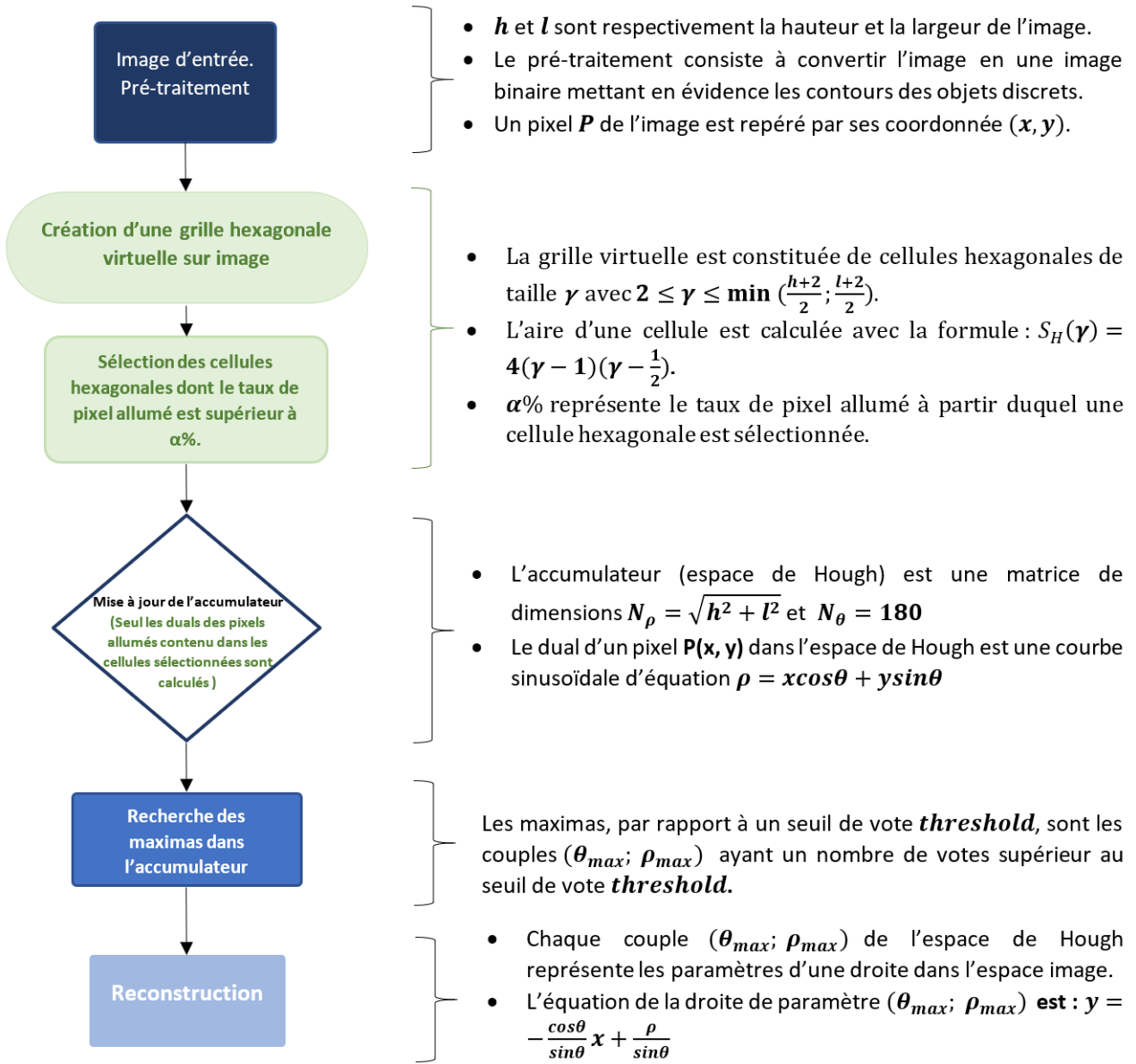


FIGURE 4.5 – Les étapes de la technique de la transformée de Hough hexagonale

la méthode. En outre, nous comparons les résultats de la transformée de Hough hexagonale avec ceux de la Transformée de Hough Standard pour évaluer son efficacité et sa précision. Enfin, nous discutons également les performances de la transformée de Hough hexagonale par rapport à sa version parallélisée.

3.1 Cas de l'image de l'immeuble

Dans cette section, nous examinons les performances de la Transformée de Hough Hexagonale appliqué à une image représentant un immeuble. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans

TABLEAU 4.1 – Resultats des simulations sur l’image 1.11(c) de l’immeuble avec l’algorithme 20 de la THH avec les paramètres fixes (Seuil=62 et $\alpha = 14\%$) et γ variant entre 2 et 64

γ	S_H	Droites détectées	temps(sec)
2	6	212	0.57
4	42	223	0.53
6	110	157	0.49
8	210	77	0.44
10	342	109	0.46
12	506	40	0.40
14	702	51	0.39
26	2550	21	0.33
30	3422	24	0.33
64	16002	15	0.32

ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l’algorithme THH en fonction des caractéristiques de l’image de l’immeuble.

3.1.1 Variation de γ

Le tableau 4.1 résume les résultats de simulations effectuées sur l’image de l’immeuble en utilisant l’algorithme de la THH avec des paramètres fixes (Seuil=62 et $\alpha = 14\%$), tandis que le paramètre γ varie entre 2 et 64. Les colonnes indiquent les valeurs de γ , l’aire S_H des cellules hexagonale, le nombre de lignes détectées et le temps de traitement pour chaque configuration de paramètres.

L’analyse du tableau révèle des variations du nombre de lignes détectées et du temps de traitement en fonction des valeurs de γ . le temps de traitement diminue à mesure que γ augmente.

La figure 4.6 illustrent graphiquement ces données. Le premier graphique de la figure montre la relation entre la taille des cellules et le nombre de lignes détectées. De façon générale, le graphique du nombre de droites détectées en fonction la taille des cellules est décroissante. Cependant, l’augmentation de la taille des cellules n’implique pas toujours une diminution du nombre de droites détectées. Par exemple, avec $S_H = 42$ 223 droites ont été détectées contre 212 droites avec $S_H = 6$. Le deuxième graphique illustre la relation entre la taille des cellules et le temps de traitement. On constate une tendance décroissante : à mesure que la taille des cellules hexagonales augmente, le temps de traitement diminue.

3.1.2 Variation de α

La table 4.2 présente les résultats de simulations sur l’image 1.11(c) de l’immeuble en utilisant l’algorithme 20 de la THH. Les simulations ont été réalisées avec des paramètres fixes (Seuil=60 et $\gamma = 4$), tandis que le paramètre α a été modifié entre 10% et 35%. Les colonnes indiquent les valeurs de α , le nombre de lignes détectées et le temps de traitement.

La figure 4.7 contient des graphiques représentant les données de la table 4.2 :

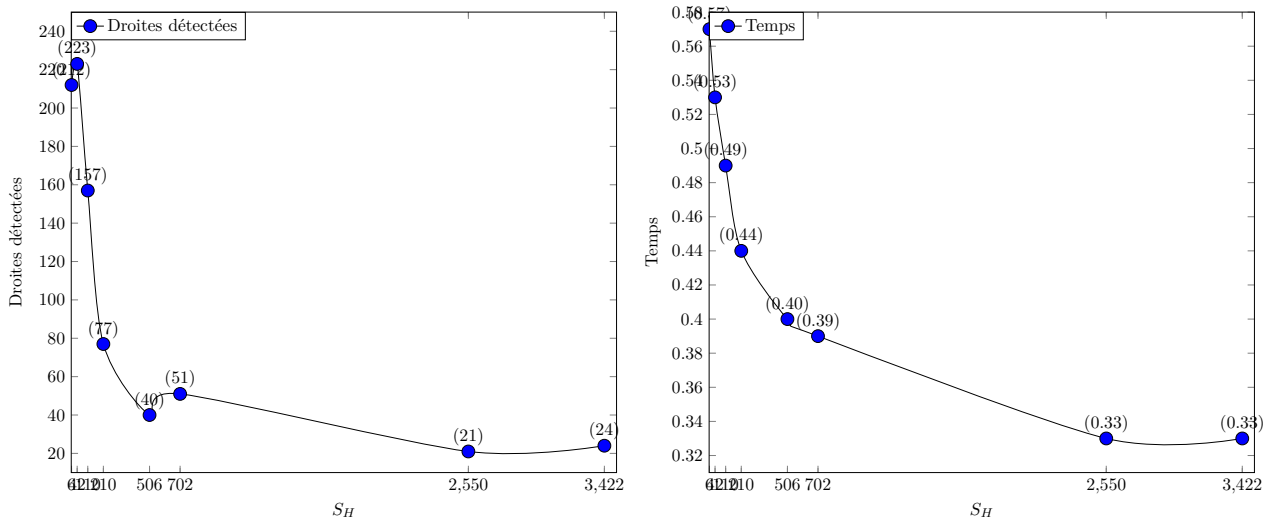


FIGURE 4.6 – Courbe représentative des données du tableau 4.1

TABEAU 4.2 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=60 and $\gamma = 4$) et α variant entre 10% et 35%

$\alpha(\%)$	Nbre de droites détectées	temps(sec)
10	492	0.58
15	236	0.48
20	106	0.42
25	34	0.33
30	3	0.24
35	1	0.16

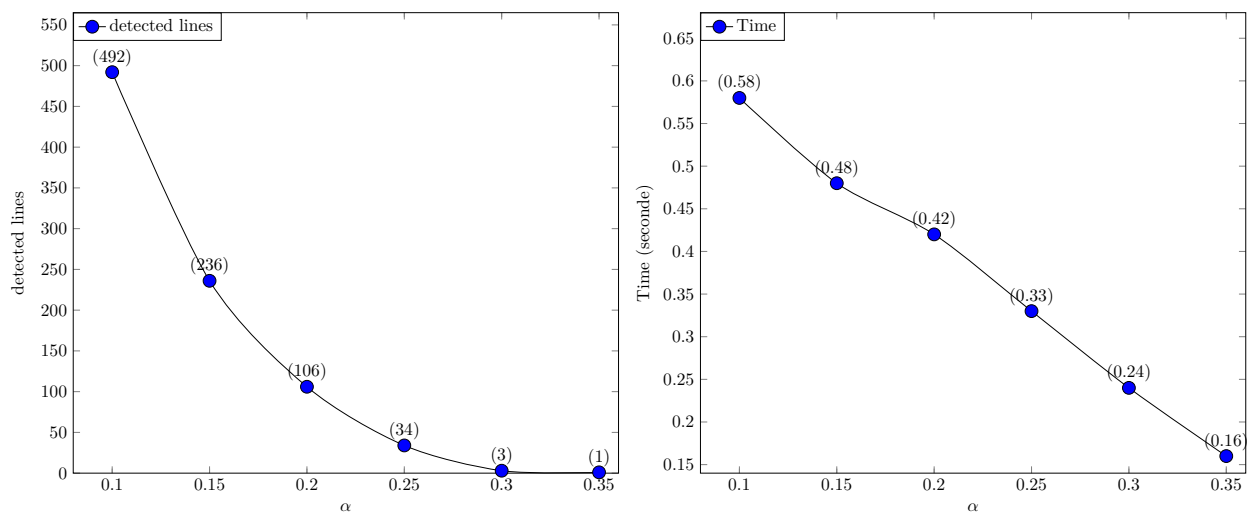


FIGURE 4.7 – Courbe représentative des données du tableau 4.2

Le premier graphique illustre la relation entre le taux α et le nombre de lignes détectées par l'algorithme 20 de la transformée de Hough hexagonale. On observe une décroissance du nombre de lignes détectées à mesure que α augmente. Cette tendance suggère qu'avec des valeurs plus élevées de α , moins de lignes sont détectées, indiquant une sensibilité de l'algorithme à ce paramètre. Des valeurs trop élevées de α peuvent entraîner des détections non pertinentes de droites

Le deuxième graphique représente le temps de traitement en fonction du taux α . On observe également une décroissance du temps de traitement à mesure que α augmente. Cette diminution du temps de traitement est associée à la valeur de α , car plus α augmente, moins on a de cellules à traiter conduisant naturellement à une accélération du processus.

L'analyse du tableau révèle des variations du nombre de lignes détectées et du temps de traitement en fonction des valeurs de α . À mesure que α augmente, le nombre de lignes détectées diminue, passant de 492 lignes avec $\alpha = 10\%$ à seulement 1 ligne avec $\alpha = 35\%$. De même, le temps de traitement diminue à mesure que α augmente.

La figure 4.7 illustrent graphiquement ces données. Le premier graphique de la figure montre la relation entre α et le nombre de lignes détectées. On observe une tendance décroissante : à mesure que α augmente, le nombre de lignes détectées diminue. Le deuxième graphique de la figure illustre la relation entre α et le temps de traitement. Encore une fois, on constate une tendance décroissante : à mesure que α augmente, le temps de traitement diminue.

Ces résultats mettent en évidence l'importance du paramètre α dans l'algorithme de la transformée de Hough hexagonale. Des valeurs élevées de α conduisent non seulement à une réduction du nombre de lignes détectées mais également à une diminution du temps de traitement. Cependant, il faut noter que des valeurs trop élevées de α peuvent conduire à des détections moins précises ou nulles.

3.2 Cas de l'image de l'autoroute

Dans cette section, nous examinons les performances de l'algorithme de la transformée de Hough hexagonale appliqué à une image représentant une autoroute. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la THH en fonction des caractéristiques de l'image de l'immeuble.

TABLEAU 4.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=60 and $\alpha = 10\%$) et γ variant entre 2 et 40

γ	$S_H(px^2)$	Nbre de droites détectées	temps(sec)
2	6	80	0.28
3	20	69	0.26
4	42	62	0.24
5	72	60	0.24
6	110	54	0.23
7	156	32	0.21
8	210	30	0.22
9	272	17	0.20
10	342	16	0.20
14	702	22	0.20
20	1482	25	0.21
32	3 906	10	0.16
40	6162	4	0.13

3.2.1 Variation de la taille des cellules de la grille

Le tableau 4.3 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute du chapitre 1 en utilisant l'algorithme 20 de la transformée de Hough hexagonale. Les simulations ont été réalisées avec des paramètres fixes (Seuil=60 et $\alpha = 10\%$), tandis que le paramètre γ a été modifié entre 2 et 40. La table 4.3 comporte quatre colonnes que sont : La première colonne représente les différentes valeurs du paramètre γ , variant de 2 à 10. La deuxième colonne indique la surface (l'aire) S_H de la cellule hexagonale H en pixels carrés. La troisième colonne montre le nombre de lignes détectées pour chaque valeur de γ . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

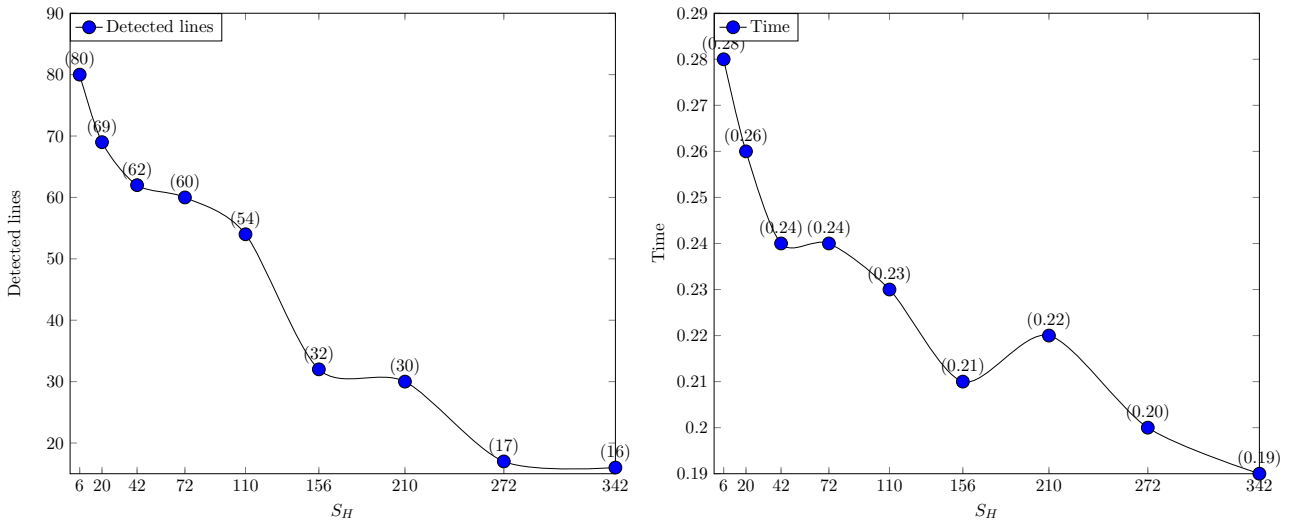


FIGURE 4.8 – Courbe représentative des données du tableau 4.3

La figure 4.8 est la représentation graphique du tableau 4.3. Elle contient deux graphiques :

Le premier graphique illustre la variation du nombre de lignes détectées par l'algorithme THH en

TABLEAU 4.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=60 and $\gamma = 3$) et α variant entre 10% et 30%

α	Nbre de droites détectées	temps(sec)
10	69	0.25
15	52	0.24
20	34	0.22
25	17	0.20
30	0	0.13
(pour $\alpha = 11$ et <i>seuil</i> = 51)	31	0.13

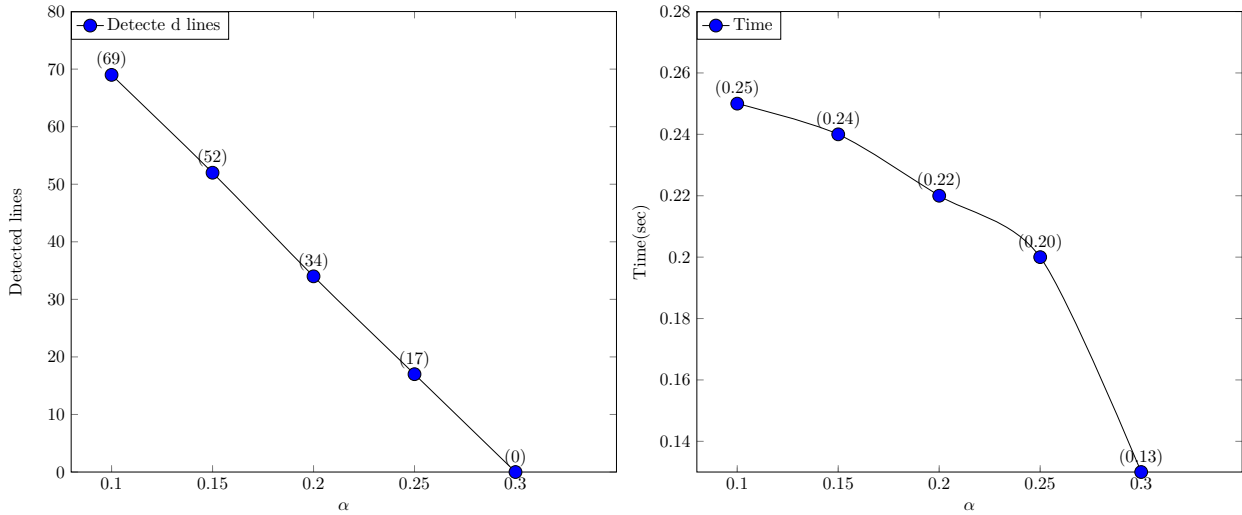


FIGURE 4.9 – Courbe représentative des données du tableau 4.4

fonction de la taille des cellules hexagonales évalué par leur surface S_H . Pour les valeurs de S_H allant de 6 à 342, on observe, en générale, une tendance à la diminution du nombre de lignes détectées en fonction de la taille des cellules hexagonales. Cependant, une augmentation de la taille des cellules n'implique toujours pas une diminution du nombre de droites détectées. Par exemple, avec $S_H = 1482$ 25 droites ont été détectées contre 16 droites avec $S_H = 342$. Le deuxième graphique représente le temps de traitement en fonction de la taille des cellules hexagonale. On constate une légère diminution du temps de traitement à mesure que la surface de l'hexagone augmente.

3.2.2 Variation de α

Le tableau 4.4 présente les résultats de simulations sur une image d'une autoroute en utilisant l'algorithme 20 de la THH. Les simulations ont été réalisées avec des paramètres fixes (Seuil=60 et $\gamma = 3$), tandis que le paramètre α a été modifié entre 10% et 30%. La première colonne représente les différentes valeurs du paramètre α , variant de 10% à 30%. La deuxième colonne montre le nombre de lignes détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse du tableau 4.4 révèle des variations du nombre de lignes détectées et du temps de traitement en fonction des valeurs de α . À mesure que α augmente, le nombre de lignes détectées

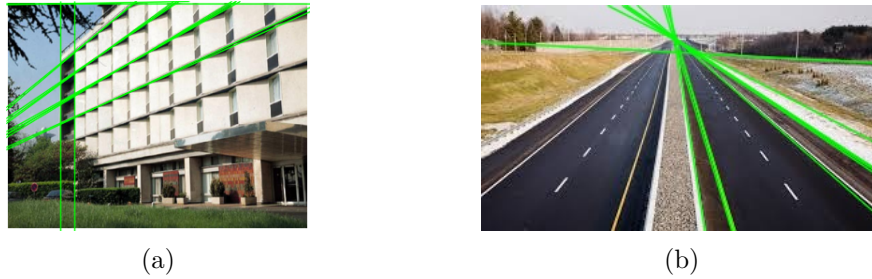


FIGURE 4.10 – (a) THH sur l'immeuble ($\gamma = 64$, $\alpha = 14$, $seuil = 62$), (b) THH sur l'autoroute ($\gamma = 30$, $\alpha = 10$, $seuil = 60$)

TABLEAU 4.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
25	25697	0,54	0,00002
116	10	0,49	0,049

diminue, passant de 69 lignes avec $\alpha = 10\%$ à 0 ligne avec $\alpha = 30\%$. Des valeurs trop élevées de α peuvent entraîner une détérioration de la qualité de détection. De manière similaire, le temps de traitement diminue à mesure que α augmente, indiquant une amélioration du temps de traitement avec des valeurs de α plus élevées et une précision de détection acceptable

La figure 4.9 illustrent graphiquement ces données. La première figure montre la relation entre α et le nombre de lignes détectées. On observe une tendance décroissante : à mesure que α augmente, le nombre de lignes détectées diminue. La deuxième figure illustre la relation entre α et le temps de traitement. Encore une fois, on constate une tendance décroissante : à mesure que α augmente, le temps de traitement diminue.

Ainsi, ces résultats mettent en évidence l'importance des paramètres $\gamma, \alpha, seuil$ dans l'algorithme de détection de lignes THH. Les cellules de grandes taille offre une précision acceptable et un temps de traitement plus rapide, en témoignent les données des tableaux 4.1 et 4.3, ainsi que les résultats visuels dans la figure 4.10.

3.3 Comparaison de transformée de Hough hexagonale et transformée de Hough standard

Nous procédons dans cette section à une comparaison de la transformée de Hough hexagonale,

TABLEAU 4.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
67	47	0,23	0,0048
100	10	0,21	0,021



FIGURE 4.11 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=25) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)



FIGURE 4.12 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=67) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough hexagonale sont illustré dans les tableaux 4.5 et 4.7 respectivement. En ce qui concerne la précision, pour un seuil de 25 votes, la transformée de Hough standard a effectué une détection de 25697 droites. Cette détection est non pertinente comme le montre la première image de la figure 4.11. par contre, avec le même seuil de votes, la THH a détecté 10 droites. Les résultats visuels illustrés par la deuxième image de la figure 4.13 montre que les droites détectées suivent bien des lignes de l'immeuble indiquant des détections pertinentes hors mis trois superflus.

Les paramètres supplémentaires utilisés par la transformée de Hough hexagonale pour parvenir à ce résultat sont : la taille γ et le pourcentage α de pixels allumés de chaque cellule hexagonale.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 4.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 4.11 illustre bien ces résultats. Ces droites détectées suivent bien des lignes de l'immeuble indiquant des détections pertinentes hormis celles des côtés délimitant l'image.

Pour ce qui est du temps de traitement, le tableau 4.5 montre que la THS à détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La transformée de Hough hexagonale a détecté le même nombre de droite en 0.14 seconde soit 0.014 seconde par droite détecté comme illustré dans le tableau 4.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,49}{0,14} = 3,5$ (T_h représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de

TABLEAU 4.7 – Résultats de simulation avec la THH appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)

γ	$S_H(px^2)$	$\alpha(\%)$	seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
3	20	30	50	37	0.30	0.0081
4	42	37	25	10	0.14	0.014

TABLEAU 4.8 – Résultats de simulation avec la THH appliquée sur l'image 1.11(a) de l'autoroute (temps1= temps de détection de toutes les droites; temps2=temps de détection d'une seule droite)

γ	$S_H(px^2)$	$\alpha(\%)$	Threshold	number of detected lines	time1(sec)	time2(sec)
2	6	50	39	33	0.15	0.0045
3	20	25	67	11	0.20	0.018

l'algorithme standard).



FIGURE 4.13 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres : (a) Seuil=50, $\alpha = 30\%$, $\gamma = 3$; (b) Seuil=25, $\alpha = 37\%$, $\gamma = 4$

De même avec l'image de l'autoroute, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough hexagonale sont illustré dans les tableaux 4.6 et 4.8 respectivement.

En ce qui concerne la précision, pour un seuil de 67 votes, la THS a effectué une détection de 47 droites. sur les 47, non seulement plusieurs sont non pertinentes mais aussi une même bord à été détectée plusieurs fois comme le montre la première image de la figure 4.12. par contre, avec le même seuil de votes, la transformée de Hough hexagonale a détecté 11 droites. Les résultats visuels illustrés par la deuxième image de la figure 4.14 montre une amélioration.

Les paramètres supplémentaires utilisés par la transformée de Hough hexagonale pour parvenir à ce résultat sont : la taille γ des cellules hexagonale et le pourcentage α de pixels allumés de chaque cellule hexagonale.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 4.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 4.12 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 4.6 montre que la THS à détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La transformée de Hough hexagonale a détecté quasiment le même nombre de droite en 0.20 seconde soit 0.018 seconde par droite détecté comme illustré dans le tableau 4.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en

outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,21}{0,20} = 1,05$ (T_h représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).



FIGURE 4.14 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres : (a) Seuil=39, $\alpha = 50\%$, $S_H = 6$ pour $\gamma = 2$; (b) Seuil=67, $\alpha = 25\%$, $S_H = 20$ pour $\gamma = 3$

3.4 Comparaison de THH et sa version parallélisée

Dans cette section, nous examinons et comparons les performances de la transformée de Hough hexagonale par rapport à sa version parallélisée. Nous explorons les variations du temps d'exécution en fonction de différents paramètres tels que α et S_H (surface des cellules hexagonales), en nous appuyant sur deux cas d'étude distincts : l'immeuble et l'autoroute. Cette comparaison vise à évaluer l'impact de la parallélisation sur les performances de la transformée de Hough hexagonale et à identifier les cas de figure où la transformée de Hough hexagonale parallélisée offre des avantages par rapport à sa version séquentielle.

3.4.1 Cas de l'image de l'immeuble

La figure 4.15 présente deux graphiques comparant les performances des algorithmes de THH et THHP. Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et S_H).

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 35%. Pour les petites valeurs de α (entre 10% et 30%), la courbe de transformée de Hough hexagonale parallélisée est en dessous de celle de la transformée de Hough hexagonale. Cela implique une amélioration des performances de la transformée de Hough hexagonale par la programmation parallèle pour les petites valeurs de α .

Pour des valeurs de α plus élevées (entre 30% et 35%), la courbe de la transformée de Hough hexagonale parallélisée passe au dessus de celle de la transformée de Hough hexagonale. Cela suggère que transformée de Hough hexagonale parallélisée est moins performante pour les valeurs élevées de α .

La parallélisation de la transformée de Hough hexagonale n'a pas affectée la précision de la détection.

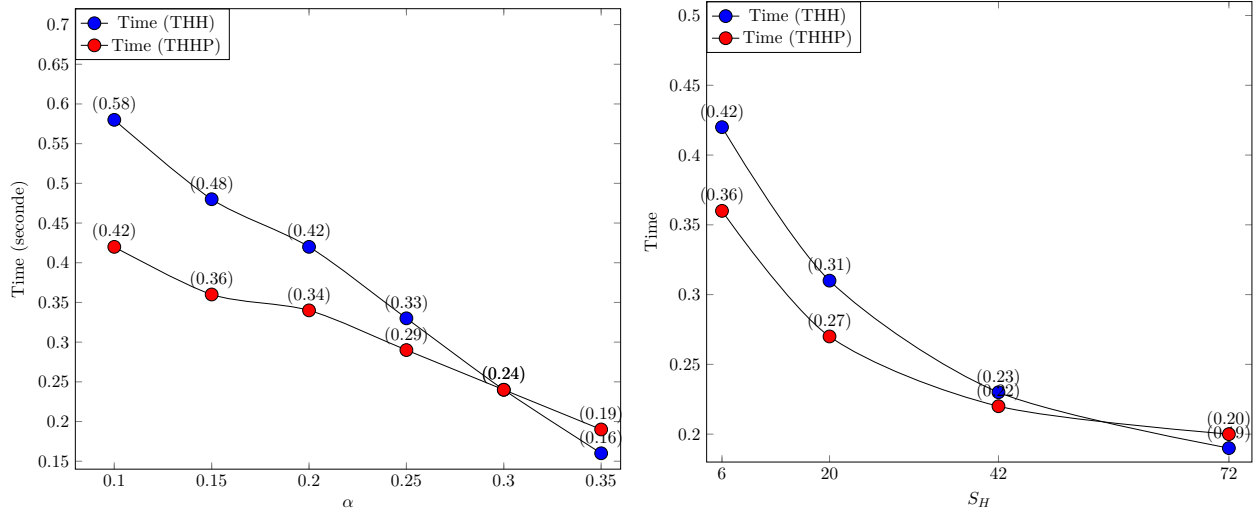


FIGURE 4.15 – Cas de l’immeuble : (à gauche) Comparaison des variations du temps d’exécution en fonction de α et les paramètres (seuil=60, $\gamma = 4$), (à droite) Comparaison des variations du temps d’exécution en fonction de la surface des cellules et les paramètres (seuil=50, $\alpha = 30\%$) des algorithmes 20 et 24

Dans le deuxième graphique, les variations du temps d’exécution sont comparées en fonction de la surface des cellules hexagonales, avec les deux algorithmes de THH et THHP. Les surfaces des cellules varient de $6 px^2$ à $72 px^2$. La courbe de la transformée de Hough hexagonale parallélisée est en dessous de celle de la transformée de Hough hexagonale pour les surfaces inférieure a $57 px^2$. De 57 à $72 px^2$, la courbe de la transformée de Hough hexagonale parallélisée passe au dessus de la courbe de la transformée de Hough hexagonale. Cela suggère que la transformée de Hough hexagonale parallélisée améliore le temps de détection de la transformée de Hough hexagonale pour les petites valeurs de S_H et moins rapide que la transformée de Hough hexagonale pour des valeurs de S_H plus élevés.

3.4.2 Cas de l’image de l’autoroute

La figure 4.16 présente deux graphiques comparant les performances des algorithmes de transformée de Hough hexagonale et de transformée de Hough hexagonale parallélisée. Les graphiques comparent les variations du temps d’exécution en fonction de différents paramètres (α et S_H). Dans le premier graphique, nous comparons les variations du temps d’exécution en fonction du paramètre α , avec des valeurs allant de 10% à 30%. Pour des valeurs de α plus basses (entre 10% et 25%), la courbe de la transformée de Hough hexagonale parallélisée se situe en dessous de celle de la transformée de Hough hexagonale. Cela suggère que la méthode de la transformée de Hough hexagonale est plus performante pour les petites valeurs de α . En revanche, pour les valeurs élevées de α (au moins 30%), la courbe de la transformée de Hough hexagonale parallélisée passe au-dessus de celle de la transformée de Hough hexagonale. Cela suggère que la transformée de Hough hexagonale parallélisée est moins rapide pour les élevées valeurs de α .

Dans le deuxième graphique, les variations du temps d’exécution sont comparées en fonction de

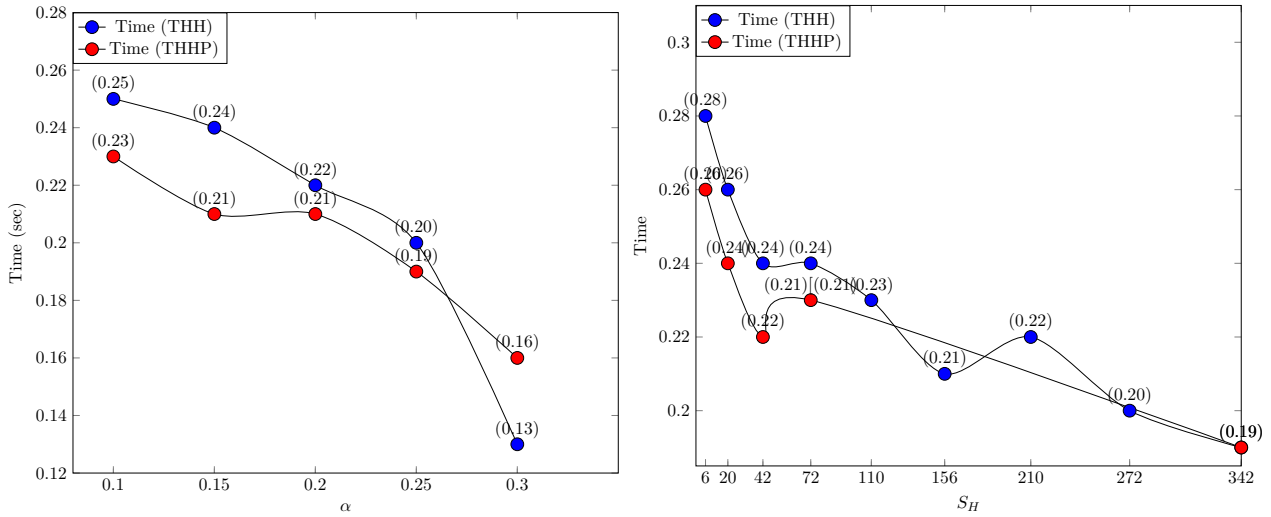


FIGURE 4.16 – Cas de l’autoroute : (à gauche) Comparaison des variations du temps d’exécution en fonction de α et les paramètres (seuil=60, $\gamma = 3$), (à droite) Comparaison des variations du temps d’exécution en fonction de la surface des cellules et les paramètres (seuil=60, $\alpha = 10\%$) des l’algorithmes 20 et 24

la surface des cellules hexagonales, avec les deux algorithmes de THH et THHP. Les surfaces des cellules varient de 6 à 342 px^2 . La courbe de la transformée de Hough hexagonale parallélisée est en dessous de celle de la transformée de Hough hexagonale pour les surfaces inférieures à 110 px^2 . Après 110 px^2 , la courbe de la transformée de Hough hexagonale parallélisée et de la transformée de Hough hexagonale sont entrelacées. Cela suggère que la transformée de Hough hexagonale parallélisée est plus performante que la transformée de Hough hexagonale pour les petites valeurs de S_H , c’est-à-dire les cellules de petite taille.

Ainsi, l’analyse comparative entre la transformée de Hough hexagonale et sa version parallélisée met en lumière plusieurs conclusions pertinentes. Dans les deux cas d’étude examinés, à savoir l’immeuble et l’autoroute, nous observons des tendances similaires quant à l’impact de la parallélisation sur les performances. Pour des valeurs de paramètres α et S_H (surface des cellules hexagonales) relativement faibles, la THHP démontre une amélioration du temps d’exécution par rapport à la THH. Cependant, cette tendance s’inverse pour des valeurs de paramètres plus élevées, où la transformée de Hough hexagonale prend l’avantage. La parallélisation de la transformée de Hough hexagonale n’affecte pas la précision de la détection.

En considérant ces observations, il est recommandé d’utiliser la transformée de Hough hexagonale lorsque des seuils α plus élevés sont privilégiés ou lorsque la taille des cellules est importante. En revanche, la transformée de Hough hexagonale parallélisée est préférable pour des seuils α plus bas et des tailles de cellules modérées. Le choix entre les deux dépendra donc des valeurs requises de α et S_H pour la détection.

Conclusion

L'étude approfondie de la transformée de Hough hexagonale révèle son potentiel en tant qu'alternative prometteuse à la méthode de la Transformée de Hough Standard pour la détection de lignes droites dans les images.

Les simulations ont démontré que la transformée de Hough hexagonale offre une amélioration en termes de temps de traitement et en termes de précision de détection des lignes. Après ajustement des valeurs de γ , α et le seuil de vote, nous avons pu observer une réduction du temps d'exécution tout en maintenant une détection acceptable des lignes droites. Aussi, la réduction du nombre de pixels à calculer à travers la variable α permet des économies computationnelles. Les cellules de grandes tailles offre une précision acceptable et un temps de traitement plus rapide. De plus, notre analyse de la complexité temporelle a montré que la transformée de Hough hexagonale présente une complexité similaire à celle de la transformée de Hough standard, ce qui confirme sa faisabilité pratique pour des applications en temps réel.

L'analyse comparant la transformée de Hough hexagonale et sa version parallélisée révèle que pour des valeurs relativement faibles des paramètres α et S_H (surface des cellules hexagonales), la THHP montre une réduction du temps d'exécution par rapport à la THH. Toutefois, cette tendance s'inverse lorsque les valeurs de ces paramètres augmentent, favorisant alors la transformée de Hough hexagonale. Notamment, il est important de noter que la parallélisation de la transformée de Hough hexagonale n'affecte pas la précision de la détection.

Cependant, malgré ses avantages, la transformée de Hough hexagonale n'est pas exempte de limitations. Ce sont : la perte d'informations présentes dans l'ensemble complet des pixels allumés des cellules hexagonales, la sensibilité aux taux α de pixels allumés et à la taille des cellules hexagonales, la complexité de sa mise en œuvre et la dépendance au contexte. Dans le chapitre suivant, nous aborderons la transformée de Hough octogonale.

Transformée de Hough Octogonale

Introduction	107
1 Description de la méthode	108
2 Complexité de la transformée de Hough octogonale	113
3 Simulations et discussions	115
Conclusion	126

Introduction

L'analyse et la détection de structures linéaires dans les images jouent un rôle crucial dans de nombreuses applications, allant de la vision par ordinateur à la robotique en passant par la cartographie. Parmi les techniques les plus utilisées pour cette tâche, la Transformée de Hough (TH) a longtemps été un pilier fondamental. Cependant, l'adaptation de la transformée de Hough à des besoins spécifiques et la recherche de performances optimales sont toujours des défis importants.

Dans cette analyse, nous explorons une variante novatrice de la transformée de Hough, connue sous le nom de transformée de Hough octogonale (THO), qui vise à surmonter certaines des limitations de l'approche standard. La transformée de Hough octogonale propose une approche basée sur une grille octogonale virtuelle, offrant ainsi une plus grande flexibilité dans la manipulation des paramètres et une réduction potentielle de temps de traitement par rapport à la THS.

Dans ce chapitre, nous examinons en détail l'algorithme de la transformée de Hough octogonale, en mettant en évidence ses principes fondamentaux, sa méthodologie de détection de lignes, et les mécanismes qui sous-tendent son fonctionnement. Nous explorons également les implications pratiques de la transformée de Hough octogonale à travers des simulations sur une image d'immeuble et une image d'autoroute de la figure 1.11, en comparant ses performances avec celles de la transformée de

Hough. Aussi, nous proposons une version parallélisée de la transformée de Hough octogonale dans l’optique de réduire d’avantage le temps de traitement.

Notre analyse se concentre sur plusieurs aspects clés, notamment la flexibilité des paramètres, la réduction du temps de calcul, la qualité de la détection et les perspectives d’amélioration. En examinant ces aspects, nous visons à évaluer le potentiel de la THO en tant qu’approche alternative pour la détection de lignes dans les images, et à identifier les domaines où elle pourrait offrir des avantages par rapport aux méthodes.

1 Description de la méthode

Dans cette section, nous présentons en détail la technique de la transformée de Hough octogonale dont les étapes sont décrites dans la figure 5.4. Nous commençons par décrire l’algorithme de maillage octogonal, qui permet de diviser une image en cellules régulières pour une analyse plus précise. Ensuite, nous abordons le processus de sélection des cellules en fonction du taux de pixels activés à l’intérieur de chaque cellule, suivi de la reconnaissance des droites discrètes grâce à la transformée de Hough octogonale. Chaque algorithme est accompagné d’explications détaillées, d’illustrations visuelles et de formules mathématiques pour une compréhension complète du processus. Enfin, nous présentons une méthode pour visualiser graphiquement les droites identifiées, facilitant ainsi l’interprétation des résultats obtenus.

1.1 Maillage de l’image

L’algorithme 27 est l’algorithme de maillage octogonale. Il prend en paramètre les dimensions de l’image (Nl : la hauteur de l’image, Nc : la largeur de l’image), la taille γ de la cellule octogonale et retourne un tableau de 3 entiers dont la première valeur est la taille de la cellule octogonale et les deux dernières valeurs sont les résidus respectivement de la hauteur et de la largeur de l’image.

Une cellule octogonale (figure 5.1) est localisée par ses huit (8) sommets A, B, C, D, E, F, G et H. Les coordonnées de chaque sommet est fonction de γ comme illustré dans la deuxième boucle itérative ”pour” de l’algorithme 30.

1.2 Sélection des cellules

Le critère de sélection d’une cellule octogonale au sein de cette grille repose sur le taux de pixels activés à l’intérieur de cette cellule octogonale particulier. Ce taux est déterminé à l’aide de l’algorithme 29, qui vise à calculer le taux de pixels allumés dans une région octogonale délimitée par les points spécifiés. Il prend en entrée une image (*img*) et les coordonnées des points délimitant l’octogone (A, B, D, E, F, G, H, I, J). L’algorithme initialise deux compteurs, k et l , qui représentent respectivement

Algorithme 27 : Construction de la grille octogonale

Fonction *maillage_oct*(*image*, γ : entiers) : tableau d'entiers

γ représente la taille des mailles octogonales;

Variables : *tab* : tableau de 3 entiers ;

Sorties : *tab* : tableau de 3 entiers;

Debut

$Nl \leftarrow$ la hauteur de l'image;

$Nc \leftarrow$ la largeur de l'image ;

$tab[0] \leftarrow \gamma$; % La taille de l'octogone.

$tab[1] \leftarrow Nl \bmod (3\gamma - 2)$;

$tab[2] \leftarrow Nc \bmod (3\gamma - 2)$;

Retourner : *tab*;

fin

fin

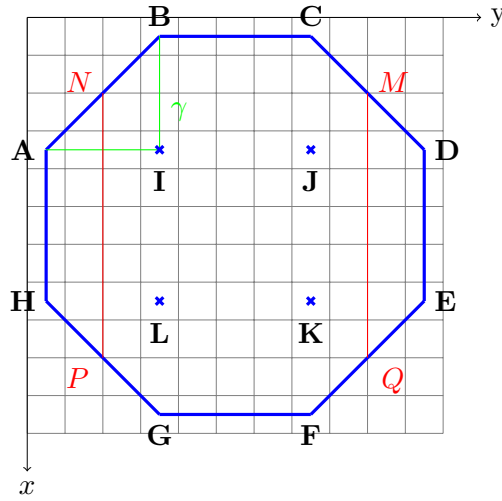


FIGURE 5.1 – Octogone ABCDEFGH

le nombre de pixels allumés et éteints dans la région octogonale. En parcourant les lignes de l'image, il examine les pixels à l'intérieur de l'octogone et calcul le pourcentage ou le taux de pixels allumés dans la cellule octogonale. Finalement, l'algorithme retourne le taux de pixels allumés dans la région octogonale, calculé comme la division de k par la somme de k et l . Ce taux donne une mesure de la proportion de pixels allumés par rapport au total dans la cellule octogonale. Le volume d'une cellule octogonale est calculé selon le lemme 3.

Lemme 3 (L'aire d'une cellule octogonale) Soit γ la taille d'un octogone. La surface de l'octogone, représentée par la partie verte de la figure 5.3, est calculée à l'aide de la formule suivante :

$$S_O(\gamma) = 7\gamma^2 - 4\gamma + 1 \quad (5.1)$$

Preuve 3 (lemme 3) Considérons γ comme la taille de l'octogone (5.1).

Calcul de la surface $S_c(\gamma)$ du carré dans lequel l'octogone est inscrit :

$S_r(\gamma) = (3\gamma - 1)^2$ où $(3\gamma - 1)$ représentent le coté du carré.

Calcul de la surface $S_t(\gamma)$ des quatre triangles rectangles (dont les angles droits correspondent aux sommets du carré) : $S_t(\gamma) = 4(\gamma^2 - \frac{\gamma^2+\gamma}{2})$ où $\frac{\gamma^2+\gamma}{2}$ représente la surface du triangle ABI .

Calcul de la surface de l'octogone : $S_O(\gamma) = S_c(\gamma) - S_t(\gamma)$. Ce qui donne, après développement et simplification : $S_O(\gamma) = 7\gamma^2 - 4\gamma + 1$

Par exemple, pour $\gamma = 4$, $S_O(4) = 7 \times 4^2 - 4 \times 4 + 1 = 97\text{px}^2$, comme illustré dans la figure 5.3. L'unité de mesure de la surface est en pixels carrés (px^2).

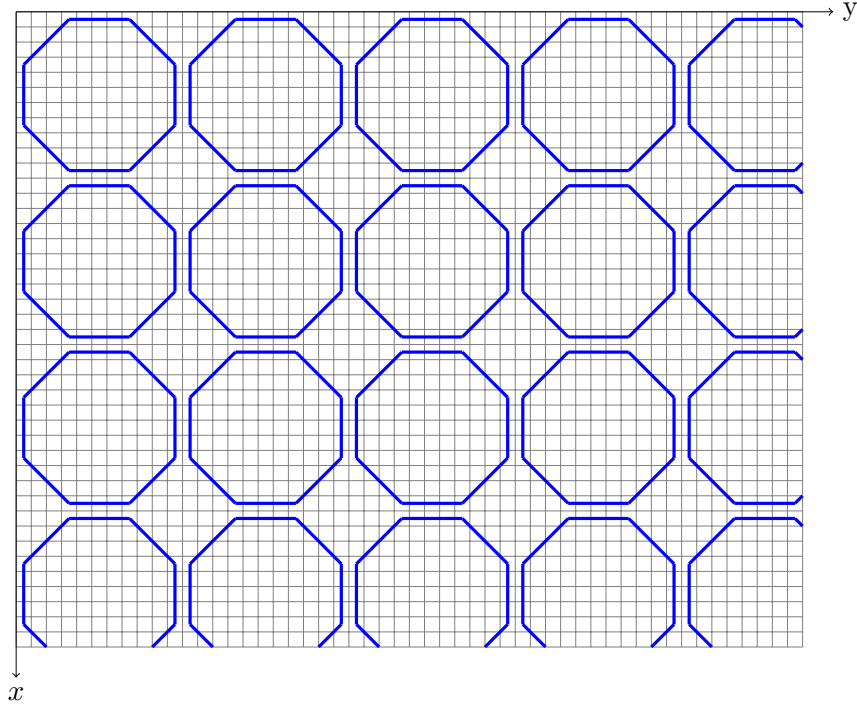


FIGURE 5.2 – Grille octogonale superposée à l'espace image.

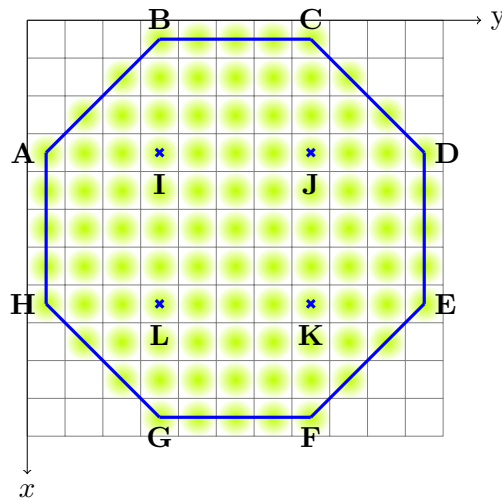


FIGURE 5.3 – Mise en évidence des pixels de l'octogone ABCDEFGH

Algorithme 28 : Mise à jour de l'accumulateur

```
Procedure Acc_update() : vide
    % Mise à jour de l'accumulateur
    if img[z, t]  $\neq$  0 then
        for itheta dans range(accum_row) do
            theta  $\leftarrow$  itheta  $\times$  dtheta;
            rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
            irho  $\leftarrow$  int(rho);
            if irho > 0 et irho < accum_column then
                accum[itheta][irho]  $\leftarrow$  accum[itheta][irho] + 1
            end
        end
    end
fin
```

1.3 Parallélisation de la transformée de Hough octogonale

La parallélisation de la transformée de Hough octogonale nécessite une adaptation de l'algorithme pour tirer parti des capacités de traitement simultané offertes par le parallélisme. La parallélisation concerne l'accumulateur c'est-à-dire l'espace des paramètres encore appelé l'espace de Hough. Les algorithmes 32 et 31 sont les versions parallélisées des algorithmes 30 et 28. Les étapes principales de la parallélisation sont :

- importation de la bibliothèque *pypm*⁵ : Importer une bibliothèque en Python est nécessaire pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction *import pypm* est utilisée pour charger la bibliothèque *pypm*.
- création de l'accumulateur de type *pypm* : Dans l'algorithme séquentiel, l'espace des paramètres (l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la transformée de Hough, est transformé en une structure de données partagée grâce à *pypm*. En effet, *pypm* permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.

L'instruction "*accum_pypm* $\leftarrow$ *pypm.shared.array([accum_row, accum_column])*" de l'algorithme 32 crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable "*accum_pypm*".

- construction parallèle : L'instruction "*with pypm.Parallel(4) as p*" de l'algorithme 32 indique que nous souhaitons exécuter le bloc de code à l'intérieur de la déclaration "with" en parallèle avec 4 processus.
- parallélisation de l'algorithme 28 de mise à jours de l'accumulateur : La fonction "*p.range*" génère une séquence d'indices à partir de laquelle chaque cœur du CPU obtiendra une plage d'indices spécifique. Cela permet de diviser le travail entre les cœurs du CPU de manière équitable. Ainsi la boucle "for

⁵. <https://github.com/classner/pypm/tree/main>

Algorithme 29 : Taux de pixels allumes

```

Fonction count_oct(img : image, A, B, D, E, F, G, H, I, J : tableaux de deux entiers) : réel
  Variables : k, l : entiers;
  Debut
    k ← 0; l ← 0;
    pour A[1] ≤ y ≤ D[1] faire
      si A[1] ≤ y ≤ I[1] − 1 alors
        NP ← |y − A[1]|;
        pour A[0] − NP ≤ x ≤ H[0] + NP faire
          si img[x, y] ≠ 0 alors
            | k ← k + 1;
          sinon
            | l ← l + 1;
          finsi
        fin
      finsi
      si I[1] ≤ y ≤ J[1] alors
        pour B[0] ≤ x ≤ G[0] faire
          si img[x, y] ≠ 0 alors
            | k ← k + 1;
          sinon
            | l ← l + 1;
          finsi
        fin
      finsi
      si J[1] ≤ y ≤ D[1] alors
        MQ ← |y − D[1]|;
        pour D[0] − MQ ≤ x ≤ E[0] + MQ faire
          si img[x, y] ≠ 0 alors
            | k ← k + 1;
          sinon
            | l ← l + 1;
          finsi
        fin
      finsi
    fin
    Retourner :  $\frac{k}{k + l}$ ;
  fin

```

theta in *p.range(accum_row)*” de l’algorithme 31 utilise les indices compris entre 0 et *accum_row* − 1 pour itérer à travers l’accumulateur par sections, où chaque section est traitée par un processus différent.

1.4 Reconnaissance des droites discrètes

L’algorithme 30 de la ”Reconnaissance de droites discrètes par la transformée de Hough octogonale” utilise une approche octogonale pour détecter des droites discrètes dans une image. La fonction principale, appelée *Octogonal*, prend en entrée une image (*img*), un paramètre γ représentant la taille de l’octogone, un paramètre α représentant le taux de pixels allumés et le seuil de vote.

L’algorithme commence par initialiser les variables nécessaires, notamment les tableaux pour les points de l’octogone (*A*, *B*, *C*, *D*, *E*, *F*, *G*, *H*, *I*, *J*, *K*, *L*), les dimensions de l’image, et la matrice d’accumulateur. Ensuite, il effectue un maillage octogonal de l’image à l’aide de la fonction *maillage_oct* (Algorithme 27).

Le processus de détection des droites discrètes se fait en parcourant les octogones de la grille générée. Pour chaque octogone, il vérifie s’il y a suffisamment de points allumés dans la région correspondante et met à jour l’accumulateur en conséquence.

Le cœur de l’algorithme se trouve dans la double boucle *for* qui parcourt les positions des octogones dans l’image. Pour chaque position, il calcule les coordonnées des points de l’octogone, vérifie le nombre de points allumés à l’intérieur de l’octogone, et met à jour l’accumulateur. La mise à jour de l’accumulateur se fait à l’aide de l’algorithme 28.

Algorithme 30 : Reconnaissance de droites discrètes par la transformée de Hough octogonale

```

Fonction Octogonal(image,  $\gamma, \alpha, \text{seuil}$ ) : tableau
Pre-condition :  $1 \leq \gamma \leq \min(\frac{Nl+1}{2}, \frac{Nc+1}{2})$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
Donnees :  $\alpha$ 
Variables : tab : tableau de 3 entiers; A, B, C, D, E, F, G, H, I, J, K, L : tableaux de deux entiers;
accum_ligne, accum_colon, irho, Nl, Nc : entier; accum : matrice accumulateur ;
dtheta, drho, rho, theta, DE : réels;
Debut
    % dimension de l'image
    Nl  $\leftarrow$  la hauteur de l'image; Nc  $\leftarrow$  la largeur de l'image;
    % dimensions de la matrice 'accum'
    accum_ligne  $\leftarrow$  180; accum_colon  $\leftarrow E[\sqrt{(Nc^2 + Nl^2)}]$ ;
    dtheta =  $\pi/180$ ;
    tab  $\leftarrow$  maillage_oct(Nl, Nc,  $\gamma$ ) ;
    Ht = Nl - tab[1]; La = Nc - tab[2];
    H1 = Ht + 3 * gamma - 2; L1 = La + 3 * gamma - 2;
    pour  $3\gamma - 2 \leq y \leq L1$  par pas de  $3\gamma - 1$  faire
        pour  $3\gamma - 2 \leq x \leq H1$  par pas de  $3\gamma - 1$  faire
            A[0] =  $x - 2tab[0] + 1$ ; A[1] =  $y - 3tab[0] + 2$ ; B[0] =  $x - 3tab[0] + 2$ ; B[1] =  $y - 2tab[0] + 1$ ; C[0] =  $x - 3tab[0] + 2$ ;
            C[1] =  $y - tab[0] + 1$ ; D[0] =  $x - tab[0] + 1$ ; D[1] =  $y$ ;
            E[0] =  $x - tab[0] + 1$ ; E[1] =  $y$ ; F[0] =  $x$ ; F[1] =  $y - tab[0] + 1$ ; G[0] =  $x$ ; G[1] =  $y - 2tab[0] + 1$ ;
            H[0] =  $x - tab[0] + 1$ ; H[1] =  $y - 3tab[0] + 2$ ;
            I[0] =  $x - 2tab[0] + 1$ ; I[1] =  $y - 2tab[0] + 1$ ; J[0] =  $x - 2tab[0] + 1$ ; J[1] =  $y - tab[0] + 1$ ;
            K[0] =  $x - tab[0] + 1$ ; K[1] =  $y - tab[0] + 1$ ; L[0] =  $x - tab[0] + 1$ ; L[1] =  $y - 2tab[0] + 1$ ;
            si comptage(img, A, B, D, E, F, G, H, I, J)  $\geq \alpha$  alors
                pour A[1]  $\leq j \leq D[1]$  faire
                    si A[1]  $\leq j \leq I[1] - 1$  alors
                        NP  $\leftarrow |j - A[1]|$ ;
                        pour A[0] - NP  $\leq i \leq H[0] + NP$  faire
                            Acc.update();
                        fin
                    finsi
                    si I[1]  $\leq j \leq J[1]$  alors
                        pour B[0]  $\leq i \leq G[0] + 1$  faire
                            Acc.update();
                        fin
                    finsi
                    si J[1]  $\leq j \leq D[1]$  alors
                        MQ  $\leftarrow |j - D[1]|$ ;
                        pour D[0] - MQ  $\leq i \leq E[0] + MQ$  faire
                            Acc.update();
                        fin
                    finsi
                fin
            finsi
        fin
    fin
    Rechercher les maxima dans l'accumulateur et placer les couple ( $\rho, \theta$ ) dans une liste 'lignes';
    Retourner la liste 'lignes';
fin

```

Enfin, il retourne une liste appelée 'lignes' contenant les couples $(\theta; \rho)$ correspondant aux paramètres des droites détectées.

L'objectif de l'algorithme 33 est de représenter graphiquement les droites identifiées par la méthode de la Transformée de Hough. Il nécessite en entrée une image (*image*), la liste des paramètres de droites détectées (*lignes*) générée par l'algorithme 30, ainsi qu'une couleur (*colors*). L'algorithme itère à travers la liste des paramètres de droites détectées, reconstruisant chaque droite correspondante sur l'image. Les droites reconstruites sont alors colorées avec la nuance spécifiée par *colors*.

Cette étape de reconstruction vise à mettre en évidence les droites identifiées de manière visuelle sur l'image originale. Cela facilite l'interprétation et l'analyse des résultats obtenus grâce à la Transformée de Hough.

2 Complexité de la transformée de Hough octogonale

La fonction *maillage_oct* dans l'algorithme 27 récupère les dimensions d'une image et assigne des valeurs à un tableau, puis retourne ce tableau. Chaque opération est de coût constant, donc la

Algorithme 31 : Mise à jour parallèle de l'accumulateur

```
Procedure Acc_updateP() : vide
  % Mise à jour de l'accumulateur
  if img[z, t]  $\neq$  0 then
    for itheta dans p.range(accum_row) do
      theta  $\leftarrow$  itheta  $\times$  dtheta;
      rho  $\leftarrow$  t  $\times$  cos(theta) + z  $\times$  sin(theta);
      irho  $\leftarrow$  int(rho);
      if irho > 0 et irho < accum_column then
        | accum_pymp[itheta][irho]  $\leftarrow$  accum_pymp[itheta][irho] + 1
      end
    end
  end
fin
```

complexité totale de la fonction est $O(1)$.

La fonction *count_oct* dans l'algorithme 29 : initialise des variables *k* et *l*, puis elle parcourt les pixels à l'intérieur de l'octogone à l'aide de boucles dont le nombre d'itérations est $7\gamma^2 - 4\gamma + 1$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité du bloc formé par les boucles est $O(7\gamma^2 - 4\gamma + 1) = O(\gamma^2)$, où $7\gamma^2 - 4\gamma + 1$ est le nombre de pixels à l'intérieur de l'octogone. Ainsi, la complexité totale de la fonction *count_oct* est $O(\gamma^2)$.

La fonction *Acc_update* dans l'algorithme 28 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de *accum_row* = 180, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction *Acc_update* est $O(1)$.

L'algorithme 30 de la transformée de Hough octogonale (THO) commence par l'initialisation des variables et du tableau *tab*, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction *maillage_oct* dont la complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux boucles imbriquées parcourant les points à l'intérieur de chaque octogone à l'intérieur du quelle se trouve des opérations de coût constant et la fonction *Acc_update* de complexité $O(1)$. La complexité de ces boucles est donc $O(7\gamma^2 - 4\gamma + 1) = O(\gamma^2)$.

Le coût des deux boucles principales imbriquées parcourant les hexagones de la grille, dont le nombre total d'itérations dépend de la taille de l'image et de la taille des cellules hexagonales, contenant un bloc d'opérations d'affectation de coût constant, un test de condition sur la fonction *count_oct* de complexité $O(\gamma^2)$, les deux boucles imbriquées de coût $7\gamma^2 - 4\gamma + 1$ parcourant les pixels à l'intérieur de l'hexagone, est $Nl/(3\gamma - 1) \times Nc/(3\gamma - 1) \times (1 + (7\gamma^2 - 4\gamma + 1) + 1 + (7\gamma^2 - 4\gamma + 1))$, donc de complexité $O(Nl/(3\gamma - 1) \times Nc/(3\gamma - 1) \times (1 + (7\gamma^2 - 4\gamma + 1) + 1 + (7\gamma^2 - 4\gamma + 1))) = O(Nl \times Nc)$.

En considérant ces points, la complexité totale de l'algorithme 30 de la THO est égal à $O(Nl \times Nc)$,

Algorithme 32 : Reconnaissance de droites discrètes par la Transformée de Hough Octogonale Parallélisée

```

Fonction OctogonalP(image,  $\gamma$ ,  $\alpha$ , Threshold, cpu_count) : tableau
  Pre-condition :  $1 \leq \gamma \leq \min(\frac{Nl+1}{2}, \frac{Nc+1}{2})$ ,  $0 \leq \alpha \leq 1$ ,  $0 < \text{seuil}$ 
  Variables : tab : tableau de 3 entiers; A, B, C, D, E, F, G, H, I, J, K, L : tableaux de deux entier;
  accum_row, accum_column, irho, Nl, Nc : entier; accum : matrice accumulateur ;
  dtheta, drho, rho, theta, DE : reels;
  Debut
    import pypm % importation de la bibliothèque pypm % dimension de l'image
    Nl  $\leftarrow$  la hauteur de l'image; Nc  $\leftarrow$  la largeur de l'image;
    % dimensions de la matrice 'accum'
    accum_row  $\leftarrow$  180; accum_column  $\leftarrow E[\sqrt{(Nc^2 + Nl^2)}]$ ;
    accum_pypm  $\leftarrow$  pypm.shared.array([accum_row, accum_column]); % creation de l'accumulateur de type pypm
    dtheta =  $\pi/180$ ; tab  $\leftarrow$  maillage_oct(Nl, Nc,  $\gamma$ ) ;
    Ht = Nl - tab[1]; La = Nc - tab[2];
    H1 = Ht + 3 * gamma - 2; L1 = La + 3 * gamma - 2;
    % Démarrage du traitement parallèle.
    With pypm.parallel(cpu_count) as p :
      pour  $3\gamma - 2 \leq y \leq L1$  par pas de  $3\gamma - 1$  faire
        pour  $3\gamma - 2 \leq x \leq H1$  par pas de  $3\gamma - 1$  faire
          A[0] = x - 2tab[0] + 1; A[1] = y - 3tab[0] + 2; B[0] = x - 3tab[0] + 2; B[1] = y - 2tab[0] + 1;
          C[0] = x - 3tab[0] + 2; C[1] = y - tab[0] + 1; D[0] = x - tab[0] + 1; D[1] = y;
          E[0] = x - tab[0] + 1; E[1] = y; F[0] = x; F[1] = y - tab[0] + 1; G[0] = x; G[1] = y - 2tab[0] + 1;
          H[0] = x - tab[0] + 1; H[1] = y - 3tab[0] + 2;
          I[0] = x - 2tab[0] + 1; I[1] = y - 2tab[0] + 1; J[0] = x - 2tab[0] + 1; J[1] = y - tab[0] + 1;
          K[0] = x - tab[0] + 1; K[1] = y - tab[0] + 1; L[0] = x - tab[0] + 1; L[1] = y - 2tab[0] + 1;
          si comptage(img, A, B, D, E, F, G, H, I, J)  $\geq \alpha$  alors
            pour A[1]  $\leq j \leq D[1] faire
              si A[1]  $\leq j \leq I[1] - 1 alors
                NP  $\leftarrow |j - A[1]|$ ;
                pour A[0] - NP  $\leq i \leq H[0] + NP faire
                  Acc.updateP();
                fin
              finsi
              si I[1]  $\leq j \leq J[1] alors
                pour B[0]  $\leq i \leq G[0] + 1 faire
                  Acc.updateP();
                fin
              finsi
              si J[1]  $\leq j \leq D[1] alors
                MQ  $\leftarrow |j - D[1]|$ ;
                pour D[0] - MQ  $\leq i \leq E[0] + MQ faire
                  Acc.updateP();
                fin
              finsi
            fin
          finsi
        fin
      fin
    fin
  Rechercher les maxima dans l'accumulateur et placer les couple ( $\rho, \theta$ ) dans une liste 'lignes';
  Retourner la liste 'lignes';
fin$$$$$$$ 
```

Algorithme 33 : Reconstruction des droites détectées

```

Fonction PlotHoughLine(image, lignes, colors) : image
  Parcourir 'lignes' la liste des paramètres des droites détectés, puis construire chaque
  droite correspondant en couleur 'colors' sur l'image 'image'.
fin

```

où *Nl* et *Nc* sont le nombre de lignes et de colonnes de l'image respectivement, γ est la taille des cellules octogonale. Ce qui correspond à la complexité de la THS.

3 Simulations et discussions

La présente section se concentre sur les résultats de simulations obtenus à partir de l'application de l'algorithme de la transformée de Hough octogonale sur des images représentatives. Nous examinons en détail les performances de la transformée de Hough octogonale dans deux scénarios distincts : la détection de lignes dans une image d'immeuble et dans une image d'autoroute. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

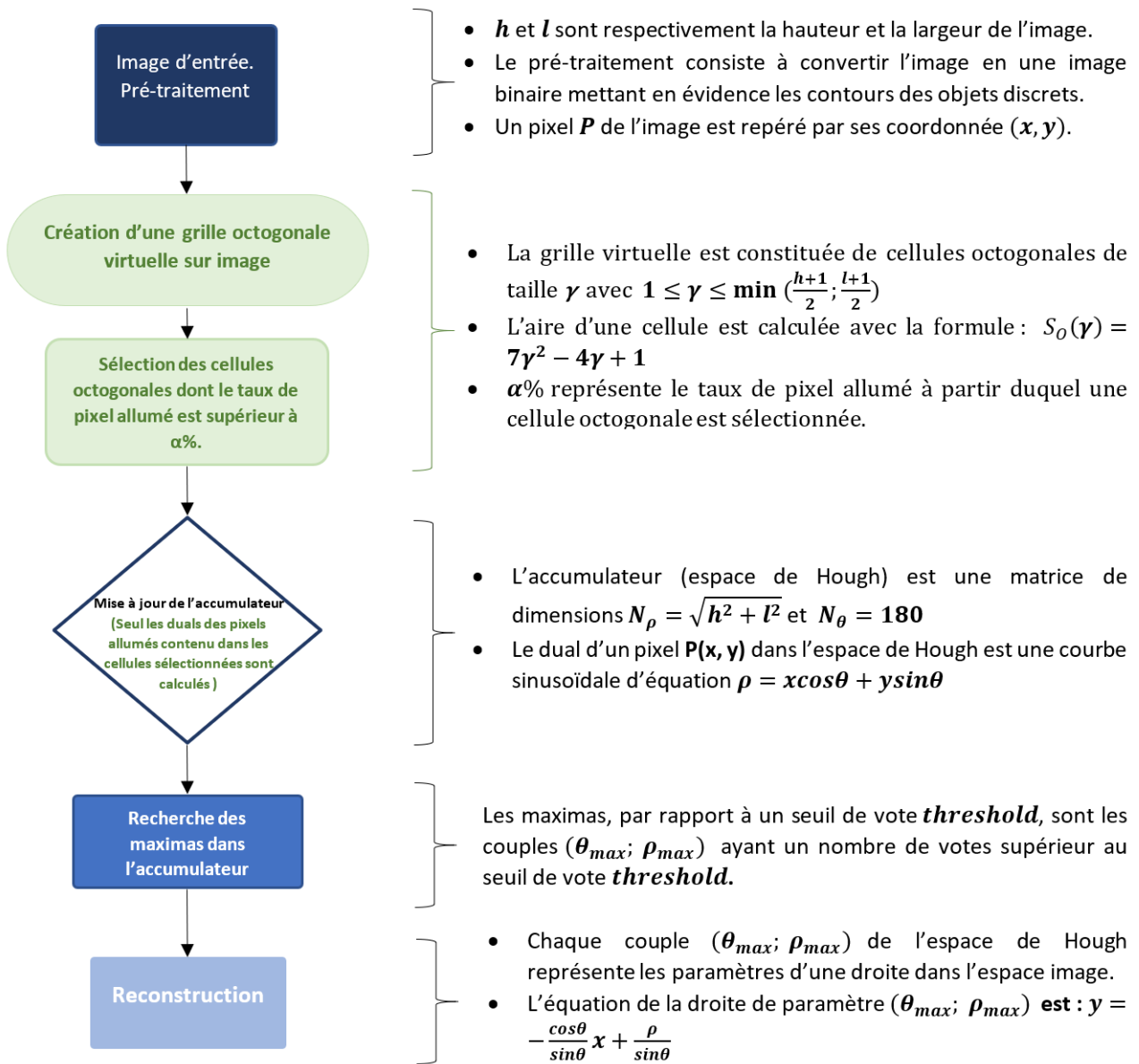


FIGURE 5.4 – Les étapes de la technique de la transformée de Hough octogonale

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Pour chaque scénario, nous explorons les variations des paramètres clés de la transformée de Hough octogonale, tels que le seuil de vote, α et γ , afin d'évaluer leur impact sur la précision et l'efficacité de la détection. De plus, nous comparons les résultats de la transformée de Hough octogonale avec ceux de la transformée de Hough standard dans le contexte de la détection de lignes. Enfin, nous présentons une analyse comparative entre la transformée de Hough octogonale et sa version parallélisée (THOP), mettant en lumière les avantages et les limites de chacune des approches. Ces discussions visent à fournir un aperçu complet des performances de la transformée de Hough octogonale dans différents contextes d'application et à identifier les meilleures pratiques pour son utilisation efficace.

TABLEAU 5.1 – Résultats des simulations sur l’image 1.11(c) de l’immeuble avec l’algorithme 30 de la THO avec les paramètres fixes ($\gamma = 3$ et $Seuil = 60$) et α variant entre 10% et 35%

$\alpha(\%)$	Nbre de droites détectées	temps(sec)
10	101	0,42
15	65	0,39
20	17	0,31
25	9	0,25
30	2	0,17
35	0	0,10

3.1 Cas de l’image de l’immeuble

Dans cette section, nous examinons les performances de l’algorithme de détection de lignes de la transformée de Hough octogonale appliqué à une image représentant un immeuble. L’objectif est d’explorer l’impact des paramètres clés de l’algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l’algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l’algorithme de la transformée de Hough octogonale en fonction des caractéristiques de l’image de l’immeuble.

3.1.1 Variations de α

Le tableau 5.1 présente les résultats de simulations sur l’image 1.11(c) de l’immeuble en utilisant l’algorithme 30 de la transformée de Hough octogonale. Les simulations ont été réalisées avec des paramètres fixes ($\gamma = 3$ et $Seuil = 60$), tandis que le paramètre α a été modifié entre 10% et 35%. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 35%. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

Deux graphiques sont fournis sur la figure 5.5 : l’un représentant le nombre de lignes détectées en fonction de α , et l’autre représentant le temps de traitement en fonction de α . Dans le premier graphique, on observe une diminution du nombre de lignes détectées à mesure que α augmente. Cela suggère que l’augmentation de α conduit à une détection moins précise des lignes. Dans le deuxième graphique, le temps de traitement diminue à mesure que α augmente. Cela indique que des valeurs plus élevées de α conduisent à une exécution plus rapide de l’algorithme.

Une faible valeur de α (10%) donne un grand nombre de lignes détectées (101), mais cela nécessite plus de temps de traitement (0,42 sec). À mesure que α augmente, le nombre de lignes détectées diminue, le temps de traitement diminue également. Des valeurs trop élevées de α peuvent entraîner une détection défectueuse.

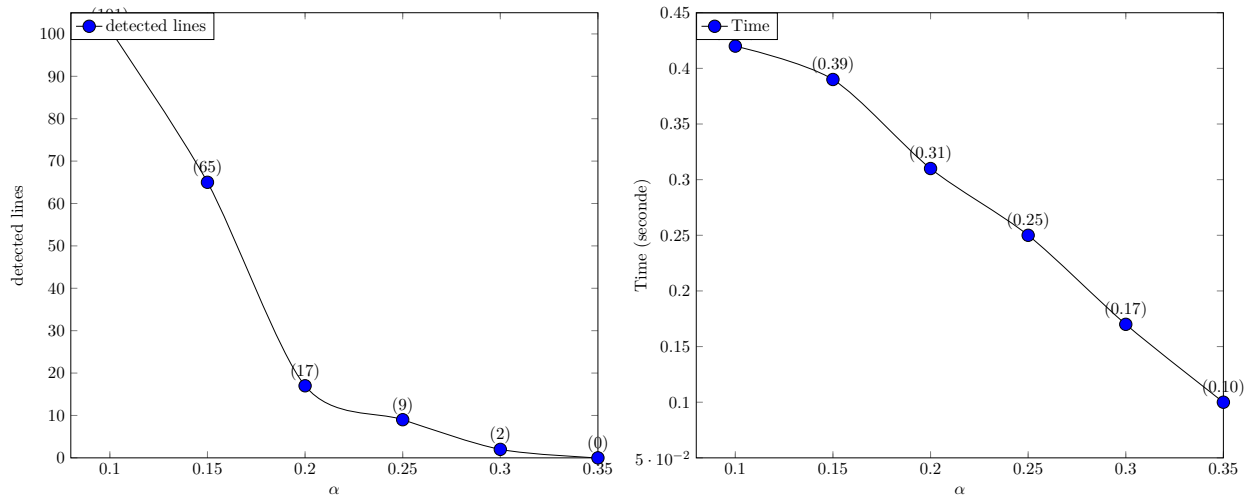


FIGURE 5.5 – Courbe représentative des données du tableau 5.1

TABLEAU 5.2 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres fixes ($Seuil = 50$ et $\alpha = 20\%$) et γ variant entre 2 et 82

γ	S_O	droites détectées	temps(sec)
2	21	299	0,39
3	52	70	0,31
4	97	73	0,28
5	156	15	0,22
6	229	3	0,20
7	316	1	0,16
8	417	0	0,14
82	46 741	0	0,14

α	γ	$S_O(\gamma)$	Nbre de droites détectées	temps(sec)
11	54	20197	31	0,22

3.1.2 Variations de la taille des cellules

Le tableau présente les résultats des simulations sur l'image 1.11(c) de l'immeuble en utilisant l'algorithme 30 de la transformée de Hough octogonale. Les paramètres fixes sont $Seuil = 50$ et $\alpha = 20\%$, tandis que γ varie entre 2 et 8. Les colonnes incluent γ , la surface octogonale S_O , le nombre de lignes détectées, et le temps en secondes.

Les deux courbes présentées dans la figure 5.6 sont les représentations graphique des données du tableau 5.2. Le premier représente le nombre de lignes détectées en fonction de la surface octogonale, et le second représente le temps de traitement en fonction de la surface octogonale. Dans le premier graphique, on observe une tendance décroissante du nombre de lignes détectées pour les valeurs de γ allant de 2 à 82. Par contre, lorsque nous baissions la valeur de α à 11%, 31 droites est détectées avec $\gamma = 54$, suggérant ainsi que l'augmentation de la taille des cellules n'entraîne pas toujours la diminution du nombre de droites détectées. Dans le second graphique, le temps de traitement diminue également à mesure que la surface octogonale augmente, indiquant une exécution plus rapide de l'algorithme pour des valeurs plus élevées de γ .

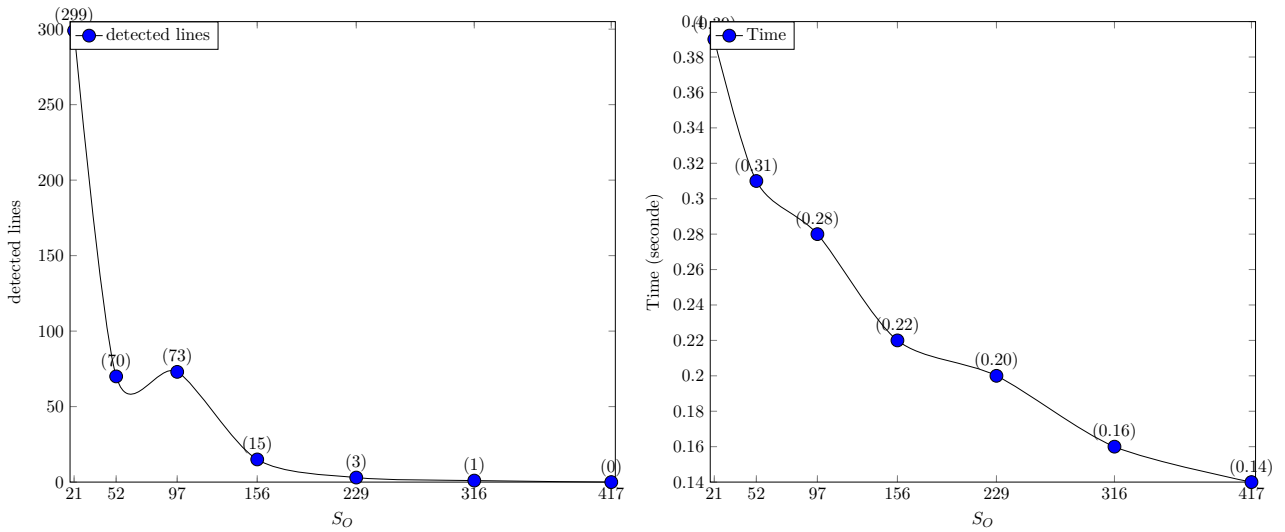


FIGURE 5.6 – Courbe représentative des données du tableau 5.2

3.2 Cas de l'image de l'autoroute

Dans cette section, nous examinons les performances de l'algorithme de détection de lignes de la transformée de Hough octogonale appliqué à une image représentant une autoroute. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la transformée de Hough octogonale en fonction des caractéristiques de l'image de l'autoroute.

3.2.1 Variations de α

Le tableau 5.3 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute en utilisant l'algorithme 30 de la transformée de Hough octogonale. Les simulations ont été réalisées avec des paramètres fixes ($\gamma = 3$ et $Seuil = 40$), tandis que le paramètre α a été modifié entre 10% et 30%. Le tableau comporte trois colonnes : la première colonne représente les différentes valeurs du paramètre α , qui varie de 10% à 30%. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

Les deux courbes présentées dans la figure 5.7 sont les représentations graphique des données du tableau 5.3. Le premier graphique illustre le nombre de lignes détectées en fonction de α , tandis que le second représente le temps de traitement en fonction de α . Dans le premier graphique, on constate une diminution du nombre de lignes détectées à mesure que α augmente, ce qui suggère une détection moins fréquente avec des valeurs plus élevées de α . Cependant, une augmentation exagérée des valeurs

TABLEAU 5.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la THO avec les paramètres fixes ($\gamma = 3$ et $Seuil = 40$) et α variant entre 10% et 30%

$\alpha(\%)$	Nbre de droites détectées	temps(sec)
10	158	0,18
15	121	0.16
20	20	0,12
25	2	0,09
30	0	0,06

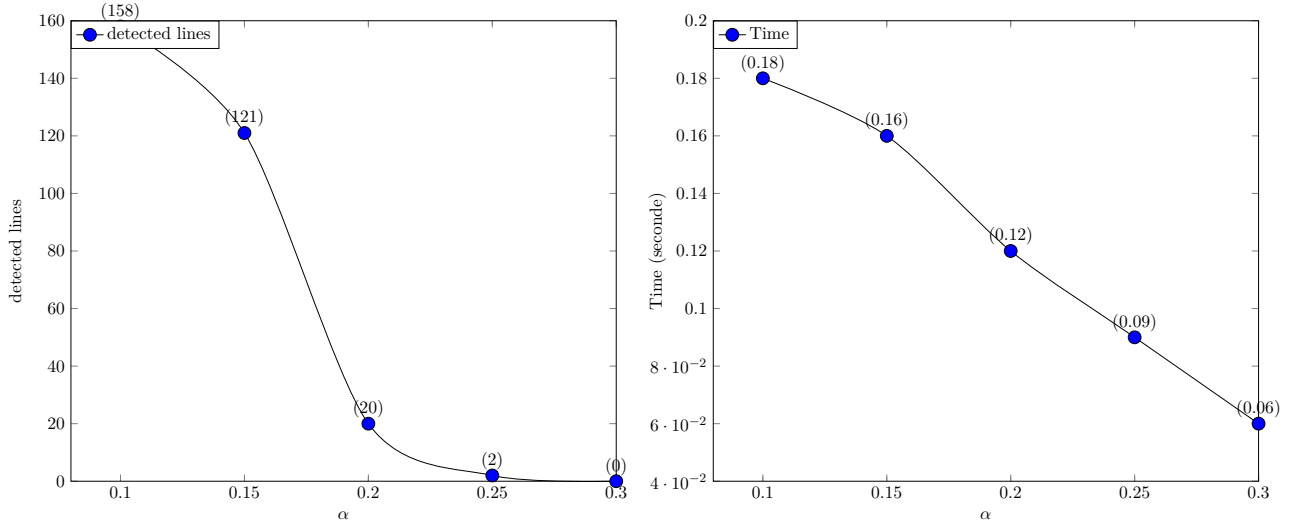


FIGURE 5.7 – Courbe représentative des données du tableau 5.3

de α peut détériorer la qualité de la détection. Dans le second graphique, le temps de traitement diminue également à mesure que α augmente, indiquant une exécution plus rapide de l'algorithme pour des valeurs plus élevées de α .

3.2.2 Variations de la taille des cellules

Le tableau 5.4 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute en utilisant l'algorithme 30 de la transformée de Hough octogonale. Les simulations ont été réalisées avec des paramètres fixes ($Seuil = 40$ et $\alpha = 20\%$) et γ variant entre 2 et 5. Le tableau 5.4 comporte quatre colonnes : la première colonne représente les différentes valeurs du paramètre γ , variant de 2 à 5. La deuxième colonne indique la surface (l'aire) S_O de la cellule octogonale en pixels carrés. La troisième colonne montre le nombre de lignes détectées pour chaque valeur de γ . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

Les deux courbes dans la figure 5.8 représentent les données du tableau 5.4. Le premier graphique illustre le nombre de lignes détectées en fonction de la surface octogonale, tandis que le second montre le temps de traitement en fonction de cette même surface. Dans le premier graphique, on observe une diminution jusqu'à 0 du nombre de lignes détectées pour γ allant de 2 à 56. Cependant, lorsque nous baissions la valeur de α à 10% par exemple, 9 droites sont détectées lorsque $\gamma = 33$. Ce qui

TABLEAU 5.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la THO avec les paramètres fixes (Seuil=40 et $\alpha = 20\%$) et γ variant entre 2 et 56

γ	$S_O(\gamma)$	Nbre de droites détectées	temps(sec)
2	21	95	0,16
3	52	20	0,12
4	97	2	0,09
5	156	0	0,08
56	21 729	0	0.04

α	γ	$S_O(\gamma)$	Nbre de droites détectées	temps(sec)
10	33	7 492	9	0.09
12	34	7 957	12	0.09

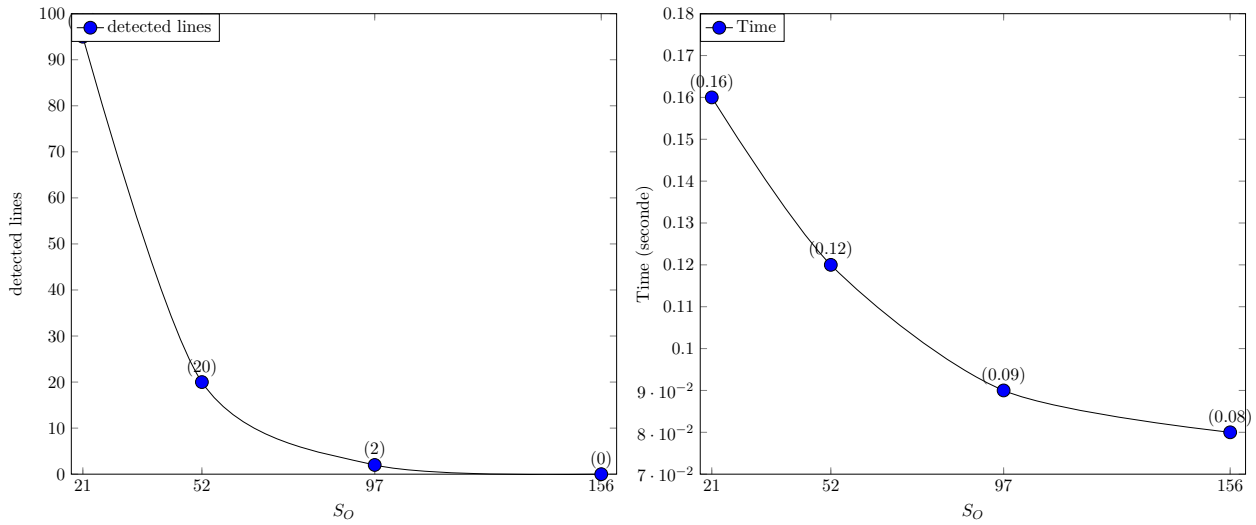


FIGURE 5.8 – Courbe représentative des données du tableau 5.4

montre que l'augmentation de la taille des cellules n'implique pas systématique la diminution du nombre de droites détectées. Dans le second graphique, le temps de traitement diminue également avec l'augmentation de la surface des cellules, indiquant une exécution plus rapide de l'algorithme pour des surfaces octogonales plus grandes.

Ainsi, ces résultats mettent en évidence l'importance des paramètres $\gamma, \alpha, seuil$ dans l'algorithme de détection de lignes de la transformée de Hough octogonale. Les cellules de grandes taille offre une précision acceptable et un temps de traitement plus rapide, en témoignent les données des tableaux 5.2 et 5.4, ainsi que les résultats visuels dans la figure 5.9.

3.3 Comparaison de la Transformée de Hough Octogonale et Transformée de Hough Standard

Nous procédons dans cette section à une comparaison de la transformée de Hough octogonale, méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard



FIGURE 5.9 – (a) THO sur l'immeuble ($\gamma = 54$, $\alpha = 11\%$, *seuil* = 50), (b) THO sur l'autoroute ($\gamma = 33$, $\alpha = 10$, *seuil* = 40)

TABLEAU 5.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

Threshold	Droites détectées	temps1(sec)	temps2(sec)
53	1991	0,6	0,0003
116	10	0,49	0,049

et la transformée de Hough octogonale sont illustré dans les tableaux 5.5 et 5.7 respectivement.

En ce qui concerne la précision, pour un seuil de 53 votes, la transformée de Hough standard a effectué une détection de 1991 droites. Cette détection est non pertinente comme le montre la première image de la figure 5.10. par contre, avec le même seuil de votes, la transformée de Hough octogonale a détecté 11 droites. Les résultats visuels illustrés par la première image de la figure 5.12 montre que les droites détectées suivent bien les lignes de l'immeuble suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough octogonale pour parvenir à ce résultat sont : la taille de cellule définit par γ et le pourcentage α de pixels allumés de chaque cellule octogonale.

Pour avoir une bonne détection de lignes droites avec la THS, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 5.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 5.10 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'immeuble indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 5.5 montre que la THS a détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La THO a détecté quasiment le même nombre de droite en 0.22 seconde soit 0.02 seconde par droite détecté comme illustré dans le tableau 5.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,49}{0,22} = 2,22$ (T_h représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).

De même avec l'image de l'autoroute, les résultats des expériences avec la THS et la THO sont illustré dans les tableaux 5.6 et 5.8 respectivement.

En ce qui concerne la précision, pour un seuil de 38 votes, la THS a effectué une détection de 751



FIGURE 5.10 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

TABLEAU 5.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

Threshold	Droites détectées	time1(sec)	time2(sec)
38	751	0,24	0,0003
100	10	0,21	0,021

droites. Cette détection est non pertinente comme le montre la première image de la figure 5.11. par contre, avec le même seuil de votes, la transformée de Hough octogonale a détecté 10 droites. Les résultats visuels illustrés par la première image de la figure 5.13 montre que les droites détectées suivent bien les lignes de l'autoroute suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough octogonale pour parvenir à ce résultat sont : la taille de cellule octogonale définit par γ et le pourcentage α de pixels allumés de chaque cellule octogonale.

Pour avoir une bonne détection de lignes droites avec la THS, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 5.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 5.11 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 5.6 montre que la THS a détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La transformée de Hough octogonale a détecté le même nombre de droite en 0.11 seconde soit 0.011 seconde par droite détecté comme illustré dans le tableau 5.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h} = \frac{0,21}{0,11} = 1,9$ (T_h représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).

TABLEAU 5.7 – Résultats de simulation avec la THO appliquée sur l'image 1.11(c) de l'immeuble (temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite détectée)

γ	$S_O(px^2)$	$\alpha(\%)$	seuil	Droites détectées	temps1(sec)	temps2(sec)
2	6	30	53	11	0.22	0.02
3	20	25	51	23	0.25	0.0092



FIGURE 5.11 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20); (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)



FIGURE 5.12 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=53, $\alpha = 30\%$, $\gamma = 2$; (b) Seuil=51, $\alpha = 25\%$, $\gamma = 3$

3.4 Comparaison de la Transformée de Hough Octogonale et la Transformée de Hough Octogonale Parallélisée

3.4.1 Cas de l'image de l'immeuble

La figure 5.14 présente deux graphiques comparant les performances des algorithmes de la transformée de Hough octogonale et de la transformée de Hough octogonale parallélisée. Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et S_O).

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 35%. Pour des valeurs de α plus basses (entre 10% et 28%), la courbe de la transformée de Hough octogonale parallélisée est inférieure à celle de la transformée de Hough octogonale, suggérant ainsi que la transformée de Hough octogonale parallélisée est plus performante pour des valeurs de α plus faibles. En revanche, pour des valeurs de α plus élevées (entre 28% et 35%), la courbe de la transformée de Hough octogonale parallélisée passe au-dessus de celle de la transformée de Hough octogonale, indiquant que la transformée de Hough octogonale parallélisée est moins performante pour des valeurs de α plus élevées.

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de la surface des cellules hexagonales, avec les deux algorithmes de la THO et de la THOP. Les surfaces des cellules varient de 21 à 417 px^2 . La courbe de la transformée de Hough octogonale parallélisée est en

TABEAU 5.8 – Résultats de simulation avec la THO appliquée sur l'image 1.11(a) de l'autoroute (temps1 = temps mis de toutes les droites détectées; temps2 = temps mis d'une droite détectée)

γ	$S_O(px^2)$	$\alpha(\%)$	seuil	Droites détectées	temps1(sec)	temps2(sec)
2	6	30	38	10	0.11	0.011
3	20	20	40	20	0.12	0.006



FIGURE 5.13 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=38, $\alpha = 30\%$ $gamma = 2$; (b) Seuil=40, $\alpha = 20\%$, $gamma = 3$

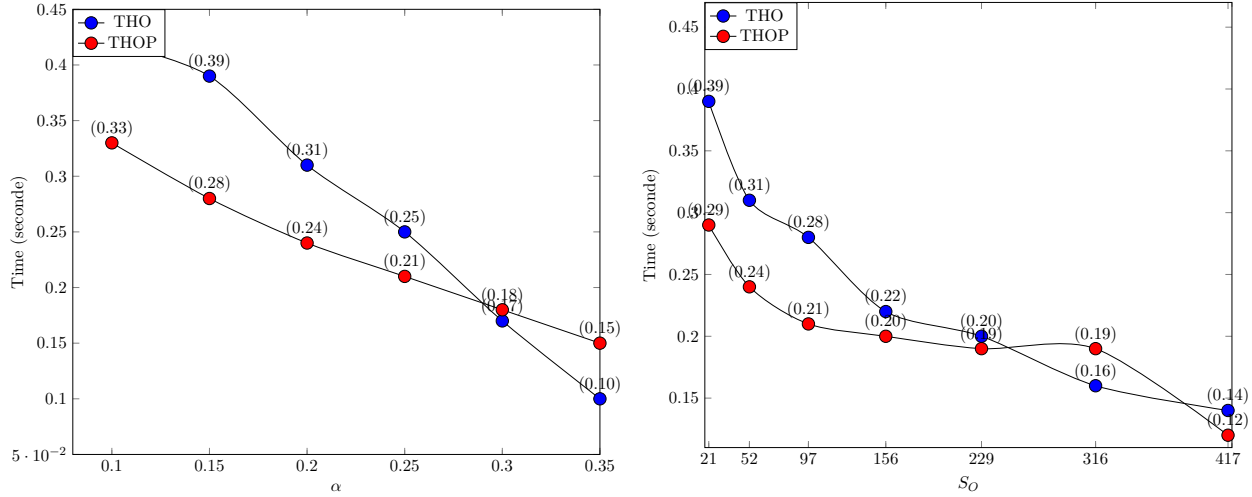


FIGURE 5.14 – Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=100, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=70, $\alpha = 20\%$) des algorithmes 30 et 32

dessous de celle de transformée de Hough octogonale pour les surfaces inférieures à 229. Aux alentours de 229, la courbe de THOP passe au-dessus de celle de THO. Cela suggère que la transformée de Hough octogonale parallélisée est plus performante que la transformée de Hough octogonale pour les petites valeurs de S_O , mais moins performante pour les valeurs élevées de S_O .

3.4.2 Cas de l'image de l'autoroute

La figure 5.15 présente deux graphiques comparant les performances des algorithmes de transformée de Hough octogonale (THO) et de transformée de Hough octogonale parallélisée (THOP). Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et S_O).

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 35%. Pour les petites valeurs de α (entre 10% et 15%), la courbe de la transformée de Hough octogonale parallélisée est en dessous de celle de la transformée de Hough octogonale. Cela indique que la transformée de Hough octogonale parallélisée est plus performante avec les petites valeurs de α . Pour les valeurs plus élevées de α (entre 15% et 30%), la courbe de la transformée de Hough octogonale parallélisée passe au dessus de celle de la transformée de Hough octogonale. Cela suggère que la transformée de Hough octogonale parallélisée

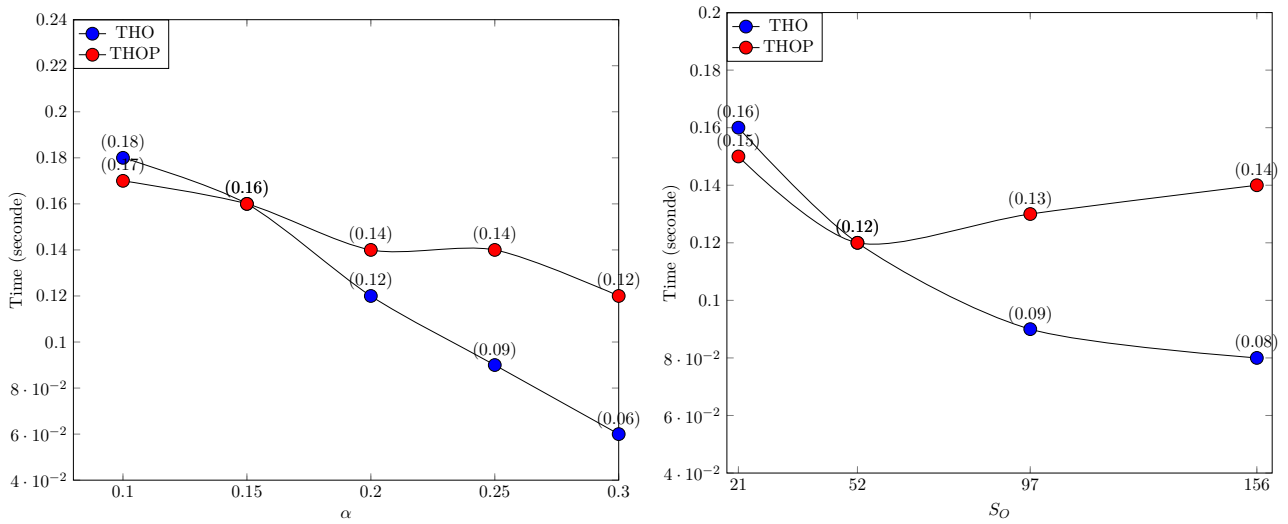


FIGURE 5.15 – Cas de l’autoroute : (à gauche) Comparaison des variations du temps d’exécution en fonction de α et les paramètres (threshold=70, $\gamma = 4$), (à droite) Comparaison des variations du temps d’exécution en fonction de la surface des cellules et les paramètres (threshold=50, $\alpha = 0, 2$) des algorithmes 30 et 32

est moins performante avec des valeurs de α plus élevées.

Dans le deuxième graphique (à droite), les variations du temps d’exécution sont comparées en fonction de la surface des cellules octogonales, avec les deux algorithmes de la THO et de la THOP. Les surfaces des cellules varient de 21 à 156 px^2 . La courbe de la transformée de Hough octogonale parallélisée est en dessous de celle de la transformée de Hough octogonale pour les surfaces inférieures à 52. À partir de 52, la courbe de la transformée de Hough octogonale parallélisée passe au-dessus de la courbe de la transformée de Hough octogonale. Cela suggère que la transformée de Hough octogonale parallélisée est plus performante que la transformée de Hough octogonale avec les petites valeurs de S_O et moins performante que la transformée de Hough octogonale avec des valeurs plus de S_O .

Ainsi, l’analyse des deux graphiques met en lumière les performances relatives de la transformée de Hough octogonale et de sa version parallèle (THOP) en fonction de α , de la taille des cellules octogonales et du seuil de vote. Pour les petites valeurs de α , la transformée de Hough octogonale parallélisée est plus rapide que la transformée de Hough octogonale. Cette performance bascule rapidement en faveur de la transformée de Hough octogonale à mesure que les valeurs de α augmentent. Il en est de même, en ce qui concerne la taille des cellules octogonales, la THOP est plus rapide par rapport à la transformée de Hough octogonale avec les cellules de petites tailles. Cependant, lorsque la taille des cellules devient plus grande, la THO devient plus performante.

Conclusion

Dans cette étude, nous avons examiné la transformée de Hough octogonale en tant qu’approche novatrice pour la détection de lignes droites dans les images. En comparaison avec la transformée de

Hough standard, la transformée de Hough octogonale offre plusieurs avantages potentiels, notamment une réduction du temps de calcul et une qualité de détection raisonnable. De plus, en diminuant le nombre de pixels à calculer grâce à la variable α , la transformée de Hough octogonale offre des économies computationnelles, ce qui peut être avantageux pour le traitement d'images à grande échelle.

En ajustant les paramètres tels que γ , α et le seuil de vote, les simulations ont démontré que la transformée de Hough octogonale peut réduire le temps de calcul tout en maintenant une qualité de détection acceptable.

L'analyse comparative de transformée de Hough octogonale et sa version parallélisée a montré que la THOP est plus performante avec les petites cellules, tandis que la THO est préférable pour des cellules plus grandes. Ainsi, le choix entre les deux dépend des valeurs spécifiques de α et de la taille des cellules.

Néanmoins, même si la transformée de Hough octogonale présente des avantages, elle comporte des limitations. Celles-ci incluent la perte d'informations provenant de l'ensemble complet des pixels allumés des cellules octogonales, la sensibilité aux taux α de pixels allumés et à la taille des cellules octogonales, la complexité de sa mise en œuvre, et sa dépendance au contexte. Dans le chapitre suivant, nous aborderons la parallélisation de la transformée de Hough généralisée pour la reconnaissance d'empreintes digitales.

Reconnaissance d'Empreintes Digitales dans une Grille Virtuelle

Introduction	128
1 La biométrie	129
2 Les empreintes digitales	130
3 Structure d'un système de reconnaissance d'empreintes digitales	131
4 Méthodologie	136
5 Expérimentations et discussions	138
Conclusion	142

Introduction

L'identification biométrique, qui repose sur des caractéristiques physiologiques ou comportementales uniques, représente un domaine clé de la sécurité et des nouvelles technologies de l'information et de la communication. Parmi les nombreuses techniques biométriques, la reconnaissance des empreintes digitales occupe une place prépondérante en raison de sa fiabilité et de sa large adoption dans divers domaines, allant de la sécurité des bâtiments à la lutte contre la fraude.

Cependant, la mise en œuvre d'un système de reconnaissance d'empreintes digitales efficace présente des défis, notamment en ce qui concerne le temps de traitement, en particulier pour les bases de données volumineuses. Pour relever ces défis, les chercheurs explorent des approches innovantes telles que la programmation parallèle, qui permet d'exploiter les ressources matérielles et informatiques pour accélérer les calculs.

Dans ce chapitre, notre étude se concentre sur l'intégration des grilles virtuelles dans le processus d'appariement dans le cadre de la reconnaissance d'empreintes digitales. Cette approche promet d'améliorer non seulement les performances en termes de temps de traitement, mais également la précision dans l'identification des empreintes digitales. Nous avons proposé également une approche hybride qui consiste à combiner le parallélisme aux grilles virtuelles.

Dans la suite du chapitre, nous présenterons d'abord un aperçu de la biométrie et des systèmes biométriques, en mettant en lumière l'importance des empreintes digitales dans ce domaine. Ensuite, nous explorerons en détail la structure d'un système complet de reconnaissance d'empreintes digitales, en mettant l'accent sur les étapes clés telles que l'acquisition, le pré-traitement, l'extraction des minuties et la comparaison. Nous discuterons également des différentes approches de comparaison des minuties, en mettant en évidence leurs avantages et leurs inconvénients. Par la suite, nous détaillerons notre méthode proposée, qui intègre la programmation parallèle combiné à une grille rectangulaire virtuelle pour accélérer le processus d'appariement des empreintes digitales. Nous présenterons ensuite les résultats de nos expérimentations. Enfin, nous conclurons en discutant des implications de nos résultats.

1 La biométrie

1.1 Définition

La biométrie est une discipline qui se concentre sur l'identification et l'authentification des individus en se basant sur des caractéristiques physiologiques ou comportementales uniques. Ces caractéristiques peuvent inclure des traits physiques tels que les empreintes digitales, la reconnaissance faciale, l'iris ou la rétine de l'œil, ainsi que des traits comportementaux tels que la voix ou la manière de taper sur un clavier. Ainsi, la biométrie permet d'authentifier l'identité des individus de manière plus fiable que les méthodes classiques telles que les mots de passe ou les cartes d'identité. Elle est largement utilisée dans divers domaines tels que la sécurité, les systèmes d'accès aux bâtiments, les contrôles aux frontières, la lutte contre la fraude et même dans certains dispositifs de paiement.

1.2 Les systèmes biométriques

Un système biométrique est fondamentalement un système de reconnaissance de formes qui opère en collectant des données biométriques à partir d'un individu, en extrayant un ensemble de caractéristiques de ces données, puis en les comparant à une signature stockée dans une base de données. Selon le contexte d'utilisation, un système biométrique peut fonctionner soit en mode de vérification, où l'identité d'une personne est vérifiée par rapport à une signature préalablement enregistrée, soit en mode d'identification, où l'identité de la personne est recherchée dans une base de données pour trouver une correspondance.

Généralement, tous les systèmes biométriques suivent un schéma de fonctionnement commun, composé des deux processus principaux suivants :

- Processus d'enregistrement : Ce processus vise à capturer et à enregistrer les caractéristiques biométriques des utilisateurs dans une base de données. Pendant cette étape, les données biométriques uniques de chaque individu, telles que les empreintes digitales, les traits du visage ou les schémas vocaux, sont collectées et stockées de manière sécurisée pour une utilisation ultérieure.
- Processus d'identification/vérification : Ce processus intervient lorsque quelqu'un cherche à s'identifier ou à être vérifié par le système biométrique. Lorsqu'une personne présente ses caractéristiques biométriques au système, celles-ci sont comparées à celles enregistrées dans la base de données. Dans le processus d'identification, le système recherche l'ensemble de la base de données pour trouver une correspondance avec les caractéristiques présentées. Dans le processus de vérification, la comparaison se fait avec les données biométriques spécifiques de l'utilisateur qui se présente, afin de vérifier son identité.

2 Les empreintes digitales

2.1 Historique

Les empreintes digitales ont une longue histoire d'utilisation dans différentes cultures à travers le monde. Les premières traces de leur utilisation remontent à l'Égypte ancienne, datant de plus de 4000 ans, et les Chinois les utilisaient également dès le troisième siècle avant Jésus-Christ pour signer des documents officiels, bien qu'ils ne comprennent pas alors leur unicité pour chaque individu [102].

Ce n'est qu'au XIXe siècle que des progrès significatifs ont été réalisés dans la compréhension scientifique des empreintes digitales. En 1856, l'anglais William Herschel [5], après avoir observé l'utilisation des empreintes digitales comme signature sur des documents par la population indienne qu'il dirigeait, a commencé à reconnaître leur unicité et leur constance dans le temps. Puis, en 1888, le britannique Francis Galton a publié une étude détaillée sur les empreintes digitales, établissant leurs caractéristiques uniques.

L'adoption officielle des empreintes digitales dans le système judiciaire anglais a eu lieu en 1894, et cette technique a ensuite été largement développée dans les enquêtes criminelles, devenant un outil essentiel pour résoudre de nombreuses affaires. Aujourd'hui, les empreintes digitales restent largement utilisées et reconnues comme une méthode d'identification fiable, jouant un rôle crucial dans les domaines de la sécurité, de la justice et de la technologie.

2.2 Caractéristiques des empreintes digitales

Une empreinte digitale est composée d'un ensemble de lignes localement parallèles formant un motif unique pour chaque individu. Dans cette empreinte, on distingue deux éléments principaux : les stries

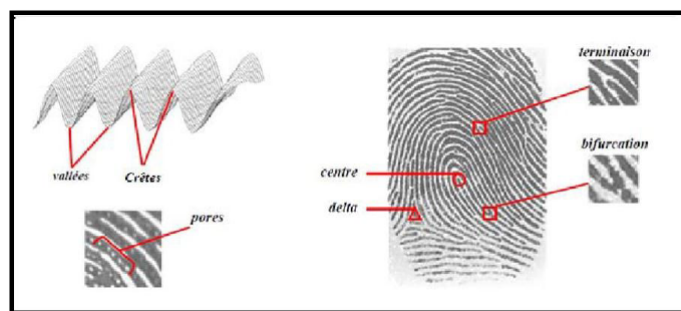


FIGURE 6.1 – Empreinte digitale [5]

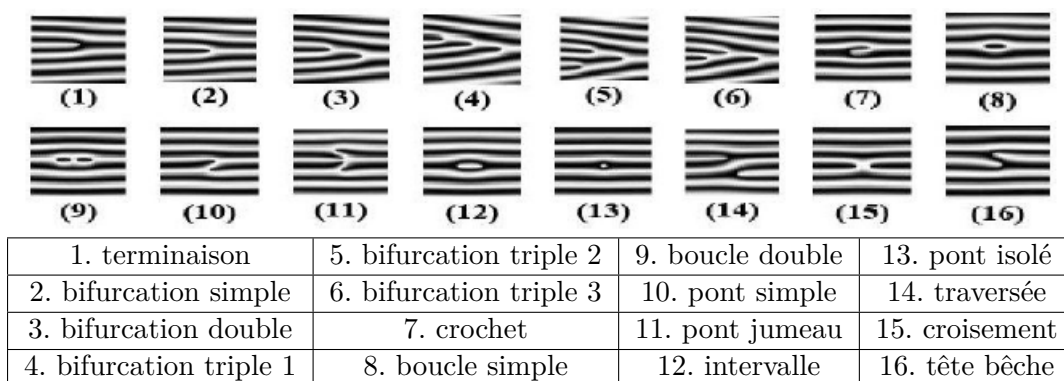


FIGURE 6.2 – Les différents types de minutie [5]

(ou crêtes) et les sillons comme illustré dans la figure 6.1. Les empreintes digitales présentent des stries qui contiennent en leur centre un ensemble de pores régulièrement espacés. Chaque empreinte possède des caractéristiques singulières à la fois globales et locales. Les caractéristiques globales comprennent les centres, qui sont des points où les stries convergent, et les deltas, qui sont des points où les stries divergent [5]. Les caractéristiques locales comprennent les minuties, qui sont des points spécifiques de bifurcation ou de terminaison le long des stries.

Bien qu'il existe seize types différents de minuties identifiés par plusieurs études et illustré dans la figure 6.2, les algorithmes de traitement d'empreintes digitales se concentrent généralement sur les bifurcations et les terminaisons, car ces caractéristiques peuvent être utilisées pour dériver les autres types de minuties par combinaison.

3 Structure d'un système de reconnaissance d'empreintes digitales

Un système complet de reconnaissance d'empreintes digitales représente une chaîne de processus qui prend en entrée l'empreinte digitale d'un utilisateur et renvoie en sortie un résultat, permettant ainsi de déterminer si l'utilisateur a accès ou non à des éléments nécessitant une protection. La mise en œuvre d'un tel système a été le sujet de nombreuses recherches, avec une variété de méthodes de traitement proposées [103]. Cependant, tous ces systèmes suivent une structure similaire, comme illustré dans la figure 6.4.

La première phase d'un système de reconnaissance d'empreintes digitales consiste à obtenir une

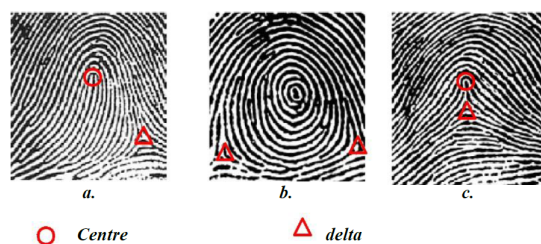


FIGURE 6.3 – Les trois principales classes d’empreintes, boucle (a), spire (b), arche (c) [5]

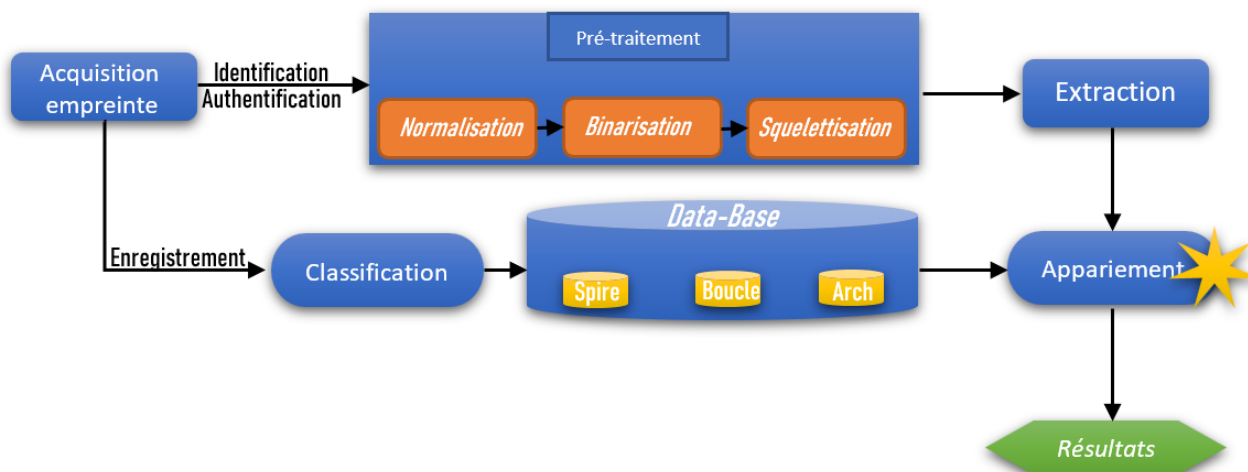


FIGURE 6.4 – Architecture générale d’un système complet de reconnaissance d’empreintes

image de l’empreinte de l’utilisateur (acquisition), qui est ensuite soumise à un pré-traitement pour extraire les informations pertinentes de l’image (extraction de la signature). Ensuite, un traitement supplémentaire peut être effectué pour éliminer les éventuelles fausses informations qui auraient pu être introduites dans la chaîne de traitement.

Si le système est destiné à simplement créer une base de données, la signature peut être compressée, puis stockée dans la base de données à l’aide d’une technique d’archivage (classification). La classification des empreintes digitales se fait en fonction de la position et du nombre de centres et de deltas, ce qui permet de les regrouper en différentes catégories selon leur caractéristiques globales. Les principales familles, illustrées dans la figure 6.3, sont les boucles, les spires et les arches.

En revanche, pour un système d’identification, toutes les empreintes présentes dans la base de données pouvant correspondre à celle de l’utilisateur (modèle identique) sont désarchivées et comparées une à une avec celle de l’utilisateur (appariement). Si une correspondance est trouvée, des informations personnelles concernant l’utilisateur sont renvoyées par le système.

Dans le cas d’un système de vérification, il n’y a qu’une seule comparaison effectuée, et un résultat binaire est renvoyé, permettant soit l’acceptation, soit le rejet de l’utilisateur.

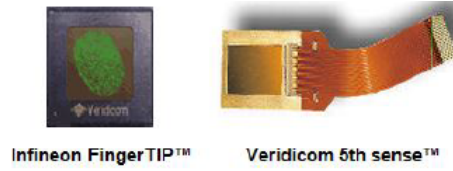


FIGURE 6.5 – Capteur capacitif [6]

3.1 Acquisition d’empreintes digitales

Pour capturer l’image d’une empreinte digitale, on fait appel à un capteur d’empreinte. Ces capteurs peuvent prendre différentes formes, parmi lesquelles on trouve une variété d’options telles que les capteurs optiques d’empreinte, les capteurs électriques thermiques, les capteurs capacitifs, les capteurs de champ électrique, les capteurs de pression, et bien d’autres encore. La méthode capacitive (Figure. 6.5) est l’une des techniques les plus répandues pour la capture d’empreintes digitales. Comme les autres capteurs, le capteur capacitif reproduit l’image des creux et des reliefs qui composent une empreinte digitale. Ce capteur utilise des condensateurs électriques pour mesurer l’empreinte, et il se compose d’une rangée de cellules minuscules. Chaque cellule comprend deux plaques conductrices recouvertes par un revêtement protecteur.

L’avantage principal de ces capteurs est qu’ils nécessitent une véritable empreinte digitale pour fonctionner. Cependant, ils peuvent rencontrer des difficultés lors de l’acquisition de données à partir de doigts secs ou humides, ce qui peut affecter leur précision et leur fiabilité dans certaines situations.

3.2 Pré-traitement

Lorsqu’une image d’empreinte digitale est capturée, aujourd’hui par le biais d’un scanner, elle contient de nombreuses informations redondantes. Les problèmes liés aux cicatrices, aux doigts trop secs ou trop humides, ou à une pression incorrecte doivent également être surmontés pour obtenir une image acceptable. C’est pourquoi un pré-traitement est indispensable. Les étapes du pré-traitement sont : la normalisation, la binarisation qui consiste à transformer la valeur des pixels en 0 et 1 et la squelettisation qui consiste à réduire les contours de l’image d’empreinte à la taille d’un pixel afin de favoriser l’extraction des minuties.

3.3 Extraction des minuties

Les minuties sont des caractéristiques uniques (bifurcations, terminaisons) de l’empreinte digitale qui permettent son identification et sa vérification [104]. Chaque minutie m_i est représentée par un quadruplet de coordonnées : $m_i = (x_i, y_i, \theta_i, t_i)$ [105]. Les coordonnées x_i et y_i correspondent à la position de la minutie dans l’image, θ_i représente sa direction, généralement calculée à partir de l’orientation locale de la crête, et t_i indique si la minutie est de type terminaison ou bifurcation comme indique la figure 6.6. Les bifurcations et les terminaisons sont des points clés pour la comparaison des

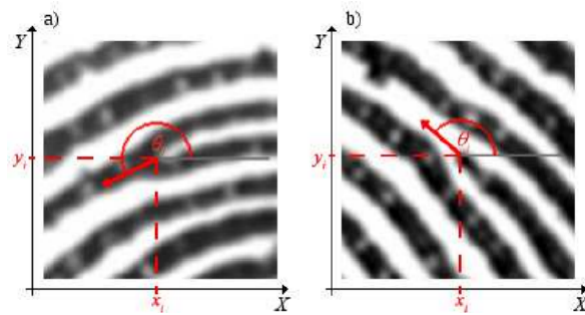


FIGURE 6.6 – (a) Bifurcation ; (b) Termination

empreintes digitales. Pour chaque pixel noir dans chaque bloc de l’empreinte digitale, nous calculons l’indicateur de Crossing Number (CN) [104], qui est une mesure de la topologie locale. La formule pour l’indicateur CN est la suivante :

$$CN = \frac{1}{2} \sum_{i=1}^8 |p_i - p_{i+1}| \quad \text{with } p_9 = p_1$$

Les pixels voisins $p_1, \dots, p_9$ sont disposés dans l’ordre des aiguilles d’une montre. En utilisant les propriétés du CN chaque pixel de crête peut être classé comme une terminaison, une bifurcation ou un point sans minutie. En particulier, un pixel isolé aura un CN de 0, une terminaison aura un CN de 1, une crête continue aura un CN de 2, une bifurcation aura un CN de 3 et un croisement aura un CN de 4 [104].

3.4 Comparaison des minuties

La comparaison des minuties est une étape cruciale dans les systèmes de reconnaissance d’empreintes digitales. Elle comporte trois parties : l’alignement, la correspondance et la génération du score de correspondance. L’algorithme de la transformée de Hough généralisée à été adapté, décrit dans l’algorithme 34, à la comparaison des minuties [106]. Cette comparaison vise à évaluer la similarité entre les minuties extraites de l’empreinte digitale à authentifier et celles stockées dans une base de données.

Pour déterminer si deux ensembles de minuties proviennent de la même empreinte digitale, nous employons une méthode de superposition exhaustive, explorant toutes les combinaisons possibles. Nous examinons chaque minutie m_Q de la base de données modèle et tentons de la superposer à la minutie m_T de l’empreinte digitale à authentifier. La figure 6.7 montre la superposition de deux empreintes digitales. Ce processus implique le déplacement de la deuxième empreinte digitale, réalisant ainsi une translation afin d’aligner les coordonnées de m_T sur celles de m_Q . De plus, nous ajustons l’orientation de la deuxième empreinte digitale pour aligner les orientations de m_T et m_Q . La correspondance, décrite dans l’algorithme 35, implique l’identification des minuties appariées, c’est-à-dire celles des deux ensembles ayant des coordonnées (abscisse et ordonnée) similaires dans une certaine marge de tolérance, ainsi qu’une orientation similaire dans une certaine marge de tolérance. Toutes les minuties

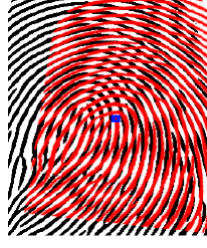


FIGURE 6.7 – Superposition de deux empreintes digitales

appariées après l'alignement sont énumérées et stockées dans une liste.

le score de correspondance, calculé à l'aide de la formule 6.1, est une mesure qui évalue à quel point deux ensembles de minuties sont similaires ou compatibles. Un score de correspondance plus élevé est associé à une meilleure précision de la reconnaissance. Cela signifie que les deux empreintes digitales sont plus susceptibles d'appartenir à la même personne si le score de correspondance est élevé.

$$S_{core} = \frac{N_c}{N_T} \quad (6.1)$$

N_c représente le nombre de minuties correspondant, N_T le nombre de minuties de l'empreinte à authentifier et

Algorithme 34 : Transformée de Hough Généralisée

Fonction $THG(m_Q, m_T, \text{seuil_larg}, \text{seuil_long}, \text{seuil_rot})$: Tableau, réel

Variables : liste : Tableau

$\{x_i^T, y_i^T, \theta_i^T\}_{i=1}^N, \{x_j^Q, y_j^Q, \theta_j^Q\}_{j=1}^M \leftarrow m_Q, m_T$;

Initialisation de la liste des correspondances;

Initialisation de la matrice d'accumulation A ;

for $i=1,2,3,\dots,N$ **do**

for $j=1,2,3,\dots,M$ **do**

 % orientations des minuties;

$\Delta\theta = \min(|\theta_i^T - \theta_j^Q|, 360^\circ - |\theta_i^T - \theta_j^Q|)$;

$$\begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} x_i^T \\ y_i^T \end{bmatrix} - \begin{bmatrix} \cos(\Delta\theta) & \sin(\Delta\theta) \\ -\sin(\Delta\theta) & \cos(\Delta\theta) \end{bmatrix} \begin{bmatrix} x_j^Q \\ y_j^Q \end{bmatrix}$$

$A(\Delta x, \Delta y, \Delta\theta) \leftarrow A(\Delta x, \Delta y, \Delta\theta) + 1$;

si $(0 \leq \Delta x \leq \text{seuil_larg})$ **et** $(0 \leq \Delta y \leq \text{seuil_long})$ **et** $(\Delta\theta \leq \text{seuil_rot})$ **alors**

 ajouter (i, j) à liste % liste des correspondances

finsi

end

end

$n \leftarrow$ nombre total des correspondants;

$$\text{score} = \frac{n}{N}$$

Retourner : accum, score

fin

Algorithme 35 : Recherche des paires de minuties correspondantes de deux empreintes digitales

Fonction *Correspondance*($m_Q, m_T, accum, Sc$) : *Tableau*

Variables : *liste* : *Tableau*

$\{x_i^T, y_i^T, \theta_i^T\}_{i=1}^N, \{x_j^Q, y_j^Q, \theta_j^Q\}_{j=1}^M \leftarrow m_Q, m_T$;

Initialisation de la liste des correspondances ;

for $i=1,2,3,\dots,N$ **do**

for $j=1,2,3,\dots,M$ **do**

 % orientations des minuties;

$\Delta\theta = \min(|\theta_i^T - \theta_j^Q|, 360^\circ - |\theta_i^T - \theta_j^Q|)$;

$$\begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} x_i^T \\ y_i^T \end{bmatrix} - \begin{bmatrix} \cos(\Delta\theta) & \sin(\Delta\theta) \\ -\sin(\Delta\theta) & \cos(\Delta\theta) \end{bmatrix} \begin{bmatrix} x_j^Q \\ y_j^Q \end{bmatrix}$$

si $(0 \leq \Delta x \leq \text{seuil_larg})$ **et** $(0 \leq \Delta y \leq \text{seuil_long})$ **et** $(\Delta\theta \leq \text{seuil_rot})$ **alors**

si $accum[\Delta y, \Delta x, \Delta\theta] \geq Sc$ **alors**

 ajouter (i, j) à *correspondances* % liste des correspondances

finsi

finsi

end

end

Retourner : *correspondances*

fin

4 Méthodologie

L'adaptation de la THG à la détection des empreintes digitales offre une précision satisfaisante de 99% d'après les travaux de Cynthia S. Mlambo et al [106]. Toutefois, son principal inconvénient réside dans son temps de traitement et l'utilisation de la mémoire. Face à cette problématique, notre objectif est d'optimiser cette approche en réduisant significativement le temps de traitement.

Pour ce faire, nous avons adopté une approche basée sur l'utilisation des grilles virtuelles et la programmation parallèle. La phase clé de notre optimisation réside dans l'étape d'appariement, représentée dans la Figure 6.4. Cette étape consiste à comparer chaque empreinte digitale de la base de données avec l'empreinte à authentifier en utilisant une approche adaptée de la Transformée de Hough généralisée décrit par l'algorithme 34.

l'approche par la grille virtuelle, décrite par l'algorithme 36, consiste à superposer une grille rectangulaire virtuelle de deux cellules sur l'empreinte digitale. Pour chaque empreinte digitale modèle de la base de données, l'appariement se fait d'abord avec la cellule supérieur de la grille virtuelle. Lorsque le nombre de paires de minuties correspondantes (Nc) est supérieur ou égal au seuil de correspondance (Sc), alors l'empreinte digital test et l'empreinte digitale modèle sont considéré comme identique avec un score de similarité. Dans le cas contraire, on passe à la cellule inférieur de la grille virtuelle. Lorsque le nombre de paires de minuties correspondantes (Nc) est supérieur ou égal au seuil de correspondance (Sc), alors l'empreinte digital test et l'empreinte digitale modèle sont considéré comme identique avec

un score de similarité. Dans le cas contraire, l’empreinte digital test et l’empreinte digitale modèle ne sont pas identiques et on passe à l’empreinte modèle suivante de la base de données.

Algorithme 36 : Reconnaissance d’empreintes digitales dans une grille rectangulaire virtuelle.

```

Fonction FingerprintVG(BD, mT, seuil_larg, seuil_long, seuil_rot, Sc) :Vide
    miT  $\leftarrow$  pré-traitement et extraction des minuties de mT;
    VG  $\leftarrow$  Génération et superposition de la grille virtuelle de deux cellules rectangulaires
        sur miT;
    % Sc : seuil de correspondance.
    h  $\leftarrow$  0;
    for mQ de BD do
        for chaque cellule C de VG do
            if C = 1ère cellule de VG then
                accum, Score  $\leftarrow$  THG(mQ, mT, seuil_larg, seuil_long, seuil_rot);
                correspondances  $\leftarrow$  Correspondance(mQ, mT, accum, Sc);
                Nc  $\leftarrow$  len(correspondances); % nombre de paires de minuties
                    correspondantes.
                if Nc  $\geq$  Sc then
                    Afficher : les empreintes digitales mQ et mT sont identiques avec un score de
                        similarité de Score;
                    h  $\leftarrow$  1;
                end
            else
                if h = 1 then
                    if Nc  $\geq$  Sc then
                        Afficher : les empreintes digitales mQ et mT sont identiques avec un
                            score de similarité de Score;
                        h  $\leftarrow$  1;
                    else
                        Afficher : les empreintes digitales mQ et mT ne sont pas identiques avec
                            un score de similarité de Score;
                    end
                end
            end
        end
    end
fin

```

La parallélisation consiste à diviser la base de données en sous-ensembles, en fonction du nombre de cœurs du processeur disponibles. La transformée de Hough généralisée est alors appliqué simultanément à chaque sous-ensemble pour l’appariement en utilisant l’approche des grilles virtuelles. C’est donc une approche hybride. Ce processus parallèle permet à chaque cœur de travailler de manière indépendante sur son propre lot de données, comme illustré dans la Figure 6.8.

Une fois l’appariement effectué, chaque processus renvoie un résultat partiel. Par la suite, ces résultats sont agrégés pour déterminer le résultat final. Plus précisément, si au moins l’un des processus identifie une correspondance positive entre l’empreinte à authentifier et une empreinte de la base de données, le résultat global est positif. En revanche, si tous les processus concluent à une absence de

correspondance, le résultat global est négatif.

Algorithme 37 : Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle combiné au parallélisme

```

Fonction FingerprintVGP(BD, mT, seuil_larg, seuil_long, seuil_rot, Sc, n_cpu) :Vide
  miT ← pré-traitement et extraction des minuties de mT;
  VG ← Génération et superposition de la grille virtuelle de deux cellules rectangulaires
    sur miT;
  % Sc : seuil de correspondance.
  % n_cpu : nombre de processus pour le parallélisme.
  h ← 0;
  With pymp.parallel(n_cpu) as p :
    for mQ de p(BD) do
      for chaque cellule C de VG do
        if C = 1ère cellule de VG then
          accum, Score ← THG(mQ, mT, seuil_larg, seuil_long, seuil_rot);
          correspondances ← Correspondance(mQ, mT, accum, Sc);
          Nc ← len(correspondances); % nombre de paires de minuties
            correspondantes.
          if Nc ≥ Sc then
            Afficher : les empreintes digitales mQ et mT sont identiques avec un
              score de similarité de Score;
            h ← 1;
          end
        else
          if h = 1 then
            if Nc ≥ Sc then
              Afficher : les empreintes digitales mQ et mT sont identiques avec un
                score de similarité de Score;
              h ← 1;
            else
              Afficher : les empreintes digitales mQ et mT ne sont pas identiques
                avec un score de similarité de Score;
            end
          end
        end
      end
    end
  end
  fin
fin

```

5 Expérimentations et discussions

Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : kali linux, 64 bits.

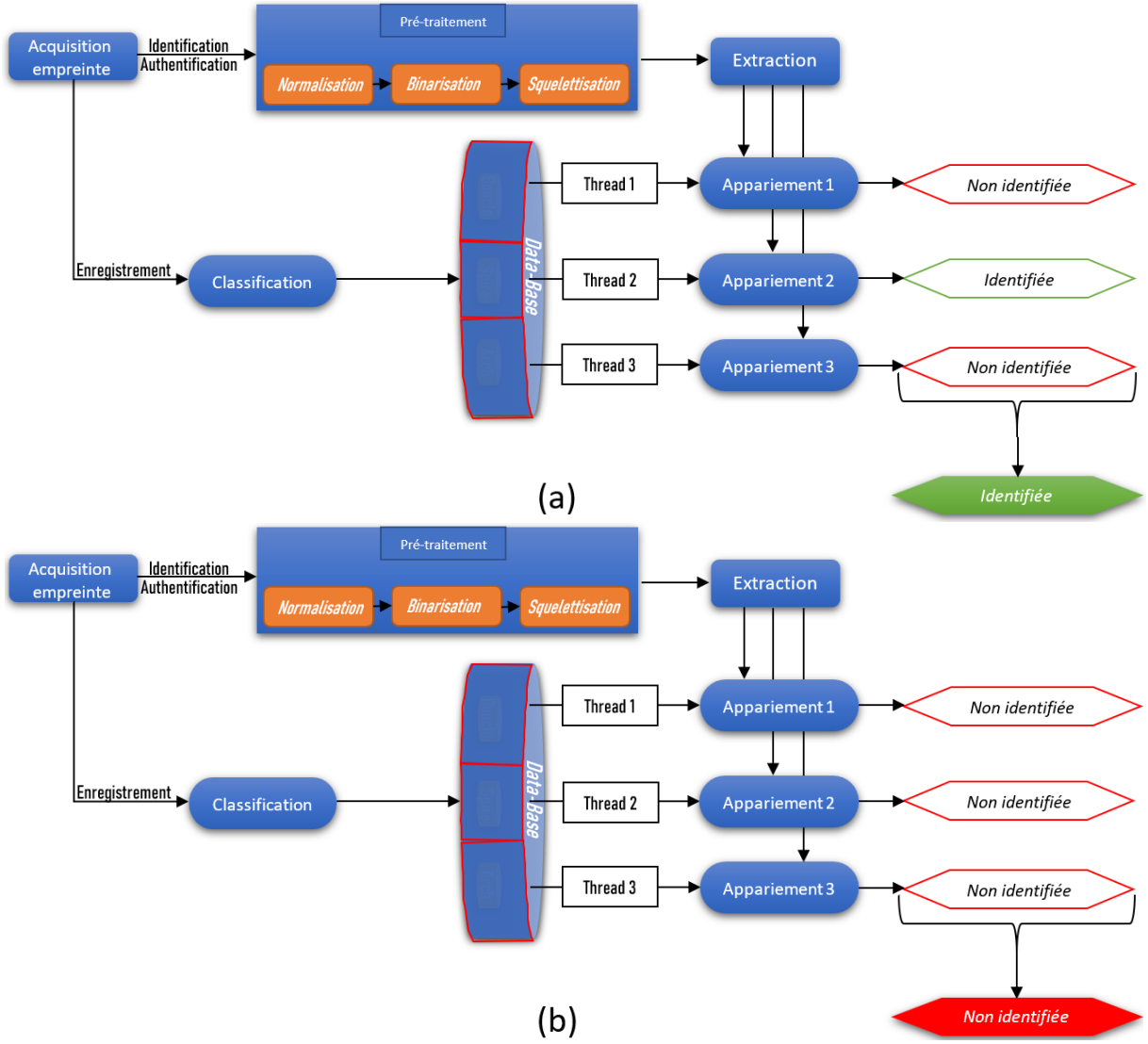


FIGURE 6.8 – Parallélisation de l'appariement des empreintes digitales. (a) Empreinte identifiée ; (b) Empreinte non identifiée

Le langage de programmation utilisé est python.

Les expérimentations ont été menées sur un échantillon restreint de données, où les empreintes digitales ont été acquises dans diverses orientations et emplacements [107], ainsi que sur la base de données publique FVC2004 [108].

5.1 Performance temporelle

Le tableau 6.1 présente le temps de traitement de chaque approche (l'approche séquentielle, l'approche basée sur les grilles virtuelles et celle basée sur les grilles virtuelles combiné a la programmation parallèle) en fonction du nombre de comparaisons d'empreintes digitales.

L'analyse des graphiques de la figure 6.10 révèle que l'approche basée sur les grilles virtuelles est plus performante que celle séquentielle. Le facteur d'accélération est donner par $A = \frac{T_s}{T_h} = \frac{7.42}{4.82} = 1,6$ où T_h représentant le temps d'exécution de l'algorithme de l'approche basée sur la grille rectangulaire

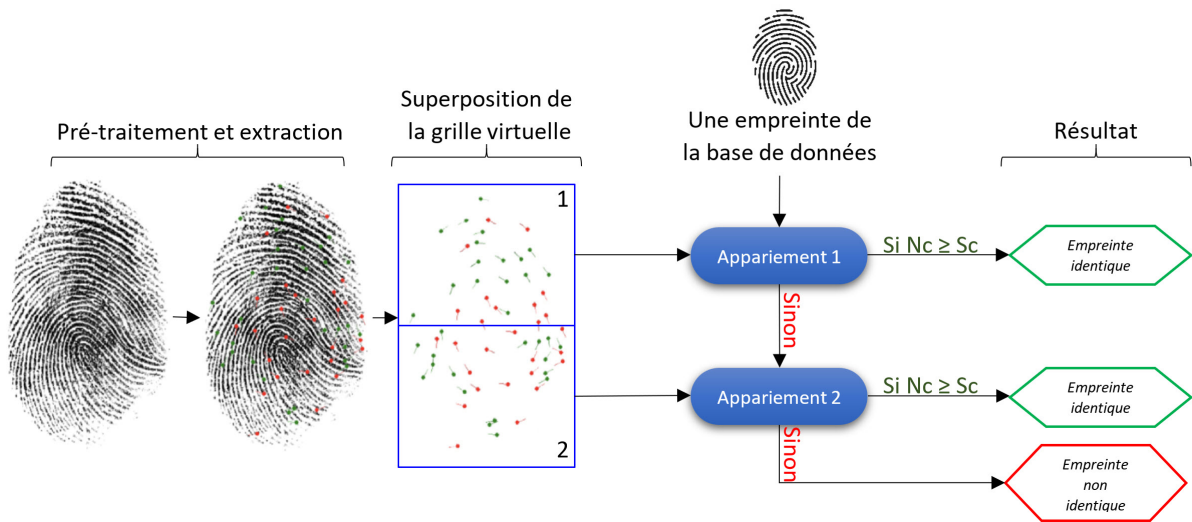


FIGURE 6.9 – Appariement de deux empreintes digitales avec une grille rectangulaire

virtuelle et T_s désigne le temps d'exécution de l'algorithme séquentielle. Quant à l'approche basée sur

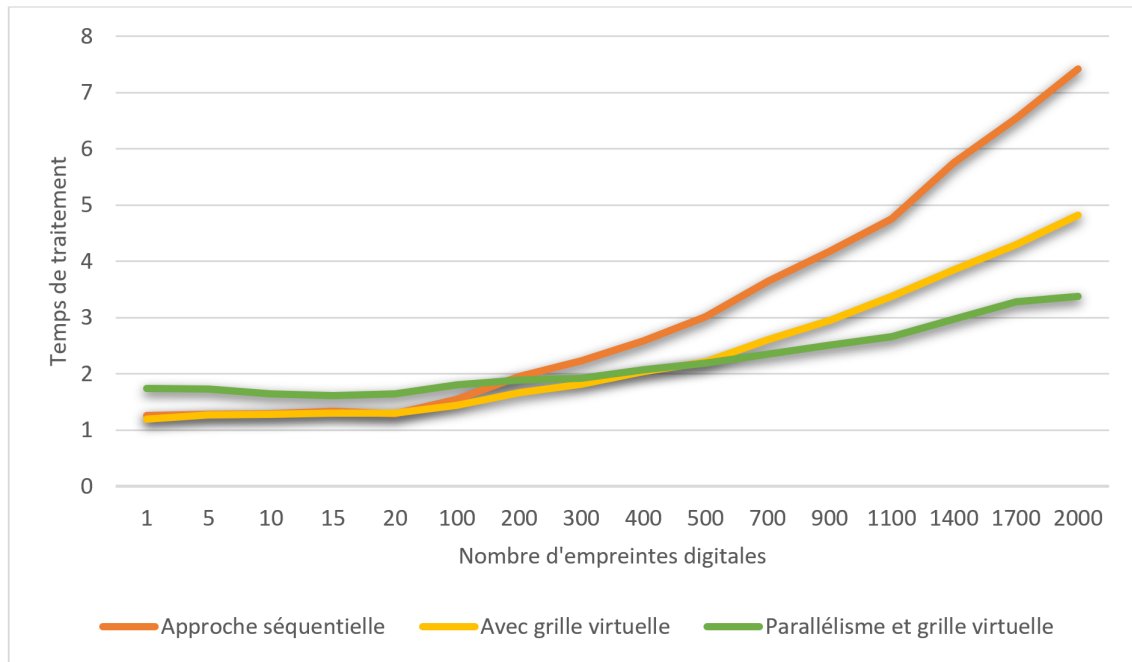


FIGURE 6.10 – Comparaison du temps de traitement en fonction du nombre d'empreintes

la grille rectangulaire virtuelle combiné au parallélisme, elle est moins performante que celle basée sur la grille virtuelle pour les bases de données contenant moins de 500 empreintes digitales. Cependant, à mesure que le nombre de comparaisons augmente, l'approche basée sur la grille rectangulaire virtuelle combiné au parallélisme devient de plus en plus avantageuse en termes de temps de traitement. Pour des nombres plus élevés de comparaisons (par exemple 500 et plus), elle réduit de manière significative le temps de traitement par rapport à l'approche basée sur la grille virtuelle. Cette réduction est particulièrement notable pour des nombres élevés de comparaisons, où le temps de traitement parallèle peut être jusqu'à plusieurs fois inférieur à celui du traitement séquentiel (voir la figure 6.10). Cette

performance temporelle est évaluée à travers le calcul du facteur d'accélération $A = \frac{T_s}{T_h} = \frac{7.42}{3.38} = 2,19$ où T_h représentant le temps d'exécution de l'algorithme de l'approche basée sur la grille rectangulaire virtuelle combiné au parallélisme (4 threads utilisés), et T_s désigne le temps d'exécution de l'algorithme séquentielle. Le fait que ce facteur d'accélération dépasse largement 1 met en lumière une amélioration significative de la rapidité de l'approche parallélisée de la reconnaissance des empreintes digitales.

Nombre de comparaison	1	5	10	15	20	100	200	300	400	500	700	900	1100	1400	1700	2000
Approche séquentielle (sans la grille virtuelle)	1,26	1,28	1,29	1,33	1,29	1,55	1,95	2,23	2,59	3,01	3,64	4,18	4,76	5,75	6,54	7,42
Approche séquentielle (avec la grille virtuelle)	1,20	1,27	1,28	1,3	1,3	1,44	1,67	1,82	2,04	2,21	2,61	2,95	3,38	3,85	4,30	4,82
Approche parallélisée (avec la grille virtuelle)	1,74	1,73	1,64	1,61	1,64	1,81	1,89	1,92	2,07	2,19	2,35	2,51	2,66	2,97	3,28	3,38

TABLEAU 6.1 – Comparaison du temps de traitement en fonction du nombre d'empreintes

5.2 Précision

Pour évaluer la précision de notre programme, nous disposons de deux bases de données distinctes : la première est la base modèle, qui comprend 500 empreintes digitales. La seconde est la base de test, comprenant 100 empreintes digitales, dont 50 appartiennent à la base de données modèle et 50 ne lui appartiennent pas. Ensuite, chaque empreinte de la base de test est comparée à toutes les empreintes de la base modèle, totalisant ainsi 50 000 comparaisons.

Les résultats sont visualisés dans les matrices de confusion de la figure 6.11. Une matrice de confusion est un tableau qui permet d'évaluer les performances d'un modèle de classification. Elle compare les prédictions du modèle aux valeurs réelles et fournit une vue d'ensemble des résultats pour chaque classe. la structure générale d'une matrice de confusion pour un problème binaire (deux classes) est donné dans le tableau 6.2. La précision est calculé a l'aide de la formule $P = \frac{VP}{VP + FP}$.

L'analyse des matrices de confusion illustrée dans la figure 6.11 révèle des résultats significatives. Pour un seuil de correspondance égal à 2, les résultats illustrés dans la matrice de confusion donnent une précision de $P = 0,69$. Pour un seuil de correspondance égal à 3, les résultats illustrés dans la matrice de confusion donnent une précision de $P = 0,98$. Pour un seuil de correspondance égal à 7, les résultats illustrés dans la matrice de confusion donnent une précision de $P = 1$.

Ainsi les Seuils de correspondance $Sc = 3 ; 4 ; 5$ et 6 sont recommandé. En effet, $Sc \leq 2$ laisse passer des faux positifs et $Sc \geq 7$ pourrait non seulement générer des faux négatifs, mais également réduire les chances de détecter de véritables correspondances si les empreintes sont partiellement dégradées. Cette recommandation permet de s'assurer qu'il y a un minimum de cohérence entre les minuties des deux

	Prédiction : Négatif	Prédiction : Positif
Réel : Négatif	Vrais Négatifs (VN)	Faux Positifs (FP)
Réel : Positif	Faux Négatifs (FN)	Vrais Positifs (VP)

TABLEAU 6.2 – Structure générale d’une matrice de confusion

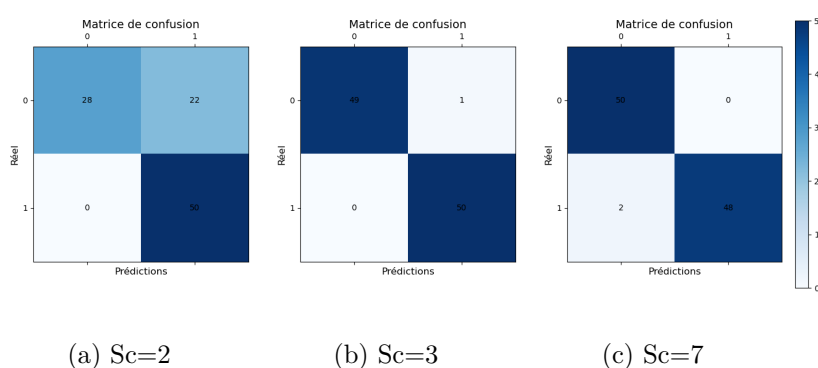


FIGURE 6.11 – Matrices de confusion

empreintes, même avec des variations mineures causées par le découpage en cellules rectangulaires, en présence de bruit ou de légères différences de positionnement.

Conclusion

Ce chapitre a mis en lumière l’importance cruciale de l’application de la transformée de Hough généralisée dans les grilles virtuelles, ainsi que le parallélisme dans le domaine de la reconnaissance d’empreintes digitales. Nous avons démontré que l’intégration des grilles virtuelles dans la transformée de Hough généralisée a considérablement amélioré les performances en termes de temps de traitement tout en préservant la précision de l’identification des empreintes digitales. Combinée à la programmation parallèle, notre méthode réduit encore plus le temps de traitement particulièrement dans les bases de données volumineuses.

Architecture de la Boîte à Outils

Introduction	143
1 Conception	144
2 Interface graphique	148
3 Impact	150
4 Limitations	151
5 Validation des hypothèses de recherche	151
Conclusion	152

Introduction

La détection et l'analyse des formes géométriques dans les images sont des problèmes fondamentaux en vision par ordinateur. Parmi les techniques couramment utilisées pour résoudre ces problèmes, la transformée de Hough occupe une place prépondérante grâce à sa robustesse et à son efficacité. Elle permet de détecter des formes paramétriques telles que des lignes et des formes arbitraires dans des images bruitées ou incomplètes. Dans ce sens, une boîte à outils permettant d'utiliser divers transformée de Hough pour divers objectifs pourrait être une aubaine. Une aubaine, pour la détection de droites discrètes, d'empreintes digitales via la transformée de Hough. L'objectif de ce chapitre est de fournir une compréhension complète de l'architecture de la boîte à outils développé dans le cadre de ces travaux. Cette boîte à outils est un paquet de modules qui offre une variété de fonctionnalités pour la détection de lignes par application de la transformée de Hough sur différentes grilles virtuelles (rectangulaires, hexagonales, triangulaires et octogonales) et pour la reconnaissance d'empreintes digitales utilisant la transformée de Hough généralisée dans une grille rectangulaire virtuelle. En plus de ses capacités de traitement, le paquet est conçu pour être utilisé en environnement multiprocesseur,

ce qui permet d'accélérer les calculs grâce à la parallélisation. En outre, Elle est dotée d'une interface graphique pour permettre aux utilisateurs de visualiser et d'interagir facilement avec les résultats.

Dans la suite du chapitre, nous commencerons par une étude des cas d'utilisation et de l'architecture de la boîte à outils. Ensuite, nous décrirons les modules internes, notamment ceux dédiés à la détection de droites et à la reconnaissance d'empreintes digitales, ainsi que les versions parallélisées de ces fonctionnalités pour des performances optimales. Aussi, nous explorerons l'interface utilisateur graphique (Graphic User Interface en anglais et abrégé par GUI) qui permet d'utiliser la boîte à outils de manière intuitive. En outre, nous explorerons les potentiels domaines d'impact de la boîte à outils. En fin, nous discuterons de ses limitations.

1 Conception

Dans cette section, nous décrivons la conception de la boîte à outils de la transformée de Hough, en détaillant les principaux aspects techniques et fonctionnels. Nous commencerons par étudier les cas d'utilisation de la boîte à outils. Ensuite, nous décrirons les fonctionnalités principales de l'outil, en mettant en avant ses capacités et son utilité. Enfin, nous présenterons les technologies et les outils utilisés pour développer cette boîte à outils.

1.1 Cas d'utilisation

La boîte à outils de la transformée de Hough possède deux cas d'utilisation distincts : un système de détection de droites et un système de reconnaissance d'empreintes digitales. La figure 7.1 illustre les cas d'utilisation de la boîte à outils [109]. Chaque système de la boîte à outils contient des algo-

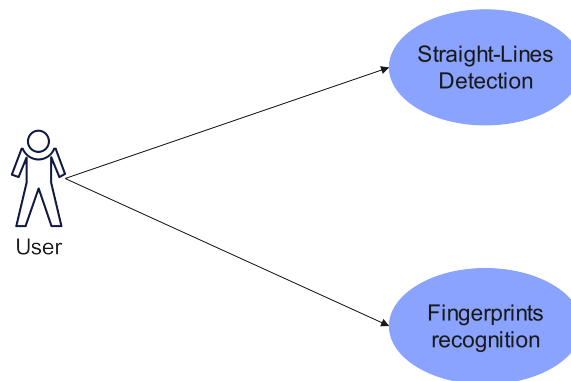


FIGURE 7.1 – Interactions entre les utilisateurs et le système

rithmes séquentiels et parallélisés. L'utilisation de l'un ou de l'autre est conditionné par les valeurs des paramètres d'entrés.

Pour la détection de droites, la version parallélisée est préférable à la version séquentielle lorsque la valeur du taux de pixels allumé *alpha* ainsi que la taille des cellules sont petites.

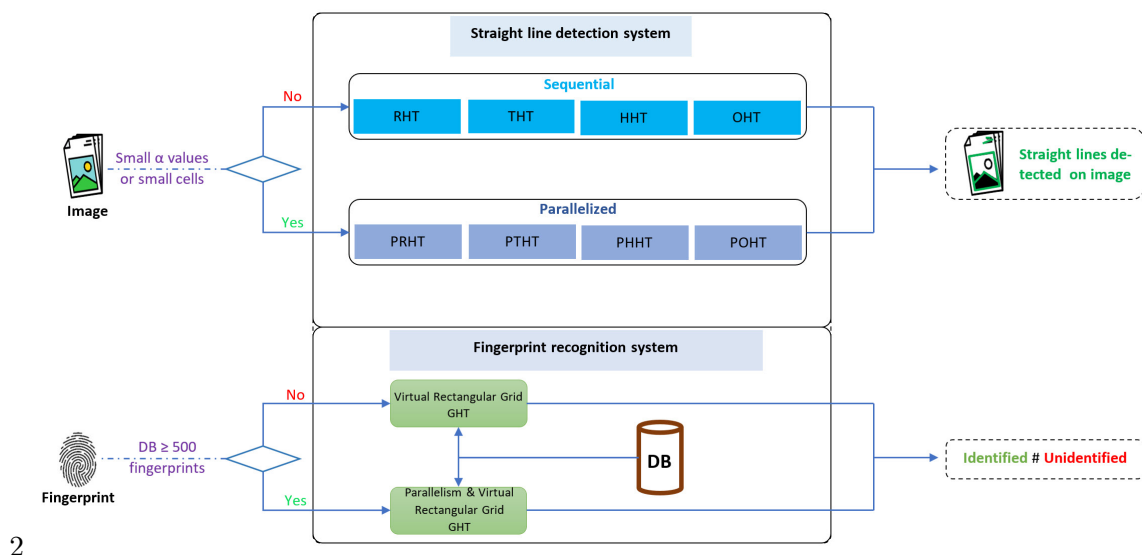


FIGURE 7.2 – Activités dans la boîte à outils

Pour la reconnaissance d’empreintes digitales, lorsque la base de données d’empreintes digitales est volumineuse (supérieur à 500), il est préférable d’utiliser la parallélisation.

La figure 7.2 illustre l’organisation et les conditions d’utilisation des fonctionnalités de la boîte à outils [109].

1.2 Description

Nous présentons, ici, les détails de l’architecture de la boîte à outils, en mettant en lumière les différents modules et composants qui la composent. Nous explorons en détail sa structure interne, en examinant comment les différentes fonctionnalités sont organisées et interconnectées. Nous abordons également les principaux modules de la boîte à outils, en décrivant leurs rôles et leurs responsabilités respectifs dans le cadre de la détection de lignes droites et de la reconnaissance d’empreintes digitales.

1.2.1 Structure interne

La structure interne de la boîte à outils est illustrée dans la figure 7.3. Le répertoire principal *HoughVG/* contient le package principal également appelé *HoughVG*.

Le fichier *__init__.py* est nécessaire pour indiquer que le répertoire est un package Python. Les modules spécifiés sont placés à l’intérieur du package *HoughVG*.

Le module *__main__.py* peut être utilisé pour lancer la bibliothèque en tant qu’application autonome s’il est exécuté en tant que script principal. *setup.py* est un fichier de configuration courant utilisé pour installer et distribuer des packages Python.

Le dossier *test/* contient des modules de test pour les différents composants de la bibliothèque. Le fichier *README.md* contient des informations sur l’utilisation de la bibliothèque, des exemples, des instructions d’installation.

Le fichier *LICENCE* : La boîte à outils est distribuée sous la licence MIT⁶ qui donne à toute personne recevant le logiciel (et ses fichiers) le droit illimité de l'utiliser, le copier, le modifier, le fusionner, le publier, le distribuer, le vendre et le sous-licencier.

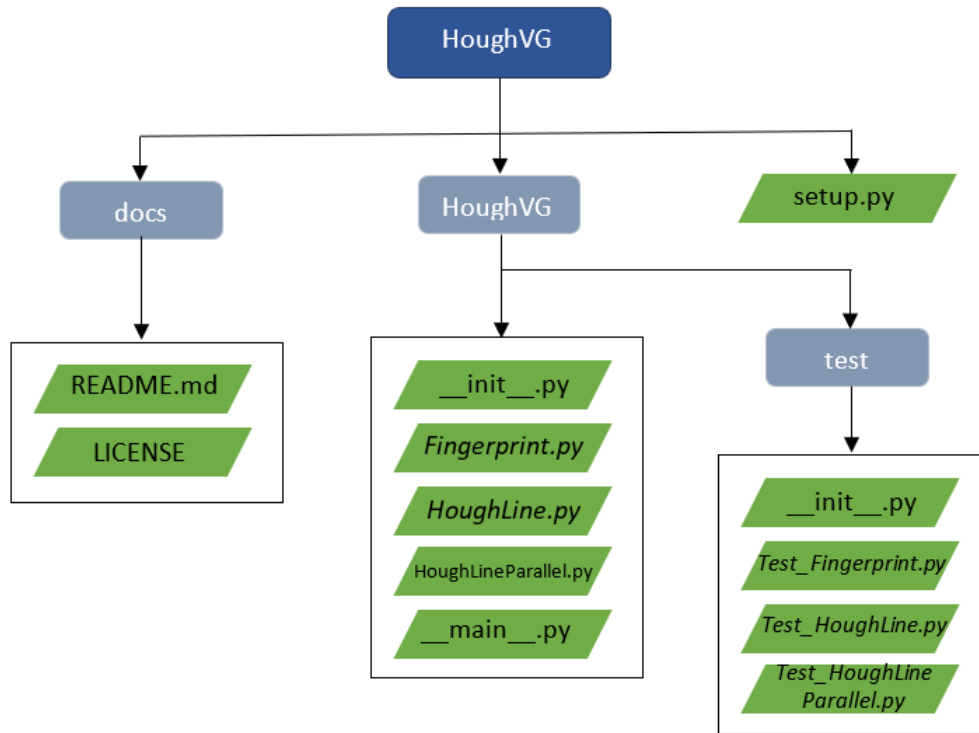


FIGURE 7.3 – Description de la structure interne de HoughVG

1.2.2 Modules

- Module HoughLine : il est composée des fonctions des variantes de la transformée de Hough standard appliqué sur des grilles virtuelles pour la détection de droites décrites dans les chapitres 2, 3, 4 et 5 :

- La fonction de la transformée de Hough rectangulaire (THR) a quatre paramètres d'entrées : l'image, la taille (l, L) des cellules rectangulaires de la grille rectangulaire virtuelle, le taux de pixel allumé α des cellules rectangulaires et le seuil de vote. l et L représentant respectivement la largeur et la longueur des cellules rectangulaire de la grille avec $0 \leq l \leq Nl$ et $0 \leq L \leq Nc$ où Nl et Nc sont respectivement la hauteur et la largeur de l'image *image*. α est un nombre réel compris entre 0 et 1
- La fonction de la transformée de Hough triangulaire (THT) a également quatre paramètres d'entrées : l'image, la taille (h, b) des cellules triangulaires de la grille triangulaire virtuelle, le taux de pixel allumé α des cellules triangulaires et le seuil de vote. Les nombres h et b représentent respectivement la hauteur et la base des cellules triangulaires de la grille avec $0 \leq h \leq Nl$ et

6. https://fr.wikipedia.org/wiki/Licence_MIT

$0 \leq b \leq Nc$ où Nl et Nc sont respectivement la hauteur et la largeur de l'image *imge*. α est un nombre réel compris entre 0 et 1.

- La fonction de la transformée de Hough hexagonale (THH) a aussi quatre paramètres d'entrées : l'image, la taille γ des cellules hexagonale de la grille hexagonale virtuelle, le taux de pixel allumé α des cellules hexagonales et le seuil de vote. $2 \leq \gamma \leq \min(\frac{Nl+2}{2}, \frac{Nc+2}{2})$, défini la taille des cellules hexagonales de la grille virtuelle. Le réel α est compris entre 0 et 1.
- La fonction de la transformée de Hough octogonale (THO) a aussi quatre paramètres d'entrées : l'image, la taille γ des cellules octogonale de la grille octogonale virtuelle, le taux de pixel allumé α des cellules octogonale et le seuil de vote. $1 \leq \gamma \leq \min(\frac{Nl+1}{2}, \frac{Nc+1}{2})$, défini la taille des cellules octogonales de la grille virtuelle et α est un nombre réel compris entre 0 et 1.

- Module HoughLineParallel : Ce module contient des versions parallélisées des fonctionnalités du module HoughLine pour améliorer davantage leurs performances. Les fonctions disponibles sont décrivent dans les même chapitres 2, 3, 4 et 5.

- Module Fingerprint : Ce module propose des fonctionnalités liées à la reconnaissance d'empreintes digitales avec la transformée de Hough généralisée (THG). Les fonctions principales *FingerprintVG* et sa version parallélisée *FingerprintVGP* sont illustré respectivement dans les algorithmes 36 et 37 au chapitre 6.

La fonction *FingerprintVG* prend en paramètre une base de données modèle *BD*, Une empreinte digitale test m_T , un seuil long *seuil_long*, un seuil large *seuil_larg*, un seuil de rotation *seuil_rot* et un seuil de correspondance *Sc*. Lorsque m_T a été correctement identifié à une empreintes digitale dans *BD* alors la fonction *FingerprintVG* affiche un message que l'empreinte digitale m_T a été identifié avec un certain score.

La fonction *FingerprintVGP* est la version parallèle de la fonction *fingerprintVG*. le paramètre *n_cpu* représente le nombre de cœurs du CPU utilisé pour la parallélisation.

Le seuil large *seuil_larg* définit la distance maximale autorisée entre deux minuties pour qu'elles soient considérées comme correspondantes. Un seuil large permet d'accepter des correspondances même si les minuties ne sont pas très proches, ce qui peut être utile pour compenser des erreurs mineures dans le placement des minuties.

Le seuil long *seuil_long*, quant à lui, spécifie la distance maximale pour accepter une correspondance lors de la superposition des minuties. L'utilisation d'un seuil long autorise des correspondances même si les minuties sont relativement éloignées, ce qui peut être bénéfique pour gérer des distorsions ou des variations dans la taille de l'empreinte digitale.

Enfin, le seuil de rotation *seuil_rot* intervient en définissant l'écart angulaire maximal autorisé entre les orientations des minuties correspondantes. Ce paramètre est crucial pour prendre en compte les variations d'orientation entre les minuties, car les empreintes digitales peuvent être capturées avec des orientations différentes.

1.3 Les dépendances

L'environnement logiciel choisi pour développer la boîte à outils est Python 3.11. Les fichiers exécutables en Python ont l'extension ".py", et l'environnement de développement que nous utilisons est la version 1.79.2 de Visual Studio, car il intègre de nombreuses bibliothèques d'usage scientifique indispensables au bon fonctionnement de la bibliothèque *HoughVG*. Nous pouvons citer :

- pypm 0.5.0 : c'est un outil pour la programmation parallèle en Python. Conçue pour tirer parti des architectures multiprocesseurs, pypm facilite l'implémentation de programmes parallèles en fournissant des structures de contrôle simples et des mécanismes de synchronisation. Il faut noter que pypm ne marche que sur les machines Linux ⁷.
- numpy 1.24.2 : c'est une bibliothèque destinée à manipuler des matrices ou tableaux multidimensionnels ainsi que des fonctions mathématiques opérant sur ces tableaux.
- matplotlib 3.6.3 : c'est une bibliothèque complète pour la création de visualisations statiques, animées et interactives en Python.
- scipy 1.10.1 : c'est une bibliothèque open-source utilisée pour résoudre des problèmes mathématiques, scientifiques, d'ingénierie et techniques.
- opencv 7.0.72 : c'est une bibliothèque libre de vision par ordinateur écrite en C et C++, utilisable sous Linux, Windows et Mac OS X. Des interfaces ont été développées pour Python, Matlab et d'autres langages. opencv est orientée vers des applications en temps réel, visant à aider les développeurs à construire rapidement des applications sophistiquées de vision à l'aide d'une infrastructure simple de vision par ordinateur.

2 Interface graphique

L'interface graphique de la boîte à outils *HoughVG* de la transformée de Hough a été conçue pour permettre aux utilisateurs de naviguer facilement et d'exploiter pleinement les fonctionnalités offertes. Le tableau de bord, présenté sur la figure 7.4, s'affiche au lancement de la boîte à outils. Le tableau de bord présente deux boutons d'option pour le système de détection de droites et le système de reconnaissance d'empreintes digitales.

2.1 Scénarios d'utilisation du système de détection de droites

Lorsque l'utilisateur sélectionne la détection de droites, l'interface graphique envoie la requête au système qui retourne ensuite les différents types de transformée de Hough. L'interface graphique les récupère puis les affiche.

L'utilisateur peut alors choisir le type de TH envoyer la requête à travers l'interface graphique au système. Le système de détection de droites envoie alors les paramètres correspondant aux types de

⁷. <https://github.com/classner/pypm>

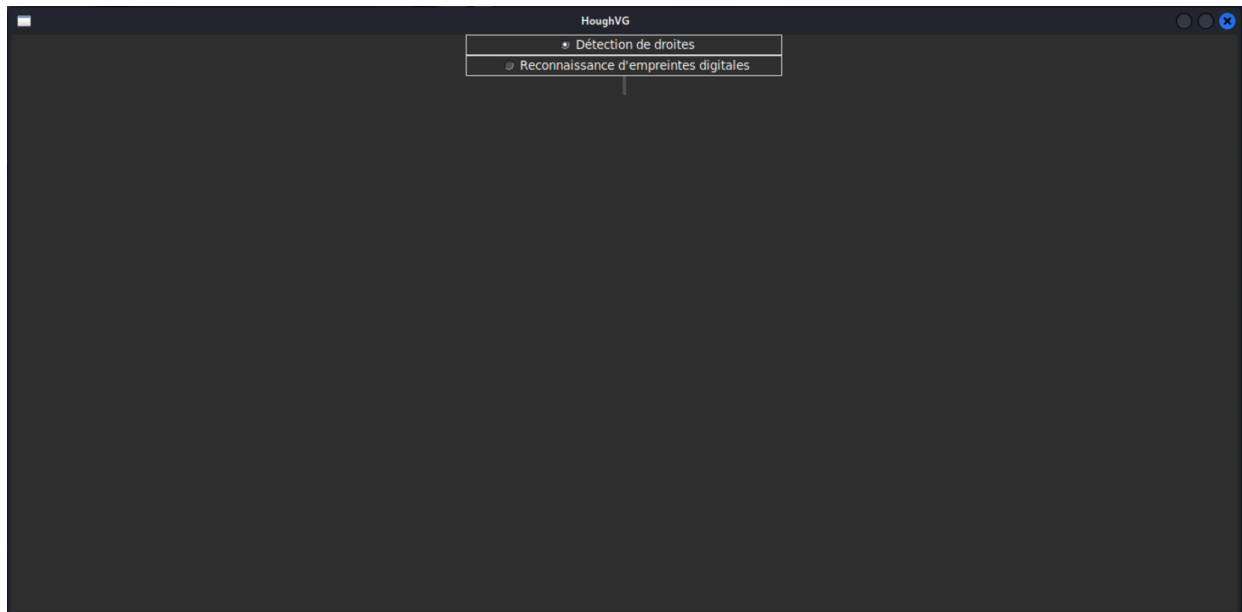


FIGURE 7.4 – Tableau de bord

transformée de Hough choisi à l'interface utilisateur qui les affiche.

L'utilisateur entre alors les paramètres et envoi, a travers l'interface utilisateur, au système. Le système traite les données envoi le résultat de la détection à l'interface graphique qui l'affiche.

L'utilisateur passe alors à l'analyse et à l'interprétation du résultat affiché.

Ces scénario sont illustré dans la figure 7.5 [109].

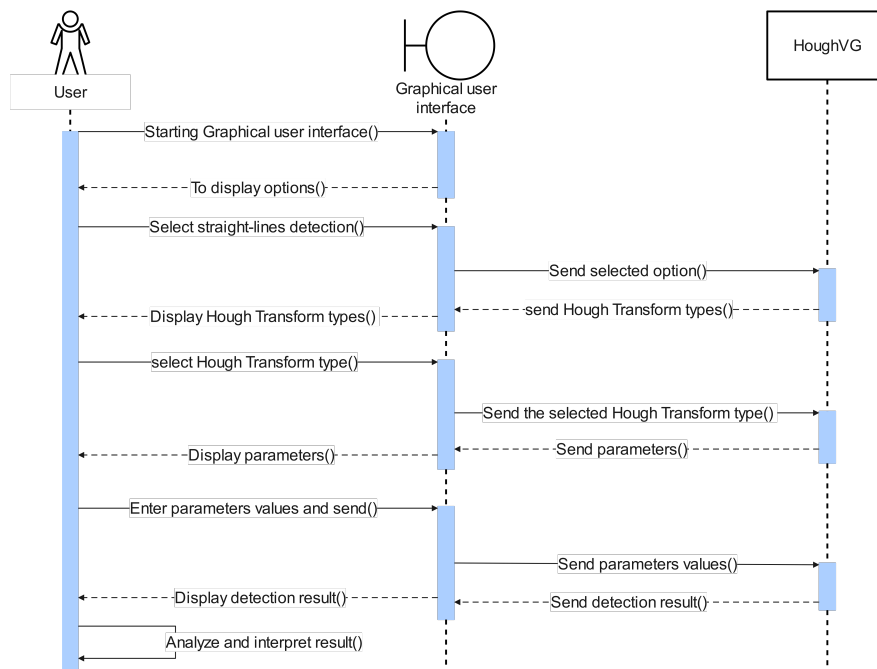


FIGURE 7.5 – Séquence des scénarios d'utilisation du système de détection des droites

2.2 Scénarios d'utilisation du système de reconnaissance d'empreintes digitales

Lorsque l'utilisateur sélectionne la reconnaissance d'empreintes, l'interface graphique envoie la requête au système qui retourne ensuite les paramètres correspondant. L'interface graphique les récupère puis les affiche.

L'utilisateur entre alors les paramètres et envoie, à travers l'interface utilisateur, au système. Le système traite les données puis envoie le résultat à l'interface graphique qui l'affiche.

L'utilisateur passe alors à l'analyse et à l'interprétation du résultat affiché.

Ces scénarios sont illustrés dans la figure 7.6 [109].

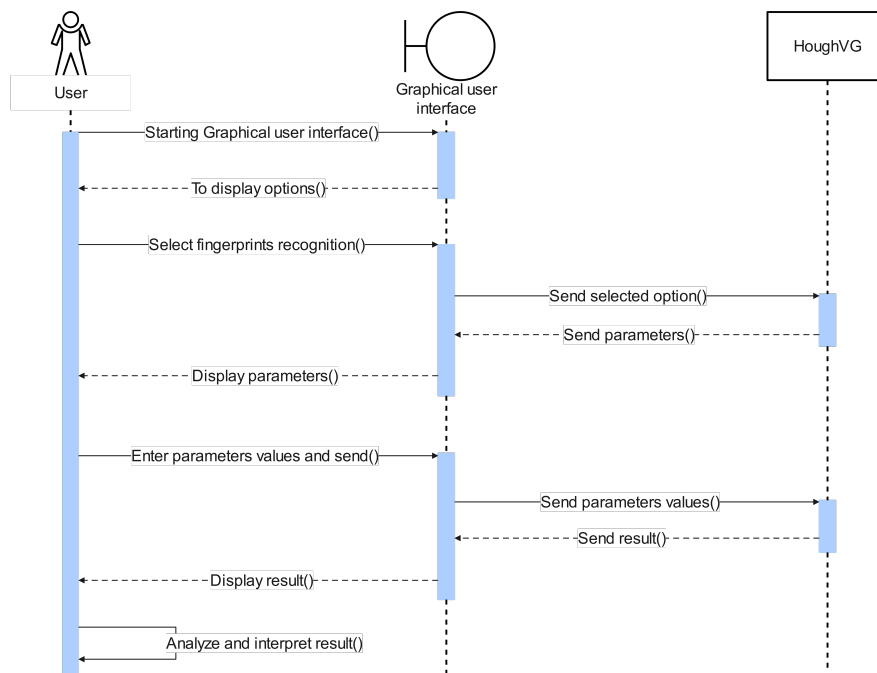


FIGURE 7.6 – Séquence des scénarios d'utilisation du système de reconnaissance d'empreintes digitales

3 Impact

La boîte à outils peut être utile dans :

- Recherche Académique et Industrielle : La disponibilité de ces outils optimisés peut stimuler la recherche de nouvelles applications et algorithmes basés sur la transformée de Hough. Les chercheurs peuvent se concentrer sur l'innovation plutôt que sur les aspects techniques de l'implémentation.
- Communauté Open-Source : Étant donné sa nature de source ouverte, HoughVG est accessible aux développeurs qui peuvent non seulement l'utiliser dans un projet mais également contribuer à son développement continu.
- Industrie et Robotique : La détection précise des droites est cruciale pour la navigation robotique,

l'assemblage automatisé et la surveillance industrielle. Ce logiciel peut permettre aux robots de mieux comprendre leur environnement et de prendre des décisions plus rapides et précises.

- Imagerie médicale : Dans le domaine médical où une détection rapide et précise est essentielle, ce logiciel peut améliorer la détection des structures linéaires dans les images, comme les rayons X et les IRM. Cela permet aux médecins de poser des diagnostics plus précis et de planifier des interventions avec plus de confiance.
- Sécurité et biométrie : Dans les systèmes de gestion des empreintes digitales, comme ceux utilisés par les forces de l'ordre ou les services de sécurité, le traitement en masse est courant. Les variantes optimisées par parallélisme permettent de traiter un grand volume d'empreintes digitales de manière plus rapide et plus efficace.
- Transport : Les véhicules autonomes dépendent de systèmes de vision sophistiqués pour naviguer de manière sûre. La transformée de Hough optimisée peut améliorer la capacité de ces systèmes à détecter et interpréter l'environnement routier.

4 Limitations

Nous pouvons énumérer deux limites majeures. La première est que HoughVG ne peut que détecter les droites et reconnaître les empreintes digitales. Cela limite son domaine d'application. La seconde est que l'ajustement des paramètres optimales pour la détection de droites, n'est pas automatique. Il se fait manuellement en fonction de l'image. Ce qui peut s'avérer pénible.

5 Validation des hypothèses de recherche

Dans le cadre de cette étude, nous avons formulé deux hypothèses clés visant à améliorer l'efficacité de la transformée de Hough pour la détection d'objets discrets. Les résultats expérimentaux confirment ces hypothèses, comme détaillé ci-dessous :

- Impact des grilles virtuelles sur le temps de traitement et la précision : L'intégration des grilles virtuelles a permis une réduction significative du temps de traitement, tout en maintenant un niveau de précision comparable à celui de la méthode traditionnelle.

Pour la détection des droites discrètes, nous avons obtenu un facteur d'accélération $A = 3,26$ (sur l'image de l'immeuble) et $A = 1,9$ (sur l'image de l'autoroute) avec la grille rectangulaire, un facteur d'accélération $A = 2,28$ (sur l'image de l'immeuble) et $A = 1,31$ (sur l'image de l'autoroute) avec la grille triangulaire, un facteur d'accélération $A = 3,5$ (sur l'image de l'immeuble) et $A = 1,05$ (sur l'image de l'autoroute) avec la grille hexagonale, un facteur d'accélération $A = 2,22$ (sur l'image de l'immeuble) et $A = 1,9$ (sur l'image de l'autoroute) avec la grille octogonale le tout assortie d'une perte négligeable de précision. Pour la reconnaissance des empreintes digitales le facteur d'accélération est

Images	Variantes de la TH	Seuil de votes	Taille cellulaire(px)	Surface cellulaire(px ²)	Taux de pixels allumés(α%)	Droites détectées	Temps (sec)	Facteur d'accélération
Immeuble	THS	116				10	0,49	
	THH	50	y=4	42	37	10	0,14	3,5
	THO	53	y=2	6	30	11	0,22	2,22
	THR	35	(l=5, l=7)	35	35	11	0,15	3,26
	THT	38	(h=7, b=5)	16	40	11	0,17	2,28
Autoroute	THS	100				10	0,21	
	THH	67	y=3	20	25	11	0,20	1,05
	THO	38	y=2	6	30	10	0,11	1,9
	THR	51	(l=3, l=5)	15	40	11	0,11	1,9
	THT	57	(h=5, b=3)	7	30	11	0,16	1,31

TABLEAU 7.1 – Facteurs d'accélération

de 1,6 avec une précision pouvant aller jusqu'à 1 en fonction du seuil de correspondance. Ces résultats sont illustrés dans le tableau 7.1

- Combinaison des deux approches : Pour la détection des droites discrètes, la combinaison des grilles virtuelles et le parallélisme a apporté une amélioration dans le temps de traitement dans le cas des grilles de petites cellules et des faibles taux α de pixel allumé comme illustré dans le tableau 7.2. En ce qui concerne la reconnaissance des empreintes digitales, cette combinaison a produit des résultats qui corroborent l'hypothèse dans le cas des bases de données volumineuse (au moins 500 empreintes digitales) avec un facteur d'accélération $A = 2,19$. Cette approche hybride a pu maintenir également la qualité de la précision. Ces résultats démontrent que l'optimisation proposée, basées sur l'utilisation

Paramètres fixés	Grilles virtuelles	Seuil de votes	Surface cellulaire	Taux de pixels allumés(%)	Comparaison des temps d'exécution
Seuil de votes et Surface cellulaire	Hexagonale	60	42	$10 \leq \alpha \leq 30$	$THHP \leq THH$
				$30 \leq \alpha \leq 35$	$THHP \geq THH$
	Octogonale	100	97	$10 \leq \alpha \leq 28$	$THOP \leq THO$
				$28 \leq \alpha \leq 35$	$THOP \geq THO$
	Rectangulaire	70	15	$10 \leq \alpha \leq 35$	$THRP \leq THR$
Seuil de votes et Taux de pixels allumés	Triangulaire	60	7	$35 \leq \alpha \leq 45$	$THRP \geq THR$
				$10 \leq \alpha \leq 55$	$THTP \leq THT$
	Hexagonale	50	$6 \leq S_H \leq 57$	$57 \leq S_H \leq 72$	$THHP \leq THH$
					$THHP \geq THH$
	Octogonale	70	$21 \leq S_O \leq 229$	$229 \leq S_O \leq 417$	$THOP \leq THO$
Seuil de votes et Surface cellulaire	Rectangulaire	50	$8 \leq S_R \leq 48$	$48 \leq S_R \leq 99$	$THRP \leq THR$
					$THRP \geq THR$
	Triangulaire	80	$4 \leq S_T \leq 29$		$THTP \leq THT$

(a) Immeuble

(b) Autoroute

TABLEAU 7.2 – Comparaison du temps de traitement de l'approche hybride et celle basée sur les grilles virtuelles

de grilles virtuelles est une solution pour améliorer la transformée de Hough. La combinaison avec le parallélisme est également Elles permettent non seulement d'accélérer les traitements, mais aussi de maintenir une bonne précision, répondant ainsi aux exigences des applications en temps réel.

Conclusion

La boîte à outils HoughVG, basée sur la transformée de Hough, intègre des algorithmes optimisés pour la détection de droites et la reconnaissance d'empreintes digitales, cette boîte à outils offre des solutions pour une variété d'applications.

L'organisation modulaire de HoughVG, ainsi que son interface graphique simplifié, rendent l'outil accessible à un large éventail d'utilisateurs, des chercheurs académiques aux professionnels de l'industrie. Les fonctionnalités de la boîte à outils améliorent considérablement les performances, permettant

de traiter de grandes quantités de données rapidement et efficacement.

Cependant, la boîte à outils n'est pas sans limitations. La qualité des résultats dépend fortement du paramétrage précis des algorithmes. De plus, les cas d'utilisation se limitent aux droites et aux empreintes digitales.

Malgré ces défis, l'impact de HoughVG est significatif. Elle permet non seulement d'améliorer les systèmes de détection de droites et de reconnaissance d'empreintes digitales existants, mais ouvre également la voie à de nouvelles recherches et développements dans le domaine de la vision par ordinateur. Les futures améliorations pourraient inclure l'extension de ses capacités à d'autres formes géométriques et l'automatisation du paramétrage pour simplifier davantage son utilisation.

Conclusion Générale

Ce document met en évidence la richesse et la diversité des variantes de la transformée de Hough, chacune offrant des perspectives uniques dans la détection d'objets dans les images. Les analyses des transformées de Hough rectangulaire, triangulaire, hexagonale et octogonale révèlent les nuances de chaque approche, en mettant en évidence leurs avantages respectifs en termes de précision, de flexibilité et de complexité algorithmique. Le seuil de vote est un paramètre très crucial. En effet l'efficacité de ces approches repose sur l'utilisation de seuil de vote faible.

Ainsi, les quatre variantes de la transformée de Hough (THR, THT, THH, THO) proposée offre chacun des possibilités de réduction du temps de traitement tout en maintenant une bonne précision détection des droites discrètes. En comparaison, la transformée de Hough rectangulaire offre un temps de traitement plus rapide que la transformée de Hough triangulaire. De plus, en termes de flexibilité des paramètres, la transformée de Hough rectangulaire est préférable. En effet, il est possible d'ajuster les paramètres de la transformée de Hough rectangulaire pour cibler la détection soit sur les droites verticales, soit sur les droites horizontales.

Aussi, de façon générale, la combinaison du parallélisme et des nouvelles variantes de la transformée de Hough dans le cadre de la détection des droites discrètes améliore le temps de traitement pour les petites valeurs de α et les grilles virtuelles de petites cellules, mais cette amélioration diminue à mesure que les valeurs de α et la taille des cellules augmentent.

En outre, l'introduction des grilles virtuelles dans le processus de la reconnaissance des empreintes digitales a permis d'améliorer le temps de traitement. La combinaison du parallélisme et des grilles virtuelles offre des solutions pour répondre aux exigences de traitement en temps réel et pour gérer efficacement de grandes quantités de données dans le cadre de la reconnaissance d'empreintes digitales. Par contre, pour les petites bases de données, il est préférable d'utiliser l'approche des grilles virtuelles.

Enfin, une boîte à outils nommée HoughVG, sous forme d'une bibliothèque Python munie d'une interface graphique utilisateur et regroupant toutes les implémentations d'algorithmes, a été mise en place.

Les travaux futurs viseront à intégrer les pratiques relatives aux régions d'intérêt (ROI) dans le pré-traitement des images pour améliorer la précision de la détection, intégrer les algorithmes d'intelligence artificielle notamment les CNNs dans notre approche, étendre la boîte à outils à la reconnaissance d'autres formes dans les domaines de la santé et de l'agriculture.

Bibliographie

- [1] A. Sere, *Transformations Analytiques appliquées aux Images multi-échelles et bruitées* Laboratoire de Traitement de l'Information et la Communication, Université de Ouagadougou, juin 2013.
- [2] M. Rodriguez, *Redimensionnement adaptatif et reconnaissance de primitives discrètes* Université de Poitiers (France), Thèse en informatique & applications, 2011.
- [3] A. SERE, Y. TRAORE, and F. T. OUEDRAOGO, *Towards New Analytical Straight Line Definitions and Detection with the Hough Transform Method* International Journal of Engineering Trends and Technology (IJETT), vol. 62, 2018. DOI : 10.14445/22315381/IJETT-V62P212.
- [4] C. A. D. G. Traoré and A. Séré, *Straight-Line Detection with the Hough Transform Method Based on a Rectangular Grid* pp. 599–610, 2022.
- [5] A. bilal, *Conception d'un système de Reconnaissance des empreintes digitales par apprentissage* Université des sciences et de la technologie d'Oran Mohammed Boudiaf, 2012.
- [6] *Conception d'un système de Reconnaissance des empreintes digitales par apprentissage* Université des sciences et de la technologie d'Oran Mohammed Boudiaf, faculté de génie électrique département d'électronique, 2012.
- [7] D. Ballard, *Generalizing the Hough transform to detect arbitrary shapes* Pattern Recognition, vol. 13, no. 2, pp. 111–122, 1981. DOI : [https://doi.org/10.1016/0031-3203\(81\)90009-1](https://doi.org/10.1016/0031-3203(81)90009-1).
- [8] A. Katru and A. Kumar, *Improved Parallel Lane Detection Using Modified Additive Hough Transform* International Journal of Image, Graphics and Signal Processing(IJIGSP), vol. 8, pp. 10–17, 2016. DOI : 10.5815/ijigsp.2016.11.02.
- [9] D. Geman and B. Jedynak, *An active testing model for tracking roads in satellite images* IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 18, no. 1, pp. 1–14, 1996. DOI : 10.1109/34.476006.
- [10] F. Benkouider, L. Hamami, A. Abdellaoui, and M. Salmon, *Extraction de routes par classification supervisée et par réseaux de neurones artificiels à partir d'image spot : cas d'une ville oasienne (algérie)* Teledetection, vol. 11, pp. 237–249, Aug. 2012.

- [11] A. Sere, C. A. D. G. Traore, Y. Traore, and O. Sie, “Towards traffic saturation detection based on the hough transform method,” in *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2* (K. Arai, S. Kapoor, and R. Bhatia, eds.), (Cham), pp. 263–270, Springer International Publishing, 2021.
- [12] M. Marzougui, A. Alasiry, Y. Kortli, and J. Baili, *A Lane Tracking Method Based on Progressive Probabilistic Hough Transform* IEEE Access, vol. 8, pp. 84893–84905, 2020.
- [13] C. Yang and J. Collins, *Improvement of Honey Bee Tracking on 2D Video with Hough Transform and Kalman Filter* J Sign Process Syst, vol. 90, pp. 1639–1650, 2018. DOI : <https://doi.org/10.1007/s11265-017-1307-x>.
- [14] E. Widyaningrum, B. Gorte, and R. Lindenbergh, *Automatic Building Outline Extraction from ALS Point Clouds by Ordered Points Aided Hough Transform* Remote Sensing, vol. 11, no. 14, 2019. DOI : 10.3390/rs11141727.
- [15] B. Liao, J. Li, Z. Ju, and G. Ouyang, “Hand gesture recognition with generalized hough transform and dc-cnn using realsense,” in *2018 Eighth International Conference on Information Science and Technology (ICIST)*, pp. 84–90, 2018. DOI : 10.1109/ICIST.2018.8426125.
- [16] G. Lin, Y. Tang, and X. Z. et al., *Fruit detection in natural environment using partial shape matching and probabilistic Hough transform* Precision Agric, vol. 21, pp. 160–177, 2020.
- [17] G. A. Soares, D. Abdala, and M. Escarpinati, “Plantation rows identification by means of image tiling and hough transform,” in *Proceedings of the 13th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISGRAPP 2018) - Volume 4 : VISAPP*, pp. 453–459, INSTICC, SciTePress, 2018. DOI : 10.5220/0006657704530459.
- [18] W. Winterhalter, F. V. Fleckenstein, C. Dornhege, and W. Burgard, *Crop Row Detection on Tiny Plants With the Pattern Hough Transform* IEEE Robotics and Automation Letters, vol. 3, no. 4, pp. 3394–3401, 2018. DOI : 10.1109/LRA.2018.2852841.
- [19] C. Campi, A. Perasso, M. C. Beltrametti, A. M. Massone, G. Sambuceti, and M. Piana, *Pattern recognition in medical imaging by means of the Hough transform of curves* pp. 280–283, 2013. DOI : 10.1109/ISPA.2013.6703753.
- [20] R. Vijayarajeswari, P. Parthasarathy, S. Vivekanandan, and A. A. Basha, *Classification of mammogram for early detection of breast cancer using SVM classifier and Hough transform* Measurement, vol. 146, pp. 800–805, 2019. DOI : <https://doi.org/10.1016/j.measurement.2019.05.083>.
- [21] C. Zhang, F. Wang, Y. Zou, J. Dimyadi, B. H. Guo, and L. Hou, *Automated UAV image-to-BIM registration for building façade inspection using improved generalised Hough transform* Automation in Construction, vol. 153, p. 104957, 2023. DOI : <https://doi.org/10.1016/j.autcon.2023.104957>.

- [22] P. N and C. Y., *Transformée de Hough et détection de linéaments sur images satellitaires et modèles numériques sur terrain* Unité de Géomatique, Département de Géographie, Université de Liège, Belgique, vol. 54, pp. 145–56, 2010.
- [23] A. C. Freeman, W. Shi, and B. Hwang, “Enhancing surveillance camera fov quality via semantic line detection and classification with deep hough transform,” 2024.
- [24] P. Hough, *Method and means for recognizing complex patterns* In United States Patent, vol. 3069654, pp. 47–64, 1962.
- [25] J. Illingworth and J. Kittler, *A survey of the hough transform* Computer Vision, Graphics, and Image Processing, vol. 44, no. 1, pp. 87–116, 1988. DOI : [https://doi.org/10.1016/S0734-189X\(88\)80033-1](https://doi.org/10.1016/S0734-189X(88)80033-1).
- [26] J. Cha, R. Cofer, and S. Kozaitis, *Extended Hough transform for linear feature detection* Pattern Recognition, vol. 39, no. 6, pp. 1034–1043, 2006. DOI : <https://doi.org/10.1016/j.patcog.2005.05.014>.
- [27] C. Galamhos, J. Matas, and J. Kittler, “Progressive probabilistic hough transform for line detection,” in *Proceedings. 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No PR00149)*, vol. 1, pp. 554–560 Vol. 1, 1999. DOI : [10.1109/CVPR.1999.786993](https://doi.org/10.1109/CVPR.1999.786993).
- [28] L. S. Davis, *Hierarchical generalized Hough transforms and line-segment based generalized Hough transforms* Pattern Recognition, vol. 15, no. 4, pp. 277–285, 1982. DOI : [https://doi.org/10.1016/0031-3203\(82\)90030-9](https://doi.org/10.1016/0031-3203(82)90030-9).
- [29] H. Rhody and C. F. Carlson, *Hough Circle Transform* Center for Imaging Science, Rochester Institute of Technology, 2005.
- [30] N. Kiryati, Y. Eldar, and A. Bruckstein, *A probabilistic Hough transform* Pattern Recognition, vol. 24, no. 4, pp. 303–316, 1991. DOI : [https://doi.org/10.1016/0031-3203\(91\)90073-E](https://doi.org/10.1016/0031-3203(91)90073-E).
- [31] D. Luo, X. He, Q. Teng, and Q. Tao, *Triplet circular Hough transform for circle detection in Journal of electronics (China)* vol. 19, pp. 356–362, 2002.
- [32] R. O. Duda and P. E. Hart, *Use of the Hough transformation to detect lines and curves in pictures* Commun. ACM, vol. 15, pp. 11–15, 1972.
- [33] A. Y. S. Chia, M. K. H. Leung, H.-L. Eng, and S. Rahardja, “Ellipse detection with hough transform in one dimensional parametric space,” in *2007 IEEE International Conference on Image Processing*, vol. 5, pp. V – 333–V – 336, 2007. DOI : [10.1109/ICIP.2007.4379833](https://doi.org/10.1109/ICIP.2007.4379833).
- [34] W. Lu and J. Tan, *Detection of incomplete ellipse in images with strong noise by iterative randomized Hough transform (IRHT)* Pattern Recognition, vol. 41, no. 4, pp. 1268–1279, 2008. DOI : <https://doi.org/10.1016/j.patcog.2007.09.006>.

- [35] C. Huang, *Elliptical feature extraction via an improved Hough transform* Pattern Recognition Letters, vol. 10, no. 2, pp. 93–100, 1989. DOI : [https://doi.org/10.1016/0167-8655\(89\)90073-1](https://doi.org/10.1016/0167-8655(89)90073-1).
- [36] C. Daul, P. Graebbling, and E. Hirsch, *From the Hough Transform to a New Approach for the Detection and Approximation of Elliptical Arcs* Computer Vision and Image Understanding, vol. 72, no. 3, pp. 215–236, 1998. DOI : <https://doi.org/10.1006/cviu.1998.0696>.
- [37] H. Kälviäinen, P. Hirvonen, L. Xu, and E. Oja, *Probabilistic and non-probabilistic Hough transforms : overview and comparisons* Image and Vision Computing, vol. 13, no. 4, pp. 239–252, 1995. DOI : [https://doi.org/10.1016/0262-8856\(95\)99713-B](https://doi.org/10.1016/0262-8856(95)99713-B).
- [38] K. Khoshelham, “Extending generalized hough transform to detect 3d objects in laser range data,” in *Proceedings of the ISPRS workshop Laser Scanning 2007 and SilviLasser 2007, Espoo, Finland, 12-14 September 2007 / ed. by P. Rönholm, H. Hyypä and J. Hyypä. (ISPRS : Volume XXXVI, part 3/W52. pp.206-210* (P. Rönholm, H. Hyypä, and J. Hyypä, eds.), pp. 206–210, International Society for Photogrammetry and Remote Sensing (ISPRS), 2007.
- [39] J. Lopez-Krahe, T. Alamo-Cantarero, and E. Davila-Gonzalez, *Discrete Hough Transform Applied to Small Size Pattern Recognition* éditeur Télécom, 1994. DOI : 10.13140/RG.2.2.10125.13283.
- [40] I. Svalbe, *Natural representations for straight lines and the Hough transform on discrete arrays* IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 11, no. 9, pp. 941–950, 1989. DOI : 10.1109/34.35497.
- [41] S. Maji and J. Malik, “Object detection using a max-margin hough transform,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1038–1045, 2009. DOI : 10.1109/CVPR.2009.5206693.
- [42] Lopez-Krahe and Pousset, “7 - transformée de hough discrète et bornée. application à la détection de droites parallèles et du réseau routier,” 1988.
- [43] A. Sere, O. Sie, and E. Andres, *Extended Standard Hough Transform for analytical line recognition* pp. 412–422, 2012. DOI : 10.1109/SETIT.2012.6481950.
- [44] K. Zhao, Q. Han, C.-B. Zhang, J. Xu, and M.-M. Cheng, *Deep Hough Transform for Semantic Line Detection* IEEE Transactions on Pattern Analysis and Machine Intelligence, p. 1–1, 2021. DOI : 10.1109/tpami.2021.3077129.
- [45] A. Gabrielli, F. Alfonsi, and F. Del Corso, *Simulated Hough Transform Model Optimized for Straight-Line Recognition Using Frontier FPGA Devices* Electronics, vol. 11, no. 4, 2022. DOI : 10.3390/electronics11040517.
- [46] Q. Liang, J. Long, Y. Nan, G. Coppola, K. Zou, D. Zhang, and W. Sun, *Angle aided circle detection based on randomized Hough transform and its application in welding spots detec-*

- tion Mathematical Biosciences and Engineering, vol. 16, no. 3, pp. 1244–1257, 2019. DOI : 10.3934/mbe.2019060.
- [47] K. GOTO and H. TSUKUNE, *Extracting Elliptical Figures from an Edge Vector Field* Transactions of the Society of Instrument and Control Engineers, vol. 20, no. 4, pp. 329–336, 1984. DOI : 10.9746/sicetr1965.20.329.
- [48] D. Bailey, Y. Chang, and S. Le Moan, *Analysing Arbitrary Curves from the Line Hough Transform* Journal of Imaging, vol. 6, no. 4, 2020. DOI : 10.3390/jimaging6040026.
- [49] M.-L. Torrente, S. Biasotti, and B. Falcidieno, *Recognition of feature curves on 3D shapes using an algebraic approach to Hough transforms* Pattern Recognition, vol. 73, pp. 111–130, 2018. DOI : <https://doi.org/10.1016/j.patcog.2017.08.008>.
- [50] C. Conti, L. Romani, and D. Schenone, *Semi-automatic spline fitting of planar curvilinear profiles in digital images using the Hough transform* Pattern Recognition, vol. 74, pp. 64–76, 2018. DOI : <https://doi.org/10.1016/j.patcog.2017.09.017>.
- [51] C. Romanengo, B. Falcidieno, and S. Biasotti, *Hough transform based recognition of space curves* Journal of Computational and Applied Mathematics, vol. 415, p. 114504, 2022. DOI : <https://doi.org/10.1016/j.cam.2022.114504>.
- [52] A. Sere, F. T. Ouedraogo, and B. Zerbo, “An improvement of the standard hough transform method based on geometric shapes,” in *Advances in Information and Communication Networks* (K. Arai, S. Kapoor, and R. Bhatia, eds.), (Cham), pp. 369–384, Springer International Publishing, 2019.
- [53] W. Yang, Y. Li, T. Hu, R. Fuchikami, and T. Ikenaga, “Relative vectors clustering and temporal constraint based generalized Hough transform for high frame rate and ultra-low delay arbitrary shape detection,” in *Third International Conference on Computer Vision and Information Technology (CVIT 2022)* (J. Ma, ed.), vol. 12590, p. 1259002, International Society for Optics and Photonics, SPIE, 2023. DOI : 10.1117/12.2670011.
- [54] N. Kiryati, Y. Eldar, and A. Bruckstein, *A probabilistic Hough transform* Pattern Recognition, vol. 24, no. 4, pp. 303–316, 1991. DOI : [https://doi.org/10.1016/0031-3203\(91\)90073-E](https://doi.org/10.1016/0031-3203(91)90073-E).
- [55] M. A. Javeed, M. A. Ghaffar, M. A. Ashraf, N. Zubair, A. S. M. Metwally, E. M. Tag-Eldin, P. Bocchetta, M. S. Javed, and X. Jiang, *Lane Line Detection and Object Scene Segmentation Using Otsu Thresholding and the Fast Hough Transform for Intelligent Vehicles in Complex Road Conditions* Electronics, vol. 12, no. 5, 2023. DOI : <https://doi.org/10.3390/electronics12051079>.
- [56] A. C. Huang, C. Yuan, S. H. Meng, and T. J. Huang, *Design of Fatigue Driving Behavior Detection Based on Circle Hough Transform* Big Data, vol. 11, no. 1, pp. 1–17, 2023. DOI : <https://doi.org/10.1089/big.2021.0166>.

- [57] D. A. M. Somda and A. Sere, “Traffic saturation detection using hough transform and vanet,” EAI, 5 2023. DOI : 10.4108/eai.24-11-2022.2329807.
- [58] M. Michałowska, J. Rapiński, and J. Janicka, *Tree position estimation from TLS data using hough transform and robust least-squares circle fitting* Remote Sensing Applications : Society and Environment, vol. 29, p. 100863, 2023. DOI : <https://doi.org/10.1016/j.rsase.2022.100863>.
- [59] W. Hou, *Research on automatic hole making technology of industrial robot based on Hough circle detection algorithm* International Journal of Wireless and Mobile Computing, vol. 24, no. 3/4, pp. 352–360, 2023. DOI : <https://doi.org/10.1504/IJWMC.2023.131325>.
- [60] C. Jugade, D. Hartgers, S. Mohamed, D. Goswami, A. Nelson, G. v. d. Veen, and K. Goossens, *Improved Positioning Precision using a Multi-rate Multi-sensor in Industrial Motion Control Systems* 2023 European Control Conference (ECC), pp. 1–8, 2023. DOI : 10.23919/ECC57647.2023.10178138.
- [61] C. S. Mlambo, *A study on Hough transform-based fingerprint alignment algorithms*. PhD thesis, University of Johannesburg, 2015.
- [62] K. Anitha, T. Sowmya, V. Srija, K. N. V. Rao, and G. Vinatha, *Cryptographic Fingerprint Matching Using The Descriptor-Based Hough Transform* IJSART, vol. 7, pp. 2395–1052, 2021.
- [63] S. Nitika and S. G. Nasib, *Hough Transform Based Fingerprint Matching Using Minutiae Extraction* International Journal of Advanced Research in Computer Science, vol. 4, p. 117, 2013.
- [64] J. Qin, S. Tang, C. Han, and T. Guo, “Partial fingerprint identification algorithm based on the modified generalized hough transform on mobile device,” in *International Conference on Graphic and Image Processing*, 2018.
- [65] F. J. Calero-Castro, S. Pereira, I. Laga, P. Villanueva, G. Suárez-Artacho, C. Cepeda-Franco, P. de la Cruz-Ojeda, E. Navarro-Villarán, S. Dios-Barbeito, M. J. Serrano, C. Fresno, and J. Padillo-Ruiz, *Quantification and Characterization of CTCs and Clusters in Pancreatic Cancer by Means of the Hough Transform Algorithm* International Journal of Molecular Sciences, vol. 24, no. 5, 2023. DOI : 10.3390/ijms24054278.
- [66] A. Sere, *Transformations Analytiques appliquées aux Images multi-échelles et bruitées* Laboratoire de Traitement de l’Information et la Communication, Université de Ouagadougou, juin 2013.
- [67] E. Andres, *Modélisation analytique discrète d’objets géométriques* Université de Poitiers (France), Thèse d’habilitation, 2000.
- [68] M. Dexet, *Architecture d’un modeleur géométrique à base topologique d’objets discrets et méthodes de reconstruction en dimensions 2 et 3* Université de Poitiers (France), Thèse en informatique et applications, 2006.

- [69] R. Jean-Pierre, *Géométrie discrète, calcul en nombres entiers et algorithmique* Université Louis Paster, p. 1991.
- [70] A. Vacavant, *Géométrie discrète sur grilles irrégulières isothétiques* Université Lumière Lyon 2, Thèse en informatique, 2008.
- [71] Freeman, *Algorithm for Generating a Digital Straight Line on a Triangular Grid* IEEE Transactions on Computers, vol. C-28, no. 2, pp. 150–152, 1979. DOI : 10.1109/TC.1979.1675305.
- [72] Luczak and Rosenfeld, *Distance on a Hexagonal Grid* IEEE Transactions on Computers, vol. C-25, no. 5, pp. 532–533, 1976. DOI : 10.1109/TC.1976.1674642.
- [73] B. Kumar, P. Gupta, and K. Pahwa, *Square Pixels to Hexagonal Pixel Structure Representation Technique* International Journal of Signal Processing, Image Processing and Pattern Recognition, vol. 7, pp. 137–144, 2014.
- [74] Y. Lin, S. L. Pintea, and J. C. van Gemert, *Deep Hough-Transform Line Priors* arXiv e-prints, p. arXiv :2007.09493, July 2020. DOI : 10.48550/arXiv.2007.09493.
- [75] J. Wang, P. Fu, and R. X. Gao, *Machine vision intelligence for product defect inspection based on deep learning and Hough transform* Journal of Manufacturing Systems, vol. 51, pp. 52–60, 2019. DOI : <https://doi.org/10.1016/j.jmsy.2019.03.002>.
- [76] M. Kowdiki and A. Khaparde, *Adaptive hough transform with optimized deep learning followed by dynamic time warping for hand gesture recognition* Multimed Tools Appl 81, p. 2095–2126, 2022. DOI : <https://doi.org/10.1007/s11042-021-11469-9>.
- [77] Q.-B. Ta and J.-T. Kim, *Monitoring of Corroded and Loosened Bolts in Steel Structures via Deep Learning and Hough Transforms* Sensors, vol. 20, no. 23, 2020. DOI : 10.3390/s20236888.
- [78] L. Huan, W. Li, and Q. Yujian, “Vehicle logo retrieval based on hough transform and deep learning,” in *2017 IEEE International Conference on Computer Vision Workshops (ICCVW)*, pp. 967–973, 2017. DOI : 10.1109/ICCVW.2017.118.
- [79] A. Sheshkus, A. Ingacheva, and D. Nikolaev, *Vanishing points detection using combination of fast Hough transform and deep learning* vol. 10696, p. 106960H, 2018. DOI : 10.1117/12.2310170.
- [80] W. Zhou, C. Qin, J. Chang, Y. Liu, Y. Chen, M. Feng, R. Wang, W. Yang, and J. Yao, *Standardized measurement of mid-surface shift of brain based on deep Hough transform* Computerized Medical Imaging and Graphics, vol. 108, p. 102284, 2023. DOI : <https://doi.org/10.1016/j.compmedimag.2023.102284>.
- [81] J.-Q. Zhang, H.-B. Duan, J.-L. Chen, A. Shamir, and M. Wang, *HoughLaneNet : Lane detection with deep hough transform and dynamic convolution* Computers and Graphics, vol. 116, pp. 82–92, 2023. DOI : <https://doi.org/10.1016/j.cag.2023.08.012>.
- [82] M. SE, E. R, and D. F, *Champ de hauteurs de la transformée de Hough standard* Laboratoire MOIVRE, Département d’informatique, Faculté des sciences, Université de Sherbrooke, 2005.

- [83] J. Matas, C. Galambos, and J. Kittler, “Progressive probabilistic hough transform,” in *British Machine Vision Conference*, 1998.
- [84] C. Kimme, D. Ballard, and J. Sklansky, *Finding circles by an array of accumulators* Commun. ACM, vol. 18, p. 120–122, feb 1975. DOI : 10.1145/360666.360677.
- [85] H. Yuen, J. Princen, J. Illingworth, and J. Kittler, *Comparative study of Hough Transform methods for circle finding* Image and Vision Computing, vol. 8, no. 1, pp. 71–77, 1990. DOI : [https://doi.org/10.1016/0262-8856\(90\)90059-E](https://doi.org/10.1016/0262-8856(90)90059-E).
- [86] M. M. A. Coelho., D. N. Sugimoto., G. A. Melo., V. V. Curtis., and J. de Melo Bezerra., *A MapReduce based Approach for Circle Detection* pp. 454–459, 2019. DOI : 10.5220/0007827604540459.
- [87] Tsuji and Matsumoto, *Detection of Ellipses by a Modified Hough Transformation* IEEE Transactions on Computers, vol. C-27, no. 8, pp. 777–781, 1978. DOI : 10.1109/TC.1978.1675191.
- [88] E. Davies, *Finding ellipses using the generalised Hough transform* Pattern Recognition Letters, vol. 9, no. 2, pp. 87–96, 1989. DOI : [https://doi.org/10.1016/0167-8655\(89\)90041-X](https://doi.org/10.1016/0167-8655(89)90041-X).
- [89] J. Illingworth and J. Kittler, *The Adaptive Hough Transform* IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-9, no. 5, pp. 690–698, 1987. DOI : 10.1109/TPAMI.1987.4767964.
- [90] A. Séré, D. Colazzo, and O. Sié, *A Hough Transform Based On a MapReduce Algorithm* 2016.
- [91] H. Li, M. A. Lavin, and R. J. Le Master, *Fast Hough transform : A hierarchical approach* Computer Vision, Graphics, and Image Processing, vol. 36, no. 2, pp. 139–161, 1986. DOI : [https://doi.org/10.1016/0734-189X\(86\)90073-3](https://doi.org/10.1016/0734-189X(86)90073-3).
- [92] Y. Li and X. Gan, *An Integrated Fast Hough Transform for Multidimensional Data* IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 9, pp. 11365–11373, 2023. DOI : 10.1109/TPAMI.2023.3269202.
- [93] E. GABRIEL, *Automatic Multi-Scale and Multi-Object Pedestrian and Car Detection in Digital Images Based on the Discriminative Generalized Hough Transform and Deep Convolutional Neural Networks* 2019.
- [94] J. Gall, A. Yao, N. Razavi, L. Van Gool, and V. Lempitsky, *Hough Forests for Object Detection, Tracking, and Action Recognition* IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 11, pp. 2188–2202, 2011. DOI : 10.1109/TPAMI.2011.70.
- [95] D. Karimanzira and H. Renkewitz, “Detection and localization of an underwater docking station in acoustic images using machine learning and generalized fuzzy hough transform,” in *Machine Learning for Cyber Physical Systems* (J. Beyerer, A. Maier, and O. Niggemann, eds.), (Berlin, Heidelberg), pp. 23–31, Springer Berlin Heidelberg, 2021.

- [96] R. Yam-Uicab, J. Lopez-Martinez, and J. Trejo-Sanchez, *A fast Hough Transform algorithm for straight lines detection in an image using GPU parallel computing with CUDA-C J Supercomput*, vol. 73, p. 4823–4842, 2017. DOI : <https://doi.org/10.1007/s11227-017-2051-5>.
- [97] J. Sun and E. Roosenbrand, “Flight contrail segmentation via augmented transfer learning with novel sr loss function in hough space,” 2023.
- [98] sadaf shafi, A. Atif, and H. Shafi, *Automated Document Orientation Correction Using Skew and Inversion Rectification With Hough Line Transformation and Machine Learning Preprints*, 2023. DOI : [10.20944/preprints202303.0326.v1](https://doi.org/10.20944/preprints202303.0326.v1).
- [99] S. Kulandaivel and R. Jeyachitra, *Combined image Hough transform based simultaneous multi-parameter optical performance monitoring for intelligent optical networks* Optical Fiber Technology, vol. 79, p. 103357, 2023. DOI : <https://doi.org/10.1016/j.yofte.2023.103357>.
- [100] K. Oh, D. Seo, I.-S. Oh, and G. Kim, *Hough Soft-Argmax Based Tillage Boundary Detection for Autonomous Tractor* p. 27, 2013. DOI : <http://dx.doi.org/10.2139/ssrn.4442845>.
- [101] A. SERE, M. OUEDRAOGO, and A. K. ATIAMPO, “Parallel hough transform based on object dual and pypm library,” 2022.
- [102] *France, Ministère de l’intérieur et des outre-mer, Gendarmerie National* mai 2024.
- [103] N. Yager and A. Amin, *Fingerprint verification based on minutiae features : a review* Pattern Analysis and Applications, vol. 7, pp. 94–113, 2004. DOI : <https://doi.org/10.1007/s10044-003-0201-2>.
- [104] W. Lukasz, *A minutae based matching algorithms in fingerprint recognition* medical informatics and technologies, vol. 13, pp. 66–71, 2009.
- [105] M. Mehtre, B. Finger, *Fingerprint image analysis for automatic identification* Machine Vision and Applications, vol. 6, pp. 124–139, 1993. DOI : <https://doi.org/10.1007/BF01211936>.
- [106] F. V. N. Cynthia S. Mlambo, Mmamelatelo E. Matheka, *A Study of Hough Transform-based Fingerprint Alignment Algorithms* International Journal of Computer Applications, vol. 103, pp. 1–8, October 2014. DOI : [10.5120/18091-9158](https://doi.org/10.5120/18091-9158).
- [107] Neurotechnology, <https://www.neurotechnology.com/download.html> avril 2024.
- [108] F. Fingerprint vérification competition, <http://bias.csr.unibo.it/fvc2004/download.asp> avril 2024.
- [109] M. Ouedraogo, A. Sere, and C. A. D. G. Traore, *HoughVG :Hough Transform Toolbox for Straight-Line Detection and Fingerprint Recognition* Elsevier, Software Impacts, vol. 22, 2024. DOI : [10.1016/j.simpa.2024.100709](https://doi.org/10.1016/j.simpa.2024.100709).



Maître de conférences en informatique

Vérification après la soutenance

5%
Textes
suspects



1% Similitudes
< 1% similitudes entre
guillemets
< 1% parmi des sources
mentionnées
4% Langues non reconnues

Nom du document: main_14_01_2025.pdf
ID du document: fd8b89f28a2a025489aa73d28b378690c326f921
Taille du document d'origine: 15,47 Mo
Auteur: sere abdoulaye

Déposant: sere abdoulaye
Date de dépôt: 02/02/2025
Type de dépôt: url_submission
date de fin d'analyse: 02/02/2025

Nombre de mots: 75 359
Nombre de caractères: 426 825

Emplacement des similitudes dans le document:



Sources des similitudes

Sources principales détectées

N°	Description	Similitudes	Emplacements	Informations complémentaires
1	doi.org Straight-Line Recognition Using a Triangular Grid SpringerLink https://doi.org/10.1007/978-3-030-98012-2_45 1 source similaire	< 1%		Mots identiques: < 1% (69 mots)
2	www.memoireonline.com Memoire Online - Etude des méthodes de reconnaissances-dempre... https://www.memoireonline.com/08/21/12173/Etude-des-methodes-de-reconnaissances-dempre... 4 sources similaires	< 1%		Mots identiques: < 1% (71 mots)
3	www.ijser.org https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf 2 sources similaires	< 1%		Mots identiques: < 1% (65 mots)
4	dspace.univ-tlemcen.dz http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf 10 sources similaires	< 1%		Mots identiques: < 1% (62 mots)
5	theses.hal.science https://theses.hal.science/tel-00009742v1/document 5 sources similaires	< 1%		Mots identiques: < 1% (42 mots)

Sources avec similitudes accidentelles

N°	Description	Similitudes	Emplacements	Informations complémentaires
1	bib.univ-oeb.dz http://bib.univ-oeb.dz:8080/jspui/bitstream/123456789/6791/1/memoire.pdf.pdf	< 1%		Mots identiques: < 1% (40 mots)
2	fr.slideshare.net Biométrie d'Empreinte Digitale PDF https://fr.slideshare.net/slideshow/biomtrie-dempreinte-digitale/34684848	< 1%		Mots identiques: < 1% (25 mots)
3	dspace.univ-tlemcen.dz http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf	< 1%		Mots identiques: < 1% (30 mots)
4	liris.cnrs.fr https://liris.cnrs.fr/Documents/Liris-3688.pdf	< 1%		Mots identiques: < 1% (29 mots)
5	Document d'un autre utilisateur #e1c200 Le document provient d'un autre groupe	< 1%		Mots identiques: < 1% (23 mots)

Sources mentionnées (sans similitudes détectées) Ces sources ont été citées dans le document sans trouver de similitudes.

- <http://dx.doi.org/10.14569/IJACSA.2022.0131084>
- <http://dx.doi.org/10.4108/eai>
- <http://dx.doi.org/10.4108/eai.18-12-2023.2348107>
- https://fr.wikipedia.org/wiki/Licence_MIT

Secrétariat Général

Université Nazi BONI

École Doctorale Sciences et Techniques
N° d'ordre :...

Thèse de doctorat unique en informatique
En vue de l'obtention du grade de docteur de l'Université Nazi BONI

Spécialité : Traitement d'Images

Vers une boîte à outils de la transformée de
Hough pour la détection des objets discrets

dans des grilles virtuelles

Présentée par : OUEDRAOGO Moïse

Soutenue le 9 Janvier 2025, devant le jury composé de :

Président : Pr Oumarou SIE, Professeur titulaire en informatique, Université Aube Nouvelle

Examineur : Pr Malo SADOUANOUAN, Professeur titulaire en informatique, Université Nazi BONI

Rapporteur : Pr Frédéric T. OUEDRAOGO, Professeur titulaire en informatique, Université Norbert Zongo

Rapporteur : Dr Telesphore B. TIENDREBEOGO, Maître de Conférences en informatique, Université Nazi BONI

Rapporteur : Dr Tiémoman KONE, Maître de recherche en informatique, Université virtuelle de Côte d'Ivoire

Directeur de thèse : Dr Abdoulaye SERE, Maître de Conférences en informatique, Université Nazi BONI

Année universitaire 2024/2025

Dédicace

Je dédie ce modeste travail :

À Jésus-Christ, pour le don de l'intelligence, de la sagesse, de la protection et de la
vie.

À mes parents, OUEDRAOGO Moumouni Sylvain et OUEDRAOGO Nongasida
Rosalie. Ils ont pris soin de moi et m'ont toujours encouragé. Ils prient pour moi
tous les jours.

Une dédicace spéciale à ma maman. Elle a veillé sans relâche sur moi et s'est sacrifiée
pour mes études. Elle est, en grande partie, la raison pour laquelle je me suis toujours
battu dans la vie.

À ma grande sœur OUEDRAOGO Monique et à mes petits frères OUEDRAOGO
Élisée Wendéfangé, OUEDRAOGO Ezékiel Bouwendmanegré, OUEDRAOGO En-
ock Relwendé qui m'ont été d'un grand soutien.

À l'amour de ma vie, BAFIOGO Marcelline, qui me soutient sans faille, même lorsque
tout le monde est contre moi. Durant ces trois années de travaux, elle a été ma force
et mon inspiration.

À mon Pasteur, l'Apôtre OUEDRAOGO Paul, qui n'a cessé de prier pour moi jour
et nuit.

- Moïse OUEDRAOGO

Remerciements

Je tiens à exprimer ma profonde gratitude aux entités et aux personnes sans lesquelles ce projet n'aurait jamais vu le jour. Mes sincères remerciements vont en particulier :

Aux membres du jury qui ont accepté de lire ce travail et de l'évaluer.

Au Dr Abdoulaye SERE. Il a eu confiance en moi et a été d'un soutien indéfectible durant les moments difficiles de la thèse. C'est un homme très ouvert, rigoureux, travailleur et très humble. j'ai beaucoup appris à ses côtés. C'est un mentor pour moi.

Au Pr Tiemonan KONE, Directeur Général de l'Université Virtuelle de Côte d'Ivoire (UVCI), et ses collaborateurs pour l'accueil chaleureux au sein de l'Unité de Recherche et d'Expertise Numérique (UREN) lors de mon séjour scientifique à l'UVCI.

Au Dr André Conseibo, Responsable du Laboratoire L@mia de l'Université Norbert Zongo de m'avoir ouvert les portes du laboratoire L@mia pour mon séjour de recherche. Le Pr Frédéric Ouedraogo, malgré qu'il était en voyage, a veillé à ce que je sois bien installé et n'a cessé de prendre de mes nouvelles durant tous mon séjour. Les doctorants du laboratoire, notamment Emile OUEDRAOGO, Jean SAMA, Augustin KABORE, ont favorisé mon intégration au sein du laboratoire.



Au Pr Boureima SANGARE, Dr Telesphore TIENDREBEOGO, Dr SOME Borli Michel Jonas,

pour leurs soutiens, conseils et encouragements.



Au Dr Ouedraogo Thomas, Dr Boulou Mahamadi, Dr Traoré Cheick Amed, Traoré Go Issa, Bammogo Moussa, Somda Augustin, Dabonné Hoda, Ouedraogo Arthur, Kaboré Abdoulaye. Nous avons

partagé de nombreuses expériences durant ces années de thèse.

ii

Abstract

The main objective of this thesis is to develop a toolbox for the Hough transform to efficiently detect discrete objects in images, by integrating temporal optimization techniques, parallel programming and specific adaptation for different types of applications. To this end, discrete mathematics (topology and geometry) has enabled us, on the one hand, to propose and demonstrate results relating to digital images. On the other hand, we developed algorithms simulated to corroborate our theoretical results. These theoretical results and their simulations concern straight line detection and fingerprint recognition.

In the case of discrete line detection, the proposed method consists in superimposing a virtual grid (rectangular, triangular, hexagonal or octagonal) on the image and calculating the dual of the cells in this virtual grid whose rate or percentage of lit pixels exceeds a given percentage ($0\% \leq \alpha \leq 100\%$).

The voting threshold of the Standard Hough transform is a criterion that determines the number of votes required for a line to be considered detected. It is an important parameter influencing the quality and reliability of line detection in an image. A low vote threshold includes noise and therefore degrades detection quality. Our approach takes advantage of this limitation to optimize the analytical recognition of straight lines. Simulations were carried out on an image of a building and on an image

of a freeway. The results demonstrated the method's ability to detect straight lines, while offering additional flexibility thanks to the possibility of adjusting the virtual grid parameters. The virtual grid approach enabled a reduction in computation time with an acceleration factor $A = 3, 26$ (on the building image) and $A = 1, 9$ (on the freeway image) with the rectangular grid, an acceleration factor $A = 2, 28$ (on the building image) and $A = 1, 31$ (on the freeway image) with the triangular grid, an acceleration factor $A = 3, 5$ (on the building image) and $A = 1, 05$ (on the freeway image) with the hexagonal grid, an acceleration factor $A = 2, 22$ (on the building image) and $A = 1, 9$ (on the freeway image) with the octagonal grid. By reducing the number of pixels to be calculated thanks to the α variable and the size of the virtual grid cells, the method offers significant computational savings. However, despite its advantages, the method proposed for discrete line detection is not without its limitations. These are : the loss of information present in the full set of lit pixels in the virtual grid cells, sensitivity to alpha rates of lit pixels and to the size of the virtual grid cells, the complexity of its implementation and context dependence.

A comparative analysis of the rectangular Hough transform and the triangular Hough transform reveals that the rectangular Hough transform offers superior processing speed to the triangular Hough transform, making it ideal for applications requiring fast detection. As far as accuracy is concerned, visual illustrations of the results confirm the relevance of the lines detected by both methods, although parameter adjustments may be necessary to optimize performance. However, the rectangular Hough

iii

transform is more flexible than the triangular Hough transform in that it offers targeted detection of vertical and horizontal lines.

In addition, to further improve our method, we combined parallel programming and virtual grids. Simulation results show that this hybrid approach reduces processing time for low α rates and small cell sizes. However, this efficiency diminishes as the value of α and/or cell size increases. It's worth pointing out that combining parallelism with triangular grids delivers better results than with rectangular, hexagonal and octagonal grids.

With regard to fingerprint recognition, our method consists, firstly, in using a virtual rectangular grid to reduce processing time, and secondly, in combining the virtual grid and parallelism to further improve processing time. Experimental results reveal that the virtual rectangular grid approach improved processing time with an acceleration factor $A = 1, 60$. The combination of parallelism and virtual grid accelerated the fingerprint identification process for large databases (at least 500 fingerprints) with a speed-up factor of $A = 2, 19$. For smaller databases (less than 500 fingerprints), this hybrid approach is less effective.

Finally, a toolbox named HoughVG with a graphical user interface, incorporating all algorithm implementations, has been developed.

Keywords : Hough transform, patterns recognition, virtual grid, parallelism, reconstruction.

iv

Résumé

L'objectif principal de cette thèse est de développer une boîte à outils pour la transformée de Hough afin de détecter efficacement des objets discrets dans les images, en intégrant des techniques d'optimisation temporelle, de programmation parallèle et d'adaptation spécifique pour différents types

d'applications. Pour ce faire, les mathématiques discrètes (topologie et géométrie) nous ont permis, d'une part de proposer et de démontrer des résultats relatifs aux images numériques. D'autre part, nous avons développé des algorithmes que nous avons simulés afin de corroborer nos résultats théoriques. Ces résultats théoriques et leurs simulations concernent la détection de lignes droites et la reconnaissance d'empreintes digitales.

Dans le cas de la détection des droites discrètes, la méthode proposée consiste à superposer une



grille virtuelle (rectangulaire, triangulaire, hexagonale ou octogonale) sur l'image et à calculer le dual

des cellules de cette grille virtuelle dont le taux ou pourcentage de pixels allumés dépasse un certain pourcentage α donné ($0\% \leq \alpha \leq 100\%$). Le seuil de vote de la transformée de Hough Standard est un critère qui détermine le nombre de votes nécessaires pour qu'une ligne soit considérée comme détectée. C'est un paramètre important qui influence la qualité et la fiabilité de la détection des lignes dans une image. Un faible seuil de vote inclut le bruit et, par conséquent, dégrade la qualité de la détection. Notre approche exploite cette limitation pour optimiser la reconnaissance analytique des lignes droites. Les simulations ont été réalisées sur des images d'un immeuble et d'une autoroute. Les résultats ont démontré la capacité de la méthode à détecter les lignes droites, tout en offrant une flexibilité supplémentaire grâce à la possibilité d'ajuster les paramètres de la grille virtuelle. L'approche des grilles virtuelles a permis la réduction du temps de calcul avec un facteur d'accélération $A = 3, 26$ (sur l'image de l'immeuble) et $A = 1, 9$ (sur l'image de l'autoroute) avec la grille rectangulaire, un facteur d'accélération $A = 2, 28$ (sur l'image de l'immeuble) et $A = 1, 31$ (sur l'image de l'autoroute) avec la grille triangulaire, un facteur d'accélération $A = 3, 5$ (sur l'image de l'immeuble) et $A = 1, 05$ (sur l'image de l'autoroute) avec la grille hexagonale, un facteur d'accélération $A = 2, 22$ (sur l'image de l'immeuble) et $A = 1, 9$ (sur l'image de l'autoroute) avec la grille octogonale. Du fait de la réduction du nombre de pixels à calculer grâce à la variable α et la taille des cellules de la grille virtuelle, la méthode offre des économies computationnelles importantes. Cependant, malgré ses avantages, la méthode proposée dans le cadre de la détection des droites discrètes n'est pas exempte de limitations. Ce sont : la perte d'informations présentes dans l'ensemble complet des pixels allumés des cellules de la grille virtuelle, la sensibilité aux taux α de pixels allumés et à la taille des cellules de la grille virtuelle, la complexité de sa mise en œuvre et la dépendance au contexte.

Une analyse comparative de la transformée de Hough rectangulaire et de la transformée de Hough triangulaire révèle que La transformée de Hough rectangulaire traite plus rapidement les données que

la version triangulaire, la rendant idéale pour les applications nécessitant une détection rapide. En ce qui concerne la précision, les illustrations visuelles des résultats confirment la pertinence des lignes détectées par les deux méthodes, bien que des ajustements de paramètres puissent être nécessaires pour optimiser les performances. Cependant, la transformée de Hough rectangulaire est plus flexible que la transformée de Hough triangulaire en ce sens qu'elle offre une capacité de détection ciblé des droites verticales et horizontales.

De plus, dans l'objectif d'améliorer d'avantage notre méthode, nous avons combiné la programmation parallèle et les grilles virtuelles. Les résultats des simulations démontrent que cette approche hybride réduit le temps de traitement pour les valeurs basses du taux α ainsi que pour les cellules

de petite taille. Cependant, cette efficacité diminue lorsque la valeur de α et/ou la taille des cellules augmente. Il convient de souligner que la combinaison du parallélisme avec les grilles triangulaires offre de meilleurs résultats qu'avec les grilles rectangulaires, hexagonales et octogonales.

En ce qui concerne la reconnaissance des empreintes digitales, notre méthode consiste, dans un premier temps, à utiliser une grille rectangulaire virtuelle pour réduire le temps de traitement, dans un deuxième temps, à combiner la grille virtuelle et le parallélisme en vue d'améliorer d'avantage le temps de traitement. Les résultats expérimentaux révèlent que l'approche basée sur la grille rectangulaire virtuelle à amélioré le temps de traitement avec un facteur d'accélération $A = 1.60$. La combinaison du parallélisme et de la grille virtuelle à accélérer le processus d'identification des empreintes digitales dans le cas des bases de données volumineuses (au moins 500 empreintes digitales) avec un facteur d'accélération $A = 2.19$. Pour les bases de données moins volumineuses (moins de 500 empreintes digitales), cette approche hybride est moins efficace.

Enfin, une boîte à outils nommée HoughVG doté d'une interface graphique utilisateur, regroupant l'ensemble des implémentations des algorithmes, a été implémentée.

Mots clés : transformée de Hough, reconnaissance de formes, grille virtuelle, parallélisme, reconstruction

vi

Table des matières

Dédicace i

Remerciements ii

Abstract iii

Résumé v

Table des matières xi

Sigles et acronymes xiii

Listes des algorithmes xv

Liste des figures xxi

Liste des tableaux xxiv

Introduction générale 1

1 Contexte de la recherche 1

2 Problématique 2

3 Objectif de la thèse 2

4 Hypothèses de recherche 3

5 Organisation du document 3

Publications personnelles 5

1 Panorama de la Transformée de Hough 6

Introduction 6

1 Notions de géométrie discrète

 7

1.1 Notion de pavage



1.2 Topologie discrète



..... 8

1.3 Modèles de discrétisation 9

1.

4 Droite analytique passant par des hexagones ou des octogones 11

2 Notions des grilles virtuelles 123 Origine et historique de la transformation de Hough



..... 13

4 Définition et étapes de la transformation de Hough Standard 14



5 Variantes de la Transformée de Hough 16

5.1 Transformée de Hough probabiliste 16

5.2 Transformée de Hough probabiliste progressive 17

5.3



dspace.univ-tlemcen.dz

[http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf](http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire%20de%20magister%20HAOUZI.pdf)

Transformée de Hough pour la détection de courbes 18

5.4 Transformée de Hough pour la détection de
cercles, d'arcs, et d'ellipses 18

5.5 Transformée de Hough pour la détection d'ellipses 18

5.6 Transformée de Hough généralisée



..... 19



5.7 Transformée de Hough adaptative 19

5.8 Transformée de Hough basée sur l'algorithme Map-Reduce 19



5.9 Transformée de Hough rapide 20

5.10 Transformée de Hough rapide intégrée (Integrated Fast Hough Transform) ... 20

6 Avancées de la Transformée de Hough 20

6.1 Optimisation de l'espace d'accumulation 21

6.2 Utilisation d'espaces d'accumulation pyramidaux 21

6.3 Adaptation à des formes complexes



..... 22

6.4 Détection multi-objet



..... 22

6.5 Intégration avec l'apprentissage automatique



..... 22

6.6 Optimisation matérielle 2

7 Applications de la transformée de Hough



..... 23

7.1 Transport 24

7.2 Images satellitaires 25

7.3 Industrie 26

7.4 Domaine de l'intelligence artificielle 27

7.5 Médecine



..... 27

7.6 Élevage 28

7.7 Réseaux de communication optique 29

7.8 Construction et bâtiment 29

7.9 Agriculture



..... 30

7.10 Sécurité 32

8 Domaines émergents



..... 33

vii
i

Conclusion 34

2 Transformée de Hough Rectangulaire 35

Introduction 35

1 Description de la méthode 36

1.1 Maillage de l'image 36

1.2 Sélection des cellules



..... 36



1.3 Parallélisation de la transformée de Hough rectangulaire	37
--	----

1.4 Reconnaissance des droites discrètes	
--	--



.	39
-----------	----



2 Complexité de la Transformée de Hough Rectangulaire	40
---	----

3 Simulations et discussions	42
--	----

3.1 Cas de l'image de l'immeuble	43
--	----

3.2 Cas de l'image de l'autoroute	46
---	----

3.3 Comparaison de la Transformée de Hough Rectangulaire et Transformée de Hough Standard	49
--	----

3.4 Comparaison de la Transformée de Hough Rectangulaire et sa version parallélisée	51
---	----

3.5 Comparaison avec d'autres approches	
---	--



.	54
-----------	----

Conclusion	55
----------------------	----

3 Transformée de Hough Triangulaire	57
-------------------------------------	----

Introduction	57
------------------------	----

1 Description de la méthode	58
---------------------------------------	----

1.1 Maillage triangulaire de l'image	58
--	----

1.2 Parallélisation de la transformée de Hough triangulaire	60
---	----

1.3 Sélection des cellules triangulaires	61
--	----

1.4 Reconnaissance des droites discrètes	
--	--



.	63
-----------	----



2 Complexité de la transformée de Hough triangulaire	65
--	----

3 Simulations et discussions	66
--	----

3.1 Cas de l'image de l'immeuble	67
--	----

3.2 Cas de l'image de l'autoroute	
-----------------------------------	--



.	69
-----------	----

4 Analyse comparative	72
---------------------------------	----

4.1 Transformée de Hough Triangulaire et Transformée de Hough Standard	73
--	----

4.2 Transformée de Hough Triangulaire et sa version parallélisée	75
--	----

4.3 Transformée de Hough Rectangulaire et Transformée de Hough Triangulaire . .	77
---	----

Conclusion	81
----------------------	----

4 Transformée de Hough Hexagonale 83

Introduction 83

1 Description de la méthode 84

1.1 Maillage Hexagonal de l'image 84

1.2 Sélection des cellules de la grille virtuelle



..... 85



1.3 Parallélisation de la transformée de Hough hexagonale 87

1.4 Reconnaissance des droites discrètes



..... 89



2 Complexité de la transformée de Hough hexagonale 90

3 Simulations et discussions



..... 93

3.1 Cas de l'image de l'immeuble 94

3.2 Cas de l'image de l'autoroute 97

3.3 Comparaison de transformée de Hough hexagonale et transformée de Hough

standard



..... 100

3.4 Comparaison de THH et sa version parallélisée 103

Conclusion 106

5 Transformée de Hough Octogonale 107

Introduction 107

1 Description de la méthode 108

1.1 Maillage de l'image 108

1.2 Sélection des cellules



..... 108



1.3 Parallélisation de la transformée de Hough octogonale 111

1.4 Reconnaissance des droites discrètes .



..... 112



2 Complexité de la transformée de Hough octogonale
.... 113

3 Simulations et discussions 115

3.1 Cas de l'image de l'immeuble 117

3.2 Cas de l'image de l'autoroute 119

3.3 Comparaison de la Transformée de Hough Octogonale et Transformée de Hough

Standard



..... 121

3.
4 Comparaison de la Transformée de Hough Octogonale et la Transformée de
Hough Octogonale Parallélisée



..... 12

4

Conclusion 126

6 Reconnaissance d'Empreintes Digitales dans une Grille Virtuelle 128

Introduction 128

1 La biométrie



..... 129

1.1 Définition



..... 129

x

1.2 Les systèmes biométriques
..... 129

2 Les empreintes digitales 130

2.1 Historique



..... 130

2.
2 Caractéristiques des empreintes digitales 130

3 Structure d'un système de reconnaissance d'empreintes digitales 131

3.1 Acquisition d'empreintes digitales



..... 133

3.2 Pré-traitement



..... 133

3.3 Extraction des minuties



..... 133

3.4 Comparaison des minuties 134

4 Méthodologie 136

5 Expérimentations et discussions



..... 13

8

5.1 Performance temporelle



..... 139

5.2 Précision



..... 141

Conclusion 142

7 Architecture de la Boîte à Outils 143

Introduction 143

1 Conception



..... 1

44

1.1 Cas d'utilisation



..... 144

1.2 Description



..... 145

1.3 Les dépendances



..... 148

2 Interface graphique 148

2.1 Scénarios d'utilisation du système de détection de droites 148

2.2 Scénarios d'utilisation du système de reconnaissance d'empreintes digitales .. 150



3 Impact 150

4 Limitations 151

5 Validation des hypothèses de recherche

 151

Conclusion 152

Conclusion générale 154

xi

Sigles et acronymes

AHT : Transformée de Hough Adaptative

BIM Building Information Modeling

BIM Mogèle d'Information du Batiment

CIHT : Combined image Hough Transform

CN Crossing Number

CPU : Processeur ou "Central Processing Unit" en anglais

FHT : Transformée de Hough Rapide

GPS Système de Positionnement Global

GUI : Interface graphique utilisateur

HoughVG Hough transform on Virtual Grids

IFHT :



Transformée de Hough Rapide Intégrée

MT-CNN :
Multi-task convolutional neural network

OCR : Reconnaissance Optique de Caractères

OHT : Transformée de Hough assistée par des points ordonnés

PHT :



Transformée de Hough Probabiliste

PPHT :
Transformée de Hough Probabiliste Progressive

SVM Machine à Vecteur de Support

TH :



transformée de Hough

THG : Transformée de Hough Generalisée

THH : Transformée de Hough Hexagonale

THHP : Transformée de Hough Hexagonale Parallélisée

xii

THO : Transformée de Hough Octogonale

THOP : Transformée de Hough Octogonale Parallélisée

THR : Transformée de Hough Rectangulaire

THRP : Transformée de Hough Rectangulaire Parallélisée

THS : Transformée de Hough Standard

THT : Transformée de Hough Triangulaire

THTP : Transformée de Hough Triangulaire Parallélisée

UAV :

Drones ou 'Unmanned Aerial Vehicles' en anglais

VANET : Réseau Ad-Hoc de véhicules

xiii

Liste des Algorithmes

1 Reconnaissance de droites discrètes par la Transformée de Hough Standard 17

2 Mise à jour de l'accumulateur



..... 18

3 Construction de la grille rectangulaire



..... 36

4 Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire 38

5 Mise à jour de l'accumulateur 38

6 Taux de pixels allumés 39

7 Mise à jour parallèle de l'accumulateur 40

8 Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire Pa-
rallélisée 41

9 Reconstruction des droites détectées 41

10 Construction de la grille triangulaire



..... 59

11 Mise à jour de l'accumulateur 60

12 Taux de pixels allumés 61

13 Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire 62

14 Mise à jour parallèle de l'accumulateur 63

15 Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire Parallélisée 64

16 Reconstruction des droites détectées 65

17 Construction de la grille hexagonale



..... 85

18 Calcul du taux de pixels allumés 87

19 Mise à jour de l'accumulateur 88

20 Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 1) 89

21 Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 2) 90

22 Sélection de tous les pixels compris entre A et B. 91

23 Mise à jour parallèle de l'accumulateur 91

xiv

24 Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée

 (partie 1) 92

25 Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée

 (partie 2) 93

26 Reconstruction des droites détectées

 93

27 Construction de la grille octogonale

 109

28 Mise à jour de l'accumulateur 111

29 Taux de pixels allumes 112

30 Reconnaissance de droites discrètes par la transformée de Hough octogonale 113

31 Mise à jour parallèle de l'accumulateur 114

32 Reconnaissance de droite discrètes par la Transformée de Hough Octogonale Parallélisée 115

33 Reconstruction des droites détectées 115

34 Transformée de Hough Généralisée 135

35 Recherche des paires de minuties correspondantes de deux empreintes digitales 136

36 Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle. 137

37 Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle combiné au parallélisme 138

xv

Table des figures

1 Organisation du document

 4

1.1 Pavage régulier [1] 8

1.2 Pavage irrégulier [1] 8

1.3 Exemple de voisinage en dimension 2 : (a) Un pixel et ses quatre 1-voisins. (b) Le même pixel et ses huit 0-voisins. [2] 9

1.4 Représentation de la droite discrète arithmétique de Reveillès D(5, 8, -1, 8), les inégalités

sont $-1 \leq 5x - 8y < 7$. (a) chaque point discret (x, y) est étiqueté par $5x - 8y$. (b) la

droite discrète obtenue [2] 10

1.5 Segments de droites discrètes : (a) Mince, (b) Nâif, (c) Standard, (d) Épais 10

1.6 (a) Un hexagone dans un pixel carré ; (b) Un octogone dans un pixel carré [3] 12

1.7 Une grille virtuelle isothétique 13

1.8 Un point A et son dual



..... 15

1.9 Une droite et son dual



..... 15

1.10 Les étapes de la technique de la transformée de Hough standard 16

1.11 Images pour les expérimentations



..... 16

1.

12 Images binaires pour les expérimentations



..... 17

2.1 Cellule rectangulaire



..... 37

2.2 Grille rectangulaire superposée à l'espace image



..... 37

2.3 Les étapes de la technique de la transformée de Hough rectangulaire 42

2.4 Courbe représentative des données du tableau 2.1 44

2.5 (a) Détection de lignes verticales de l'immeuble avec la THR (l = 168, L = 1, SR = 168,

$\alpha = 30\%$, seuil = 50), (b) détection de bords de l'autoroute avec la THR (l = 29,

L = 238, SR = 6902, $\alpha = 10\%$, seuil = 40) 45

2.6 Courbe représentative des données du tableau 2.2 46

2.7 Courbe représentative des données du tableau 2.3 47

2.8 Courbe représentative des données du tableau 2.4 48

xvi

2.9 (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de

la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble

avec l'algorithme 1 de la THS (Seuil=116) 49

2.10 (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de

la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute

avec l'algorithme 1 de la THS (Seuil=100)	50
2.11 Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=40, $\alpha = 40\%$, $L = 5$ et $l = 3$; (b) Seuil=40, $\alpha = 40\%$, $L = 5$ et $l = 3$	51
2.12 Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=51, $\alpha = 40\%$, $L = 5$ et $l = 3$; (b) Seuil=45, $\alpha = 25\%$, $L = 7$ et $l = 5$	52
2.13 Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et seuil = 70), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres (seuil = 50, $\alpha = 30\%$) des l'algorithmes 4 et 8	52
2.14 Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et seuil = 40), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres (seuil = 40, $\alpha = 30\%$) des l'algorithmes 4 et 8	53
2.15 Lignes droites détectées sur le pentagone avec l'approche de Traoré et al [4]	54
2.16 Lignes droites détectées sur l'autoroute avec l'approche de Traoré et al [4]	54
2.17 Lignes droites détectées sur l'immeuble avec l'approche de Traoré et al [4]	54
2.18 Lignes droites détectées avec notre méthode	55

3.1 Deux triangles rectangles (ABC) et (FGH) avec respectivement des droites parallèles (AB) et (DE), et (FG) et (KL)	
---	--

	59
--	----

3.2 Une grille triangulaire superposée à l'espace image	59
---	----

3.3 Sélection des pixels d'une cellule triangulaire	
---	--

	62
--	----

3.4 Les étapes de la technique de la transformée de Hough triangulaire	65
--	----

3.5 Représentation graphique des données du tableau 3.1	68
---	----

3.6 (a) THT sur l'immeuble ($h = 219$, $b = 178$, $\alpha = 48$, seuil = 61), (b) THT sur l'autoroute ($h = 92$, $b = 300$, $\alpha = 10$, seuil = 60)	69
--	----

3.7 Représentation graphique des données du tableau 3.2	70
---	----

3.8 Représentation graphique des données du tableau 3.3	71
---	----

3.9 Représentation graphique des données du tableau 3.4	72
---	----

3.10 (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)	73
---	----

3.11 (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute	
---	--

avec l'algorithme 1 de la THS (Seuil=100) 74

3.12 Détection de lignes droites sur l'image 1.11(a)de l'immeuble avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=38, $\alpha = 45\%$, $h = 5$ et $b = 3$; (b) Seuil=38, $\alpha = 40\%$, $h = 7$ et $b = 5$ 74

3.13 Détection de lignes droites sur l'image1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=57, $\alpha = 30\%$, $h = 5$ et $b = 3$; (b) Seuil=35, $\alpha = 30\%$, $h = 7$ et $b = 5$ 75

3.14 Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($h = 3$, $b = 5$ et Seuil = 60), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules triangulaires et les paramètres (Seuil = 80, $\alpha = 20\%$) des algorithmes 13 et 15 76

3.15 Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($h = 4$, $b = 2$ et Seuil = 60), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules triangulaires et les paramètres (Seuil = 60, $\alpha = 20\%$) des algorithmes 13 et 15 77

3.16 Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=35, $\alpha = 0.35$, $L = 7$ et $l = 5$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.4$, $h = 7$ et $b = 5$



. 79

3.17 Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=90, $\alpha = 0.1$, $L = 5$ et $l = 3$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.45$, $h = 5$ et $b = 3$



. 80

3.18 Détection de lignes droites sur l'image 1.11 de l'autoroute avec, (a) le THR avec les paramètres : Seuil=51, $\alpha = 0.4$, $L = 5$ et $l = 3$; (b) le THT avec les paramètres : Seuil=57, $\alpha = 0.3$, $h = 5$ et $b = 3$ 80

4.1 Un hexagone ABCDEF



. 85

4.2 Représentation des pixels d'une cellule hexagonale 85

4.3 Une grille hexagonale superposée sur un espace image 86

4.4 La prise en compte des pixels exclus d'une cellule hexagonale 86

4.5 Les étapes de la technique de la transformée de Hough hexagonale 94

4.6 Courbe représentative des données du tableau 4.1 96

xviii

4.7 Courbe représentative des données du tableau 4.2 96

4.8 Courbe représentative des données du tableau 4.3 98

4.9 Courbe représentative des données du tableau 4.4 99

4.10 (a) THH sur l'immeuble ($\gamma = 64$, $\alpha = 14$, seuil = 62), (b) THH sur l'autoroute

($\gamma = 30$, $\alpha = 10$, seuil = 60) 100

4.11 (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1

de la THS (Seuil=25) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble

avec l'algorithme 1 de la THS (Seuil=116) 101

4.12 (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de

la THS (Seuil=67) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute

avec l'algorithme 1 de la THS (Seuil=100) 101

4.13 Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la

THH avec les paramètres : (a) Seuil=50, $\alpha = 30\%$, $\gamma = 3$; (b) Seuil=25, $\alpha = 37\%$, $\gamma = 4$

4.14 Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la

THH avec les paramètres : (a) Seuil=39, $\alpha = 50\%$, SH = 6 pour $\gamma = 2$; (b) Seuil=67,

$\alpha = 25\%$, SH = 20 pour $\gamma = 3$



..... 103

4.15 Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en

fonction de α et les paramètres (seuil=60, $\gamma = 4$), (à droite) Comparaison des variations

du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=50,

$\alpha = 30\%$) des algorithmes 20 et 24 104

4.16 Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en

fonction de α et les paramètres (seuil=60, $\gamma = 3$), (à droite) Comparaison des variations

du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=60,

$\alpha = 10\%$) des algorithmes 20 et 24 105

5.1 Octogone ABCDEFGH



..... 109

5.2 Grille octogonale superposée à l'espace image 110

5.3 Mise en évidence des pixels de l'octogone ABCDEFGH 110

5.4 Les étapes de la technique de la transformée de Hough octogonale 116

5.5 Courbe représentative des données du tableau 5.1 118

5.6 Courbe représentative des données du tableau 5.2 119

5.7 Courbe représentative des données du tableau 5.3 120

5.8 Courbe représentative des données du tableau 5.4 121

5.9 (a) THO sur l'immeuble ($\gamma = 54$, $\alpha = 11\%$, seuil = 50), (b) THO sur l'autoroute

($\gamma = 33$, $\alpha = 10$, seuil = 40) 122

xix

5.10 (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1

de la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble

avec l'algorithme 1 de la THS (Seuil=116) 123

5.11 (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1

de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute

avec l'algorithme 1 de la THS (Seuil=100)	124
5.12 Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=53, $\alpha = 30\%$, $\gamma = 2$; (b) Seuil=51, $\alpha = 25\%$, $\gamma = 3$	124
5.13 Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=38, $\alpha = 30\%$ gamma = 2 ; (b) Seuil=40, $\alpha = 20\%$, gamma = 3	125
5.14 Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=100, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=70, $\alpha = 20\%$) des l'algorithmes 30 et 32	125
5.15 Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (threshold=70, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (threshold=50, $\alpha = 0$, 2) des l'algorithmes 30 et 32	126
6.1 Empreinte digitale [5]	131
6.2 Les différents types de minutie [5]	
	
.	131
6.	
3 Les trois principales classes d'empreintes, boucle (a), spire (b), arche (c) [5]	132
6.4 Architecture générale d'un système complet de reconnaissance d'empreintes	132
6.5 Capteur capacitif [6]	
	
.	1
33	
6.6 (a) Bifurcation ; (b) Terminaison	
	
.	134
6.	
7 Superposition de deux empreintes digitales	
	
.	135
6.8 Parallélisation de l'appariement des empreintes digitales. (a) Empreinte identifiée ; (b) Empreinte non identifiée	139
6.9 Appariement de deux empreintes digitales avec une grille rectangulaire	140
6.10 Comparaison du temps de traitement en fonction du nombre d'empreintes	140
6.11 Matrices de confusion	142
7.1 Interactions entre les utilisateurs et le système	144
7.2 Activités dans la boîte à outils	
	
.	145
7.3 Description de la structure interne de HoughVG	146
7.4 Tableau de bord	149

7.5 Séquence des scénarios d'utilisation du système de détection des droites 149

7.6 Séquence des scénarios d'utilisation du système de reconnaissance d'empreintes digitales 150

Liste des tableaux

2.1 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la

THR avec les paramètres fixes ($l = 3$, $L = 5$ and Seuil = 70) et α variant entre 10%

et 45% où l et L représentent respectivement la largeur et la longueur de la cellule

rectangulaire. 43

2.2 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la

THR avec les paramètres fixes ($\alpha = 30\%$ et Seuil = 50) et $1 \leq L \leq NI$ et $1 \leq l \leq Nc$

où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire. 44

2.3 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la

THR avec les paramètres fixes ($l = 3$, $L = 5$ and Seuil = 40) et α variant entre 10%

et 50% où l et L représentent respectivement la largeur et la longueur de la cellule

rectangulaire. 47

2.4 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la

THR avec les paramètres fixes ($\alpha = 30\%$ et Seuil = 40), $1 \leq L \leq NI$ et $1 \leq l \leq Nc$ où

l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire. 48

2.5 Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1=

temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite) 49

2.6 Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (temps1=

temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite) 49

2.7 Résultats de simulation avec la THR appliquée sur l'image 1.11(c) de l'immeuble (time1=

temps de détection de toutes les droites ; time2=temps de détection d'une seule droite) 51

2.8 Résultats de simulation avec la THR appliquée sur l'image 1.11(a) de l'autoroute

(time1= temps de détection de toutes les droites ; time2=temps de détection d'une

seule droite) 51

2.9 Approche proposée 53

2.10 Approche de Traoré et al [4] 55

3.1 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la

THT avec les paramètres fixes ($h = 5$, $b = 3$ et Seuil = 60) et α variant entre 10% et

55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire. 68

3.2 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la

THT avec les paramètres fixes ($\alpha = 20\%$ et Seuil = 80), $1 \leq h \leq NI$ et $1 \leq b \leq Nc$,

où h et b représentent respectivement la hauteur et la base de la cellule triangulaire. . 69

3.3 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la

THT avec les paramètres fixes ($h = 4$, $b = 2$ et Seuil = 60) et α variant entre 10% et

55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire. 70

3.4 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la

THT avec les paramètres fixes ($\alpha = 20\%$ et Seuil = 60), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$,

où h et b représentent respectivement la hauteur et la base de la cellule triangulaire. . 72

3.5 Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1=

temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite) 73

3.6 Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1=

temps de détection de toutes les droites ; time2=temps de détection d'une seule droite) 73

3.7 Résultats de simulation avec la THT sur l'image de l'immeuble (temps1= temps de

détection de toutes les droites ; temps2=temps de détection d'une seule droite) . . . 74

3.8 Résultats de simulation avec la THT sur l'image de l'autoroute (temps1= temps de

détection de toutes les droites ; temps2=temps de détection d'une seule droite) . . . 75

4.1 Resultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la

THH avec les paramètres fixes (Seuil=62 et $\alpha = 14\%$) et γ variant entre 2 et 64 . . . 95

4.2 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la

THH avec les paramètres fixes (Seuil=60 and $\gamma = 4$) et α variant entre 10% et 35% . 96

4.3 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la

THH avec les paramètres fixes (Seuil=60 and $\alpha = 10\%$) et γ variant entre 2 et 40 . . 98

4.4 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la

THH avec les paramètres fixes (Seuil=60 and $\gamma = 3$) et α variant entre 10% et 30% . 99

4.5 Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1=

temps de détection de toutes les droites ; time2=temps de détection d'une seule droite) 100

4.6 Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1=

temps de détection de toutes les droites ; time2=temps de détection d'une seule droite) 100

4.7 Résultats de simulation avec la THH appliquée sur l'image 1.11(c) de l'immeuble

(temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une

seule droite) 102

xxiii

4.8 Résultats de simulation avec la THH appliquée sur l'image 1.11(a) de l'autoroute

(temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une

seule droite) 102

5.1 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la

THO avec les paramètres fixes ($\gamma = 3$ et Seuil = 60) et α variant entre 10% et 35% . 117

5.2 Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la

THO avec les paramètres fixes (Seuil = 50 et $\alpha = 20\%$) et γ variant entre 2 et 82 . . 118

5.3 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la

THO avec les paramètres fixes ($\gamma = 3$ et Seuil = 40) et α variant entre 10% et 30% . 120

5.4 Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la

THO avec les paramètres fixes (Seuil=40 et $\alpha = 20\%$) et γ variant entre 2 et 56 . . . 121

5.5 Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1=

temps de détection de toutes les droites ; time2=temps de détection d'une seule droite) 122

5.6 Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1=

temps de détection de toutes les droites ; time2=temps de détection d'une seule droite) 123

5.7 Résultats de simulation avec la THO appliquée sur l'image 1.11(c) de l'immeuble

(temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite

détectée) 123

5.8 Résultats de simulation avec la THO appliquée sur l'image 1.11(a) de l'autoroute

(temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite

détectée) 124

6.1 Comparaison du temps de traitement en fonction du nombre d'empreintes 141

6.2 Structure générale d'une matrice de confusion 142

7.1 Facteurs d'accélération 152

7.2 Comparaison du temps de traitement de l'approche hybride et celle basée sur les grilles

virtuelles 152

xxiv

Introduction Générale

1 Contexte de la recherche

Dans le domaine de la vision par ordinateur et du traitement d'image, la détection d'objets discrets est une problématique essentielle qui trouve de nombreuses applications pratiques. Que ce soit pour la reconnaissance de formes [7], la conduite autonome [8], la détection des routes [9–12], la surveillance de sécurité, l'élevage [13], la construction des bâtiments [14], la chiologie [15], l'agriculture [16–18], la médecine [19, 20], les véhicules autonomes [21–23], la capacité à identifier et localiser des objets spécifiques dans une image de façon précise et rapide est cruciale de nos jours où la rapidité et l'efficacité sont devenues des exigences quotidiennes.

Pour illustrer ce fait, prenons l'exemple 1 de l'incident qui s'est produit en Arizona au États-Unis le 18 mars 2018 où une piétonne traversant un passage piéton de nuit a été mortellement heurté par un automobile autonome de type Volvo réaménagé. Cet exemple montre à quel point les algorithmes des systèmes des véhicules autonome doivent être optimisé pour les permettre de détecter rapidement et avec précision les obstacles pour éviter les accidents.

Parmi les différentes approches de détection des objets discrets, la transformée de Hough est largement utilisée depuis son introduction en 1962 par Paul Hough [24]. Au fil des années, les chercheurs ont développé plusieurs variations [7, 25–42] de cette technique qui a fait ses preuves en identifiant avec succès des formes géométriques, telles que des lignes [43–45], des cercles [46] ou des ellipses [47], des courbes [32, 48–51], ainsi que des formes arbitraires [52, 53] dans des images complexes, même en présence de bruit et d'occlusions partielles.

La transformation de Hough repose sur une représentation paramétrique des formes utilisant un espace image et un espace de paramètre. Cette approche permet de détecter des objets indépendamment de leur position, leur orientation et leur échelle, offrant ainsi une grande robustesse dans des scénarios variés. En effet, au lieu de rechercher directement les objets dans l'image, la transformée de Hough convertit le problème de détection en une recherche de correspondances dans un espace de paramètres.



1. <https://www.bfmtv.com/economie/entreprises/industries/accident-mortel-le-logiciel-d-uber-etait-incapable-de-reconnaitre-un-pieton-traversant-hors-des-clous-AN-201911060101>.

Cette représentation paramétrique permet de traiter les variations géométriques des objets, ce qui la rend particulièrement adaptée à la détection de formes régulières telles que les lignes, les cercles et les ellipses. Cependant, elle peut être coûteuse en termes de temps de traitement. La méthode traditionnelle de la transformée de Hough nécessite de calculer et d'accumuler les votes pour chaque pixel de l'image, ce qui peut être intensif en ressources computationnelles, en particulier pour des images de haute résolution ou dans des applications nécessitant un traitement en temps réel. Cette contrainte de temps de traitement peut limiter l'applicabilité de la transformée de Hough dans des applications pratiques où des performances rapides sont essentielles.

2 Problématique

La détection d'objets discrets dans des images reste un défi majeur en vision par ordinateur. Bien que la transformation de Hough soit une méthode bien établie pour cette tâche, son implémentation peut être complexe et exigeante en termes de ressources.

Ainsi, comment concevoir une boîte à outils basée sur la transformée de Hough, optimisée pour des performances temporelles accrues et adaptée spécifiquement à la détection d'objets discrets, tels que les lignes dans des images et les caractéristiques des empreintes digitales ? Pour résoudre cette problématique, nous devons répondre à plusieurs questions clés :

- comment réduire le temps de traitement de la transformée de Hough en utilisant des grilles virtuelles, pour améliorer la détection des objets discrets ?
- quel impact la combinaison des grilles virtuelles et du parallélisme a-t-elle sur les performances de la détection des objets discrets par la transformée de Hough ?
- quels sont les défis et les opportunités rencontrés lors de l'implémentation d'une boîte à outils intégrant les solutions proposées ?
- comment cette boîte à outils peut-elle être utilisée dans des applications pratiques de traitement d'images ?

3 Objectif de la thèse

L'objectif principal de cette thèse est de développer une boîte à outils de la transformée de Hough en vue de détecter efficacement des objets discrets dans les images, en intégrant des techniques d'optimisation temporelle, pour différents types d'applications, notamment la détection de lignes droites et la reconnaissance d'empreintes digitales.

Pour atteindre cet objectif global, nous nous fixons plusieurs sous-objectifs :

- optimiser le temps de traitement de la transformée de Hough en utilisant des grilles virtuelles : pour améliorer la détection des objets discrets dans les images, notre premier sous-objectif

2

consiste à optimiser le temps de traitement de la transformée de Hough en utilisant des grilles virtuelles. Nous cherchons à développer des méthodes efficaces pour superposer ces grilles sur les images, en explorant différents types telles que les grilles rectangulaires, triangulaires, hexagonales et octogonales et à évaluer les performances des différents types de grilles virtuelles pour la transformée de Hough, en termes de précision et de temps de calcul.

- combiner la méthode des grilles virtuelles et le parallélisme : notre deuxième sous-objectif est de

combiner l'approche des grilles virtuelles et la programmation parallèle. Nous visons à évaluer

l'impact de cette approche hybride sur le temps de traitement. Notre objectif est d'identifier les

configurations de grille pour lesquelles le parallélisme offre les meilleurs gains de performance.

- développer une boîte à outils : notre troisième sous-objectif est de développer une boîte à outils

intégrant les implémentations des algorithmes de l'approche basée sur les grilles virtuelles et de

l'approche Hybride.

4 Hypothèses de recherche

Pour orienter notre recherche, nous formulons plusieurs hypothèses clés qui guideront le développement

et l'optimisation de la transformée de Hough :

- l'intégration des grilles virtuelles dans la méthode de la transformée de Hough contribue à réduire

le temps de traitement tout en maintenant une bonne précision dans la détection d'objets discrets.

- La combinaison du parallélisme et des grilles virtuelles permet d'accélérer le temps de traitement

et de maintenir une bonne précision.

5 Organisation du document

Ce document est organisé comme suit :

- L'introduction générale présente le sujet de la recherche, explique le contexte et expose les

questions, les objectifs et les hypothèses de recherche.

- Le chapitre 1 (Panorama de la transformée de Hough) examine les bases théoriques de la TH,

son évolution historique, ainsi que les avancées récentes dans le domaine. Elle explore également

les différentes variantes de la TH et leurs applications dans divers domaines, de la vision par

ordinateur à la médecine en passant par l'industrie et l'agriculture.

- Les chapitres 2, 3, 4 et 5 explorent en détails les variantes de la transformée de Hough proposées

dans le cadre de cette thèse, notamment la transformée de Hough rectangulaire, la transformée de



Hough triangulaire, la transformée de Hough hexagonale et la transformée de Hough octogonale

respectivement.

3

- Le chapitre 6 présente une application pratique de la THG dans le domaine de la biométrie,

en particulier dans la reconnaissance d'empreintes digitales. Elle décrit une approche basée sur

l'utilisation des grilles virtuelles pour accélérer le processus de reconnaissance d'empreintes di-

giales.

- Le chapitre 7 présente l'environnement de travail et l'architecture logicielle de la boîte à outils.

Il détaille les différentes ses fonctionnalités et offre des conseils pratiques pour son utilisation.

- La conclusion générale résume les principaux résultats et contributions de la recherche, discute

des limites et propose des pistes pour des recherches futures.

Figure 1 – Organisation du document

4

Publications personnelles

- Möise Ouedraogo, Abdoulaye Sere, Cheick Amed Diloma Gabriel Traore1, HoughVG :Hough

Impacts, 2024,



<https://doi.org/10.1016/j.simpa.2024.100709>

• M $\ddot{o}$ ise Ouedraogo, Abdoulaye Sere, B.M.J. Some, C. A.G.D. Traore. Straight-Line Recognition

Using a Triangular Grid. In



thesai.org
https://thesai.org/Downloads/Volume13No10/Paper_84-Parallel_Hough_Transform_based_on_Object_Dual_and_Pymp_Library.pdf

: Arai, K. (eds) *Advances in Information and Communication*.

FICC 2022. *Lecture Notes in Networks and Systems*, vol 438. Springer, Cham. 2022 https://doi.org/10.1007/978-3-030-98012-2_45



https://doi.org/10.1007/978-3-030-98012-2_45

• M $\ddot{o}$ ise Ouedraogo, Abdoulaye Séré, Straight-Line Recognition in a Virtual Hexagonal Grid using

Hough Transform, in EAI INTERSOL, 2024

• Abdoulaye Sere, M $\ddot{o}$ ise Ouedraogo and Armand Kodjo Atiampo, Parallel Hough Transform based

on Object Dual and Pymp Library. *International Journal of Advanced Computer Science and*

Applications(IJACSA), 13(10), 2022. <http://dx.doi.org/10.14569/IJACSA.2022.0131084>

• A. Zerbo, M. Ouedraogo, A. Sere



doi.org | Parallel Fingerprint Recognition Using Generalized Hough Transform in a Virtual Grid | SpringerLink
https://doi.org/10.1007/978-3-031-47457-6_35

(2023). *Parallel Fingerprint Recognition Using Generalized*

Hough Transform in a Virtual Grid. In : Arai, K. (eds) *Proceedings of the Future Technologies*

Conference (FTC) 2023, Volume 3. FTC 2023. *Lecture Notes in Networks and Systems*, vol 815.

Springer, Cham. https://doi.org/10.1007/978-3-031-47457-6_35

• Ali Zerbo, M $\ddot{o}$ ise Ouedraogo, Abdoulaye Sere, Mamadou Diarra, Comparison of Joblib and

Pymp for Parallel Fingerprint Recognition, EAI JRI, 2024, <http://dx.doi.org/10.4108/eai.18-12-2023.2348107>

18-12-2023.2348107

5

https://doi.org/10.1007/978-3-030-98012-2_45
https://doi.org/10.1007/978-3-030-98012-2_45
<http://dx.doi.org/10.14569/IJACSA.2022.0131084>
https://doi.org/10.1007/978-3-031-47457-6_35
<http://dx.doi.org/10.4108/eai.18-12-2023.2348107>
<http://dx.doi.org/10.4108/eai.18-12-2023.2348107>

1

Ch
ap

itr
e

Panorama de la Transformée de

Hough

Introduction 6

1 Notions de géométrie discrète



..... 7

2 Notions des grilles virtuelles 12

3 Origine et historique de la transformation de Hough



..... 13

4 Définition et étapes de la transformation de Hough Standard 14



5 Variantes de la Transformée de Hough 16

6 Avancées de la Transformée de Hough



..... 20

7 Applications de la transformée de Hough



..... 23

8 Domaines émergents 33

Conclusion 34

Introduction

La Transformée de Hough, proposée initialement en 1962 par Paul Hough, est une méthode adaptée à la détection de formes géométriques dans des images numériques bruitées [24]. Au fil des années, cette technique a été largement utilisée dans le domaine de la vision par ordinateur pour détecter des objets discrets tels que des lignes droites, des cercles et des ellipses, et des formes géométriques paramétriques et arbitraires.

La Transformée de Hough a connu de nombreuses évolutions et des variantes ont été développées pour relever les défis associés à la détection d'objets dans des images réelles. Des recherches appro-

6

fondies ont été entreprises pour améliorer l'efficacité et la précision de cette méthode, tout en tenant compte des contraintes de temps de calcul et des ressources matérielles.

Plusieurs travaux scientifiques sur la transformée de Hough ont été réalisés dans plusieurs domaines clés. Parmi ces travaux, les techniques d'optimisation visant à réduire la complexité du calcul de l'accumulateur, les approches probabilistes [54] permettant une détection plus rapide et efficace d'objets discrets, ainsi que les adaptations spécifiques pour la détection de formes non-linéaires et complexes [7, 32] dans des images ont été réalisées.

Les applications de la Transformée de Hough s'étendent à divers domaines tels que le transport [55–57] pour améliorer la détection des voies par les véhicules autonomes, les images satellitaires [22, 58] améliorer la précision de la détection des satellites, l'industrie [59, 60] dans le but d'améliorer les systèmes automatique de production, la sécurité [61–64], la détection d'objets dans les images médicales [19,20,65]. Son utilité dans des environnements variés témoigne de sa polyvalence et de son efficacité dans la résolution de problèmes pratiques.

Cependant, malgré les succès obtenus, des défis subsistent et suscitent un intérêt continu pour la

recherche en Transformée de Hough. Parmi ces défis, on peut citer la prise en compte d'occlusions, la détection d'objets dans des environnements bruités, l'optimisation de l'accumulateur pour la détection rapide et précise, ainsi que l'extension de la méthode pour des tâches de détection plus complexes et spécifiques.

Dans ce chapitre, nous donnerons d'abord quelques notions de la géométrie discrète, ensuite nous explorerons l'origine et l'historique de la Transformée de Hough, ses principes fondamentaux, les avancées de la transformée de Hough. Nous mettrons également l'accent sur les différentes applications et les domaines émergents où la Transformée de Hough continue de jouer un rôle essentiel dans l'analyse d'images numériques.

1 Notions de géométrie discrète

Cette section est consacrée aux notations et définitions des éléments de la géométrie discrète.

Quelques relations topologiques (voisinage, connexité) entre les différents éléments composant les espaces discrets sont présentées.

1.1 Notion de pavage

Définition 1 (Pavé, espace discret, pavage, objet discret, partie pavable [66])

- (i) Un pavé est une cellule (composant élémentaire) d'un espace discret ;
- (ii) Un espace discret est un sous-ensemble de points isolés (points discrets) de l'espace euclidien

$\mathbb{R}^n$;

7

Figure 1.1 – Pavage régulier [1]

Figure 1.2 – Pavage irrégulier [1]

- (iii) Un pavage est une partition dénombrable de compactes (pavés) d'intérieurs non vide dans un espace euclidien ;

- (iv) Un pavage régulier (respectivement irrégulier) est une partition dénombrable de compactes (pavés) identiques (respectivement non-identiques) d'intérieurs non vide dans un espace euclidien ;

- (v) Un objet discret est un ensemble de pavés ;

- (vi) Une partie pavable de $\mathbb{R}^n$ est toute réunion d'un nombre fini de pavés fermés et bornés de $\mathbb{R}^n$.

Définition 2

- (i) Une grille régulière est un espace discret constitué de pavés identiques
- (ii) Une grille irrégulière est un espace discret constitué de pavés non-identiques

La figure 1.1 montre trois exemples de grilles régulières et la figure 1.2 montre deux exemples de grilles irrégulières.

Remarque 1

◆ Une image numérique est vue comme un espace discret où chaque point lumineux de l'image est un point discret dans l'espace discret.

◆ La discrétisation est le fait de décomposer un espace (une image) en points discrets.

1.2 Topologie discrète

Dans cette section, nous introduisons les notions de voisinage et de connexité entre points discrets afin d'établir des relations d'adjacence et d'incidence entre les pavés d'un espace discret.

Figure 1.3 – Exemple de voisinage en dimension 2 : (a) Un pixel et ses quatre 1-voisins. (b) Le même pixel et ses huit 0-voisins. [2]

Dans le cadre des espaces discrets classiques, nous utilisons la définition 3 de voisinage entre des points discrets.

Définition 3 (k-voisinage [67]) Soit $k \in \llbracket 0, n - 1 \rrbracket$. Deux points discrets $p = (p_1, \dots, p_n) \in \mathbb{Z}^n$ et

$q = (q_1, \dots, q_n) \in \mathbb{Z}^n$ sont dits k-voisins si $\forall i \in \llbracket 1, n \rrbracket, |p_i - q_i| \leq 1$ et $k \leq \sum_{i=1}^n |p_i - q_i|$.

En pratique, dans un espace discret de dimension 2, deux pixels sont 1-voisins s'ils ont une arête commun et 0-voisins s'ils ont une arête commune ou un sommet commun. La figure 1.3 donne une illustration de voisinage en dimension 2.

Remarque 2 ([4]) Deux points discrets k-voisins sont aussi k-adjacents. D'où l'équivalence des notions de k-voisinage et de k-adjacence. En dimension 2 si : p et q sont 0-voisins sans être 1-voisins alors $d(p; q) = \sqrt{2}$ unité ; p et q sont 1-voisins alors $d(p; q) = 1$ unité.

1.3 Modèles de discrétisation

La discrétisation est l'opération qui transforme un objet continu dans un espace euclidien en un objet discret décrit dans un espace discret. La discrétisation d'un objet continu se fait grâce à l'utilisation de méthodes ou modèles de discrétisation telles que les modèles Naïfs, Super-ouverture et Standard [68].

Définition 4 (Droite discrète analytique arithmétique [69]) Une droite discrète de paramètres (a, b, μ) et d'épaisseur arithmétique ω est définie comme l'ensemble des points entiers (x, y) vérifiant la double inégalité $\mu \leq ax + by < \mu + \omega$ avec $(a, b, \mu, \omega) \in \mathbb{Z}^4$ telle que a et b soient premiers entre eux. Une telle droite est notée $D(a, b, \mu, \omega)$. μ est la borne inférieure de la droite discrète et a sa pente.

Cette première définition de droite analytique discrète fut généralisée aux hyperplans analytiques discrets par Jean Pierre Reveillès [69].



Définition 5 (Hyperplans analytiques discrets [69]) En dimension n, l'hyperplan analytique dis-

cret de paramètres $A = (a_1, \dots, a_n) \in \mathbb{R}^n, \mu \in \mathbb{R}$ et $\omega \in \mathbb{R}$ est l'ensemble des points discrets

$X = (x_1, \dots, x_n) \in \mathbb{R}^n$ vérifiant $\mu \leq \sum_{i=1}^n a_i x_i < \mu + \omega$.

Figure 1.4 – Représentation de la droite discrète arithmétique de Reveillès $D(5, 8, -1, 8)$, les inégalités sont $-1 \leq 5x - 8y < 7$. (a) chaque point discret (x, y) est étiqueté par $5x - 8y$. (b) la droite discrète obtenue [2]

Figure 1.5 – Segments de droites discrètes : (a) Mince, (b) Naïf, (c) Standard, (d) Épais

Si l'inégalité de droite est large, on parle d'hyperplan analytique discret fermé.

En fonction de son épaisseur arithmétique, un hyperplan analytique peut être qualifié de diverses manières.

Définition 6 (Hyperplan analytique discret [67]) Soit H un hyperplan analytique discret de paramètres $A = (a_1, \dots, a_n) \in \mathbb{R}^n, \mu \in \mathbb{R}$ et $\omega \in \mathbb{R}$. Alors :

— H est un hyperplan Nârf si $\omega = \max_{1 \leq i \leq n} |a_i|$;

— H est un hyperplan Standard si $\omega = \sum_{i=1}^n |a_i|$;

— H est dit mince si $\omega < \max_{1 \leq i \leq n} |a_i|$;

— H est dit épais si $\omega > \sum_{i=1}^n |a_i|$.

La figure 1.5 montre quatre exemples en dimension 2 de droites discrètes mince, naïve, standard et épaisse. Dans ce document, il est question de la détection des droites discrètes standard.

10

1.4 Droite analytique passant par des hexagones ou des octogones

Les pixels sont généralement représentés sous forme carré. Séré et al [3] ont proposé de nouvelles définitions de droites analytiques à travers des hexagones ou des octogones virtuels. La figure 1.6 présent deux pixels contenant un hexagone et un octogonale respectivement.

Définition 7 (Droite analytique hexagonale croissante [3]) Soit A(xa, ya) et B(xb, yb) le centre de deux hexagones respectivement. nous fixons $U = (yb - ya)$, $V = (xa - xb)$ et $T = -xa(yb - ya) - ya(xa - xb)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite hexagonale croissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

$$\begin{aligned} &\bullet \text{ si } 0 \leq \alpha \leq \pi \\ &3 \text{ alors } -(d \cdot |U| \\ &4 + d \cdot |V| \\ &\sqrt{3} \\ &4) \leq U \cdot x + V \cdot y + T \leq (d \cdot |U| \\ &4 + d \cdot |V| \\ &\sqrt{3} \\ &4) ; \end{aligned}$$

$$\begin{aligned} &\bullet \text{ si } \pi \\ &3 \leq \alpha \leq \pi \\ &2 \text{ alors } -d \cdot |U| \\ &4 \leq U \cdot x + V \cdot y + T \leq d \cdot |U| \\ &4 \end{aligned}$$

Définition 8 (Droite analytique hexagonale décroissante [3]) Soit A(xa, ya) et B(xb, yb) le centre de deux hexagones respectivement. nous fixons $U = (yb - ya)$, $V = (xa - xb)$ et $T = -xa(yb - ya) - ya(xa - xb)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite hexagonale croissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

$$\begin{aligned} &\bullet \text{ si } 2\pi \\ &3 \leq \alpha \leq \pi \text{ alors } -(d \cdot |U| \\ &4 + d \cdot |V| \\ &\sqrt{3} \\ &4) \leq U \cdot x + V \cdot y + T \leq (d \cdot |U| \\ &4 + d \cdot |V| \\ &\sqrt{3} \\ &4) ; \end{aligned}$$

$$\begin{aligned} &\bullet \text{ si } \pi \\ &2 \leq \alpha \leq 2\pi \end{aligned}$$

$$3 \text{ alors } -d. |U|$$

$$4 \leq U.x + V.y + T \leq d. |U|$$

4

Définition 9 (Droite analytique octogonale croissante [3]) Soit A(xa, ya) et B(xb, yb) le centre

de deux hexagones respectivement. nous fixons $U = (yb - ya)$, $V = (xa - xb)$ et $T = -xa(yb - ya) -$

$ya(xa - xb)$. Soit α un angle créé par la droite (AB) avec la droite horizontale. Une droite hexagonale

croissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

$$\bullet \text{ si } \pi$$

$$2 \geq \alpha \geq \pi$$

$$4 \text{ alors } -(|U|$$

$$2 +$$

$$\sqrt$$

$$2 |V|$$

$$2(2+$$

$$\sqrt$$

$$2)) \leq U.x + V.y + T \leq |U|$$

$$2 +$$

$$\sqrt$$

$$2 |V|$$

$$2(2+$$

$$\sqrt$$

2) ;

$$\bullet \text{ si } 0 \leq \alpha \leq \pi$$

$$4 \text{ alors } -(|V|$$

$$2 +$$

$$\sqrt$$

$$2 |U|$$

$$2(2+$$

$$\sqrt$$

$$2)) \leq U.x + V.y + T \leq |V|$$

$$2 +$$

$$\sqrt$$

$$2 |U|$$

$$2(2+$$

$$\sqrt$$

$$2)$$

Définition 10 (Droite analytique octogonale décroissante [3]) Soit A(xa, ya) et B(xb, yb) le centre

de deux hexagones respectivement. nous fixons $U = (yb - ya)$, $V = (xa - xb)$ et $T = -xa(yb - ya) -$

$ya(xa - xb)$. Soit $\hat{\alpha}$ un angle créé par la droite (AB) avec la droite horizontale. Une droite hexagonale

croissante est un ensemble de points $M(x, y) \in \mathbb{Z}^2$ qui vérifie :

$$\bullet \text{ si } 3\pi$$

$$4 \geq \alpha \geq \pi$$

$$2 \text{ alors } -(|U|$$

$$2 +$$

$$\sqrt$$

$$2 |V|$$

$$2(2+$$

$$\sqrt$$

$$2)) \leq U.x + V.y + T \leq |U|$$

$$2 +$$

$$\sqrt$$

$$2 |V|$$

$$2(2+$$

$$\sqrt$$

$$2);$$

$$\bullet \text{ si } 3\pi$$

$$4 \leq \alpha \leq \pi \text{ alors } -(|V|$$

$$2 +$$

$$\sqrt$$

$$2|U|$$

$$2(2+$$

$$\sqrt$$

$$2)) \leq U.x + V.y + T \leq |V|$$

$$2 +$$

$$\sqrt$$

$$2|U|$$

$$2(2+$$

$$\sqrt$$

$$2)$$

$$11$$

Figure 1.6 – (a) Un hexagone dans un pixel carré ; (b) Un octogone dans un pixel carré [3]

2 Notions des grilles virtuelles

Une grille virtuelle est une structure imaginaire composée de lignes horizontales, verticales et/ou oblique qui divisent une image en cellules.

Il existe deux types de grille virtuelle : les grilles régulières et les grilles irrégulières. La grille régulière est la plus simple, avec des cellules de même type et de taille égale réparties de manière régulière sur toute l'image. Par exemple, pour une grille dont les cellules sont des rectangles, on parle de grille rectangulaire, pour une grille dont les cellules sont des triangles, on parle de grille triangulaire, pour une grille dont les cellules sont des hexagones, on parle de grille hexagonale et pour une grille dont les cellules sont des octogones, on parle de grille octogonale. Nous pouvons voir une illustration de ces grilles virtuelles sur les figures 2.2, 3.2, 4.3 et 5.2 respectivement. Quant à la grille irrégulière, elle est composée de formes différentes de cellules réparties sur l'image.

Antoine Vacavant [70], dans ses travaux de thèse en 2008, a présenté les grilles irrégulières isothétiques. Une grille irrégulière isothétique est une grille composée de cellules de formes et de tailles variées, mais dont les côtés restent parallèles aux axes horizontalement et verticalement comme illustré sur la figure 1.7. Il a fait mention également des grilles triangulaires, hexagonales abordées par Herbert Freeman [71] en 1979 et E. Luczak [72] en 1976 respectivement.

D'après B. Kumar [73], Les structures de pixels hexagonaux offrent plusieurs avantages par rapport aux pixels carrés, comme une symétrie circulaire plus élevée, un ajustement uniforme des pixels, une complexité de traitement réduite, une meilleure qualité d'image, une connectivité cohérente et une isotropie plus élevée.

En 2019, Traoré et al [4] ont proposé une nouvelle approche de la transformée de Hough standard qui consiste à appliquer la TH sur une grille rectangulaire virtuelle superposée sur l'image dans le but de réduire son temps de traitement.

L'application de la TH, implique généralement des calculs intensifs. En considérant l'ensemble des pixels de l'image, ces calculs peuvent devenir coûteux en termes de temps de traitement. La grille virtuelle permet de filtrer les cellules de la grille qui ne contiennent pas un nombre suffisant de pixels allumés pour contribuer de manière significative à la détection de motifs. Cette approche permet de réduire le nombre total de calculs. Au lieu de traiter l'ensemble des pixels de l'image,

Figure 1.7 – Une grille virtuelle isothétique

nous ne considérons que les cellules dont le taux de pixel allumé dépasse un pourcentage α donné, économisant ainsi des ressources computationnelles. Comment reconnaître des objets discrets, notamment des droites discrètes ?

3 Origine et historique de la transformation de Hough

La vision par ordinateur a connu des avancées spectaculaires au cours des dernières décennies, permettant aux machines de comprendre et d'analyser des images numériques de manière semblable à la perception humaine. Au cœur de cette discipline, la détection d'objets discrets occupe une place prépondérante, permettant l'identification et la localisation précise d'entités spécifiques dans une image. Pour réaliser cette tâche complexe, la transformation de Hough émerge comme une méthode classique et puissante qui s'appuie sur une approche paramétrique, offrant une grande robustesse face aux variations géométriques des objets dans l'image.

La Transformation de Hough est une méthode de traitement d'image novatrice qui a été proposée pour la première fois par Paul Hough dans un article [24] publié en 1962 intitulé "Méthode et moyens pour reconnaître des motifs complexes". Cependant, la méthode n'a pas été largement reconnue à l'époque et est restée relativement méconnue pendant plusieurs années.

C'est en 1972 que la Transformation de Hough a été réintroduite et popularisée par Richard O. Duda et Peter E. Hart dans leur article intitulé "Use of the Hough Transformation to Detect Lines and Curves in Pictures" (Utilisation de la Transformation de Hough pour détecter des lignes et des courbes dans des images) [32]. Dans cet article, Duda et Hart ont démontré comment la méthode de Hough pouvait être utilisée pour détecter des lignes droites dans des images en représentant ces lignes dans un espace de paramètres appelé espace de Hough.

L'idée centrale de la Transformation de Hough réside dans sa capacité à représenter des formes géométriques, telles que des lignes droites, des cercles, des ellipses, et d'autres objets, sous forme de points dans un espace de paramètres. Cette représentation paramétrique permet de détecter ces formes indépendamment de leur position, leur orientation et leur échelle dans l'image, ce qui la rend

13

très robuste aux variations géométriques.

Initialement utilisée pour détecter des lignes dans des images, la Transformation de Hough a été étendue pour détecter d'autres formes géométriques en adaptant la représentation paramétrique appropriée pour chaque forme. Ainsi, des variantes de la méthode de Hough ont été développées pour détecter des cercles, des ellipses et d'autres objets discrets, élargissant considérablement son champ d'application.

Depuis sa réintroduction par Duda et Hart, la Transformation de Hough est devenue une méthode fondamentale de traitement d'images et un outil puissant pour la détection de formes géométriques et d'objets discrets dans de nombreux domaines de la vision par ordinateur. Elle a été largement utilisée dans des applications pratiques telles que la conduite autonome, la reconnaissance de formes, la robotique, la vision industrielle, la détection d'objets médicaux, et bien d'autres.

Au fil des années, de nombreuses améliorations et optimisations, notamment la transformée de Hough combinée à l'intelligence artificielle [44, 74–81], ont été apportées à la méthode de Hough, rendant son utilisation plus efficace et plus précise. Aujourd'hui, la Transformation de Hough continue

d'être un sujet de recherche actif, avec de nouvelles avancées et applications émergentes dans le domaine de la vision par ordinateur.

4 Définition et étapes de la transformation de Hough Standard

Définition 11 (Transformée de Hough standard) Soient $I_2 \subset \mathbb{R}^2$ l'espace image c'est-à-dire une image de largeur l , de hauteur h et (x, y) un point de I_2 de paramètres (θ, ρ) dans l'espace des paramètres P_2 . La transformée de Hough standard de (x, y) est la courbe sinusoidale $S(x, y)$ dans P_2

définie par $S(x, y) = \{(\theta, \rho) \in [0, \pi] \times [-\sqrt{l^2 + h^2}, \sqrt{l^2 + h^2}] \mid$

$\sqrt{l^2 + h^2},$

$\sqrt{l^2 + h^2}/\rho = x \cos \theta + y \sin \theta\}$.

Par exemple pour $A(1; 1) \in I_2$, la transformée de Hough standard du point A est $S(1, 1) = \{(\theta, \rho) \in$

$[0, \pi] \times [-\sqrt{l^2 + h^2},$

$\sqrt{l^2 + h^2},$

$\sqrt{l^2 + h^2}/\rho = \cos \theta + \sin \theta\}$. Ce resultat est illustré dans la figure 1.8

Propriete 1 (Transformée de Hough standard [82])

- Un point dans l'espace d'image I_2 , correspond à une sinusöide dans l'espace des paramètres P_2 .

Cela est illustré par la Figure 1.8 en (a) et (b).

- Des points appartenant à la même sinusöide dans l'espace des paramètres P_2 correspondent à des droites qui se coupent en un même point dans l'espace image I_2

- Un point de l'espace des paramètres P_2 correspond à une droite dans l'espace image I_2 . La figure 1.9 présente un exemple.

- Des points appartenant à une même droite (D) dans l'espace d'observation I_2 correspondent à des sinusöides qui se coupent en un même point dans l'espace des paramètres P_2 . La figure 1.9 présente également cette relation.

14

Figure 1.8 – Un point A et son dual

Figure 1.9 – Une droite et son dual

Les étapes de l'application de la transformée de Hough standard sur une image sont :

- pré-traitement de l'image : Il s'agit d'appliquer un filtre sur l'image pour mettre en évidence les contours des objets discrets. La figure 1.11 illustre respectivement une image d'une route, un pentagone et une image d'un bâtiment. La figure 1.12 est le résultat de l'application du filtre de Canny sur les images de la figure 1.11.

- mise à jours de l'accumulateur : la mise à jours de l'accumulateur consiste à calculer le dual des points de l'image et à mettre à jours l'espace des paramètres par incrémentations des valeurs des cellules de l'accumulateur comme illustre l'algorithme 1 et 2. Le dual de chaque pixel représente un vote dans l'accumulateur.

- détection des maxima dans l'accumulateur : une fois que tous les votes ont été accumulés, on recherche les maxima dans l'accumulateur en utilisant un seuil de vote. les maxima sont les points (θ_{max}, p_{max}) de l'accumulateur ayant un nombre de votes supérieur au seuil de vote donné.

- reconstruction : Les paramètres (θ_{max}, p_{max}) de l'espace des paramètres sont associés à la droite

$\Delta : y = -\cos \theta$

$\sin\theta$
 $x + \rho$

$\sin\theta$
dans l'espace image.

15

Figure 1.10 – Les étapes de la technique de la transformée de Hough standard

Figure 1.11 – Images pour les expérimentations

5 Variantes de la Transformée de Hough

Les variantes de la Transformée de Hough sont des extensions ou des adaptations de l'algorithme de base pour répondre à des problèmes spécifiques de détection d'objets géométriques dans les images numériques. Voici quelques-unes des principales variantes de la Transformée de Hough :

5.1 Transformée de Hough probabiliste

La Transformée de Hough probabiliste (PHT) est une variante de la Transformée de Hough classique utilisée pour détecter des formes géométriques, (telles que des lignes, des cercles, des ellipses) dans des images. Elle a été développée par Kiryati et al [54] pour améliorer la robustesse et l'efficacité de la Transformée de Hough classique, en particulier dans des conditions où l'exactitude des détections

16

Figure 1.12 – Images binaires pour les expérimentations

Algorithme 1 : Reconnaissance de droites discrètes par la Transformée de Hough Standard
Fonction Standard(Imge,threshold) :tableau

Pre-condition : $0 < \text{threshold}$
Variables : accum ligne, accum colon, irho, NI, Nc : entier ; accum : matrice

accumulateur ;
dtheta, drho, rho, theta : réels;
Debut

% dimension de l'image
NI ← la hauteur de l'image ; Nc ← la largeur de l'image;
% dimensions de la matrice 'accum'
accum ligne ← 180 ; accum column ← E[

√
(Nc2 + NI2)];

dtheta = $\pi/180$;
pour $0 \leq y \leq Nc$ faire

pour $0 \leq x \leq NI$ faire
Acc update();

fin
fin

fin
Rechercher les maxima dans l'accumulateur et placer les couples (p, θ) dans une liste

'lignes';
Retourner la liste 'lignes';

fin

est compromise par des données bruitées ou des objets partiels. Contrairement à la Transformée de Hough classique, qui accumule des votes pour les points d'intersection dans un espace discret, la Transformée de Hough probabiliste attribue des probabilités aux paramètres des formes candidates, permettant ainsi de gérer de manière plus flexible les incertitudes liées à la détection.

5.2 Transformée de Hough probabiliste progressive

La transformée de Hough probabiliste progressive (PPHT) présentée par Matas et al [83], offre des

avantages significatifs par rapport à la transformée de Hough probabiliste [54]. Contrairement à PHT, qui effectue la transformation de Hough standard sur une fraction pré-sélectionnée de points d'entrée, PPHT réalise la détection de ligne avec un calcul minimal en exploitant les différentes fractions de votes nécessaires pour une détection fiable des lignes avec un nombre variable de points de support. Cela élimine la nécessité de spécifier la fraction de points utilisée pour voter ad hoc ou sur la base de connaissances préalables. Au lieu de cela, l'algorithme s'adapte à la complexité inhérente des données. PPHT est particulièrement bien adapté aux applications en temps réel avec un temps de traitement fixe, car le vote et la détection de ligne se produisent simultanément, les fonctionnalités les plus

17

Algorithme 2 : Mise à jour de l'accumulateur
 Procedure Acc update() : vide

```
% Mise à jour de l'accumulateur
if img[z, t] = 0 then

for itheta dans range(accum row) do
theta ← itheta × dtheta;
rho ← t × cos(theta) + z × sin(theta);
irho ← int(rho);
if irho > 0 et irho < accum column then

accum[itheta][irho] ← accum[itheta][irho] + 1
end

end
end

fin
```

importantes étant identifiées en premier. Les résultats expérimentaux démontrent que PPHT surpasse la transformée de Hough standard dans de nombreux scénarios.

5.3 Transformée de Hough pour la détection de courbes

Dans les années 1972, soit 10 ans après l'introduction de la transformée de Hough classique [24], Duda et Hart [32] ont introduit une nouvelle approche pour détecter des courbes plus généraux. Cette approche est basée sur la transformée de Hough classique et met en avant le fait que l'utilisation de paramètres angle-rayon, par opposition aux paramètres pente-interception, simplifie davantage les calculs nécessaires. En outre, des interprétations alternatives sont proposées, offrant une perspective supplémentaire sur les raisons sous-jacentes à l'efficacité de cette méthode.

5.4 Transformée de Hough pour la détection de cercles, d'arcs, et d'ellipses

Cette variante est spécifiquement conçue pour la détection de cercles dans les images. En 1975 Kimme et al dans leur article [84] décrivent une procédure qui se concentre sur l'identification efficace de cercles approximatifs et d'arcs circulaires dans des images numérisées avec des bords améliorés. La procédure est une extension et une amélioration d'un concept de détection de courbes (arcs, cercles et ellipses en particulier) introduit par Duda et Hart [32]. Cela implique que la nouvelle procédure s'inspire de leur travail et développe leurs idées pour améliorer la précision, l'efficacité ou l'applicabilité de la détection de cercles. Princen et al dans leur article [85] ont étudié un certain nombre de méthodes de détection de cercles basées sur des variations de la transformée de Hough. Coelho et al [86] quant à eux utilisent une approche basée sur MapReduce pour la détection des cercles.

5.5 Transformée de Hough pour la détection d'ellipses

La Transformée de Hough est une technique précieuse pour détecter des lignes droites dans des images, en particulier lorsque les contours sont accentués. Cependant, lorsqu'il s'agit d'étendre cette

18

méthode pour détecter des ellipses, un problème se pose : le temps de calcul requis peut devenir excessivement long. Dans ce contexte, Tsuji et Matsumoto dans leurs travaux [87] en 1978 ont proposés une approche modifiée visant à surmonter cette limitation. L'approche fonctionne de manière itérative. Elle recherche des groupes d'ellipses presque complètes en explorant deux espaces de paramètres différents. Cette approche séparée dans les espaces de paramètres permet de réduire la complexité du calcul et d'accélérer le processus de détection. Suite à ces travaux, Davies [88] a montré par une autre approche qu'un seul plan dans l'espace des paramètres peut être utilisé, avec un gain d'efficacité conséquent. La méthode est particulièrement adaptée aux ellipses de faible excentricité.

5.6 Transformée de Hough généralisée

Ballard et al [7] ont proposé une généralisation de la Transformée de Hough classique, permettant de détecter des formes géométriques arbitraires, telles que des polygones, des courbes complexes ou d'autres formes définies par un ensemble de points. Elle utilise un modèle de référence de l'objet à détecter et recherche les occurrences de ce modèle dans l'image en utilisant la Transformée de Hough. La Transformée de Hough classique est limitée à la détection de lignes droites et de cercles. En généralisant la Transformée de Hough pour détecter des formes arbitraires, cette approche constitue



Document d'un autre utilisateur

Le document provient d'un autre groupe

une avancée significative dans le domaine de la détection d'objets et de formes dans des images. La

méthode proposée a ouvert de nouvelles possibilités pour la détection de formes complexes, élargissant ainsi l'applicabilité de la Transformée de Hough dans diverses applications dans le domaine du traitement d'images.

5.7 Transformée de Hough adaptative

Dans les années 1987 les chercheurs, Illingworth et Kittler [89] ont présenté une approche novatrice appelée "Transformée de Hough Adaptative (AHT)" pour la détection efficace de formes bidimensionnelles. Cette méthode étend la Transformée de Hough en utilisant un réseau d'accumulateurs plus restreint tout en intégrant une stratégie d'accumulation et de recherche itérative. Cette approche permet d'identifier de manière précise les pics significatifs dans les espaces de paramètres de la Transformée de Hough. L'AHT se révèle nettement supérieure à l'implémentation standard de la TH en termes de stockage et de calculs requis.

5.8 Transformée de Hough basée sur l'algorithme Map-Reduce

La Transformée de Hough basée sur l'algorithme Map-Reduce est une version parallélisée de la transformée de Hough traditionnelle, conçue pour tirer parti des capacités de calcul distribué offertes par le modèle de programmation Map-Reduce. Abdoulaye SERE et al [90] ont proposés une composition de la transformée de Hough et de Map-Reduce dans l'environnements informatiques distribués

19

Hadoop. Ces travaux se sont focalisés sur la détection des droites, mais ils ont été étendus par Mateus Coelho et al [86] pour proposer la détection de cercle avec Map-Reduce. Ces variantes de la Transformée de Hough offrent des solutions adaptées à différents types de formes géométriques et aux exigences spécifiques des applications.

5.9 Transformée de Hough rapide

La transformée de Hough rapide est une approche, intitulée "Fast Hough Transform" (FHT),

introduite en 1986 par Li et al [91] qui vise à améliorer l'efficacité du processus de détection en

utilisant une méthode hiérarchique pour réduire les calculs nécessaires.

Les auteurs considèrent que les caractéristiques de l'image "votent" pour des ensembles de points

qui sont associés à des hyperplans dans un espace de paramètres. Plutôt que de traiter l'ensemble

de l'espace des paramètres, l'algorithme divise cet espace en hypercubes de résolutions variables. Ils

effectuent ensuite la transformée de Hough uniquement sur les hypercubes où les votes dépassent un certain seuil.

L'approche hiérarchique permet de réduire les coûts de calcul et de stockage, elle ne traite que les

parties de l'espace des paramètres susceptibles de contenir des informations importantes.

5.10 Transformée de Hough rapide intégrée (Integrated Fast Hough Transform)

La détection de lignes, de plans et d'hyperplans au sein de données multidimensionnelles trouve

de nombreuses applications dans les domaines de la vision artificielle et de l'intelligence artificielle. La

contribution de Li et al [92] consiste en la proposition d'une approche novatrice nommée Transformée

de Hough Rapide Intégrée ou "Integrated Fast Hough Transform" (IFHT).

L'espace des paramètres dans l'IFHT est représenté par un unique arbre de type k , ce qui permet

d'adopter une stratégie de stockage hiérarchique. L'IFHT modifie fondamentalement la manière dont

les données sont ajustées aux moindres carrés, par rapport à la méthode de Transformée de Hough

Rapide (FHT) développée par Li et al. Dans l'IFHT, cet ajustement se réalise en prenant en compte

les erreurs d'observation sur toutes les dimensions, ce qui le rend plus adapté et résistant au bruit des données.

Dans des tests effectués sur des données simulées avec divers niveaux de bruit et de paramètres,

l'IFHT surpasse significativement la méthode de Transformée de Hough Rapide de Li [91] en termes

de robustesse et de précision.

6 Avancées de la Transformée de Hough

Les avancées de la Transformée de Hough ont été nombreuses au fil des années, et elles ont permis

d'améliorer son efficacité, sa précision et son adaptabilité à différents domaines d'application. Voici

20

quelques-unes des principales avancées de la Transformée de Hough

6.1 Optimisation de l'espace d'accumulation

L'optimisation de l'espace d'accumulation est une technique importante pour accélérer la Trans-

formée de Hough et réduire la complexité computationnelle en réduisant la taille de l'espace d'accu-

mulation tout en maintenant une précision raisonnable pour la détection d'objets discrets. L'espace

d'accumulation est une matrice multidimensionnelle utilisée pour enregistrer les votes lors de la re-

cherche de lignes ou de formes dans l'image. Cependant, cette matrice peut devenir très grande, ce

qui rend le processus de vote coûteux en termes de mémoire et de temps de calcul.

La Transformée de Hough probabiliste est une variante de la Transformée de Hough classique

introduite par Kiryati et al [54]. Les auteurs ont proposé une variation de la transformée de Hough

classique qui vise à résoudre certains des problèmes inhérents à la détection de formes complexes dans

des images bruitées. La transformée de Hough probabiliste opère en échantillonnant aléatoirement un

sous-ensemble des pixels de l'image et en utilisant ces échantillons pour déterminer les paramètres de

la forme recherchée. Cette approche probabiliste permet de réduire les temps de calcul et de mieux

gérer les scénarios où les objets peuvent être partiellement cachés ou présenter des variations d'échelle.

L'optimisation de l'espace d'accumulation dans le contexte de la Transformée de Hough probabiliste

repose sur l'idée que, plutôt que de stocker l'accumulation pour chaque point de l'image, on peut

estimer les paramètres de transformation en utilisant uniquement un échantillon aléatoire de points.

Ce processus probabiliste permet de maintenir une représentation compacte de l'espace d'accumulation

tout en réduisant la complexité temporelle de l'algorithme.

6.2 Utilisation d'espaces d'accumulation pyramidaux

Pour des échelles différentes ou des niveaux de résolution d'image, on peut créer des espaces d'ac-

cumulation pyramidaux. Cela permet de détecter des formes à différentes échelles, tout en réduisant la

complexité computationnelle. En 2004, Lorenz et Hardeberg dans [93] ont présentés une généralisation

multiscale de la Transformée de Hough pour l'analyse d'images. La méthode proposée vise à améliorer

la robustesse de la Transformée de Hough aux variations d'échelle dans les images, ce qui est une

limitation courante de la version traditionnelle de la Transformée de Hough.

La généralisation multiscale repose sur la construction d'une pyramide d'échelles dans l'espace

d'accumulation. Au lieu d'utiliser une échelle fixe pour détecter les objets discrets, les auteurs utilisent

plusieurs échelles de manière pyramidale pour représenter les objets à différentes résolutions. Cela

permet de détecter des objets de différentes tailles dans l'image.

Le processus de détection commence par la création de plusieurs niveaux de résolution dans l'espace

d'accumulation. Ensuite, des seuils adaptatifs sont appliqués pour détecter les objets candidats à

21

chaque niveau. Les détections à différents niveaux sont ensuite combinées pour obtenir les détections

finales.

Les résultats expérimentaux démontrent l'efficacité de la méthode proposée. La généralisation

multiscale améliore la détection d'objets discrets dans des images contenant des variations d'échelle,

et elle est robuste face au bruit et aux déformations.

6.3 Adaptation à des formes complexes

L'adaptation de la Transformée de Hough à des formes complexes est un domaine de recherche

important en vision par ordinateur. La Transformée de Hough classique est conçue pour détecter des

formes géométriques simples telles que des lignes droites, des cercles, et des ellipses. Cependant, de

nombreuses applications réelles impliquent la détection de formes plus complexes et non linéaires,

telles que des objets naturels, des structures anatomiques, ou des motifs abstraits. Pour adapter la

Transformée de Hough à des formes complexes, une approche a été proposée. Il s'agit de l'extension

de l'espace des paramètres abordée par Ballard et Brown dans [7], où ils proposent la THG, qui étend

l'espace des paramètres pour inclure des descripteurs plus riches et contextuels. Au lieu de se limiter à

des paramètres géométriques, l'espace des paramètres est étendu pour inclure des descripteurs locaux

ou globaux qui décrivent des caractéristiques spécifiques des formes recherchées.

6.4 Détection multi-objet

Plutôt que de se limiter à détecter un seul objet à la fois, cette approche permet la détection

simultanée de plusieurs occurrences d'objets dans une seule passe de la Transformée de Hough. Cette

approche offre une manière efficace de détecter des objets multiples dans des images complexes, ce

qui est crucial dans de nombreuses applications de vision par ordinateur telles que la robotique, la

reconnaissance de formes, la surveillance et l'analyse d'images médicales. Eric GABRIEL et al [93]

a proposé une approche innovante pour la détection automatique de piétons et de voitures dans des

images numériques basée sur la transformée de Hough généralisée discriminante et les réseaux de neurones à convolution profonde.

6.5 Intégration avec l'apprentissage automatique

L'utilisation de techniques d'optimisation avancées et d'apprentissage automatique (comme les réseaux de neurones) permet d'améliorer la précision de la détection en optimisant les paramètres de l'algorithme en fonction des données spécifiques. Des chercheurs ont proposés des approches pour intégrer la transformée de Hough avec l'apprentissage automatique.

Gall et al [94] ont proposé une approche novatrice appelée "Hough Forest" pour la détection d'objets, le suivi et la reconnaissance d'actions dans des vidéos. L'Hough Forest est une méthode d'apprentissage automatique basée sur les arbres de décision, inspirée par la Transformée de Hough,

qui vise à améliorer la robustesse et la précision des tâches de détection et de suivi d'objets dans des scènes complexes.

Qi Han et al [44] ont présenté également une méthode qui intègre la Transformée de Hough classique avec des techniques d'apprentissage profond (deep learning) pour la détection sémantique des lignes dans les images. En effet, le problème de détection des lignes sémantiques dans le domaine spatial est transformé en repérage de points individuels dans le domaine paramétrique, rendant les étapes de post-traitement, c'est-à-dire la suppression non maximale, plus efficaces.

Divas Karimanzira et Helge Renkewitz [95] utilisent la transformée de Hough généralisée combiné avec l'apprentissage automatique (machine learning) pour la détection et la localisation de stations d'accueil sous-marines dans des images acoustiques.

6.6 Optimisation matérielle

L'optimisation matérielle dans le cadre de la Transformée de Hough vise à améliorer l'efficacité et la rapidité de cette transformation en utilisant des techniques spécifiques au matériel informatique sur lequel elle est implémentée. La Transformée de Hough est une opération coûteuse en termes de temps de calcul, surtout pour les images de grande taille, en raison de la complexité de son algorithme.

L'optimisation matérielle cherche à exploiter les caractéristiques du matériel cible pour accélérer le processus de transformation de Hough et réduire la consommation de ressources. Les architectures de processeurs modernes offrent des possibilités de parallélisation, notamment en utilisant des unités de calcul vectorielles et des unités de calcul graphiques (GPU). Ce qui a favorisé Yam-Uicab et al [96] de présenter des stratégies d'optimisation pour accélérer la Transformée de Hough appliquée à la détection de caractéristiques linéaires en utilisant le GPU. Les auteurs exploitent les capacités de parallélisation offertes par les GPU pour accélérer le processus de transformation de Hough.

Ces avancées ont rendu la Transformée de Hough plus performante et polyvalente dans la détection d'objets discrets dans les images numériques. Elles ont ouvert de nouvelles perspectives pour l'utilisation de cette méthode dans divers domaines, allant de la robotique à la vision industrielle, en passant par la reconnaissance de formes et la détection d'objets médicaux. Les progrès continus dans ce domaine promettent de rendre la Transformée de Hough encore plus efficace et pertinente pour des applications futures en vision par ordinateur.

7 Applications de la transformée de Hough

Cette section explore les diverses applications de la transformée de Hough dans différents domaines, mettant en évidence son importance et son efficacité dans des contextes tels que le transport,

la surveillance environnementale, l'industrie, la médecine, l'élevage et l'agriculture. Ces applications démontrent comment la transformée de Hough contribue de manière significative à la résolution de

23

problèmes pratiques.

7.1 Transport

Une méthode pour améliorer la précision de la détection des voies dans le contexte des véhicules autonomes a été présentée en 2023 par Javeed et al [55]. Au cours des dernières décennies, les véhicules autonomes ont gagné en popularité en raison de leur potentiel pour réduire les accidents de la route causés par des erreurs humaines. Dans ce contexte, il devient crucial d'intégrer des systèmes intelligents de détection des voies dans les véhicules autonomes. La détection précise des voies est une fonctionnalité essentielle pour les systèmes avancés de conduite. Cette étude propose une approche améliorée pour détecter les voies en utilisant une version étendue de l'algorithme de détection des bords de Canny, soutenue par la transformée de Hough rapide. Pour améliorer la qualité des données en entrée, une étape de lissage de l'image avec un filtre de flou gaussien est effectuée, réduisant ainsi le bruit indésirable. Cette méthodologie a été testée sur différentes séquences d'images. Ensuite, un ajustement des moindres carrés a été appliqué dans cette région pour suivre la trajectoire de la voie. Les résultats des expériences ont montré que le système proposé présente une détection des voies robuste et précise. L'approche s'est avérée efficace en termes de rapidité de traitement et de précision d'identification, répondant ainsi aux exigences de la détection des voies pour les systèmes légers de conduite automatique.

Somda et Sere [57] ont proposés une approche novatrice pour résoudre le problème de la congestion routière en exploitant la technologie du Système de Positionnement Global (GPS) intégrée aux véhicules, associée à des serveurs de station de contrôle. La solution proposée vise à atténuer la congestion routière sans nécessiter l'installation de nombreuses caméras ou d'infrastructures supplémentaires sur les routes. Le cœur de cette étude tourne autour de la mise en œuvre de la méthode de transformation de Hough au sein des Réseaux Véhiculaires Ad-Hoc (VANET) pour une détection et une surveillance efficaces de la congestion routière. En établissant un réseau VANET, l'échange de données liées à la sécurité entre les véhicules devient possible, améliorant ainsi la sécurité des usagers de la route. En réponse aux préoccupations croissantes concernant la congestion routière et l'augmentation des accidents de la route, divers scénarios de circulation ont été formulés, conçus et testés.

Katru Amandeep et Anil Kumar [8] ont proposés une technique utilisant la transformée de Hough pour améliorer la détection des voies dans les systèmes Ad-Hoc véhiculaires en temps réel. La recherche se concentre sur l'optimisation de la détection des voies en exploitant la transformée de Hough. Cette méthode est utilisée pour identifier les voies courbes, tandis que la conversion du parallélisme des données vise à augmenter la vitesse de la technique proposée..

Pour une comparaison exhaustive des performances, les résultats de l'algorithme proposé seront mis en parallèle avec ceux des algorithmes de détection de voies déjà existants. Dans le contexte des transports intelligents, qui sont en constante évolution pour accroître la sécurité routière et réduire les

24

incidents, cette nouvelle technique basée sur la transformée de Hough vise à surmonter les limites des approches existantes.

L'algorithme proposé a été soigneusement conçu et implémenté dans l'environnement MATLAB, fournissant ainsi une plate-forme de développement et d'expérimentation solide pour évaluer son efficacité et ses performances.

Le domaine de l'aviation présente d'importants défis environnementaux, notamment en ce qui concerne l'impact du transport aérien sur le changement climatique. Les traînées de condensation émises par les avions contribuent particulièrement à ce problème en raison de leur potentiel à aggraver le réchauffement planétaire. Cependant, la détection des traînées de condensation à partir d'images satellites s'avère être une tâche complexe et ancienne. Les méthodes classiques de traitement d'images informatique montrent leurs limites dans des conditions d'imagerie changeantes. De plus, les approches traditionnelles de l'apprentissage automatique, impliquant des réseaux de neurones convolutifs classiques, se heurtent à des obstacles tels que le manque de données annotées spécifiquement pour les traînées de condensation et des processus d'apprentissage inadaptés à ce domaine. Dans ce présent article de Sun et al [97], une approche novatrice est introduite, reposant sur l'utilisation de l'apprentissage par transfert amélioré. Cette approche permet d'identifier de manière précise les traînées de condensation en utilisant un minimum de données. En parallèle, une nouvelle fonction de perte appelée "SR Loss" est proposée, laquelle améliore la détection des traînées en transformant l'espace de l'image en espace de Hough. Cette recherche ouvre de nouvelles perspectives pour la détection basée sur l'apprentissage automatique dans le domaine de l'aéronautique, en offrant des solutions pour pallier le manque d'ensembles de données volumineux annotés manuellement. De plus, ces avancées contribuent significativement à l'amélioration des modèles de détection des traînées de condensation.

7.2 Images satellitaires

Les travaux de Maja Michalowska et al [58] décrivent l'utilisation de données de balayage laser terrestre dans la gestion et la planification forestières. Ces données permettent d'obtenir des informations géospatiales pour évaluer les caractéristiques des arbres et des peuplements forestiers. L'approche présentée dans l'article combine la transformée de Hough circulaire, le débruitage des données et une méthode robuste d'ajustement au cercle des moindres carrés pour extraire avec précision les positions des troncs d'arbres à partir des données de balayage laser. L'étude commence par la détection initiale des positions des tiges d'arbres grâce à la transformée de Hough circulaire. Ensuite, les résultats sont filtrés pour éliminer les points non pertinents, ce qui améliore la qualité de la détection. Les données tridimensionnelles du balayage laser sont ensuite analysées en utilisant une méthode d'ajustement au cercle des moindres carrés, permettant d'identifier précisément les positions des tiges d'arbres. Les résultats obtenus sont prometteurs. Environ 94,8 % des arbres ont été localisés avec une précision moyenne de 87,2 %. Les erreurs moyennes pour la position du tronc et le rayon de l'arbre mesuré au

25

sol sont respectivement de 1,64 cm et 1,15 cm. Cette méthodologie offre des applications potentielles étendues. Elle peut être utilisée comme base pour analyser automatiquement des caractéristiques telles que l'essence, la hauteur et l'étendue du houppier des arbres. De plus, cette approche peut être adaptée à la détection d'autres objets de forme circulaire, comme des lampes ou des pylônes électriques, lorsque des données de balayage laser sont disponibles. Les positions détectées peuvent être intégrées dans des systèmes d'information géographique ou des applications industrielles pour répondre à des exigences légales liées à la position géodésique des objets.

En 2010, Poncelet et al [22] ont développé une technique basée sur la méthode de la transformée

de Hough pour identifier des linéaments dans des images et appliquée aux résultats obtenus par des détecteurs de contours. Après avoir testé ses performances sur une image bien-structurée, représentant un paysage régulier de la région de Hesbaye, cette méthode a été mise en œuvre sur une image Landsat TM5 et un modèle numérique de terrain couvrant les



popups.uliege.be
<https://popups.uliege.be/0770-7576/index.php?id=1036&file=1>

massifs montagneux des Cima d'Asta et des

Lagorai dans les Dolomites. Les résultats issus de ces deux types de données spatiales sont comparés

et mis en relation avec la structure géologique de la zone. Les avantages et inconvénients spécifiques de chaque approche sont examinés. Il est évoqué que la détection des formations naturelles présente des challenges et que la question de l'échelle spatiale de reconnaissance est importante. Par ailleurs, les limitations du programme implémenté sont étudiées. Bien que fonctionnel, l'efficacité du détecteur de contours joue un rôle crucial, et les effets de normalisation appliqués aux zones d'intérêt peuvent introduire des biais dans le calcul des fréquences, qui ne sont pas encore complètement maîtrisés.

7.3 Industrie

En raison des normes de qualité de plus en plus exigeantes dans l'industrie mécanique moderne concernant la création de trous, les méthodes conventionnelles de détection manuelle ne sont plus viables. Il est essentiel de mettre en place un système mécanisé pour assurer un contrôle de qualité en temps réel et une production automatisée de trous. C'est dans cette optique que Wenfeng Hou [59] combine la technique de détection de cercles de Hough avec un système automatisé de repérage de trous, opéré par un robot industriel. La démarche de recherche implique une analyse approfondie de la structure globale du système de localisation automatique des trous. Cela englobe la confirmation des emplacements critiques ainsi que des paramètres nécessaires pour la détection. Par la suite, la méthode de détection de cercles de Hough est mise en œuvre et son efficacité est évaluée au travers d'expérimentations. Les résultats montrent une précision moyenne de détection de 98,0 % pour quatre types de défauts. Cette réussite souligne l'efficacité et la faisabilité de l'intégration de la détection de cercles de Hough au système de perçage automatisé utilisant des robots industriels. De plus, cette approche propose des perspectives réalisables pour des applications connexes.

26

7.4 Domaine de l'intelligence artificielle

La correction du désalignement des documents pose un problème fondamental lors du traitement d'images de documents. Les approches existantes présentent des limitations. Par exemple, la méthode de transformation de ligne de Hough peut induire une inversion erronée des images, tandis que les modèles d'apprentissage profond exigent d'importantes ressources humaines et informatiques, sans garantir une correction complète incluant l'orientation. Les méthodes basées sur l'OCR éprouvent également des difficultés à interpréter le texte en cas d'inclinaison. Shafi et al [98] ont proposés, en 2023, une nouvelle méthode d'apprentissage profond, à la fois simple et économique, pour remédier à l'inclinaison et à l'orientation des documents. L'approche restreint l'espace de recherche du modèle d'apprentissage automatique en prédisant si une image est inversée, sans nécessiter la prédiction d'un angle entre 0 et 360 degrés. Ils ont raffiné un modèle MobileNetV2, pré-entraîné sur Imagenet, en utilisant seulement 200 images, obtenant ainsi des résultats prometteurs. Cette méthode se révèle utile pour des tâches automatisées telles que l'extraction de données via la technologie OCR, réduisant substantiellement la nécessité d'interventions manuelles.

La reconnaissance des mouvements de la main joue un rôle essentiel dans l'interaction entre l'humain et la machine. Avec l'avancement des caméras capables de capturer la profondeur, la fusion des images en couleur et en profondeur permet d'obtenir des informations plus riches pour la détection des gestes de la main. Dans ce document, nous présentons un système de détection de gestes de la main qui se base sur les données enregistrées par la caméra frontale Intel RealSense SR300. Étant donné que les pixels des images en profondeur provenant du dispositif RealSense diffèrent de ceux des images en couleur, le système de détection proposé par Liao et al [15] associe les images en profondeur aux images en couleur en utilisant une technique appelée transformée de Hough généralisée. Cette méthode permet de séparer la main du contexte complexe de l'image en couleur en se basant sur les informations de profondeur. Ensuite, le système identifie divers gestes de la main grâce à un réseau de neurones convolutif à double canal spécialement conçu, qui intègre deux flux d'entrée distincts : les images en couleur et les images en profondeur. De plus, les auteurs ont constitué une base de données comprenant 24 types de gestes de la main, représentant chacun une lettre de l'alphabet anglais. Cette base contient un total de 168 000 images, réparties équitablement entre les images RVB et les images en profondeur, soit 84 000 de chaque. Les résultats expérimentaux obtenus en utilisant cette nouvelle base de données de gestes de la main démontrent l'efficacité de l'approche, avec un taux de précision de reconnaissance atteignant 99,4 %.

7.5 Médecine



La méthode de la transformée de Hough est un algorithme traditionnel de détection de formes

qui a été largement employé pour repérer des lignes droites, des cercles ainsi que des ellipses au sein

27

d'images. Récemment, des concepts issus de la géométrie algébrique ont été employés pour étendre cette approche à des catégories particulières de courbes. Campi Cristina et al exposent dans l'article [19] des résultats préliminaires qui illustrent la potentialité d'utiliser cette technique pour identifier des structures osseuses denses dans des images cliniques obtenues par tomodensitométrie (scanner) et qui pourrait ainsi servir d'outil informatique efficace pour des applications en hématologie et oncologie.

Le cancer du sein constitue l'un des problèmes majeurs en matière de santé à l'échelle mondiale.

La détection précoce des anomalies liées au cancer du sein offre les meilleures perspectives de guérison.

Parmi les techniques les plus performantes et largement adoptées pour repérer et diagnostiquer ce can-

cer figure la mammographie. L'article de Vijayarajeswari et al [20] se focalisent sur la catégorisation

des mammographies en utilisant les propriétés extraites via la transformée de Hough. La transformée

de Hough, une méthode bidimensionnelle, est employée pour isoler les caractéristiques spécifiques

d'une forme dans une image. La détection des micro-calcifications et des masses représente deux indi-

cateurs cruciaux du risque, et leur identification automatisée revêt une importance primordiale pour

le dépistage précoce du cancer du sein. Étant donné que les masses sont souvent ambiguës par rapport

au tissu environnant, la localisation et la caractérisation automatisées des masses s'avèrent également

complexes. Cet article explore les approches de classification et d'extraction de caractéristiques. Ici,

la transformée de Hough est mise en œuvre pour repérer les éléments distinctifs au sein des images

mammographiques, lesquels sont ensuite classés à l'aide d'une machine à vecteurs de support (SVM).

L'intégration du SVM se traduit par une précision de classification accrue. La méthodologie est testée

sur un échantillon de 95 images mammographiques recueillies et catégorisées par le biais d'un SVM.

Les résultats obtenus démontrent l'efficacité de l'approche proposée pour la classification précise des anomalies au sein des mammographies.

7.6 Élevage

Dans leur papier publié en 2018, C. Yang et al [13] ont présenté une nouvelle méthode pour améliorer le suivi des abeilles. Dans le cadre de cette étude, des vidéos en 2D ont été enregistrées à une fréquence de 50 Hz. La première étape du modèle consiste à suivre les abeilles en combinant le seuillage des couleurs et la détection du premier plan. Ensuite, les abeilles sont représentées sous forme de taches dans une image binaire, et ces taches sont analysées pour estimer les positions et les directions de déplacement des abeilles. Néanmoins, le suivi devient difficile lorsque les abeilles se croisent dans l'image. L'article résout ce problème en appliquant la transformée de Hough standard à la recherche des abeilles lorsqu'elles se croisent. Ensuite, un filtre de Kalman est utilisé pour suivre les abeilles en utilisant leurs positions estimées. Étant donné la fréquence élevée des images (50 Hz), les mouvements des abeilles présentent une grande variabilité, ce qui rend leur suivi fiable complexe. Pour pallier cela, des ajustements sont apportés au filtre de Kalman pour s'adapter à cette situation. Étant donné que plusieurs abeilles sont suivies en même temps, l'algorithme d'affectation hongrois

28

est mis en œuvre pour attribuer les prédictions et les mesures à chaque abeille individuellement. Les résultats de l'expérience démontrent que le suivi des abeilles dans les vidéos 2D à 50 Hz est réalisé de manière fiable, surtout en situation de vue rapprochée.

7.7 Réseaux de communication optique

Les réseaux optiques flexibles en pleine émergence nécessitent des méthodes de surveillance intelligentes pour garantir la qualité du signal et atteindre les niveaux de performance souhaités. Ainsi, Sindhumitha et al [99] ont introduit un réseau de neurones à convolution multitâche (MT-CNN) conçu pour une surveillance polyvalente à l'aide d'une technique novatrice de transformation combinée d'image de Hough (CIHT). Ce système innovant permet de surveiller efficacement les performances optiques à multiples paramètres en utilisant la transformation de Hough (HT) pour extraire les caractéristiques de manière efficiente. Ils ont réalisé des simulations approfondies afin de démontrer les avantages de l'utilisation de la transformation de Hough ainsi que de l'augmentation des données, confirmant ainsi les résultats obtenus.

7.8 Construction et bâtiment

Malgré l'utilisation croissante des drones pour inspecter les façades des bâtiments, leurs avantages restent sous-exploités en raison des difficultés à obtenir une vue d'ensemble rapide et complète de l'état actuel des bâtiments à partir d'images morcelées. Ainsi, Cheng Zhang et al [21] ont présenté une méthode visant à résoudre ce problème. Cette méthode se déroule en trois étapes principales : premièrement, les informations de position et les paramètres optiques des images capturées par les drones sont utilisés pour configurer des caméras virtuelles dans un modèle d'information du bâtiment (BIM), créant ainsi des images modélisées. Deuxièmement, une technique améliorée basée sur la transformée de Hough généralisée est employée pour extraire les éléments de la façade du bâtiment, quelles que soient leurs formes. Enfin, dans la troisième étape, les images des drones sont projetées sur une orthophoto et intégrées dans des vues en deux dimensions et en trois dimensions. Pour automatiser ce processus, un prototype appelé Dynamo a été développé. Les expérimentations informatiques et sur le

terrain ont validé cette approche en montrant une marge d'erreur moyenne de 21 mm lors de l'enregistrement des images dans le modèle BIM (Building Information Modeling). Ces résultats illustrent le potentiel de l'approche proposée pour faciliter l'inspection des façades de bâtiments en utilisant des drones.

De multiples applications urbaines requièrent des polygones de bâtiments en tant que données d'entrée. Cependant, extraire manuellement ces polygones à partir de nuages de points implique un investissement considérable en termes de temps et d'efforts. La méthode de la transformée de Hough est reconnue pour extraire des caractéristiques linéaires, mais les approches actuelles basées sur cette méthode manquent de souplesse pour extraire de manière efficace les contours de bâtiments complexes

29

et variés. L'information concernant l'ordre des points est souvent négligée. En exploitant les points de bord de bâtiments ordonnés, Elyta et al [14] ont développé une nouvelle variante de la transformée de Hough appelée "transformée de Hough assistée par des points ordonnés" (OHT). Cette méthode permet d'extraire avec précision les contours de bâtiments à partir de nuages de points LiDAR aéroportés. Cette approche consiste tout d'abord à construire une matrice d'accumulation de Hough en utilisant un système de vote dans un espace paramétrique linéaire (θ, r). La dispersion des angles dans chaque colonne est employée pour identifier les orientations prédominantes des bâtiments. Les auteurs ont mis au point une méthode de filtrage et de regroupement hiérarchique pour obtenir des lignes précises à partir des points saillants détectés et des points de bord de bâtiments ordonnés. L'utilisation d'une matrice de liste de points ordonnés, comprenant les points de bord de bâtiments disposés dans un ordre spécifique, permet de détecter des segments de ligne dans des directions variées, ce qui aboutit à l'obtention de polygones de toit de bâtiments de grande qualité. La méthode a été soumise à des tests sur trois ensembles de données distincts : un nouvel ensemble à Makassar, en Indonésie, ainsi que deux ensembles de référence à Vaihingen, en Allemagne, caractérisés par des particularités différentes. L'algorithme de la méthode est la première application de la transformée de Hough offrant une grande adaptabilité, étant en mesure de traiter des bâtiments aux contours de longueurs et d'orientations variées. Les résultats montrent des taux élevés d'exhaustivité (entre 90,1 % et 96,4 %) ainsi que de précision (tous supérieurs à 96 %). L'exactitude de l'alignement angulaire des bâtiments présente un écart type moyen entre 0,2 et 0,57 m. Par ailleurs, le score de qualité (89,6 %) pour le benchmark Vaihingen-B dépasse les performances des méthodes concurrentes de pointe. Bien que les solutions pour l'ensemble de données difficile Vaihingen-A ne soient pas encore disponibles, nous obtenons un score de qualité de 93,2 %. Enfin, ils ont démontré l'efficacité de la méthode sur des bâtiments complexes à orientations variées près du musée EYE à Amsterdam.

7.9 Agriculture

Ces dernières années, la recherche dans le domaine de l'automatisation agricole a particulièrement mis l'accent sur le développement des tracteurs autonomes. Une tâche cruciale pour la planification de la conduite autonome concerne la détection des frontières du sol labouré. Récemment, plusieurs études ont exploré des approches basées sur la vision artificielle pour cette détection, et des progrès significatifs sont en cours. Cependant, la plupart des méthodes existantes se concentrent principalement sur des caractéristiques locales, ce qui entraîne des performances en deçà du potentiel optimal. En outre, les méthodes précédentes souffrent généralement d'interférences locales, nécessitant ainsi une étape de post-traitement heuristique afin d'obtenir les contours définitifs des zones labourées. C'est

dans ce contexte que Oh Knaghan et al [100] ont introduit une nouvelle approche de détection des frontières du sol labouré, basée sur l'apprentissage profond, capable d'appréhender les caractéristiques contextuelles des lignes. L'approche proposée repose sur une fusion entre le soft-argmax et la trans-

30

formation de Hough. Grâce à l'utilisation du soft-argmax de Hough, les auteurs transforment les caractéristiques profondes issues des réseaux de neurones convolutifs en caractéristiques contextuelles liées aux frontières du sol labouré. Ensuite, ils effectuent une régression directe des paramètres de la transformation de Hough en explorant l'espace vectoriel transformé. Étant donné que les lignes droites sont paramétrées par ces paramètres, la méthode que nous avançons propose un apprentissage intégral et ne nécessite aucune phase de traitement ultérieur. Pour évaluer notre proposition, les auteurs mènent une expérimentation en créant une importante base de données spécifique à la détection des frontières du sol labouré. Les résultats des expériences confirment l'efficacité de notre méthode par rapport aux approches existantes en matière de détection des frontières du sol labouré.

G. Lin et al [16] ont présenté une nouvelle méthode pour repérer des fruits dans des milieux naturels, adaptée aux robots de récolte automatisée, aux systèmes d'évaluation des rendements et aux dispositifs de contrôle de qualité. Contrairement aux techniques couramment basées sur la couleur, qui souffrent de sensibilité aux variations de lumière et aux contrastes faibles entre les fruits et les feuilles, l'approche proposée se fonde sur les caractéristiques des contours. Tout d'abord, un indicateur de forme distinctif est dérivé pour représenter les propriétés géométriques d'une partie quelconque, et est utilisé dans une correspondance partielle à double sens des formes. Cela permet de repérer les parties d'intérêt correspondant à des parties d'un contour de référence. Ensuite, une nouvelle transformation de Hough probabiliste est élaborée pour combiner ces morceaux afin de détecter les candidats-fruits. Enfin, un classificateur à vecteur de support, formé sur des caractéristiques de couleur et de texture, vérifie tous les candidats-fruits. Des ensembles de données couvrant différents types de fruits tels que les agrumes, les tomates, les citrouilles, les courges amères, les serviettes et les mangues ont été mis à disposition. Les expériences menées sur ces ensembles de données ont démontré que cette approche est compétitive pour la détection d'une variété de fruits, qu'ils soient verts ou oranges, de forme circulaire ou non, dans des environnements naturels.

La détection des rangs de café sur des champs imagés par UAV reste un défi à relever. Bien que de nombreuses méthodes aient abordé cette problématique en utilisant la transformée de Hough, cette approche présuppose à tort que les rangées de culture sont linéaires, ce qui n'est pas le cas dans les images prises depuis les airs. La contribution de Soares et al [17] consiste en un système de tuiles qui parvient à approximer de manière satisfaisante les rangées au sein de chaque tuile sous forme de segments droits, rendant ainsi possible l'utilisation de la transformée de Hough. Les résultats expérimentaux, comparés à la réalité du terrain, suggèrent que notre méthode permet d'effectivement se rapprocher des rangées de culture réelles.

Dans le contexte de l'agriculture durable, il y a une demande croissante pour des solutions robotiques. En particulier, les robots conçus pour l'agriculture de précision ouvrent de nouvelles perspectives. Ces applications requièrent généralement une grande précision dans leur système de navigation. Un élément clé pour une navigation fiable est la capacité à détecter de manière robuste les rangées de

cultures. Cependant, repérer les cultures à partir de données visuelles ou laser devient particulièrement complexe lorsque les plantes sont soit très petites, soit si grandes qu'elles se confondent. Winterhalter et al [18] ont exposé une méthodologie pour segmenter de manière fiable les plantes à différents stades de croissance. De plus, ils introduisent un nouvel algorithme destiné à détecter avec robustesse les rangées de cultures. Cet algorithme adapte la transformée de Hough utilisée pour la détection de lignes, permettant ainsi de repérer un modèle de lignes parallèles équidistantes. L'algorithme est en mesure d'estimer simultanément l'angle, le décalage latéral et l'espacement entre les rangées de cultures. Il se révèle particulièrement adapté pour identifier les plantes très petites. Des expérimentations approfondies, menées sur divers jeux de données réelles provenant de cultures de types et de tailles différentes, démontrent que notre algorithme produit des résultats fiables et précis.

7.10 Sécurité

La transformée de Hough est une technique de traitement d'image qui permet de détecter des formes géométriques telles que des lignes, des cercles ou des ellipses. Cette technique est largement utilisée dans le domaine de la sécurité pour la détection d'objets ou de personnes dans des images de vidéo-surveillance. Elle peut également être utilisée pour la reconnaissance des empreintes digitales. Les empreintes digitales sont un moyen courant d'identification et de vérification de l'identité dans de nombreux domaines, tels que les services de police, les douanes, les contrôles d'accès. L'article [61] présente une étude sur les algorithmes d'alignement d'empreintes digitales basés sur la Transformée de Hough. Les auteurs classifient les différentes approches basées sur la Transformée de Hough en trois catégories : l'alignement basé sur la correspondance locale, l'alignement basé sur la rotation discrétisée et l'alignement basé sur les paires correspondantes. Les performances de ces approches sont comparées en termes de précision, de temps de calcul et d'utilisation de la mémoire. Les expériences ont été réalisées sur une base de données d'empreintes digitales capturées dans différentes orientations et positions. Les résultats montrent que chaque approche a ses avantages et ses inconvénients en fonction des conditions spécifiques des empreintes digitales. Cette approche offre une méthode robuste et fiable pour l'identification biométrique basée sur les empreintes digitales.

La Transformée de Hough joue un rôle crucial dans la méthode de correspondance d'empreintes digitales. Elle est utilisée par K. Anitha [62] pour aligner et faire correspondre les empreintes digitales en extrayant des lignes droites qui approximent chaque crête d'empreinte digitale individuellement. La Transformée de Hough est appliquée à chaque crête séparément pour détecter les lignes droites qui correspondent le mieux à la crête. Les pics de l'espace de Hough pour chaque crête sont ensuite utilisés pour estimer les paramètres de rotation et de translation nécessaires pour enregistrer les images d'empreintes digitales de requête et de modèle. Ce processus d'alignement simplifie la correspondance des empreintes digitales en calculant un score de correspondance basé sur l'alignement des crêtes. Dans l'ensemble, la Transformée de Hough est essentielle pour faciliter l'alignement et la comparaison des

32

empreintes digitales dans l'algorithme présenté dans le document.

La transforme de Hough linéaire a été utilisé par Nitika et al [63], en combinaison avec la correspondance des minuties pour la reconnaissance des empreintes digitales.

La technologie d'identification par empreintes digitales partielles, qui est principalement utilisée dans les appareils dotés d'une petite surface de détection tels que les téléphones portables, les disques durs et les ordinateurs, a fait l'objet d'une attention accrue ces dernières années en raison des avantages

uniques qu'elle présente. Toutefois, en raison de l'absence de points caractéristiques suffisants, la méthode conventionnelle ne donne pas de bons résultats dans la situation décrite ci-dessus. Qin, Jin et al [64] ont proposé une nouvelle technique de comparaison d'empreintes digitales qui utilise les crêtes comme caractéristiques pour traiter les images d'empreintes digitales partielles et combine la transformée de Hough généralisée modifiée et la stratégie de notation basée sur l'apprentissage automatique. L'algorithme peut répondre efficacement aux exigences de temps réel et d'économie d'espace des appareils à ressources limitées. Les expériences menées sur une base de données interne indiquent que l'algorithme proposé présente d'excellentes performances.

8 Domaines émergents

Les avancées rapides en vision par ordinateur et le développement de nouvelles technologies ont ouvert la voie à de nombreux domaines émergents où la Transformée de Hough pourrait jouer un rôle important. Voici quelques-uns de ces domaines émergents :

♦Véhicules autonomes : la détection d'objets et de lignes est essentielle pour la navigation des véhicules autonomes.

**7**

Document d'un autre utilisateur
Le document provient d'un autre groupe

La Transformée de Hough peut être utilisée pour détecter les lignes de la route,

les panneaux de signalisation, les véhicules et les piétons, contribuant ainsi à la sécurité et à la prise de décision dans ces véhicules.

♦Réalité augmentée et réalité virtuelle

**8**

Document d'un autre utilisateur
Le document provient d'un autre groupe

: la Transformée de Hough peut être utilisée pour détecter

et suivre des objets en temps réel, ce qui est essentiel pour des applications de réalité augmentée et de réalité virtuelle. Elle peut aider à suivre les mouvements des utilisateurs et à intégrer des objets virtuels dans des environnements réels.

♦Agriculture de précision : l'utilisation de drones et de robots dans l'agriculture de précision nécessite la détection et la reconnaissance d'objets tels que les cultures, les maladies des plantes et les débris dans les champs. La Transformée de Hough pourrait être utile pour détecter et suivre ces objets dans des images aériennes.

♦Médecine et imagerie médicale : la détection d'objets anatomiques et de structures médicales est essentielle dans l'imagerie médicale. La Transformée de Hough pourrait être appliquée pour détecter des organes, des vaisseaux sanguins et des lésions dans des images médicales, contribuant ainsi au diagnostic et à la planification de traitements médicaux.

♦Surveillance et sécurité : la Transformée de Hough peut être utilisée dans des systèmes de surveillance pour détecter des objets suspects ou des comportements anormaux. Elle peut être appliquée dans des systèmes de vidéo surveillance pour détecter des intrusions, des véhicules suspects ou des colis abandonnés.

♦Fabrication et contrôle qualité : dans le domaine industriel, la Transformée de Hough peut être utilisée pour détecter des défauts dans des pièces fabriquées, vérifier la qualité des produits et aider à l'automatisation des processus de fabrication.

◆ Robotique collaborative : la détection et la localisation précises des objets sont essentielles dans

la robotique collaborative où les robots doivent interagir de manière sûre et efficace avec des objets et des humains.

◆ Détection d'objets dans des vidéos en temps réel : avec la demande croissante de surveillance

vidéo en temps réel, la Transformée de Hough peut jouer un rôle essentiel dans la détection rapide d'objets dans des vidéos en direct.

Ces domaines émergents représentent des opportunités prometteuses pour la Transformée de Hough et son intégration avec d'autres techniques avancées de vision par ordinateur et d'apprentissage automatique. En explorant ces domaines, la Transformée de Hough pourrait continuer à évoluer et à se développer pour répondre aux besoins en constante évolution de la société moderne.

Conclusion

La Transformée de Hough demeure une méthode important dans le domaine du traitement d'images. Depuis son introduction initiale pour la détection de lignes droites dans les années 1960, elle a été étendue et adaptée pour détecter une gamme variée de formes géométriques, allant des cercles et des ellipses aux formes arbitraires définies par un ensemble de points.

Les nombreuses variantes de la Transformée de Hough offrent des solutions spécifiques aux défis rencontrés dans différents domaines d'application, que ce soit dans l'industrie, la médecine, l'agriculture, les véhicules autonomes ou la sécurité. Que ce soit pour la détection d'objets dans des images de surveillance, l'identification de structures anatomiques dans des images médicales, ou la navigation des véhicules autonomes, la Transformée de Hough continue d'être un outil indispensable.

Avec les avancées rapides en intelligence artificielle, ainsi que l'émergence de nouveaux domaines tels que la réalité augmentée et la médecine personnalisée, il est probable que la Transformée de Hough continuera à jouer un rôle important dans l'analyse et la compréhension des données visuelles.

Dans la suite, nous proposerons la reconnaissance des objets discrets par de nouvelles approches de la transformée de Hough introduite dans le cadre de ces travaux.

34

2

Ch
ap

itr
e

Transformée de Hough Rectangulaire

Introduction 35

1 Description de la méthode



..... 36

2 Complexité de la Transformée de Hough Rectangulaire 40

3 Simulations et discussions
..... 42

Conclusion 55

Introduction

La détection de lignes droites dans les images est un problème fondamental en vision par ordinateur, largement étudié pour ses applications dans des domaines tels que la robotique, la reconnaissance de

formes, la cartographie. Parmi les nombreuses techniques développées pour résoudre ce problème, la Transformée de Hough est l'une des approches les plus connues et les plus utilisées.

Ce chapitre fournit une nouvelle approche de la transformée de Hough pour la détection des droites discrètes. Il s'agit de la transformée de Hough rectangulaire. Nous décrivons sa méthode de fonctionnement, ses étapes clés et ses paramètres. Nous menons également des simulations sur des images représentatives pour évaluer les performances de la transformée de Hough rectangulaire dans la détection de lignes droites.

De plus, nous comparons les performances de la transformée de Hough rectangulaire avec celles de la Transformée de Hough Standard, avec sa version parallélisée THRP, ainsi qu'avec une approche proposée par Traore et al [4]. Cette comparaison nous permet d'évaluer l'efficacité de la THR par rapport aux méthodes existantes en termes de précision de détection et de temps d'exécution.

35

1 Description de la méthode

Dans cette section, nous décrivons la technique de la transformée de Hough rectangulaire. Les différentes étapes sont illustrées dans la figure 2.3. En outre, nous détaillons l'algorithme 6 de calcul du taux de pixels allumés dans une région rectangulaire. Enfin, nous discutons également des considérations pratiques pour déterminer la taille optimale des cellules, afin d'assurer une bonne détection des droites tout en minimisant le temps de traitement.

1.1 Maillage de l'image

L'algorithme 3 est l'algorithme de maillage rectangulaire. Il prend en paramètre les dimensions de l'image (NI : la hauteur de l'image, Nc : la largeur de l'image), les dimensions du rectangle (L :longueur, l : largeur) et retourne un tableau de 4 entiers dont les deux premières valeurs sont les dimensions de la cellule rectangulaire et les deux dernières valeurs sont les résidus respectivement de la hauteur et de la largeur de l'image. Les résidus de la hauteur sont les pixels restants après avoir subdivisé la hauteur de l'image en parties égales de longueur l. c'est le reste de la division euclidienne de NI par l. De même, les résidus de la largeur sont les pixels restants après avoir subdivisé la largeur de l'image en parties égales de longueur L. c'est le reste de la division euclidienne de Nc par L.

Algorithme 3 : Construction de la grille rectangulaire
Fonction maillage_rect(NI, Nc,l, L : entiers) : tableau d'entiers

```
Variables : tab : tableau de 6 entiers
Output : tab;
begin
tab[0]← L ; % Longueur du rectangle
tab[1]← l ; % Largeur du rectangle
tab[2]← NI mod l ; % le reste de la division euclidienne de NI par l
tab[3]← Nc mod L ; % le reste de la division euclidienne de NI par l
return tab;

fin
```

La grille rectangulaire, représenté par la figure 2.2, issue du maillage de l'espace image est constitue de cellules rectangulaires de longueur AB et de largeur AD comme illustré dans la figure 2.1.

1.2 Sélection des cellules

Le critère de sélection d'un rectangle dans la grille repose sur le pourcentage de pixels allumés à l'intérieur de ce rectangle. Ce pourcentage est déterminé à l'aide de l'algorithme 6 décrit comme suit :

36

x

A

D

B

C

(a) Rectangle ABCD

y

x

A

D

B

C

(b) Surface du rectangle ABCD

Figure 2.1 – Cellule rectangulaire

y

x

Figure 2.2 – Grille rectangulaire superposée à l'espace image

- Paramètres d'entrées : *img*, représentant l'image sur laquelle l'algorithme opère, et *A*, *B*, *D* trois tableaux de deux entiers chacun, représentant les coordonnées de trois sommet d'un rectangle.
- Paramètre de sorties : l'algorithme renvoie un nombre réel représentant le taux de pixels allumés dans la région rectangulaire spécifiée.
- Variables : *k* et *l* sont des variables entières initialisées à 0. Elles sont utilisées pour compter les pixels allumés et éteints, respectivement.
- Les boucles : deux boucles imbriquées parcourent les pixels dans la région rectangulaire spécifiée.
- L'analyse des pixels : pour chaque pixel aux coordonnées (*x*, *y*), l'algorithme vérifie si la valeur du pixel dans l'image n'est pas égale à 0. Si c'est vrai, il incrémente *k* (nombre de pixels allumés) ; sinon, il incrémente *l* (nombre de pixels éteints).
- Enfin, l'algorithme renvoie le rapport de pixels allumés sur le nombre total de pixels dans la région, calculé comme $k/(k + l)$.

1.3 Parallélisation de la transformée de Hough rectangulaire

La parallélisation de la transformée de Hough rectangulaire nécessite une adaptation de l'algorithme pour tirer parti des capacités de traitement simultané offertes par le parallélisme. La parallélisation concerne l'accumulateur c'est-à-dire l'espace des paramètres encore appelé l'espace de

37

Algorithme 4 : Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire

Fonction Rectangular(*img*,*l*,*L*, α ,seuil) :tableau

Pre-condition : $1 \leq L \leq NI$, $1 \leq l \leq Nc$, $0 \leq \alpha \leq 1$, $0 < \text{seuil}$

Variables : *tab* : tableau de 6 entiers ;

A, *B*, *D* : tableau de 2 entiers ; *accum row*, *accum column*, *irho*, *NI*, *Nc* : entiers;

accum : matrice de dimensions "*accum row* × *accum column*";

dtheta, *rho*, *theta* : real;

Begin

%les dimensions de l'image ;



NI ← nombre de ligne de l'image ; *Nc* ← nombre de colonne de l'image;

```
% the dimensions of "accum";
```

```
accum row ← 180 ; accum column ← E(  
√
```

```
(Nc2 + NI2));
```

```
dtheta =  $\pi/180$  ; tab ← maillage rect(NI, Nc, l, L) ;
```

```
H = NI - tab[2] ; L = Nc - tab[3] ; H1 = H + tab[1] + 1 ; L1 = L + tab[0] + 1 ;
```

```
for x allant de tab[1] à H1 par pas de tab[1]+1 do
```

```
for y allant de tab[0] à L1 par pas de tab[0]+1 do
```

```
A[0] = x - tab[1] ; A[1] = y - tab[0] ; B[0] = x - tab[1] ; B[1] = y ; D[0] = x ; D[1] = y - tab[0] ;
```

```
if count(img, A, B, D) ≥  $\alpha$  then
```

```
for z allant de A[0] à D[0] do
```

```
for t allant de A[1] à B[1] do
```

```
Acc update();
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
Rechercher les maxima dans l'accumulateur et placer les couples (p,  $\theta$ ) dans une liste 'lignes';
```

```
Retourner la liste 'lignes';
```

```
fin
```

```
fin
```

Algorithme 5 : Mise à jour de l'accumulateur

Procédure Acc update() : vide

```
% Mise à jour de l'accumulateur
```

```
if img[z, t] = 0 then
```

```
for itheta dans range(accum row) do
```

```
theta ← itheta × dtheta;
```



```
rho ← t × cos(theta) + z × sin(theta);
```

```
irho ← int(rho);
```

```
if irho > 0 et irho < accum column then
```

```
accum[itheta][irho] ← accum[itheta][irho] + 1
```

```
end
```

```
end
```

```
end
```

```
fin
```

Hough.

Les algorithmes 8 et 7 sont les versions parallélisées des algorithmes 4 et 5. Les étapes princi-

pales de la parallélisation sont :

— importation de la bibliothèque pypm 2 : importer une bibliothèque en Python est nécessaire

pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction import

pypm est utilisée pour charger la bibliothèque pypm.

— création de l'accumulateur de type pypm : dans l'algorithme séquentiel, l'espace des paramètres

(l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la

transformée de Hough, est transformé en une structure de données partagée grâce à pypm. En

effet, pypm permet de créer et de gérer des tableaux partagés entre différents threads, ce qui

peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.

2. <https://github.com/classner/pypm/tree/main>

38

Algorithme 6 : Taux de pixels allumés

Fonction count rect(img : image, A, B, D : tableaux) : réel

Variables : k, l : entiers;

```

begin
k ← 0 ; % initialisation du compteur du nombre de pixels allumés
l ← 0 ; % initialisation du compteur du nombre de pixels éteints
for x allant de A[0] à D[0] do

    for y allant de A[1] à B[1] do
        if img[x, y] = 0 then

            k ← k + 1 ;
        else

            l ← l + 1 ;
        end
    end

end

return k

k + l
; % le taux de pixels allumés

fin

```



L'instruction "accum pypm ← pypm.shared.array([accum row, accum column])" de l'algorithme

8 crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable "accum pypm".

— construction parallèle : l'instruction "with pypm.



Parallel(4) as p" de l'algorithme 8 indique qu

e nous souhaitons exécuter le bloc de code à l'intérieur de la déclaration "with" en parallèle avec 4 processus.

— parallélisation de l'algorithme 5 de mise à jours de l'accumulateur : la fonction "p.range" génère une séquence d'indices à partir de laquelle chaque cœur du CPU obtiendra une plage d'indices spécifique. Cela permet de diviser le travail entre les cœurs du CPU de manière équitable. Ainsi la boucle "for itheta in p.range(accum row)" de l'algorithme 7 utilise les indices compris entre 0 et accum row – 1 pour itérer à travers l'accumulateur par sections, où chaque section est traitée par un processus différent.

1.4 Reconnaissance des droites discrètes

L'algorithme 4 de reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire prend une image en entrée et utilise une grille générée avec des paramètres spécifiques. Il parcourt ensuite cette grille, définissant des régions rectangulaires à partir de points délimités par les points A, B, C, D. Si le taux de pixels allumés dans une région dépasse un seuil défini par α , la fonction Acc update (l'algorithme 5) est appelée pour mettre à jour la matrice accumulateur. Après avoir exploré toutes les régions, l'algorithme recherche les maxima dans l'accumulateur, puis place les couples $(\theta; p)$ (les coordonnées des maxima dans l'accumulateur) dans une liste. Les paramètres des droites détectées correspondent aux coordonnées des maxima de l'accumulateur. Cette liste est ensuite ren-

Algorithme 7 : Mise à jour parallèle de l'accumulateur
Procédure Acc updateP() : vide

```

% Mise à jour de l'accumulateur
if img[z, t] = 0 then

```

```
for itheta dans p.range(accum row) do
% Calcul des coordonnées polaire du pixel "img[z,
```



```
tj". theta ← itheta× dtheta;
rho ← t× cos(theta) + z × sin(theta);
irho ← int(rho);
if irho > 0 et irho < accum column then

accum pypm[itheta][irho] ← accum pypm[itheta][irho] + 1
end

end
end

fin
```

voyée,
fournissant les paramètres des droites discrètes détectées dans l'image.

L'algorithme 9 a pour objectif de visualiser les droites détectées par la méthode de la Transformée de Hough. Il prend en entrée une image (imge), la liste de paramètres de droites détectées (lignes) renvoyée par l'algorithme 4, et une couleur (colors). L'algorithme parcourt la liste des paramètres de droites détectées et reconstruit chaque droite correspondante sur l'image. Les droites reconstruites sont de couleur colors. Cette reconstruction permet de mettre en évidence les droites détectées visuellement sur l'image d'origine, facilitant ainsi l'interprétation et l'analyse des résultats de la Transformée de Hough.

2 Complexité de la Transformée de Hough Rectangulaire

La fonction maillage rect dans l'algorithme 3 récupère les dimensions d'une image et assigne des valeurs à un tableau, puis retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

La fonction count rect dans l'algorithme 6 : initialise des variables k et l1, puis elle parcourt les pixels à l'intérieur du rectangle à l'aide de boucles dont le nombre d'itérations est $L \times l$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité du bloc formé par les boucles est $O(L \times l)$, où $L \times l$ est le nombre de pixels à l'intérieur du rectangle (l'aire du rectangle). Ainsi, la complexité totale de la fonction count rect est $O(L \times l)$.

La fonction Acc update dans l'algorithme 5 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de accum row = 180, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction Acc update est $O(1)$.

L'algorithme 4 de la Transformée de Hough rectangulaire (THR) commence par l'initialisation des

40

Algorithme 8 : Reconnaissance de droites discrètes par la Transformée de Hough Rectangulaire Parallélisée

Fonction RectangularP(imge,l,L,α,seuil,cpu count) :tableau
Pre-condition : $1 \leq L \leq NI$, $1 \leq l \leq Nc$, $0 \leq \alpha \leq 1$, $0 < \text{seuil}$
Variables : tab : tableau de 6 entiers ;
A, B, D : tableau de 2 entiers ; accum row, accum column, irho, NI, Nc : entiers;
accum : matrice de dimensions "accum row × accum column";
dtheta, rho, theta : real;
Begin

```
import pypm ;% importation de la bibliothèque pypm
%les dimensions de l'image ;
```



NI ← nombre de ligne de l'image ; Nc ← nombre de colonne de l'image;

% the dimensions of "accum";

accum row ← 180 ; accum column ← E(
√

(Nc2 + NI2));



accum pypm ← pypm.shared.array([accum row, accum column]) % création de l'accumulateur de type pypm;

dtheta = $\pi/180$; tab ← maillagerectP (NI,

Nc, l, L) ;

H = NI - tab[4] ; L = Nc - tab[5] ; H1 = H + tab[3] + 1 ; L1 = L + tab[1] + 1 ;



% Démarrage du traitement parallèle

With pypm.

parallel(cpu count) as p :

for x allant de tab[1] à H1 par pas de tab[1]+1 do

for y allant de tab[0] à L1 par pas de tab[0]+1 do

A[0] = x - tab[1] ; A[1] = y - tab[0] ; B[0] = x - tab[1] ; B[1] = y ; C[0] = x ; C[1] = y - tab[0] ;

if count(img, A, B, D) ≥ α then

for z allant de A[0] à D[0] do

for t allant de A[1] à B[1] do

Acc update();

end

end

end

end

end

fin

Rechercher les maxima dans l'accumulateur et placer les couples (p, θ) dans une liste 'lignes';

Retourner la liste 'lignes';

fin

fin

Algorithme 9 : Reconstruction des droites détectées

Fonction PlotHoughLine(imge, lignes, colors) : image

Parcourir 'lignes' la liste des paramètres des droites détectées, puis construire chaque droite correspondant en couleur 'colors' sur l'image 'imge'.

fin

variables et du tableau tab, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes

ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction maillage rect dont la

complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux

boucles imbriquées parcourant les points à l'intérieur de chaque rectangle à l'intérieur du quelle se

trouve des opérations de coût constant et la fonction Acc update de complexité $O(1)$. La complexité

de ces boucles est donc $O(L \times l)$.

Le coût des deux boucles principales imbriquées parcourant les rectangles de la grille, dont le

nombre total d'itérations dépend de la taille de l'image et de la taille des cellules rectangulaires,

contenant un bloc d'opérations d'affectation de coût constant, un test de condition sur la fonction

count rect de complexité $O(L \times l)$, les deux boucles imbriquées de coût $L \times l$ parcourant les pixels à

l'intérieur du rectangle, est $(NI/L \times Nc/l) \times (1 + (L \times l) + 1 + (L \times l))$, donc de complexité $O((NI/L \times$

$Nc/l) \times (1 + (L \times l) + 1 + (L \times l))) = O(NI \times Nc)$.

En considérant ces points, la complexité totale de l'algorithme 4 de la transformée de Hough

rectangulaire est égal à $O(NI \times Nc)$, où NI et Nc sont le nombre de lignes et de colonnes de l'image

respectivement. Ce qui correspond à la complexité de la transformée de Hough standard.

Figure 2.3 – Les étapes de la technique de la transformée de Hough rectangulaire

3 Simulations et discussions

Cette section se concentre sur les résultats de simulations obtenus en appliquant l'algorithme de la transformée de Hough rectangulaire à deux cas d'étude distincts : une image représentant un immeuble et une autre dépeignant une autoroute illustré dans la figure 1.11. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Les simulations ont été menées en variant différents paramètres de l'algorithme, tels que α , la taille des cellules rectangulaires SR, et le seuil de vote. Les résultats sont analysés en termes du nombre de lignes

Tableau 2.1 – Résultats des simulations sur l'image1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres fixes ($l = 3$, $L = 5$ and $\text{Seuil} = 70$) et α variant entre 10% et 45% où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

α	Droites détectées	temps(sec)
10	155	0,49
15	134	0,48
20	134	0,49
25	51	0,39
30	32	0,33
35	2	0,20
40	2	0,20
45	0	0,13

détectées, de la précision et du temps de calcul. Cette section présente également une comparaison entre la transformée de Hough rectangulaire et sa version parallélisée, ainsi qu'une confrontation avec l'approche de Traoré et al [4]. Les discussions qui suivent mettent en lumière les tendances observées dans les résultats des simulations, ainsi que les avantages et les limitations de chaque approche.

3.1 Cas de l'image de l'immeuble

Dans cette section, nous examinons les performances de l'algorithme de détection de lignes de la THR appliqué à une image représentant un immeuble. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir L , l et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la transformée de Hough rectangulaire en fonction des caractéristiques de l'image de l'immeuble.

3.1.1Variation de α

Le tableau 2.1 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble en utilisant l'algorithme 4 de la THR. Les simulations ont été réalisées avec des paramètres fixes ($l = 3$, $L = 5$ et $\text{Seuil} = 70$), tandis que le paramètre α a été modifié entre 10% et 45%. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 45%. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse des résultats de la simulation sur l'image de l'immeuble avec l'algorithme de la THR et

les paramètres fixes ($l = 3$, $L = 5$ et $\text{Seuil} = 70$) avec α variant entre 10% et 45% révèle deux tendances

majeures. Premièrement, en ce qui concerne le nombre de lignes détectées, on observe une diminution

43

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45
0

20

40

60

80

100

120

140

160 (155)

(134) (134)

(51)

(32)

(2) (2) (0)

α

de
te

ct
ed

lin
es

detected lines

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45

0.15

0.2

0.25

0.3

0.35

0.4

0.45

0.5

0.55

(0.49) (0.48) (0.49)

(0.39)

(0.33)

(0.20) (0.20)

(0.13)

α

T
im

e

(s

ec
on

de
)

Time

Figure 2.4 – Courbe représentative des données du tableau 2.1

Tableau 2.2 – Résultats des simulations sur l'image1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres fixes ($\alpha = 30\%$ et Seuil = 50) et $1 \leq L \leq NI$ et $1 \leq I \leq Nc$ où I et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

I L SR Nbre de droites détectées temps(sec)
2 4 8 263 0,



35
3 5 15 162 0,33
4 6 24 27 0,23
5 7 35 22 0,22
6 8 48 5 0,18
7 9 63 2 0,18
8 10 80 2 0,16
9 11 99 0 0,
13
168 1 168 14 0.10
171 1 171 14 0.10

progressive à mesure que α augmente. À $\alpha = 10\%$, 155 lignes sont détectées, mais ce nombre diminue régulièrement pour atteindre 0 lignes à $\alpha = 45\%$. Cette tendance suggère que des valeurs plus élevées de α conduisent à une détection moins nombreuse mais plus sélective des lignes droites. Cependant, à partir des certaines valeurs de α , THR ne détecte aucune droites. Deuxièmement, en ce qui concerne le temps de calcul, on observe une diminution correspondante à mesure que α augmente. Pour $\alpha = 10\%$, le temps de calcul est de 0,49 secondes, et il diminue progressivement pour atteindre 0,13 secondes à $\alpha = 45\%$. La diminution du temps de calcul est plus marquée après $\alpha = 30\%$, passant de 0,33 à 0,13 secondes. Cette tendance suggère qu'opter pour des valeurs élevées de α peut améliorer le temps de traitement. La courbe représentative dans la figure 2.4 confirme visuellement ces tendances, avec une décroissance du nombre de lignes détectées d'une part, et une décroissance du temps de calcul d'autre part.

44

Figure 2.5 – (a) Détection de lignes verticales de l'immeuble avec la THR ($I = 168$, $L = 1$, $SR = 168$, $\alpha = 30\%$, seuil = 50), (b) détection de bords de l'autoroute avec la THR ($I = 29$, $L = 238$, $SR = 6902$, $\alpha = 10\%$, seuil = 40)

3.1.2 Variation de la taille des cellules rectangulaires

Le tableau 2.2 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR. Les simulations ont été réalisées avec des paramètres fixes ($\alpha = 30\%$ et Seuil = 50), $1 \leq L \leq NI$ et $1 \leq I \leq Nc$. Le tableau comporte cinq colonnes : La première et la deuxième colonne représentent respectivement les différentes valeurs de la largeur I et la longueur L du rectangle. La troisième colonne représente l'aire SR du rectangle. La quatrième colonne montre le nombre de droites détectées pour chaque valeur de SR. La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse des données présentées dans le tableau et les graphiques correspondant de la figure 2.6) permet de tirer plusieurs conclusions importantes.

Premièrement, en ce qui concerne le nombre de droites détectées, une tendance claire émerge.

En effet, il y a une diminution régulière du nombre de lignes détectées à mesure que la surface des

cellules augmente. Mais à partir d'un certain range, à SR = 168, cette tendance est compromise.

Ainsi l'augmentation de la taille des cellules de la grille n'entraîne pas toujours la diminution des droites détectées. Deuxièmement, le temps de calcul varie également en fonction de la surface des cellules rectangulaires. Bien que les cellules de plus petite surface nécessitent généralement plus de temps de calcul, avec un maximum à 0, 35 secondes pour la cellule de surface 8, le temps de calcul diminue globalement avec l'augmentation de la surface des cellules. La cellule de plus grande surface (l = 9, L = 11) atteint un temps de calcul minimal de 0, 13 secondes, suggérant une amélioration du temps de traitement pour les cellules de grande taille.

La courbe représentative dans la figure 2.6 confirme visuellement ces tendances, montrant une décroissance du nombre de lignes détectées et une diminution du temps de calcul à mesure que la surface des cellules rectangulaires augmente.

Ainsi, les simulations réalisées sur l'image représentant l'immeuble démontrent que l'algorithme de la transformée de Hough rectangulaire est capable de détecter, avec une précision acceptable, les droites discrètes présentes dans l'image de l'immeuble. En ajustant les paramètres appropriés, comme

45

8 15 24 35 48 63 80 99 1681710

20

40

60

80

100

120

140

160

180

200

220

240

260
(263)

(162)

(27) (22)
(5) (2) (2) (0)

P(14)P(14)

SR

D
et

ec
te

d
lin

es
Detected lines

8 15 24 35 48 63 80 99 168171

0.1

0.15

0.2

0.25

0.3

0.35

0.



4

(0.35)

(0.33)

(0.23)

(0.22)

(0.18)(0.18)

(0.16)

(0.13)

(0.10)(0.

10)

SR

T

im

e

Time

Figure 2.6 – Courbe représentative des données du tableau 2.2

la taille des cellules rectangulaires défini par SR, α et le seuil de vote, nous avons pu minimiser le temps de calcul tout en maintenant une précision raisonnable comme illustre la première image la figure 2.5.

3.2 Cas de l'image de l'autoroute

Dans cette section, nous explorons l'application de la transformée de Hough rectangulaire à le cas de la détection de lignes dans l'image d'une autoroute. Nous étudions les performances de la transformée de Hough rectangulaire en ajustant différents paramètres, tels que le pourcentage α de pixels allumés des cellules rectangulaires et la taille des cellules rectangulaires.

3.2.1 Variation de α

Le tableau 2.3 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute en utilisant l'algorithme 4 de la transformée de Hough rectangulaire. Les simulations ont été réalisées avec des paramètres fixes ($l = 3$, $L = 5$ and Seuil = 40) et α variant entre 10% et 50%. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 50%. Ce paramètre définit la taille de la cellule Rectangulaire. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α .

Les résultats de la simulation sur l'image de l'autoroute en utilisant l'algorithme de la transformée de Hough rectangulaire avec des paramètres fixes ($l = 3$, $L = 5$ et Seuil = 40) et une variation de α entre 10% et 50% sont analysés.

En ce qui concerne le nombre de lignes détectées, on observe une tendance à la diminution à mesure que α augmente. À $\alpha = 10\%$, 493 lignes sont détectées, et ce nombre diminue progressivement pour atteindre 0 lignes à $\alpha = 50\%$. Ainsi, des valeurs trop élevées de α peuvent entraîner une détection Nulle.

Tableau 2.3 – Résultats des simulations sur l'image1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres fixes (l= 3, L = 5 and Seuil = 40) et α variant entre 10% et 50% où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

α (%) Droites détectées temps(sec)

10 493 0,21
15 384 0,20
20 384 0,21
25 195 0,16
30 118 0,15
35 25 0,11
40 25 0,11
45 4 0,07
50 0 0,05

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.50

50

100

150

200

250

300

350

400

450

500 (493)

(384) (384)

(195)

(118)

(25) (25)

(4) (0)

α

D
et

ec
te

d
lin

es

Detecte d lignes

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.54 · 10⁻²

6 · 10⁻²

8 · 10⁻²

0.1

0.12

0.14

0.16

0.18

0.2

0.



22 (0.21)
(0.20)

(0.21)

(0.16)
(0.15)

(0.11) (0.11)

(0.07)

(0.
05)

α

T
im

e(
se

c)

Time

Figure 2.7 – Courbe représentative des données du tableau 2.3

Le temps de calcul, quant à lui, montre une décroissance avec l'augmentation de α . À $\alpha = 10\%$, le temps de calcul est de 0,21 seconde, et il diminue progressivement pour atteindre 0,05 seconde à $\alpha = 50\%$. La réduction du temps de calcul est plus marquée après $\alpha = 35\%$, passant de 0,15 à 0,05 secondes.

Le graphique des données du tableau 2.3, représenté dans la figure 2.7, confirme visuellement ces tendances, montrant une décroissance du nombre de lignes détectées et du temps de calcul. Ces résultats suggèrent que des valeurs plus élevées de α conduisent à une détection plus sélective des lignes, tout en améliorant le temps de calcul.

3.2.2 Variation de la taille des cellules rectangulaires

Les résultats de la simulation sur l'image de l'autoroute en utilisant l'algorithme 4 de la transformée de Hough rectangulaire avec une variation de SR sont analysés. SR représente l'aire d'une cellule rectangulaire. Les paramètres fixes incluent un taux α de 30% et un seuil de 40 votes, tandis que les dimensions du rectangle, représentées par l et L, varient respectivement entre 2 et 6 et entre 4 et 8.

47

Tableau 2.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres fixes ($\alpha = 30\%$ et Seuil = 40), $1 \leq L \leq Nl$ et $1 \leq l \leq Nc$ où l et L représentent respectivement la largeur et la longueur de la cellule rectangulaire.

l	L	SR	Droites détectées	temps(sec)
2	4	8	184	0,16
3	5	15	118	0,14
4	6	24	21	0,12
5	7	35	2	0,08
6	8	48	0	0,06

l	L	SR	Droites détectées	temps(sec)
29	84	111	0	0,12

$\alpha = 10$	29	217	10	0,07
29	238	17	0	0,08

Les résultats sont présentés dans le Tableau 2.4 et visualisés dans la Figure 2.8. Le tableau comporte cinq colonnes : La première et la deuxième colonne représentent respectivement les différentes valeurs de la largeur l et la longueur L du rectangle. La troisième colonne représente l'aire SR du rectangle. La quatrième colonne montre le nombre de droites détectées pour chaque valeur de SR. La dernière

colonne présente le temps de traitement en secondes pour chaque simulation.

En ce qui concerne le nombre de lignes détectées, à mesure que l'aire des cellules augmente, il diminue, passant de 184 droites détectées pour une surface de 8 px2 à 0 pour les aires supérieurs ou égales 48 px2. Cependant, en diminuant le pourcentage de pixels allumés α à 10%, nous remarquons que il est possible de détecter, avec une précision acceptable, des droites avec des cellules de grandes taille comme le montre le tableau et la deuxième image de la figure 2.5. Aussi, le temps de calcul diminue avec l'augmentation de l'aire SR des cellules rectangulaires, indiquant une amélioration du temps de traitement avec les grandes cellules rectangulaires. Les représentations graphiques 2.8 confirme ces résultats.

8 15 24 35 480

20

40

60

80

100

120

140

160

180

(184)

(118)

(21)

(2) (0)

SR

D

et

ec

te

d

lin

es

Detected lines

8 15 24 35 484 · 10⁻²

6 · 10⁻²

8 · 10⁻²

0.1

0.12

0.14

0.16

0.18

(0.16)

(0.14)

(0.12)

(0.08)

(0.06)

SR

T
im

e

Time

Figure 2.8 – Courbe représentative des données du tableau 2.4

Ainsi, les résultats obtenus ont démontré la capacité de cette méthode à identifier avec une précision

48

Tableau 2.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble
(temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

Seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
40	9281	0,49	0,00005
116	10	0,49	0,049

Tableau 2.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute
(temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

Seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
51	169	0,24	0,0014
100	10	0,21	0,021

acceptable les lignes de l'immeuble ainsi que celles délimitant l'autoroute. Les données des tableaux 2.2

et 2.4 ainsi que les résultats visuels représentés dans la figure 2.5 suggèrent qu'avec les grandes cellules,

il est possible d'avoir des économies computationnelles tout en gardant une précision acceptable. Avec

la transformée de Hough rectangulaire, il est possible de cibler et détecter les droites en fonction de

leurs position : soit verticale, soit horizontale. Pour les droites verticales, il faut fixer la longueur L de

la cellule rectangulaire à 1 et prendre la largeur l de la cellule rectangulaire dans un voisinage de la

hauteur de l'image. Pour les droites horizontales, il faut fixer la largeur l à 1 et prendre la longueur L

dans un voisinage de la largeur de l'image.

3.3 Comparaison de la Transformée de Hough Rectangulaire et Transformée de

Hough Standard

(a) (b)

Figure 2.9 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

Nous procédons dans cette section à une comparaison de la transformée de Hough rectangulaire,

méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble

et une image d'une autoroute comme illustré dans la figure 1.11.

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard

et la transformée de Hough rectangulaire sont illustré dans les tableaux 2.5 et 2.7 respectivement.

En ce qui concerne la précision, pour un seuil de 40 votes, la THS a effectué une détection de 9281

droites. Cette détection est non pertinente comme le montre la première image de la figure 2.9. par

contre, avec le même seuil de votes, la THR a détecté 11 droites. Les résultats visuels illustrés par la

49

(a) (b)

Figure 2.10 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

deuxième image de la figure 2.11 montre que les droites détectées suivent bien les lignes de l'immeuble

Les paramètres supplémentaires utilisés par la transformée de Hough rectangulaire pour parvenir à ce résultat sont : les dimensions (I; L) et le pourcentage α de pixels allumés de chaque cellule rectangulaire.

Pour ce qui est du temps de traitement, le tableau 2.5 montre que la transformée de Hough standard a détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La transformée de Hough rectangulaire a détecté quasiment le même nombre de droites en 0.15 seconde soit 0.013 seconde par droite détecté comme illustré dans le tableau 2.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté

0,15 = 3,26 (Th représente le temps d'exécution de l'algorithme de notre méthode, tandis que Ts signifie le temps d'exécution de l'algorithme standard).

En ce qui concerne la précision, pour un seuil de 51 votes, la THS a effectué une détection de 169 droites. Cette détection est non pertinente comme le montre la première image de la figure 2.10. par contre, avec le même seuil de votes, la transformée de Hough rectangulaire a détecté 11 droites. Les résultats visuels illustrés par la première image de la figure 2.12 montre que les droites détectées suivent bien les lignes de l'autoroute suggérant des détections pertinentes.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 2.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 2.10 illustre bien ces résultats. Les 10

(1,



Tableau 2.8 – Résultats de simulation avec la THR appliquée sur l'image 1.11(a) de l'autoroute
(time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

(1,



L) SR a seuil number of detected lines time1(sec) time2(sec)
(3,

5) 15 0,4 51 11 0,11 0,01
(5,7) 35 0,25 45 20 0,13 0,0065

droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 2.6 montre que la THS à détecté 10 droites en 0.21

seconde soit 0.021 seconde par droites. La THR a détecté quasiment le même nombre de droite en 0.11

seconde soit 0.01 seconde par droite détecté comme illustré dans le tableau 2.8. Ce qui suggère une

net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul

du facteur d'accélération noté A, où $A = \frac{T_s}{T_h}$

$= 0,21$
 $0,11 = 1,9$ (T_h représente le temps d'exécution de

l'algorithme de notre méthode, tandis que T_s signifie le temps d'exécution de l'algorithme standard).

(a) (b)

Figure 2.11 – Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=40, $\alpha = 40\%$, $L = 5$ et $I = 3$; (b) Seuil=40, $\alpha = 40\%$, $L = 5$ et $I = 3$

3.4 Comparaison de la Transformée de Hough Rectangulaire et sa version pa-

rallélisée

Dans cette section, nous procédons à une analyse comparative des performances de la transformée

de Hough rectangulaire et de sa version parallélisée (THRP) sur deux images tests : une image d'im-

meuble et une image d'autoroute représentées sur la figure 1.11.

3.4.1 Cas de l'image de l'immeuble

La figure 2.13 présente deux graphiques comparant les performances des algorithmes de transformée

de Hough rectangulaire et de transformée de Hough rectangulaire parallèle. Les graphiques comparent

les variations du temps d'exécution en fonction de différents paramètres (α et SR).

51

(a) (b)

Figure 2.12 – Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 4 de la THR avec les paramètres : (a) Seuil=51, $\alpha = 40\%$, $L = 5$ et $I = 3$; (b) Seuil=45, $\alpha = 25\%$, $L = 7$ et $I = 5$

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45
0.15

0.2

0.25

0.3

0.35

0.4

0.45

0.5

0.55

(0.49) (0.



48) (0.49)

(0.39)

(0.33)

(0.20) (0.20)

(0.33) (0.33) (0.33)

(0.26)
(0.24)

(0.20) (0.21)

(0.16)

α

T
im

es

THR
THRP

8 15 24 35 48 63 80 99

0.15

0.2

0.25

0.3

0.35

0.4

(0.35)

(0.33)

(0.23)
(0.22)

(0.18) (0.18)

(0.16)

(0.13)

(0.27)

(0.25)

(0.21)

(0.19)
(0.
18)

(0.19)

(0.17) (0.17)

SR

T
im

e

THR
THRP

Figure 2.13 – Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et seuil = 70), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres (seuil = 50, $\alpha = 30\%$) des l'algorithmes 4 et 8

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 45%. Pour des valeurs de α plus basses (entre 10% et 35%), la courbe de la transformée de Hough rectangulaire parallélisée est en dessous de celle de la transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire parallélisée est plus rapide pour les petites valeurs de α . Pour des valeurs de α plus élevés (entre 35% et 45%), la courbe de la transformée de Hough rectangulaire parallélisée passe au dessus de celle de

la transformée de Hough rectangulaire. Cela suggère que la version parallélisée est moins rapide pour les valeurs élevées de α .

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de la surface des cellules rectangulaires, avec les deux algorithmes de la THR et la THRP. Les surfaces des cellules varient de 8 px2 à 99 px2. La courbe de la transformée de Hough rectangulaire parallélisée est en dessous de celle de la transformée de Hough rectangulaire pour les surfaces inférieure à 48 px2. A 48 px2, la courbe de la transformée de Hough rectangulaire parallélisée passe au dessus de la courbe de la transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire est moins rapide que sa version parallélisée pour les petites valeurs de SR et est plus rapide pour les grandes valeurs de SR.

52

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.54 · 10⁻²

6 · 10⁻²

8 · 10⁻²

0.1

0.12

0.14

0.16

0.18

0.2

0.22

0.24

(0.21)

(0.20)

(0.21)

(0.



16)

(0.15)

(0.11) (0.11)

(0.07)

(0.05)

(0.17)

(0.18)

(0.17)

(0.16)

(0.15) (0.15)

(0.14)

(0.

13)

(0.11)

α

T

im

e

THR

THRP

8 15 24 35 484 · 10⁻²

6 · 10⁻²

8 · 10⁻²

0.1

0.12

0.14

0.16

0.18

(0.



16)

(0.14)

(0.12)

(0.08)

(0.06)

(0.17)

(0.15) (0.15)

(0.

12)

(0.11)

SR

T

im

e

THR

THRP

Figure 2.14 – Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($l = 3$, $L = 5$ et seuil = 40), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules rectangulaires et les paramètres (seuil = 40, $\alpha = 30\%$) des l'algorithmes 4 et 8

3.4.2 Cas de l'image de l'autoroute

La figure 2.14 présente deux graphiques comparant les performances des algorithmes de la THR et la THRP. Les graphiques comparent les variations du temps d'exécution en fonction des paramètres α et SR.

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 50%. Pour les petites valeurs de α (entre 10% et 25%), la courbe de la transformée de Hough rectangulaire parallélisée est en dessous de celle de transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire parallélisée est plus rapide pour ces valeurs de α . Pour des valeurs de α plus élevés (entre 30% et 50%), la courbe de la



transformée de Hough rectangulaire parallélisée passe au dessus de celle de la transformée de Hough rectangulaire. Cela suggère que la transformée de Hough rectangulaire parallélisée est moins rapide pour des valeurs de α supérieur à 30%.

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de la

surface des cellules rectangulaires, avec les deux algorithmes de la transformée de Hough rectangulaire et sa version parallélisée. Les surfaces des cellules varient de 8 à 48 px2. La courbe de transformée de Hough rectangulaire parallélisée est au dessus de celle de la transformée de Hough rectangulaire pour toutes les valeurs de SR. Cela suggère que la transformée de Hough rectangulaire parallélisée est moins rapide que la transformée de Hough rectangulaire dans ce cas.

α (%) (L,I) seuil Droites Temps (sec)
20 (3,6) 30 5 0,10

Tableau 2.9 – Approche proposée

53

3.5 Comparaison avec d'autres approches

Dans cette section, nous comparons notre approche de détection de lignes à celle proposée par Traoré et al [4]. Cette comparaison vise à évaluer les performances relatives des deux méthodes en termes de précision dans la détection des lignes d'une part, et de leur performance temporelle d'autre part. Nous examinerons en détail les résultats visuels obtenus ainsi que les temps d'exécution pour éclairer les forces et les faiblesses de chaque approche. Cette analyse comparative permettra de déterminer les avantages et les limitations de chaque méthode, fournissant ainsi des indications précieuses pour le choix de l'approche la plus appropriée en fonction des besoins spécifiques de l'application envisagée.

Dans l'approche de la transformée de Hough rectangulaire proposée dans ce document, les dimensions des cellules rectangulaires sont paramétrées contrairement à l'approche proposée par Traoré et al [4]. Ce qui donne une plus grande flexibilité dans l'ajustement des paramètres.

3.5.1 Précision

les résultats visuels de la simulation de l'algorithme de l'approche de Traoré et al [4] dans la figure 2.15 montre une détection acceptable de deux côtés du pentagone et une détection non-parfaite des trois autres côtés avec les paramètres mentionnés dans le tableau 2.10. La figure 2.16 illustre la détection des lignes délimitant l'autoroute avec l'approche de Amed [4]. Trois de ces lignes ont été bien détectées. Les quatre autres détectations ont été défectueuse. La figure 2.17 illustre la détection des lignes de l'immeuble avec l'approche de Traoré et al [4]. Aucune ligne de l'immeuble n'a été détecté.

Figure 2.15 – Lignes droites détectées sur le pentagone avec l'approche de Traoré et al [4]

Figure 2.16 – Lignes droites détectées sur l'autoroute avec l'approche de Traoré et al [4]

Figure 2.17 – Lignes droites détectées sur l'immeuble avec l'approche de Traoré et al [4]

La première de la figure 2.18 illustre les résultats visuels de simulations de notre algorithme sur l'image du pentagone en utilisant les paramètres mentionnés dans le tableau 2.9. Elle montre une

54

détection acceptable de quatre côtés du pentagone et un côté non détecté. La deuxième image de la figure 2.18 montre une détection acceptable de cinq lignes délimitant l'autoroute et la troisième image de la même figure montre une détection claire de lignes horizontale et verticale.

(a) (b) (c)

Figure 2.18 – Lignes droites détectées avec notre méthode

α (%) seuil Droites Temps (sec)
10 80 5 0,17

Tableau 2.10 – Approche de Traoré et al [4]

Ainsi, les résultats obtenus montrent que notre méthode présente des performances supérieures

dans plusieurs aspects par rapport à l'approche de Traoré et al.

Dans le cas de la détection des côtés du pentagone, notre méthode parvient à détecter quatre côtés

sur cinq de manière acceptable, tandis que l'approche de Traoré et al ne parvient qu'à une détection

acceptable de deux côtés. De même, pour la détection des lignes délimitant l'autoroute, notre méthode

réussit à détecter cinq lignes sur sept de manière satisfaisante, alors que l'approche de Traoré et al

n'en détecte que trois correctement. En outre, alors que l'approche de Traoré et al ne parvient pas à

détecter les lignes de l'immeuble, notre méthode parvient à identifier les lignes de l'immeuble.

3.5.2 Performance temporelle

En ce qui concerne la performance temporelle, les résultats obtenus montrent des différences entre

l'approche de Traoré et al et la méthode proposée dans ce chapitre. L'approche de Traoré et al

nécessite un temps d'exécution moyen de 0,17 secondes pour détecter cinq lignes avec un seuil de 80 et

un paramètre α de 10%, tandis que notre méthode atteint une meilleure performance avec un temps

d'exécution moyen de seulement 0,10 secondes pour détecter cinq lignes, utilisant un seuil de 30 et

un paramètre α de 20%, avec des dimensions (L,l) fixées à (3,6). Ces résultats mettent en évidence

l'amélioration du temps de traitement de la méthode proposée.

Conclusion

L'étude de la technique de la transformée de Hough rectangulaire a révélé ses capacités promet-

teuses dans la détection de lignes droites dans les images. Nous avons décrit en détail la méthode de

fonctionnement de la transformée de Hough rectangulaire, mettant en lumière ses étapes clés et ses pa-

55

ramètres ajustables, ainsi que ses avantages par rapport à d'autres approches telles que la transformée

de Hough standard.

Les simulations ont démontré la capacité de la transformée de Hough rectangulaire à détecter avec

une précision acceptable les lignes droites, tout en offrant une flexibilité supplémentaire grâce à la pos-

sibilité d'ajuster les paramètres de maillage et la réduction du temps de calcul. Aussi, avec la réduction

du nombre de pixels à calculer grâce à la variable α , la transformée de Hough rectangulaire offre des

économies computationnelles importantes. aussi, avec la transformée de Hough rectangulaire, il est

possible de cibler et détecter les droites en fonction de leurs position : soit verticale, soit horizontale.

De plus, nos comparaisons avec l'approche proposée par Traoré et al [4] ont mis en évidence les

performances supérieures de la transformée de Hough rectangulaire en termes de précision de détection

et de temps d'exécution. Aussi, le choix entre transformée de Hough rectangulaire et transformée de

Hough rectangulaire parallélisée dépend des valeurs spécifiques de α et de SR dans un contexte donné.

Globalement, THRP semble être plus performante pour les petites valeurs de α et de SR, tandis

que transformée de Hough rectangulaire pourrait être préférable pour les valeurs plus élevées de ces

paramètres. Néanmoins, même si elle présente des avantages, la transformée de Hough rectangulaire

comporte des limitations. Celles-ci incluent la perte d'informations provenant de l'ensemble complet

des pixels allumés des cellules rectangulaires, la sensibilité aux taux α de pixels allumés et à la taille

des cellules rectangulaires, la complexité de sa mise en œuvre, sa dépendance au contexte ainsi que

la sensibilité aux paramètres. Dans le chapitre suivant, nous aborderons la transformée de Hough

triangulaire.

Introduction 57

1 Description de la méthode



..... 58

2 Complexité de latransformée de Hough triangulaire 65

3 Simulations et discussions
..... 66

4 Analyse comparative 72

Conclusion 81

Introduction

La détection de lignes droites dans les images constitue une étape cruciale dans de nombreuses applications de vision par ordinateur. Parmi les nombreuses méthodes développées à cet effet, la transformée de Hough (TH) est largement reconnue pour sa robustesse et sa capacité à détecter des structures linéaires dans des images bruitées. Cependant, son utilisation standard comporte des limites, notamment en termes de performance temporelle.

Dans ce chapitre, nous introduisons une nouvelle approche de la transformée de Hough standard (THS) dénommée transformée de Hough triangulaire (THT). La transformée de Hough triangulaire repose sur le concept de grille virtuelle, où des cellules triangulaires virtuelles sont générées sur l'image. Seules les cellules ayant un certains taux de pixel allumés sont considéré dans les calculs.

Dans ce chapitre, nous présentons en détail la méthode de la transformée de Hough triangulaire, en commençant par la description du maillage triangulaire de l'image et de la sélection des cellules triangulaires. Nous discutons ensuite des techniques de reconnaissance des droites discrètes utilisées dans la transformée de Hough triangulaire et de son application pratique dans la détection de lignes droites dans les images. Aussi, nous nous penchons sur la comparaison des performances entre la

57

transformée de Hough triangulaire et sa version parallélisée (THTP).

Nous évaluons les performances de la transformée de Hough triangulaire à travers des simulations sur des images, en comparant ses résultats avec ceux de la transformée de Hough standard. Enfin, nous discutons des avantages et des limitations de la transformée de Hough triangulaire, ainsi que de ses implications potentielles pour les applications de vision par ordinateur.

1 Description de la méthode

La description de la méthode présentée dans cette section vise à détailler le processus de maillage triangulaire de l'image ainsi que les techniques de reconnaissance des droites discrètes utilisant la transformée de Hough triangulaire. Les étapes de la méthode sont illustrées dans la figure 3.4. Le maillage triangulaire, fondé sur le théorème de Thalès et accompagné d'algorithmes spécifiques, permet de subdiviser l'espace image en une grille virtuelle de triangles. Cette subdivision sert de base à la

détection de droites discrètes dans l'image.

Par la suite, des simulations pour la détection des droites discrètes sont effectué à l'aide de la transformée de Hough triangulaire, où chaque pixel allumé du triangle de la grille est utilisé comme élément de vote dans l'accumulateur. Les détails des algorithmes utilisés pour parcourir la grille, compter les votes et détecter les droites sont discutés en détail, offrant ainsi un aperçu complet du processus de reconnaissance.

1.1 Maillage triangulaire de l'image

Le maillage triangulaire de l'image consiste à générer une grille triangulaire virtuelle sur l'espace image. La grille est constituée de cellules triangulaires. Le théorème de Thalès est utilisé ici pour calculer le dual de chaque cellule. Nous rappelons ce théorème comme suit :

Theoreme 1 (Thales) Soit (ABC) un triangle rectangle. Soient D et E deux points sur les droites (AC) et (BC) respectivement, de manière à avoir la droite (DE) parallèle à la droite (AB). Alors :
CD
CA = DE

AB = CE
CB

Cela signifie que le théorème de Thalès met en évidence des relations de proportionnalité entre des droites parallèles. Par exemple, avec la relation
(

$$\frac{CD}{CA} = \frac{DE}{AB}$$

)

du théorème de Thalès, nous obtenons

$$DE = \frac{CD \times AB}{CA}$$

donc DE = CD×AB
CA , comme illustré dans la figure 3.1.

Dans la figure 3.1, les segments [AB], [AC] sont respectivement la base et la hauteur du triangle (ABC) : la base est liée à l'axe (y), tandis que la hauteur est associée à l'axe (x).

58

y

x

A

C

B

D E

F

H

G

KL

Figure 3.1 – Deux triangles rectangles (ABC) et (FGH) avec respectivement des droites parallèles (AB) et (DE), et (FG) et (KL).

La hauteur et la base des triangles rectangles sont passées en paramètres dans l'algorithme 10.

Une illustration de deux triangles rectangles est présentée dans la figure 3.1.

Algorithme 10 : Construction de la grille triangulaire
Fonction maillage tri(Nl, Nc,h, b : entiers) : tableau d'entiers

Variables : tab : tableau de 6 entiers
Output : tab;

```

begin
tab[1] ← b ; % la base du triangle
tab[3] ← h ; % la hauteur du triangle
tab[0] ← int[Nl/h] ; % nombre de partis de longueur tab[3] sur la hauteur de l'image
tab[2] ← int[Nc/b] ; % nombre de partis de longueur tab[1] sur la largeur de l'image
tab[4] ← Nl mod h ; % le reste dans la division euclidienne de Nl par h
tab[5] ← Nc mod b ; % le reste dans la division euclidienne de Nc par b
return tab;

fin

y

x

```

Figure 3.2 – Une grille triangulaire superposée à l'espace image

59

Algorithme 11 : Mise à jour de l'accumulateur
Procédure Acc update() : vide

```

% Mise à jour de l'accumulateur
if img[z, t] = 0 then

for itheta dans range(accum row) do
% Calcul des coordonnées polaires du pixel "img[z,

```



```

t]".
theta ← itheta × dtheta;
rho ← t × cos(theta) + z × sin(theta);

```

```

irho ← int(rho);
if irho > 0 et irho < accum column then

```

```

accum[itheta][irho] ← accum[itheta][irho] + 1
end

```

```

end
end

```

```

fin

```

Après la création de la grille triangulaire, présenté dans la figure 3.2, le calcul du dual du triangle

dépend du nombre de pixels allumés dans chaque triangle.

1.2 Parallélisation de la transformée de Hough triangulaire

La parallélisation de la transformée de Hough triangulaire nécessite une adaptation de l'algorithme afin de tirer pleinement parti des capacités de traitement simultané offertes par le parallélisme. Cette parallélisation concerne principalement l'accumulateur, également appelé l'espace des paramètres ou l'espace de Hough. Les algorithmes 15 et 14 sont les versions parallélisées des algorithmes 13 et 11.

Les principales étapes de cette parallélisation sont les suivantes :

- importation de la bibliothèque pypm 3 : importer une bibliothèque en Python est nécessaire pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction `import pypm` est utilisée pour charger la bibliothèque pypm.
- création de l'accumulateur de type pypm : dans l'algorithme séquentiel, l'espace des paramètres (l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la transformée de Hough, est transformé en une structure de données partagée grâce à pypm. En effet, pypm permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.



L'instruction `"accum pypm ← pypm.shared.array([accum row, accum column])"` de l'algorithme 15

crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable

"accum pypm".

- construction parallèle : l'instruction "with pypm.



Parallel(4) as p" de l'algorithme 15 indique qu
e

nous souhaitons exécuter le bloc de code à l'intérieur de la déclaration "with" en parallèle avec 4

processus.



3. <https://github.com/classner/pypm/tree/main>

60

Algorithme 12 :

Taux de pixels allumés

Fonction count tri(img : image, A, B, C : 3 tableaux de deux entiers) : réel

Variables : k, l : integers;

D : table of two integers

begin

k ← 0 ; % Initialisation du nombre de pixels allumés

l ← 0 ; % Initialisation du nombre de pixels éteints

if B[1]≠A[1] and C[0]≠A[0] then

for x allant de A[0] à C[0] do

DE ←

|x- C[0]| × |A[1]- B[1]|

|A[0]- C[0]|

;

y0 ← A[1] + round(DE);

for y allant de A[1] à y0 do

if img[x, y] = 0 then

k ← k + 1 ;

else

l ← l + 1 ;

end

end

end

end

if B[1]≠A[1] and C[0]≠A[0] then

for x allant de C[0] à A[0] do

DE ←

|x- C[0]| × |A[1]- B[1]|

|A[0]- C[0]|

;

y0 ← A[1]- E[DE];

%E[DE] est la partie entière de DE for y allant de y0 à A[1] do

if img[x, y] = 0 then

k ← k + 1 ;

else

l ← l + 1 ;

end

end

end

end

return

k

k + l
;

fin

- parallélisation de l'algorithme 11 de mise à jours de l'accumulateur : la fonction "p.range" génère une séquence d'indices à partir de laquelle chaque cœur du CPU obtiendra une plage d'indices spécifique.

Cela permet de diviser le travail entre les cœurs de manière équitable. Ainsi la boucle "for itheta

in p.range(accum row)" de l'algorithme 14 utilise les indices compris entre 0 et accum row- 1 pour

itérer à travers l'accumulateur par sections, où chaque section est traitée par un processus différent.

1.3 Sélection des cellules triangulaires

La sélection d'une cellule triangulaire dépend de son taux de pixel allumé. L'algorithme 12 calcule le

taux de pixels allumés dans une région triangulaire définie par trois points spécifiés par les tableaux A,

B, et C. L'image d'entrée est représentée par img. L'algorithme utilise deux boucles pour parcourir

les pixels de la région délimitée par les points du triangle. Selon la configuration du triangle (si la

coordonnée y de B est supérieure ou égale à celle de A, et la coordonnée x de C est supérieure ou

égale à celle de A), l'algorithme calcule la valeur de DE et détermine la portée de l'itération y. Ensuite,

il compte le nombre de pixels allumés (k) et éteints (l) dans la région définie par le triangle. Le résultat

renvoyé est le taux de pixels allumés dans cette région, calculé comme le rapport de k sur la somme

de k et l.

Le parcours des pixels du triangle se fait à l'aide du théorème de Thalès avec l'instruction suivante :



$DE \leftarrow |x - C[0]| \times |A[1] - B[1]|$
 $|A[0] - C[0]| \cdot (3.$
1)

Il convient de noter que, pour une cellule particulière de la grille, tous les pixels ne sont pas pris

61

Algorithme 13 : Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire

Fonction Triangular(img,h,b,a,seuil) :Tableau

Pre-condition : $1 \leq h \leq NI$ et $1 \leq b \leq Nc$, $0 \leq a \leq 1$, $0 < \text{seuil}$

Variables : tab : tableau de 6 entiers ; A, B, C : tableau de 2 entiers ; accum row, accum column, irho, NI, Nc : entiers;

accum : matrice de dimensions "accum row $\times$ accum column" ; dtheta, rho, theta, DE : réels;

Begin

NI $\leftarrow$ nombre de ligne de l'image ; Nc $\leftarrow$ nombre de colonne de l'image ; % the dimensions of the image ;

accum row $\leftarrow$ 180 ; accum column $\leftarrow$ E(
 $\sqrt{$

$(Nc^2 + NI^2)$); % les dimensions de la matrice "accum";

dtheta = $\pi/180$; tab $\leftarrow$ maillage tri(NI, Nc) ;

H = NI - tab[4] ; L = Nc - tab[5] ;

% parcours des triangles de la grille sans tenir compte des résidus

for x allant de tab[3] à NI par pas de tab[3]+1 do

for y allant de tab[1] à Nc par pas de tab[1]+1 do

A[0] = x - tab[3] ; A[1] = y - tab[1] ; B[0] = x - tab[3] ; B[1] = y ; C[0] = x ; C[1] = y - tab[1] ; D = A;

if B[1] $\neq$ A[1] et C[0] $\neq$ A[0] then

if count(img, A,B, C) $\geq$ a then

for z allant de A[0] à C[0] do

DE $\leftarrow$

$|x - C[0]| \times |A[1] - B[1]|$

$|A[0] - C[0]|$

; y0 $\leftarrow$ A[1] + round(DE);

for t allant de A[1] à y0 do

```
Acc update();

end
end

end
end
A[0] = x ; A[1] = y ; B[0] = x- tab[3] ; B[1] = y ; C[0] = x ; C[1] = y - tab[1] ; D = A;
if B[1]i=A[1] et C[0]i=A[0] then

if count(img, A,B, C)≥ α then
for z allant de C[0] à A[0] do

DE ←
|x- C[0]| × |A[1]- B[1]|

|A[0]- C[0]|
; y0 ← A[1]- E[DE];

for t allant de y0 à A[1] do
Acc update();

end
end

end
end

end
end
% traitement des résidus if tab[4] = 0 then

for z allant de H et NI do
for t allant de 0 et Nc do

Acc update();
end

end
end
if tab[5] = 0 then

for z allant de 0 and NI do
for t allant de L and Nc do

Acc update();
end

end
end
Rechercher les maxima dans l'accumulateur et placer les couples (p, θ) dans une liste 'lignes';
Retourner la liste 'lignes';

fin
fin

y

x

A

C

B

D

(a)

y

x

A

C

B

D

(b)

y
```

x

A

C

B

D

(c)

y

x

A

C

B

D

(d)

Figure 3.3 – Sélection des pixels d'une cellule triangulaire

en compte lors du calcul du taux. Les pixels exclus, en revanche, seront considérés dans les cellules voisines, comme illustré dans la figure 3.3. Cette exclusion vise à garantir qu'un même pixel ne soit pas pris en compte deux fois dans deux triangles distincts. L'aire d'une cellule triangulaire est calculé selon le lemme 1.

Lemme 1 (L'aire d'une cellule triangulaire) Soit b la base et h la hauteur du triangle. L'aire du

62

Algorithme 14 : Mise à jour parallèle de l'accumulateur

Procédure Acc updateP() : vide

% Mise à jour de l'accumulateur

if img[z, t] = 0 then

for itheta dans p.



range(accum row) do

theta ← itheta × dtheta;

rho ← tx cos(theta) + z × sin(theta);

irho ← int(rho);

if irho > 0 et irho < accum column then

accum pyp[itheta][irho] ← accum pyp[itheta][irho] + 1

end

end

end

fin

triangle,

représentée par la partie verte de la figure 3.3(b), est calculée à l'aide de la formule suivante :

$ST(b, h) = b(h-1) + 2$

2 (3.2)

Preuve 1 (lemme 1) Considérons b la base et h la hauteur du triangle.

La formule d'un triangle ordinaire est donnée par $b \times h$

2 . L'espace image est discret et les cellules

triangulaires ont leur diagonale commune deux à deux. Ainsi, les pixels qui sont sur une diagonale

commune à deux cellules seront exclu du calcul de l'aire du triangle pour n'est pas que des cellules

soit pris en compte deux fois dans les calculs. Par conséquent, la surface du triangle dans ce cas est $b(h-1)$

$2 + 1$. Ce qui donne $ST(b, h) = b(h-1) + 2$

2 .

Remarque 3 Étant donné que l'espace image est discrète, toute valeur de $ST(b, h)$ non entier doit

être arrondi à l'entier qui lui est immédiatement supérieur.

Par exemple, pour $b = 7$ et $h = 13$, $ST(7, 13) = 7(13-1)+2$
 $2 = 43$ px², comme illustré dans la figure

3.3. L'unité de mesure de la surface est en pixels carrés (px²). Aussi pour $b = 5$ et $h = 8$, $ST(5, 8) =$
 $5(8-1)+2$

$2 = 18$, 5px². d'après la remarque 3, 18, 5 est arrondi à 19. Par conséquent on écrit $ST(5, 8) = 19$.

1.4 Reconnaissance des droites discrètes

L'algorithme 13 de reconnaissance de droites discrètes par la transformée de Hough triangulaire vise

à détecter des droites discrètes dans une image en utilisant une variante triangulaire de la Transformée

de Hough. Les paramètres de l'algorithme incluent l'image d'entrée (img), les dimensions de la grille

triangulaire (h, b), un pourcentage α de pixels allumés dans chaque cellule de la grille, et un seuil global

(Seuil). L'algorithme parcourt une grille triangulaire générée par la fonction maillage tri(Algorithme

10) et explore les triangles formés en utilisant des coordonnées spécifiques (A, B, C). Pour chaque

triangle, les pixels de l'image sont examinés en fonction de certaines conditions et un comptage est

63

Algorithme 15 : Reconnaissance de droites discrètes par la Transformée de Hough Triangulaire Parallélisée

Fonction TriangularP(img,h,b, α ,seuil,cpu count) :Tableau

Pre-condition : $1 \leq h \leq NI$ et $1 \leq b \leq Nc$, $0 \leq \alpha \leq 1$, $0 < \text{seuil}$

Variables : tab : tableau de 6 entiers ; A, B, C : tableau de 2 entiers ; accum row, accum column, irho, NI, Nc : entiers;

accum : matrice de dimensions "accum row $\times$ accum column" ; dtheta, rho, theta, DE : réels;

Begin

import pypm ;% importation de la bibliothèque pypm

NI ← nombre de ligne de l'image ; Nc ← nombre de colonne de l'image ;% the dimensions of the image ;

accum row ← 180 ; accum column ← E(
 $\sqrt{$

$(Nc^2 + NI^2)$) ;% les dimensions de la matrice "accum";



accum pypm ← pypm.shared.array([accum row,accum column]) % création de l'accumulateur de type pypm;

dtheta = $\pi/180$; tab ← maillage tri(NI,

Nc) ;

H = NI - tab[4] ; L = Nc - tab[5] ;

% parcours des triangles de la grille sans tenir compte des résidus

% Démarrage du traitement parallèle

With pypm.parallel(cpu count) as p :

for x allant de tab[3] à NI par pas de tab[3]+1 do

for y allant de tab[1] à Nc par pas de tab[1]+1 do

A[0] = x - tab[3] ; A[1] = y - tab[1] ; B[0] = x - tab[3] ; B[1] = y ; C[0] = x ; C[1] = y - tab[1] ; D = A;

if B[1]_i=A[1] et C[0]_i=A[0] then

if count(img, A,B, C) $\geq \alpha$ then

for z allant de A[0] à C[0] do

DE ←

$|x - C[0]| \times |A[1] - B[1]|$

$|A[0] - C[0]|$

; y0 ← A[1] + round(DE);

for t allant de A[1] à y0 do

Acc update();

end

end

end

end

A[0] = x ; A[1] = y ; B[0] = x - tab[3] ; B[1] = y ; C[0] = x ; C[1] = y - tab[1] ; D = A;

if B[1]_i=A[1] et C[0]_i=A[0] then

```

if count(img, A,B, C)≥ α then
for z allant de C[0] à A[0] do

    DE ←
    |x- C[0]| × |A[1]- B[1]|

    |A[0]- C[0]|
; y0 ← A[1]- E[DE];

for t allant de y0 à A[1] do
Acc update();

end
end

end
end

end
end
% traitement des résidus if tab[4] = 0 then

for z allant de H et NI do
for t allant de 0 et Nc do

Acc update();
end

end
end
if tab[5] = 0 then

for z allant de 0 à NI do
for t allant de L à Nc do

Acc update();
end

end
end

fin
Rechercher les maxima dans l'accumulateur et placer les couples (p, θ) dans une liste 'lignes';
Retourner la liste 'lignes';

fin
fin

```

effectué. La mise à jour de l'accumulateur se fait en fonction de ces pixels. Les triangles de la grille sont parcourus sans tenir compte des résidus, et ensuite, les résidus sont traités séparément. Finalement, les maxima de l'accumulateur sont recherchés, les couples (θ, p) correspondants sont ajoutés à une liste 'lignes', qui est ensuite renvoyée.

L'algorithme 16 génère une représentation visuelle des droites détectées par la méthode de la Transformée de Hough. Il reçoit en entrée une image (img), une liste de paramètres de droites détectées (lignes) générée par l'algorithme 13, et une couleur (colors). L'algorithme parcourt la liste des paramètres de droites détectées et reconstruit chaque droite correspondante sur l'image. Les droites reconstruites sont colorées avec la couleur colors. Cette reconstruction permet de visualiser les droites détectées sur l'image d'origine, facilitant ainsi l'interprétation et l'analyse des résultats de la Trans-

64

formée de Hough.

Figure 3.4 – Les étapes de la technique de la transformée de Hough triangulaire

Algorithme 16 :



Reconstruction des droites détectées
Fonction PlotHoughLine(img,lignes,colors) :image



Parcourir 'lignes' la liste des paramètres des droites détectés, puis construire chaque droite correspondant en couleur 'colors' sur l'image 'img'.

fin

2 Complexité de la transformée de Hough triangulaire

La fonction maillage tri dans l'algorithme 10 initialise un tableau et lui assigne des valeurs, puis retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

65

La fonction count tri dans l'algorithme 12 : initialise des variables k, l et D, puis effectue des tests de conditions pour déterminer l'orientation du triangle. Toutes ces opérations sont de coût constant, donc $O(1)$. Ensuite, elle parcourt les pixels à l'intérieur du triangle à l'aide de boucles dont le nombre d'itérations est $b(h-1)+2$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité d'un bloc formé par les boucles est $O(b(h-1)+2)$ = $O(b \times h)$, où $b(h-1)+2$

2 est le nombre de

pixels à l'intérieur du triangle. Ainsi, la complexité totale de la fonction count tri est $O(b \times h)$.

La fonction Acc update dans l'algorithme 11 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de accum row = 180, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction Acc update est $O(1)$.

L'algorithme 13 de la transformée de Hough triangulaire commence par l'initialisation des variables et du tableau tab, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction maillage tri dont la complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux boucles imbriquées parcourant les points à l'intérieur de chaque triangle à l'intérieur du quelle se trouve des opérations de coût constant et la fonction Acc update de complexité $O(1)$. La complexité de ces boucles est donc $O(b(h-1)+2)$ = $O(b \times h)$, où $b(h-1)+2$

2 est le nombre de pixels à

l'intérieur du triangle. Le coût des deux boucles principales imbriquées parcourant les triangles de la grille, dont le nombre total d'itérations dépend de la taille de l'image et de la taille des cellules triangulaires, contenant un test de condition de coût constant, un deuxième test de condition sur

la fonction count tri2 de complexité $O(b \times h)$, deux boucles imbriquées de coût $b(h-1)+2$ parcourant

les pixels à l'intérieur du triangle, est $NI/h \times Nc/b \times (1 + 1 + 1 + (b(h-1)+2))$, donc de complexité

$O(NI/h \times Nc/b \times (1 + 1 + 1 + (b(h-1)+2)))$ = $O(NI \times Nc)$. Aussi la complexité des deux boucles

parcourant les résidus dans les dimensions de l'image est $O(Nc)$ et $O(NI)$ respectivement.

En considérant ces points, la complexité totale de l'algorithme 13 de la THT est égal à $O(NI \times$

$Nc) + O(Nc) + O(NI) = O(NI \times Nc)$, où NI et Nc sont le nombre de lignes et de colonnes de l'image

respectivement, h et b sont les dimensions des cellules triangulaire. Ce qui correspond à la complexité

3 Simulations et discussions

Cette section offre une analyse détaillée des résultats obtenus à partir des simulations menées sur les algorithmes de la transformée de Hough triangulaire appliqués à deux images : l'image d'un bâtiment et l'image d'une autoroute représenté dans la figure 1.11. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

66

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Ces simulations ont été réalisées en variant les paramètres clés tels que le seuil de vote, la taille ST des cellules triangulaires, et le paramètre α . Les résultats obtenus ont été examinés afin de déterminer l'impact de ces paramètres sur la performance de l'algorithme en termes de détection de lignes droites. De plus, une comparaison entre la transformée de Hough triangulaire et sa version parallélisée (THTP) a été effectuée pour évaluer l'algorithme parallèle. Cette section offre ainsi une compréhension des performances et des limitations de la transformée de Hough triangulaire dans les contextes de l'image de l'immeuble et de l'autoroute, ainsi que des perspectives pour d'éventuelles améliorations.

3.1 Cas de l'image de l'immeuble

Dans cette section, nous explorons l'application de la transformée de Hough triangulaire à le cas de la détection de lignes dans l'image d'un immeuble. Nous étudions les performances de la transformée de Hough triangulaire en ajustant différents paramètres, tels que le pourcentage α de pixels allumés et la taille des cellules rectangulaires.

3.1.0.1 Variation de α

Le tableau 3.1 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble en utilisant l'algorithme 13 de la transformée de Hough triangulaire. Les simulations ont été réalisées avec les paramètres fixes ($h = 5$, $b = 3$ et Seuil = 60) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 55%. Ce paramètre définit la taille de la cellule triangulaire. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse des résultats de simulation sur l'image de l'immeuble, en utilisant l'algorithme de la transformée de Hough triangulaire, met en lumière l'influence du paramètre α sur la performance de l'algorithme :

- Tout d'abord, en examinant le nombre de lignes détectées, on constate qu'il diminue à mesure que α augmente. En effet, de 388 droites détectées avec $\alpha = 10\%$, ce nombre a diminué jusqu'à 0 avec $\alpha = 55\%$. Des valeurs trop élevées de α peuvent entraîner une détection nulle.
- En ce qui concerne le temps de traitement, il y a une amélioration. Le temps de calcul reste relativement constant pour α compris entre 0,1 et 0,45, puis diminue à $\alpha = 50\%$ et $\alpha = 55\%$.

67

Tableau 3.1 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 13 de la

THT avec les paramètres fixes (h = 5, b = 3 et Seuil = 60) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

α (%) Nbre de droites détectées temps(sec)

10 388 0,49
15 154 0,44
20 154 0, 44
25 154 0,



43
30 28 0,33
35 28 0,34
40 9 0.27
45 4 0,21
50 4 0,21
55 0 0,
16

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.550

50

100

150

200

250

300

350

400 (388)

(154) (154) (154)

(28) (28)
(9) (4) (4) (0)

Rate

de
te

ct
ed

lin
es

detected lines

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.55
0.15

0.2

0.25

0.3

0.35

0.4

0.45

0.5

0.55

(0.



49)

(0.44) (0.44) (0.43)

(0.33) (0.34)

(0.27)

(0.21) (0.21)

(0.16)

Rate

T
im

e
(s

ec
on

de
)

Time

Figure 3.5 – Représentation graphique des données du tableau 3.1

La représentation graphique des données du tableau représenté dans la figure 3.5 confirme ces observations, montrant la décroissance du nombre de lignes détectées à mesure que α augmente, et une baisse du temps de calcul pour les valeurs de α plus élevées.

3.1.0.2 Variation de la taille des cellules triangulaires

Le tableau 3.2 et les graphiques associés, illustrés dans la Figure 3.7, présentent les résultats de simulations utilisant l’Algorithme 13 de la transformée de Hough triangulaire sur l’image 1.11(c) de l’immeuble. L’algorithme est appliqué avec des paramètres fixes ($\alpha = 20\%$ et Seuil = 80) tout en faisant varier les paramètres h et b , représentant respectivement la hauteur et la base de la cellule triangulaire.

Dans l’analyse du tableau 3.2, on observe que les valeurs de h et b augmentent progressivement, entraînant une augmentation correspondante de l’aire ST des cellules triangulaires. Cette croissance est attendue, car h et b définissent la taille de la cellule triangulaire. A partir de ST = 4 jusqu’à ST = 29, le nombre de lignes détectées diminue à mesure que la taille de la cellule augmente. Pour 68

Figure 3.6 – (a) THT sur l’immeuble ($h = 219$, $b = 178$, $\alpha = 48$, seuil = 61), (b) THT sur l’autoroute ($h = 92$, $b = 300$, $\alpha = 10$, seuil = 60)

Tableau 3.2 – Résultats des simulations sur l’image 1.11(c) de l’immeuble avec l’algorithme 13 de la THT avec les paramètres fixes ($\alpha = 20\%$ et Seuil = 80), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$, où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

h b ST (h, b) Nbre de droites détectées temps(sec)
2 4 4 221 0,



59
3 5 7 105 0,53
4 6 11 85 0,52
5 7 16 49 0,48
6 8 22 39 0,
41
7 9 29 29 0,40
131 117 7606 59 0,36

h b ST (h, b) Nbre de droites détectées temps(sec)
2 163 800 42 0,28

seuil = 61, $\alpha = 48$ 2 187 95 25 0,25
219 178 19403 38 0,26

ST = 7606, 59 droites ont été détecté contre 29 pour ST = 29 l’aire d’une plus petite cellule. Ainsi, de façon générale, le nombre de droites détectées n’a pas de tendance spécifique par rapport à la taille de

cellules. En ce qui concerne le temps de traitement, il diminue lorsque la taille des cellules augmente.

3.2 Cas de l'image de l'autoroute

Dans cette section, nous examinons les performances de l'algorithme de détection de lignes de la transformée de Hough triangulaire appliqué à une image représentant une autoroute. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir h , b et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la transformée de Hough triangulaire en fonction des caractéristiques de l'image de l'autoroute.

3.2.0.1 Variation de α Le tableau 3.3 présente les résultats de simulations utilisant l'algorithme 13 de la transformée de Hough triangulaire sur l'image 1.11(a) de l'autoroute. Les paramètres de l'algorithme sont fixés à $h = 4$, $b = 2$, et Seuil = 60, tandis que le paramètre α varie entre 10%

69

4 7 11 16 22 29

40

60

80

100

120

140

160

180

200

220

240
(221)

(105)

(85)

(49)
(39)

(29)

ST

de
te

ct
ed

lin
es

detected lines

4 7 11 16 22 290.35

0.4

0.45

0.5

0.55

0.6

0.65

(0.59)

(0.53)
(0.52)

(0.48)

(0.41)
(0.40)

ST

T
im

e
(s

ec
on

de
)

Time

Figure 3.7 – Représentation graphique des données du tableau 3.2

Tableau 3.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres fixes ($h = 4$, $b = 2$ et $\text{Seuil} = 60$) et α variant entre 10% et 55% où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

$\alpha(\%)$ Droites détectées temps(sec)
10 80 0,



24
15 80 0.24
20 80 0.24
25 51 0.22
30 18 0,19
35 18 0.19
40 18 0,19
45 8 0.16
50 8 0,16
55 2 0.
12

70

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.55

10

20

30

40

50

60

70

80

90

(80) (80) (80)

(51)

(18) (18) (18)

(8) (8)
(2)
α
de
te
ct
ed
lin
es
detected lines
0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.55
0.12
0.14
0.16
0.18
0.2
0.22
0.24
(0.24) (0.

24) (0.24)
(0.22)
(0.19) (0.19) (0.19)
(0.16) (0.16)
(0.
12)
α
T
im
e
(s
ec
on
de
)
Time

Figure 3.8 – Représentation graphique des données du tableau 3.3

et 55%. Les colonnes indiquent le nombre de lignes détectées et le temps de traitement pour chaque valeur de α .

Les résultats indiquent que, pour des valeurs de α comprises entre 10% et 20%, le nombre de lignes détectées reste constant à 80, et le temps de calcul associé est également stable à 0.24 seconde. À partir de $\alpha = 25\%$, on observe une diminution progressive du nombre de lignes détectées, atteignant 2 à $\alpha = 55\%$. Le temps de calcul diminue légèrement à mesure que α augmente.

L'analyse des graphiques illustrés dans la figure 3.8 montre que jusqu'à $\alpha = 20\%$, la performance de l'algorithme est constante. Au-delà de cette valeur, le nombre de lignes détectées diminue progressivement, et à $\alpha = 55\%$, seulement 2 lignes sont détectées. Cependant, le temps de calcul diminue légèrement avec l'augmentation de α .

Ainsi, ces résultats suggèrent que l'algorithme de détection de lignes est sensible au paramètre α .

Des valeurs plus élevées de α conduisent à une réduction du nombre de lignes détectées mais également à une diminution du temps de traitement, ce qui peut être bénéfique pour des applications nécessitant une exécution rapide de l'algorithme. Cependant, des valeurs trop élevées de α peuvent entraîner de détections nulles

3.2.0.2 Variation de la taille des cellules triangulaire Le tableau 3.4 présente les résultats de simulations utilisant l'algorithme 13 de la transformée de Hough triangulaire sur l'image 1.11(a) de l'autoroute. Les paramètres de l'algorithme sont fixés avec $\alpha = 20\%$ et Seuil = 60, tandis que $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$. Les colonnes indiquent la hauteur h , la base b , la surface ST , le nombre de lignes détectées, et le temps de traitement pour chaque configuration de paramètres. L'analyse du tableau 3.4, dont les graphiques sont représentés dans la figure 3.4, révèle des variations significatives du nombre de lignes détectées et du temps de traitement en fonction des valeurs de h et b . À mesure que la taille de la cellule triangulaire augmente, allant de 4×2 à 29×2 , le nombre de lignes détectées

71

Tableau 3.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres fixes ($\alpha = 20\%$ et Seuil = 60), $1 \leq h \leq Nl$ et $1 \leq b \leq Nc$, où h et b représentent respectivement la hauteur et la base de la cellule triangulaire.

h b ST Droites détectées temps(sec)
2 4 4 165 0,



25
3 5 7 96 0,23
4 6 11 84 0.22
5 7 16 50 0,20
6 8 22 15 0,17
7 9 29 17 0,
16

h b ST Droites détectées temps(sec)
(pour $\alpha = 10$) 168 151 12 610 29 0.15

92 300 13 651 8 0.13

4 7 11 16 22 29

20

40

60

80

100

120

140

160

(165)

(96)

(84)

(50)

(15) (17)

ST

de

te

ct

ed

lin
es

detected lines

4 7 11 16 22 290.16

0.17

0.18

0.19

0.2

0.21

0.22

0.23

0.24

0.25

0.26
(0.25)

(0.23)

(0.22)

(0.20)

(0.17)

(0.16)

ST

T
im

e
(s

ec
on

de
)

Time

Figure 3.9 – Représentation graphique des données du tableau 3.4

diminue, passant de 165 droites à 15 droites. Pour ST = 29 le nombre de droites détectées est monté à 29. Ainsi, la tendance observée précédemment n'est pas générale. Le temps de traitement diminue à mesure que la taille de la cellule augmente, indiquant un temps de traitement réduit avec des cellules triangulaires plus grandes.

Ainsi, ces résultats mettent en évidence l'importance des paramètres h et b dans l'algorithme de la transformée de Hough triangulaire. Les cellules de grandes taille offre une précision acceptable et un temps de traitement plus rapide, en témoignent les données des tableaux 3.2 et 3.4, ainsi que les résultats visuels dans la figure 3.6.

4 Analyse comparative

Dans cette section, nous comparons les performances de deux méthodes de détection de lignes droites : la transformée de Hough triangulaire et sa version standard. Nous appliquons ces deux méthodes sur deux images différentes : une image d'un immeuble et une image d'une autoroute. Les

Tableau 3.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

Seuil Droites détectées temps1(sec) temps2(sec)
38 11271 0,55 0,000049
116 10 0,49 0,049

Tableau 3.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

Seuil Droites détectées temps1(sec) temps2(sec)
57 99 0,23 0,0023
100 10 0,21 0,021

résultats de ces comparaisons sont illustrés et analysés en termes de précision de détection et de temps de traitement, fournissant ainsi une évaluation complète de l'efficacité et de la rapidité des algorithmes. De plus, nous examinerons les performances de la transformée de Hough triangulaire par rapport à sa version parallélisée (THTP). Nous comparons également la performance temporelle et la précision de la Transformée de Hough Rectangulaire et de la transformée de Hough triangulaire. Ces analyses permettront de démontrer les avantages et les inconvénients de chaque méthode dans différents contextes d'application.

4.1 Transformée de Hough Triangulaire et Transformée de Hough Standard

Nous procédons dans cette section à une comparaison de la transformée de Hough triangulaire, méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.

(a) (b)

Figure 3.10 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough triangulaire sont illustré dans les tableaux 3.5 et 3.7 respectivement. En ce qui concerne la précision, pour un seuil de 38 votes, la THS a effectué une détection de 11271 droites. Cette détection est non pertinente comme le montre la première image de la figure 3.10. par contre, avec le même seuil de votes, la THT a détecté 11 droites. Les résultats visuels illustrés par la deuxième image de la figure 3.12 montre que les droites détectées suivent bien les lignes de l'immeuble suggérant des détections pertinentes.

73

(a) (b)

Figure 3.11 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

Tableau 3.7 – Résultats de simulation avec la THT sur l'image de l'immeuble (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

(h, b) ST α(%) seuil Nbre de droites détectées temps1(sec) temps2(sec)
(5,



3) 7 45 38 22 0.0095
(7,5) 16 40 38 11 0.
17 0.015

Les paramètres supplémentaires utilisés par la transformée de Hough triangulaire pour parvenir à ce résultat sont : les dimensions (h; b) et le pourcentage α de pixels allumés de chaque cellule triangulaire. Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 3.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 3.10 illustre bien ces résultats. Les 10

droites détectées suivent bien les lignes de l'immeuble indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 3.5 montre que la transformée de Hough standard à détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La transformée de Hough triangulaire a détecté quasiment le même nombre de droite en 0.17 seconde soit 0.015 seconde par droite détecté comme illustré dans le tableau 3.7. Ce qui suggère une net amélioration du temps de traitement.

Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A, où

$$A = \frac{T_s}{T_h}$$

= 0,49
0,17 = 2, 28 (Th représente le temps d'exécution de l'algorithme de la méthode proposée,

tandis que Ts représente le temps d'exécution de l'algorithme standard).

(a) (b)

Figure 3.12 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=38, α = 45%, h = 5 et b = 3 ; (b) Seuil=38, α = 40%, h = 7 et b = 5

De même avec l'image de l'autoroute, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough triangulaire sont illustré dans les tableaux 3.6 et 3.8 respective-

ment.

En ce qui concerne la précision, pour un seuil de 57 votes, la THS a effectué une détection de 99

74

Tableau 3.8 – Résultats de simulation avec la THT sur l'image de l'autoroute (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

(h, b) ST α(%) Seuil Nbre de droites détectées temps1(sec) temps2(sec)

(5,



3)	7	30	57	11	0.16	0.014
(7,5)	16	30	35	25	0.	
12	0.0048					

droites. Cette détection est non pertinente comme le montre la première image de la figure 3.11. par contre, avec le même seuil de votes, la THT a détecté 11 droites. Les résultats visuels illustrés par la première image de la figure 3.13 montre que les droites détectées suivent bien les lignes de l'autoroute suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough triangulaire pour parvenir à ce résultat sont : les dimensions (h; b) et le pourcentage α de pixels allumés de chaque cellule triangulaire.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 3.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 3.11 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 3.6 montre que la transformée de Hough standard à détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La transformée de Hough triangulaire a détecté quasiment le même nombre de droite en 0.16 seconde soit 0.014 seconde par droite détecté comme illustré dans le tableau 3.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le calcul du facteur d'accélération noté A,

où $A = \frac{T_s}{T_h}$

= 0,21
0,16 = 1, 31 (Th représente le temps d'exécution de l'algorithme de la méthode proposée,

tandis que Ts représente le temps d'exécution de l'algorithme standard).

(a) (b)

Figure 3.13 – Détection de lignes droites sur l'image1.11(a) de l'autoroute avec l'algorithme 13 de la THT avec les paramètres : (a) Seuil=57, $\alpha = 30\%$, $h = 5$ et $b = 3$; (b) Seuil=35, $\alpha = 30\%$, $h = 7$ et $b = 5$

4.2 Transformée de Hough Triangulaire et sa version parallélisée

4.2.1 Cas de l'image de l'immeuble

La figure 3.14 présente deux graphiques comparant les performances des algorithmes de transformée de Hough triangulaire et de transformée de Hough triangulaire parallélisée (THTP). Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et ST).

75

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.55
0.15

0.2

0.25

0.3

0.35

0.4

0.45

0.5

0.



55

(0.49)

(0.44) (0.44) (0.43)

(0.33) (0.34)

(0.27)

(0.21) (0.21)

(0.16)

(0.37)

(0.28) (0.29) (0.29)

(0.21) (0.22)

(0.
18) (0.17) (0.16)
(0.14)

α

T
im

e
(s

ec
on

de
)

THT
THTP

4 7 11 16 22 29
0.25



Figure 3.14 – Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($h = 3$, $b = 5$ et Seuil = 60), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules triangulaires et les paramètres (Seuil = 80, $\alpha = 20\%$) des l'algorithmes 13 et 15

Dans le premier graphique (à gauche), les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 55%. Pour toutes ces valeurs de α , la courbe de la transformée de Hough triangulaire parallélisée est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire parallélisée est plus rapide que la transformée de Hough triangulaire.

Dans le deuxième graphique (à droite), les variations du temps d'exécution sont comparées en fonction de la surface des cellules rectangulaires, avec les deux algorithmes de la THT et la THTP. Les surfaces des cellules varient de 4 à 29 px2. Pour toutes ces valeurs, la courbe de la transformée de Hough triangulaire parallélisée est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire parallélisée est plus rapide que la transformée de Hough triangulaire.

4.2.2 Cas de l'image de l'autoroute

La figure 3.15 présente deux graphiques comparant les performances des algorithmes de transformée

de Hough triangulaire et de transformée de Hough triangulaire parallélisée (THTP). Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et ST). Dans le premier graphique (à gauche), les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 55%. Pour toutes ces valeurs de α , la courbe de la transformée de Hough triangulaire parallélisée est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire est moins rapide que sa version parallélisée.

Dans le deuxième graphique (à droite), les variations du temps d'exécution sont comparées en

76

0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.55

0.12

0.14

0.16

0.18

0.2

0.22

0.24

(0.24) (0.



24) (0.24)

(0.22)

(0.19) (0.19) (0.19)

(0.16) (0.16)

(0.12)

(0.19) (0.19) (0.19)

(0.17)

(0.16) (0.16) (0.16)

(0.

14) (0.14)

(0.12)

α

T

im

e

(s

ec

on

de

)

THT

THTP

4 7 11 16 22 290.14

0.16

0.18

0.2

0.22

0.24

0.



26
(0.25)

(0.23)

(0.22)

(0.20)

(0.17)

(0.16)

(0.19)

(0.18)

(0.19)

(0.17)

(0.15)

(0.
14)

ST

T
im

e
(s

ec
on

de
)

THT
THTP

Figure 3.15 – Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres ($h = 4$, $b = 2$ et Seuil = 60), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules triangulaires et les paramètres (Seuil = 60, $\alpha = 20\%$) des l'algorithmes 13 et 15

fonction de la surface des cellules triangulaires, avec les deux algorithmes de la THT et la THTP.

Les surfaces des cellules varient de 4 à 29 px2. Pour toutes ces valeurs, la courbe de la transformée de Hough triangulaire est en dessous de celle de la transformée de Hough triangulaire. Cela suggère que la transformée de Hough triangulaire parallélisée est plus rapide que la transformée de Hough triangulaire.

Ainsi, l'analyse des deux graphiques met en lumière les performances relatives de la transformée de Hough triangulaire et sa version parallélisée en fonction des paramètres : α et la taille des cellules triangulaires. Pour les petites valeurs de α , la transformée de Hough triangulaire parallélisée montre des temps d'exécution plus rapides par rapport à THT. Cependant, cette supériorité diminue lorsque α devient plus élevé. De plus, en ce qui concerne la taille des cellules triangulaire, la transformée de Hough triangulaire parallélisée est plus performante tant pour les cellules de petites tailles que pour les cellules de taille plus grande. De plus la parallélisation n'a pas affecté la précision de la détection. La version parallélisé de la transformée de Hough triangulaire est donc la meilleur option pour une détection rapide dans le cas de l'image de l'immeuble et de l'autoroute. L'article [101] corobore ces résultats.

4.3 Transformée de Hough Rectangulaire et Transforméede Hough Triangulaire

Dans cette section, nous entreprenons une analyse comparative de la transformée de Hough rectangulaire et de la transformée de Hough triangulaire, examinant leurs méthodologies respectives, leurs complexités computationnelles et leurs performances en matière de détection de lignes droites. Cela permettra de mettre en lumière les avantages et les limitations de chaque approche, fournissant des indications pour choisir la méthode la plus adaptée à des applications spécifiques.

77

4.3.1 Performance temporelle

Dans cette sous-section, nous examinons en détails les temps de traitement associés à la transformée de Hough rectangulaire et à la transformée de Hough triangulaire pour la détection de lignes dans des images d'immeubles et d'autoroutes. Nous présentons les résultats de nos simulations, mettant en évidence les différences de temps de détection entre les deux méthodes. Cette analyse nous permettra de déterminer la méthode la plus efficace en termes de vitesse de traitement pour chaque scénario d'imagerie spécifique.

4.3.1.1 Cas de l'image de l'immeuble Le tableau 2.7 contient les résultats des simulations de la transformée de Hough rectangulaire appliquée à l'image de l'immeuble. Au niveau de la deuxième ligne, 11 lignes droites ont été détectées en 0.15 secondes ou 0.013 secondes par ligne, avec les paramètres $(l, L) = (5, 7)$, $SR = 35$, $\alpha = 0.35$ et $\text{Seuil} = 35$ où $SR(l, L)$ représente la surface du rectangle de largeur l et de longueur L .

Le tableau 3.7 contient les résultats des simulations de la transformée de Hough triangulaire appliquée à l'image de l'immeuble. Au niveau de la deuxième ligne, 11 lignes droites ont également été détectées, mais avec un temps de détection relatif élevé. Le temps de détection est de 0.17 secondes ou 0.015 secondes par ligne, avec les paramètres $(h, b) = (7, 5)$, $ST = 16$, $\alpha = 0.4$ et $\text{Seuil} = 38$ où $ST(h, b)$ représente l'aire du rectangle de hauteur h et de base b .

Le temps de détection est relativement faible dans les deux cas, ce qui indique que la THR et la THT sont efficaces pour détecter les lignes droites dans les images.

Les résultats des deux tableaux (tableau 2.7, tableau 3.7) révèlent que la transformée de Hough rectangulaire est plus rapide que la transformée de Hough triangulaire pour détecter les lignes droites dans l'image de l'immeuble.

4.3.1.2 Cas de l'image de l'autoroute Le tableau 2.8 contient les résultats des simulations de la transformée de Hough rectangulaire appliquée à l'image de l'autoroute. Au niveau de la première ligne, 11 lignes droites ont été détectées en 0.11 secondes ou 0.01 secondes par ligne, avec les paramètres $(l, L) = (3, 5)$, $SR = 15$, $\alpha = 0.4$ et $\text{Seuil} = 51$ où $SR(l, L)$ représente l'aire du rectangle de largeur l et de longueur L .

Le tableau 3.8 contient les résultats des simulations de la transformée de Hough triangulaire appliquée à l'image de l'autoroute. Au niveau de la première ligne, 11 lignes droites ont également été détectées, mais avec un temps de détection relatif élevé. 0, 16 secondes ou 0, 014 secondes par ligne, avec les paramètres $(h, b) = (5, 3)$, $ST = 7$, $\alpha = 0.3$ et $\text{Seuil} = 57$ où $ST(h, b)$ représente l'aire du rectangle de hauteur h et de base b .

Le temps de détection est relativement faible dans les deux cas, ce qui indique que THR et THT sont efficaces pour détecter les lignes droites dans les images.

78

Les résultats des deux tableaux montrent que la transformée de Hough rectangulaire est plus rapide que la transformée de Hough triangulaire pour détecter les lignes droites dans l'image de la route. En résumé, les deux méthodes de détection de lignes ont démontré des performances dans les deux scénarios d'imagerie, avec des résultats comparables en termes de précision et de vitesse. Cependant, la méthode de la transformée de Hough rectangulaire a montré un léger avantage en termes de temps de traitement dans les deux cas, ce qui suggère son application potentielle dans des environnements où la vitesse de traitement est cruciale.

4.3.2 Précision

La précision est un aspect crucial dans l'évaluation des performances des algorithmes de détection de lignes, car elle détermine la fiabilité des résultats obtenus. Dans cette sous-section, nous examinons attentivement la précision de deux méthodes de détection de lignes : la transformée de Hough rectangulaire et la transformée de Hough triangulaire, appliquées à des images d'immeubles et d'autoroutes. Nous analysons les résultats visuels des deux méthodes, en évaluant la pertinence des lignes détectées par rapport aux contours réels des objets dans les images.

4.3.2.1 Cas de l'image de l'immeuble Examinons la première image de la figure 3.16, la sortie visuelle de l'algorithme de la transformée de Hough rectangulaire. Il s'agit de la sortie visuelle des résultats du tableau 2.7. Les 11 lignes détectées sont toutes pertinentes. Aucune ligne indésirable n'a été détectée. Il est clair que 4 arêtes de bâtiment ont été détectées. Cela signifie que la même arête a été détectée plusieurs fois. Les lignes détectées sont toutes obliques par rapport au plan de l'image et correspondent aux lignes horizontales de l'immeuble. Aucune ligne verticale de l'immeuble n'a été détectée.

(a) (b)

Figure 3.16 – Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=35, $\alpha = 0.35$, $L = 7$ et $l = 5$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.4$, $h = 7$ et $b = 5$.

Il en va de même pour la deuxième image de la figure 3.16, le résultat de l'algorithme de la THT.

Il s'agit de la sortie visuelle des données du tableau 3.7. Comme pour l'algorithme de la THR, les 11 lignes détectées sont toutes pertinentes (mais ce ne sont pas les mêmes). Aucune ligne indésirable n'a été détectée. Il est clair que 4 arêtes du bâtiment ont été détectées. Cela implique que les arêtes ont

79

été détectées plusieurs fois. Les lignes détectées sont toutes obliques par rapport au plan de l'image et correspondent aux lignes horizontales du bâtiment. Aucune ligne verticale du bâtiment n'a été détectée.

Remarque 4 Les paramètres peuvent être affinés en fonction des besoins de l'application afin d'obtenir un rendu visuel plus efficace et plus précis. Par exemple, la première image de la figure 3.17, résultant de la transformée de Hough rectangulaire, illustre la détection de lignes horizontales et verticales avec les paramètres : Seuil=90, $\alpha = 0.1$, $L = 5$ et $l = 3$. Il en va de même pour la seconde image, résultant de la transformée de Hough triangulaire, avec les paramètres : Seuil=38, $\alpha = 0.45$, $h = 5$ et $b = 3$.

(a) (b)

Figure 3.17 – Détection de lignes droites sur l'image 1.11 de l'immeuble avec, (a) la THR avec les paramètres : Seuil=90, $\alpha = 0.1$, $L = 5$ et $l = 3$; (b) la THT avec les paramètres : Seuil=38, $\alpha = 0.45$, $h = 5$ et $b = 3$.

4.3.2.2 Cas de l'image de l'autoroute Examinons la première image de la figure 3.18, qui

représente le rendu visuel de l'algorithme de la THR. Cette illustration est la sortie graphique des résultats énumérés dans le tableau 2.8. Dans ce contexte, les 11 lignes détectées sont toutes pertinentes, sans aucune ligne superflue. Il est à noter que quatre bords de la route ont été identifiés, ce qui suggère que la même détection a pu être faite plusieurs fois. Ce qui est intéressant, c'est que les bords délimitant la route ont été clairement identifiés. De même, la deuxième image de la figure 3.18 résulte (a) (b)

Figure 3.18 – Détection de lignes droites sur l'image 1.11 de l'autoroute avec, (a) le THR avec les paramètres : Seuil=51, $\alpha = 0.4$, $L = 5$ et $l = 3$; (b) le THT avec les paramètres : Seuil=57, $\alpha = 0.3$, $h = 5$ et $b = 3$.

de l'application de l'algorithme de la THT. Cette représentation graphique correspond à la sortie visuelle des données présentées dans le tableau 3.8. Comme dans le cas de l'algorithme de la THR, 80

les 11 lignes détectées sont pertinentes, bien qu'elles ne correspondent pas nécessairement aux mêmes lignes. Une ligne superflue est identifiée. Cette fois, cinq bords de la route sont clairement détectés, ce qui suggère la possibilité d'une détection répétée de certains éléments.

Dans cette analyse comparative de la précision entre la transformée de Hough rectangulaire et la transformée de Hough triangulaire, nous avons examiné les résultats visuels des deux méthodes appliquées à des images d'immeubles et d'autoroutes. Dans chaque cas, les lignes détectées ont été évaluées par rapport aux contours réels des objets dans les images.

Les résultats obtenus ont montré que dans l'ensemble, les lignes détectées par les deux méthodes étaient pertinentes, avec peu ou pas de lignes superflues. Cependant, des cas de détections répétées de certaines arêtes ont été observés.. Cela souligne l'importance de l'ajustement précis des paramètres de chaque méthode pour optimiser la pertinence des détections.

Conclusion

la transformée de Hough triangulaire offre une approche novatrice pour la détection de lignes droites dans les images.



En exploitant le maillage triangulaire de l'image, la transformée de Hough triangulaire

permet une détection acceptable des lignes droites, tout en réduisant le temps de traitement, et ayant une complexité comparable à celle de la Transformée de Hough Standard.

Au cours de cette étude, nous avons examiné la méthode de la transformée de Hough triangulaire, en mettant en lumière ses différentes étapes, de la génération de la grille triangulaire virtuelle à la reconnaissance des droites discrètes. Nous avons également réalisé des simulations sur deux images pour évaluer les performances de la transformée de Hough triangulaire, en comparant ses résultats avec ceux de la transformée de Hough standard. L'étude de la complexité temporelle de l'algorithme



de la transformée de Hough triangulaire a révélé que la transformée de Hough triangulaire a la même complexité que la transformée de Hough standard.

Après ajustement des paramètres de la transformée de Hough triangulaire, nous avons pu observer une amélioration de la vitesse de détection par rapport à la THS pour les faibles seuils de vote, tout en maintenant une précision raisonnable. De plus, avec la réduction du nombre de pixels à calculer grâce à

la variable α , la transformée de Hough triangulaire offre des économies computationnelles importantes.

Les cellules de grandes tailles offre une précision acceptable et un temps de traitement plus rapide.

L'analyse comparative entre la transformée de Hough triangulaire et sa version parallélisée THTP révèle que la THTP est plus rapide pour des petites valeurs de seuil α , offrant des temps d'exécution plus courts. Cependant, cette efficacité diminue avec l'augmentation de α . Ainsi, la version parallélisée de la transformée de Hough triangulaire est recommandée pour une détection rapide, notamment dans les cas des images d'immeuble et d'autoroute.



L'analyse comparative de la transformée de Hough rectangulaire et de la transformée de Hough

81

triangulaire a mis en lumière plusieurs constats importants.

Tout d'abord, en ce qui concerne les performances temporelles, nous avons constaté que la transformée de Hough rectangulaire a tendance à être plus rapide que la transformée de Hough triangulaire dans la détection de lignes. En ce qui concerne la précision, nos résultats ont montré que les deux méthodes ont produit des résultats acceptables.

Cependant, la transformée de Hough rectangulaire est plus flexible que la transformée de Hough triangulaire en ce sens qu'elle offre une capacité de détection ciblé des droites verticales et horizontales.

Cependant, malgré ses avantages, la transformée de Hough triangulaire présente également certaines limites, notamment en termes de sensibilité aux paramètres (α et (b, h)), de complexité de mise en œuvre, de perte d'informations provenant de l'ensemble complet des pixels allumés des cellules triangulaires et la dépendance du contexte. Dans le chapitre suivant, nous aborderons la transformée de Hough hexagonale.

82

4

Ch
ap

itr
e

Transformée de Hough Hexagonale

Introduction	83
1 Description de la méthode	84
2 Complexité de la transformée de Hough hexagonale	90
3 Simulations et discussions	93
Conclusion	106

Introduction

La détection de lignes droites dans les images constitue une tâche fondamentale dans le domaine de la vision par ordinateur et de l'analyse d'images. Parmi les méthodes les plus couramment utilisées pour cette tâche, la Transformée de Hough (TH) est largement répandue pour sa capacité à détecter des motifs linéaires, même en présence de bruit.

Cependant, malgré sa robustesse, la méthode de la Transformée de Hough Standard (THS) présente des limitations, notamment en termes de temps de traitement et de sensibilité aux paramètres. Pour

surmonter ces défis, une approche innovante a émergé : la Transformée de Hough Hexagonale (THH). Dans cette étude, nous examinons en détail la méthode de la transformée de Hough hexagonale, explorant ses principes fondamentaux, ses mécanismes de fonctionnement et ses applications potentielles. Nous menons également des simulations pour évaluer les performances de la transformée de Hough hexagonale dans divers scénarios, en comparant ses résultats avec ceux de la méthode de la THS. Aussi, nous discutons des implications de nos résultats et des perspectives pour l'avenir de la détection de lignes dans les images. Enfin, nous examinons les performances comparatives de la transformée de Hough hexagonale et de sa version parallélisée (THHP) en fonction du taux α et de la taille des cellules hexagonales γ .

83

1 Description de la méthode

Dans cette section, nous présentons une approche novatrice de la Transformée de Hough standard appliquée sur une grille hexagonale virtuelle. Cette technique, dont les étapes sont décrites dans la figure 4.5, repose sur la subdivision de l'image en cellules hexagonales régulières, offrant ainsi une représentation alternative qui permet de réduire le temps de traitement et d'améliorer la détection des lignes. Nous décrivons en détail le processus de création de la grille hexagonale virtuelle, ainsi que l'algorithme de la THH.

En outre, nous discutons des techniques de sélection de cellules de la grille et de reconnaissance de droites discrètes. Enfin, nous présentons une méthode de visualisation des lignes détectées pour une interprétation des résultats.

1.1 Maillage Hexagonal de l'image

La conception d'une grille hexagonale virtuelle à l'aide du maillage de l'espace image consiste à diviser une image en cellules hexagonales et à positionner ces cellules de manière à couvrir l'image de manière cohérente. La grille hexagonale virtuelle est composée de cellules hexagonales ayant les mêmes caractéristiques. Il s'agit donc d'un maillage régulier comme illustré dans la figure 4.2.

Définition 12 (Hexagone) Un hexagone est un polygone à six côtés.

γ représente la taille de l'hexagone.

AB, BC, CD, DE, EF et FA sont les côtés de l'hexagone illustré dans la figure 4.1.

L'hexagone est subdivisé en trois parties comme illustré dans la figure 4.1(a) :

— le triangle isocèle ABF

— le rectangle BCEF

— le triangle isocèle CDE

Le théorème de Thalès est utilisé pour parcourir les pixels dans la zone triangulaire isocèle de chaque hexagone.

Le théorème de Thalès montre la proportionnalité entre les lignes parallèles. Par exemple, en

utilisant l'équation
(

$$\frac{AP}{AG} = \frac{NP}{BG}$$

BG

)
du théorème, on peut trouver que $NP = AP \times BG$

AG . Si $BG = \gamma$ et

$AG = \gamma$, alors $NP = AP$. Ainsi, $NP = PK$ car ABF est un triangle isocèle. De manière similaire,

nous avons $MQ = QR = QD$ comme indiqué dans la figure 4.1(b).

L'algorithme 17 est utilisé pour la conception de la grille hexagonale virtuelle. Avec les paramètres d'entrée (y et $image$), l'algorithme 17 renvoie un tableau de 3 entiers. La première valeur dans le

84

y

x

A

B C

D

EF

(a)

y

x

A

B C

D

EF

G H

Y

P Q

K

N

R

M

(b)

Figure 4.1 – Un hexagone ABCDEF

Algorithme 17 : Construction de la grille hexagonale
Fonction maillage hex($image$, y : entier) : tableau d'entiers

```
%y est la taille de l'hexagone;
Variables : tab : tableau of 3 entiers ;
Sorties : tab : tableau of 3 entiers ;
Debut

NI ← la hauteur de l'image;
Nc ← la largeur de l'image ;
tab[0] ← y;
tab[1] ← NI mod(2y – 2) ;% résidus sur la hauteur de l'image
tab[2] ← Nc mod(4y – 2) ;% résidus sur la largeur de l'image
Retourner : tab;

fin
fin
```

tableau est y exprimé en pixels et noté px , la deuxième et la troisième valeurs sont les résidus de ligne et de colonne, respectivement.

1.2 Sélection des cellules de la grille virtuelle

Le critère de sélection d'un hexagone dans la grille d'hexagones est le taux de pixels allumés dans l'hexagone. Ce taux est calculé avec l'algorithme 18. Il convient de noter que, pour un hexagone donné de la grille, tous les pixels ne sont pas pris en compte dans le calcul du taux. Dans la figure 4.2, les pixels de couleur verte sont ceux pris en compte dans le calcul. Les autres seront pris en compte dans

les hexagones voisins, comme illustré dans la figure 4.4. L'exclusion des pixels de couleur rouge et

y
x
A
B C
D
EF
G H
Y
Y

Figure 4.2 – Représentation des pixels d'une cellule hexagonale

85

y
x

Figure 4.3 – Une grille hexagonale superposée sur un espace image

magenta (avec l'algorithme 22) permet de ne pas prendre en compte deux fois le même pixel dans deux hexagones différents.

y
x
2 4
57
1
3
6

Figure 4.4 – La prise en compte des pixels exclus d'une cellule hexagonale

Le volume d'une cellule Hexagonale est calculé selon le lemme 2.

Lemme 2 (L'aire d'une cellule hexagonale) Soit y la taille d'un hexagone. La surface de l'hexagone, représentée par la partie verte de la figure 4.2, est calculée à l'aide de la formule suivante :

$$SH(y) = \frac{4(y - 1)(y - 1)}{2} \quad (4.1)$$

Preuve 2 (lemme 2) Considérons y comme la taille de l'hexagone.

Calcul de la surface $Sr(y)$ du rectangle dans lequel l'hexagone est inscrit :

$Sr(y) = L \times l = (3y - 1)(2y - 1)$ où $L = (3y - 1)$ et $l = (2y - 1)$ représentent respectivement la longueur et la largeur du rectangle.

Calcul de la surface $St(y)$ des quatre triangles rectangles (aux sommets du rectangle) : $St(y) =$

$$\frac{4(y^2 - y)}{2}$$

86

Algorithme 18 : Calcul du taux de pixels allumés
Fonction count_hex(img : image, A, B, C, D, E, F, G, H : tableau de deux entiers) : réel

Variables : k, l : entiers ;
Debut

k ← 0 ; l ← 0 ; MAT 1 ← Rely(A, F) ; MAT 2 ← Rely(F, E) ; MAT 3 ← Rely(E, D) ;

```

for  $A[1] \leq y \leq D[1]$  do

    if  $A[1] \leq y \leq G[1]$  then
         $NP \leftarrow |y - A[1]|$ 
        for  $A[0] - NP \leq x \leq A[0] + NP$  do

            if (x,y) est un element de MAT1 then
                ne rien faire;

            else
                if  $0 \leq x \leq Nl$  et  $0 \leq y \leq Nc$  then

                    if  $img[x, y] = 0$  then
                         $k \leftarrow k + 1$  ;

                    else

                         $l \leftarrow l + 1$  ;

                    end

                end

            end

        end

    end

    if  $G[1] \leq y \leq H[1]$  then

        for  $B[0] \leq x \leq F[0]$  do
            if (x,y) est un element de MAT2 then

                ne rien faire;
            else

                if  $0 \leq x \leq Nl$  and  $0 \leq y \leq Nc$  then
                    if  $img[x, y] = 0$  then

                         $k \leftarrow k + 1$  ;
                    else

                         $l \leftarrow l + 1$  ;

                    end

                end

            end

        end

    end

    if  $H[1] \leq y \leq D[1]$  then

         $MQ \leftarrow |y - D[1]|$ 
        for  $A[0] - MQ \leq x \leq A[0] + MQ$  do

            if (x,y) est un element de MAT3 then
                ne rien faire;

            else
                if  $0 \leq x \leq Nl$  and  $0 \leq y \leq Nc$  then

                    if  $img[x, y] = 0$  then
                         $k \leftarrow k + 1$  ;

                    else

                         $l \leftarrow l + 1$  ;

                    end

                end

            end

        end

    end

    end
    Retourner : k

k+l
;

fin
fin

```

Calcul de la surface $Se(y)$ de l'ensemble des pixels exclus : $Se(y) = (y-1) + 2(y-1) + 2 = 3y-1$

où $(y-1)$ représente le nombre de pixels rouges en bas de l'hexagone, $2 \times (y-1)$ représente le nombre

de pixels de couleur magenta sur les côtés gauche et droit de l'hexagone, et 2 représente les deux pixels

rouges en haut de l'hexagone.

Calcul de la surface de l'hexagone (seulement la partie verte de la figure 4.2) : $SH(y) = Sr(y) -$

$(St(y) + Se(y))$. Ce qui donne, après simplification et factorisation : $SH(y) = 4(y - 1)(y - 1/2)$

Par exemple, pour $y = 4$, $SH(4) = 4(4 - 1)(4 - 1/2) = 4 \times 3 \times 3,5 = 42 \text{ px}^2$, comme illustré dans la

figure 4.2. L'unité de mesure de la surface est en pixels carrés (px²).

1.3 Parallélisation de la transformée de Hough hexagonale

La parallélisation de la transformée de Hough hexagonal nécessite une adaptation de l'algorithme

pour tirer parti des capacités de traitement simultanée offertes par le parallélisme. La parallélisation

concerne l'accumulateur c'est-à-dire l'espace des paramètres encore appelé l'espace de Hough. Les

87

Algorithme 19 : Mise à jour de l'accumulateur
Procédure Acc update() : vide

```
% Mise à jour de l'accumulateur
if img[z, t] = 0 then

for itheta dans range(accum row) do
theta ← itheta × dtheta;
rho ← t × cos(theta) + z × sin(theta);
irho ← int(rho);
if irho > 0 et irho < accum column then

accum[itheta][irho] ← accum[itheta][irho] + 1
end

end
end

fin
```

algorithmes 24 et 23 sont les versions parallélisées des algorithmes 20 et 19. Les étapes principales de la parallélisation sont :

- importation de la bibliothèque pypm 4 : Importer une bibliothèque en Python est nécessaire pour accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction `import pypm` est utilisée pour charger la bibliothèque pypm.
- création de l'accumulateur de type pypm : Dans l'algorithme séquentiel, l'espace des paramètres (l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la transformée de Hough, est transformé en une structure de données partagée grâce à pypm. En effet, pypm permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour éviter les conflits de mémoire dans des environnements parallèles.



L'instruction `"accum pypm ← pypm.shared.array([accum row, accum column])"` de l'algorithme 24

crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable `"accum pypm"`.

- construction parallèle : L'instruction `"with pypm."`



`Parallel(4) as p"` de l'algorithme 24 indique

que nous souhaitons exécuter le bloc de code à l'intérieur de la déclaration `"with"` en parallèle avec 4 processus.

- parallélisation de l'algorithme 19 de mise à jours de l'accumulateur : La fonction "p.range"

génère une séquence d'indices à partir de laquelle chaque cœur du CPU obtiendra une plage d'indices

spécifique. Cela permet de diviser le travail entre les cœurs du CPU de manière équitable. Ainsi la

boucle "for itheta in p.range(accum row)" de l'algorithme 23 utilise les indices compris entre 0 et

accum row- 1 pour itérer à travers l'accumulateur par sections, où chaque section est traitée par un

processus différent.

4. <https://github.com/classner/pymp/tree/main>

88

Algorithme 20 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 1)

Fonction Hexagonal(image,y,q,seuil) :tableau

Pre-condition : $2 \leq y \leq \min(NI+2$

$2, Nc+2$

$2), 0 \leq \alpha \leq 1, 0 < \text{seuil}$

Variables : tab : tableau de 6 entiers ; A, B, C,D, E, F, G, H : tableaux de deux entiers ;



accum ligne, accum colon, irho, NI, Nc :

entiers ; accum : matrice ; dtheta, drho, rho, theta, DE

: réels;

Debut

NI ← la hauteur de l'image ; Nc ← la largeur de l'image ; % les dimensions de l'image ;

accum row ← 180 ; accum column ← E(

√

(Nc2 + NI2)) ; % les dimensions de la matrice "accum";



dtheta = $\pi/180$; tab ← maillage hex(image, y) ; Ht ← NI- tab[1] ; La ← Nc- tab[2] ; H1 ← Ht + 2y - 2 ; L1 ← La + 4y - 2 ;

for 3y - 2 ≤ y ≤ L1 par pas de 4y - 2 do

for 2y - 2 ≤ x ≤ H1 par pas de 2y - 2

do

A[0] = x- tab[0] + 1 ; A[1] = y - 3tab[0] + 2 ; B[0] = x- 2tab[0] + 2 ; B[1] = y - 2tab[0] + 1 ; C[0] = x- 2tab[0] + 2 ;

C[1] = y - tab[0] + 1 ; D[0] = x- tab[0] + 1 ; D[1] = y ; E[0] = x ; E[1] = y - tab[0] + 1 ; F [0] = x ;

F [1] = y - 2tab[0] + 1 ; G[0] = x- tab[0] + 1 ; G[1] = y - 2tab[0] + 1 ; H[0] = x- tab[0] + 1 ; H[1] = y - tab[0] + 1 ;



if count(img, A,B, D, E, F, G, H) ≥ α then

MAT 1 ← Rely(A, F) ;MAT 2 ← Rely(F, E) ;MAT 3 ← Rely(E,

D) ;

for A[1] ≤ j ≤ D[1] do

if A[1] ≤ j ≤ G[1]- 1 then

NP ← |j - A[1]|

for A[0]-NP ≤ i ≤ A[0] + NP do

if (i,j) est un element de MAT1 then

ne rien faire;

else

Acc update();

end

end

end

if G[1] ≤ j ≤ H[1] then

for B[0] ≤ i ≤ F [0] do

if (i,j)est un element de MAT2 then

ne rien faire;

else

Acc update()

```

end
;

end
end
if  $H[1] \leq j \leq D[1]$  then

MQ ← |j - D[1]|
for  $A[0] - MQ \leq i \leq A[0] + MQ$  do

if (i,j) est un element de MAT3 then
ne rien faire;

else
Acc update()

end
;

end
end

end
end

end
end

```

1.4 Reconnaissance des droites discrètes

L'algorithme 20 de la reconnaissance de droites discrètes par la Transformée de Hough Hexagonale

prend en entrée une image (img), un paramètre γ représentant la taille de l'hexagone, un paramètre α représentant le taux de pixels allumés dans l'hexagone, et le seuil de votes.

L'algorithme commence par initialiser les variables nécessaires, notamment les tableaux pour les points de l'hexagone (A, B, C, D, E, F, G, H), les dimensions de l'image, et la matrice d'accumulateur.

Ensuite, il effectue un maillage hexagonal de l'image à l'aide de la fonction maillage hex représenté par l'algorithme 17.

Le processus de détection des droites discrètes se fait en parcourant les hexagones de la grille

générée. Pour chaque hexagone, il vérifie s'il y a suffisamment de points allumés dans la région correspondante et met à jour l'accumulateur en conséquence.

Le cœur de l'algorithme se trouve dans la double boucle for qui parcourt les positions des hexagones

dans l'image. Pour chaque position, il calcule les coordonnées des points de l'hexagone, vérifie le nombre

de points allumés à l'intérieur de l'hexagone, et met à jour l'accumulateur à l'aide de l'algorithme 18

89

Algorithme 21 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale (partie 2)

```

for  $\gamma - 1 \leq y \leq L1$  par pas de  $4\gamma - 2$  do
for  $\gamma - 1 \leq x \leq H1$  par pas de  $2\gamma - 2$  do

```

```

A[0] = x - tab[0] + 1 ; A[1] = y - 3tab[0] + 2 ; B[0] = x - 2tab[0] + 2 ; B[1] = y - 2tab[0] + 1 ; C[0] = x - 2tab[0] + 2 ;
C[1] = y - tab[0] + 1 ; D[0] = x - tab[0] + 1 ; D[1] = y ; E[0] = x ; E[1] = y - tab[0] + 1 ; F[0] = x ;
F[1] = y - 2tab[0] + 1 ; G[0] = x - tab[0] + 1 ; G[1] = y - 2tab[0] + 1 ; H[0] = x - tab[0] + 1 ; H[1] = y - tab[0] + 1 ;

```



```

if count(img, A,B, C, D, E, F, G, H) ≥  $\alpha$  then

```

```

MAT 1 ← Rely(A, F) ; MAT 2 ← Rely(F, E) ; MAT 3 ← Rely(E,
D);
for  $A[1] \leq j \leq D[1]$  do

```

```

if  $A[1] \leq j \leq G[1] - 1$  then
NP ← |j - A[1]|
for  $A[0] - NP \leq i \leq A[0] + NP$  do

```

```

if (i,j) est un element de MAT1 then
ne rien faire;

```

```

else

```

```
Acc update()

end
;

end
end
if  $G[1] \leq j \leq H[1]$  then

for  $B[0] \leq i \leq F[0]$  do
if (i,j) est un element de MAT2 then

ne rien faire;
else

Acc update();
end

end
end
if  $H[1] \leq j \leq D[1]$  then

 $MQ \leftarrow |j - D[1]|$ 
for  $A[0] - MQ \leq i \leq A[0] + MQ$  do

if (i,j) est un element de MAT3 then
ne rien faire;

else
Acc update()

end
;

end
end

end
end
Rechercher les maxima dans l'accumulateur et placer les couples (p,  $\theta$ ) dans une liste 'lignes';
Retourner la liste 'lignes';

fin
fin
```

et de l'algorithme 19 respectivement.

Enfin, il retourne une liste appelée lignes contenant les couples (θ , p) correspondant aux paramètres des droites détectées.

L'objectif de l'algorithme 26 est de visualiser les droites identifiées à l'aide de la méthode de la Transformée de Hough. Il requiert une image en entrée (imge), la liste des paramètres de droites détectées (lignes) renvoyée par l'algorithme 20, ainsi qu'une spécification de couleur (colors). L'algorithme parcourt la liste des paramètres de droites détectées et reconstitue chaque droite correspondante sur l'image, attribuant la couleur colors à ces droites reconstruites. Cette étape de reconstruction vise à mettre en évidence de manière visuelle les droites détectées sur l'image d'origine, ce qui facilite l'interprétation et l'analyse des résultats obtenus par la Transformée de Hough.

2 Complexité de la transformée de Hough hexagonale

La fonction Relate dans l'algorithme 22 assigne des valeurs à un tableau, puis retourne ce tableau.

Chaque opération est de coût constant, donc la complexité totale de la fonction est $O(1)$.

La fonction maillage hex dans l'algorithme 17 initialise un tableau et lui assigne des valeurs, puis

90

Algorithme 22 : Sélection de tous les pixels compris entre A et B.
Fonction Relate(A, B : tableaux de deux entiers) : liste

Variables : MAT :une liste de couple d'entiers;
Debut

MAT ← les coordonnées des pixels
compris entre A et B, A et B exclus ;
Retourner : MAT ;

fin
fin

Algorithme 23 : Mise à jour parallèle de l'accumulateur
Procédure Acc updateP() : vide

% Mise à jour de l'accumulateur
if img[z, t] = 0 then

for itheta dans p.



```
range(accum row) do  
theta ← itheta × dtheta;  
rho ← t × cos(theta) + z × sin(theta);
```

```
irho ← int(rho);  
if irho > 0 et irho < accum column then
```

```
accum pypm[itheta][irho] ← accum pypm[itheta][irho] + 1  
end
```

```
end  
end
```

fin

retourne ce tableau. Chaque opération est de coût constant, donc la complexité totale de la fonction
est $O(1)$.

La fonction count hex dans l'algorithme 18 : initialise des variables k, l et D, puis elle parcourt les
pixels à l'intérieur de l'hexagone à l'aide de boucles dont le nombre d'itérations est $4(y-1)(y-1/2)$.

Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité du bloc formé par
les boucles est $O(4(y-1)(y-1/2)) = O(y^2)$, où $4(y-1)(y-1/2)$

est le nombre de pixels à l'intérieur de

l'hexagone. Ainsi, la complexité totale de la fonction count hex est $O(y^2)$.

La fonction Acc update dans l'algorithme 19 commence par un test de condition, dont le coût
est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend
uniquement de la valeur de accum row = 180, donc sa complexité est $O(1)$. À l'intérieur de cette
boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction Acc update est
 $O(1)$.

L'algorithme 20 de la transformée de Hough hexagonal commence par l'initialisation des variables
et du tableau tab, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces
opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction maillage hex dont la
complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux
boucles imbriquées parcourant les points à l'intérieur de chaque hexagonale à l'intérieur du quelle se
trouve des opérations de coût constant et la fonction Acc update de complexité $O(1)$. La complexité
91

Algorithme 24 : Reconnaissance de droites discrètes par la Transformée de Hough Hexa-
gonale Parallélisée (partie 1)

Fonction HexagonalP(imge, y, α, seuil, cpu count) : tableau
Pre-condition : $2 \leq y \leq \min(NI+2$

$2, Nc+2$
 $2), 0 \leq \alpha \leq 1, 0 < \text{seuil}$

Données : α

Variables : tab : tableau de 6 entiers ; A, B, C,D, E, F, G, H : tableau de deux entiers;
accum ligne, accum colon, irho, NI, Nc : entiers ; accum : accumulator ; Seuil, α , dtheta, drho, rho, theta, DE : réels;
Debut

```
import pypm ; % importation de la bibliothèque pypm
NI ← number of rows of image ; Nc ← number of columns of image ; % the dimensions of the image ;
 $\alpha = 0.8$  % The rate of lit pixels from which the triangle is selected ; threshold ← 150 % the minimum number of votes ;
```

```
accum row ← 180 ; accum column ← E(
√
```

```
(Nc2 + NI2)) ; % the dimensions of the matrix "accum";
```



```
accum pypm ← pypm.shared.array([accum row,
accum column]) ; % creation de l'accumul teur de type pypm
```



```
y ← 3 ; dtheta =  $\pi/180$  ; tab ← mesh(image, y) ; Ht ← NI - tab[3] ; La ← Nc - tab[4] ; H1 ← Ht + 2y - 2 ; L1 ← La + 4y - 2;
% Démarrage du traitement parallèle
With pypm.
parallel(cpu count) as p :
```

```
for 3y - 2 ≤ y ≤ L1 par pas de 4y - 2 do
for 2y - 2 ≤ x ≤ H1 par pas de 2y - 2 do
```

```
A[0] = x - tab[0] + 1 ; A[1] = y - 3tab[0] + 2 ; B[0] = x - 2tab[0] + 2 ; B[1] = y - 2tab[0] + 1 ;
C[0] = x - 2tab[0] + 2 ;
```

```
C[1] = y - tab[0] + 1 ; D[0] = x - tab[0] + 1 ; D[1] = y ; E[0] = x ; E[1] = y - tab[0] + 1 ; F [0] = x ;
F [1] = y - 2tab[0] + 1 ; G[0] = x - tab[0] + 1 ; G[1] = y - 2tab[0] + 1 ; H[0] = x - tab[0] + 1 ; H[1] = y - tab[0] + 1 ;
```



```
if count(img, A,B, D, E, F, G, H) ≥  $\alpha$  then
```

```
MAT 1 ← Rely(A, F) ;MAT 2 ← Rely(F, E) ;MAT 3 ← Rely(E,
D);
for A[1] ≤ j ≤ D[1] do
```

```
if A[1] ≤ j ≤ G[1] - 1 then
NP ← |j - A[1]|
for A[0] - NP ≤ i ≤ A[0] + NP do
```

```
if (i,j) est un element de MAT1 then
ne rien faire;
```

```
else
Acc updateP();
```

```
end
end
```

```
end
if G[1] ≤ j ≤ H[1] then
```

```
for B[0] ≤ i ≤ F [0] do
if (i,j)est un element de MAT2 then
```

```
ne rien faire;
else
```

```
Acc updateP() ;
end
```

```
end
end
if H[1] ≤ j ≤ D[1] then
```

```
MQ ← |j - D[1]|
for A[0] - MQ ≤ i ≤ A[0] + MQ do
```

```
if (i,j)est un element de MAT3 then
ne rien faire;
```

```
else
Acc updateP();
```

```
end
end
```

```
end
end
```

```
end
end
```

```
end
```

de ces boucles est donc $O(4(y-1)(y-1-2)) = O(y^2)$

Le coût des deux boucles principales imbriquées parcourant les hexagones de la grille, dont le

nombre total d'itérations dépend de la taille de l'image et de la taille des cellules hexagonales, contenant



un bloc d'opérations d'affectation de coût constant,
un test de condition sur la fonction count hex

de complexité $O(y^2)$, trois affectations appelant la fonction Rely de coût constant, les deux boucles

imbriquées de coût $4(y-1)(y-1-2)$ parcourant les pixels à l'intérieur de l'hexagone,



est $NI/(4y-2) \times$

$Nc/(2y-2) \times (1+4(y-1)(y-1-2)+1+4(y-1)(y-1-2))$,

donc de complexité $O(NI/(4y-2) \times Nc/(2y-$

$2) \times (1+4(y-1)(y-1-2)+1+4(y-1)(y-1-2))$

$2))) = O(NI \times Nc)$.

En considérant ces points, la complexité totale de l'algorithme 20 de la THH est égal à $O(NI \times Nc)$,

où NI et Nc sont le nombre de lignes et de colonnes de l'image respectivement, y est la taille des cellules

hexagonale. Ce qui correspond à la complexité de la THS.

92

Algorithme 25 : Reconnaissance de droites discrètes par la Transformée de Hough Hexagonale Parallélisée (partie 2)

```
for  $y-1 \leq y \leq L1$  par pas de  $4y-2$  do
for  $y-1 \leq x \leq H1$  par pas de  $2y-2$  do
```

```
A[0] =  $x - \text{tab}[0] + 1$  ; A[1] =  $y - 3\text{tab}[0] + 2$  ; B[0] =  $x - 2\text{tab}[0] + 2$  ; B[1] =  $y - 2\text{tab}[0] + 1$  ;
C[0] =  $x - 2\text{tab}[0] + 2$  ;
```

```
C[1] =  $y - \text{tab}[0] + 1$  ; D[0] =  $x - \text{tab}[0] + 1$  ; D[1] =  $y$  ; E[0] =  $x$  ; E[1] =  $y - \text{tab}[0] + 1$  ; F[0] =  $x$  ;
F[1] =  $y - 2\text{tab}[0] + 1$  ; G[0] =  $x - \text{tab}[0] + 1$  ; G[1] =  $y - 2\text{tab}[0] + 1$  ; H[0] =  $x - \text{tab}[0] + 1$  ; H[1] =  $y - \text{tab}[0] + 1$  ;
```



```
if count(img, A,B, C, D, E, F, G, H)  $\geq \alpha$  then
```

```
MAT 1  $\leftarrow$  Rely(A, F) ; MAT 2  $\leftarrow$  Rely(F, E) ; MAT 3  $\leftarrow$  Rely(E,
D);
for A[1]  $\leq j \leq D[1]$  do
```

```
if A[1]  $\leq j \leq G[1]-1$  then
NP  $\leftarrow |j - A[1]|$ 
for A[0]-NP  $\leq i \leq A[0] + NP$  do
```

```
if (i,j) est un element de MAT1 then
ne rien faire;
```

```
else
Acc updateP();
```

```
end
end
```

```
end
if G[1]  $\leq j \leq H[1]$  then
```

```
for B[0] ≤ i ≤ F [0] do
if (i,j) est un element de MAT2 then

ne rien faire;
else

Acc updateP();
end

end
end
if H[1] ≤ j ≤ D[1] then

MQ ← |j -D[1]|
for A[0]-MQ ≤ i ≤ A[0] + MQ do

if (i,j) est un element de MAT3 then
ne rien faire;

else
Acc updateP();

end
end

end
end

end
end

end
fin
Rechercher les maxima dans l'accumulateur et placer les couple (p, θ) dans une liste 'lignes';
Retourner la liste 'lignes';

fin
fin
```

Algorithme 26 : Reconstruction des droites détectées
Fonction PlotHoughLine(imge,lignes,colors) :image

Parcourir 'lignes' la liste des paramètres des droites détectées, puis construire chaque droite correspondant en couleur 'colors' sur l'image 'imge'.

fin

3 Simulations et discussions

Dans cette section, nous présentons les résultats des simulations et des discussions sur l'application de la Transformée de Hough Hexagonale à deux images différentes : celle d'un immeuble et celle d'une autoroute représentées sur la figure 1.11. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Nous examinons les effets de trois paramètres clés, le seuil de vote, γ et α , sur les performances de

Figure 4.5 – Les étapes de la technique de la transformée de Hough hexagonale la méthode. En outre, nous comparons les résultats de la transformée de Hough hexagonale avec ceux de la Transformée de Hough Standard pour évaluer son efficacité et sa précision. Enfin, nous discutons également les performances de la transformée de Hough hexagonale par rapport à sa version parallélisée.

3.1 Cas de l'image de l'immeuble

Dans cette section, nous examinons les performances de la Transformée de Hough Hexagonale appliqué à une image représentant un immeuble. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement.

Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans

94

Tableau 4.1 – Resultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=62 et $\alpha = 14\%$) et γ variant entre 2 et 64

γ	SH	Droites détectées	temps(sec)
2	6	212	0.57
4	42	223	0.53
6	110	157	0.49
8	210	77	0.44
10	342	109	0.46
12	506	40	0.40
14	702	51	0.39
26	2550	21	0.33
30	3422	24	0.33
64	16002	15	0.32

ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme THH en fonction des caractéristiques de l'image de l'immeuble.

3.1.1 Variation de γ

Le tableau 4.1 résume les résultats de simulations effectuées sur l'image de l'immeuble en utilisant l'algorithme de la THH avec des paramètres fixes (Seuil=62 et $\alpha = 14\%$), tandis que le paramètre γ varie entre 2 et 64. Les colonnes indiquent les valeurs de γ , l'aire SHdes cellules hexagonale, le nombre de lignes détectées et le temps de traitement pour chaque configuration de paramètres.

L'analyse du tableau révèle des variations du nombre de lignes détectées et du temps de traitement en fonction des valeurs de γ . le temps de traitement diminue à mesure que γ augmente.

La figure 4.6 illustrent graphiquement ces données. Le premier graphique de la figure montre la relation entre la taille des cellules et le nombre de lignes détectées. De façon générale, le graphique du nombre de droites détectées en fonction la taille des cellules est décroissante. Cependant, l'augmentation de la taille des cellules n'implique pas toujours une diminution du nombre de droites détectées. Par exemple, avec SH = 42 223 droites ont été détectées contre 212 droites avec SH = 6. Le deuxième graphique illustre la relation entre la taille des cellules et le temps de traitement. On constate une tendance décroissante : à mesure que la taille des cellules hexagonales augmente, le temps de traitement diminue.

3.1.2 Variation de α

La table 4.2 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble en utilisant l'algorithme 20 de la THH. Les simulations ont été réalisées avec des paramètres fixes (Seuil=60 et $\gamma = 4$), tandis que le paramètre α a été modifié entre 10% et 35%. Les colonnes indiquent les valeurs de α , le nombre de lignes détectées et le temps de traitement.

La figure 4.7 contient des graphiques représentant les données de la table 4.2 :

64	21	10	210	506	702	2,550	3,422
20							
40							
60							
80							
100							

120

140

160

180

200

220

240

(212)

(223)

(157)

(77)

(40)

(51)

(21) (24)

SH

D

ro

ite

s

dé

te

ct

ée

s

Droites détectées

642110210 506 702 2,550 3,422

0.32

0.34

0.36

0.38

0.4

0.42

0.44

0.46

0.48

0.5

0.52

0.54

0.56

0.58(0.57)

(0.53)

(0.49)

(0.44)

(0.40)

(0.39)

(0.33) (0.33)

SH

Te
m

ps

Temps

Figure 4.6 – Courbe représentative des données du tableau 4.1

Tableau 4.2 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=60 and $\gamma = 4$) et α variant entre 10% et 35%

α (%)	Nbre de droites détectées	temps(sec)
10	492	0.58
15	236	0.48
20	106	0.42
25	34	0.33
30	3	0.24
35	1	0.16

0.1 0.15 0.2 0.25 0.3 0.350

50

100

150

200

250

300

350

400

450

500

550

(492)

(236)

(106)

(34)

(3) (1)

α

de
te

ct
ed

lin
es

detected lines

0.1 0.15 0.2 0.25 0.3 0.35
0.15

0.2

0.25

0.3

0.35

0.4

0.45

0.5

0.55

0.6
0.65
(0.58)
(0.48)
(0.42)
(0.33)
(0.24)
(0.16)
 α
T
im
e
(s
ec
on
de
)
Time

Figure 4.7 – Courbe représentative des données du tableau 4.2

Le premier graphique illustre la relation entre le taux α et le nombre de lignes détectées par l'algorithme 20 de la transformée de Hough hexagonale. On observe une décroissance du nombre de lignes détectées à mesure que α augmente. Cette tendance suggère qu'avec des valeurs plus élevées de α , moins de lignes sont détecté, indiquant une sensibilité de l'algorithme à ce paramètre. Des valeurs trop élevées de α peuvent entrainer des détections non pertinentes de droites

Le deuxième graphique représente le temps de traitement en fonction du taux α . On observe également une décroissance du temps de traitement à mesure que α augmente. Cette diminution du temps de traitement est associée à la valeur de α , car plus α augmente, moins on a de cellules a traiter conduisant naturellement à une accélération du processus.

L'analyse du tableau révèle des variations du nombre de lignes détectées et du temps de traitement en fonction des valeurs de α . À mesure que α augmente, le nombre de lignes détectées diminue, passant de 492 lignes avec $\alpha = 10\%$ à seulement 1 ligne avec $\alpha = 35\%$. De même, le temps de traitement diminue à mesure que α augmente.

La figure 4.7 illustrent graphiquement ces données. Le premier graphique de la figure montre la relation entre α et le nombre de lignes détectées. On observe une tendance décroissante : à mesure que α augmente, le nombre de lignes détectées diminue. Le deuxième graphique de la figure illustre la relation entre α et le temps de traitement. Encore une fois, on constate une tendance décroissante : à mesure que α augmente, le temps de traitement diminue.

Ces résultats mettent en évidence l'importance du paramètre α dans l'algorithme de la transformée de Hough hexagonale. Des valeurs élevées de α conduisent non seulement à une réduction du nombre de lignes détectées mais également à une diminution du temps de traitement. Cependant, il faut noter que des valeurs trop élevées de α peuvent conduire à des détections moins précises ou nulles.

3.2 Cas de l'image de l'autoroute

Dans cette section, nous examinons les performances de l'algorithme de la transformée de Hough

hexagonale appliqué à une image représentant une autoroute. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la THH en fonction des caractéristiques de l'image de l'immeuble.

97

Tableau 4.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=60 and $\alpha = 10\%$) et γ variant entre 2 et 40

γ	SH(px2)	Nbre de droites détectées	temps(sec)
2	6	80	0.28
3	20	69	0.26
4	42	62	0.24
5	72	60	0.24
6	110	54	0.23
7	156	32	0.21
8	210	30	0.22
9	272	17	0.20
10	342	16	0.20
14	702	22	0.20
20	1482	25	0.21
32	3906	10	0.16
40	6162	4	0.13

3.2.1 Variation de la taille des cellules de la grille

Le tableau 4.3 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute du chapitre

1 en utilisant l'algorithme 20 de la transformée de Hough hexagonale. Les simulations ont été réalisées avec des paramètres fixes (Seuil=60 et $\alpha = 10\%$), tandis que le paramètre γ a été modifié entre 2 et

40. La table 4.3 comporte quatre colonnes que sont : La première colonne représente les différentes

valeurs du paramètre γ , variant de 2 à 10. La deuxième colonne indique la surface (l'aire) SH de la

cellule hexagonale H en pixels carrés. La troisième colonne montre le nombre de lignes détectées pour

chaque valeur de γ . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

6 20 42 72 110 156 210 272 342

20

30

40

50

60

70

80

90

(80)

(69)

(62) (60)

(54)

(32) (30)

(17) (16)

SH

D
et

ec
te

d
lin

es

Detected lines

6 20 42 72 110 156 210 272 3420.19

0.2

0.21

0.22

0.23

0.24

0.25

0.26

0.27

0.28

0.29
(0.



28)

(0.26)

(0.24) (0.24)

(0.23)

(0.21)

(0.22)

(0.20)

(0.
19)

SH

T
im

e

Time

Figure 4.8 – Courbe représentative des données du tableau 4.3

La figure 4.8 est la représentation graphique du tableau 4.3. Elle contient deux graphiques :

Le premier graphique illustre la variation du nombre de lignes détectées par l'algorithme THH en

98

Tableau 4.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 20 de la THH avec les paramètres fixes (Seuil=60 and $\gamma = 3$) et α variant entre 10% et 30%

α	Nbre de droites détectées	temps(sec)
10	69	0.25
15	52	0.24
20	34	0.22
25	17	0.20
30	0	0.13
(pour $\alpha = 11$ et seuil = 51)	31	19 0.13

0.1 0.15 0.2 0.25 0.30

10

20

30

40

50

60

70

80

(69)

(52)

(34)

(17)

(0)

α

D
et

ec
te

d
lin

es

Detecte d lines

0.1 0.15 0.2 0.25 0.3

0.14

0.16

0.18

0.2

0.22

0.24

0.26

0.28

(0.25)
(0.24)

(0.22)

(0.20)

(0.13)

α

T
im

e(
se

c)

Time

Figure 4.9 – Courbe représentative des données du tableau 4.4

fonction de la taille des cellules hexagonales évalué par leur surface SH . Pour les valeurs de SH allant

de 6 à 342, on observe, en générale, une tendance à la diminution du nombre de lignes détectées en fonction de la taille des cellules hexagonales. Cependant, une augmentation de la taille des cellules n'implique toujours pas une diminution du nombre de droites détectées. Par exemple, avec SH = 1482 25 droites ont été détectées contre 16 droites avec SH = 342. Le deuxième graphique représente le temps de traitement en fonction de la taille des cellules hexagonale. On constate une légère diminution du temps de traitement à mesure que la surface de l'hexagone augmente.

3.2.2 Variation de α

Le tableau 4.4 présente les résultats de simulations sur une image d'une autoroute en utilisant l'algorithme 20 de la THH. Les simulations ont été réalisées avec des paramètres fixes (Seuil=60 et $\gamma = 3$), tandis que le paramètre α a été modifié entre 10% et 30%. La première colonne représente les différentes valeurs du paramètre α , variant de 10% à 30%. La deuxième colonne montre le nombre de lignes détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

L'analyse du tableau 4.4 révèle des variations du nombre de lignes détectées et du temps de traitement en fonction des valeurs de α . À mesure que α augmente, le nombre de lignes détectées

99

(a) (b)

Figure 4.10 – (a) THH sur l'immeuble ($\gamma = 64$, $\alpha = 14$, seuil = 62), (b) THH sur l'autoroute ($\gamma = 30, \alpha = 10$, seuil = 60)

Tableau 4.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
25	25697	0,54	0,00002
116	10	0,49	0,049

diminue, passant de 69 lignes avec $\alpha = 10\%$ à 0 ligne avec $\alpha = 30\%$. Des valeurs trop élevées de α peuvent entrainer une détérioration de la qualité de détection. De manière similaire, le temps de traitement diminue à mesure que α augmente, indiquant une amélioration du temps de traitement avec des valeurs de α plus élevées et une précision de détection acceptable

La figure 4.9 illustrent graphiquement ces données. La première figure montre la relation entre α et le nombre de lignes détectées. On observe une tendance décroissante : à mesure que α augmente, le nombre de lignes détectées diminue. La deuxième figure illustre la relation entre α et le temps de traitement. Encore une fois, on constate une tendance décroissante : à mesure que α augmente, le temps de traitement diminue.

Ainsi, ces résultats mettent en évidence l'importance des paramètres γ , α , seuil dans l'algorithme de détection de lignes THH. Les cellules de grandes taille offre une précision acceptable et un temps de traitement plus rapide, en témoignent les données des tableaux 4.1 et 4.3, ainsi que les résultats visuels dans la figure 4.10.

3.3 Comparaison de transformée de Hough hexagonale et transformée de Hough standard

Nous procédons dans cette section à une comparaison de la transformée de Hough hexagonale,

Tableau 4.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
67	47	0,23	0,0048
100	10	0,21	0,021

(a) (b)

Figure 4.11 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=25) ; (b) Détection de lignes droites sur l'image1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

(a) (b)

Figure 4.12 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=67) ; (b) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough hexagonale sont illustré dans les tableaux 4.5 et 4.7 respectivement.

En ce qui concerne la précision, pour un seuil de 25 votes, la transformée de Hough standard a effectué une détection de 25697 droites. Cette détection est non pertinente comme le montre la première image de la figure 4.11. par contre, avec le même seuil de votes, la THH a détecté 10 droites. Les résultats visuels illustrés par la deuxième image de la figure 4.13 montre que les droites détectées suivent bien des lignes de l'immeuble indiquant des détections pertinentes hors mis trois superflus.

Les paramètres supplémentaires utilisés par la transformée de Hough hexagonale pour parvenir à ce résultat sont : la taille y et le pourcentage α de pixels allumés de chaque cellule hexagonale.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 4.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 4.11 illustre bien ces résultats. Ces droites détectées suivent bien des lignes de l'immeuble indiquant des détections pertinentes hormis celles des côtés délimitant l'image.

Pour ce qui est du temps de traitement, le tableau 4.5 montre que la THS à détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La transformée de Hough hexagonale a détecté le même nombre de droite en 0.14 seconde soit 0.014 seconde par droite détecté comme illustré dans le tableau 4.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée

à travers le calcul du facteur d'accélération noté A , où $A = \frac{T_s}{T_h}$

$$= \frac{0,49}{0,14} = 3,5 \text{ (} T_h \text{ représente le temps}$$

d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de

Tableau 4.7 – Résultats de simulation avec la THH appliquée sur l'image 1.11(c) de l'immeuble (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

y	$SH(px2)$	$\alpha(\%)$	seuil	Nbre de droites détectées	temps1(sec)	temps2(sec)
3	20	30	50	37	0.30	0.0081
4	42	37	25	10	0.14	0.014

Tableau 4.8 – Résultats de simulation avec la THH appliquée sur l'image 1.11(a) de l'autoroute (temps1= temps de détection de toutes les droites ; temps2=temps de détection d'une seule droite)

y	SH(px2)	$\alpha(\%)$	Threshold number of detected lines	time1(sec)	time2(sec)	
2	6	50	39	33	0.15	0.0045
3	20	25	67	11	0.20	0.018

l'algorithme standard).

(a) (b)

Figure 4.13 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres : (a) Seuil=50, $\alpha = 30\%$, $\gamma = 3$; (b) Seuil=25, $\alpha = 37\%$, $\gamma = 4$

De même avec l'image de l'autoroute, les résultats des expériences avec la transformée de Hough standard et la transformée de Hough hexagonale sont illustré dans les tableaux 4.6 et 4.8 respectivement.

En ce qui concerne la précision, pour un seuil de 67 votes, la THS a effectué une détection de 47 droites. sur les 47, non seulement plusieurs sont non pertinentes mais aussi une même bord à été détectée plusieurs fois comme lemontre la première image de la figure 4.12. par contre, avec le même seuil de votes, la transformée de Hough hexagonale a détecté 11 droites. Les résultats visuels illustrés par la deuxième image de la figure 4.14 montre une amélioration.

Les paramètres supplémentaires utilisés par la transformée de Hough hexagonale pour parvenir à ce résultat sont : la taille γ des cellules hexagonale et le pourcentage α de pixels allumés de chaque cellule hexagonale.

Pour avoir une bonne détection de lignes droites avec la transformée de Hough standard, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 4.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 4.12 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 4.6 montre que la THS à détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La transformée de Hough hexagonale a détecté quasiment le même nombre de droite en 0.20 seconde soit 0.018 seconde par droite détecté comme illustré dans le tableau 4.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en

102

outre quantifiée à travers le calcul du facteur d'accélération noté A, où $A = T_s / T_h$

$$= 0,21 / 0,20 = 1,05 \text{ (} T_h \text{)}$$

représente le temps d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).

(a) (b)

Figure 4.14 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 20 de la THH avec les paramètres : (a) Seuil=39, $\alpha = 50\%$, SH = 6 pour $\gamma = 2$; (b) Seuil=67, $\alpha = 25\%$, SH = 20 pour $\gamma = 3$

3.4 Comparaison de THH et sa version parallélisée

Dans cette section, nous examinons et comparons les performances de la transformée de Hough hexagonale par rapport à sa version parallélisée. Nous explorons les variations du temps d'exécution en fonction de différents paramètres tels que α et SH (surface des cellules hexagonales), en nous appuyant sur deux cas d'étude distincts : l'immeuble et l'autoroute. Cette comparaison vise à évaluer l'impact de la parallélisation sur les performances de la transformée de Hough hexagonale et à identifier les cas de figure où la transformée de Hough hexagonale parallélisée offre des avantages par rapport à sa version séquentielle.

3.4.1 Cas de l'image de l'immeuble

La figure 4.15 présente deux graphiques comparant les performances des algorithmes de THH et THHP. Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et SH).

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du

paramètre α , avec des valeurs allant de 10% à 35%. Pour les petites valeurs de α (entre 10% et 30%), la courbe de transformée de Hough hexagonale parallélisée est en dessous de celle de la transformée de Hough hexagonale. Cela implique une amélioration des performances de la transformée de Hough hexagonale par la programmation parallèle pour les petites valeurs de α .

Pour des valeurs de α plus élevés (entre 30% et 35%), la courbe de la transformée de Hough hexagonale parallélisée passe au dessus de celle de la transformée de Hough hexagonale. Cela suggère que transformée de Hough hexagonale parallélisée est moins performante pour les valeurs élevées de α .

La parallélisation de la transformée de Hough hexagonale n'a pas affectée la précision de la détection.

103

0.1 0.15 0.2 0.25 0.3 0.35
0.15

0.2

0.25

0.3

0.35

0.4

0.45

0.5

0.55

0.6

0.65

0.7

(0.58)

(0.48)

(0.42)

(0.33)

(0.24)

(0.16)

(0.42)

(0.36)

(0.34)

(0.29)

(0.24)

(0.19)

α

T
im

e
(s

ec
on

de
)

Time (THH)
Time (THHP)

6 20 42 72

0.2

0.25

0.3

0.35

0.4

0.45

0.5

(0.42)

(0.31)

(0.23)

(0.19)

(0.36)

(0.27)

(0.22)

(0.20)

SH

T
im

e

Time (THH)
Time (THHP)

Figure 4.15 – Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=60, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=50, $\alpha = 30\%$) des algorithmes 20 et 24

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de la surface des cellules hexagonales, avec les deux algorithmes de THH et THHP. Les surfaces des cellules varient de 6 px2 à 72 px2. La courbe de la transformée de Hough hexagonale parallélisée est en dessous de celle de la transformée de Hough hexagonale pour les surfaces inférieure a 57 px2. De 57 à 72 px2, la courbe de la transformée de Hough hexagonale parallélisée passe au dessus de la courbe de la transformée de Hough hexagonale. Cela suggère que la transformée de Hough hexagonale parallélisée améliore le temps de détection de la transformée de Hough hexagonale pour les petites valeurs de SH et moins rapide que la transformée de Hough hexagonale pour des valeurs de SH plus élevés.

3.4.2 Cas de l'image de l'autoroute

La figure 4.16 présente deux graphiques comparant les performances des algorithmes de transformée de Hough hexagonale et de transformée de Hough hexagonale parallélisée. Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et SH). Dans le premier graphique, nous comparons les variations du temps d'exécution en fonction du paramètre α , avec des valeurs allant de 10% à 30%. Pour des valeurs de α plus basses (entre 10% et 25%), la courbe de la transformée de Hough hexagonale parallélisée se situe en dessous de celle de la transformée de Hough hexagonale. Cela suggère que la méthode de la transformée de Hough hexagonale est plus performante pour les petites valeurs de α . En revanche, pour les valeurs élevées de α (au moins 30%), la courbe de la transformée de Hough hexagonale parallélisée passe au-dessus de celle de la transformée de Hough

hexagonale. Cela suggère que la transformée de Hough hexagonale parallélisée est moins rapide pour les élevées valeurs de α .

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de

104

0.1 0.15 0.2 0.25 0.30.12

0.14

0.16

0.18

0.2

0.22

0.24

0.26

0.28

(0.25)
(0.24)

(0.22)

(0.20)

(0.13)

(0.23)

(0.21) (0.21)

(0.19)

(0.16)

α

T
im

e
(s

ec
)

Time (THH)
Time (THHP)

6 20 42 72 110 156 210 272 342

0.2

0.22

0.24

0.26

0.28

0.3

(0.28)

(0.26)

(0.24) (0.24)

(0.23)

(0.21)

(0.22)

(0.20)

(0.19)

(0.26)

(0.24)

(0.22)

(0.21) [(0.21)

(0.19)

SH

T
im

e

Time (THH)

Time (THHP)

Figure 4.16 – Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=60, $\gamma = 3$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=60, $\alpha = 10\%$) des algorithmes 20 et 24

la surface des cellules hexagonales, avec les deux algorithmes de THH et THHP. Les surfaces des cellules varient de 6 à 342 px2. La courbe de la transformée de Hough hexagonale parallélisée est en dessous de celle de la transformée de Hough hexagonale pour les surfaces inférieures à 110 px2. Après 110 px2, la courbe de la transformée de Hough hexagonale parallélisée et de la transformée de Hough hexagonale sont entrelacées. Cela suggère que la transformée de Hough hexagonale parallélisée est plus performante que la transformée de Hough hexagonale pour les petites valeurs de SH, c'est-à-dire les cellules de petite taille.

Ainsi, l'analyse comparative entre la transformée de Hough hexagonale et sa version parallélisée met en lumière plusieurs conclusions pertinentes. Dans les deux cas d'étude examinés, à savoir l'immeuble et l'autoroute, nous observons des tendances similaires quant à l'impact de la parallélisation sur les performances. Pour des valeurs de paramètres α et SH (surface des cellules hexagonales) relativement faibles, la THHP démontre une amélioration du temps d'exécution par rapport à la THH. Cependant, cette tendance s'inverse pour des valeurs de paramètres plus élevées, où la transformée de Hough hexagonale prend l'avantage. La parallélisation de la transformée de Hough hexagonale n'affecte pas la précision de la détection.

En considérant ces observations, il est recommandé d'utiliser la transformée de Hough hexagonale lorsque des seuils α plus élevés sont privilégiés ou lorsque la taille des cellules est importante. En revanche, la transformée de Hough hexagonale parallélisée est préférable pour des seuils α plus bas et des tailles de cellules modérées. Le choix entre les deux dépendra donc des valeurs requises de α et SH pour la détection.

105

Conclusion

L'étude approfondie de la transformée de Hough hexagonale révèle son potentiel en tant qu'alternative prometteuse à la méthode



dspace.centre-univ-mila.dz

[https://dspace.centre-univ-mila.dz/jspui/bitstream/123456789/3659/1/Conception et mise en oeuvre d'un système de contrôle d'accès basé sur la reconnaissance faciale.pdf](https://dspace.centre-univ-mila.dz/jspui/bitstream/123456789/3659/1/Conception%20et%20mise%20en%20oeuvre%20d'un%20syst%C3%A8me%20de%20contr%C3%B4le%20d'acc%C3%A8s%20bas%C3%A9%20sur%20la%20reconnaissance%20faciale.pdf)

de la Transformée de Hough Standard pour la détection de lignes

droites dans les images.

Les simulations ont démontré que la transformée de Hough hexagonale offre une amélioration en

termes de temps de traitement et en termes de précision de détection des lignes. Après ajustement des valeurs de γ , α et le seuil de vote, nous avons pu observer une réduction du temps d'exécution tout en maintenant une détection acceptable des lignes droites. Aussi, la réduction du nombre de pixels à calculer à travers la variable α permet des économies computationnelles. Les cellules de grandes tailles offre une précision acceptable et un temps de traitement plus rapide. De plus, notre analyse de la complexité temporelle a montré que la transformée de Hough hexagonale présente une complexité similaire à celle de la transformée de Hough standard, ce qui confirme sa faisabilité pratique pour des applications en temps réel.

L'analyse comparant la transformée de Hough hexagonale et sa version parallélisée révèle que pour des valeurs relativement faibles des paramètres α et SH (surface des cellules hexagonales), la THHP montre une réduction du temps d'exécution par rapport à la THH. Toutefois, cette tendance s'inverse lorsque les valeurs de ces paramètres augmentent, favorisant alors la transformée de Hough hexagonale. Notamment, il est important de noter que la parallélisation de la transformée de Hough hexagonale n'affecte pas la précision de la détection.

Cependant, malgré ses avantages, la transformée de Hough hexagonale n'est pas exempte de limitations. Ce sont : la perte d'informations présentes dans l'ensemble complet des pixels allumés des cellules hexagonales, la sensibilité aux taux α de pixels allumés et à la taille des cellules hexagonales, la complexité de sa mise en œuvre et la dépendance au contexte. Dans le chapitre suivant, nous aborderons la transformée de Hough octogonale.

106

5

Ch
ap

itr
e

Transformée de Hough Octogonale

Introduction	107
1 Description de la méthode	108
2 Complexité de la transformée de Hough octogonale	113
3 Simulations et discussions	115
Conclusion	126

Introduction

L'analyse et la détection de structures linéaires dans les images jouent un rôle crucial dans de nombreuses applications, allant de la vision par ordinateur à la robotique en passant par la cartographie. Parmi les techniques les plus utilisées pour cette tâche, la Transformée de Hough (TH) a longtemps été un pilier fondamental. Cependant, l'adaptation de la transformée de Hough à des besoins spécifiques et la recherche de performances optimales sont toujours des défis importants.

Dans cette analyse, nous explorons une variante novatrice de la transformée de Hough, connue sous le nom de transformée de Hough octogonale (THO), qui vise à surmonter certaines des limitations de l'approche standard. La transformée de Hough octogonale propose une approche basée sur une grille octogonale virtuelle, offrant ainsi une plus grande flexibilité dans la manipulation des paramètres et une réduction potentielle de temps de traitement par rapport à la THS.

Dans ce chapitre, nous examinons en détail l'algorithme de la transformée de Hough octogonale, en mettant en évidence ses principes fondamentaux, sa méthodologie de détection de lignes, et les mécanismes qui sous-tendent son fonctionnement. Nous explorons également les implications pratiques de la transformée de Hough octogonale à travers des simulations sur une image d'immeuble et une image d'autoroute de la figure 1.11, en comparant ses performances avec celles de la transformée de

107

Hough. Aussi, nous proposons une version parallélisée de la transformée de Hough octogonale dans l'optique de réduire d'avantage le temps de traitement.

Notre analyse se concentre sur plusieurs aspects clés, notamment la flexibilité des paramètres, la réduction du temps de calcul, la qualité de la détection et les perspectives d'amélioration. En examinant ces aspects, nous visons à évaluer le potentiel de la THO en tant qu'approche alternative pour la détection de lignes dans les images, et à identifier les domaines où elle pourrait offrir des avantages par rapport aux méthodes.

1 Description de la méthode

Dans cette section, nous présentons en détail la technique de la transformée de Hough octogonale dont les étapes sont décrites dans la figure 5.4. Nous commençons par décrire l'algorithme de maillage octogonal, qui permet de diviser une image en cellules régulières pour une analyse plus précise. Ensuite, nous abordons le processus de sélection des cellules en fonction du taux de pixels activés à l'intérieur de chaque cellule, suivi de la reconnaissance des droites discrètes grâce à la transformée de Hough octogonale. Chaque algorithme est accompagné d'explications détaillées, d'illustrations visuelles et de formules mathématiques pour une compréhension complète du processus. Enfin, nous présentons une méthode pour visualiser graphiquement les droites identifiées, facilitant ainsi l'interprétation des résultats obtenus.

1.1 Maillage de l'image

L'algorithme 27 est l'algorithme de maillage octogonale. Il prend en paramètre les dimensions de l'image (Nl : la hauteur de l'image, Nc : la largeur de l'image), la taille y de la cellule octogonale et retourne un tableau de 3 entiers dont la première valeur est la taille de la cellule octogonale et les deux dernières valeurs sont les résidus respectivement de la hauteur et de la largeur de l'image.

Une cellule octogonale (figure 5.1) est localisée par ses huit (8) sommets A, B, C, D, E, F, G et H.

Les coordonnées de chaque sommet est fonction de y comme illustré dans la deuxième boucle itérative "pour" de l'algorithme 30.

1.2 Sélection des cellules

Le critère de sélection d'une cellule octogonale au sein de cette grille repose sur le taux de pixels activés à l'intérieur de cette cellule octogonale particulier. Ce taux est déterminé à l'aide de l'algorithme 29, qui vise à calculer le taux de pixels allumés dans une région octogonale délimitée par les points spécifiés. Il prend en entrée une image (img) et les coordonnées des points délimitant l'octogone (A, B, D, E, F, G, H, I, J). L'algorithme initialise deux compteurs, k et l, qui représentent respectivement

Algorithme 27 : Construction de la grille octogonale
Fonction maillage oct(image, y : entiers) : tableau d'entiers

y représente la taille des mailles octogonales;
Variables : tab : tableau de 3 entiers ;
Sorties : tab : tableau de 3 entiers;

Debut

```
NI ← la hauteur de l'image;
Nc ← la largeur de l'image ;
tab[0] ← y ; % La taille de l'octogone.
tab[1] ← NI mod (3y - 2);
tab[2] ← Nc mod (3y - 2);
Retourner : tab;
```

fin
fin

y

x

A

B C

D

E

FG

H

I J

KL

Y

P

N

Q

M

Figure 5.1 – Octogone ABCDEFGH

le nombre de pixels allumés et éteints dans la région octogonale. En parcourant les lignes de l'image, il examine les pixels à l'intérieur de l'octogone et calcul le pourcentage ou le taux de pixels allumés dans la cellule octogonale. Finalement, l'algorithme retourne le taux de pixels allumés dans la région octogonale, calculé comme la division de k par la somme de k et l. Ce taux donne une mesure de la proportion de pixels allumés par rapport au total dans la cellule octogonale. Le volume d'une cellule octogonale est calculé selon le lemme 3.

Lemme 3 (L'aire d'une cellule octogonale) Soit y la taille d'un octogone. La surface de l'octogone, représentée par la partie verte de la figure 5.3, est calculée à l'aide de la formule suivante :

$SO(y) = 7y^2 - 4y + 1$ (5.1)

Preuve 3 (lemme 3) Considérons y comme la taille de l'octogone (5.1).

Calcul de la surface $Sc(y)$ du carré dans lequel l'octogone est inscrit :

$Sr(y) = (3y - 1)^2$ où $(3y - 1)$ représentent le coté du carré.

Calcul de la surface $St(y)$ des quatre triangles rectangles (dont les angles droits correspondent aux sommets du carré) : $St(y) = 4(y^2 - y^2 + y^2)$ où $y^2 + y^2$

2 représente la surface du triangle ABI.

Calcul de la surface de l'octogone : $SO(y) = Sc(y) - St(y)$. Ce qui donne, après développement et simplification : $SO(y) = 7y^2 - 4y + 1$

Par exemple, pour $y = 4$, $SO(4) = 7 \times 4^2 - 4 \times 4 + 1 = 97px^2$, comme illustré dans la figure 5.3.

L'unité de mesure de la surface est en pixels carrés (px^2).

y

x

Figure 5.2 – Grille octogonale superposée à l'espace image.

y

x

A

B C

D

E

FG

H

I J

KL

Figure 5.3 – Mise en évidence des pixels de l'octogone ABCDEFGH

110

Algorithme 28 : Mise à jour de l'accumulateur

Procédure Acc update() : vide

% Mise à jour de l'accumulateur

if img[z, ϑ] = 0 then

for itheta dans range(accum row) do

theta ← itheta × dtheta;

rho ← tx cos(theta) + z sin(theta);

irho ← int(rho);

if irho > 0 et irho < accum column then

accum[itheta][irho] ← accum[itheta][irho] + 1

end

end

end

fin

1.3 Parallélisation de la transformée de Hough octogonale

La parallélisation de la transformée de Hough octogonale nécessite une adaptation de l'algorithme

pour tirer parti des capacités de traitement simultané offertes par le parallélisme. La parallélisation

concerne l'accumulateur c'est-à-dire l'espace des paramètres encore appelé l'espace de Hough. Les al-

gorithmes 32 et 31 sont les versions parallélisées des algorithmes 30 et 28. Les étapes principales de la

parallélisation sont :

- importation de la bibliothèque pypm 5 : Importer une bibliothèque en Python est nécessaire pour

accéder aux fonctionnalités étendues que ces bibliothèques fournissent. L'instruction import pypm est

utilisée pour charger la bibliothèque pypm.

- création de l'accumulateur de type pypm : Dans l'algorithme séquentiel, l'espace des paramètres

(l'accumulateur) est un simple tableau à double entrées. L'accumulateur, fondamental dans la trans-

formée de Hough, est transformé en une structure de données partagée grâce à pypm. En effet, pypm

permet de créer et de gérer des tableaux partagés entre différents threads, ce qui peut être utile pour

éviter les conflits de mémoire dans des environnements parallèles.

L'instruction "accum pypm ← pypm.shared.array([accum row, accum column])" de l'algorithme 32

crée un tableau partagé à double entrée dont tous les éléments sont initialisés à 0 et l'affecte à la variable

"accum pypm".

- construction parallèle : L’instruction “with pypm.Parallel(4) as p” de l’algorithme 32 indique que

nous souhaitons exécuter le bloc de code à l’intérieur de la déclaration “with” en parallèle avec 4

processus.

- parallélisation de l’algorithme 28 de mise à jours de l’accumulateur : La fonction “p.range” génère une

séquence d’indices à partir de laquelle chaque cœur du CPU obtiendra une plage d’indices spécifique.

Cela permet de diviser le travail entre les cœurs du CPU de manière équitable. Ainsi la boucle “for

5. <https://github.com/classner/pypm/tree/main>

111

Algorithme 29 : Taux de pixels allumes

Fonction count oct(img : image, A, B, D, E, F, G, H, I, J : tableaux de deux entiers) : réel

Variables : k, l : entiers;

Debut

k ← 0 ; l ← 0;

pour A[1] ≤ y ≤ D[1] faire

si A[1] ≤ y ≤ I[1] − 1 alors

NP ← |y − A[1]|;

pour A[0] − NP ≤ x ≤ H[0] + NP faire

si img[x, y] = 0 alors

k ← k + 1 ;

sinon

l ← l + 1 ;

finsi

fin

finsi

si I[1] ≤ y ≤ J[1] alors

pour B[0] ≤ x ≤ G[0] faire

si img[x, y] = 0 alors

k ← k + 1 ;

sinon

l ← l + 1 ;

finsi

fin

finsi

si J[1] ≤ y ≤ D[1] alors

MQ ← |y − D[1]|;

pour D[0] − MQ ≤ x ≤ E[0] + MQ faire

si img[x, y] = 0 alors

k ← k + 1 ;

sinon

l ← l + 1 ;

finsi

fin

finsi

fin

Retourner :

k

k + l

;

fin

fin

itheta in p.range(accum row)” de l’algorithme 31 utilise les indices compris entre 0 et accum row − 1

pour itérer à travers l’accumulateur par sections, où chaque section est traitée par un processus

différent.

1.4 Reconnaissance des droites discrètes

L'algorithme 30 de la "Reconnaissance de droites discrètes par la transformée de Hough octogonale" utilise une approche octogonale pour détecter des droites discrètes dans une image. La fonction principale, appelée Octogonal, prend en entrée une image (img), un paramètre γ représentant la taille de l'octogone, un paramètre α représentant le taux de pixels allumés et le seuil de vote.

L'algorithme commence par initialiser les variables nécessaires, notamment les tableaux pour les points de l'octogone (A, B, C, D, E, F, G, H, I, J, K, L), les dimensions de l'image, et la matrice d'accumulateur. Ensuite, il effectue un maillage octogonal de l'image à l'aide de la fonction maillage oct (Algorithme 27).

Le processus de détection des droites discrètes se fait en parcourant les octogones de la grille générée. Pour chaque octogone, il vérifie s'il y a suffisamment de points allumés dans la région correspondante et met à jour l'accumulateur en conséquence.

Le cœur de l'algorithme se trouve dans la double boucle for qui parcourt les positions des octogones dans l'image. Pour chaque position, il calcule les coordonnées des points de l'octogone, vérifie le nombre de points allumés à l'intérieur de l'octogone, et met à jour l'accumulateur. La mise à jour de l'accumulateur se fait à l'aide de l'algorithme 28.

112

Algorithme 30 : Reconnaissance de droites discrètes par la transformée de Hough octogonale

Fonction Octogonal(imge, γ , α , seuil) : tableau
Pre-condition : $1 \leq \gamma \leq \min(NI+1$

$2, Nc+1$
 $2)$, $0 \leq \alpha \leq 1$, $0 < \text{seuil}$

Donnees : α
Variables : tab : tableau de 3 entiers ; A, B, C, D, E, F, G, H, I, J, K, L : tableaux de deux entiers;
accum ligne, accum colon, irho, NI, Nc : entier ; accum : matrice accumulateur ;
dtheta, drho, rho, theta, DE : réels;
Debut

% dimension de l'image
NI ← la hauteur de l'image ; Nc ← la largeur de l'image;
% dimensions de la matrice 'accum'
accum ligne ← 180 ; accum column ← E[

$\sqrt{}$
($Nc^2 + NI^2$);

dtheta = $\pi/180$;
tab ← maillage oct(NI, Nc, γ) ;
Ht = NI - tab[1] ; La = Nc - tab[2] ;
H1 = Ht + 3 * gamma - 2 ; L1 = La + 3 * gamma - 2 ;
pour $3\gamma - 2 \leq y \leq L1$ par pas de $3\gamma - 1$ faire

pour $3\gamma - 2 \leq x \leq H1$ par pas de $3\gamma - 1$ faire
A[0] = $x - 2\text{tab}[0] + 1$; A[1] = $y - 3\text{tab}[0] + 2$; B[0] = $x - 3\text{tab}[0] + 2$; B[1] = $y - 2\text{tab}[0] + 1$; C[0] = $x - 3\text{tab}[0] + 2$;

C[1] = $y - \text{tab}[0] + 1$; D[0] = $x - \text{tab}[0] + 1$; D[1] = y ;
E[0] = $x - \text{tab}[0] + 1$; E[1] = y ; F[0] = x ; F[1] = $y - \text{tab}[0] + 1$; G[0] = x ; G[1] = $y - 2\text{tab}[0] + 1$;
H[0] = $x - \text{tab}[0] + 1$; H[1] = $y - 3\text{tab}[0] + 2$;
I[0] = $x - 2\text{tab}[0] + 1$; I[1] = $y - 2\text{tab}[0] + 1$; J[0] = $x - 2\text{tab}[0] + 1$; J[1] = $y - \text{tab}[0] + 1$;
K[0] = $x - \text{tab}[0] + 1$; K[1] = $y - \text{tab}[0] + 1$; L[0] = $x - \text{tab}[0] + 1$; L[1] = $y - 2\text{tab}[0] + 1$;
si comptage(img, A, B, D, E, F, G, H, I, J) $\geq \alpha$ alors

pour A[1] $\leq j \leq D[1]$ faire
si A[1] $\leq j \leq I[1] - 1$ alors

NP ← $|j - A[1]|$;
pour A[0] - NP $\leq i \leq H[0] + NP$ faire

Acc update();
fin

finsi

```
si  $l[1] \leq j \leq J[1]$  alors
```

```
pour  $B[0] \leq i \leq G[0] + 1$  faire  
Acc update();
```

```
fin  
finsi  
si  $J[1] \leq j \leq D[1]$  alors
```

```
 $MQ \leftarrow |j - D[1]|$  ;  
pour  $D[0] - MQ \leq i \leq E[0] + MQ$  faire
```

```
Acc update();  
fin
```

```
finsi  
fin
```

```
finsi  
fin
```

```
fin  
fin  
Rechercher les maxima dans l'accumulateur et placer le couple  $(\rho, \theta)$  dans une liste 'lignes';  
Retourner la liste 'lignes';
```

```
fin
```

Enfin, il retourne une liste appelée 'lignes' contenant les couples $(\theta; \rho)$ correspondant aux paramètres des droites détectées.

L'objectif de l'algorithme 33 est de représenter graphiquement les droites identifiées par la méthode de la Transformée de Hough. Il nécessite en entrée une image (img), la liste des paramètres de droites détectées (lignes) générée par l'algorithme 30, ainsi qu'une couleur (colors). L'algorithme itère à travers la liste des paramètres de droites détectées, reconstruisant chaque droite correspondante sur l'image. Les droites reconstruites sont alors colorées avec la nuance spécifiée par colors.

Cette étape de reconstruction vise à mettre en évidence les droites identifiées de manière visuelle sur l'image originale. Cela facilite l'interprétation et l'analyse des résultats obtenus grâce à la Transformée de Hough.

2 Complexité de la transformée de Hough octogonale

La fonction maillage oct dans l'algorithme 27 récupère les dimensions d'une image et assigne des valeurs à un tableau, puis retourne ce tableau. Chaque opération est de coût constant, donc la

113

Algorithme 31 : Mise à jour parallèle de l'accumulateur
Procédure Acc updateP() : vide

```
% Mise à jour de l'accumulateur  
if img[z, t] = 0 then
```

```
for itheta dans p.range(accum row) do  
theta ← itheta × dtheta;  
rho ← t × cos(theta) + z × sin(theta);  
irho ← int(rho);  
if irho > 0 et irho < accum column then
```

```
accum pypm[itheta][irho] ← accum pypm[itheta][irho] + 1  
end
```

```
end  
end
```

```
fin
```

complexité totale de la fonction est $O(1)$.

La fonction count oct dans l'algorithme 29 : initialise des variables k et l, puis elle parcourt les pixels à l'intérieur de l'octogone à l'aide de boucles dont le nombre d'itérations est $7y^2 - 4y + 1$. Le coût des opérations à l'intérieur des boucles est constant. Donc la complexité du bloc formé par les

boucles est $O(7\gamma^2 - 4\gamma + 1) = O(\gamma^2)$, où $7\gamma^2 - 4\gamma + 1$ est le nombre de pixels à l'intérieur de l'octogone.

Ainsi, la complexité totale de la fonction count oct est $O(\gamma^2)$.

La fonction Acc update dans l'algorithme 28 commence par un test de condition, dont le coût est constant. Ensuite, elle parcourt une boucle dont le nombre d'itérations est constant et dépend uniquement de la valeur de accum row = 180, donc sa complexité est $O(1)$. À l'intérieur de cette boucle, les opérations restent constantes. Ainsi, la complexité totale de la fonction Acc update est $O(1)$.

L'algorithme 30 de la transformée de Hough octogonale (THO) commence par l'initialisation des variables et du tableau tab, ainsi que le calcul des dimensions de l'image et de l'accumulateur, toutes ces opérations ont un coût constant, donc $O(1)$. Ensuite, il appelle la fonction maillage oct dont la complexité est $O(1)$ comme expliqué précédemment. À l'intérieur des boucles principales, il y a deux boucles imbriquées parcourant les points à l'intérieur de chaque octogone à l'intérieur du quelle se trouve des opérations de coût constant et la fonction Acc update de complexité $O(1)$. La complexité de ces boucles est donc $O(7\gamma^2 - 4\gamma + 1) = O(\gamma^2)$

Le coût des deux boucles principales imbriquées parcourant les hexagones de la grille, dont le nombre total d'itérations dépend de la taille de l'image et de la taille des cellules hexagonales, contenant un bloc d'opérations d'affectation de coût constant, un test de condition sur la fonction count oct de complexité $O(\gamma^2)$, les deux boucles imbriquées de coût $7\gamma^2 - 4\gamma + 1$ parcourant les pixels à l'intérieur de l'hexagone, est $NI/(3\gamma - 1) \times Nc/(3\gamma - 1) \times (1 + (7\gamma^2 - 4\gamma + 1) + 1 + (7\gamma^2 - 4\gamma + 1))$, donc de complexité $O(NI/(3\gamma - 1) \times Nc/(3\gamma - 1) \times (1 + (7\gamma^2 - 4\gamma + 1) + 1 + (7\gamma^2 - 4\gamma + 1))) = O(NI \times Nc)$.

En considérant ces points, la complexité totale de l'algorithme 30 de la THO est égal à $O(NI \times Nc)$,

114

Algorithme 32 : Reconnaissance de droite discrètes par la Transformée de Hough Octogonale Parallélisée

Fonction OctogonalP(imge,y,α,Threshold,cpu count) :tableau
Pre-condition : $1 \leq \gamma \leq \min(NI+1$

2 , Nc+1
2), $0 \leq \alpha \leq 1$, $0 < \text{seuil}$

Variables : tab : tableau de 3 entiers ; A, B, C,D, E, F, G, H, I, J, K, L : tableaux de deux entier;
accum row, accum column, irho, NI, Nc : entier ; accum : matrice accumulateur ;
dtheta, drho, rho, theta, DE : reels;
Debut

import pypm % importation de la bibliothèque pypm % dimension de l'image
NI ← la hauteur de l'image ; Nc ← la largeur de l'image;
% dimensions de la matrice 'accum'
accum row ← 180 ; accum column ← E[

√
(Nc2 + NI2)];

accum pypm ← pypm.shared.array([accum row, accum column]); % creation de l'accumulateur de type pypm
dtheta = π/180 ; tab ← maillage oct(NI, Nc, γ) ;
Ht = NI- tab[1] ; La = Nc- tab[2];
H1 = Ht + 3 * gamma- 2 ; L1 = La + 3 * gamma- 2;
% Démarrage du traitement parallèle.
With pypm.parallel(cpu count) as p :

pour $3\gamma - 2 \leq y \leq L1$ par pas de $3\gamma - 1$ faire
pour $3\gamma - 2 \leq x \leq H1$ par pas de $3\gamma - 1$ faire

A[0] = x- 2tab[0] + 1 ; A[1] = y - 3tab[0] + 2 ; B[0] = x- 3tab[0] + 2 ; B[1] = y - 2tab[0] + 1 ;
C[0] = x- 3tab[0] + 2 ; C[1] = y - tab[0] + 1 ; D[0] = x- tab[0] + 1 ; D[1] = y ;

E[0] = x- tab[0] + 1 ; E[1] = y ; F [0] = x ; F [1] = y - tab[0] + 1 ; G[0] = x ; G[1] = y - 2tab[0] + 1 ;
H[0] = x- tab[0] + 1 ; H[1] = y - 3tab[0] + 2 ;
I[0] = x- 2tab[0] + 1 ; I[1] = y - 2tab[0] + 1 ; J[0] = x- 2tab[0] + 1 ; J[1] = y - tab[0] + 1 ;
K[0] = x- tab[0] + 1 ; K[1] = y - tab[0] + 1 ; L[0] = x- tab[0] + 1 ; L[1] = y - 2tab[0] + 1 ;
si comptage(img, A,B, D, E, F, G, H, I, J) ≥ α alors

```
pour A[1] ≤ j ≤ D[1] faire
si A[1] ≤ j ≤ I[1]- 1 alors

NP ← |j - A[1]|;
pour A[0]-NP ≤ i ≤ H[0] + NP faire

Acc updateP();
fin

finsi
si I[1] ≤ j ≤ J[1] alors

pour B[0] ≤ i ≤ G[0] + 1 faire
Acc updateP();

fin
finsi
si J[1] ≤ j ≤ D[1] alors

MQ ← |j -D[1]| ;
pour D[0]-MQ ≤ i ≤ E[0] + MQ faire

Acc updateP();
fin

finsi
fin

finsi
fin

fin
fin

fin
Rechercher les maxima dans l'accumulateur et placer les couple (p, θ) dans une liste 'lignes';
Retourner la liste 'lignes';

fin
```

Algorithme 33 : Reconstruction des droites détectées
Fonction PlotHoughLine(imge,lignes,colors) :image

Parcourir 'lignes' la liste des paramètres des droites détectés, puis construire chaque droite correspondant en couleur 'colors' sur l'image 'imge'.

fin

où NI et Nc sont le nombre de lignes et de colonnes de l'image respectivement, γ est la taille des cellules octogonale. Ce qui correspond à la complexité de la THS.

3 Simulations et discussions

La présente section se concentre sur les résultats de simulations obtenus à partir de l'application de l'algorithme de la transformée de Hough octogonale sur des images représentatives. Nous examinons en détail les performances de la transformée de Hough octogonale dans deux scénarios distincts : la détection de lignes dans une image d'immeuble et dans une image d'autoroute. Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin d'effectuer les simulations :

115

Figure 5.4 – Les étapes de la technique de la transformée de Hough octogonale

- Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz
- RAM : 8 Go
- Système d'exploitation : , 64 bits.

Pour chaque scénario, nous explorons les variations des paramètres clés de la transformée de Hough octogonale, tels que le seuil de vote, α et γ, afin d'évaluer leur impact sur la précision et l'efficacité de la détection. De plus, nous comparons les résultats de la transformée de Hough octogonale avec ceux de la transformée de Hough standard dans le contexte de la détection de lignes. Enfin, nous présentons une analyse comparative entre la transformée de Hough octogonale et sa version parallélisée (THOP),

mettant en lumière les avantages et les limites de chacune des approches. Ces discussions visent à fournir un aperçu complet des performances de la transformée de Hough octogonale dans différents contextes d'application et à identifier les meilleures pratiques pour son utilisation efficace.

116

Tableau 5.1 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres fixes ($\gamma = 3$ et Seuil = 60) et α variant entre 10% et 35%

$\alpha(\%)$	Nbre de droites détectées	temps(sec)
10	101	0,42
15	65	0,39
20	17	0,31
25	9	0,25
30	2	0,17
35	0	0,10

3.1 Cas de l'image de l'immeuble

Dans cette section, nous examinons les performances de l'algorithme de détection de lignes de la transformée de Hough octogonale appliqué à une image représentant un immeuble. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la transformée de Hough octogonale en fonction des caractéristiques de l'image de l'immeuble.

3.1.1 Variations de α

Le tableau 5.1 présente les résultats de simulations sur l'image 1.11(c) de l'immeuble en utilisant l'algorithme 30 de la transformée de Hough octogonale. Les simulations ont été réalisées avec des paramètres fixes ($\gamma = 3$ et Seuil = 60), tandis que le paramètre α a été modifié entre 10% et 35%. Le tableau comporte trois colonnes : La première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 35%. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation. Deux graphiques sont fournis sur la figure 5.5 : l'un représentant le nombre de lignes détectées en fonction de α , et l'autre représentant le temps de traitement en fonction de α . Dans le premier graphique, on observe une diminution du nombre de lignes détectées à mesure que α augmente. Cela suggère que l'augmentation de α conduit à une détection moins précise des lignes. Dans le deuxième graphique, le temps de traitement diminue à mesure que α augmente. Cela indique que des valeurs plus élevées de α conduisent à une exécution plus rapide de l'algorithme.

Une faible valeur de α (10%) donne un grand nombre de lignes détectées (101), mais cela nécessite plus de temps de traitement (0,42 sec). À mesure que α augmente, le nombre de lignes détectées diminue, le temps de traitement diminue également. Des valeurs trop élevées de α peuvent entraîner une détection défectueuse.

117

0.1 0.15 0.2 0.25 0.3 0.350

10

20

30

40

50

60

70

80

90

100
(101)

(65)

(17)

(9)
(2) (0)

α

de
te

ct
ed

lin
es

detected lines

0.1 0.15 0.2 0.25 0.3 0.355 · 10⁻²

0.1

0.15

0.2

0.25

0.3

0.35

0.4

0.45
(0.42)

(0.39)

(0.31)

(0.25)

(0.17)

(0.10)

α

T
im

e
(s

ec
on

de
)

Time

Figure 5.5 – Courbe représentative des données du tableau 5.1

Tableau 5.2 – Résultats des simulations sur l'image 1.11(c) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres fixes (Seuil = 50 et α = 20%) et γ variant entre 2 et 82

γ SO droites détectées temps(sec)
2 21 299 0,39
3 52 70 0,31
4 97 73 0,28
5 156 15 0,22
6 229 3 0,20
7 316 1 0,16
8 417 0 0,14
82 46 741 0 0,14

α γ SO(γ) Nbre de droites détectées temps(sec)
11 54 20197 31 0,22

3.1.2 Variations de la taille des cellules

Le tableau présente les résultats des simulations sur l'image 1.11(c) de l'immeuble en utilisant

l'algorithme 30 de la transformée de Hough octogonale. Les paramètres fixes sont Seuil = 50 et

α = 20%, tandis que γ varie entre 2 et 8. Les colonnes incluent γ, la surface octogonale SO, le nombre de lignes détectées, et le temps en secondes.

Les deux courbes présentées dans la figure 5.6 sont les représentations graphique des données du tableau 5.2. Le premier représente le nombre de lignes détectées en fonction de la surface octogonale, et le second représente le temps de traitement en fonction de la surface octogonale. Dans le premier graphique, on observe une tendance décroissante du nombre de lignes détectées pour les valeurs de γ allant de 2 à 82. Par contre, lorsque nous baissons la valeur de α à 11%, 31 droites est détectées avec γ = 54, suggérant ainsi que l'augmentation de la taille des cellules n'entraîne pas toujours la diminution du nombre de droites détectées. Dans le second graphique, le temps de traitement diminue également à mesure que la surface octogonale augmente, indiquant une exécution plus rapide de l'algorithme pour des valeurs plus élevées de γ.

118

21 52 97 156 229 316 4170

50

100

150

200

250

300
(299)

(70) (73)

(15)
(3) (1) (0)

SO

de
te

ct
ed

lin
es

detected lines

21 52 97 156 229 316 4170.14

0.16

0.18

0.2



Figure 5.6 – Courbe représentative des données du tableau 5.2

3.2 Cas de l'image de l'autoroute

Dans cette section, nous examinons les performances de l'algorithme de détection de lignes de la transformée de Hough octogonale appliqué à une image représentant une autoroute. L'objectif est d'explorer l'impact des paramètres clés de l'algorithme, à savoir γ et α , sur la précision de la détection des lignes et le temps de traitement. Pour ce faire, nous avons mené une série de simulations en variant ces paramètres dans des plages spécifiques, tout en maintenant les autres paramètres fixes. Les résultats sont présentés sous forme de tableaux et de graphiques, offrant ainsi une analyse détaillée des performances de l'algorithme dans ce contexte spécifique. Cette étude nous permettra de mieux comprendre les ajustements nécessaires pour optimiser l'algorithme de la transformée de Hough octogonale en fonction des caractéristiques de l'image de l'autoroute.

3.2.1 Variations de α

Le tableau 5.3 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute en utilisant l'algorithme 30 de la transformée de Hough octogonale. Les simulations ont été réalisées avec des paramètres fixes ($\gamma = 3$ et Seuil = 40), tandis que le paramètre α a été modifié entre 10% et 30%. Le tableau comporte trois colonnes : la première colonne représente les différentes valeurs du paramètre α , qui varient de 10% à 30%. La deuxième colonne montre le nombre de droites détectées pour chaque valeur de α . La dernière colonne présente le temps de traitement en secondes pour chaque simulation. Les deux courbes présentées dans la figure 5.7 sont les représentations graphique des données du

tableau 5.3. Le premier graphique illustre le nombre de lignes détectées en fonction de α , tandis que le second représente le temps de traitement en fonction de α . Dans le premier graphique, on constate une diminution du nombre de lignes détectées à mesure que α augmente, ce qui suggère une détection moins fréquente avec des valeurs plus élevées de α . Cependant, une augmentation exagérée des valeurs

119

Tableau 5.3 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la THO avec les paramètres fixes ($\gamma = 3$ et Seuil = 40) et α variant entre 10% et 30%

α (%) Nbre de droites détectées temps(sec)
 10 158 0,18
 15 121 0,16
 20 20 0,12
 25 2 0,09
 30 0 0,06

0.1 0.15 0.2 0.25 0.30

20

40

60

80

100

120

140

160 (158)

(121)

(20)

(2) (0)

α

de
te

ct
ed

lin
es

detected lines

0.1 0.15 0.2 0.25 0.34 · 10^{−2}

6 · 10^{−2}

8 · 10^{−2}

0.1

0.12

0.14

0.16

0.18

0.2

(0.18)

(0.16)

(0.12)

(0.09)

(0.06)

α

T
im

e
(s

ec
on

de
)

Time

Figure 5.7 – Courbe représentative des données du tableau 5.3

de α peut détériorer la qualité de la détection. Dans le second graphique, le temps de traitement diminue également à mesure que α augmente, indiquant une exécution plus rapide de l'algorithme pour des valeurs plus élevées de α .

3.2.2 Variations de la taille des cellules

Le tableau 5.4 présente les résultats de simulations sur l'image 1.11(a) de l'autoroute en utilisant l'algorithme 30 de la transformée de Hough octogonale. Les simulations ont été réalisées avec des paramètres fixes (Seuil = 40 et $\alpha = 20\%$) et γ variant entre 2 et 5. Le tableau 5.4 comporte quatre colonnes : la première colonne représente les différentes valeurs du paramètre γ , variant de 2 à 5. La deuxième colonne indique la surface (l'aire) SO de la cellule octogonale en pixels carrés. La troisième colonne montre le nombre de lignes détectées pour chaque valeur de γ . La dernière colonne présente le temps de traitement en secondes pour chaque simulation.

Les deux courbes dans la figure 5.8 représentent les données du tableau 5.4. Le premier graphique illustre le nombre de lignes détectées en fonction de la surface octogonale, tandis que le second montre le temps de traitement en fonction de cette même surface. Dans le premier graphique, on observe une diminution jusqu'à 0 du nombre de lignes détectées pour γ allant de 2 à 56. Cependant, lorsque nous baissions la valeur de α à 10% par exemple, 9 droites sont détectées lorsque $\gamma = 33$. Ce qui

120

Tableau 5.4 – Résultats des simulations sur l'image 1.11(a) de l'autoroute avec l'algorithme 30 de la THO avec les paramètres fixes (Seuil=40 et $\alpha = 20\%$) et γ variant entre 2 et 56

γ SO(γ) Nbre de droites détectées temps(sec)

2 21 95 0,16

3 52 20 0,12

4 97 2 0,09

5 156 0 0,08

56 21 729 0 0,04

α γ SO(γ) Nbre de droites détectées temps(sec)

10 33 7 492 9 0,09

12 34 7 957 12 0,09

21 52 97 1560

10

20

30

40

50

60

70

80

90

100(95)

(20)

(2) (0)

SO

de
te

ct
ed

lin
es

detected lines

21 52 97 1567 · 10⁻²

8 · 10⁻²

9 · 10⁻²

0.1

0.11

0.12

0.13

0.14

0.15

0.16

0.17

0.18

(0.16)

(0.12)

(0.09)

(0.08)

SO

T
im

e
(s

ec
on

de
)

Time

Figure 5.8 – Courbe représentative des données du tableau 5.4

montre que l'augmentation de la taille des cellules n'implique pas systématique la diminution du nombre de droites détectées. Dans le second graphique, le temps de traitement diminue également avec l'augmentation de la surface des cellules, indiquant une exécution plus rapide de l'algorithme pour des surfaces octogonales plus grandes.

Ainsi, ces résultats mettent en évidence l'importance des paramètres γ , α , seuil dans l'algorithme de détection de lignes de la transformée de Hough octogonale. Les cellules de grandes taille offre une précision acceptable et un temps de traitement plus rapide, en témoignent les données des tableaux

5.2 et 5.4, ainsi que les résultats visuels dans la figure 5.9.

3.3 Comparaison de la Transformée de Hough Octogonale et Transformée de

Hough Standard

Nous procédons dans cette section à une comparaison de la transformée de Hough octogonale, méthode proposée dans ce chapitre, et sa version standard sur deux images. Une image d'un immeuble et une image d'une autoroute comme illustré dans la figure 1.11.

Avec l'image de l'immeuble, les résultats des expériences avec la transformée de Hough standard

121

(a) (b)

Figure 5.9 – (a) THO sur l'immeuble ($\gamma = 54$, $\alpha = 11\%$, seuil = 50), (b) THO sur l'autoroute ($\gamma = 33$, $\alpha = 10$, seuil = 40)

Tableau 5.5 – Résultats de simulation avec la THS appliquée sur l'image 1.11(c) de l'immeuble (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

Threshold	Droites détectées	temps1(sec)	temps2(sec)
53	1991	0,6	0,0003
116	10	0,49	0,049

et la transformée de Hough octogonale sont illustré dans les tableaux 5.5 et 5.7 respectivement.

En ce qui concerne la précision, pour un seuil de 53 votes, la transformée de Hough standard a effectué une détection de 1991 droites. Cette détection est non pertinente comme le montre la première image de la figure 5.10. par contre, avec le même seuil de votes, la transformée de Hough octogonale a détecté 11 droites. Les résultats visuels illustrés par la première image de la figure 5.12 montre que les droites détectées suivent bien les lignes de l'immeuble suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough octogonale pour parvenir à ce résultat sont : la taille de cellule définit par γ et le pourcentage α de pixels allumés de chaque cellule octogonale.

Pour avoir une bonne détection de lignes droites avec la THS, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 5.5 montre 10 droites détectées avec un seuil de 116 votes. La deuxième image de la figure 5.10 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'immeuble indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 5.5 montre que la THS a détecté 10 droites en 0.49 seconde soit 0.049 seconde par droites. La THO a détecté quasiment le même nombre de droite en 0.22 seconde soit 0.02 seconde par droite détecté comme illustré dans le tableau 5.7. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée à travers le

calcul du facteur d'accélération noté A, où $A = \frac{T_s}{T_h}$

$$= \frac{0,49}{0,22} = 2,22$$
 (T_h représente le temps d'exécution

de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).

De même avec l'image de l'autoroute, les résultats des expériences avec la THS et la THO sont illustré dans les tableaux 5.6 et 5.8 respectivement.

En ce qui concerne la précision, pour un seuil de 38 votes, la THS a effectué une détection de 751

122

(a) (b)

Figure 5.10 – (a) Détection de lignes droites sur l'image 1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=40) ; (b) Détection de lignes droites sur l'image1.11(c) de l'immeuble avec l'algorithme 1 de la THS (Seuil=116)

Tableau 5.6 – Résultats de simulation avec la THS appliquée sur l'image 1.11(a) de l'autoroute (time1= temps de détection de toutes les droites ; time2=temps de détection d'une seule droite)

Threshold	Droites détectées	time1(sec)	time2(sec)
38	751	0,24	0,0003
100	10	0,21	0,021

droites. Cette détection est non pertinente comme le montre la première image de la figure 5.11. par contre, avec le même seuil de votes, la transformée de Hough octogonale a détecté 10 droites. Les résultats visuels illustrés par la première image de la figure 5.13 montre que les droites détectées suivent bien les lignes de l'autoroute suggérant des détections pertinentes.

Les paramètres supplémentaires utilisés par la transformée de Hough octogonale pour parvenir à ce résultat sont : la taille de cellule octogonale définit par γ et le pourcentage α de pixels allumés de chaque cellule octogonale.

Pour avoir une bonne détection de lignes droites avec la THS, il faut augmenter le seuil de votes. Par exemple, la deuxième ligne du tableau 5.6 montre 10 droites détectées avec un seuil de 100 votes. La deuxième image de la figure 5.11 illustre bien ces résultats. Les 10 droites détectées suivent bien les lignes de l'autoroute indiquant des détections pertinentes.

Pour ce qui est du temps de traitement, le tableau 5.6 montre que la THS a détecté 10 droites en 0.21 seconde soit 0.021 seconde par droites. La transformée de Hough octogonale a détecté le même nombre de droite en 0.11 seconde soit 0.011 seconde par droite détecté comme illustré dans le tableau 5.8. Ce qui suggère une net amélioration du temps de traitement. Cette amélioration est en outre quantifiée

à travers le calcul du facteur d'accélération noté A, où $A = \frac{T_s}{T_h}$

$$= \frac{0,21}{0,11} = 1,9 \text{ (} T_h \text{ représente le temps}$$

d'exécution de l'algorithme de la méthode proposée, tandis que T_s représente le temps d'exécution de l'algorithme standard).

Tableau 5.7 – Résultats de simulation avec la THO appliquée sur l'image 1.11(c) de l'immeuble (temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite détectée)

γ	SO(px2)	$\alpha(\%)$	seuil	Droites détectées	temps1(sec)	temps2(sec)
2	6	30	53	11	0.22	0.02
3	20	25	51	23	0.25	0.0092

123

(a) (b)

Figure 5.11 – (a) Détection de lignes droites sur l'image 1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=20) ; (b) Détection de lignes droites sur l'image1.11(a) de l'autoroute avec l'algorithme 1 de la THS (Seuil=100)

(a) (b)

Figure 5.12 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=53, $\alpha = 30\%$, $\gamma = 2$; (b) Seuil=51, $\alpha = 25\%$, $\gamma = 3$

3.4 Comparaison de la Transformée de Hough Octogonale et la Transformée de Hough Octogonale Parallélisée

3.4.1 Cas de l'image de l'immeuble

La figure 5.14 présente deux graphiques comparant les performances des algorithmes de la transformée de Hough octogonale et de la transformée de Hough octogonale parallélisée. Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et SO).

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 35%. Pour des valeurs de α plus basses (entre 10% et 28%), la courbe de la transformée de Hough octogonale parallélisée est inférieure à celle de la transformée de Hough octogonale, suggérant ainsi que la transformée de Hough octogonale parallélisée est plus performante pour des valeurs de α plus faibles. En revanche, pour des valeurs de α plus élevées (entre 28% et 35%), la courbe de la transformée de Hough octogonale parallélisée passe au-dessus de celle de la transformée de Hough octogonale, indiquant que la transformée de Hough octogonale parallélisée est moins performante pour des valeurs de α plus élevées.

Dans le deuxième graphique, les variations du temps d'exécution sont comparées en fonction de la surface des cellules hexagonales, avec les deux algorithmes de la THO et de la THOP. Les surfaces des cellules varient de 21 à 417 px2. La courbe de la transformée de Hough octogonale parallélisée est en

Tableau 5.8 – Résultats de simulation avec la THO appliquée sur l'image 1.11(a) de l'autoroute (temps1 = temps mis de toutes les droites détectées ; temps2 = temps mis d'une droite détectée)

y	SO(px2)	α (%)	seuil	Droites détectées	temps1(sec)	temps2(sec)
2	6	30	38	10	0.11	0.011
3	20	20	40	20	0.12	0.006

124

(a) (b)

Figure 5.13 – Détection de lignes droites sur l'image 1.11(a) de l'immeuble avec l'algorithme 30 de la THO avec les paramètres : (a) Seuil=38, α = 30% gamma = 2 ; (b) Seuil=40, α = 20%, gamma = 3

0.1 0.15 0.2 0.25 0.3 0.355 · 10⁻²

0.1

0.15

0.2

0.25

0.3

0.35

0.4

0.45

(0.42)

(0.39)

(0.31)

(0.25)

(0.17)

(0.10)

(0.33)

(0.28)

(0.24)

(0.21)

(0.18)

(0.15)

α

T

im

e

(s

ec
on

de
)

THO
THOP

21 52 97 156 229 316 417

0.15

0.2

0.25

0.3

0.35

0.4

0.45

(0.39)

(0.31)

(0.28)

(0.22)
(0.20)

(0.16)
(0.14)

(0.29)

(0.24)

(0.21)
(0.20)

(0.19) (0.19)

(0.12)

SO

T
im

e
(s

ec
on

de
)

THO
THOP

Figure 5.14 – Cas de l'immeuble : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (seuil=100, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (seuil=70, $\alpha = 20\%$) des algorithmes 30 et 32

dessous de celle de transformée de Hough octogonale pour les surfaces inférieures à 229. Aux alentours de 229, la courbe de THOP passe au-dessus de celle de THO. Cela suggère que la transformée de Hough octogonale parallélisée est plus performante que la transformée de Hough octogonale pour les petites valeurs de SO, mais moins performante pour les valeurs élevées de SO.

3.4.2 Cas de l'image de l'autoroute

La figure 5.15 présente deux graphiques comparant les performances des algorithmes de transformée de Hough octogonale (THO) et de transformée de Hough octogonale parallélisée (THOP). Les graphiques comparent les variations du temps d'exécution en fonction de différents paramètres (α et

50).

Dans le premier graphique, les variations du temps d'exécution sont comparées en fonction du paramètre α , avec des valeurs allant de 10% à 35%. Pour les petites valeurs de α (entre 10% et 15%), la courbe de la transformée de Hough octogonale parallélisée est en dessous de celle de la transformée de Hough octogonale. Cela indique que la transformée de Hough octogonale parallélisée est plus performant avec les petites valeurs de α . Pour les valeurs plus élevées de α (entre 15% et 30%), la courbe de la transformée de Hough octogonale parallélisée passe au dessus de celle de la transformée de Hough octogonale. Cela suggère que la transformée de Hough octogonale parallélisée

125

0.1 0.15 0.2 0.25 0.34 · 10⁻²

6 · 10⁻²

8 · 10⁻²

0.1

0.12

0.14

0.16

0.18

0.2

0.22

0.24

(0.18)

(0.16)

(0.12)

(0.09)

(0.06)

(0.17)

(0.16)

(0.14) (0.14)

(0.12)

α

T

im

e

(s

ec

on

de

)

THO

THOP

21 52 97 1564 · 10⁻²

6 · 10⁻²

8 · 10⁻²

0.1

0.12

0.14

0.16

0.18

0.2

(0.16)

(0.12)

(0.09)

(0.08)

(0.15)

(0.12)

(0.13)

(0.14)

SO

T
im

e
(s

ec
on

de
)

THO
THOP

Figure 5.15 – Cas de l'autoroute : (à gauche) Comparaison des variations du temps d'exécution en fonction de α et les paramètres (threshold=70, $\gamma = 4$), (à droite) Comparaison des variations du temps d'exécution en fonction de la surface des cellules et les paramètres (threshold=50, $\alpha = 0, 2$) des algorithmes 30 et 32

est moins performante avec des valeurs de α plus élevées.

Dans le deuxième graphique (à droite), les variations du temps d'exécution sont comparées en

fonction de la surface des cellules octogonales, avec les deux algorithmes de la THO et de la THOP.

Les surfaces des cellules varient de 21 à 156 px2. La courbe de la transformée de Hough octogonale parallélisée est en dessous de celle de la transformée de Hough octogonale pour les surfaces inférieure à 52. À partir de 52, la courbe de la transformée de Hough octogonale parallélisée passe au-dessus de la courbe de la transformée de Hough octogonale. Cela suggère que la transformée de Hough octogonale parallélisée est plus performante que la transformée de Hough octogonale avec les petites valeurs de SO et moins performante que la transformée de Hough octogonale avec des valeurs plus de SO.

Ainsi, l'analyse des deux graphiques met en lumière les performances relatives de la transformée de Hough octogonale et de sa version parallèle (THOP) en fonction de α , de la taille des cellules octogonales et du seuil de vote. Pour les petites valeurs de α , la transformée de Hough octogonale parallélisée est plus rapide que la transformée de Hough octogonale. Cette performance bascule rapidement en faveur de la transformée de Hough octogonale à mesure que les valeurs de α augmentent. Il en est de même, en ce qui concerne la taille des cellules octogonales, la THOP est plus rapide par rapport à la transformée de Hough octogonale avec les cellules de petites tailles. Cependant, lorsque la taille des cellules devient plus grande, la THO devient plus performante.

Conclusion

Dans cette étude, nous avons examiné la transformée de Hough octogonale en tant qu'approche novatrice pour la détection de lignes droites dans les images. En comparaison avec la transformée de

Hough standard, la transformée de Hough octogonale offre plusieurs avantages potentiels, notamment une réduction du temps de calcul et une qualité de détection raisonnable. De plus, en diminuant le nombre de pixels à calculer grâce à la variable α , la transformée de Hough octogonale offre des économies computationnelles, ce qui peut être avantageux pour le traitement d'images à grande échelle. En ajustant les paramètres tels que γ , α et le seuil de vote, les simulations ont démontré que la transformée de Hough octogonale peut réduire le temps de calcul tout en maintenant une qualité de détection acceptable.

L'analyse comparative de transformée de Hough octogonale et sa version parallélisée a montré que la THOP est plus performante avec les petites cellules, tandis que la THO est préférable pour des cellules plus grandes. Ainsi, le choix entre les deux dépend des valeurs spécifiques de α et de la taille des cellules.

Néanmoins, même si la transformée de Hough octogonale présente des avantages, elle comporte des limitations. Celles-ci incluent la perte d'informations provenant de l'ensemble complet des pixels allumés des cellules octogonales, la sensibilité aux taux α de pixels allumés et à la taille des cellules octogonales, la complexité de sa mise en œuvre, et sa dépendance au contexte. Dans le chapitre suivant, nous aborderons la parallélisation de la transformée de Hough généralisée pour la reconnaissance d'empreintes digitales.

127

6

Ch
ap

itr
e

Reconnaissance d'Empreintes

Digitales dans une Grille Virtuelle

Introduction	128
1 La biométrie	129
2 Les empreintes digitales	130
3 Structure d'un système de reconnaissance d'empreintes digitales	131
4 Méthodologie	136
5 Expérimentations et discussions	138
Conclusion	142

Introduction

L'identification biométrique, qui repose sur des caractéristiques physiologiques ou comportementales uniques, représente un domaine clé de la sécurité et des nouvelles technologies de l'information et de la communication. Parmi les nombreuses techniques biométriques, la reconnaissance des empreintes digitales occupe une place prépondérante en raison de sa fiabilité et de sa large adoption dans divers domaines, allant de la sécurité des bâtiments à la lutte contre la fraude. Cependant, la mise en œuvre d'un système de reconnaissance d'empreintes digitales efficace présente des défis, notamment en ce qui concerne le temps de traitement, en particulier pour les bases de données volumineuses. Pour relever ces défis, les chercheurs explorent des approches innovantes telles

que la programmation parallèle, qui permet d'exploiter les ressources matérielles et informatiques pour accélérer les calculs.

128

Dans ce chapitre, notre étude se concentre sur l'intégration des grilles virtuelles dans le processus d'appariement dans le cadre de la reconnaissance d'empreintes digitales. Cette approche promet d'améliorer non seulement les performances en termes de temps de traitement, mais également la précision dans l'identification des empreintes digitales. Nous avons proposé également une approche hybride qui consiste à combiner le parallélisme aux grilles virtuelles.

Dans la suite du chapitre, nous présenterons d'abord un aperçu de la biométrie et des systèmes biométriques, en mettant en lumière l'importance des empreintes digitales dans ce domaine. Ensuite, nous explorerons en détail la structure d'un système complet de reconnaissance d'empreintes digitales, en mettant l'accent sur les étapes clés telles que l'acquisition, le pré-traitement, l'extraction des minuties et la comparaison. Nous discuterons également des différentes approches de comparaison des minuties, en mettant en évidence leurs avantages et leurs inconvénients. Par la suite, nous détaillerons notre méthode proposée, qui intègre la programmation parallèle combiné à une grille rectangulaire virtuelle pour accélérer le processus d'appariement des empreintes digitales. Nous présenterons ensuite les résultats de nos expérimentations. Enfin, nous conclurons en discutant des implications de nos résultats.

1 La biométrie

1.1 Définition

La biométrie est une discipline qui se concentre sur l'identification et l'authentification des individus en se basant sur des caractéristiques physiologiques ou comportementales uniques. Ces caractéristiques peuvent inclure des traits physiques tels que les empreintes digitales, la reconnaissance faciale, l'iris ou la rétine de l'œil, ainsi que des traits comportementaux tels que la voix ou la manière de taper sur un clavier. Ainsi, la biométrie permet d'authentifier l'identité des individus de manière plus fiable que les méthodes classiques telles que les mots de passe ou les cartes d'identité. Elle est largement utilisée dans divers domaines tels que la sécurité, les systèmes d'accès aux bâtiments, les contrôles aux frontières, la lutte contre la fraude et même dans certains dispositifs de paiement.

1.2



10

Document d'un autre utilisateur

Le document provient d'un autre groupe

Les systèmes biométriques

Un système biométrique est fondamentalement un système de reconnaissance de formes qui opère

en collectant des données biométriques à partir d'un individu, en extrayant un ensemble de caractéristiques de ces données, puis en les comparant à une signature stockée dans une base de données.

Selon le contexte d'utilisation, un système biométrique peut fonctionner soit en mode de vérification, où l'identité d'une personne est vérifiée par rapport à une signature préalablement enregistrée, soit en mode d'identification, où l'identité de la personne est recherchée dans une base de données pour trouver une correspondance.

129

Généralement, tous les systèmes biométriques suivent un schéma de fonctionnement commun, composé des deux processus principaux suivants :

- Processus d'enregistrement : Ce processus vise à capturer et à enregistrer les caractéristiques biométriques

des utilisateurs dans une base de données. Pendant cette étape, les données biométriques uniques de chaque individu, telles que les empreintes digitales, les traits du visage ou les schémas vocaux, sont collectées et stockées de manière sécurisée pour une utilisation ultérieure.

- Processus d'identification/vérification : Ce processus intervient lorsque quelqu'un cherche à s'identifier ou à être vérifié par le système biométrique. Lorsqu'une personne présente ses caractéristiques biométriques au système, celles-ci sont comparées à celles enregistrées dans la base de données. Dans le processus d'identification, le système recherche l'ensemble de la base de données pour trouver une correspondance avec les caractéristiques présentées. Dans le processus de vérification, la comparaison se fait avec les données biométriques spécifiques de l'utilisateur qui se présente, afin de vérifier son identité.

2 Les empreintes digitales

2.1 Historique

Les empreintes digitales ont une longue histoire d'utilisation dans différentes cultures à travers le monde. Les premières traces de leur utilisation remontent à l'Égypte ancienne, datant de plus de 4000 ans, et les Chinois les utilisaient également dès le troisième siècle avant Jésus-Christ pour signer des documents officiels, bien qu'ils ne comprennent pas alors leur unicité pour chaque individu [102]. Ce n'est qu'au XIXe siècle que des progrès significatifs ont été réalisés dans la compréhension scientifique des empreintes digitales. En 1856, l'anglais William Herschel [5], après avoir observé l'utilisation des empreintes digitales comme signature sur des documents par la population indienne qu'il dirigeait, a commencé à reconnaître leur unicité et leur constance dans le temps. Puis, en 1888, le britannique Francis Galton a publié une étude détaillée sur les empreintes digitales, établissant leurs caractéristiques uniques.

L'adoption officielle des empreintes digitales dans le système judiciaire anglais a eu lieu en 1894, et cette technique a ensuite été largement développée dans les enquêtes criminelles, devenant un outil essentiel pour résoudre de nombreuses affaires. Aujourd'hui, les empreintes digitales restent largement utilisées et reconnues comme une méthode d'identification fiable, jouant un rôle crucial dans les domaines de la sécurité, de la justice et de la technologie.

2.2 Caractéristiques des empreintes digitales

Une empreinte digitale est composée d'un



theses.hal.science
<https://theses.hal.science/tel-00009742v1/document>

ensemble de lignes localement parallèles formant un motif

unique pour chaque individu. Dans cette empreinte, on distingue deux éléments principaux : les stries

Figure 6.1 – Empreinte digitale [5]

- 1. terminaison 5. bifurcation triple 2 9. boucle double 13. pont isolé
- 2. bifurcation simple 6. bifurcation triple 3 10. pont simple 14. traversée
- 3. bifurcation double 7. crochet 11. pont jumeau 15. croisement
- 4. bifurcation triple 1 8. boucle simple 12. intervalle 16. tête bêche

Figure 6.2 – Les différents types de minutie [5]

(ou crêtes) et les sillons comme illustré dans la figure 6.1. Les empreintes digitales présentent des stries qui



theses.hal.science
<https://theses.hal.science/tel-00009742v1/document>

contiennent en leur centre un ensemble de pores régulièrement espacés.

Chaque empreinte possède

des caractéristiques singulières à la fois globales et locales. Les caractéristiques globales comprennent les centres, qui sont des points où les stries convergent, et les deltas, qui sont des points où les stries divergent [5]. Les caractéristiques locales comprennent les minuties, qui sont des points spécifiques de bifurcation ou de terminaison le long des stries.

Bien qu'il existe seize types différents de minuties identifiés par plusieurs études et illustré dans la figure 6.2, les algorithmes de traitement d'empreintes digitales se concentrent généralement sur les bifurcations et les terminaisons, car ces caractéristiques peuvent être utilisées pour dériver les autres types de minuties par combinaison.

3 Structure d'un système



theses.hal.science

<https://theses.hal.science/tel-00009742v1/document>

de reconnaissance d'empreintes digitales

Un système complet de reconnaissance d'empreintes digitales

représente une chaîne de processus

qui prend en entrée l'empreinte digitale d'un utilisateur et renvoie en sortie un résultat, permettant ainsi de déterminer si l'utilisateur a accès ou non à des éléments nécessitant une protection. La mise en œuvre d'un tel système a été le sujet de nombreuses recherches, avec une variété de méthodes de traitement proposées [103]. Cependant, tous ces systèmes suivent une structure similaire, comme illustré dans la figure 6.4.

La première phase d'un système de reconnaissance d'empreintes digitales consiste à obtenir une

131

Figure 6.3 – Les trois principales classes d'empreintes, boucle (a), spire (b), arche (c) [5]

Figure 6.4 – Architecture générale d'



bib.univ-oeb.dz

<http://bib.univ-oeb.dz:8080/jspui/bitstream/123456789/6791/1/mémoire.pdf.pdf>

un système complet de reconnaissance d'empreintes

image de l'empreinte de l'utilisateur (acquisition),

qui est ensuite soumise à un pré-traitement pour

extraire les informations pertinentes de l'image (extraction de la signature). Ensuite, un traitement supplémentaire peut être effectué pour éliminer les éventuelles fausses informations qui auraient pu être introduites dans la chaîne de traitement.

Si le système est destiné à simplement créer une base de données, la signature peut être compressée, puis stockée dans la base de données à l'aide d'une technique d'archivage (classification). La classification des empreintes digitales se fait en fonction de la position et du nombre de centres et de deltas, ce qui permet de les regrouper en différentes catégories selon leur caractéristiques globales. Les principales familles, illustrées dans la figure 6.3, sont les boucles, les spires et les arches.

En revanche, pour un système d'identification, toutes les



www.ummo.dz

<https://www.ummo.dz/dspace/bitstream/ummo/8362/1/OuldAmerDjamel.pdf>

empreintes présentes dans la base de



Document d'un autre utilisateur

Le document provient d'un autre groupe

données pouvant correspondre à celle de l'utilisateur (modèle identique) sont désarchivées et comparées

une à une avec celle de l'utilisateur (appariement). Si une correspondance est trouvée, des informations

personnelles concernant l'utilisateur sont renvoyées par le système.

Dans le cas d'un système de vérification, il n'y a qu'une seule comparaison effectuée, et un résultat binaire est renvoyé, permettant soit l'acceptation, soit le rejet de l'utilisateur.

132

Figure 6.5 – Capteur capacitif [6]

3.1 Acquisition d'empreintes digitales

Pour capturer l'image d'une empreinte digitale, on fait appel à un capteur d'empreinte. Ces capteurs peuvent prendre différentes formes, parmi lesquelles on trouve une variété d'options telles que les capteurs optiques d'empreinte, les capteurs électriques thermiques, les capteurs capacitifs, les capteurs de champ électrique, les capteurs de pression, et bien d'autres encore. La méthode capacitive (Figure. 6.5) est l'une des techniques les plus répandues pour la capture d'empreintes digitales. Comme les autres capteurs, le capteur capacitif reproduit l'image des creux et des reliefs qui composent une empreinte digitale. Ce capteur utilise des condensateurs électriques pour mesurer l'empreinte, et il se compose d'une

 **fr.slideshare.net** | Biométrie d'Empreinte Digitale | PDF
<https://fr.slideshare.net/slideshow/biomtrie-dempreinte-digitale/34684848>

rangée de cellules minuscules. Chaque cellule comprend deux plaques conductrices recouvertes par un revêtement protecteur.

L'avantage principal de ces capteurs est qu'ils nécessitent une véritable empreinte digitale pour fonctionner. Cependant, ils peuvent rencontrer des difficultés lors de l'acquisition de données à partir de doigts secs ou humides, ce qui peut affecter leur précision et leur fiabilité dans certaines situations.

3.2 Pré-traitement

Lorsqu'une image d'empreinte digitale est capturée, aujourd'hui par le biais d'un scanner, elle contient de nombreuses informations redondantes. Les problèmes liés aux cicatrices, aux doigts trop secs ou trop humides, ou à une pression incorrecte doivent également être surmontés pour obtenir une image acceptable. C'est pourquoi un pré-traitement est indispensable. Les étapes du pré-traitement sont : la normalisation, la binarisation qui consiste à transformer la valeur des pixels en 0 et 1 et la squelettisation qui consiste à réduire les contours de l'image d'empreinte à la taille d'un pixel afin de favoriser l'extraction des minuties.

3.3 Extraction des minuties

Les minuties sont des caractéristiques uniques (bifurcations, terminaisons) de l'empreinte digitale qui permettent son identification et sa vérification [104]. Chaque minutie m_i est représentée par un quadruplet de coordonnées : $m_i = (x_i, y_i, \theta_i, t_i)$ [105]. Les coordonnées x_i et y_i correspondent à la position de la minutie dans l'image, θ_i représente sa direction, généralement calculée à partir de l'orientation locale de la crête, et t_i indique si la minutie est de type terminaison ou bifurcation comme indique la figure 6.6. Les bifurcations et les terminaisons sont des points clés pour la comparaison des

133

Figure 6.6 – (a) Bifurcation ; (b) Terminaison

empreintes digitales. Pour chaque pixel noir dans chaque bloc de l'empreinte digitale, nous calculons l'indicateur de Crossing Number (CN) [104], qui est une mesure de la topologie locale. La formule pour l'indicateur CN est la suivante :

$CN = 1$

$$\sum_{i=1}^8 |p_i - p_{i+1}| \text{ with } p_9 = p_1$$

Les pixels voisins $p_1, \dots, p_9$ sont disposés dans l'ordre des aiguilles d'une montre. En utilisant les propriétés du CN chaque pixel de crête peut être classé comme une terminaison, une bifurcation ou un point sans minutie. En particulier, un pixel isolé aura un CN de 0, une terminaison aura un CN de 1, une crête continue aura un CN de 2, une bifurcation aura un CN de 3 et un croisement aura un CN de 4 [104].

3.4 Comparaison des minuties

La comparaison des minuties est une étape cruciale dans les systèmes de reconnaissance d'empreintes digitales. Elle comporte trois parties : l'alignement, la correspondance et la génération du score de correspondance. L'algorithme de la transformée de Hough généralisée à été adapté, décrit dans l'algorithme 34, à la comparaison des minuties [106]. Cette comparaison vise à évaluer la similarité entre les minuties extraites de l'empreinte digitale à authentifier et celles stockées dans une base de données.

Pour déterminer si deux ensembles de minuties proviennent de la même empreinte digitale, nous employons une méthode de superposition exhaustive, explorant toutes les combinaisons possibles. Nous examinons chaque minutie m_Q de la base de données modèle et tentons de la superposer à la minutie m_T de l'empreinte digitale à authentifier. La figure 6.7 montre la superposition de deux empreintes digitales. Ce processus implique le déplacement de la deuxième empreinte digitale, réalisant ainsi une translation afin d'aligner les coordonnées de m_T sur celles de m_Q . De plus, nous ajustons l'orientation de la deuxième empreinte digitale pour aligner les orientations de m_T et m_Q . La correspondance, décrite dans l'algorithme 35, implique l'identification des minuties appariées, c'est-à-dire celles des deux ensembles ayant des coordonnées (abscisse et ordonnée) similaires dans une certaine marge de tolérance, ainsi qu'une orientation similaire dans une certaine marge de tolérance. Toutes les minuties

134

Figure 6.7 – Superposition de deux empreintes digitales

appariées après l'alignement sont énumérées et stockées dans une liste. le score de correspondance, calculé à l'aide de la formule 6.1, est une mesure qui évalue à quel point deux ensembles de minuties sont similaires ou compatibles. Un score de correspondance plus élevé est associé à une meilleure précision de la reconnaissance. Cela signifie que les deux empreintes digitales sont plus susceptibles d'appartenir à la même personne si le score de correspondance est élevé.

Score = N_c

$$\frac{N_T}{N_c} \tag{6.1}$$

N_c représente le nombre de minuties correspondant, N_T le nombre de minuties de l'empreinte à authentifier et

Algorithme 34 : Transformée de Hough Généralisée
Fonction THG(m_Q, m_T , seuil larg, seuil long, seuil rot) : Tableau, réel

Variables : liste : Tableau
{ x_T

i, y_T
 i, θ_T

$i \in \{1, \dots, N_i\}, \{x_Q$
 j, y_Q

```

j ,  $\theta_Q$ 
j }Mj=1 ← mQ, mT ;

Initialisation de la liste des correspondances ;
Initialisation de la matrice d'accumulation A ;
for i=1,2,3,...,N do

    for j=1,2,3,...,M do
        % orientations des minuties;
         $\Delta\theta = \min(|\theta_T$ 

        i –  $\theta_Q$ 
        j | ,  $360^\circ - |\theta_T$ 

        i –  $\theta_Q$ 
        j | ) ;[

         $\Delta x$ 
         $\Delta y$ 

        ]
        =

        [
        xT

        i

        yT
        i

        ]
        –

        [
         $\cos(\Delta\theta) \sin(\Delta\theta)$ 
         $-\sin(\Delta\theta) \cos(\Delta\theta)$ 

        ] [
        xQ

        j

        yQ
        j

        ]

        A( $\Delta x$ ,  $\Delta y$ ,  $\Delta\theta$ ) ← A( $\Delta x$ ,  $\Delta y$ ,  $\Delta\theta$ ) + 1;
        si ( $0 \leq \Delta x \leq \text{seuil larg}$ ) et ( $0 \leq \Delta y \leq \text{seuil long}$ ) et ( $\Delta\theta \leq \text{seuil rot}$ ) alors

            ajouter (i, j) à liste % liste des correspondances
        fin

    end
end
n ← nombre total des correspondants;
score = n

N
Retourner : accum, score

fin

135

```

Algorithme 35 : Recherche des paires de minuties correspondantes de deux empreintes digitales

Fonction Correspondance(mQ, mT , accum, Sc) : Tableau
 Variables : liste : Tableau
 {xT

```

i , yT
i ,  $\theta_T$ 

i }Ni=1 , {xQ
j , yQ

```

```

j ,  $\theta_Q$ 
j }Mj=1 ← mQ, mT ;

```

Initialisation de la liste des correspondances ;
 for i=1,2,3,...,N do

```

for j=1,2,3,...,M do
% orientations des minuties;
 $\Delta\theta = \min(|\theta_T$ 

 $i - \theta_Q$ 
 $j |, 360^\circ - |\theta_T$ 

 $i - \theta_Q$ 
 $j |) : [$ 

 $\Delta x$ 
 $\Delta y$ 

 $] =$ 

 $[$ 
 $x_T$ 

 $i$ 

 $y_T$ 
 $i$ 

 $] =$ 

 $[$ 
 $\cos(\Delta\theta) \sin(\Delta\theta)$ 
 $-\sin(\Delta\theta) \cos(\Delta\theta)$ 

 $] [$ 
 $x_Q$ 

 $j$ 

 $y_Q$ 
 $j$ 

 $]$ 

si  $(0 \leq \Delta x \leq \text{seuil larg})$  et  $(0 \leq \Delta y \leq \text{seuil long})$  et  $(\Delta\theta \leq \text{seuil rot})$  alors
si  $\text{accum}[\Delta y, \Delta x, \Delta\theta] \geq S_c$  alors

ajouter  $(i, j)$  à correspondances % liste des correspondances
fin si

fin si
end

end
Retourner : correspondances

fin

```

4 Méthodologie

L'adaptation de la THG à la détection des empreintes digitales offre une précision satisfaisante de

99% d'après les travaux de Cynthia S. Mlambo et al [106]. Toutefois, son principal inconvénient réside

dans son temps de traitement et l'utilisation de la mémoire. Face à cette problématique, notre objectif est d'optimiser cette approche en réduisant significativement le temps de traitement.

Pour ce faire, nous avons adopté une approche basée sur l'utilisation des grilles virtuelles et

la programmation parallèle. La phase clé de notre optimisation réside dans l'étape d'appariement,

représentée dans la Figure 6.4. Cette étape consiste à comparer chaque empreinte digitale de la base

de données avec l'empreinte à authentifier en utilisant une approche adaptée de la Transformée de

Hough généralisée décrit par l'algorithme 34.

l'approche par la grille virtuelle, décrite par l'algorithme 36, consiste à superposer une grille rec-

tangulaire virtuelle de deux cellules sur l'empreinte digitale. Pour chaque empreinte digitale modèle de

la base de données, l'appariement se fait d'abord avec la cellule supérieur de la grille virtuelle. Lorsque

le nombre de paires de minuties correspondantes (N_c) est supérieur ou égal au seuil de correspondance

(S_c), alors l'empreinte digital test et l'empreinte digitale modèle sont considéré comme identique avec

un score de similarité. Dans le cas contraire, on passe à la cellule inférieure de la grille virtuelle. Lorsque le nombre de paires de minuties correspondantes (N_c) est supérieur ou égal au seuil de correspondance (Sc), alors l'empreinte digital test et l'empreinte digitale modèle sont considéré comme identique avec

136

un score de similarité. Dans le cas contraire, l'empreinte digital test et l'empreinte digitale modèle ne sont pas identiques et on passe à l'empreinte modèle suivante de la base de données.

Algorithme 36 : Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle.

Fonction FingerprintVG(BD, mT, seuil larg, seuil long, seuil rot, Sc) :Vide

miT ← pré-traitement et extraction des minuties de mT ;

V G ← Génération et superposition de la grille virtuelle de deux cellules rectangulaires sur miT ;

% Sc : seuil de correspondance.

h ← 0 ;

for mQ de BD do

for chaque cellule C de V G do

if C = 1^{ere} cellule de VG then

accum, Score ← THG(mQ, mT, seuil larg,



seuil long, seuil rot) ;

correspondances ← Correspondance(mQ, mT, accum, Sc) ;

N_c ← len(correspondances) ;

% nombre de paires de minuties correspondantes.

if $N_c \geq Sc$ then

Afficher : les empreintes digitales mQ et mT sont identiques avec un score de similarité de Score ;

h ← 1 ;

end

else

if h = 1 then

if $N_c \geq Sc$ then

Afficher : les empreintes digitales mQ et mT sont identiques avec un score de similarité de Score ;

h ← 1 ;

else

Afficher : les empreintes digitales mQ et mT ne sont pas identiques avec un score de similarité de Score ;

end

end

end

end

end

fin

La parallélisation consiste à diviser la base de données en sous-ensembles, en fonction du nombre de cœurs du processeur disponibles. La transformée de Hough généralisée est alors appliqué simultanément à chaque sous-ensemble pour l'appariement en utilisant l'approche des grilles virtuelles. C'est donc une approche hybride. Ce processus parallèle permet à chaque cœur de travailler de manière indépendante sur son propre lot de données, comme illustré dans la Figure 6.8.

Une fois l'appariement effectué, chaque processus renvoie un résultat partiel. Par la suite, ces résultats sont agrégés pour déterminer le résultat final. Plus précisément, si au moins l'un des processus identifie une correspondance positive entre l'empreinte à authentifier et une empreinte de la base de données, le résultat global est positif. En revanche, si tous les processus concluent à une absence de

correspondance, le résultat global est négatif.

Algorithme 37 : Reconnaissance d'empreintes digitales dans une grille rectangulaire virtuelle combiné au parallélisme

Fonction FingerprintVGP(BD, mT, seuil larg, seuil long, seuil rot, Sc, n cpu) :Vide
 mIT ← pré-traitement et extraction des minuties de mT ;
 V G ← Génération et superposition de la grille virtuelle de deux cellules rectangulaires sur mIT ;

% Sc : seuil de correspondance.

% n cpu : nombre de processus pour le parallélisme.



h ← 0 ;

With `pypm.parallel(n cpu)` as p :

for mQ de p(BD) do
 for chaque cellule C de V G do

if C = 1ere cellule de VG then
 accum, Score ← THG(mQ, mT, seuil larg,



seuil long, seuil rot) ;

correspondances ← Correspondance(mQ, mT, accum, Sc) ;

Nc ← len(correspondances) ;

% nombre de paires de minuties
 correspondantes.

if Nc ≥ Sc then
 Afficher : les empreintes digitales mQ et mT sont identiques avec un
 score de similarité de Score ;

h ← 1 ;
 end

else
 if h = 1 then

if Nc ≥ Sc then
 Afficher : les empreintes digitales mQ et mT sont identiques avec un
 score de similarité de Score ;

h ← 1 ;
 else

Afficher : les empreintes digitales mQ et mT ne sont pas identiques
 avec un score de similarité de Score ;

end
 end

end
 end

end
 fin

fin

5 Expérimentations et discussions

Un ordinateur MSI de modèle VECTOR GP76 doté des spécifications suivantes a été utilisé afin

d'effectuer les simulations :

• Processor : Intel(R) Core(TM) i7 CPU M 480 @ 2.67GHz 2.67GHz

• RAM : 8 Go

• Système d'exploitation : kali linux, 64 bits.

Empreinte non identifiée

Le langage de programmation utilisé est python.

Les expérimentations ont été menées sur un échantillon restreint de données, où les empreintes digitales ont été acquises dans diverses orientations et emplacements [107], ainsi que sur la base de données publique FVC2004 [108].

5.1 Performance temporelle

Le tableau 6.1 présente le temps de traitement de chaque approche (l'approche séquentielle, l'approche basée sur les grilles virtuelles et celle basée sur les grilles virtuelles combiné a la programmation parallèle) en fonction du nombre de comparaisons d'empreintes digitales.

L'analyse des graphiques de la figure 6.10 révèle que l'approche basée sur les grilles virtuelles est plus performante que celle séquentielle. Le facteur d'accélération est donner par $A = \frac{T_s}{T_h}$

$$= 7.42$$
$$4.82 = 1, 6$$

où T_h représentant le temps d'exécution de l'algorithme de l'approche basée sur la grille rectangulaire

139

Figure 6.9 – Appariement de deux empreintes digitales avec une grille rectangulaire

virtuelle et T_s désigne le temps d'exécution de l'algorithme séquentielle. Quant à l'approche basée sur

Figure 6.10 – Comparaison du temps de traitement en fonction du nombre d'empreintes

la grille rectangulaire virtuelle combiné au parallélisme, elle est moins performante que celle basée sur la grille virtuelle pour les bases de données contenant moins de 500 empreintes digitales. Cependant, à mesure que le nombre de comparaisons augmente, l'approche basée sur la grille rectangulaire virtuelle combiné au parallélisme devient de plus en plus avantageuse en termes de temps de traitement. Pour des nombres plus élevés de comparaisons (par exemple 500 et plus), elle réduit de manière significative le temps de traitement par rapport à l'approche basée sur la grille virtuelle. Cette réduction est particulièrement notable pour des nombres élevés de comparaisons, où le temps de traitement parallèle peut être jusqu'à plusieurs fois inférieur à celui du traitement séquentiel (voir la figure 6.10). Cette

140

performance temporelle est évaluée à travers le calcul du facteur d'accélération $A = \frac{T_s}{T_h}$

$$= 7.42$$
$$3.38 = 2, 19$$

où T_h représentant le temps d'exécution de l'algorithme de l'approche basée sur la grille rectangulaire virtuelle combiné au parallélisme (4 threads utilisés), et T_s désigne le temps d'exécution de l'algorithme séquentielle. Le fait que ce facteur d'accélération dépasse largement 1 met en lumière une amélioration significative de la rapidité de l'approche parallélisée de la reconnaissance des empreintes digitales.

Tableau 6.1 – Comparaison du temps de traitement en fonction du nombre d'empreintes

5.2 Précision

Pour évaluer la précision de notre programme, nous disposons de deux bases de données distinctes : la première est la base modèle, qui comprend 500 empreintes digitales. La seconde est la base de test, comprenant 100 empreintes digitales, dont 50 appartiennent à la base de données modèle et 50 ne lui appartiennent pas. Ensuite, chaque empreinte de la base de test est comparée à toutes les empreintes de la base modèle, totalisant ainsi 50 000 comparaisons.

Les résultats sont visualisés dans les matrices de confusion de la figure 6.11. Une matrice de confusion est un tableau qui permet d'évaluer les performances d'un modèle de classification. Elle compare les prédictions du modèle aux valeurs réelles et fournit une vue d'ensemble des résultats pour chaque classe. la structure générale d'une matrice de confusion pour un problème binaire (deux classes) est donné dans le tableau 6.2. La précision est calculé a l'aide de la formule $P = \frac{V P}{V P + F P}$

$$P = \frac{V P}{V P + F P}$$

L'analyse des matrices de confusion illustrée dans la figure 6.11 révèle des résultats significatives. Pour un seuil de correspondance égal à 2, les résultats illustrés dans la matrice de confusion donnent une précision de $P = 0,69$. Pour un seuil de correspondance égal à 3, les résultats illustrés dans la matrice de confusion donnent une précision de $P = 0,98$. Pour un seuil de correspondance égal à 7, les résultats illustrés dans la matrice de confusion donnent une précision de $P = 1$. Ainsi les Seuils de correspondance $Sc = 3 ; 4 ; 5$ et 6 sont recommandé. En effet, $Sc \leq 2$ laisse passer des faux positifs et $Sc \geq 7$ pourrait non seulement générer des faux négatifs, mais également réduire les chances de détecter de véritables correspondances si les empreintes sont partiellement dégradées. Cette recommandation permet de s'assurer qu'il y a un minimum de cohérence entre les minuties des deux

Tableau 6.2 – Structure générale d'une matrice de confusion

(a) $Sc=2$ (b) $Sc=3$ (c) $Sc=7$

Figure 6.11 – Matrices de confusion

empreintes, même avec des variations mineures causées par le découpage en cellules rectangulaires, en présence de bruit ou de légères différences de positionnement.

Conclusion

Ce chapitre a mis en lumière l'importance cruciale de l'application de la transformée de Hough généralisée dans les grilles virtuelles, ainsi que le parallélisme dans le domaine de la reconnaissance d'empreintes digitales. Nous avons démontré que l'intégration des grilles virtuelles dans la transformée de Hough généralisée a considérablement amélioré les performances en termes de temps de traitement tout en préservant la précision de l'identification des empreintes digitales. Combinée à la programmation parallèle, notre méthode réduit encore plus le temps de traitement particulièrement dans les bases de données volumineuses.

Introduction	143
1 Conception	144
2 Interface graphique	148
3 Impact	150



Introduction

La détection et l'analyse des formes géométriques dans les images sont des problèmes fondamentaux en vision par ordinateur. Parmi les techniques couramment utilisées pour résoudre ces problèmes, la transformée de Hough occupe une place prépondérante grâce à sa robustesse et à son efficacité. Elle permet de détecter des formes paramétriques telles que des lignes et des formes arbitraires dans des images bruitées ou incomplètes. Dans ce sens, une boîte à outils permettant d'utiliser divers transformée de Hough pour divers objectifs pourrait être une aubaine. Une aubaine, pour la détection de droites discrètes, d'empreintes digitales via la transformée de Hough. L'objectif de ce chapitre est de fournir une compréhension complète de l'architecture de la boîte à outils développé dans le cadre de ces travaux. Cette boîte à outils est un paquet de modules qui offre une variété de fonctionnalités pour la détection de lignes par application de la transformée de Hough sur différentes grilles virtuelles (rectangulaires, hexagonales, triangulaires et octogonales) et pour la reconnaissance d'empreintes digitales utilisant la transformée de Hough généralisée dans une grille rectangulaire virtuelle. En plus de ses capacités de traitement, le paquet est conçu pour être utilisé en environnement multiprocesseur,

143

ce qui permet d'accélérer les calculs grâce à la parallélisation. En outre, Elle est dotée d'une interface graphique pour permettre aux utilisateurs de visualiser et d'interagir facilement avec les résultats. Dans la suite du chapitre, nous commencerons par une étude des cas d'utilisation et de l'architecture de la boîte à outils. Ensuite, nous décrirons les modules internes, notamment ceux dédiés à la détection de droites et à la reconnaissance d'empreintes digitales, ainsi que les versions parallélisées de ces fonctionnalités pour des performances optimales. Aussi, nous explorerons l'interface utilisateur graphique (Graphic User Interface en anglais et abrégé par GUI) qui permet d'utiliser la boîte à outils de manière intuitive. En outre, nous explorerons les potentiels domaines d'impact de la boîte à outils. En fin, nous discuterons de ses limitations.

1 Conception

Dans cette section, nous décrivons la conception de la boîte à outils de la transformée de Hough, en détaillant les principaux aspects techniques et fonctionnels. Nous commencerons par étudier les cas d'utilisation de la boîte à outils. Ensuite, nous décrirons les fonctionnalités principales de l'outil, en mettant en avant ses capacités et son utilité. Enfin, nous présenterons les technologies et les outils utilisés pour développer cette boîte à outils.

1.1 Cas d'utilisation

La boîte à outils de la transformée de Hough possède deux cas d'utilisation distincts : un système de détection de droites et un système de reconnaissance d'empreintes digitales. La figure 7.1 illustre les cas d'utilisation de la boîte à outils [109]. Chaque système de la boîte à outils contient des algorithmes séquentiels et parallélisés. L'utilisation de l'un ou de l'autre est conditionné par les valeurs des paramètres d'entrés.

Pour la détection de droites, la version parallélisée est préférable à la version séquentielle lorsque la valeur du taux de pixels allumé alpha ainsi que la taille des cellules sont petites.

144

2

Figure 7.2 – Activités dans la boîte à outils

Pour la reconnaissance d'empreintes digitales, lorsque la base de données d'empreintes digitales est volumineuse (supérieur à 500), il est préférable d'utiliser la parallélisation.

La figure 7.2 illustre l'organisation et les conditions d'utilisation des fonctionnalités de la boîte à outils [109].

1.2 Description

Nous présentons, ici, les détails de l'architecture de la boîte à outils, en mettant en lumière les différents modules et composants qui la composent. Nous explorons en détail sa structure interne, en examinant comment les différentes fonctionnalités sont organisées et interconnectées. Nous abordons également les principaux modules de la boîte à outils, en décrivant leurs rôles et leurs responsabilités respectifs dans le cadre de la détection de lignes droites et de la reconnaissance d'empreintes digitales.

1.2.1 Structure interne

La structure interne de la boîte à outils est illustrée dans la figure 7.3. Le répertoire principal HoughV G/ contient le package principal également appelé HoughV G.

Le fichier init .py est nécessaire pour indiquer que le répertoire est un package Python. Les modules spécifiés sont placés à l'intérieur du package HoughV G.

Le module main .py peut être utilisé pour lancer la bibliothèque en tant qu'application autonome s'il est exécuté en tant que script principal. setup.py est un fichier de configuration courant utilisé pour installer et distribuer des packages Python.

Le dossier test/ contient des modules de test pour les différents composants de la bibliothèque. Le fichier README.md contient des informations sur l'utilisation de la bibliothèque, des exemples, des instructions d'installation.

145

Le fichier LICENCE : La boîte à outils est distribuée sous la licence MIT 6 qui donne à toute personne recevant le logiciel (et ses fichiers) le droit illimité de l'utiliser, le copier, le modifier, le fusionner, le publier, le distribuer, le vendre et le sous-licencier.

Figure 7.3 – Description de la structure interne de HoughVG

1.2.2 Modules

• Module HoughLine : il est composée des fonctions des variantes de la transformée de Hough standard appliqué sur des grilles virtuelles pour la détection de droites décrites dans les chapitres 2, 3, 4 et 5 :

— La fonction de la transformée de Hough rectangulaire (THR) a quatre paramètres d'entrées : l'image, la taille (l, L) des cellules rectangulaires de la grille rectangulaire virtuelle, le taux de pixel allumé α des cellules rectangulaires et le seuil de vote. l et L représentant respectivement la largeur et la longueur des cellules rectangulaire de la grille avec $0 \leq l \leq Nl$ et $0 \leq L \leq Nc$ où Nl et Nc sont respectivement la hauteur et la largeur de l'image image. α est un nombre réel compris entre 0 et 1

— La fonction de la transformée de Hough triangulaire (THT) a également quatre paramètres

d'entrées : l'image, la taille (h, b) des cellules triangulaires de la grille triangulaire virtuelle, le

taux de pixel allumé α des cellules triangulaires et le seuil de vote. Les nombres h et b représentent

respectivement la hauteur et la base des cellules triangulaires de la grille avec $0 \leq h \leq NI$ et

6. https://fr.wikipedia.org/wiki/Licence_MIT

146

https://fr.wikipedia.org/wiki/Licence_MIT

$0 \leq b \leq Nc$ où NI et Nc sont respectivement la hauteur et la largeur de l'image *imge*. α est un

nombre réel compris entre 0 et 1.

— La fonction de la transformée de Hough hexagonale (THH) a aussi quatre paramètres d'entrées :

l'image, la taille γ des cellules hexagonale de la grille hexagonale virtuelle, le taux de pixel

allumé α des cellules hexagonales et le seuil de vote. $2 \leq \gamma \leq \min(NI+2, Nc+2)$

2), défini la taille

des cellules hexagonales de la grille virtuelle. Le réel α est compris entre 0 et 1.

— La fonction de la transformée de Hough octogonale (THO) a aussi quatre paramètres d'entrées :

l'image, la taille γ des cellules octogonale de la grille octogonale virtuelle, le taux de pixel allumé

α des cellules octogonale et le seuil de vote. $1 \leq \gamma \leq \min(NI+1, Nc+1)$

2), défini la taille des cellules

octogonales de la grille virtuelle et α est un nombre réel compris entre 0 et 1.

• Module HoughLineParallel : Ce module contient des versions parallélisées des fonctionnalités

du module HoughLine pour améliorer davantage leurs performances. Les fonctions disponibles sont

décrivent dans les même chapitres 2, 3, 4 et 5.

•Module Fingerprint : Ce module propose des fonctionnalités liées à la reconnaissance d'empreintes

digitales avec la transformée de Hough généralisée (THG). Les fonctions principales FingerprintV G

et sa version parallélisée FingerprintV GP sont illustré respectivement dans les algorithmes 36 et 37

au chapitre 6.

La fonction FingerprintV G prend en paramètre une base de données modèle BD, Une empreinte

digitale test mT , un seuil long seuil long, un seuil large seuil larg, un seuil de rotation seuil rot et

un seuil de correspondance Sc. Lorsque mT a été correctement identifié à une empreintes digitale dans

BD alors la fonction FingerprintV G affiche un message que l'empreinte digitale mT a été identifié

avec un certain score.

La fonction FingerprintV GP est la version parallèle de la fonction fingerprintV G. le paramètre

n cpu représente le nombre de cœurs du CPU utilisé pour la parallélisation.

Le seuil large seuil larg définit la distance maximale autorisée entre deux minuties pour qu'elles

soient considérées comme correspondantes. Un seuil large permet d'accepter des correspondances même

si les minuties ne sont pas très proches, ce qui peut être utile pour compenser des erreurs mineures

dans le placement des minuties.

Le seuil long seuil long, quant à lui, spécifie la distance maximale pour accepter une correspon-

dance lors de la superposition des minuties. L'utilisation d'un seuil long autorise des correspondances

même si les minuties sont relativement éloignées, ce qui peut être bénéfique pour gérer des distorsions

ou des variations dans la taille de l'empreinte digitale.

Enfin, le seuil de rotation seuil rot intervient en définissant l'écart angulaire maximal autorisé entre les orientations des minuties correspondantes. Ce paramètre est crucial pour prendre en compte les variations d'orientation entre les minuties, car les empreintes digitales peuvent être capturées avec des orientations différentes.

147

1.3 Les dépendances

L'environnement logiciel choisi pour développer la boîte à outils est Python 3.11 Les fichiers exécutables en Python ont l'extension ".py", et l'environnement de développement que nous utilisons est la version 1.79.2 de Visual Studio, car il intègre de nombreuses bibliothèques d'usage scientifique indispensables au bon fonctionnement de la bibliothèque HoughV G. Nous pouvons citer :

- pypm 0.5.0 : c'est un outil pour la programmation parallèle en Python. Conçue pour tirer parti des architectures multiprocesseurs, pypm facilite l'implémentation de programmes parallèles en fournissant des structures de contrôle simples et des mécanismes de synchronisation. Il faut noter que pypm ne marche que sur les machines Linux 7.
- numpy 1.24.2 : c'est une bibliothèque



www.memoireonline.com | Memoire Online - Etude des méthodes de reconnaissances d'empreinte digitale a l'aide du deep learning - Jean-Edmond DASSE
<https://www.memoireonline.com/08/21/12173/Etude-des-methodes-de-reconnaissances-dempreinte-digitale-a-laide-du-deep-learning.html>

destinée à manipuler des matrices ou tableaux multidimen-

sionnels ainsi que des fonctions mathématiques opérant sur ces tableaux.

- matplotlib 3.6.3 : c'est une bibliothèque complète pour la création de visualisations statiques, animées et interactives en Python.
- scipy 1.10.1 : c'est une bibliothèque open-source utilisée pour résoudre des problèmes mathématiques, scientifiques, d'ingénierie et techniques.
- opencv 7.0.72 : c'est



www.memoireonline.com | Memoire Online - Etude des méthodes de reconnaissances d'empreinte digitale a l'aide du deep learning - Jean-Edmond DASSE
<https://www.memoireonline.com/08/21/12173/Etude-des-methodes-de-reconnaissances-dempreinte-digitale-a-laide-du-deep-learning.html>

une bibliothèque libre de vision par ordinateur écrite en C et C++, utilisable

sous Linux, Windows et Mac OS X. Des interfaces ont été développées pour Python, Matlab et d'autres

langages. opencv est orientée vers des applications en temps réel, visant à aider les développeurs à

construire rapidement des applications sophistiquées de vision à l'aide d'une infrastructure simple de

vision par ordinateur.

2 Interface graphique

L'interface graphique de la boîte à outils HoughV G de la transformée de Hough a été conçue pour permettre aux utilisateurs de naviguer facilement et d'exploiter pleinement les fonctionnalités offertes. le tableau de bord, présenté sur la figure 7.4, s'affiche au lancement de la boîte à outils. Le tableau de bord présente deux boutons d'option pour le système de détection de droites et le système de reconnaissance d'empreintes digitales.

2.1 Scénarios d'utilisation du système de détection de droites

Lorsque l'utilisateur sélectionne la détection de droites, l'interface graphique envoie la requête au système qui retourne ensuite les différents types de transformée de Hough. L'interface graphique les récupère puis les affiche.

L'utilisateur peut alors choisir le type de TH envoyer la requête à travers l'interface graphique au système. Le système de détection de droites envoie alors les paramètres correspondant au types de

Figure 7.4 – Tableau de bord

transformée de Hough choisi à l'interface utilisateur qui les affiche.

L'utilisateur entre alors les paramètres et envoi, a travers l'interface utilisateur, au système. Le

système traite les données envoi le résultat de la détection à l'interface graphique qui l'affiche.

L'utilisateur passe alors à l'analyse et à l'interprétation du résultat affiché.

Ces scénarios sont illustré dans la figure 7.5 [109].

Figure 7.5 – Séquence des scénarios d'utilisation du système de détection des droites

2.2 Scénarios d'utilisation du système de reconnaissance d'empreintes digitales

Lorsque l'utilisateur sélectionne la reconnaissance d'empreintes, l'interface graphique envoi la

requête au système qui retourne ensuite les paramètres correspondant. L'interface graphique les

récupère puis les affiche.

L'utilisateur entre alors les paramètres et envoi, a travers l'interface utilisateur, au système. Le

système traite les données puis envoi le résultat à l'interface graphique qui l'affiche.

L'utilisateur passe alors à l'analyse et à l'interprétation du résultat affiché.

Ces scénarios sont illustré dans la figure 7.6 [109].

Figure 7.6 – Séquence des scénarios d'utilisation du système de reconnaissance d'empreintes digitales

3 Impact

La boîte à outils peut être utile dans :

— Recherche Académique et Industrielle : La disponibilité de ces outils optimisés peut stimu-

ler la recherche de nouvelles applications et algorithmes basés sur la transformée de Hough.

Les chercheurs peuvent se concentrer sur l'innovation plutôt que sur les aspects techniques de

l'implémentation.

— Communauté Open-Source : Étant donné sa nature de source ouverte, HoughVG est accessible

aux développeurs qui peuvent non seulement l'utiliser dans projet mais également contribuer à

son développement continu.

— Industrie et Robotique : La détection précise des droites est cruciale pour la navigation robotique,

l'assemblage automatisé et la surveillance industrielle. Ce logiciel peut permettre aux robots de

mieux comprendre leur environnement et de prendre des décisions plus rapides et précises.

— Imagerie médicale : Dans le domaine médical où une détection rapide et précise est essentielle,

ce logiciel peut améliorer la détection des structures linéaires dans les images, comme les rayons

X et les IRM. Cela permet aux médecins de poser des diagnostics plus précis et de planifier des

interventions avec plus de confiance.

— Sécurité et biométrie : Dans les systèmes de gestion des empreintes digitales, comme ceux utilisés

par les forces de l'ordre ou les services de sécurité, le traitement en masse est courant. Les va-

riantes optimisées par parallélisme permettent de traiter un grand volume d'empreintes digitales

de manière plus rapide et plus efficace.

— Transport : Les véhicules autonomes dépendent de systèmes de vision sophistiqués pour naviguer de manière sûre. La transformée de Hough optimisée peut améliorer la capacité de ces systèmes à détecter et interpréter l'environnement routier.

4 Limitations

Nous pouvons énumérer deux limites majeures. La première est que HoughVG ne peut que détecter les droites et reconnaître les empreintes digitales. Cela limite son domaine d'application. La seconde est que l'ajustement des paramètres optimales pour la détection de droites, n'est pas automatique. Il se fait manuellement en fonction de l'image. Ce qui peut s'avérer pénible.

5 Validation des hypothèses de recherche

Dans le cadre de cette étude, nous avons formulé deux hypothèses clés visant à améliorer l'efficacité de la transformée de Hough pour la détection d'objets discrets. Les résultats expérimentaux confirment ces hypothèses, comme détaillé ci-dessous :

• Impact des grilles virtuelles sur le temps de traitement et la précision : L'intégration des grilles virtuelles a permis une réduction significative du temps de traitement, tout en maintenant un niveau de précision comparable à celui de la méthode traditionnelle.

Pour la détection des droites discrètes, nous avons obtenu un facteur d'accélération $A = 3,26$ (sur l'image de l'immeuble) et $A = 1,9$ (sur l'image de l'autoroute) avec la grille rectangulaire, un facteur d'accélération $A = 2,28$ (sur l'image de l'immeuble) et $A = 1,31$ (sur l'image de l'autoroute) avec la grille triangulaire, un facteur d'accélération $A = 3,5$ (sur l'image de l'immeuble) et $A = 1,05$ (sur l'image de l'autoroute) avec la grille hexagonale, un facteur d'accélération $A = 2,22$ (sur l'image de l'immeuble) et $A = 1,9$ (sur l'image de l'autoroute) avec la grille octogonale. Le tout assorti d'une perte négligeable de précision. Pour la reconnaissance des empreintes digitales le facteur d'accélération est 151

Tableau 7.1 – Facteurs d'accélération

de 1,6 avec une précision pouvant aller jusqu'à 1 en fonction du seuil de correspondance. Ces résultats sont illustrés dans le tableau 7.1

• Combinaison des deux approches : Pour la détection des droites discrètes, la combinaison des grilles virtuelles et le parallélisme a apporté une amélioration dans le temps de traitement dans le cas des grilles de petites cellules et des faibles taux α de pixel allumé comme illustré dans le tableau 7.2. En ce qui concerne la reconnaissance des empreintes digitales, cette combinaison a produit des résultats qui corroborent l'hypothèse dans le cas des bases de données volumineuse (au moins 500 empreintes digitales) avec un facteur d'accélération $A = 2,19$. Cette approche hybride a pu maintenir également la qualité de la précision. Ces résultats démontrent que l'optimisation proposée, basée sur l'utilisation (a) Immeuble (b) Autoroute

Tableau 7.2 – Comparaison du temps de traitement de l'approche hybride et celle basée sur les grilles virtuelles

de grilles virtuelles est une solution pour améliorer la transformée de Hough. La combinaison avec le parallélisme est également. Elles permettent non seulement d'accélérer les traitements, mais aussi de maintenir une bonne précision, répondant ainsi aux exigences des applications en temps réel.

Conclusion

La boîte à outils HoughVG, basée sur la transformée de Hough, intègre des algorithmes optimisés pour la détection de droites et la reconnaissance d'empreintes digitales, cette boîte à outils offre des

solutions pour une variété d'applications.

L'organisation modulaire de HoughVG, ainsi que son interface graphique simplifié, rendent l'outil accessible à un large éventail d'utilisateurs, des chercheurs académiques aux professionnels de l'industrie. Les fonctionnalités de la boîte à outils améliorent considérablement les performances, permettant

152

de traiter de grandes quantités de données rapidement et efficacement.

Cependant, la boîte à outils n'est pas sans limitations. La qualité des résultats dépend fortement du paramétrage précis des algorithmes. De plus, les cas d'utilisation se limitent aux droites et aux empreintes digitales.

Malgré ces défis, l'impact de HoughVG est significatif. Elle permet non seulement d'améliorer les systèmes de détection de droites et de reconnaissance d'empreintes digitales existants, mais ouvre également la voie à de nouvelles recherches et développements dans le domaine de la vision par ordinateur. Les futures améliorations pourraient inclure l'extension de ses capacités à d'autres formes géométriques et l'automatisation du paramétrage pour simplifier davantage son utilisation.

153

Conclusion Générale

Ce document met en évidence la richesse et la diversité des variantes de la transformée de Hough, chacune offrant des perspectives uniques dans la détection d'objets dans les images. Les analyses des transformées de Hough rectangulaire, triangulaire, hexagonale et octogonale révèlent les nuances de chaque approche, en mettant en évidence leurs avantages respectifs en termes de précision, de flexibilité et de complexité algorithmique. Le seuil de vote est un paramètre très crucial. En effet l'efficacité de ces approches repose sur l'utilisation de seuil de vote faible.

Ainsi, les quatre variantes de la transformée de Hough (THR, THT, THH, THO) proposée offre chacun des possibilités de réduction du temps de traitement tout en maintenant une bonne précision de détection des droites discrètes. En comparaison, la transformée de Hough rectangulaire offre un temps de traitement plus rapide que la transformée de Hough triangulaire. De plus, en termes de flexibilité des paramètres, la transformée de Hough rectangulaire est préférable. En effet, il est possible d'ajuster les paramètres de la transformée de Hough rectangulaire pour cibler la détection soit sur les droites verticales, soit sur les droites horizontales.

Aussi, de façon générale, la combinaison du parallélisme et des nouvelles variantes de la transformée de Hough dans le cadre de la détection des droites discrètes améliore le temps de traitement pour les petites valeurs de α et les grilles virtuelles de petites cellules, mais cette amélioration diminue à mesure que les valeurs de α et la taille des cellules augmentent.

En outre, l'introduction des grilles virtuelles dans le processus de la reconnaissance des empreintes digitales a permis d'améliorer le temps de traitement. La combinaison du parallélisme et des grilles virtuelles offre des solutions pour répondre aux exigences de traitement en temps réel et pour gérer efficacement de grandes quantités de données dans le cadre de la reconnaissance d'empreintes digitales.

Par contre, pour les petites bases de données, il est préférable d'utiliser l'approche des grilles virtuelles.

Enfin, une boîte à outils nommée HoughVG, sous forme d'une bibliothèque Python munie d'une interface graphique utilisateur et regroupant toutes les implémentations d'algorithmes, a été mise en place.

Les travaux futurs viseront à intégrer les pratiques relatives aux régions d'intérêt (ROI) dans le pré-traitement des images pour améliorer la précision de la détection, intégrer les algorithmes d'intelligence artificielle notamment les CNNs dans notre approche, étendre la boîte à outils à la reconnaissance d'autres formes dans les domaines de la santé et de l'agriculture.

155

Bibliographie

[1] A. Sere, Transformations Analytiques appliquées aux Images multi-échelles et bruitées Laboratoire de Traitement de l'Information et la Communication, Université de Ouagadougou, juin 2013.

[2] M. Rodriguez, Redimensionnement adaptatif et reconnaissance de primitives discrètes Université de Poitiers (France), Thèse en informatique & applications, 2011.

[3] A. SERE, Y. TRAORE, and F. T. OUEDRAOGO, Towards New Analytical Straight Line Definitions and Detection with the Hough Transform Method International Journal of Engineering Trends and Technology (IJETT), vol. 62, 2018. DOI : 10.14445/22315381/IJETT-V62P212.

[4] C. A. D. G. Traoré and A. Séré, Straight-Line Detection with the Hough Transform Method Based on a Rectangular Grid pp. 599–610, 2022.

[5] A. bilal, Conception d'un

 **20** **bib.univ-oeb.dz**
<http://bib.univ-oeb.dz:8080/jspui/bitstream/123456789/6791/1/mémoire.pdf.pdf>

système de Reconnaissance des empreintes digitales par apprentissage

Université des sciences et de la technologie d'Oran Mohammed Boudiaf, 2012.

[6] Conception d'un système de Reconnaissance des empreintes digitales par apprentissage Univer-

site des sciences et de la technologie d'oran mohammed boudiaf, faculte de genie electrique département d'electronique, 2012.

[7] D. Ballard, Generalizing the Hough transform to detect arbitrary shapes Pattern Recognition, vol. 13, no. 2, pp. 111–122, 1981.



DOI : [https://doi.org/10.1016/0031-3203\(81\)90009-1](https://doi.org/10.1016/0031-3203(81)90009-1).

[8] A. Katru and A. Kumar, Improved Parallel Lane Detection Using Modified Additive Hough Transform International Journal of Image, Graphics and Signal Processing(IJIGSP), vol. 8, pp. 10–17, 2016. DOI : 10.5815/ijigsp.2016.11.02.

[9] D. Geman and B. Jedynak,

 **21** **doi.org** | Straight-Line Recognition Using a Triangular Grid | SpringerLink
https://doi.org/10.1007/978-3-030-98012-2_45

An active testing model for tracking roads in satellite images

 **22** **dspace.univ-tlemcen.dz**
[http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf](http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire%20de%20magister%20HAOUZI.pdf)

IEEE

Transactions on Pattern Analysis and Machine Intelligence, vol. 18,

 **23** **theses.hal.science**
<https://theses.hal.science/tel-00009742v1/document>

no.

1, pp. 1–14, 1996.



DOI :

10.1109/34.476006.

[10] F. Benkouider, L. Hamami, A. Abdellaoui,
and M. Salmon, Extraction de routes par classification

supervisée et par réseaux de neurones artificiels à partir d'image spot : cas d'une ville oasienne

(algérie) Teledetection, vol. 11, pp. 237–249, Aug. 2012.

156

[11] A. Sere, C. A. D. G. Traore, Y. Traore, and O. Sie, "Towards traffic saturation detection based
on the hough transform method," in Proceedings of the Future Technologies Conference (FTC)

2020, Volume 2 (K. Arai, S. Kapoor, and R. Bhatia, eds.), (Cham), pp. 263–270, Springer

International Publishing, 2021.

[12] M. Marzougui, A. Alasiry, Y. Kortli, and J. Baili, A Lane Tracking Method Based on Progressive

Probabilistic Hough Transform IEEE Access, vol. 8, pp. 84893–84905, 2020.

[13] C. Yang and J. Collins, Improvement of Honey Bee Tracking on 2D Video with Hough

Transform and Kalman Filter J Sign Process Syst, vol. 90, pp. 1639–1650, 2018.



DOI :

<https://doi.org/10.1007/s11265-017-1307-x>.

[14] E. Widyaningrum, B. Gorte,
and R. Lindenberg,



www.mdpi.com | Automatic Building Outline Extraction from ALS Point Clouds by Ordered Points Aided Hough Transform
<https://www.mdpi.com/2072-4292/11/14/1727>

Automatic Building Outline Extraction from

ALS Point Clouds by Ordered Points Aided Hough Transform

Remote Sensing, vol. 11, no. 14,

2019. DOI : 10.3390/rs11141727.

[15] B. Liao, J. Li, Z. Ju, and G. Ouyang, "Hand



link.springer.com | Application Development for Hand Gestures Recognition with Using a Depth Camera | SpringerLink
https://link.springer.com/chapter/10.1007/978-3-030-57672-1_5

gesture recognition with generalized hough transform

and dc-cnn using realsense,"

in 2018 Eighth International Conference on Information Science

and Technology (ICIST), pp. 84–90, 2018. DOI : 10.1109/ICIST.2018.8426125.

[16] G. Lin, Y. Tang, and X. Z. et al., Fruit detection in natural environment using partial shape

matching and probabilistic Hough transform Precision Agric, vol. 21, pp. 160–177, 2020.

[17] G. A. Soares, D. Abdala, and M. Escarpinati, "Plantation rows identification by means of

image tiling and hough transform,"



www.scitepress.org
<https://www.scitepress.org/PublishedPapers/2018/66577/66577.pdf>

in Proceedings of the 13th International Joint Confe-

rence on Computer Vision, Imaging and Computer Graphics Theory and Applications

(VI-

SIGRAPP 2018) - Volume 4 :



[18] W. Winterhalter, F. V. Fleckenstein, C. Dornhege, and W. Burgard, Crop Row Detection on

Tiny Plants With the Pattern Hough Transform IEEE Robotics and Automation Letters, vol. 3, no. 4, pp. 3394–3401, 2018. DOI : 10.1109/LRA.2018.2852841.

[19] C. Campi, A. Perasso, M. C. Beltrametti, A. M. Massone, G. Sambuceti, and M. Piana, Pattern recognition in medical imaging by means of the Hough transform of curves pp.



[20] R. Vijayarajeswari, P. Parthasarathy, S. Vivekanandan, and A. A. Basha, Classification of mam-

mogram for early detection of breast cancer using SVM classifier and Hough transform Measurement, vol. 146, pp. 800–805, 2019. DOI : https ://doi.org/10.1016/j.measurement.2019.05.083.

[21] C. Zhang, F. Wang, Y. Zou, J. Dimyadi, B. H. Guo, and L. Hou, Automated UAV image-to-BIM registration for building façade inspection using improved generalized Hough transform Automation in Construction, vol. 153, p. 104957, 2023. DOI : https ://doi.org/10.1016/j.autcon.2023.104957.

[22] P. N and C. Y.,



[23] A. C. Freeman, W. Shi, and B. Hwang, “Enhancing surveillance camera fov quality via semantic line detection and classification with deep hough transform,” 2024.

[24] P. Hough, Method and means for recognizing complex patterns In United States Patent, vol. 3069654, pp. 47–64, 1962.

[25] J. Illingworth and J. Kittler, A survey of the hough transform Computer Vision, Graphics, and Image Processing, vol. 44, no. 1, pp. 87–116, 1988.



[26] J. Cha, R. Cofer, and S. Kozaitis, Extended Hough transform for linear feature detection Pattern Recognition, vol. 39, no. 6, pp. 1034–1043, 2006. DOI : https ://doi.org/10.1016/j.patcog.2005.05.014.

[27] C. Galamhos, J. Matas, and J. Kittler, “Progressive probabilistic hough transform for line detection,” in Proceedings. 1999



(Cat. No PR00149), vol. 1, pp. 554–560 Vol. 1, 1999. DOI :

10.1109/CVPR.1999.786993.

[28] L. S. Davis, Hierarchical generalized Hough transforms and line-segment based generalized Hough transforms Pattern Recognition, vol. 15, no. 4, pp. 277–285, 1982.



DOI :

[https://doi.org/10.1016/0031-3203\(82\)90030-9](https://doi.org/10.1016/0031-3203(82)90030-9).

[29] H. Rhody and C. F. Carlson, Hough Circle Transform Center for Imaging Science, Rochester Institute of Technology, 2005.

[30] N. Kiryati, Y. Eldar, and A. Bruckstein, A probabilistic Hough transform Pattern Recognition, vol. 24, no. 4, pp. 303–316, 1991.



DOI : [https://doi.org/10.1016/0031-3203\(91\)90073-E](https://doi.org/10.1016/0031-3203(91)90073-E).

[31] D. Luo, X. He, Q. Teng, and Q. Tao, Triplet circular Hough transform for circle detection in Journal of electronics (China) vol. 19, pp. 356–362, 2002.

[32] R. O. Duda and P. E. Hart,

29

popups.uliege.be
<https://popups.uliege.be/0770-7576/index.php?id=1036&file=1>

Use

30

dspace.univ-tlemcen.dz
[http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf](http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire%20de%20magister%20HAOUZI.pdf)

of the Hough transformation to detect lines and curves in

pictures Commun. ACM, vol. 15, pp. 11–15, 1972.

[33] A. Y. S. Chia, M. K. H. Leung, H.-L. Eng, and S. Rahardja, “Ellipse detection with hough transform in one dimensional parametric space,” in 2007 IEEE International Conference on Image Processing, vol. 5, pp. V – 333–V – 336, 2007. DOI : 10.1109/ICIP.2007.4379833.

[34] W. Lu and J. Tan,

31

www.ijser.org
https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf

Detection of incomplete ellipse in images with strong noise by iterative

randomized Hough transform (IRHT) Pattern Recognition, vol. 41, no. 4, pp. 1268–1279, 2008.

DOI : <https://doi.org/10.1016/j.patcog.2007.09.006>.



1016/j.patcog.2007.09.006.

158

[35] C. Huang, Elliptical feature extraction via an improved Hough transform Pattern Recognition Letters, vol. 10, no. 2, pp. 93–100, 1989.



DOI : [https://doi.org/10.1016/0167-8655\(89\)90073-1](https://doi.org/10.1016/0167-8655(89)90073-1).

[36] C. Daul, P. Graebbling, and E. Hirsch,

32

www.ijser.org
https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf

From the Hough Transform to a New Approach for the

Detection and Approximation of Elliptical Arcs Computer Vision and Image Understanding,

vol. 72, no. 3, pp. 215–236, 1998. DOI : <https://doi.org/10.1006/cviu.1998.0696>.

[37] H. Kälviäinen, P. Hirvonen, L. Xu, and



www.ijser.org

https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf

E. Oja, Probabilistic and non-probabilistic Hough trans-

forms : overview and

comparisons Image and Vision Computing, vol. 13, no. 4, pp. 239–252,

1995. DOI : [https://doi.org/10.1016/0262-8856\(95\)99713-B](https://doi.org/10.1016/0262-8856(95)99713-B).

[38] K. Khoshelham, “Extending generalized hough transform to detect 3d objects in laser range

data,” in Proceedings of the ISPRS workshop Laser Scanning 2007 and SilviLasser 2007, Espoo,

Finland, 12-14 September 2007 / ed. by P. Rönholm, H. Hyypä and J. Hyypä. (ISPRS :

Volume XXXVI, part 3/W52. pp.206-210 (P. Rönholm, H. Hyypä, and J. Hyypä, eds.),

pp. 206–210, International Society for Photogrammetry and Remote Sensing (ISPRS), 2007.

[39] J. Lopez-Krahe, T.



Alamo-Cantarero, and E. Davila-Gonzalez,

Discrete Hough Trans-

form Applied to Small Size Pattern Recognition éditeur Télécom, 1994. DOI :

10.13140/RG.2.2.10125.13283.

[40] I.



www.ijser.org

https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf

Svalbe, Natural representations for straight lines and the Hough transform on discrete arrays

IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 11, no. 9, pp. 941–950,

1989. DOI : 10.1109/34.35497.

[41] S. Maji and J. Malik, “Object detection using a max-margin hough transform,” in 2009

IEEE Conference on Computer Vision and Pattern Recognition, pp. 1038–1045, 2009. DOI :

10.



1109/CVPR.2009.5206693.

[42] Lopez-Krahe and Pousset,

“7 - transformée de hough discrète et bornée. application à la détection

de droites parallèles et du réseau routier,” 1988.

[43] A. Sere, O. Sie, and E. Andres, Extended Standard Hough Transform for analytical line recogni-

tion pp. 412–422, 2012. DOI : 10.1109/SETIT.2012.6481950.

[44] K. Zhao, Q. Han, C.-B. Zhang, J. Xu, and M.-M. Cheng, Deep Hough Transform for Semantic

Line Detection IEEE Transactions on Pattern Analysis and Machine Intelligence, p. 1–1, 2021.

DOI : 10.1109/tpami.2021.3077129.

[45] A. Gabrielli, F. Alfonsi, and F. Del Corso, Simulated Hough Transform Model Optimized for

Straight-Line Recognition Using Frontier FPGA Devices Electronics, vol. 11, no. 4, 2022. DOI :

10.3390/electronics11040517.

[46] Q. Liang, J. Long, Y. Nan, G. Coppola, K. Zou, D. Zhang, and W. Sun, Angle aided circle

detection based on randomized Hough transform and its application in welding spots detec-

tion Mathematical Biosciences and Engineering, vol. 16, no. 3, pp. 1244–1257, 2019. DOI :

10.3934/mbe.2019060.

[47] K. GOTO and H. TSUKUNE, Extracting Elliptical Figures from an Edge Vector Field Transactions of the Society of Instrument and Control Engineers, vol. 20, no. 4, pp. 329–336, 1984.

DOI : 10.9746/sicetr1965.20.329.

[48] D. Bailey, Y. Chang, and S. Le Moan, Analysing Arbitrary Curves from the Line Hough Transform Journal of Imaging, vol. 6, no. 4, 2020. DOI : 10.3390/jimaging6040026.

[49] M.-L. Torrente, S. Biasotti, and B. Falcidieno, Recognition of feature curves on 3D shapes using an algebraic approach to Hough transforms Pattern Recognition, vol. 73, pp. 111–130, 2018.

DOI : <https://doi.org/10.1016/j.patcog.2017.08.008>.

[50] C. Conti, L. Romani, and D. Schenone, Semi-automatic spline fitting of planar curvilinear profiles in digital images using the Hough transform Pattern Recognition, vol. 74, pp. 64–76, 2018. DOI :

<https://doi.org/10.1016/j.patcog.2017.09.017>.

[51] C. Romanengo, B. Falcidieno, and S. Biasotti, Hough transform based recognition of space curves Journal of Computational and Applied Mathematics, vol. 415, p. 114504, 2022. DOI :

<https://doi.org/10.1016/j.cam.2022.114504>.

[52] A. Sere, F. T. Ouedraogo, and B. Zerbo, “An

 **35** doi.org | Straight-Line Recognition Using a Triangular Grid | SpringerLink
https://doi.org/10.1007/978-3-030-98012-2_45

improvement of the standard hough transform

method based on geometric shapes,”

in Advances in Information and Communication Networks

(K. Arai, S. Kapoor, and R. Bhatia, eds.), (Cham), pp. 369–384, Springer International Publishing, 2019.

[53] W. Yang, Y. Li, T. Hu, R. Fuchikami, and T. Ikenaga, “Relative vectors clustering and temporal constraint based generalized Hough transform for high frame rate and ultra-low delay arbitrary shape detection,” in Third International Conference on Computer Vision and Information Technology (CVIT 2022) (J. Ma, ed.), vol. 12590, p. 1259002, International Society for Optics and Photonics, SPIE, 2023. DOI : 10.1117/12.2670011.

[54] N. Kiryati, Y. Eldar, and A. Bruckstein, A probabilistic Hough transform Pattern Recognition, vol. 24, no. 4, pp. 303–316, 1991.



DOI : [https://doi.org/10.1016/0031-3203\(91\)90073-E](https://doi.org/10.1016/0031-3203(91)90073-E).

[55] M. A. Javeed, M. A. Ghaffar, M. A. Ashraf, N. Zubair, A. S. M. Metwally, E. M. Tag-Eldin, P. Bocchetta, M. S. Javed, and X. Jiang, Lane Line Detection and Object Scene Segmentation Using Otsu Thresholding and the Fast Hough Transform for Intelligent Vehicles in Complex Road Conditions Electronics, vol. 12, no. 5, 2023. DOI : <https://doi.org/10.3390/electronics12051079>.

[56] A. C. Huang, C. Yuan, S. H. Meng, and T. J. Huang, Design of Fatigue Driving Behavior Detection Based on Circle Hough Transform Big Data, vol. 11, no. 1, pp. 1–17, 2023. DOI : <https://doi.org/10.1089/big.2021.0166>.

[57] D. A. M. Somda and A. Sere, "Traffic saturation detection using hough transform and vanet,"

EAI, 5 2023. DOI : 10.4108/eai.24-11-2022.2329807.

[58] M. Micha lowska, J. Rapiński, and J. Janicka, Tree position estimation from TLS data using

hough transform and robust least-squares circle fitting Remote Sensing Applications : Society

and Environment, vol. 29, p. 100863, 2023. DOI : <https://doi.org/10.1016/j.rsase.2022.100863>.

[59] W. Hou, Research on automatic hole making technology of industrial robot based on Hough circle

detection algorithm International Journal of Wireless and Mobile Computing, vol. 24, no. 3/4,

pp. 352–360, 2023. DOI : <https://doi.org/10.1016/j.ijwmc.2023.131325>.



1504/ijwmc.2023.131325.

[60] C. Jugade, D. Hartgers, S. Mohamed,

D. Goswami, A. Nelson, G. v. d. Veen, and K. Goos-

sens, Improved Positioning Precision using a Multi-rate Multi-sensor in Industrial Mo-

tion Control Systems 2023 European Control Conference (ECC), pp. 1–8, 2023. DOI :

10.23919/ECC57647.2023.10178138.

[61] C. S. Mlambo, A study on Hough transform-based fingerprint alignment algorithms. PhD thesis,

University of Johannesburg, 2015.

[62] K. Anitha, T. Sowmya, V. Srijja, K. N. V. Rao, and G. Vinatha, Cryptographic Fingerprint

Matching Using The Descriptor-Based Hough Transform IJSART, vol. 7, pp. 2395–1052, 2021.

[63] S. Nitika and S. G. Nasib, Hough Transform Based Fingerprint Matching Using Minutiae Ex-

traction International Journal of Advanced Research in Computer Science, vol. 4, p. 117, 2013.

[64] J. Qin, S. Tang, C. Han, and T. Guo, "Partial fingerprint identification algorithm based on the

modified generalized hough transform on mobile device," in International Conference on Graphic

and Image Processing, 2018.

[65] F. J. Calero-Castro, S. Pereira, I. Laga, P. Villanueva, G. Suárez-Artacho, C. Cepeda-Franco,

P. de la Cruz-Ojeda, E. Navarro-Villarán, S. Dios-Barbeito, M. J. Serrano, C. Fresno, and

J. Padillo-Ruiz, Quantification and Characterization of CTCs and Clusters in Pancreatic Can-

cer



www.mdpi.com | IJMS | Free Full-Text | Quantification and Characterization of CTCs and Clusters in Pancreatic Cancer by Means of the Hough Transform Algorithm
https://www.mdpi.com/1422-0067/24/5/4278/review_report

by Means of the Hough Transform Algorithm International Journal of Molecular Sciences,

vol. 24, no. 5, 2023. DOI : 10.3390/ijms24054278.

[66] A. Sere, Transformations Analytiques appliquées aux Images multi-échelles et bruitées Labora-

toire de Traitement de l'Information et la Communication, Université de Ouagadougou, juin

2013.

[67] E. Andres, Modélisation analytique discrète d'objets géométriques Université de Poitiers

(France), Thèse d'habilitation, 2000.

[68] M. Dexet,



www.ijser.org
https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf

Architecture d'un modeleur géométrique à base topologique d'objets discrets et



liris.cnrs.fr
<https://liris.cnrs.fr/Documents/Liris-3688.pdf>

méthodes de reconstruction en dimensions 2 et 3



Thèse en in-

formatique et applications,
2006.

161

[69] R. Jean-Pierre, Géométrie discrète, calcul en nombres entiers et algorithmique Université Louis Paster, p. 1991.

[70] A. Vacavant, Géométrie discrète sur grilles irrégulières isothétiques Université Lumière Lyon 2, Thèse en informatique, 2008.

[71] Freeman,



liris.cnrs.fr
<https://liris.cnrs.fr/Documents/Liris-3688.pdf>

Algorithm for Generating a Digital Straight Line on a Triangular Grid
IEEE Transac-

tions on Computers, vol. C-28, no. 2, pp. 150–152, 1979. DOI : 10.1109/TC.1979.1675305.

[72] Luczak and Rosenfeld, Distance on a Hexagonal Grid IEEE Transactions on Computers, vol. C-25, no. 5, pp. 532–533, 1976. DOI : 10.1109/TC.1976.1674642.

[73] B. Kumar, P. Gupta, and K. Pahwa, Square Pixels to Hexagonal Pixel Structure Representation Technique International Journal of Signal Processing, Image Processing and Pattern Recognition, vol. 7, pp. 137–144, 2014.

[74] Y. Lin, S. L. Pintea, and J. C. van Gemert, Deep Hough-Transform Line Priors arXiv e-prints, p. arXiv :2007.09493, July 2020. DOI : 10.48550/arXiv.2007.09493.

[75] J. Wang, P. Fu, and R. X. Gao, Machine vision intelligence for product defect inspection based on deep learning and Hough transform Journal of Manufacturing Systems, vol. 51, pp. 52–60, 2019. DOI : <https://doi.org/10.1016/j.jmsy.2019.03.002>.

[76] M. Kowdiki and A. Khaparde, Adaptive hough transform with optimized deep learning followed by dynamic time warping for hand gesture recognition Multimed Tools Appl 81, p. 2095–2126, 2022. DOI : <https://doi.org/10.1007/s11042-021-11469-9>.

[77] Q.-B. Ta and J.-T. Kim,



www.mdpi.com | Monitoring of Corroded and Loosened Bolts in Steel Structures via Deep Learning and Hough Transforms
<https://www.mdpi.com/1424-8220/20/23/6888>

Monitoring of Corroded and Loosened Bolts in Steel Structures via

Deep Learning and Hough Transforms Sensors, vol. 20, no. 23, 2020. DOI : 10.3390/s20236888.

[78] L. Huan, W. Li, and Q. Yujian, “Vehicle logo retrieval based on hough transform and deep learning,” in 2017 IEEE International Conference on Computer Vision Workshops (ICCVW), pp. 967–973, 2017. DOI : 10.1109/ICCVW.2017.118.

[79] A. Sheshkus, A. Ingacheva, and D. Nikolaev, Vanishing points detection using combination of fast Hough transform and deep learning vol. 10696, p. 106960H, 2018. DOI : 10.1117/12.2310170.

[80] W. Zhou, C. Qin, J. Chang, Y. Liu, Y. Chen, M. Feng, R. Wang, W. Yang, and J. Yao, Standardized measurement of mid-surface shift of brain based on deep Hough transform Computerized Medical Imaging and Graphics, vol. 108, p. 102284, 2023. DOI : <https://doi.org/10.1016/j.compmedimag.2023.102284>.

[81] J.-Q. Zhang, H.-B. Duan, J.-L. Chen, A. Shamir, and M. Wang, HoughLaneNet : Lane detection

with deep hough transform and dynamic convolution Computers and Graphics, vol. 116, pp. 82–

92, 2023. DOI : <https://doi.org/10.1016/j.cag.2023.08.012>.

[82] M. SE, E. R, and D. F,

**41**

doi.org | Straight-Line Recognition Using a Triangular Grid | SpringerLink
https://doi.org/10.1007/978-3-030-98012-2_45

Champ de hauteurs de la transformée de Hough standard Laboratoire

MOIVRE, Département d'informatique, Faculté des sciences, Université de Sherbrooke
2005.

162

[83] J. Matas, C. Galambos, and J. Kittler, “Progressive probabilistic hough transform,” in British
Machine Vision Conference, 1998.

[84] C. Kimme, D. Ballard, and J. Sklansky, Finding circles by an array of accumulators Commun.
ACM, vol. 18, p. 120–122, feb 1975.



DOI : [10.1145/360666.360677](https://doi.org/10.1145/360666.360677).

[85] H. Yuen, J. Princen, J. Illingworth,
and J. Kittler, Comparative study of Hough Transform

methods for circle finding Image and Vision Computing, vol. 8, no. 1, pp. 71–77, 1990.



DOI :

[https://doi.org/10.1016/0262-8856\(90\)90059-E](https://doi.org/10.1016/0262-8856(90)90059-E).

[86] M. M. A. Coelho., D. N. Sugimoto., G. A. Melo., V. V. Curtis., and J. de Melo Be-
zerra., A MapReduce based Approach for Circle Detection pp.



454–459, 2019. DOI :

[10.5220/0007827604540459](https://doi.org/10.5220/0007827604540459).

[87] Tsuji and Matsumoto, Detection of Ellipses by a Modified Hough Transformation IEEE Tran-
sactions on Computers, vol. C-27, no. 8, pp. 777–781, 1978. DOI : [10.1109/TC.1978.1675191](https://doi.org/10.1109/TC.1978.1675191).

[88] E. Davies, Finding ellipses using the generalised Hough transform Pattern Recognition Letters,
vol. 9, no. 2, pp. 87–96, 1989.



DOI : [https://doi.org/10.1016/0167-8655\(89\)90041-X](https://doi.org/10.1016/0167-8655(89)90041-X).

[89] J. Illingworth and J. Kittler, The Adaptive Hough Transform IEEE Transactions on Pat-
tern Analysis and Machine Intelligence, vol. PAMI-9, no. 5, pp. 690–698, 1987. DOI :
[10.1109/TPAMI.1987.4767964](https://doi.org/10.1109/TPAMI.1987.4767964).

[90] A. Séré, D. Colazzo, and O. Sié, A Hough Transform Based On a MapReduce Algorithm 2016.

[91] H. Li, M. A. Lavin, and R. J. Le Master, Fast Hough transform : A hierarchical approach
Computer Vision, Graphics, and Image Processing, vol. 36, no. 2, pp. 139–161, 1986.



DOI :

[https://doi.org/10.1016/0734-189X\(86\)90073-3](https://doi.org/10.1016/0734-189X(86)90073-3).

[92] Y. Li and X. Gan, An Integrated Fast Hough Transform for Multidimensional Data IEEE Tran-

sactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 9, pp. 11365–11373, 2023.

DOI : 10.1109/TPAMI.2023.3269202.

[93] E. GABRIEL, Automatic Multi-Scale and Multi-Object Pedestrian and Car Detection in Digital Images Based on the Discriminative Generalized Hough Transform and Deep Convolutional Neural Networks 2019.

[94] J. Gall, A. Yao, N. Razavi, L. Van Gool, and V. Lempitsky, Hough Forests for Object Detection, Tracking, and Action Recognition IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 11, pp. 2188–2202, 2011. DOI : 10.1109/TPAMI.2011.70.

[95] D. Karimanzira and H. Renkewitz, “Detection

42

[link.springer.com](https://link.springer.com/content/pdf/10.1007/978-3-662-62746-4_3.pdf)
https://link.springer.com/content/pdf/10.1007/978-3-662-62746-4_3.pdf

and localization of an underwater docking station

in acoustic images using machine learning and generalized fuzzy hough transform,” in Machine

Learning for Cyber Physical Systems (J. Beyerer, A. Maier, and O. Niggemann, eds.), (Berlin, Heidelberg), pp. 23–31, Springer Berlin Heidelberg, 2021.

163

[96] R. Yam-Uicab,



J. Lopez-Martinez, and J. Trejo-Sanchez,
A fast Hough Transform algorithm for

straight lines detection in an image using GPU parallel computing with CUDA-C J Supercomput, vol. 73, p. 4823–4842, 2017. DOI :



<https://doi.org/10.1007/s11227-017-2051-5>.

[97] J. Sun and E. Roosenbrand, “Flight contrail segmentation via augmented transfer learning with novel sr loss function in hough space,” 2023.

[98] sadaf shafi, A. Atif, and H. Shafi, Automated Document Orientation Correction Using Skew and Inversion Rectification With Hough Line Transformation and Machine Learning Preprints, 2023. DOI : 10.20944/preprints202303.0326.v1.

[99] S. Kulandaivel and R. Jeyachitra, Combined image Hough transform based simultaneous multi-parameter optical performance monitoring for intelligent optical networks Optical Fiber Technology, vol. 79, p. 103357, 2023. DOI : <https://doi.org/10.1016/j.yofte.2023.103357>.

[100] K. Oh, D. Seo, I.-S. Oh, and G. Kim, Hough Soft-Argmax Based Tillage Boundary Detection for Autonomous Tractor p. 27, 2013. DOI : <http://dx.doi.org/10.2139/ssrn.4442845>.



[org/10.2139/ssrn.4442845](http://dx.doi.org/10.2139/ssrn.4442845).

[101] A. SERE, M. OUEDRAOGO, and A. K. ATIAMPO,
“Parallel hough transform based on object
dual and pympl library,” 2022.

[102] France, Ministère de l'intérieur et des outre-mer, Gendarmerie National mai 2024.



[103] N. Yager and A. Amin, Fingerprint verification based on minutiae features :

Analysis and Applications, vol. 7, pp. 94–113, 2004. DOI : [https://doi.org/10.1007/s10044-003-](https://doi.org/10.1007/s10044-003-0201-2)

0201-2.

[104] W. Lukasz, A minutae based matching algorithms in fingerprint recognition medical informatics

and technologies, vol. 13, pp. 66–71, 2009.

[105] M. Mehtre, B. Fingerprint image analysis for automatic identification Machine Vision and Ap-

plications, vol. 6, pp. 124–139, 1993. DOI : <https://doi.org/10.1007/BF01211936>.

[106] F. V. N. Cynthia S. Mlambo, Mmamelatelo E. Mathekga, A Study of Hough Transform-based

Fingerprint Alignment Algorithms International Journal of Computer Applications, vol. 103,

pp. 1–8, October 2014.



DOI : [10.5120/18091-9158](https://doi.org/10.5120/18091-9158).

[107] *Neurotechnology*, [https://www.neurotechnology.com/download.](https://www.neurotechnology.com/download.html)

html avril 2024.

[108] F. Fingerprint vérification competition,



[http://bias.csr.unibo.it/fvc2004/download.](http://bias.csr.unibo.it/fvc2004/download.asp)

asp avril

2024.

[109] M. Ouedraogo, A. Sere, and C. A. D. G. Traore, HoughVG :Hough Transform Toolbox for

Straight-Line Detection and Fingerprint Recognition Elsevier, Software Impacts, vol. 22, 2024.

DOI : [10.1016/j.simpa.2024.100709](https://doi.org/10.1016/j.simpa.2024.100709).

164

Dédicace

Remerciements

Abstract

Résumé

Table des matières

Sigles et acronymes

Listes des algorithmes

Liste des figures

Liste des tableaux

Introduction générale

Contexte de la recherche

Problématique

Objectif de la thèse

Hypothèses de recherche

Organisation du document

Publications personnelles

Panorama de la Transformée de Hough

Introduction

Notions de géométrie discrète

Notions des grilles virtuelles

Origine et historique de la transformation de Hough

Définition et étapes de la transformation de Hough Standard

Variantes de la Transformée de Hough

Avancées de la Transformée de Hough

Applications de la transformée de Hough

Domaines émergents

Conclusion

Transformée de Hough Rectangulaire

Introduction

Description de la méthode

Complexité de la Transformée de Hough Rectangulaire

Simulations et discussions

Conclusion

Transformée de Hough Triangulaire

Introduction

Description de la méthode

Complexité de la transformée de Hough triangulaire

Simulations et discussions
Analyse comparative
Conclusion

Transformée de Hough Hexagonale
Introduction
Description de la méthode
Complexité de la transformée de Hough hexagonale
Simulations et discussions
Conclusion

Transformée de Hough Octogonale
Introduction
Description de la méthode
Complexité de la transformée de Hough octogonale
Simulations et discussions
Conclusion

Reconnaissance d'Empreintes Digitales dans une Grille Virtuelle
Introduction
La biométrie
Les empreintes digitales
Structure d'un système de reconnaissance d'empreintes digitales
Méthodologie
Expérimentations et discussions
Conclusion

Architecture de la Boîte à Outils
Introduction
Conception
Interface graphique
Impact
Limitations
Validation des hypothèses de recherche
Conclusion

Conclusion générale



Maître de conférences en informatique

Vérification après la soutenance

5%
Textes
suspects



1% Similitudes
< 1% similitudes entre
guillemets
< 1% parmi des sources
mentionnées
4% Langues non reconnues

Nom du document: main_14_01_2025.pdf
ID du document: fd8b89f28a2a025489aa73d28b378690c326f921
Taille du document d'origine: 15,47 Mo
Auteur: sere abdoulaye

Déposant: sere abdoulaye
Date de dépôt: 02/02/2025
Type de dépôt: url_submission
date de fin d'analyse: 02/02/2025

Nombre de mots: 75 359
Nombre de caractères: 426 825

Emplacement des similitudes dans le document:



Sources principales détectées

N°	Description	Similitudes	Emplacements	Informations complémentaires
1	doi.org Straight-Line Recognition Using a Triangular Grid SpringerLink https://doi.org/10.1007/978-3-030-98012-2_45 1 source similaire	< 1%		Mots identiques: < 1% (69 mots)
2	www.memoireonline.com Memoire Online - Etude des méthodes de reconnaissan... https://www.memoireonline.com/08/21/12173/Etude-des-methodes-de-reconnaissances-dempre... 4 sources similaires	< 1%		Mots identiques: < 1% (71 mots)
3	www.ijser.org https://www.ijser.org/thesis/publication/TH_M7RGYG.pdf 2 sources similaires	< 1%		Mots identiques: < 1% (65 mots)
4	dspace.univ-tlemcen.dz http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf 10 sources similaires	< 1%		Mots identiques: < 1% (62 mots)
5	theses.hal.science https://theses.hal.science/tel-00009742v1/document 5 sources similaires	< 1%		Mots identiques: < 1% (42 mots)

Sources avec similitudes accidentelles

N°	Description	Similitudes	Emplacements	Informations complémentaires
1	bib.univ-oeb.dz http://bib.univ-oeb.dz:8080/jspui/bitstream/123456789/6791/1/mémoire pdf.pdf	< 1%		Mots identiques: < 1% (40 mots)
2	fr.slideshare.net Biométrie d'Empreinte Digitale PDF https://fr.slideshare.net/slideshow/biomtrie-dempreinte-digitale/34684848	< 1%		Mots identiques: < 1% (25 mots)
3	dspace.univ-tlemcen.dz http://dspace.univ-tlemcen.dz/bitstream/112/3310/1/Memoire de magister HAOUZI.pdf	< 1%		Mots identiques: < 1% (30 mots)
4	liris.cnrs.fr https://liris.cnrs.fr/Documents/Liris-3688.pdf	< 1%		Mots identiques: < 1% (29 mots)
5	Document d'un autre utilisateur #e1c200 Le document provient d'un autre groupe	< 1%		Mots identiques: < 1% (23 mots)

Sources mentionnées (sans similitudes détectées)

Ces sources ont été citées dans le document sans trouver de similitudes.

- <http://dx.doi.org/10.14569/IJACSA.2022.0131084>
- <http://dx.doi.org/10.4108/eai>
- <http://dx.doi.org/10.4108/eai.18-12-2023.2348107>
- https://fr.wikipedia.org/wiki/Licence_MIT